

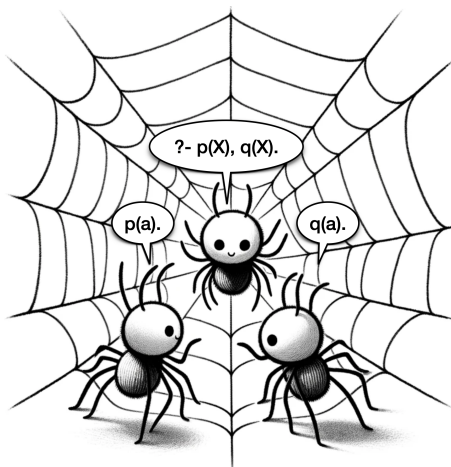
Torbjörn Lager

The Prolog Trinity ecosystem

Let's webize Prolog, prologize the Web, and
populate the Web with clever Prolog agents!

(Draft printed 2026-09-09 at 08:52:09)

Preface



In this book, we present a proposal for a profile of Prolog called *Web Prolog*. We like to think of it as a *web programming language*, or more specifically, as a *web logic programming language*, but also as a kind of simple web-based *agent programming language*. In addition, the architecture for an extension of the traditional Web that we think of as *the Prolog Web* is described, along with descriptions of some of the *Prolog agents* that might dwell there. Together, Web Prolog, Prolog agents and the Prolog Web form what we shall refer to as the *Prolog Trinity ecosystem* – a niche within the vast ecosystem of the World Wide Web.

As can be gleaned from its title, as well as from the previous paragraph, the Prolog programming language plays the lead role in the book, and Prolog programmers and Prolog system implementers are indeed among its intended readers. However, we do not think a reader has to be a very experienced Prolog programmer or implementer to get something out of it. In fact, we are presenting what we believe is a somewhat novel view of Prolog and Prolog programming, so it might even be good if your mind is a beginner's mind, still open to a somewhat different story.

Another programming language, namely Erlang, has an important role to play in the book, so Erlang developers and other actor-programming aficionados might be interested. We hope they will contribute their experience both to the future specification of Web Prolog and to its implementation. Note, however, that although Erlangers are among the intended readers, some basic knowledge of Prolog is still assumed.¹

¹ For full accounts of the language and its programming techniques, see Sterling and Shapiro's *The Art of Prolog* and (for a more advanced treatment) O'Keefe's *The Craft of Prolog* (Sterling and Shapiro, 1994; O'Keefe, 1990).

Logic programming researchers and system developers aiming for logical purity might be interested too, but should be aware that the book is focusing on a non-declarative concurrency and distribution model for Prolog, and does not purport to contribute to its logic programming aspects. It is still only “good ol’ Prolog.” Logic programming researchers might find *use* for it, however, as a way to present their own work to the world, and in particular to other Web Prolog programmers. The broader ambition is to show how the globe itself might be wrapped in pure logic, and logic programming researchers may want to find out what that means.

The book is also written with an audience of Semantic Web researchers and practitioners in mind, in the hope that it will seem relevant to some of them. After all, the Prolog community and the Semantic Web community have something in common, namely *computational logic*, and probably also a deep conviction that, on some level at least, building software agents that are *really* clever requires logic, reasoning and a relational way of thinking. We hope to be able to show that Web Prolog might serve as a *semantic web logic programming language* – a language that fits in very well among the other web logic languages defined and standardised by the W3C, and a suitable language for building semantically aware web applications, such as intelligent web agents.

Readers interested in Artificial Intelligence (AI) might also be able to get something out of the book, at least if they are somewhat familiar with Prolog. In particular, readers who, along with many others, hold the view that a lot of what happens within the field of AI in the coming decades will take place on the Web, and that intelligent conversational agents are likely to serve as the *face* of AI. Note, however, that the book has almost nothing to say about artificial neural networks and other non-symbolic approaches to AI, but is primarily concerned with symbolic approaches, focusing on the use of logic and reasoning for building web agents. Chapter 8 develops an architecture that combines *large language models* (LLMs) with Prolog rules, actor communication, and statechart control. Its service-request example shows how model proposals can be validated, confirmed, and routed under explicit policy, with human review where needed. The discussion also examines the limits of symbolic rules and the assumptions needed to combine these technologies responsibly.

The categories of potential readers outlined above – Prolog programmers and system implementers, Erlang developers, logic programming researchers, and Semantic Web practitioners – are not merely an audience for this book but also (potentially) the main stakeholders in a broader endeavour: the realisation of the Prolog Trinity ecosystem. Each of these groups brings essential expertise and perspectives that contribute to the development, refinement, and practical deployment of Web Prolog, Prolog agents, and the Prolog Web. Together, these stakeholders form a core community that might shape the Trinity, ensuring that it is not only an intellectually compelling idea but also a practical and viable extension of the Web.

It may all be a pipe dream, but it is not a castle in the air!²

² <https://blogs.bmj.com/bmj/2017/01/27/neville-goodmans-metaphor-watch-dream-a-dream-of-ivory-castles-in-the-air/>

Use of artificial intelligence and computational tools

AI systems were used extensively during the later stages of preparing this manuscript and developing its accompanying software prototypes. Their role evolved over the course of this work and became an increasingly important part of both the writing and the software-engineering process.

The origins of Web Prolog, however, substantially predate the recent development and widespread adoption of such systems. The language, its underlying architectural ideas, and an earlier proof-of-concept implementation had already been developed and were described in the author's paper *Intro to Web Prolog for Erlangers*, published in 2019. Substantial parts of the present manuscript likewise originated in work undertaken before large language models became practical tools for research and software engineering; extracts documenting this work have been made publicly available at <https://torbjornlager.github.io/>. AI systems therefore played no part in the original conception of Web Prolog. Their contribution concerned the subsequent refinement, implementation, investigation, and presentation of ideas whose development had begun considerably earlier.

In preparing the present manuscript, these systems were used as interactive tools for technical discussion and criticism, linguistic revision, investigation of technical questions, and comparison of design alternatives. They also assisted in identifying potentially relevant specifications, documentation, and literature. The author independently consulted and evaluated those sources before relying on them in the text.

In later implementation work on the Web Prolog prototype and its associated demonstrator, AI systems assisted with programming, debugging, architectural analysis, user-interface design, and system integration. This assistance concerned, among other subjects, Prolog, JavaScript, HTML, CSS, SVG, and browser technologies. Some of the later architectural and design decisions discussed in this book emerged from iterative exchanges between the author and these systems. All proposed designs, explanations, and implementation materials were reviewed by the author and were adopted, revised, or rejected at the author's discretion. The author directed the development process and was responsible for integrating, testing, and validating the resulting software.

AI systems were treated as computational tools, not as authors or authoritative sources. Their outputs were subject to critical review, and no responsibility for the work is attributed to them. The author assumes full responsibility for the manuscript's arguments and statements, the technical design and implementation of the software, and the reported experimental results.

This book thus spans two distinct periods in the development of Web Prolog: an earlier period in which its central ideas and initial implementations were developed without the assistance of AI systems, and a later period in which such models became practical tools for research, writing, and software engineering. Their role is documented here both for transparency and as part of the methodological context in which the later stages of the work were carried out.

Acknowledgements

This book has grown out of a long personal and intellectual trajectory, and I owe thanks to many people and institutions that have shaped it, directly or indirectly. I am especially grateful to the University of Gothenburg, and to the departments and programmes through which I first encountered philosophy, linguistics, computer science, computational linguistics, dialogue systems, language technology, and the Web. The former Department of Linguistics, the Department of Philosophy, and later the Department of Philosophy, Linguistics and Theory of Science have provided the academic environment in which most of the ideas in this book gradually took shape. I am also grateful to Chalmers University of Technology, the Centre for Language Technology, the Faculty of Arts, the IT Faculty, and the Swedish National Graduate School of Language Technology for forming parts of the institutional landscape in which much of this work became possible.

Several teachers and colleagues were important in my early intellectual formation. Björn Haglund and Staffan Carlshamre gave lectures in philosophy that left a lasting impression on me, and helped turn my interest toward cognition, logic, language, and the philosophy of science. Bosse Dahlbom, by helping me gain access to a DEC VAX computer at Chalmers, gave me my first real taste of networked computing, `comp.lang.prolog`, FTP archives, and the strange feeling of being intellectually present in machines located far away. That experience may have mattered more than I realised at the time.

I am also indebted to the computational linguistics and dialogue-systems community in Gothenburg and elsewhere. Joakim Nivre was both a collaborator and an important intellectual companion; without his talent for formalisation and for structuring arguments, some of the work I am most fond of would have been much weaker. Björn Beskow joined Joakim and me in the small adventure of Linguistic Instruments. Robin Cooper and Staffan Larsson helped shape the dialogue-systems environment in Gothenburg. The PLUS, TREE, TRINDI, and related dialogue and language-technology projects gave me problems to think about, systems to build, and reasons to keep returning to Prolog.

A central debt is owed to Jan Wielemaker. My work on Pengines, SWISH, and eventually Web Prolog would not have developed as it did without his help, generosity, technical insight, and long-standing work on SWI-Prolog. He kindly helped with the implementation of an earlier demonstrator, and he also came up with the original idea on which the stateless HTTP API described in this book is based. SWI-Prolog showed me that Prolog could be a practical platform for the Web, not merely a beautiful language for small examples and elegant programs. I am grateful not only to Jan, but also to the wider SWI-Prolog community and to the Prolog and logic programming communities more generally. This book is, among other things, an attempt to give something back to them.

The design ideas in this book also owe much to Erlang, Oz, and SCXML. I am grateful to the late Joe Armstrong for his enthusiastic support, for his writings and talks, and for his personal comments on early versions of these ideas. He encouraged me to think more seriously about actors, message passing, fault tolerance, and the

possibility of making Prolog look outward toward a distributed world. Richard O’Keefe and Markus Triska provided encouraging and constructive comments and suggestions, and Richard O’Keefe’s remarks on Erlang-style concurrency in Prolog were an important source of confidence. Peter van Roy and Seif Haridi, through their work on Oz and *Concepts, Techniques, and Models of Computer Programming*, helped me understand concurrency, dataflow, message passing, and multi-paradigm programming in a deeper way. Finally, my involvement in the W3C SCXML Working Group gave me valuable experience of standardisation work, and helped convince me that languages and protocols intended for the Web need not only good ideas, but also specifications, implementations, tests, communities, and patience.

I also thank the anonymous reviewers who, at different stages, offered useful suggestions and criticism. Their comments helped sharpen the presentation and reminded me that even a book driven by vision must eventually submit to the discipline of explanation.

Skärhamn, August 2026
Torbjörn Lager

Contents

1	Introduction	1
1.1	The Prolog Trinity ecosystem	2
1.2	Walking in the footsteps of inventors	3
1.3	The Prolog Trinity ecosystem concretised	4
1.3.1	Prolog nodes and Prolog actors	5
1.3.2	Relations, actors, and interaction across nodes	6
1.4	Motivations and goals	8
1.4.1	Challenges in the Prolog ecosystem	9
1.4.2	New opportunities on the Web and in artificial intelligence	10
1.4.3	The five principal goals of the Prolog Trinity	11
1.4.4	Webizing Prolog and prologizing the Web	12
1.5	Thirty years of Prolog on the Web	13
1.6	The Trinity demonstrator	15
1.7	The structure of the book	16

Part I The conceptual and architectural core

2	Web Prolog	23
2.1	The essence of Prolog	23
2.2	Web Prolog in a nutshell	25
2.2.1	Web Prolog is inspired by Erlang	28
2.2.2	Web Prolog is a multi-paradigm programming language	31
2.2.3	Web Prolog as a scripting language for the Web	33
2.3	Erlang-style message-passing concurrency in Web Prolog	37
2.3.1	Programmer talking to actor, actor talking to itself	38
2.3.2	Two utility tools for programming in the shell	39
2.3.3	Actors talking to other actors	40
2.3.4	A closer look at receive/1-2	49
2.4	Erlang-style programming examples in Web Prolog	57
2.4.1	A count server	57
2.4.2	A bigger, tastier example	59

2.4.3	Hiding the details of protocols	60
2.4.4	Using delayed sending	61
2.4.5	Prolog actors playing ping-pong	62
2.4.6	Event-driven state machines	63
2.4.7	Getting answers through backtracking	65
2.4.8	The toplevel behaviour: a preliminary sketch	66
2.5	Summary	68
3	Prolog agents	71
3.1	Prolog agents in a nutshell	71
3.2	More about Prolog actor agents	74
3.2.1	An actor agent is equipped with a private Prolog database ..	74
3.2.2	The lifecycle of a typical Prolog actor agent	75
3.2.3	The dynamic (and still private) Prolog database	77
3.2.4	Private beliefs and shared knowledge – shadowing	78
3.3	Prolog shells and other toplevel actors	80
3.3.1	A Prolog toplevel is an actor with a built-in protocol	81
3.3.2	The Prolog Toplevel Communication Protocol	82
3.3.3	Shell talking to a Prolog toplevel	84
3.3.4	Programming with toplevel actors	91
3.4	Prolog nodes are also agents	94
3.4.1	The node controller	95
3.4.2	The shared Prolog database	95
3.4.3	Deploying a node	96
3.4.4	Settings	97
3.4.5	Two transports – three web APIs	99
3.5	Node profiles	101
3.5.1	Browser talking to a RELATION node	103
3.5.2	Browser talking to an ISOBASE node	106
3.5.3	Browser talking to an ISOTOPE node	107
3.5.4	Browser talking to an ACTOR node	110
3.5.5	A terminal talking to a browser node	112
3.5.6	Social robot talking to an ACTOR node	113
3.5.7	Could there be other profiles?	115
3.6	Notes on node implementation	115
3.6.1	Statelessness as a protocol property	115
3.6.2	Semi-stateful HTTP sessions and response retrieval	116
3.6.3	Stateful WebSockets and push-driven interaction	117
3.7	Summary	118
4	The Prolog Web	119
4.1	The concurrent Prolog Web	121
4.1.1	Actor talking to remote actor	121
4.1.2	Actors playing ping-pong while on different nodes	122
4.1.3	Terminal authority across node boundaries	123

4.1.4	The node option works for toplevels too!	124
4.1.5	Node-resident actor processes	125
4.1.6	Uniform messaging across node boundaries	130
4.1.7	Signals and distributed lifetime control	131
4.2	The sequential Prolog Web	134
4.2.1	Nondeterministic remote procedure calls	135
4.2.2	Two operational realisations	137
4.2.3	Browser talking to a node talking to a node	138
4.2.4	Backtracking as pagination	140
4.2.5	Programming with <code>rpc/2-3</code>	145
4.2.6	Promise and yield over stateless HTTP	148
4.3	The programmable Prolog Web	151
4.3.1	Injecting code with the <code>src_*</code> options	152
4.3.2	Code shipping and data locality	152
4.3.3	A small design space	153
4.4	Timing and failure boundaries on the Prolog Web	153
4.4.1	Four blocking surfaces	154
4.4.2	Late replies	155
4.4.3	Design principle	155
4.5	Security on the Prolog Web	155
4.5.1	Inexpressibility: the sandbox	156
4.5.2	Invisibility: name discipline	157
4.5.3	Containment: lifetime discipline	158
4.5.4	Deployment concerns	159
4.6	Summary	160

Part II Behaviours and other generics

5	Implementing Web Prolog generics	163
5.1	The server behaviour	164
5.1.1	The client interface	166
5.1.2	A failing callback	168
5.1.3	Hot code replacement	168
5.1.4	Graceful shutdown	169
5.1.5	From a bespoke actor to a reusable behaviour	170
5.2	The supervisor behaviour	170
5.2.1	A restart we can observe	170
5.2.2	The generic mechanism	172
5.2.3	From restart to escalation	173
5.2.4	Strategies, dynamic children, and trees	174
5.3	The toplevel behaviour	175
5.3.1	The three inner states	176
5.3.2	Why <code>fail/0</code> resumes the search	178
5.3.3	Production lifecycle management	179
5.3.4	The enclosing state: abort and halt	179

5.4	The <code>rpc/2-3</code> meta-predicate	181
5.4.1	From protocol messages to Prolog nondeterminism	182
5.4.2	An actor-based reconstruction	182
5.4.3	The stateless HTTP implementation	184
5.4.4	Comparing the two realisations	186
5.5	The <code>parallel/1</code> meta-predicate	187
5.5.1	From two goals to a list	187
5.5.2	Detecting failure and errors	188
5.5.3	Race-free cleanup	189
5.5.4	Behaviour and performance	190
5.5.5	Relation to <code>concurrent/3</code>	192
5.6	The <code>first_solution/2-3</code> meta-predicate	192
5.6.1	Racing alternative solvers	192
5.6.2	Failure and error policies	194
5.6.3	An example	194
5.7	Summary	195
6	The statechart behaviour	197
6.1	STATECHARTS	197
6.2	Statechart actors	198
6.2.1	A first statechart	198
6.2.2	Running a statechart actor	200
6.3	Hierarchy and history	201
6.4	Parallelism and broadcast communication	202
6.4.1	An NPC controller	202
6.4.2	An unbounded internal cascade	204
6.4.3	What “broadcast” means here	205
6.4.4	Why not model each region as a separate actor?	205
6.5	The Web Prolog data model	206
6.5.1	Guarded transitions can derive information	206
6.5.2	Eventless transitions as forward chaining	207
6.5.3	Timed transitions	208
6.5.4	Deferred events	209
6.6	Process invocation and communication	209
6.6.1	What does a confirmation confirm?	211
6.7	The execution model in summary	212
6.8	A flat subset of SXML	213
6.9	From XML source to execution	213
6.10	Discussion	214
6.10.1	Previous work	214
6.10.2	Comparison with an ordinary message loop	214
6.10.3	Could PTCP itself be an SXML statechart?	216
6.10.4	Comparison with Erlang’s <code>gen_statem</code> behaviour	216
6.10.5	Relationship to SCXML	217
6.10.6	Toward an actor-oriented successor to SCXML	217

6.11 Summary	218
--------------------	-----

Part III The broader technological contexts

7 The Prolog Web $\approx$ the Semantic Web?	221
7.1 The Semantic Web tower	222
7.2 RDF as rdf/3	223
7.3 SPARQL-style querying in Prolog	225
7.4 OWL as an external reasoning service	227
7.5 Positioning the Prolog Web alongside the Semantic Web	228
7.6 Cooperation patterns	230
7.6.1 A: Web Prolog as a SPARQL client and post-processor	230
7.6.2 B: RDF as rdf/3, Prolog rules as the API	231
7.6.3 C: OWL reasoning as a service, Web Prolog for orchestration	231
7.6.4 Summary	232
7.7 Proof and trust	232
7.8 User interfaces and applications	233
7.9 Related work: towers of logic programming	234
7.10 Outlook: Prolog Trinity towers	235
7.11 Summary and discussion	237
8 Prolog AI agents on the Agentic Web	239
8.1 Why Prolog agents now?	240
8.2 Symbolic competence in one engine	242
8.2.1 Knowledge representation and inference	242
8.2.2 Planning as search in a state-transition model	242
8.2.3 Meta-interpreters as a control and explanation lever	243
8.2.4 An expert system with a query-the-user facility	244
8.2.5 Parsing as deduction: DCGs	245
8.2.6 Summary	245
8.3 Prolog AI agents on the Agentic Prolog Web	245
8.3.1 An expert-system agent and a simulation	246
8.3.2 Explainable AI across nodes: distributed proof trees	248
8.3.3 Meta-interpreters as generic behaviours	249
8.3.4 Parsing as remote deduction	250
8.3.5 Implementing agent frameworks in Web Prolog	250
8.3.6 Scalability and interoperability on the Web	251
8.4 Neuro-symbolic AI agents	252
8.4.1 Why hybridise?	252
8.4.2 Web Prolog as symbolic backend and agent layer	253
8.4.3 Architectural patterns for hybrid agents	254
8.4.4 A statechart-wrapped service-request agent	256
8.4.5 Open engineering and research questions	261
8.5 User-interface agents	262
8.5.1 UI agents versus MAS agents	262

8.5.2	Turn-taking and repair as control structure	263
8.5.3	Dialogue as proof: the missing-axioms view	263
8.5.4	Statecharts as dialogue managers	264
8.5.5	Dialogue as a governed interface	265
8.6	Summary	266

Part IV From architecture to paradigm

9	The Trinity as a web-native symbolic paradigm	269
9.1	The two partitions as a foundational principle	270
9.1.1	Two levels of abstraction	271
9.1.2	Unification as the universal data contract	272
9.1.3	Backtracking as pagination	273
9.2	From compatibility to interaction portability	273
9.2.1	Node profiles as interaction contracts	275
9.3	Agents as the primary abstraction	276
9.3.1	Interaction as inference	277
9.3.2	Mutual constraint	277
9.3.3	Capability-oriented security	278
9.3.4	Five complementary views	279
9.4	The logical foundation: purity, belief, and scoped inference	279
9.4.1	Pure Web Prolog and the pure Prolog Web	279
9.4.2	Epistemic reading and scoped inference	280
9.4.3	The less pure Prolog Web	281
9.5	The distribution-first model	281
9.5.1	The browser as a Web Prolog runtime	283
9.6	Control as a first-class dimension	283
9.6.1	Governance of interaction	283
9.7	Executable logic on the Web	284
9.8	Comparison to other approaches	285
9.8.1	Linda and tuple-space coordination	285
9.8.2	Concurrent Prolog and the committed-choice languages	285
9.8.3	Erlang/OTP and actor behaviours	286
9.8.4	Oz/Mozart and the multiparadigm kernel language	286
9.9	Why the Trinity fits Prolog	287
9.10	The dual process	287
9.11	Summary	288

Part V What's in it for you and your community?

10	Communities, interoperability, and adoption	291
10.1	From fragmentation to federation	292
10.1.1	Incremental convergence through PIPs	292
10.1.2	One Prolog to rule them all	292
10.1.3	The middle way: interoperability as a federating force	293

10.2	Community infrastructure and adoption	296
10.2.1	Shared objects for collaboration	296
10.2.2	Playgrounds and learning	296
10.2.3	Conditions for wider adoption	298
10.3	Opening the network to neighbouring communities	299
10.3.1	For the broader logic programming community?	299
10.3.2	For the Semantic Web community?	302
10.3.3	For the AI community?	303
10.3.4	For web programmers?	304
10.3.5	For the Erlang community?	305
10.4	Risk analysis	306
10.5	Summary	307

Part VI Final words

11	From vision to execution	313
11.1	A research and engineering agenda	314
11.2	Two routes from prototype to platform	316
11.2.1	A production-oriented SWI-Prolog platform	316
11.2.2	A federated and standardised Prolog Web	317
11.2.3	A decision that need not be made all at once	317
11.3	From incubation to standardisation	317
11.3.1	What should be standardised, and when?	318
11.3.2	Incubation and institutional division of labour	318
11.3.3	The SXML notation and SCXML	320
11.4	The first specification targets	320
11.4.1	The actor substrate as the distinctive target	321
11.4.2	Actor identity, messaging, and lifecycle	321
11.4.3	The semantics of <code>spawn/3</code>	321
11.4.4	The resumable toplevel actor	322
11.4.5	Solution cursors and the ISO liaison	322
11.4.6	Layering the first standards	322
11.5	Implementation and interoperability	323
11.5.1	Why implementation comes early	323
11.5.2	Independent implementations and test suites	323
11.6	AI-assisted implementation and standardisation	323
11.7	Shared ownership	324
11.8	Closure	325

Part VII Appendices

A	Glossary	329
----------	-----------------------	------------

B	Manual	341
B.1	Predicates for programming with core actors	341
B.2	Predicates for programming with toplevel actors	350
B.3	Predicates for programming with server actors	352
B.4	Predicates for programming with statechart actors	354
B.5	Predicates for programming with supervisor actors	355
B.6	Prolog-style generics	358
B.7	Predicates for remote Prolog-style calls	359
B.8	Common error contract	361
B.9	ISO predicates and related constructs	362
C	SXML notation reference	365
C.1	A complete minimal document	365
C.2	Vocabulary at a glance	366
C.3	XML and embedded Prolog	367
C.3.1	XML conventions	367
C.3.2	Where Prolog occurs	367
C.4	Document root and states	368
C.4.1	The <statechart> element	368
C.4.2	The <state> element	368
C.4.3	The <parallel> element	369
C.4.4	The <final> element and completion	369
C.4.5	The <history> element	370
C.5	Transitions: the <go> element	370
C.5.1	Selection and execution order	372
C.5.2	Timed transitions	372
C.6	Entry, exit, and the Prolog data model	372
C.6.1	The <onentry> and <onexit> elements	372
C.6.2	The <datamodel> element	373
C.6.3	Statechart built-ins	373
C.7	Deferred events: the <defer> element	374
C.8	Actor invocation: the <spawn> element	374
C.8.1	Common syntax and value rules	374
C.8.2	Option matrix	375
C.8.3	Actor, toplevel, server, and supervisor examples	376
C.8.4	Nested statecharts	376
C.9	Run-to-completion and event queues	377
C.10	Validation, tracing, and diagnosis	378
C.11	Relationship to SCXML 1.0	379
C.12	Authoring checklist	380
D	Node protocol reference	381
D.1	Profiles and protocol surfaces	381
D.2	Common conventions	382
D.2.1	Versioning	382

D.2.2	Terms, pids, and formats	382
D.2.3	Compression, binary formats, and transport protection	383
D.2.4	Common answer and event forms	384
D.3	The toplevel communication protocol	385
D.4	The stateless HTTP binding	386
D.5	The semi-stateful HTTP binding	386
D.6	The stateful WebSocket binding	388
D.6.1	Version negotiation	388
D.6.2	Toplevel session commands	388
D.6.3	Bare actor commands	389
D.7	Lifecycle and ownership	389
D.8	Starting a native node	389
E	How to implement a Prolog node	391
E.1	From a profile to a running node	391
E.1.1	A useful decomposition	392
E.2	A native node on top of SWI-Prolog	393
E.2.1	The stateless HTTP endpoint	393
E.2.2	Avoiding spurious recomputation	395
E.2.3	Implementing the actor primitives	399
E.2.4	Actor I/O	405
E.2.5	Private databases and source installation	406
E.2.6	Distribution and toplevel actors	406
E.3	Sandboxing, isolation, and authority control	407
E.3.1	Whitelist and blacklist strategies	407
E.3.2	Language sandboxing and host isolation	408
E.3.3	WASM and WASI	408
E.3.4	Policy permissions inside the node	408
E.3.5	Resource governance	409
E.3.6	A layered deployment strategy	409
E.4	A browser-resident node	409
E.4.1	Implementing a browser node	410
E.5	Alternative implementation approaches	412
E.5.1	Other Prolog systems and host languages	412
E.5.2	Prospects for a dedicated Web Prolog VM	412
E.5.3	Narrower profiles require less work	413
E.6	Summary	413
F	Benchmarking three actor substrates	415
F.1	What is being compared	415
F.2	Method	416
F.3	Actor lifecycle	417
F.3.1	Chained spawning	418
F.4	Steady-state message passing	420
F.4.1	A browser measurement caveat	421

F.5	RPC, pagination, and query placement	421
F.6	Interpretation and limits	422
References		423
Index		431

Chapter 1

Introduction



This book articulates a *vision* – an ambitious and forward-looking proposal – for a reconceptualisation of Prolog in the context of the modern Web. At the heart of this vision lies *Web Prolog*, a carefully crafted *profile* of the Prolog language – more precisely, an extension of a well-chosen subset – reimagined as a logic-based programming language for the Web. More than just a variant of Prolog adapted for networked environments, Web Prolog aspires to function as a foundational language for a new generation of *web-native logic programming* and *web-based agent systems*.

We propose to think of Web Prolog not only as a tool for building programs that interact over the Web, but as a language for defining logic-driven agents – *Prolog agents* – capable of reasoning, communicating, and coordinating their behaviour across distributed contexts. We use *agent* in a deliberately modest, architectural sense: it is a role assigned to an addressable computational participant encountered through a term-based interaction boundary. The term implies neither intelligence, autonomy, private state, nor a minimum lifetime; rather, this modest form is intended to provide a substrate on which stronger kinds of agency can be built.

These agents inhabit a conceptual space we call the *Prolog Web*: an architectural extension of the traditional Web, designed to accommodate structured logical interactions, intelligent agent communication, and declarative knowledge sharing.

Taken together, these three components form the *Prolog Trinity ecosystem*. This ecosystem is envisioned as a niche within the broader World Wide Web, offering a unique paradigm for symbolic AI, declarative communication, and logical coordination on the Web. It represents not merely a technical enhancement, but a philosophical stance: a reassertion of the relevance of symbolic reasoning and declarative knowledge in an era increasingly dominated by opaque statistical models.

This opening chapter serves as a prologue to that larger vision. It prepares the ground for the conceptual foundations, technical proposals, and speculative possibilities explored in the chapters ahead, and it introduces the *what*, *why* and *how* of a project that aims to bring it to life.

1.1 The Prolog Trinity ecosystem

As suggested by the simple diagram in Figure 1.1, we think of Web Prolog, Prolog agents and the Prolog Web as three different kinds of entities, playing three distinct roles in the comprehensive Prolog-based ecosystem that we think of as the Trinity.

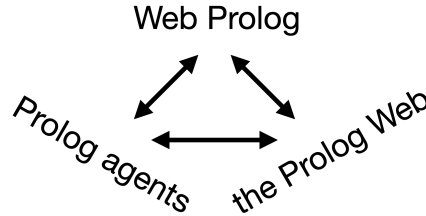


Fig. 1.1 The Prolog Trinity ecosystem: Web Prolog (a shared language), Prolog agents (active participants at node and actor scale), and the Prolog Web (their networked environment), together with their mutual dependencies.

The diagram illustrates the main idea on the highest level of abstraction we shall care about. The arrows are meant to indicate the many interdependencies and symbiotic relationships that exist between these entities, and to remind ourselves that we are looking at one and the same ecosystem from three different angles.

Put simply, on this level of abstraction the Prolog Web is a network of Prolog agents being programmable in Web Prolog and talking Web Prolog to each other. Expressed more elaborately, the Prolog Web is populated, and indeed also propped up, by Prolog agents, and provides, in the form of distributed Web Prolog databases, some of the knowledge that these agents need to do their thing. Prolog agents in the Prolog Web environment interact with other Prolog agents, using Web Prolog as their language of communication. They are also able to talk with (other kinds of) agents outside this environment such as humans, maybe through a conventional graphical user interface (a GUI), or perhaps using spoken natural language in a voice user interface (a VUI), and maybe even in a dialogue with an *embodied conversational agent* (an ECA).

It is important to understand that the Prolog Trinity ecosystem is not an island. The Prolog Web does not exist in complete isolation, cut off from the rest of the Web. Rather, it is meant to serve as an *extension* of the traditional Web, along with other extensions such as the Semantic Web and the Web of Things. Just like these, the Prolog Trinity can be regarded as a *sub-ecosystem* of the Web ecosystem in its entirety. It can also be thought of as a sub-ecosystem of the larger ecosystem surrounding Prolog – the already existing Prolog systems and the communities of logic programming researchers, system implementers, and other users of Prolog.

1.2 Walking in the footsteps of inventors

Before we continue, let us first pay tribute to five people who invented technologies that form the basis for the work presented in the book and laid the groundwork for the Prolog Trinity ecosystem. We have put them side by side in Figure 1.2.



Fig. 1.2 Five inventors whose ideas underpin the Trinity: Colmerauer, Kowalski, Armstrong, Berners-Lee, and Harel (photo montage).

On the left we find Alain Colmerauer. He invented Prolog in 1972 and built the first implementation. Sadly, he passed away in 2017. To the right of him is Robert Kowalski – who came up with a lot of the theory behind Prolog which inspired Colmerauer. He is still around, and is still doing a lot of logic programming research and development. Third from the left, we find Joe Armstrong, who invented another programming language called Erlang. He liked Prolog, and to some extent, as we shall see, Erlang was inspired by Prolog. Regretfully, Joe Armstrong died in 2019. To the right of him, we find Tim Berners-Lee. He invented the Web, the first web browser, and the protocols and algorithms allowing the Web to scale. He is still very much alive and active trying to build a better Web. Finally, on the right, we find David Harel, the inventor of Statecharts, a rather successful visual state-machine programming language. Why his work has a role to play in the Prolog Trinity ecosystem will be revealed later in the book.

These five inventors are (or were, for those that are no longer with us) scientists and engineers who have made significant contributions to the field of computer science. We do not mean to suggest that they were alone in inventing these technologies.¹ The work presented in this book is indebted also to the many, many scholars who evolved the original ideas into the shape they have today. This book tries to build on their achievements, combining chunks of their work into what we hope will emerge as a coherent whole, and preferably a whole greater than the simple sum of its parts.

¹ For example, the Association for Logic Programming has given the official title “Founders of Logic Programming” to fifteen pioneering researchers who laid the groundwork for the field (see https://en.wikipedia.org/wiki/Association_for_Logic_Programming), and Robert Virding and Mike Williams were heavily involved in the design and development of Erlang.

1.3 The Prolog Trinity ecosystem concretised

A deeper understanding of what the Prolog Trinity is all about can hopefully be gained by considering the tiny fragment of the Prolog Web suggested in Figure 1.3.



Fig. 1.3 A small fragment of the Prolog Web. Nodes (boxes) store Web Prolog programs and data (upper container) and host actor processes (lower container, shown as smileys).

On this fairly low level of abstraction, the Prolog Trinity ecosystem can be understood as a concrete network of concrete *Prolog nodes*. In Figure 1.3, nodes are shown as boxes labelled by a *domain name* at the top. The arrows symbolise connections between different nodes, and between end-user clients and nodes. As suggested by the diagram, each node is divided into two “containers,” one container dedicated to the management of actor processes, the other to the storage of Web Prolog programs and data. The boxes and the actors they contain depict the two principal scales of Prolog agent: the node as a network-facing agent and the actor as its elementary unit of concurrent computation.

1.3.1 Prolog nodes and Prolog actors

Inspired by Erlang’s notion of an actor, the *Prolog actor* is the fundamental unit of computation – and the simplest type of agent – in the Prolog Trinity ecosystem. The smiley emojis in Figure 1.3 represent Prolog actors of various kinds that live and work at nodes. When a node is computing something for a client, it is often one or more of these actors that are responsible, even if this is not evident from the outside.

Hiding an actor behind an emoji makes it look simpler than it actually is, and an actor has in fact an interesting internal structure. The diagram in Figure 1.4 shows three actors, one of which has just been spawned and one that has been opened up so that we can look inside (where, at this level of abstraction, they all look the same).



Fig. 1.4 The anatomy of a Prolog actor.

A Prolog actor is organised around a *process* – a thread of control capable of communicating with the world around it through messaging. An actor has a unique address – a *process identifier*, or *pid* for short. If we have the pid of an actor, we can message it, and since pids can themselves be part of a message, it allows other actor processes to communicate back. Furthermore, an actor has a *mailbox* that stores incoming messages. There is a *receive* operation that allows the actor to select and process messages from the mailbox, and a *send* operation that can be used to send messages to any other actor. Note that, as the figure shows, incoming messages are delivered to the mailbox, not to the process itself: sending is asynchronous, and the actor retrieves messages using the receive operation at a time of its own choosing. There is a *spawn* operation that allows an actor to create other actors – child actors as it were. Finally, every actor is equipped with a private Prolog clause database, a dedicated space where programs and data exclusive to that actor are stored.

Authorised end-user clients can connect and interact with nodes. These clients may range from web browsers running simple JavaScript-based Prolog terminals or intelligent web applications, to more sophisticated devices like VR headsets or social

robots. (Note that not all clients are end-user clients because nodes can be clients too.) Human users interacting with end-user clients typically trigger activity in the Prolog Web, where agents as well as connections between agents are more or less frequently being created, interacted with, and then destroyed. Some agents and some connections live for a long time, while others may only last for fractions of a second. This is obviously a form of dynamics that is not easy to capture in a static diagram such as Figure 1.3.

A node can be seen as a special-purpose web server serving data in the form of Prolog terms (possibly encoded in JSON) to clients as well as to other nodes. Just like a web server, a node has a domain name, makes use of HTTP and/or WebSockets, and is usually capable of serving more than one client at the same time.

We use the term *node* rather than *server*, partly because this is what Erlang uses, but a more important reason is that a node is much *more* than just a web server. A node is also a *runtime system* for Web Prolog capable of executing Web Prolog programs stored by the node and/or passed to it by the client over the web APIs. The Prolog Web is a network of such code, ready to be executed by actors, ready to be queried by clients.

1.3.2 Relations, actors, and interaction across nodes

Data and programs stored at a node come in two flavours, characterised by two kinds of Web Prolog clauses. One flavour might have clauses looking like this:

```
mortal(Who) :- human(Who).
```

Clauses of this flavour have logical interpretations, and this particular clause translates into the formula $\forall x[human(x) \rightarrow mortal(x)]$ in first-order predicate logic.

As suggested in Figure 1.3, a node can be queried from a terminal running in a web browser. Given our definition of `mortal/1` and a set of Prolog *facts* such as

```
human(socrates).
human(plato).
human(aristotle).
```

we can ask for the solutions to a query `?-mortal(Who)`. When we run the query from a terminal we enter into a kind of interaction with a Prolog process that most readers are probably familiar with – in which the system only offers one solution at a time and the user may request further solutions by typing a semicolon:

```
?- mortal(Who).
Who = socrates ;
Who = plato ;
Who = aristotle.
?-
```

Such queries can also be given logical interpretations, and the existential closure of this particular query is the formula $\exists x[mortal(x)]$. The interaction above not only establishes that there exists an x such that x is mortal, but also computes *witnesses* for x – terms that make the existential statement true. The point here is to remind ourselves that Prolog is a *logic programming language* eminently suitable for *knowledge representation*, or for other uses that require logic. The interaction also demonstrates that Prolog is a *nondeterministic* programming language, meaning that a query may have several solutions, which can be lazily iterated over using Prolog’s backtracking mechanism.

In comparison with the clause defining the predicate `mortal/1`, the following clause has a very different flavour and looks a lot more like a program, and indeed something from the Erlang world – a clause in the *style* of Erlang:

```
echo_actor :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    }).
```

As its name suggests, it implements the core of an echo server: an actor capable of receiving messages and returning them to the sender as an “echo.” In contrast to the definition of `mortal/1`, this clause cannot as a whole be given the same logical reading; both `receive/1` and `!/2` are built-in predicates with committed side effects, so the clause must also be understood procedurally – as a procedure that, when executed by an actor process, determines that actor’s *behaviour*.

These examples illustrate two characteristic uses of Prolog clauses. A clause may describe a logical relation, an actor’s sequence of actions, or a mixture of the two. We will return to this distinction later.

The predicate `spawn/2` creates a concurrent process running in parallel with the caller, binding the variable in its second argument to a process identifier:

```
?- spawn(echo_actor, Pid).
Pid = 10783471.
?-
```

An interaction exercising the echo actor might look like this:

```
?- self(Self).
Self = 81902375.
?- 10783471 ! echo(81902375, hello).
true.
?- receive({Message -> true}).
Message = echo(hello).
?-
```

Readers familiar with Erlang will recognise the pattern. Readers less familiar with Erlang only need to know, at this point, that the predicate `self/1` retrieves the pid

of the calling process – in this case the *shell*, which is itself an actor – that `!/2` sends a message to another process, and that `receive/1` waits for an incoming message. Even this tiny example already conveys the essential properties of an actor. It has its own mailbox, reacts asynchronously to messages, can run indefinitely, and maintains identity through its process identifier.

Beyond local concurrency, Web Prolog also supports two complementary modes of distributed interaction across node boundaries. The simpler of the two is sequential: a remote procedure call that lets one node query another as naturally as it would query its own database.

The built-in predicate `rpc/2-3` evaluates a goal at a URI-addressed remote node. From the caller's perspective, it behaves like an ordinary Prolog query, returning solutions one at a time through backtracking:

```
?- rpc('http://n2.org', human(Who)).
Who = socrates ;
Who = plato ;
Who = aristotle.
?-
```

The second mode of distribution is concurrent, and it builds on the actor model introduced above. The `spawn/3` predicate accepts a `node` option that takes a URI, allowing an actor to be created on a remote node:

```
?- spawn(echo_actor, Pid, [
    node('http://n4.org')
]).
Pid = 41983256@'http://n4.org'.
?-
```

The returned pid can be used for messaging exactly as a local pid would be:

```
?- 41983256@'http://n4.org' ! echo(81902375, hello).
true.
?-
```

Later chapters describe additional spawning options and actor lifecycle management.

1.4 Motivations and goals

The Trinity is not a solution in search of a problem. It responds to a set of long-standing challenges in the Prolog ecosystem and to a newer set of opportunities arising in the surrounding technological landscape. The challenges explain why a common language, agent model, and networked environment would be useful; developments on the Web and in artificial intelligence help explain why such a project is particularly timely now. This section separates those *motivations* from the *goals* by which the proposal should ultimately be judged.

1.4.1 Challenges in the Prolog ecosystem

Created in 1972, Prolog is an old programming language, but age should not be mistaken for stagnation. Many implementations remain under active development, new ones continue to appear, and established systems have acquired major facilities such as tabling and constraint logic programming libraries. Prolog also continues to support research, practical applications, books, courses, and an active – if small – community.

The survey *Fifty Years of Prolog and Beyond* (Körner et al., 2022) provides a useful assessment of this situation. Published during the 2022 Year of Prolog,² it examines the evolution and current state of Prolog systems and considers how the language and its community might develop. Its account reveals both the continuing vitality of Prolog and the difficulties facing its ecosystem.

The diversity of Prolog systems is both a strength and a source of difficulty. Different implementations have developed distinctive features in response to different research and application needs. Most share the ISO core, but they often differ in extensions, interfaces, libraries, and tools. These differences limit the portability of non-trivial programs. The survey stresses, however, that many of them are not fundamental and could be bridged through common interfaces and shared implementations.

Fragmentation also has a social and organisational dimension. Users tend to gather around particular systems, so knowledge, discussion, and support are often organised by implementation rather than shared across the Prolog community. Implementers likewise have few shared facilities for discussing interfaces, exchanging tests and libraries, or coordinating future work. Compatibility is widely valued, but the time and stewardship needed to maintain it are scarce.

The tooling picture is similarly mixed. Prolog has produced sophisticated debuggers, analysers, specialisers, and development environments, but their availability varies between systems. There is no standard package manager, support in mainstream development environments is uneven, and important work is duplicated across a small community.

Finally, Prolog faces what the survey calls the “post-desktop world of JavaScript web applications.” Browsers, mobile devices, and networked services have become central computing environments, while other languages are generally easier to deploy in them. Individual Prolog systems provide web facilities, but the ecosystem lacks a portable web-facing layer through which different systems can expose services and interact.

These concerns – portability, fragmentation, shared tooling, and the Web – motivate the Trinity. Its response is not to replace existing Prolog systems or erase their differences. It is to define common boundaries through which they can cooperate: a shared language surface, a common agent model, and an open network environment. In this way, the diversity of the Prolog ecosystem can become a federated strength rather than a barrier between its parts.

² <https://prologyear.logicprogramming.org>

1.4.2 New opportunities on the Web and in artificial intelligence

The post-desktop world that presents a challenge to Prolog also creates an opportunity. HTTP, WebSocket, JSON, web workers, WebAssembly, and container deployment provide much of the infrastructure needed to make a language runtime available as a network service. They do not solve the problems of language design, interoperability, security, or resource control, but they allow a Prolog system to participate in the Web without first having to invent its own transport and deployment environment.

Artificial intelligence presents a related opportunity. As (Körner et al., 2022) observes, data-driven methods are powerful at learning patterns from examples, but it can be difficult to express explicitly what they have learned, explain their conclusions, or constrain their behaviour. Symbolic methods offer complementary strengths: they can represent rules and background knowledge, derive conclusions through explicit inference, and make assumptions and intermediate reasoning available for inspection.

The relationship can work in both directions. Logic programming can represent rules extracted from learned models, impose constraints on what those models may infer or do, and verify proposed results before they have consequences. Learned methods can in turn guide search, acquire symbolic knowledge, or improve parts of a logic-based system. Neuro-symbolic AI should therefore be understood not as a contest between two approaches, but as an attempt to combine their different strengths.

Prolog is especially well suited to the symbolic side of this combination. Its terms provide a uniform representation for programs, data, rules, and queries. Its support for meta-programming makes it natural to construct interpreters, analysers, verifiers, and domain-specific languages. Unification, constraints, and explicit inference can support applications in planning, policy enforcement, explanation, dialogue, and the integration of heterogeneous knowledge.

Large language models make this opportunity more immediate. A model can generate text, code, and proposed actions, but it is not by itself a persistent or accountable agent. It becomes part of one only when placed inside an *agent harness*: a surrounding system that maintains state, mediates tool use, checks outputs, enforces permissions and limits, handles failure, and determines what may happen next. Logic programming is a natural candidate for the inspectable and testable parts of such a harness.

The emerging Agentic Web adds a further requirement. Agents must be addressable, able to interact over time, and capable of using services and coordinating with other agents. Web Prolog can contribute at two levels. Its logical facilities can provide rules, constraints, plans, and explanations, while its actor model can provide identity, private state, messaging, and lifecycle management. Neither level guarantees that an application is correct or safe, but together they provide a structured substrate on which stronger assurances can be built.

There is therefore a concrete deployment opportunity. A Prolog engine can be exposed through a stable web interface as a verifier, planner, policy engine, dialogue manager, constraint service, or explanation service. Conversely, a Prolog actor can

use learned models and other external systems as network services. The Web makes these forms of composition available beyond a single runtime, while a common interaction layer makes them available beyond a single Prolog implementation.

1.4.3 The five principal goals of the Prolog Trinity

Against this background, the Prolog Trinity pursues five principal goals.

1. **A simpler model for concurrent programming in Prolog.** Provide a portable actor model based on lightweight processes, private databases, mailboxes, asynchronous messaging, monitoring, linking, and supervision. The model should be easier to use than direct manipulation of operating-system threads and general enough to support a wide range of concurrent applications.
2. **A coherent model for distributed Prolog programming.** Extend querying and actor communication across node boundaries while preserving the same basic programming concepts. Distribution cannot be made indistinguishable from local computation: latency, partial failure, authority, and resource limits must remain visible where they affect program behaviour.
3. **A lingua franca for Prolog systems.** Define a common term language, operational core, and set of interaction interfaces through which different Prolog implementations can exchange goals, answers, programs, data, and messages. The aim is not to eliminate differences between systems, but to establish a shared boundary across them.
4. **A web-native language for intelligent agents.** Support addressable, stateful agents that can reason, react to events, coordinate with other agents, and interact with services and human clients. The actor model supplies the execution framework, while reusable behaviours – including statechart-based behaviours expressed in a combination of Web Prolog and XML – provide structure for more complex agents. These are building blocks rather than a prescribed cognitive architecture.
5. **Deployable Prolog nodes.** Make it straightforward to run a Prolog system as a networked service that can be discovered, secured, tested, and connected to other nodes and Web applications. A node should combine a Prolog runtime, shared programs and data, actor hosting, documented interfaces, and basic operational controls. The demonstrator developed in this book is intended to show that such a design is practical, not to serve as a production platform.

These goals rest on three commitments. First, the Trinity is federated: participating systems retain their own implementations and internal extensions. Second, participation is graded: cumulative profiles allow a node to implement only the parts of the architecture that it needs. Third, public behaviour is defined by precise, testable contracts rather than by assumptions about internal implementation. The same interfaces should also be usable by non-Prolog clients where appropriate.

The three parts of the Trinity divide the responsibility between them. *Web Prolog* provides the common language and interaction interfaces. *Prolog agents* provide the concurrent and distributed execution model. The *Prolog Web* is the open network environment formed by nodes, agents, services, and clients. The three parts are designed together: the shared language must support the agents, the agent model must work across the Web, and the Web interfaces must preserve the relevant Prolog semantics.

1.4.4 Webizing Prolog and prologizing the Web

The essential process in webizing is to take a system which is designed as a closed world, and then ask what happens when it is considered as part of an open world. Practically, this effect on a computer language is to replace the names/tokens/identifiers for URIs. Thus, where before reference could only be made to something in the same document/program/module one can with equal ease make reference to something in a different one somewhere in that abstract space which is the Web.

*Tim Berners-Lee, "Webizing existing systems," 1998*³

The relationship between Prolog and the Web runs in both directions. Webizing Prolog means making systems designed largely as closed worlds participants in an open one. Prologizing the Web means adding the distinctive capabilities of logic programming to the Web. These are complementary views of the goals just stated.

Webizing Prolog begins with identity. Uniform Resource Identifiers identify nodes, knowledge bases, and other network resources. Actor pids identify actors and include the identity of their hosting node when they cross node boundaries. Meaningful names are thereby no longer confined to a single program or runtime.⁴

The sequential and concurrent forms of interaction described in Section 1.3.2 use ordinary web infrastructure, including HTTP, WebSocket, and representations such as JSON. A node can therefore be used by non-Prolog clients as well as by other Prolog systems. Participation in an open network also brings obligations that a stand-alone interpreter can often ignore: authentication, authorisation, sandboxing, containment, and resource limits are part of webization.

Prologizing the Web means making relations and agents available as network resources. A client can submit a goal to a node and receive bindings for its variables. Because requests and answers share the same term structure, unification provides a declarative interface between client and node. A relation can support several patterns of use without being recast as a collection of unrelated endpoints. Section 9.1.2 develops this claim more precisely.

³ <https://www.w3.org/DesignIssues/Webize>.

⁴ The Semantic Web applies the same discipline to entities, relations, and concepts. Web Prolog terms can likewise contain globally identified resources and concepts, allowing services and agents to reuse established Semantic Web vocabularies. Section 7.2 develops the relationship through the representation of RDF triples as `rdf/3` facts.

Multiple solutions can be returned incrementally, one slice at a time. The concurrent interface also lets a client address a persistent agent, send it messages, and take part in an ongoing interaction. Prolog thereby contributes unification, search, stateful agents, and executable logic to the wider Web. The result is an executable logic layer *within* the Web rather than a closed network available only to Prolog clients. Chapter 4 develops the operational details, while Chapter 9 returns to webizing and prologizing as a pair of paradigm-level commitments.

1.5 Thirty years of Prolog on the Web

We do not want to give the impression that the suitability of Prolog for programming the Web has not been investigated before. On the contrary, in the mid-1990s the advent of the Web sparked a great deal of interest in the logic programming community, reflected in two workshops devoted to the subject and in the development of libraries that mostly relied on creating so-called CGI programs in Prolog. Here is a snippet of text lifted from the call for papers for the second workshop:⁵

This workshop is the second in a series intended to explore the elective affinities between Logic Programming and Internet technologies with emphasis on enhancing the World Wide Web with knowledge, deductive abilities and superior forms of interactive behavior. With the paradigm shift to highly inter-connected computers and programming tools, logic programming languages have a unique opportunity to contribute to practical Internet application development. Simplicity, remote executability, robustness, automatic memory management, are among the features some LP languages share with emerging tools like Java. Superior meta-programming and high-level distributed programming facilities, built-in grammars and dynamic databases, declarative semantics are among their competitive advantages.

Since then the Web has developed at an amazingly fast pace, and web applications have become increasingly speedier and more interactive. The Web of today is *very* different from what it was in the mid-1990s – faster, more reliable, more mobile, and with a fantastic selection of browser APIs that lets a web developer add features such as spoken interaction, video or 3D graphics to an application. There is a reason we nowadays talk about, not only the Semantic Web, but also the Conversational Web, the Mobile Web, the Web of Things, and (more recently) the Agentic Web.⁶ Furthermore, many (potentially) disruptive technologies such as automation, artificial intelligence, blockchain technology, wearable technology, augmented reality, virtual reality, robotics and quantum computing are likely to more or less depend on and be integrated into the Web. In this book we focus on artificial intelligence and its integration into the Web.

With the Web (technically) so much better and so different from what it used to be, there is, in our opinion, every reason to make a new attempt at “enhancing the World Wide Web with knowledge, deductive abilities and superior forms of interactive behavior.” In particular, “superior forms of interactive behavior” is likely to depend

⁵ <http://www.cliplab.org/lpnet/proceedings97/preface.html>

⁶ See (Lager and Myrendal, 2012) for a description of the many facets of the current Web.

on the WebSocket protocol and/or the implementation of Web Prolog in browsers or (mobile) apps. The WebSocket standard did not exist twenty years ago, which meant that every approach to web logic programming had to work solely with HTTP. While HTTP remains the most important protocol (and as we shall see, two out of the three APIs driving the Prolog Trinity ecosystem rely on it), important aspects of some of the inventions described in this book rely on the WebSocket protocol, a so called server-push technology which opens a whole slew of opportunities for developers of web applications based on Prolog.

Regarding the history of Prolog, we do not suggest that nothing has happened between then and now. From the 1990s onward, Prolog has repeatedly adapted to the web's changing landscape. Early work such as the CLIP group's PiLLoW library showed how HTML and HTTP could be handled declaratively by treating web structures as Prolog terms. With the rise of the Semantic Web in the 2000s, Prolog gained renewed relevance: SWI-Prolog's RDF store, web libraries, and the ClioPatria platform demonstrated that a Prolog system could serve simultaneously as a high-performance knowledge base and a full HTTP server (Wielemaker et al., 2008, 2016).

In the 2010s, more interactive and distributed styles of web programming led to further innovations. Pengines (Lager and Wielemaker, 2014) made server-side Prolog engines accessible from JavaScript through a simple RPC model, enabling incremental solution streaming and smooth integration with modern web applications. At the same time, Tau Prolog brought ISO-style Prolog directly into the browser, allowing entirely client-side logic applications implemented in JavaScript (Riaza, 2024).

More recently, WebAssembly has made it possible to run full Prolog systems inside the browser. SWI-Prolog can now be compiled to WASM via Emscripten, and systems such as Ciao, Stryer and Trealla have been ported as well. These provide fast, sandboxed Prolog engines that interoperate smoothly with JavaScript, opening the way for high-performance logic programming in modern web applications.

Among Prolog systems, SWI-Prolog in particular has all the right tools for building server-side support for web applications, in the form of mature libraries for protocols such as HTTP and WebSocket, and excellent support for formats such as HTML, XML and JSON (Wielemaker et al., 2012, 2008). Therefore, it might be argued that since Prolog can and has been used for building server-side support for web applications, it should already be counted as a web programming language. But since this is true also for Python, or Ruby, or Java, or just about any other general-purpose programming language in existence, it would make the notion of a web programming language more or less empty.

What makes Web Prolog a web programming language, in the particular sense intended in this book, is therefore not merely that Prolog programs can act as HTTP servers or be called from a browser. Web Prolog defines a shared, public interaction surface for Prolog computation on the Web. Independently operated nodes are addressed by URIs, expose language-neutral HTTP and WebSocket interfaces, and implement explicit profiles whose observable behaviour can be tested across different Prolog systems. These interfaces specify how goals, answers, alternatives,

messages, failures, and actor lifecycles cross a network boundary. They are also designed for interaction between parties that need not share either an implementation or a common trust domain.

In this respect, Web Prolog is intended to be more than a collection of web libraries for Prolog and less than a claim that all Prolog programming should become web programming. It is a portable web-facing profile of Prolog: a way for logic programs and Prolog actors to become addressable, interoperable components of the Web while remaining recognisably Prolog.

On that basis, we believe that Prolog should claim a more prominent position in this space. Web Prolog should be used not only for building server-side support for web applications, but should be considered as *part* of the Web, in much the same way as HTML, CSS, JavaScript, RDF and OWL are parts of the Web. In other words, as we think it deserves it, we would like it to be recognised as *a web technology in its own right*.

1.6 The Trinity demonstrator

The concepts described in the chapters ahead are accompanied by a running implementation: the *Trinity demonstrator*, a browser-based demonstrator that puts the Prolog Trinity ecosystem in working form.

It is not a finished product. It is an attempt to make the essential ideas observable, reproducible, and interactable, rather than merely readable. The demonstrator is accessible from the Trinity homepage at

<https://trinity.elfenbenstornet.se>

and its companion source code is available in the Git repository linked from that page. Readers who prefer to read the book without stopping to experiment may do so; the text is self-contained. But readers who wish to try things out – to inspect which demonstrator nodes are currently available, to run a query against a live node, to spawn an actor, to load and exercise a statechart, or simply to watch the system respond in real time – will find the demonstrator a useful companion.

The demonstrator presents a workspace on the left and a terminal pane on the right. The workspace has three modes – a *Prolog editor*, a *Statechart editor*, and an embedded *Tutorial* – while the *Terminal* executes queries against the currently selected node using whichever transport mode the node allows. An *Examples* drawer provides one-click loading of the actor and statechart examples resident on each node. A *Logger* panel records all transport-level activity, and its trace filter exposes the step-by-step execution of a statechart as an inspectable audit trail. The implementation is written in SWI-Prolog and JavaScript, and the work has been done in cooperation with several AIs.

A second browser-facing component of the demonstrator is the *Discovery Hub*, itself implemented as the node `n0` and accessible at

<https://n0.elfenbenstornet.se/discovery-hub>.

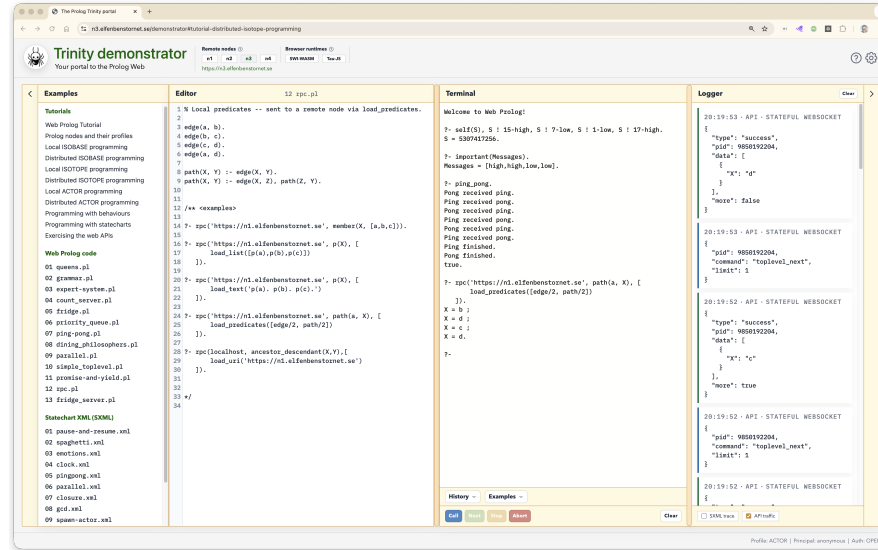


Fig. 1.5 The Trinity demonstrator.

The hub maintains a live directory of the demonstrator nodes, periodically probing them and presenting their advertised profiles, authorisation modes, database portability, and provided predicates and services. Its browser interface is backed by ordinary relations exposed through `n0`'s stateless API: in this sense, the demonstrator uses the Prolog Web to describe the Prolog Web. The hub is discussed more fully in Chapter 3 and 4.

Two caveats deserve mention. First, this is a proof-of-concept, not a production system. Browser-side execution is implemented using SWI-Prolog compiled to WebAssembly, and is described and measured later in the book. Further security hardening and more robust browser and multi-node operation remain open work. The demonstrator is hosted with best-effort availability, and readers should not be surprised if it is occasionally unreachable. Second, the implementation currently covers SWI-Prolog specifically. Chapter 10 considers the wider interoperability opportunity; Chapter 11 returns to the engineering choices that follow from this starting point.

1.7 The structure of the book

To help the reader navigate the book's central ideas and the relationships between them, the mindmap in Figure 1.6 provides a broader view of the three principal aspects of the Prolog Trinity ecosystem.

The mindmap reflects the structure of the rest of this book. Statechart actors appear under actors because they are ordinary Prolog actors whose control structure is

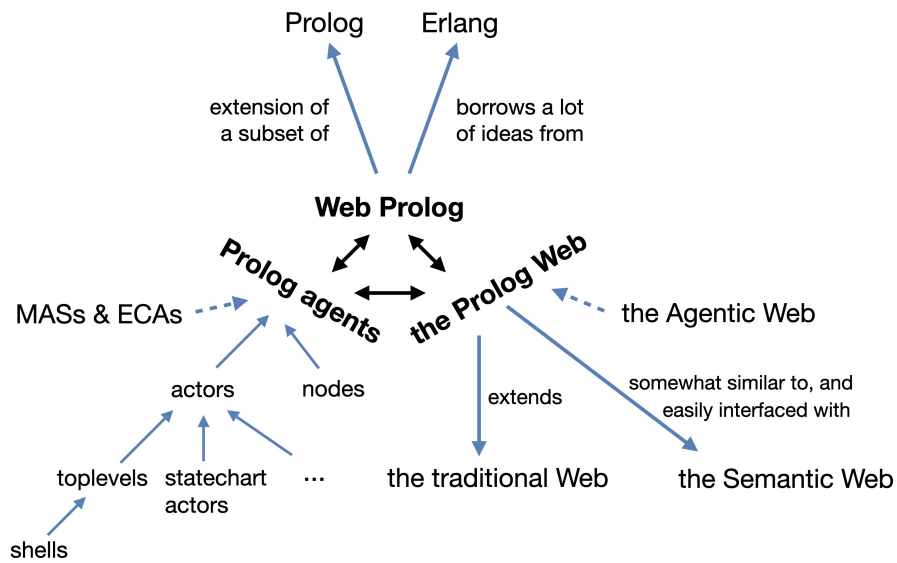


Fig. 1.6 Mind map of the Prolog Trinity ecosystem and its main conceptual threads. MASs means multi-agent systems and ECAs means embodied conversational agents. Chapter 11 redraws the map at the journey's end, distinguishing established relationships from proposed standardisation and adoption paths.

expressed using SXML, a specialised notation within Web Prolog rather than a separate runtime language. Broadly speaking, the exposition moves from foundations to consequences. It begins with the conceptual and architectural core of the Trinity, then turns to reusable behavioural abstractions built on top of that core, then places the proposal in relation to larger technological contexts, and finally steps back to ask what sort of paradigm may be taking shape, who it might matter to, and what would be required to move from vision to execution.

Part I: The conceptual and architectural core

Chapter 2 introduces Web Prolog as a multi-paradigm language for concurrent and distributed programming. It explains what is inherited from standard Prolog, what is added to support actor-style concurrency, and why Erlang is a particularly useful reference point. A range of Erlang programs are translated into Web Prolog to clarify the intended operational behaviour.

Chapter 3 develops the agent strand. It focuses on the two agent forms that matter most for the rest of the book – the Prolog toplevel actor and the Prolog node – and describes the built-in interaction protocol, node architecture, and client–node interaction patterns through a series of scenarios.

Chapter 4 presents the Prolog Web: how multiple nodes compose into an open, decentralised network. It introduces `rpc/2-3` for nondeterministic remote procedure calls, delineates the sequential and concurrent partitions, and addresses timing, failure, programmability, and security as first-class architectural concerns.

Part II: Behaviours and other generics

Chapter 5 shifts from individual actors to reusable behavioural patterns – servers, supervisors, and toplevels – showing how common coordination concerns can be packaged as generic, configurable building blocks. It also introduces higher-level constructs such as `parallel/1` and `first_solution/2`.

Chapter 6 introduces the statechart actor: an event-driven state machine whose control structure is expressed in SXML, a specialised declarative notation within Web Prolog. The chapter explores hierarchy, history, orthogonality, and process invocation, and situates SXML relative to W3C’s SCXML and Erlang’s `gen_statem`.

Part III: The broader technological contexts

Chapter 7 compares the Prolog Web with the Semantic Web and argues that the relationship is cooperative rather than competitive. It moves from conceptual comparison to concrete cooperation patterns involving RDF access, SPARQL-style querying, and OWL-based reasoning.

Chapter 8 turns to AI applications, exploring how statechart-style control and symbolic scripting can be combined for building interactive agents. It revisits classical symbolic AI strengths, examines neuro-symbolic combinations involving large language models, and discusses the governance of user-interface agents.

Part IV: From architecture to paradigm

Chapter 9 asks what conceptual pattern emerges when the Trinity’s components are considered together, and argues that they constitute a candidate paradigm for web-native symbolic computation – characterised by the orthogonality of the two partitions, interaction portability, the agent as the primary unit of structure, a logical foundation that survives distribution, a distribution-first programming model, and the treatment of interaction governance as a first-class dimension.

Part V: What's in it for you and your community?

Chapter 10 asks what the Trinity offers both to the Prolog community and to neighbouring communities. Beginning from the current state of Prolog, it compares responses to systems and community fragmentation and develops the case for federation. It then surveys the wider logic-programming landscape and opens the same network to logic programming, the Semantic Web, AI, web development, and Erlang. The chapter closes by considering the risks and conditions of adoption.

Part VI: Final words

Chapter 11 moves from vision to execution: standardisation, implementation paths, shared ownership, and the practical work required to turn the proposal into a testable reality.

Appendices

Six appendices provide supporting reference and implementation material. The first defines the book's principal terms; the second documents the Web Prolog language and predicate APIs; the third provides the SXML statechart reference; and the fourth specifies the public HTTP and WebSocket protocols of a node. The remaining two describe how a Prolog node can be implemented and compare the performance of the browser, native Prolog, and Erlang actor substrates.

Part I

The conceptual and architectural core

Chapter 2

Web Prolog

Imagine a dialect of Prolog with actors and mailboxes and send and receive – all the means necessary for powerful concurrent and distributed programming. Alternatively, think of it as a dialect of Erlang with logic variables, backtracking search and a built-in database of facts and rules – the means for logic programming, knowledge representation and reasoning. Also, think of it as a web programming language, and as a possible lingua franca for logic-based programming systems on the Web. This is what **Web Prolog** is all about.

Web Prolog – the elevator pitch

2.1 The essence of Prolog

Based on formal logic, a subject dating all the way back to antiquity and tried and tested by generations of logicians and philosophers, logic programming forms a paradigm of its own, very different from the imperative or functional programming paradigms. Prolog is generally regarded as the first logic programming language, and is arguably the most important one. Consider the following program:

```
husband(Wife, Husband) :- wife(Husband, Wife).  
  
wife(socrates, xantippa).  
wife(aristotle, pythias).
```

We have here a *rule* that translates into the following formula in predicate logic:

$$\forall x \forall y [wife(x, y) \rightarrow husband(y, x)]$$

The rule in combination with the two *facts* of the predicate `wife/2` (that need no translation) allow us to query the predicate `husband/2`. We may for example ask if it is true that Aristotle is the husband of Pythias, ask who is the husband of Pythias, ask who is the wife of Socrates, or enumerate the married couples one by one:

```

?- husband(pythias, aristotle).
true.
?- husband(pythias, Husband).
Husband = aristotle.
?- husband(Wife, socrates).
Wife = xantippa.
?- husband(Wife, Husband).
Wife = xantippa, Husband = socrates ;
Wife = pythias, Husband = aristotle.
?-

```

This simple example serves as a reminder that Prolog is not only a logic language, but also a *relational* language that can be used to check if a sentence is true, to look up the value of one argument given the value of another, or to enumerate all pairs of values. When querying a binary relation, these are the *modes* that exist.

Because Prolog can pattern-match on structured terms in clause heads, `append/3` defines list concatenation with a base case and a recursive case that peels off the head `H` of the first list and rebuilds it on the result:

```

append([], L, L).
append([H|L1], L2, [H|L]) :- append(L1, L2, L).

```

All three arguments are relational – any of them may be given or left as variables – so `?-append(Xs, Ys, Zs)` holds precisely when `Zs` is `Xs` followed by `Ys`.

Complex terms in the arguments of clauses ensure that Prolog is a *Turing complete* programming language. This is what makes it different from a language such as Datalog, which is similar in many ways, but is not Turing complete.

In order to be not only Turing complete but also a practical and efficient programming language, serious Prolog systems need to offer a lot more, and they always do. In Section 2.1 of *Fifty Years of Prolog and Beyond*, the authors provide a conceptual and minimalist definition of the important features of Prolog, and are thus able to draw a line between what can be considered a Prolog implementation and what can not. Actually, they draw *two* lines, since they distinguish the *essential* features of Prolog from *important* (yet non-essential) features. Below, we reproduce their list of features, where 1-6 are considered essential features and 7-12 less essential. As it turns out, all the essential features except for 2) and 6) are involved when querying the binary relation between the married couples, or the ternary relation between the lists, in the examples given above.

1. Horn clauses with variables in the terms and arbitrarily nested function symbols as the basic knowledge representation means for both programs (a.k.a. knowledge bases) and queries;
2. the ability to manipulate predicates and clauses as terms, so that meta-predicates can be written as ordinary predicates;
3. SLD-resolution (Kowalski, 1974) based on Robinson's principle (1965) and Kowalski's procedural semantics (Kowalski, 1974) as the basic execution mechanism;

4. unification of arbitrary terms which may contain logic variables at any position, both during SLD-resolution steps and as an explicit mechanism (for example, via the built-in `=/2`);
5. the automatic depth-first exploration of the proof tree for each logic query;
6. some control mechanism aimed at letting programmers manage the aforementioned exploration;
7. negation as failure (Clark, 1978), and other logic aspects such as disjunction or implication;
8. the possibility to alter the execution context during resolution, via ad-hoc primitives;
9. an efficient way of indexing clauses in the knowledge base, for both the read-only and read-write use cases;
10. the possibility to express definite clause grammars (DCG) and parse strings using them;
11. constraint logic programming (Jaffar and Lassez, 1987) via ad-hoc predicates or specialised rules (Frühwirth, 2009);
12. the possibility to define custom infix, prefix, or postfix operators, with arbitrary priority and associativity.

One cannot avoid noticing that, as far as we know, no other community in support of a programming language has found itself in the position of having to determine what should be considered a *real* language of this kind. Many systems can rightly claim to implement Prolog while remaining incompatible in important ways. This diversity helped motivate a shared standard. Since 1995, the core features of Prolog have been standardised by ISO, documented in a report known as ISO/IEC 13211-1:1995.¹

Prolog belongs to a much broader family of logic-programming languages and formalisms, including abductive, inductive, constraint, probabilistic, Datalog, and answer-set approaches. Web Prolog is specifically grounded in Prolog and its ISO core; Section 10.3.1 surveys the wider landscape and considers how systems with different semantics might expose selected capabilities through shared interaction surfaces.

2.2 Web Prolog in a nutshell

Web Prolog is designed as a superset of a subset of the ISO Prolog core standard. The dialect defined by ISO is well understood, and is reasonably close to most of the dialects used in Prolog textbooks. The ISO Prolog syntax specification as well as a large subset of its built-in predicates form an excellent point of departure for our attempt to create a design and a standard for Web Prolog.

The conventional Prolog source syntax of Web Prolog is exactly as in ISO Prolog, except that three infix operators (`!/2`, `if/2` and `@/2`) that are not in the standard are defined. They can easily be added by means of `op/3`, the ISO Prolog predicate

¹ <https://www.iso.org/obp/ui/#iso:std:iso-iec:13211:-1:ed-1:v1:en>

This chapter concentrates on conventional Prolog source notation. Web Prolog also provides specialised declarative notations for particular programming tasks. Most notably, SXML provides the statechart notation described in Chapter 6. It is not a separate implementation language, but an integrated notation that shares Web Prolog’s actor runtime, term and message model, logical environment, and executable content.

[illegible]

The current Web Prolog definition includes most of the predicates in this cloud; the precise portable predicate requirements are listed in Appendix B. Those predicates are necessary but not sufficient. The current definition also makes the following choices:

- In addition to the built-in predicates offered by the ISO Prolog core, nodes that accept general Prolog goals must supply a set of *standard library predicates*. These predicates are available without explicit loading. The *Prologue for Prolog* lists `member/2`, `append/3`, `length/2`, `between/3`, `select/3`, `succ/2`, `maplist/2-8`, `nth0/3`, `nth1/3`, `nth0/4`, `nth1/4`, `call_nth/2`, `foldl/4-6` and `countall/2`.² See also the public-domain set of libraries developed by the Prolog Commons Working Group.³
- Item number 10 in the list of essential and important features of Prolog points to the possibility to express Definite Clause Grammars (DCGs) and parse strings using them. DCG notation and the associated parsing primitives are specified separately as an extension, in ISO/IEC 13211-3 (Part 3: Definite clause grammar rules).
- Web-facing access is provided through web APIs and remote-call predicates described later in the book. A host system may additionally expose facilities such as `http_open/3`, but such host-library predicates are not silently part of the portable Web Prolog surface. JSON representation at the protocol boundary is specified separately in Appendix D.
- At its fullest, Web Prolog extends its ISO-based sequential core with carefully crafted concurrency and distribution primitives heavily inspired by Erlang. It uses the same kind of actor and, where appropriate, the same names for corresponding operations, while making the deliberate deviations described later in this chapter.

The first three choices largely consolidate existing Prolog and Web practice. The fourth is a more substantial language-design commitment. Even so, Erlang provides a clear precedent for the process, messaging, and failure mechanisms on which the concurrent part of Web Prolog is based.

The portable definition deliberately omits some features, built-in predicates, and libraries. Some would cross the security boundary of a public node, some duplicate facilities supplied by the node itself, and others are not yet ready for standardisation:

- Only a subset of ISO Prolog is exposed. Predicates that grant ambient access to the operating system are outside the portable surface; a node owner may provide controlled capabilities separately.
- Predicates for constructing web servers, such as those in SWI-Prolog's HTTP server library, are not portable Web Prolog facilities. A Prolog node already supplies the required server boundary.
- Constraint programming is not currently standardised beyond facilities inherited from ISO. Individual nodes may offer additional constraint predicates; a shared definition can be adopted later if implementation experience supports one.
- The portable definition does not currently standardise a module system. Nodes and, where present, actors already provide important namespace and encapsulation boundaries, while host-specific modules remain an implementation facility.

² <https://www.complang.tuwien.ac.at/ulrich/iso-prolog/prologue>

³ <http://prolog-commons.org>

Web Prolog is therefore a profile of Prolog for programming on the Web. Section 3.5 divides it into nested sub-profiles with progressively broader language and interaction surfaces. Narrower sub-profiles may omit database updates, terminal I/O, or `op/3`.

The word *profile* is used at two related levels. Web Prolog is a language profile: the portable language surface defined by this book. The node sub-profiles introduced in Section 3.5 are conformance profiles: cumulative, testable promises about the language and interaction surfaces a particular node exposes. This chapter describes the full Web Prolog language unless a profile-specific restriction is stated. At the narrowest relational boundary, the terms accepted as goals may be restricted to query patterns over an exposed schema; conformance does not require a general Prolog engine behind that surface.

2.2.1 Web Prolog is inspired by Erlang

And if you think I would prefer multi-threading in Prolog to look as much as possible like Erlang, why yes, I would. I would like that very much.

*Richard O’Keefe*⁴

Erlang is a general-purpose concurrent functional programming language developed by Joe Armstrong, Robert Virding and Mike Williams in 1986. It began as a proprietary language within Ericsson, but was released as open source in 1998. Originally designed for telephony applications, Erlang later proved highly effective for backend web systems and for a wide range of soft real-time control problems. Regarded by many as the first actor programming language to achieve broad practical success, Erlang is especially notable for its built-in support for concurrency, distribution and fault tolerance.⁵

A good overview of the language, its design intent and its underlying philosophy is given in Joe Armstrong’s Ph.D. thesis (Armstrong, 2003b). There, the role of concurrency is described as threefold:

1. In Erlang, the fundamental role of concurrency is architectural. Systems are structured as sets of communicating processes that interact exclusively by message passing, which yields a natural decomposition into well-defined components that can be independently implemented and tested.
2. A system designed in this way can also be executed efficiently on multiprocessor machines or distributed across a network. But concurrency was not introduced in Erlang merely as a mechanism for improving performance. Efficiency is only a potential benefit, and depends on whether the application can in fact be partitioned into genuinely independent tasks.

⁴ <https://groups.google.com/g/erlang-programming/c/1jdsnqZ4XfQ/m/ve9WfF12YBwJ>

⁵ The actor model itself originated with Carl Hewitt and colleagues in the early 1970s (Hewitt et al., 1973); Web Prolog’s concrete concurrency design, however, draws most directly on Erlang. Book-length introductions to Erlang include *Concurrent Programming in Erlang* and Armstrong’s *Programming Erlang* (Virding et al., 1996; Armstrong, 2007b)

3. The main purpose of this architectural discipline is to limit the consequences of software errors. As Armstrong argues, large reliable systems are best built by partitioning them into independent concurrent processes that share no data and communicate only by message passing, combined with mechanisms for monitoring and restarting processes when they fail (Armstrong, 2003b, pp. 18–19).

Armstrong refers to this paradigm as *concurrency-oriented programming* (COP). This idea is central to the design of Web Prolog, which is itself conceived as a language for COP. The actor processes introduced in this book are not merely devices for exploiting parallelism. They are the primary units of organisation in a distributed system on the Web. In that sense, they are not just concurrency mechanisms but *agents* – a perspective that will be developed more fully further ahead in the book.

Inspired by the list of essential features of Prolog given in the previous section, we may also ask what it would mean for a language to count as Erlang-style. At a minimum, such a language would seem to require the following:

1. the ability to execute a large number of actor processes concurrently;
2. the ability to enforce process isolation, so that actors share no mutable state and communicate exclusively by asynchronous message passing;
3. the ability to support network-transparent concurrent programming, allowing actors to create other actors on remote computers and communicate with them seamlessly.

Added to the essential features of ISO Prolog, these facilities provide crucial parts of the foundation for Web Prolog, and indeed for the wider Prolog Trinity ecosystem. For this synthesis to work, however, the two sets of features must be compatible. Our current view is that they are, and part of the purpose of this book is to demonstrate that claim.

Why did we choose Erlang as a source of inspiration, rather than any alternative? After all, there are other actor programming languages – Akka, Pony and E, to name a few. The main reason for the choice of Erlang is that it is arguably the most *mature* concurrency-oriented language in existence, with a larger community than the alternatives, and a community that includes a fair number of industrial users. Many millions of lines of source code have been written in Erlang, many books teaching the language have been authored, and university courses teaching concurrent and distributed programming often use Erlang. In our opinion, since we are aiming for a standard it makes a lot of sense to borrow the required concurrency-oriented features from a language as mature and battle-tested as Erlang.

Another reason for our choice is that Erlang and Prolog are in many ways remarkably similar, both when it comes to syntax, and the reliance on dynamic typing, assign-once variables, pattern matching and recursion. The similarities can be explained by the fact that historically, Erlang evolved from Prolog and the first version of Erlang was actually implemented as an interpreter in Prolog (Armstrong, 2007a). The following listings of Erlang code (on the the left) and Web Prolog code (on the right) show some of the similarities as well as some of the differences between the two languages:

```

append([], L) -> L ;
append([H|L1], L2) ->
    [H|append(L1, L2)].

append([], L, L).
append([H|L1], L2, [H|L]) :-
    append(L1, L2, L).

```

Note the use of capital letters for variables, the familiar notation for lists, and the reliance on pattern matching and recursion. A major difference is that Erlang is a functional programming language, whereas Web Prolog is relational. Since a function is just a special case of a relation this difference must not be exaggerated, but it does show in the way function calls may be nested, something that cannot be done in Prolog (or Web Prolog) since arguments are used to represent outputs as well as inputs. What *can* be done in Prolog, however, but not in Erlang, is to call `append/3` in more than one *mode* – not only to append one list to another, but also, for example, to nondeterministically split a list up into two parts. Furthermore, thanks to logic variables, `append/3` is tail-recursive in Prolog, but not in Erlang’s `append/2` (as it is written here).

Despite such differences, as long as we do deterministic computation only, and no search is involved, logic programming and functional programming are fairly similar in the way they work, and methods used to achieve success with one often transpose to the other. Again, features such as pattern matching, assign-once variables and recursion play important roles in both languages.

During its evolution, Erlang gradually shed several features that Prolog had – and still has – while at the same time developing exceptionally strong support for concurrent and distributed programming, an area where Prolog historically lacked, and still lacks, comparable built-in support. Key here is the concept of an actor, and the ways actors can be scripted. Below, the echo actor written in Web Prolog is placed side-by-side with an implementation in Erlang – a function in Erlang and a predicate in Web Prolog – both making a recursive call that implements a loop:

```

echo_actor() ->
    receive
        {echo, From, Msg} ->
            From ! {echo,Msg},
            echo_actor()
    end.

echo_actor :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    })).

```

The Web Prolog code on the right demonstrates the use of two constructs foreign to traditional Prolog, implementing the sending and receiving of messages in the style of Erlang. The programs have a similar look, and this is intentional. We have *strived* to make Web Prolog look as similar as possible to Erlang within the constraints imposed by the syntax and semantics of ISO Prolog.

A function call such as in `Pid = spawn(fun() -> echo_actor() end)` (or, but only for zero-argument functions, `spawn(fun echo_actor/0)`) can be made in order to spawn our echo actor in Erlang, while doing it in Web Prolog would use something like `spawn(echo_actor, Pid)`. These calls look somewhat similar too, but while Erlang is a higher-order language in which the spawn function takes an anonymous function as its argument, Prolog (or Web Prolog) is not a higher-order language in this sense. In Web Prolog, `spawn/2` is a *meta predicate* which expects a callable term to be passed in the first argument, a term that when called will execute a procedure that determines how the actor will behave.

2.2.2 Web Prolog is a multi-paradigm programming language

A common way to categorise programming languages is in terms of the programming paradigm(s) they support. In the very nicely organised and very well argued taxonomy of programming paradigms by Peter Van Roy in (Van Roy, 2009), depicted in Figure 2.2,⁶ it is indeed possible to pinpoint exactly where Web Prolog fits in. We have enclosed the relevant square boxes in boxes with rounded corners.



Fig. 2.2 A taxonomy of programming paradigms due to Peter Van Roy.

⁶ The diagram is available at <https://www.info.ucl.ac.be/~pvr/paradigmsDIAGRAMeng108.pdf>

This suggests that we can regard Web Prolog as a language that supports three programming models: *relational logic programming* (as in Prolog), *imperative search programming* (also as in Prolog), and (as in Erlang) *message-passing concurrent programming*.

With access to predicates for asserting and retracting clauses in the dynamic database, traditional Prolog has always, albeit somewhat reluctantly, been able to support imperative programming (and later additions such as `nb_setval/2` have of course amplified this). Also, the built-in backtracking *search* that is so useful in Prolog has little to do with logic. Thus Prolog has never really been a single-paradigm programming language, and this is reflected in Van Roy’s taxonomy. In (Van Roy, 2009) he suggests that this is not a bad thing:

A good language for large programs must support several paradigms. One approach that works surprisingly well is the *dual-paradigm language*: a language that supports one paradigm for programming in the small and another for programming in the large.

With the addition of support for the actor programming model, Web Prolog becomes a much clearer as well as (in our view) a more interesting case of a multi-paradigm programming language. In particular, it can be argued that the support for programming in the large has been greatly improved.

Any unexpected interactions between paradigms?

It is probably wise to entertain a suspicion of unexpected interactions between language features and possible impedance mismatches between the two paradigms – between Prolog’s relational, nondeterministic programming model based on logic and Erlang’s functional and message passing model. How well do the Erlang-style constructs mix with Prolog – with backtracking for example, or with the features for imperative programming? What do we get if we combine them? A kludge, or something quite beautiful?

In theory, we should be on the safe side. Sequential Erlang is basically Erlang with its data types, one-way pattern matching, functions and control structures, but without `spawn`, `send`, `receive`, and other constructs used for concurrent programming. The idea behind Web Prolog can thus be described as an attempt to “plug out” the sequential part from Erlang and “plug in” sequential Prolog instead. There seems to be no principled reason why we would not be able to replace the sequential functional language with a sequential relational language such as Prolog. Indeed, Armstrong (2003a) writes:

Erlang is a concurrent programming language with a functional core. By this we mean that the most important property of the language is that it is concurrent and that secondly, the sequential part of the language is a functional programming language.

The sequential subset of the language expresses what happens from the point it time [sic] where a process receives a message to the point in time when it emits a message. From the point of view of an external observer two systems are indistinguishable if they obey the principle of observational equivalence. *From this point of view, it does not matter what family of programming language is used to perform sequential computation.* (Our emphasis.)

Alternatively – and this has been our choice – the approach can be described as an attempt to *extend* Prolog with constructs such as `spawn`, `send` and `receive`. The idea is to keep everything that core Prolog has to offer, and extend it with a number of those primitives that make Erlang such a great language for programming message-passing concurrency. The choice between extending Prolog with Erlang-style constructs and extending Erlang with Prolog-style constructs is easy to make, and a lot has to do with syntax. Provided we can accept using a syntax which is relational rather than functional, precluding the nesting of function calls, it may be clear already at this point that the surface syntax of Prolog can easily be adapted to express the needed Erlang-style primitives. It is arguably a lot harder to express Prolog rules and other constructs using the syntax of Erlang.

2.2.3 Web Prolog as a scripting language for the Web

Scripting languages are a lot like obscenity. I can't define it, but I'll know it when I see it.

Larry Wall

Over the years, the Web has become a key delivery platform for a variety of sophisticated interactive applications in just about any conceivable domain. As a consequence, JavaScript, more or less the only game in town for programming the front-end of a web application, has become among the most commonly used programming languages on Earth. Indeed, JavaScript must be regarded as *the* web programming language of our times.

JavaScript started out as a very small, highly domain-specific scripting language that was limited to running within a web browser to dynamically modify the web page being shown, but has over time evolved into a widely portable general-purpose programming language. Modern JavaScript is a high-level, dynamically typed, interpreted multi-paradigm programming language with several essential features that make it a versatile tool for web development. JavaScript's syntax supports various programming paradigms, including imperative, functional, and object-oriented programming styles. With features such as callbacks, promises, and `async/await`, JavaScript supports asynchronous programming, enabling non-blocking operations, especially useful in web applications. JavaScript's native format for data interchange, JSON, is lightweight and widely used for data transmission. JavaScript runs on almost all modern web browsers and platforms, making it universally applicable for web development. It easily interfaces with numerous web APIs for tasks like accessing the Document Object Model (DOM), making HTTP requests, and handling events. This makes it an excellent tool for connecting disparate systems in web applications.

With the advent of Node.js,⁷ JavaScript extended its reach to server-side development. This allowed for a unified language experience across both client and server sides, simplifying the development process by allowing the same language to be used

⁷ <https://en.wikipedia.org/wiki/Node.js>

throughout the entire stack. Not having to learn more than one language in order to become a so called “full-stack developer,” and not having to switch languages when changing from working on the server-side to working on the client-side are important advantages. Also, Node.js is frequently used to build and connect microservices. Its non-blocking I/O model and event-driven architecture make it suitable for building scalable and efficient network applications.

It is clear that as a relational logic programming language with the essential and important features listed above, Prolog is very different from JavaScript in terms of paradigms supported. The addition of Erlang-style concurrency and asynchronous communication does not really narrow the gap, as those are features that work differently in Erlang. What matters here is the *role* that JavaScript plays on the Web. We want Web Prolog to play a similar role in the Prolog Trinity ecosystem as JavaScript does in the JavaScript ecosystem, just as clear-cut.

The connection between scripting languages and the Web has been noted before. For example, in his article “Scripting: higher level programming for the 21st Century,” John Ousterhout (1998) writes (about the Internet rather than the Web, but that makes little difference):

The growth of the Internet has also popularized scripting languages. The Internet is nothing more than a gluing tool. It does not create any new computations or data; it simply makes a huge number of existing things easily accessible. The ideal language for most Internet programming tasks is one that makes it possible for all the connected components to work together; that is, a scripting language.

So what about the suitability of Web Prolog for programming the Web? Can Prolog challenge JavaScript in this space, at least for some applications, such as web-based AI? For this to be possible, we must ensure that Web Prolog is

- viable as a scripting language,
- suitable for programming the Web,
- possible to implement in browsers, and
- secure.

Web Prolog as a scripting language

As regards the viability of Web Prolog as a scripting language we note that while hardly a definition, Ousterhout (1998) characterises scripting languages as follows:

1. They are suitable for “programming in the large,”
2. they are designed for “glueing” applications together,
3. they tend to be typeless, thus making it easier to connect components,
4. they are usually interpreted, thus providing rapid turnaround during development by eliminating compile times,
5. they are higher level than a system programming language in the sense that a single statement does more work on average, and
6. they allow rapid prototyping.

JavaScript does indeed fit this characterisation, but given the development described above, it should now probably be described as a general-purpose language which also happens to be suitable for scripting. By contrast, languages such as C++ and Java are not well suited to that purpose.

What about Prolog? Does it satisfy Ousterhout's characterisation? The points 3-6 in his list are probably true of the Prolog programming language in general. (With the possible exception of 4 – some systems compile, but do it very quickly.) Whether points 1 and 2 apply varies between systems. For example, the people behind the commercial SICStus Prolog appear to regard Prolog more as a language for writing problem-solving modules to be embedded into other code, written in Java, C++, Python or what have you.⁸ It is only a guess, but we suspect that the people behind SICStus Prolog would not be willing to characterise it as a scripting language, although that may simply be a business decision more than anything else. In contrast, acting as a glue language and a language for programming in the large is the ambition of SWI-Prolog, witness the following quote from the online manual:⁹

SWI-Prolog positions itself primarily as a Prolog environment for 'programming in the large' and use cases where it plays a central role in an application, i.e., where it acts as 'glue' between components.

Interestingly, the people behind Erlang also think of it as a glue language:

We use Erlang as the glue to handle all orchestration, and then we use Python, C, Julia, ... It is actually a language *intended* to act as a hub towards other languages. The interfaces could be protocols, could be RESTful APIs, or other programming languages. It's ideal for that.¹⁰

Erlang-style, network-transparent message-passing concurrency seems ideal for programming in the large on the Web, and for orchestrating both local and remote processes.

Erlang glue is likely a different kind of glue, with different properties, than Prolog glue. There is hope that combining Prolog and Erlang might yield an even stronger and more flexible glue of the required kind – a *superglue*, if you will.

Despite the good fit with Ousterhout's characterisation, neither Prolog nor Erlang are usually *advertised* as scripting languages, and they were hardly *designed* for the purpose of glueing applications together (or at least Prolog was not). They are both full-blown very flexible general-purpose programming languages with "batteries included." Like JavaScript, they should probably be regarded as general-purpose languages which also *happen* to be suitable for scripting.

Web Prolog is a web-facing Prolog profile rather than a special-purpose language. It can serve as a scripting and glue language, but also supports applications for which "scripting" is not really the right word. Its design focuses on web *logic* programming and web *agent* programming, in client-side browsers as well as on servers. Using Web Prolog, the owner of a node can, for example, build knowledge bases consisting of many millions of facts and rules, and web agents that make use of those knowledge bases.

⁸ Mats Carlsson, main developer of SICStus Prolog, personal communication

⁹ <http://www.swi-prolog.org/pldoc/man?section=swiprolog>

¹⁰ See <https://www.youtube.com/watch?v=K8nXTSPHZhs>, 8:58 into the discussion.

Is Web Prolog suitable for programming the Web?

To demonstrate that Web Prolog is suitable for programming the Web is what this book is about. But we are going further than that, as we are also aiming for a novel extension of the Web – the Prolog Web and even an Agentic Web – and a complete ecosystem, for which Web Prolog will serve as the default language.

Web Prolog running in browsers

Regarding our third point, for Web Prolog to become a viable web programming complement to JavaScript it is vital that it can be implemented in browsers too, since this comes with a number of advantages:

1. When network connection is slow, it is best to perform the majority of computations in the browser.
2. This is where, in most scenarios, the *state* of an interaction between a user and an application should preferably be represented.
3. Client-side computation reduces the demands placed on nodes.
4. This is also where we find browser APIs that lets a web developer manipulate the DOM, store data locally, and add features such as spoken interaction, video or graphics to an application.

For Web Prolog in the browser to play the same role as JavaScript in the browser currently does, it must allow Web Prolog source code to be loaded from any server on the Web by means of the HTML `<script>` element, inline or with a link.

We do not, however, go all the way in our attempt to replicate JavaScript's abilities as a scripting language. One thing that we do not at this time aim for is to make Web Prolog into a language with primitives for scripting the Document Object Model (DOM) in web browsers. With time it may well be interesting to develop such facilities, but we believe it is simply too early to try to standardise them. In fact, we suspect that JavaScript will reign supreme in that role, and that if a browser is running Web Prolog locally, it will be as a *web worker*,¹¹ with which the main JavaScript process is talking.

Web Prolog and security

As for security, similar to JavaScript, Web Prolog is a *sandboxed language*, open to the execution of untested or untrusted source code, possibly from unverified or untrusted clients without risking harm to the host machine or operating system. Therefore, Web Prolog does not include predicates for file I/O, socket programming or persistent storage, but must rely on the host environment in which it is embedded for such features.

¹¹ https://en.wikipedia.org/wiki/Web_worker

Web Prolog does indeed share some of the properties that made JavaScript succeed on the Web. A visit to a web application server starts a JavaScript process on the client, running code that has been downloaded from the application's host to the user's client. Such a process must be allowed to run there (that is, the owner of the client must allow it), and when it is (and most users allow it by default), it must execute in a way that does not harm the client. When a Web Prolog process is run on a node it must be allowed by its owner to do so, and it must run without harming the node. So one thing they share, Web Prolog and JavaScript, is the ability to run untested and untrusted source code, authored by unverified and untrusted programmers, in a sandbox on someone else's computer.

In summary, while JavaScript's initial role was confined to client-side scripting in web browsers, its reach has expanded significantly. Today, it serves as a versatile glue language that connects various components in both client and server environments, as well as in the broader web development ecosystem. When designing Web Prolog and the rest of the Prolog Trinity ecosystem, we must of course try to learn from such a popular and versatile tool for web development, with a focus on the good parts in particular.

2.3 Erlang-style message-passing concurrency in Web Prolog

In a nutshell, if you were an actor in Erlang's world, you would be a lonely person, sitting in a dark room with no window, waiting by your mailbox to get a message. Once you get a message, you react to it in a specific way: you pay the bills when receiving them, you respond to Birthday cards with a "Thank you" letter and you ignore the letters you can't understand.

Fred H  bert

Corresponding to the send, receive, and spawn operations are built-in Web Prolog predicates `!/2`, `receive/1-2`, and `spawn/1-3`, among others. Their use is demonstrated through a series of simple examples, many drawn from Erlang tutorials and introductory texts. The examples are pitched at two kinds of reader. Prolog programmers taking their first steps into concurrent programming may wish to work through them carefully, perhaps alongside the Trinity demonstrator. Experienced Prolog teachers, by contrast, are invited to read as evaluators: does this material constitute a sound basis for teaching actor-style concurrency to Prolog students, and does Erlang-style concurrent programming sit naturally within a Prolog setting? We believe it does – that Web Prolog offers a promising vehicle for teaching both logic programming and actor programming within a single, web-based system, with minimal setup overhead for instructor and student alike.

2.3.1 Programmer talking to actor, actor talking to itself

If we need to hold a Prolog-style conversation with an actor, we may want to talk to a Prolog *shell*. By using a terminal attached to a shell, we can interact with it the way we normally interact with Prolog, by querying it, updating its dynamic database, and so on. For example, here is how we instruct the shell to split a list up into a prefix and a postfix using `append/3`:

```
?- append(Xs, Ys, [a,b,c]).
Xs = [], Ys = [a, b, c] ;
Xs = [a], Ys = [b, c] ;
Xs = [a, b], Ys = [c] ;
Xs = [a, b, c], Ys = [] ;
false.
?-
```

This book, as we have warned, is not a beginner's textbook, so how Prolog is able to come up with solutions to such queries is not something we will cover. Our focus is rather on the concurrency-oriented, message-passing features of Web Prolog, so we begin instead by introducing a couple of simple messaging primitives.

First, using `self/1`, we need to determine the identity of the shell we will be talking to:

```
?- self(Self).
Self = 85234512.
?-
```

What we got back here is a *pid*, an address for an actor process. Every pid has a global form `Id@Node`, where `Id` is assigned to the process by the node and `Node` is the node's URI. This form is part of the language contract, not an opaque implementation detail: programs may copy, construct, decompose, compare, and serialise pid terms. The short decimal identifiers printed in examples are typographical stand-ins for node-assigned values, not a prescribed pid size. Thus, the full pid displayed above would be something like

```
85234512@'http://n7.org'
```

The shell has a presentation setting that may suppress the node component when the pid names a process on the shell's own node. With that setting enabled, the same pid is printed simply as 85234512. This is only a display abbreviation: internally, and whenever the pid is serialised or transmitted to another node, the node component remains part of the pid. A pid embedded in a message therefore remains a globally meaningful return address even after the message crosses a node boundary.

Global identity comes from the complete `Id@Node` pair, rather than from the identifier component alone. Constructing such a pair does not create an actor or prove that one is live. It addresses an actor only when `Id` is an identifier assigned at `Node`, and every operation remains subject to the node's authorisation policy. The

demonstrator chooses local identifiers from a large, unpredictable space and does not reuse them during a node run.

As stated above, if we know the pid of an actor process we can send it a message. Just as any other kind of actor, the shell is equipped with a mailbox, and this is where the message will end up. The syntax and semantics of the send operator is easy to explain. With a call such as `Actor ! Message` the term `Message` is sent to the mailbox of the process identified as `Actor`, either by means of a pid or (as we shall see) using a registered *name*. For example, using `!/2` with the pid of our shell we can instruct it to send *itself* a message:

```
?- 85234512 ! hello.
true.
?-
```

Unless a continuation is stated explicitly, transcript blocks in this book should be read as independent shell sessions; their illustrative pids may therefore differ. Now we can use `receive/1` to make sure that the message we sent was received by the shell and is present in its mailbox. A single receive clause with a variable in the head and `true` in the body does the job:

```
?- receive({Message -> true}).
Message = hello.
?-
```

This is one of the simplest uses of the receive operation we can think of, but `receive/1` really is more complex than this, so expect more to come further ahead in this chapter. One more thing about the above call to `receive/1` is worth a mention already at this point though: had the mailbox been empty, `receive/1` would have *blocked* the execution of the process running as a shell until a message shows up.

There is an even simpler call to `receive/1` that one can make, namely this:

```
?- receive({}).
```

This call does not wait for any specific message, which just means that it will block forever, or until the process that runs it is killed. Control-C will work and send the user back to the `?-` prompt.

2.3.2 Two utility tools for programming in the shell

Copying and pasting pids is not very convenient. Instead, we can use a small shell feature: if a variable name is prefixed with a dollar sign, the shell substitutes it with the variable's most recent binding before running the query. For example:

```
?- $Self ! hello.
true.
?-
```

Here, the shell kept track of the most recent binding of `Self` to a value (85234512 in this case) and substituted the term `$Self` with that value before the query was run.¹²

During interactive development, it is also important to inspect the shell's mailbox. Using `receive/1` is often awkward here: if the mailbox happens to be empty, `receive/1` will block, effectively freezing the session until the call is interrupted (for example, with Control-C). Moreover, if we want to examine *all* pending messages, we would need to call `receive/1` repeatedly, without necessarily knowing how many messages there are – which again risks blocking, or that messages are left behind.

A more convenient tool is the utility predicate `flush/0`. It prints (and removes) all messages currently in the shell's mailbox and, crucially, it never blocks.

Below, we demonstrate both of these utilities by first sending yet another message to our shell and then use `flush/0` to inspect and empty the contents of its mailbox:

```
?- $Self ! goodbye.
true.
?- flush.
Shell got hello
Shell got goodbye
true.
?-
```

The `$Var` substitution mechanism (or “dollar notation”) in combination with `flush/0` comes in very handy during interactive programming in the shell and we shall rely on them extensively when running examples through all of the book.

2.3.3 Actors talking to other actors

It takes two to tango, and an actor talking to itself is not very interesting. Fortunately, as already mentioned, an actor is capable of spawning other actors and then start talking to them.

The built-in predicate `spawn/1-3` is used to create new actor processes. In a call such as `spawn(Goal, Pid)` it expects a callable goal to be passed in the first argument, and will bind the variable in the second argument to the pid of the actor created. Here is how this might look:

```
?- spawn(echo_actor, Pid).
Pid = 72347585.
?-
```

¹² Dollar notation is a convenience provided by the demonstrator's shell, not part of ISO Prolog or the portable Web Prolog language. Its notation and the lifetime of remembered bindings are properties of the client interface; Web Prolog programs should not depend on them.

The goal calls a predicate which determines the behaviour of the actor. The actor process runs in parallel with the caller, the shell in this case. Here is an example script that implements a simple echo server:

```
echo_actor :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    }).
```

We have seen this piece of code before; it is now time to explain how it works. The predicate `echo_actor/0` implements a simple receive loop: each call to `receive/1` selects a message of the form `echo(From, Msg)` from the mailbox, sends `echo(Msg)` back to the actor identified by `From`, and then recurses.

To test the server interactively, we send it a message with `From` bound to the shell's own pid, obtained via `self/1`:

```
?- self(Self), $Pid ! echo(Self, hi).
Self = 85234512.
?- flush.
Shell got echo(hi)
true.
?-
```

Sending is asynchronous: `!/2` does not block waiting for a response but continues immediately. Also, sending a message to a pid always succeeds and never throws an error, even if the pid points to a non-existing process.

In that case the message is simply discarded. This means that short of having the targeted actor return a confirmation message we will not always know if the message reached it. However, the above case did not leave us in the dark, as the actual echo served as a confirmation message.

Web Prolog guarantees per-sender FIFO delivery for repeated sends to the same pid within one routing epoch (one continuous use of a single delivery route), and for repeated sends through one unchanged name binding within that epoch. There is no portable order across different senders, pid/name addressing, name rebinding, reconnects, or rerouting, nor between a mailbox message and a runtime signal. The exact message-order matrix and the separate signal channel are treated in Sections 4.1.6 and 4.1.7.

Failure, exceptions, and process termination

Several mechanisms that are easy to blur together meet at an actor boundary. We shall use the following distinctions throughout the book. An ordinary Prolog goal may *fail*, in which case it simply has no solution, or it may *throw an exception*. In SWI-Prolog, system exceptions normally have the form `error(Formal, Context)`.

The formal part classifies the error; the context is supplied by the implementation and may identify a predicate, source location, or stack context. Code may handle an exception locally with `catch/3`, arrange cleanup with predicates such as `setup_call_cleanup/3`, translate it into another exception, or rethrow it.

If an exception is not caught inside an actor, the actor terminates. A monitor does not catch that exception or keep the actor alive; it makes the termination observable by delivering `down(Pid,Ref,exception(E))`. Logical failure at the outermost actor goal is reported separately, with `false` as the reason, while successful completion uses `true`. An explicit call to `exit/1-2` may supply another reason altogether. Thus an exit reason describes process termination; it is not necessarily an error.

An exit reason should not be confused with every possible `down/3` reason. In particular, `connection_closed` discharges a remote monitor because observation was lost; it does not assert that the monitored process terminated.

An uncaught SWI-Prolog system exception is sanitised when it crosses this monitor boundary. The notification retains its formal error classification but elides the implementation-supplied context. For an undefined procedure in a private actor module, it also omits that module qualification. Thus observers receive, for example, `exception(error(existence_error(procedure,q/1),_))`, rather than implementation details such as a generated actor-module name or stack context. Code inside the actor can still catch the original exception before it reaches the boundary. Non-exception exit reasons are not rewritten.

A reusable behaviour may instead catch an exception at its protocol boundary. The toplevel behaviour developed later in the book returns `error(Pid,E)` and remains available for another query. The `rpc/2-3` interface then translates such a remote error back into the ordinary Prolog interface by throwing `E` locally. Statechart actors use yet another policy: an exception raised by executable content becomes an internal `error(E)` event that transitions may handle. These are different policies for the same underlying Prolog exception, not different exception syntaxes. Timeouts, explicit aborts, and transport failures remain separate outcomes.

The demonstrator uses SWI-Prolog's message machinery to present these diagnostics concisely by default. Its Terminal settings can instead display a Prolog exception term whose implementation-supplied context is deliberately elided. For exceptions handled by the toplevel, this is a presentation choice and the caught exception itself remains unchanged. For an uncaught actor exception, the monitor reason has already been sanitised at the actor boundary; `flush/0` faithfully prints that sanitised Prolog term from the shell's mailbox.

Erlang offers corresponding local constructs through `try/catch/after`, and its monitors likewise turn process termination into a message carrying an exit reason. Web Prolog borrows that separation between a process and its observer, but retains Prolog's own control model and SWI-Prolog's exception terms. In particular, Erlang has no counterpart to ordinary Prolog failure as a distinct process result: Web Prolog's `down/3` therefore distinguishes `false` from an uncaught `exception(E)`.

Monitoring

In a call such as `spawn(Goal, Pid, Options)` the optional third argument is a list of options used for configuring the actor. One such option is `monitor`, which if set to `true` requests that the parent process be notified when the spawned actor terminates.

In the following example, `monitor(true)` instructs the runtime to deliver a `down` message to the parent when the child terminates:

```
?- spawn(2 > 1, Pid, [
    monitor(true)
]).
Pid = 60367387.
?- flush.
Shell got down(60367387,60367387,true)
true.
?-
```

A `down` message has the form `down(Pid, Ref, Reason)`, where `Pid` identifies the monitored actor and `Ref` identifies the *monitor instance* that produced the notification. For an observed lifecycle outcome, `Reason` describes why the actor terminated. For a remote observation failure it is instead `connection_closed`, with the distinct meaning given below. When monitoring is installed via `monitor(true)`, the monitor reference is chosen to be identical to the child's `pid`, which is why the first two arguments coincide in the example.

In addition to the `monitor` option, the predicate `monitor(Pid, Ref)` is provided, which installs a monitor and returns a fresh reference in `Ref`. The reference matters whenever there may be more than one monitor, because each monitor produces its own `down` message and we must be able to tell them apart:

```
?- spawn(sleep(3), Pid),
    monitor(Pid, Ref1),
    monitor(Pid, Ref2).
Pid = 41544343,
Ref1 = 10924110,
Ref2 = 92041933.
?- flush.
Shell got down(41544343, 10924110,true)
Shell got down(41544343, 92041933,true)
true.
?-
```

Here the same parent process installs two monitors on the same child. When the child terminates after three seconds, two `down` messages are delivered, one per monitor. The distinct references (`Ref1` and `Ref2`) identify which monitor instance each message corresponds to.

Using the `monitor` option is the *safest* way to establish monitoring, because it is *atomic* with respect to process creation: either the child is spawned with monitoring

already in place, or it is not spawned at all. In contrast, calling `monitor/2` after `spawn/2-3` is a separate step and therefore not atomic. This means that a very short-lived child may terminate in the gap between the `spawn` returning and the monitor being installed, in which case the parent will never receive a `down` message. For actors that may complete immediately (or fail immediately), `monitor(true)` avoids this race condition.

For a remote monitor, `down(Pid,Ref,connection_closed)` has a distinct meaning: the runtime has lost the ability to observe the target, not learned that the target terminated. That notification discharges the monitor; a later reconnection does not silently restore it. Section 4.1.7 relates this rule to remote links and containment.

Monitoring can be stopped by passing the monitor ref to `demonitor/1-2`. When monitoring is installed using the `monitor` option, the pid of the monitored actor can also be used since it is identical to the ref.

The above example of the use of `monitor/2` was trivial, but in Chapter 5, both `monitor/2` and `demonitor/1-2` will be put to more serious use.

Registering

Calling `register(Name,Pid)` associates the atom `Name` with `Pid`. The name can be used instead of the pid when calling `!/2`. For example, we can register our shell under the name `shell`:

```
?- self(Self), register(shell, Self).
Self = 85234512.
?- shell ! hello.
true.
?- spawn(shell ! goodbye).
true.
?- flush.
Shell got hello
Shell got goodbye
true.
?-
```

Registering is useful when we want to offer a service that should always be available under a name that is easy to remember. Should a crash occur, all our system needs to do is to restart it and associate the same name with the pid of the new process.

Exiting

Web Prolog supports two predicates, `exit/1` and `exit/2`, that can be used to terminate an actor process.

If a process calls `exit(Reason)`, it deliberately terminates immediately with the supplied reason. The reason need not denote an error: it is an arbitrary nonvar Prolog term chosen by the process. If the process is monitored, the runtime delivers a `down` message containing that term. Here is a simple example where a process is spawned and monitored by the shell. Since the process exits immediately with the reason `my_reason`, the shell receives a `down` message with that atom in its third argument:

```
?- spawn(exit(my_reason), Pid, [
    monitor(true)
]).
Pid = 91325643.
?- flush.
Shell got down(91325643,91325643,my_reason)
true.
?-
```

It sometimes happens that we need to terminate an actor process by force. Our echo server, for example, can only be terminated in this way. The predicate `exit/2` can be used to terminate any actor process with a known pid. Calling `exit(Pid, Reason)` sends an exit *signal* to the actor with that pid, killing it. Signals are runtime-control operations rather than mailbox messages; their full semantics is specified in Section 4.1.7.

If we do not know the pid but do know a registered name, we can first resolve the name to its pid. To see how this works, let us spawn a new monitored echo server and register it:

```
?- spawn(echo_actor, Pid, [
    monitor(true)
]),
    register(echo_actor, Pid).
Pid = 21562390.
?-
```

Now, let us say we change our minds and want to get rid of it again:

```
?- whereis(echo_actor, Pid),
    exit(Pid, 'We changed our minds!').
Pid = 21562390.
?- flush.
Shell got down(21562390,21562390,'We changed our minds!')
true.
?-
```

By calling `exit/2` with the pid in the first argument and a term detailing the reason for exiting in the second, a signal was sent to the process that terminated it. Note that a call to `exit/2` will only accept a pid in its first argument, so if all we have is a name, the built-in predicate `whereis/2` must be used to locate it.

Note that the reason passed to `exit/1-2` can be any nonvar term, not just an atom. This means that it can be used to transfer an arbitrarily large chunk of information back to the parent. However, in most cases an atom is all that is needed, and it is worth noticing that using `true` as a reason can be used to suggest to the parent that the termination was *normal*, even though it was caused by a call to `exit/1-2`.

What might seem a bit odd is that even though 21562390 is now provably dead, trying to send it a message using a pid or calling `exit/2` still succeeds without an error, although no down message is being sent:

```
?- self(Self), $Pid ! echo(Self, bye).
Self = 85234512.
?- exit($Pid, 'We changed our minds!').
true.
?- flush.
true.
?-
```

Treating `!/2` and `exit/2` as no-ops when the pid points to a non-existent process is consistent with how it works in Erlang. However, trying to send a message using the *name* of a non-existent process generates an error:

```
?- echo_actor ! echo($Self, bye).
Error: The name 'echo_actor' is not associated with a pid.
?-
```

This too is consistent with how Erlang works.

Linking

A further spawn option, `link`, governs the lifetime of the spawned actor. If the option is set to `true`, the life of the new actor is tied to the life of its parent: should the parent terminate – for whatever reason – the child is forced to terminate too. In our proposal, the default value of this option is `true`.

Readers who know Erlang should pause here, because this is a point at which Web Prolog deliberately departs from its role model, in two ways. First, Erlang's links are bidirectional, whereas the dependence just described runs in one direction only, from parent to child. This choice follows the proposal of Svensson et al. (2010, Section 2.2) to replace Erlang's symmetric links with unidirectional failure dependencies, so that termination of the supervisor terminates the child, while termination of the child is reported separately by a monitor. Web Prolog adopts exactly that orientation for its parent-child lifetime relation. Second, Erlang's `spawn` does not link at all – linking requires `spawn_link` – whereas in Web Prolog the tie to the parent is the default and must be explicitly declined. Building the one-way relation into `spawn` and making it the default are Web Prolog's additions, motivated by the client-owner separation of the open Web and justified in Chapter 4; for now, we simply demonstrate the behaviour.

In the following example, the outer call to `spawn/2` creates an actor whose goal spawns a second, non-terminating actor and then completes. When its goal completes, the outer actor terminates – and because the inner actor was created with the default `link(true)`, it is automatically killed along with it:

```
?- spawn(spawn(receive({})), Pid).
Pid = 10782012.
?-
```

Calling the built-in predicate `actors/1` confirms that only one actor remains alive – the shell itself:

```
?- actors(Alive).
Alive = [91033267].
?-
```

The dependence does not run the other way. A parent whose child terminates – even abnormally – is unaffected:

```
?- spawn((spawn(exit(die)), receive({})), Pid).
Pid = 40276924.
?-
```

Even though the child killed itself immediately, calling `actors/1` again shows that its parent 40276924 is still alive:

```
?- actors(Alive).
Alive = [91033267, 40276924].
?-
```

The link, then, makes the life of a child dependent on the life of its parent, and never the reverse.

Since the option is on by default, declining it must be done explicitly, by passing `link(false)` when the child is spawned. Note carefully where the option goes: it belongs to the child's own spawn, and describes that child's relationship to its parent. In the following example, it is the *inner* actor that is spawned with `link(false)`, and it consequently survives the termination of the outer actor that created it:

```
?- spawn(spawn(receive({}), _, [link(false)]), Pid).
Pid = 10023812.
?- actors(Alive).
Alive = [91033267, 24917621].
?-
```

The outer actor (10023812) is gone, its goal having completed, but the inner actor (24917621) lives on – now without a living parent. Actors of this kind, deliberately detached from the hierarchy that created them, are useful for long-lived services, but they are also actors that no parent's termination will ever clean up. For a reason explained in Chapter 4, the distributed setting therefore restricts them: clients are typically required to spawn with `link(true)`, with `link(false)` reserved for node owners.

A hierarchy of actors

The lifetime dependence introduced earlier is *transitive*: because each child depends on its parent, terminating an actor tears down not just its immediate children but its entire descendant subtree. The following example makes this cascade visible.

The predicate `root/0` spawns a child and then blocks indefinitely. The child spawns a grandchild and does the same. Neither names anything or monitors anything; the only structure is the spawn hierarchy itself:

```
root :-
    spawn(child),
    receive({}).

child :-
    spawn(grandchild),
    receive({}).

grandchild :-
    receive({}).
```

We spawn the root from the shell and then confirm, using `actors/1`, that all three processes (plus the shell itself) are alive:

```
?- spawn(root, Pid).
Pid = 50318842.
?- actors(Alive).
Alive = [91033267, 50318842, 71200934, 33910576].
?-
```

The shell is 91033267; the other three are the root, the child, and the grandchild. Now we terminate the root, and *only* the root, by force:

```
?- exit($Pid, die).
true.
?- actors(Alive).
Alive = [91033267].
?-
```

The root is gone, as expected – but so are the child and the grandchild, even though we sent an exit signal to neither. This is the cascade: terminating the root made its child terminate, which in turn made the grandchild terminate. Lifetime dependence propagates down the whole hierarchy.

The reverse does not hold: terminating a child does not terminate its parent. Because lifetime dependence propagates downward, however, a client that spawns a subtree of workers cannot leave orphans behind it: when the client's own process ends, everything it spawned, however deeply nested, is reclaimed with it. A child cannot trap this downward termination; the rules governing unrefusable signals and remote links are specified in Section 4.1.7.

2.3.4 A closer look at receive/1-2

As shown already, but only for a trivial case, an actor process uses the receive primitive to extract messages from its mailbox. Since the syntax and semantics of `receive/1-2` is fairly complex, a closer look and more examples are needed. Below we give several simple examples illustrating different ways to use the `receive/1-2` predicate in Web Prolog, demonstrating how to handle different *types* of messages, use *timeouts*, apply *guards*, and more. Other examples illustrate how messages are deferred and handled in subsequent `receive/1-2` calls, demonstrating the flexibility of passing and processing messages in Web Prolog.

Basic receive

In Web Prolog, just like in Erlang, the receive operation specifies an ordered sequence of *receive clauses* delimited by semicolons. A receive clause always has a *head* (a term) and a *body* of Prolog goals. Schematically, the basic form of a receive call looks like this:

```
receive({
    Head1 -> Body1 ;
    Head2 -> Body2 ;
    ...
    HeadN -> BodyN
})
```

Often, the head is just a single term serving as a *pattern*. Any term will do, ground or non-ground, and even a bare variable is fine.

As in Erlang, `receive/1` scans the mailbox looking for the oldest message that matches a pattern in any of the receive clauses, blocking if no such message is found. If a matching clause is found, the message is removed from the mailbox and the body of the clause is called. In Web Prolog, just like in Erlang, values of any variables bound by the matching of the pattern with a message are available in the body of the clause.

Often, a `receive/1` call waits for a specific message and executes the corresponding code in the body of the receive clause if a message that matches the pattern shows up. For example, the following call waits for a message in the form of `hello(Name)` and prints a greeting when it appears:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name])
})
```

We can specify multiple patterns in a single `receive/1` call. For example, this call handles messages of either the form `hello(Name)` or the form `goodbye(Name)`, but only one of them:

```

receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name]) ;
    goodbye(Name) ->
        format("Goodbye, ~s!~n", [Name])
})

```

We can use the `_` pattern to catch any message. In the following case, any message that does not match `hello(Name)` will be caught by the `_` pattern:

```

receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name]) ;
    _ ->
        format("Unknown message received.~n")
})

```

We can nest `receive/1-2` calls. Here, after having received the `start(Name)` message, the call waits for a `continue(Msg)` message:

```

receive({
    start(Name) ->
        format("Starting with ~s.~n", [Name]),
        receive({
            continue(Msg) ->
                format("Continuing with ~s~n", [Msg])
        })
})

```

Messages deferred

If no pattern matches a message in the mailbox, the message is *deferred*, which means that the message does not match any of the patterns in the current `receive/1-2` call and remains in the process's mailbox, possibly to be handled later in the control flow of the process. The receive is still running, waiting for more messages to arrive, and for one that will match.

In the following example, the `goodbye("Bob")` message – which is the oldest and therefore first in line – does not match the clause in the first receive call and is deferred. The `hello("Alice")` message matches that clause, and then the clause in the second receive call handles the `goodbye("Bob")` message:

```

?- self(Self),
   Self ! goodbye("Bob"),
   Self ! hello("Alice"),
   receive({

```



```

        hello(Name1) ->
            format("Hello, ~s!~n", [Name1])
    }),
    receive({
        goodbye(Name2) ->
            format("Goodbye, ~s!~n", [Name2])
    }).
Hello, Alice!
Goodbye, Bob!
true.
?-

```

This behaviour is particularly useful if we expect two messages but are not sure which one will arrive first.

Guards

The head of a receive clause can, in addition to the pattern, optionally specify a *guard* in the form of a Prolog goal. A head of the form `Pattern if Goal` makes pattern matching more expressive: the clause is selected only if the incoming message matches `Pattern` *and* `Goal` succeeds. Here is an example that distinguishes positive from non-positive numbers:

```

receive({
    number(N) if N > 0 ->
        format("Positive number: ~p~n", [N]) ;
    number(N) if N <= 0 ->
        format("Non-positive number: ~p~n", [N])
})

```

This is the familiar use of guards, and it works in Erlang too: the guard is a test, a yes-or-no question about values already bound by the pattern.

But in Web Prolog a guard can be far more than a test, and the next example is worth dwelling on, because it exposes one of the deepest points of contact between the actor model and logic programming. The guard is an ordinary Prolog goal, evaluated within the actor's *logical environment* – the logical resources available to the actor – and like any Prolog goal, it can *bind variables that the pattern never mentions*:

```

?- self(Self),
   Self ! hello(xantippa),
   receive({
       hello(W) if husband(W, H) ->
           format("Hello, ~w, say hello to ~w!~n", [W,H])
   }).
Hello, xantippa, say hello to socrates!

```

```
true.  
?-
```

Look closely at what happened. The message supplied $W = \text{xantippa}$. The variable H appears nowhere in the pattern `hello(W)`; it is introduced by the guard. When `husband(W, H)` was evaluated within that logical environment, it did not merely *test* whether `xantippa` has a husband – it *looked him up*, binding H to `socrates`, and that binding then flowed into the body of the clause and did real work there.

One could imagine a version of Erlang in which guards call arbitrary pure functions, and there is nothing in the preceding example that such a program could not reproduce. The important difference is therefore not raw computational power, but what the guard itself means. An ordinary guard has the conceptual form

$$\text{message} \times \text{state} \longrightarrow \text{Boolean}.$$

It answers whether a message should be accepted. Allowing arbitrary function calls makes the Boolean test more computationally powerful, but does not by itself change this contract. If a function is also to return information needed by the clause body, the language requires an additional convention for capturing and destructuring that result. The construct is then doing more than guarding.

A Web Prolog guard instead has the conceptual form

$$\text{partial substitution} \times \text{logical environment} \longrightarrow \text{extended substitution}.$$

The pattern first contributes bindings obtained from the message. The guard then extends those bindings by proving a goal within the actor's logical environment. In the example above, matching supplies $W = \text{xantippa}$, while proving `husband(W, H)` supplies $H = \text{socrates}$. The body receives the combined substitution.

The logical environment is not confined to the actor's private database, or even to a single database. It may include the actor's private clauses, the node's shared database, imported predicates, and built-in predicates – and, once Chapter 4 has extended that environment across node boundaries, remote logical services as well. A guard can therefore combine resources from the logical environment in one goal. This is a statement about what the actor can logically ask, not about where the knowledge or computation happens.

This is important because a Prolog predicate is relational rather than tied to one direction of computation. A conjunction such as

```
authorized(User), owns(User, Printer), online(Printer)
```

does not hide a fixed lookup procedure behind a function name. It states constraints joining the message with three relations available in the actor's logical environment. Variables are shared freely across the pattern, the guard and the body; conjunction composes further constraints; and the same predicates can be queried in other modes elsewhere in the program.

An Erlang function could return a printer, a list of printers, or an iterator, but the programmer would have to choose and implement that result convention explicitly. In Web Prolog, witnesses, shared variables and relational composition are already

the ordinary meaning of goal evaluation. Arbitrary function calls make guards more permissive; logical queries make message selection relational. The selected message is not merely tested against actor state – it participates with the logical resources available to the actor in a proof.

For clause selection, however, a guard is evaluated only for its first solution, as if by `once/1`. Pattern matching and guard evaluation use a tentative substitution: if either fails, all bindings made for that candidate are discarded before another clause or mailbox message is tried, and the mailbox remains unchanged. Bindings produced by a successful guard remain available in the clause body, but backtracking through the body does not ask the guard for another solution and does not reconsider the selected message or receive clause.

For portable code, a guard must be observationally free of side effects. The runtime need not enforce that restriction, and effects performed by a non-conforming guard are not undone on failure. A guard exception is treated as a non-match: its tentative bindings and the exception are discarded, and selection continues. This follows an Erlang-like error-as-false discipline, but a guard that calls a fallible external service should translate failures into explicit results if the actor must distinguish them from an ordinary non-match. Unrestricted goal evaluation also carries operational risk: an expensive or nonterminating guard can delay message selection. The significance lies in the information a successful guard produces, not in exposing unrestricted backtracking through the mailbox. Chapter 4 returns to the additional latency and failure risks when a guard extends this environment across a node boundary.

This discipline has precedents in the committed-choice logic-programming languages. Concurrent Prolog, Parlog, and Guarded Horn Clauses use guards to determine which process clause may commit, rather than as ordinary goals through which execution may later backtrack (Shapiro, 1987; Clark and Gregory, 1986; Ueda, 1986). Web Prolog does not adopt their execution model wholesale: ordinary nondeterministic Prolog remains available within an actor, including in the body selected by a receive clause. The narrower lesson adopted here is that selection itself is committed and that a rejected guard must leave no logical bindings behind. Requiring guards to be observationally free of side effects makes that discipline coherent without pretending that arbitrary external effects can be undone. The broader relationship to committed-choice languages is discussed in Section 9.8.2.

It is this difference – a guard that genuinely *reasons* rather than merely tests – that leads us to depart from Erlang’s vocabulary and write `if` where Erlang writes `when`. The choice is deliberate. `when` suggests a side-condition on matching, a gate that opens or stays shut; `if` suggests a logical antecedent, a relation whose satisfaction is itself part of what the clause computes. The name follows the semantics: in Web Prolog, a guard is a relational query, not a restricted test. We return to this feature in Section 2.4.6, where guards that consult the actor’s logical environment become the basis for event-driven state machines whose transitions depend on what the process can establish.

Timeouts

The optional second argument of `receive/1-2` expects a list of options. The `timeout` option takes an integer or float specifying the number of seconds before a timeout occurs if no match has been found. The `on_timeout` option takes a goal that is called on timeout. If this option is omitted, its goal defaults to `true`, so the receive succeeds. An explicitly supplied timeout goal is ordinary Prolog code: its success, failure, exception, and any choice points determine the outcome of the receive on the timeout path. Here is an example of its use:

```
receive({
    hello(Name) ->
        format("Hello, ~s!\n", [Name])
}, [ timeout(5),
    on_timeout(format("No match received in 5 seconds.\n"))
])
```

If no message of the form `hello(Name)` is received within 5 seconds, the code in the `on_timeout` option is executed.

An implementation of sleep/1

Here is how a predicate `sleep/1` that suspends execution `Time` seconds can be defined:

```
sleep(Time) :-
    receive({}, [
        timeout(Time)
    ]).
```

The call to `receive/2` does not wait for messages, but when `Time` seconds have passed, it still succeeds.

An implementation of flush/0

As we noted earlier, being able to inspect the contents of the shell's mailbox during interactive programming is important, and `flush/0` is a nice tool for doing just that. Its definition also serves as yet another example of the use of the `timeout` option:

```
flush :-
    receive({
        Message ->
            format("Shell got ~q\n", [Message]),
            flush
    }, [
```

```
        timeout(0)
    ]).
```

The value `0` of the `timeout` option ensures that the loop terminates immediately if no messages remain in the mailbox. This is how the hanging of the `receive/1` call is avoided.

A simple priority queue

To demonstrate the use of the `if` operator and the `timeout` and `on_timeout` options of `receive/2` – the latter of which causes a goal to run on timeout – we show a priority queue example borrowed from Fred Hébert’s textbook on Erlang (Hébert, 2013). The purpose is to build a list of messages with those with a priority above 10 coming first:

```
important(Messages) :-
    receive({
        Priority-Message if Priority > 10 ->
            Messages = [Message|MoreMessages],
            important(MoreMessages)
    }, [ timeout(0),
        on_timeout(normal(Messages))
    ]).

normal(Messages) :-
    receive({
        _-Message ->
            Messages = [Message|MoreMessages],
            normal(MoreMessages)
    }, [ timeout(0),
        on_timeout(Messages=[])
    ]).
```

The timeout set to `0` means that it will occur immediately, but the system tries all messages currently in the mailbox first.

Below, we test this program by first sending four messages to the shell process, and then calling `important/1`:

```
?- self(S),
    S ! 15-high, S ! 7-low, S ! 1-low, S ! 17-high.
S = 34871244.
?- important(Messages).
Messages = [high,high,low,low].
?-
```

Note that the implementation of the priority-queue example relies on the deferring behaviour of `receive/1-2` and would not work without it.

One message, possibly several solutions

A call to `receive/1-2` is *single-message*, but not necessarily *single-solution*. During one invocation it selects and consumes at most one message: the oldest message accepted by any clause, using the first clause that accepts it. This selection is committed. Backtracking through the invocation never selects another message or another clause.

The number of messages consumed and the number of Prolog solutions are different matters. After committing to the message and clause, the `receive` calls the selected body as ordinary Prolog code. The body may fail, succeed once, throw an exception, or yield several solutions. Choice points left by the body enumerate further solutions of that same body; they do not inspect the mailbox again. On timeout no message is consumed, and the same principle applies to the `on_timeout` goal. The determinism declaration `nondet` therefore concerns the possible number of logical solutions, not the number of reception events performed by one invocation.

To see how it pans out in a simple corner case, consider the following two calls:

```
receive({foo(X) -> true})      receive({foo(X) -> fail})
```

The call on the left will succeed if a message matching the pattern `foo(X)` appears in the mailbox of the actor process executing the call, a term such as `foo(314)` for example. The call on the right will fail (and possibly cause backtracking) once `foo(314)` appears. Only by the first call will the variable `X` be bound (to 314). Both calls will remove the matched message from the mailbox – and, as we now explain, that removal stands even under backtracking. In both cases, if a message appears that does not match the pattern, it is deferred.

A nondeterministic body makes the distinction explicit. Assuming the mailbox initially contains one `foo(314)` message, the following `receive` has two solutions:

```
?- receive({foo(X) -> member(Y, [a,b])}).
X = 314, Y = a ;
X = 314, Y = b.
?- 
```

Both solutions come from the body selected for the same consumed message. The message is consumed only once. Producing the second solution does not inspect the mailbox again, and asking for still more solutions does not make the `receive` try another message or clause. In short, `receive/1-2` is *single-message*, not *single-solution*.

One point deserves emphasis, because it has no analogue in pure Prolog. The `send` and `receive` operations are effects on a mailbox, and like the database mutations of `assertz/1` and `retract/1`, they are not undone on backtracking. A message sent by `!/2` remains in its target mailbox even if the sending goal is later backtracked into. A message matched by a `receive` is removed whether the clause body then succeeds or fails, and is not restored by backtracking past the call. Consequently, a goal re-entered through backtracking does not see the mailbox as it was on first entry – it sees the current mailbox, reflecting every `send` and consumption (including

those of other actors) that has occurred in the meantime. Backtracking over message operations is therefore re-execution against the current state of a shared, time-varying store, not a return to the state that held on first entry. This is the same discipline Prolog programmers already accept for the dynamic database, now extended to the mailbox.

To implement a looping behaviour, Prolog programmers occasionally use a *failure-driven* loop that relies on backtracking rather than recursion. Using this technique, our echo server can be rewritten like so:

```
echo_actor :-
    repeat,
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            fail
    }).
```

For an Erlang programmer, this use of `receive/1` may come as a surprise and is not a technique that can be used in Erlang. In this particular case, a Web Prolog programmer would be advised to stick to the recursive version. However, there are cases when a failure-driven loop is the only way forward and we shall look at an important such case towards the end of this chapter.

2.4 Erlang-style programming examples in Web Prolog

Reading the code was fun – I had to do a double take – was I reading Erlang or Prolog – they often look pretty much the same.

Joe Armstrong (p.c. June 18, 2018)

Not only do Web Prolog programs *look* a lot like Erlang, they *behave* a lot like Erlang too. A good way to demonstrate this is to translate a fair number of different Erlang programming examples borrowed from tutorials and text books into Web Prolog and show that they run just like in Erlang. In this section we shall look at a count server, a fridge simulation, a technique for hiding message protocols behind predicate APIs, delayed sending, actors playing ping-pong, event-driven state machines whose transitions consult the knowledge base, and – most importantly – a demonstration that Prolog’s backtracking search can be placed under external, message-driven control. They are all concurrent programs, but only locally so. Distributed programming is dealt with in Chapter 4.

2.4.1 A count server

We have already looked at an example of a *stateless* server, namely the echo server presented earlier in this chapter. Let us now turn to *stateful* servers and show how

state may be programmed. In the following example, we demonstrate how to script an actor that can keep a *count*:

```
count_actor(Count0) :-
    receive({
        count(From) ->
            Count is Count0 + 1,
            From ! count(Count),
            count_actor(Count) ;
        stop ->
            true
    }).
```

The predicate `count_actor/1` defines a loop where a call to the built-in predicate `receive/1` tries to select a message of the form `count(From)` from the mailbox, increment the counter, send the current count back to the actor referenced by the `pid` to which the variable `From` is bound, and continue looping by making a recursive call. Note how the state of the counter – the current count, that is – is kept in the argument of the predicate. Note also that we added a second receive clause that will allow us to terminate the server without using `exit/2`. We just have to send it a message of the form `stop`.

Here is how an actor following this script can be made to perform when spawning it:

```
?- spawn(count_actor(0), Pid, [
    monitor(true)
]).
Pid = 60367387.
?-
```

In the next step the send operator `!/2` is used for sending a message to the spawned actor, instructing it to increment the count and to return the result in the form of a message:

```
?- self(Self), $Pid ! count(Self).
Self = 41167597.
?-
```

A call to `receive/1` can be made in order to collect the message arriving from the actor:

```
?- receive({Count -> true}).
Count = count(1).
?-
```

Here, `!/2` is used to send two messages to the actor that will end up in its mailbox, in the order they were sent:


```
?- $Pid ! count($Self), $Pid ! stop.
true.
?-
```

Finally, `flush/0` is used to inspect the contents of the mailbox belonging to the `toplevel` process:

```
?- flush.
Shell got count(2)
Shell got down(60367387,60367387,true)
true.
?-
```

Because the actor was monitored, a `down` message was found in addition to the current count.

2.4.2 A bigger, tastier example

As a tastier example of how a process can be made to hold an updatable state during a conversation we have adapted a fridge simulation example from Fred Hébert's introduction to Erlang (Hébert, 2013):¹³

```
fridge(FoodList0) :-
    receive({
        store(From, Food) ->
            self(Self),
            From ! Self-ok,
            fridge([Food|FoodList0]) ;
        take(From, Food) ->
            self(Self),
            ( select(Food, FoodList0, FoodList)
              -> From ! Self-ok(Food),
                fridge(FoodList)
              ; From ! Self-not_found,
                fridge(FoodList0)
            ) ;
        terminate ->
            true
    }).
```

The program creates a process allowing three operations: storing food in the fridge, taking food from the fridge, and terminating the fridge. It is only possible to take food that has been stored beforehand. Again, with the help of recursion the state

¹³ <http://learnyousomeerlang.com/more-on-multiprocessing#state-your-state>

of a process can be held entirely in the argument of the predicate. In this case we choose to store all the food as a list, and then look in that list when someone needs something.

Assuming the above program is already available, the following session creates the server process. We can then call `self/1`, `!/2` and `flush/0` from the shell in order to simulate the actions of a client:

```
?- spawn(fridge([]), Pid, [
    monitor(true)
]).
Pid = 77346122.
?- self(Me), $Pid ! store(Me, meat), $Pid ! store(Me, cheese).
Me = 97216744.
?- flush.
Shell got 77346122-ok
Shell got 77346122-ok
true.
?- self(Me), $Pid ! take(Me, cheese).
Me = 97216744.
?- flush.
Shell got 77346122-ok(cheese)
true.
?- $Pid ! terminate.
true.
?- flush.
Shell got down(77346122,77346122,true)
true.
?-
```

2.4.3 Hiding the details of protocols

In the previous example, we expected programmers to know the details of the protocol that must be followed when interacting with our fridge simulation, which forced them to make raw calls using the send operator in combination with the `flush/0` utility predicate. As stated by Fred Hébert in his book, that is often a useless burden, and good way around it is to abstract messages away with the help of predicates (or in Hébert's case, functions) dealing with receiving and sending them:

```
store(Pid, Food, Response) :-
    self(Self),
    Pid ! store(Self, Food),
    receive({
        Pid-Response -> true
    }).
```

```
take(Pid, Food, Response) :-
    self(Self),
    Pid ! take(Self, Food),
    receive({
        Pid-Response -> true
    }).
```

Calling `receive/1` immediately after having sent the message guarantees that the communication between client and server stays synchronous. Using `store/3` and `take/3`, the interaction with the fridge becomes somewhat easier:

```
?- spawn(fridge([]), Pid, [
    monitor(true)
]).
Pid = 55289322.
?- store($Pid, cheese, Response).
Response = ok.
?- take($Pid, cheese, Response).
Response = ok(cheese).
?-
```

When dealing with actors with more complex protocols, such abstractions turn out to be of considerable value.

2.4.4 Using delayed sending

Appendix B documents a built-in predicate `send/2-3` that allows a process to delay the sending of a message, but also to cancel the sending if needed, by calling `cancel/1`. This can be used to implement an alarm that can be cancelled, like so:

```
alarm :-
    receive({
        ring ->
            writeln('Alarm ringing!'),
            alarm;
        stop ->
            true
    }).
```

Here we schedule the alarm to ring in 5 seconds, with an id set to `alarm1`:

```
?- spawn(alarm, Pid),
    send(Pid, ring, [
        delay(5),
```

```

        id(alarm1)
    ]).
Pid = 60034123.
?-

```

Then we cancel the alarm after a couple of seconds:

```

?- cancel(alarm1).
true.
?-

```

Cancellation is scoped to the calling actor: `cancel(alarm1)` tries to withdraw all delayed sends with that identifier that were scheduled by this actor, without affecting another actor that happened to choose the same identifier. If cancellation succeeds, the message is withdrawn and the alarm never rings. If `cancel/1` is called too late, the alarm message has already been delivered and “Alarm ringing!” will be printed.

2.4.5 Prolog actors playing ping-pong

Since the servers in the previous sections are running in parallel to the shell and are talking to it using asynchronous messaging, we have already demonstrated the use of concurrency. Below, in a probably more convincing example inspired by a user’s guide to Erlang,¹⁴ two processes are first created and then start sending messages to each other a specified number of times:

```

ping(0, Pong_Pid) :-
    Pong_Pid ! finished,
    format('Ping finished.~n').
ping(N, Pong_Pid) :-
    self(Self),
    Pong_Pid ! ping(Self),
    receive({
        pong ->
            format('Ping received pong.~n')
    }),
    N1 is N - 1,
    ping(N1, Pong_Pid).

pong :-
    receive({
        finished ->
            format('Pong finished.~n');
        ping(Ping_Pid) ->

```

¹⁴ See http://erlang.org/doc/getting_started/conc_prog.html

```

        format('Pong received ping.~n'),
        Ping_Pid ! pong,
        pong
    }).

ping_pong :-
    spawn(pong, Pong_Pid),
    spawn ping(3, Pong_Pid)).

```

When `ping_pong/0` is called the behaviour of this program exactly mirrors the behaviour of the original version in Erlang:

```

?- ping_pong.
true.
Pong received ping.
Ping received pong.
Pong received ping.
Ping received pong.
Pong received ping.
Ping received pong.
Ping finished.
Pong finished.
?-

```

This is a concurrent program, but its execution is not done in parallel, but rather as is illustrated in Figure 2.3.



Fig. 2.3 A timeline where the upper line represents the “pinger” and the lower line the “ponger,” and where each grey bar represents a task consisting of the sending of a message (in light grey) and the reception of the response (in darker grey).

2.4.6 Event-driven state machines

Building on the guard mechanism introduced above, we now show a simple but representative application: an event-driven state machine whose transitions can draw on the actor’s logical environment. The key point is that clause selection can depend not only on the incoming message pattern, but also on arbitrary relational conditions, and any bindings produced by those conditions can be used by the transition action.

Web Prolog’s `receive` primitive can implement such machines directly. Message patterns represent events, guards represent transition conditions, and clause bodies

implement actions and state updates. Figure 2.4 depicts a machine with two states and four transitions.

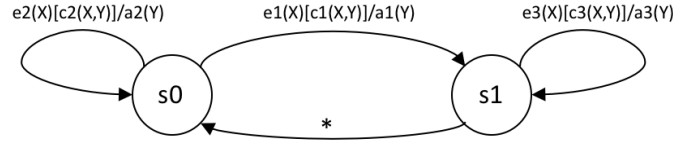


Fig. 2.4 A simple event-driven state machine. `s0` is the initial state.

Using predicate names to encode states, one possible implementation is:

```

s0 :-
    receive({
        e1(X) if c1(X,Y) ->
            a1(Y),
            s1 ;
        e2(X) if c2(X,Y) ->
            a2(Y),
            s0
    }).

s1 :-
    receive({
        e3(X) if c3(X,Y) ->
            a3(Y),
            s1 ;
        _AnyEvent ->
            s0
    }).
  
```

In state `s0` the machine waits for an event message of the form `e1(X)` or `e2(X)` in the current process's mailbox. If `e1(X)` matches and `c1(X,Y)` succeeds, the action `a1(Y)` is executed and the machine transitions to `s1`. If `e2(X)` matches and `c2(X,Y)` succeeds, the action `a2(Y)` is executed and the machine remains in `s0`. The other state is analogous: `e3(X)` triggers a guarded self-loop in `s1`, while any other event falls back to `s0`.¹⁵

The variables `Y` illustrate the guard semantics described in Section 2.3.4: event matching binds `X`, while the transition condition may derive `Y` and make it available to the action. What this example adds is a control pattern: receive clauses can encode states, events, transition conditions and actions directly. Chapter 6 develops `SXML`, a specialised statechart notation within Web Prolog, for expressing the same style of reactive programming more structurally.¹⁶

¹⁵ Note that `s0` defers unknown events while `s1` discards them; the designer must choose deliberately, a matter we return to with `SXML` in Chapter 6.

¹⁶ Event-driven state machines have historically been less prominent in the Prolog world, most likely because message-passing concurrency arrived relatively late and unevenly across Prolog systems, gaining wider traction only with facilities such as those in the ISO Prolog Threads draft. By contrast, Erlang/OTP treats this programming model as foundational, as exemplified by `gen.statem`.

2.4.7 Getting answers through backtracking

We come now to the example that most directly tests the central claim of this chapter. Back in Section 2.2.2 we worried aloud about an impedance mismatch: Prolog's model is relational and nondeterministic, driven by backtracking search; Erlang's is functional and message-passing, with no backtracking at all. When the two are combined in one language, what happens at the seam – where a goal that can produce many solutions meets a `receive` that consumes messages one at a time? Does backtracking compose with messaging, or do the two models grind against each other?

Consider a spawned goal that enumerates the solutions to `human(Who)`, but instead of collecting them all at once, sends the *first* solution back to its parent and then waits. It will produce the next solution only when told to, by a `next` message, and will stop on `stop`:

```
?- self(Self),
    spawn(( human(Who),
             Self ! Who,
             receive({
                 next -> fail ;
                 stop -> true
             })
           ), Pid).
Pid = 76123351,
Self = 90054377.
?- flush.
Shell got socrates
true.
?- $Pid ! next.
true.
?- flush.
Shell got plato
true.
?- $Pid ! stop.
true.
?-
```

Follow the control flow carefully, because the mechanism is more subtle than its three lines suggest. The spawned goal solves `human(Who)`, binding `Who = socrates`, sends that solution to the parent's mailbox with `Self ! Who`, and then blocks in `receive`. So far this is ordinary sequential Prolog with one message send. The interesting part is what `next` does. The `next` clause has body `fail`, so receiving `next` makes the `receive` *fail* – and a failure here propagates backwards into `human(Who)`, forcing it to backtrack and produce its next solution, `plato`. Execution then runs forward again through `Self ! Who`, sending `plato`, and blocks once more. Each

`next` thus drives one step of backtracking through the solution set; `stop`, whose body is `true`, lets the goal succeed and the actor terminate.

What makes this work is precisely the property established in Section 2.3.4: message selection is committed, and message operations are effects that backtracking does not undo. When the goal backtracks from `socrates`'s `receive` back into `human(Who)`, the `send` that already delivered `socrates` is *not* retracted, and `plato` is delivered as an *additional* message.

Messages now place Prolog's search under external control: each `next` requests another solution, much as pressing semicolon does at an interactive toplevel. This small actor is the starting point for the *toplevel behaviour* developed below.

The goal is still hard-coded, however, and the actor neither reports failure nor handles exceptions. The next section extends the sketch so that a client can submit goals and receive explicit information about their outcomes.

2.4.8 The toplevel behaviour: a preliminary sketch

In essence, a Prolog toplevel is a failure-driven loop running inside another loop – recursive or failure driven. The failure-driven inner loop allows a client to ask for one solution at a time to a goal, while the outer loop allows it to call several goals in a row and run each of them to completion.

Below, we show how we can build a simple Prolog toplevel by using a meta-predicate (such as `call_cleanup/2`) and by specifying a small set of custom messages carrying answers and/or the state of the process that needs to be returned to the calling process. The predicate `call_cleanup/2` is here used not only to call a goal, but also to check if any choice points remain after the goal has been called or backtracked into.¹⁷

```
simple_toplevel(Pid) :-
    simple_toplevel(Pid, []).

simple_toplevel(Pid, Options) :-
    self(Self),
    spawn(session(Pid, Self), Pid, Options).

session(Pid, Parent) :-
    receive({
        '$call'(Template, Goal) ->
            ( call_cleanup(Goal, Det=true),
              ( var(Det)
                -> Parent ! success(Pid, Template, true),
                  receive({
                      '$next' -> fail ;
                  })
              )
            )
    })
```

¹⁷ http://www.swi-prolog.org/pldoc/man?predicate=call_cleanup/2


```

        '$stop' -> true
    }},
    !
    ;   Parent ! success(Pid, Template, false)
    )
    ;   Parent ! failure(Pid)
    )
}},
session(Pid, Parent).

```

A suitable predicate API hiding the details of the protocol from the programmer can be written like so:

```

simple_toplevel_call(Pid, Template, Goal) :-
    Pid ! '$call'(Template, Goal).

```

```

simple_toplevel_next(Pid) :-
    Pid ! '$next'.

```

```

simple_toplevel_stop(Pid) :-
    Pid ! '$stop'.

```

Here is a session that shows that these predicates work as intended:

```

?- simple_toplevel(Pid, [
    monitor(true)
]).
Pid = 23891373.
?- simple_toplevel_call($Pid, Who, human(Who)).
true.
?- flush.
Shell got success(23891373,socrates,true)
true.
?- simple_toplevel_next($Pid).
true.
?- flush.
Shell got success(23891373,plato,true)
true.
?- simple_toplevel_next($Pid).
true.
?- flush.
Shell got success(23891373,aristotle,false)
true.
?-

```

Note that because of the outer recursive loop, our simple toplevel actor is again poised to run another query:

```

?- simple_toplevel_call($Pid, X, q(X)).
true.
?- flush.
Shell got down(23891373,23891373,
                exception(error(
                            existence_error(procedure,q/1),_)))
true.
?-

```

Here the actor terminated because the code for `simple_toplevel/1-2` does not say what should happen when the called goal throws an exception, as an undefined predicate does. Since the spawned process is monitored, a termination notification containing the exception classification nevertheless reached the spawning process in the form of a `down` message. The call to `flush/0` is displaying the boundary-sanitised Prolog term found in the shell's mailbox. Note that the actor runtime deliberately elides the implementation-supplied context when it forms the monitor notification. The underscore in this transcript denotes that elided, unbound argument.

A similar problem is evident should we tell the actor to execute a query that does not terminate. Using `exit/2` works, but will once again kill the entire actor process.

Chapter 3 introduces built-in toplevel predicates that address these limitations through a standardised protocol. These provide the reusable Erlang-style behaviour anticipated by this sketch.

2.5 Summary

This chapter introduced Web Prolog as a hybrid language that extends a substantial subset of ISO Prolog with Erlang-style constructs for programming concurrency. Web Prolog extends ISO Prolog with a small set of core additions – actors with private state, asynchronous message passing (`!/2`), process creation (`spawn/1-3`), and fault-handling primitives such as links, monitors, and exit signals. A program that never mentions these constructs behaves like a conventional Prolog program, while programs that do use them gain access to an Erlang-like world of concurrent processes.

We also argued that Erlang provides an unusually suitable point of departure for such an extension. Its actor model is mature, battle-tested, and close enough in syntax and spirit to Prolog that both languages can share patterns, idioms, and even didactic material. Many of the examples in this chapter are thin Prolog translations of familiar Erlang patterns. The combination of Prolog's backtracking search with Erlang-style concurrency is made workable by committing message and clause selection without suppressing the ordinary Prolog control behaviour of the selected body. Backtracking may explore that body, but it neither selects another message nor undoes mailbox effects. This makes it possible to treat a Prolog toplevel as just another actor and to encapsulate interactive sessions as behaviours implemented on top of a common concurrency substrate.

The borrowing is deliberate rather than literal. Table 2.1 summarises the principal departures established in this chapter and the reasons behind them. Most reflect either Prolog’s logical semantics or the requirements of ownership and lifetime management on an open Web.

Table 2.1 Principal departures from Erlang’s process model and their rationale.

Erlang	Web Prolog and rationale
Links are bidirectional	A parent owns a unidirectional lifetime dependence on its child. <i>Rationale:</i> client–owner separation and containment.
<code>spawn</code> is unlinked by default	Client-created children use <code>link(true)</code> by default. <i>Rationale:</i> bounded lifetime on the open Web.
Exit signals may be trapped	Containment termination is untrappable; monitors provide observation. <i>Rationale:</i> a child must not refuse owner teardown.
Guards are restricted tests introduced by <code>when</code>	<code>if</code> guards are relational queries over the logical environment. <i>Rationale:</i> Prolog’s unification and proof semantics.
Normal return has one success outcome	<code>down/3</code> distinguishes outermost success <code>true</code> from logical failure <code>false</code> . <i>Rationale:</i> Prolog’s distinct success, failure, and exception outcomes.
Per-pair ordering spans messages and other signals	FIFO is scoped to one message route; signals have no cross-channel order. <i>Rationale:</i> transport diversity and protocol-level coordination.

Taken together, these choices define Web Prolog as a multi-paradigm language. Its logical expressivity remains that of ISO Prolog, but its operational repertoire is enlarged by an actor-based concurrency model that is both network-transparent and compatible with the rest of the Prolog Trinity ecosystem. There are other possible ways of adding concurrency to Prolog, but the thesis of this chapter is simply that Erlang-style actors provide a coherent and practically manageable foundation on which the subsequent chapters can build.

Chapter 3

Prolog agents

Imagine a world of **Prolog agents**, some useful, others playful, bringing joy to games and virtual worlds; some short-lived, others long-lived, some simple, others complex – but maybe built from simpler ones. Written in Web Prolog, talking Web Prolog with other agents, using Web Prolog knowledge bases to guide their actions and conversations, making sure important abilities of clever conversational agents, such as natural language understanding, knowledge representation, reasoning and real-time interaction, are accounted for.

Prolog agents – the elevator pitch

Chapter 1 introduced Prolog agents in outline, while Chapter 2 developed the Prolog actor as Web Prolog’s elementary unit of concurrent computation. This chapter completes the first-level picture by treating actors and nodes as the two principal kinds of Prolog agent and by distinguishing the main actor behaviours used in the book.

3.1 Prolog agents in a nutshell

The wider claim made here is that the actor and the node – entities that differ enormously in scale, lifetime, and role – are most productively understood as instances of a single abstraction, the *Prolog agent*. A one-line echo actor that lives for a fraction of a second and a node that serves thousands of clients for months sit at opposite ends of a spectrum, and yet both are encountered through addressable, term-based boundaries. What unites them is not how they are built but the architectural stance we take towards them: we meet each through a protocol of goals or messages and corresponding answers, messages, or effects. Every actor and every node is an agent in this analytical sense. The term marks a role rather than a natural species and makes no claim about a minimum lifetime, private state, autonomy, or intelligence. Nor does it impose an implementation language – a node need not be written in Prolog, only able to talk it.¹

¹ In treating “agent” as an analytical lens rather than an absolute classification, we follow Russell and Norvig (2016); the term carries no commitment to autonomy or intelligence in any fuller sense.

This single vocabulary is what lets us describe the whole inhabited layer of the Prolog Web at one level of abstraction. The rest of this section sketches the first-level taxonomy. Figure 3.1 is deliberately asymmetric. It expands the actor branch into the principal programming forms used in the book, while leaving the organisation of nodes as a dashed promise to be fulfilled in Section 3.5. The sections that follow first develop actor agents and their lifecycle, then toplevel actors and their protocol, and finally nodes as agents on the Prolog Web.

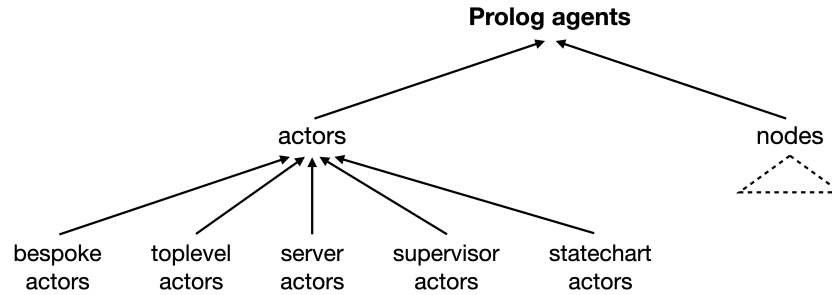


Fig. 3.1 A first-level taxonomy of Prolog agents. Arrows point towards the more general kind. The actor branch distinguishes bespoke actors from actors governed by four principal reusable behaviours used in this book. The dashed node branch announces a further division into four cumulative sub-profiles, whose names and requirements are deferred to Section 3.5.

Actors and nodes. These are the two principal architectural kinds. Actors are independently scheduled loci of concurrent computation, equipped with pids and mailboxes. Nodes operate at a larger scale: they host computation and knowledge, expose web APIs, route messages, and enforce owner policy.

Bespoke actors. A bespoke actor is programmed directly for a particular task. Its initial goal and, where it is interactive, its receive loop encode the application’s own messages, state changes, and stopping conditions. The echo actors, the actors that play ping-pong, and the hand-written count actor are simple examples. “Bespoke” does not imply short-lived or disposable: the same style can be used for a durable, named, node-resident service. It means that the application author supplies the control loop rather than instantiating one of the reusable behaviours shown alongside it in the figure.

Generic actors and behaviours. The remaining four actor branches name generic forms, more precisely actors governed by reusable *behaviours*. A behaviour factors recurring protocol and lifecycle machinery out of the application:

- A *toplevel actor* implements the Prolog Toplevel Communication Protocol (PTCP), retains a query’s suspended search, and delivers solutions incrementally.
- A *server actor* provides correlated request–response handling and threads private state from one request to the next; an application supplies the callback that interprets requests.

- A *supervisor actor* starts, monitors, terminates, and, according to policy, restarts children; an application supplies the child specifications and restart strategy.
- A *statechart actor* provides event selection and run-to-completion control; an application supplies the SXML statechart, including its states, transitions, guards, and actions.

Thus a generic actor is still a concrete running actor with a pid and a mailbox. What is generic is the control machinery that governs it.

This list is deliberately not exhaustive. Other reusable actor behaviours could be defined – for example, an *event-manager* behaviour analogous to Erlang/OTP's `gen_event`, dispatching events to dynamically installed handlers. Nor is the design space limited to patterns inherited from OTP. The toplevel behaviour already demonstrates how a reusable actor behaviour can package capabilities distinctive to Prolog; further Prolog-specific forms of search and interaction may be worth exploring.

Application-specific instances of generic behaviours. Specialising a behaviour produces an application-specific actor. A shell is a user-facing arrangement built around a toplevel actor, with terminal-oriented facilities supplied by the node controller. The expert-system agent developed later in Chapter 8 is realised as a toplevel actor, made distinctive by the inference engine and domain knowledge loaded into it. A particular callback turns the server behaviour into a particular service, and particular child specifications turn the supervisor behaviour into a concrete supervision tree. Statechart actors can likewise be specialised as agent harnesses, service-request agents, or dialogue managers. The figure therefore classifies actors primarily by where their control machinery comes from, not by every application role they may eventually play.

Organising nodes. The dashed triangle below *nodes* indicates that nodes also admit a more detailed organisation. Section 3.5 will introduce four cumulative *sub-profiles*: each successive sub-profile promises everything required by the preceding one and adds a broader language or interaction surface. This profile hierarchy is independent of how a node is realised. In particular, the distinction between a persistent native node and a node hosted in a browser is an implementation and deployment distinction, not a fifth sub-profile or an alternative profile hierarchy. The contrast is introduced in Section 3.5.5 and developed further in Sections D.8 and E.4.1.

Cross-cutting properties. An agent may be momentary or long-lived, and it need not be stateful. Some are effectively stateless, such as the echo actor; others maintain state in receive-loop parameters, in control structure, or in a private dynamic database. Statefulness therefore cuts across the branches of the figure rather than defining one of them. Actors may also combine a private workspace – including an optional private dynamic database – with shared, node-provided knowledge. Private definitions can *shadow* shared ones, allowing local specialisation without modifying the shared base; Section 3.2.4 works out the precise semantics.

3.2 More about Prolog actor agents

Actor agents are the workhorses of the concurrent strand of the Trinity: they execute Prolog code concurrently, interact by exchanging Prolog terms, and may maintain actor-local state across repeated interactions. This section makes that operational picture more concrete. We first introduce the actor's private Prolog database and the `src_*` options used to initialise it. We then sketch the lifecycle of a typical actor and explain how receive loops realise explicit communication protocols. Finally, we discuss the dynamic (still private) database as a non-logical but sometimes convenient mechanism for actor-local updates.

3.2.1 An actor agent is equipped with a private Prolog database

When running the examples in Chapter 2 we were somewhat vague about the origin of the code being executed. The only clue provided was that it might reside in the node's shared database, which defines read-only data and programs accessible to any actor living on that node. While we will delve deeper into the shared database later in this chapter, we first turn our attention to the other storage area – the one located *within* the actor itself and where programs and data exclusive to that actor are stored. While all actors have a private database, it starts out empty by default.

During the spawn operation, the (soon-to-be) parent process can initialise the private database of the (soon-to-be) child actor with clauses defining one or more predicates. Consider the following example where the `src_text` option is passed to `spawn/3`, specifying that the source code for our count server should be loaded into the actor's private database before calling the goal in the first argument:

```
?- spawn(count_actor(0), Pid, [
    src_text("
        count_actor(Count0) :-
            receive({
                count(From) ->
                    Count is Count0 + 1,
                    From ! count(Count),
                    count_actor(Count) ;
                stop ->
                    true
            }).
    "),
    monitor(true)
]).
Pid = 45092311.
?-
```


In addition to the `src_text` option, three other options can be used to populate the private database of an actor. The `src_list` option loads a list of clauses or directives, the `src_uri` option loads the content specified by a URI, and `src_predicates` takes a list of predicate indicators and loads the clauses for the indicated predicates that are accessible by the caller.

The clauses in an actor's private database represent its unique beliefs and skills, distinguishing them from the shared beliefs and skills accessible to other actor agents on the same node. In this sense, the private database can be seen as the core of the actor's individuality, encapsulating the knowledge and skills that are exclusive to it.

The `src_*` options make a kind of sharing of beliefs and skills between a parent process and a child process possible. The child is “born” with them, they are “innate.” It is up to the parent actor to determine exactly what should be passed on to the child actor. We can think of this as “learning by injection.”

3.2.2 The lifecycle of a typical Prolog actor agent

We have the black boxes that do things, and we have the messages in between. What goes on inside the black boxes does not really matter; what takes place there is abstracted over.

Joe Armstrong

The diagram in Figure 3.2 illustrates the lifecycle of a typical Prolog actor.



Fig. 3.2 Lifecycle of a Prolog actor: spawn, interact, terminate, and clean up

When an actor is spawned, it first passes through an initialisation phase where it is configured in a way that will suit its intended purpose. This includes specifying values for options such as `monitor`, `link` as well as the `src.*` options.

For example, when the `count` actor in the previous example was spawned, the source code specified by the `src_text` option was loaded into the actor's private Prolog database and the `monitor` option instructed the runtime to deliver a `down` message to the shell when the actor terminated. The call to `spawn/3` succeeded, binding the variable in the second argument to a `pid`. By this, the initialisation phase is completed, the actor is now in its *initial state*, and the loop that implements the actor's *communication protocol* is entered. Typically, processing stops at a call to `receive/1-2` where it blocks, waiting for a `count` message or a `stop` message to appear in its mailbox.

The notion of a communication protocol is central to Erlang-style concurrent programming. The loop implements the communication protocol that clients that want to interact with the actor must follow. The interaction is driven by coordinating send and receive operations, with the protocol defining the rules. The sending process transmits messages in the specified format, and the receiving process captures and interprets them. They must adhere to the syntax, semantics and timing constraints of the communication protocol:

- **Syntax:** This refers to the structure and format of the data exchanged between processes. It dictates how data is packaged, including the organisation of various data elements.
- **Semantics:** This involves the meaning and interpretation of the exchanged data. It defines the rules for understanding the content of the message, what actions should be triggered upon receiving certain data, and how to handle errors or unexpected data.
- **Timing:** This concerns the timing aspects of the communication, such as when data is sent, the pace of data exchange, and managing the order and sequence of messages so that both processes can coordinate despite delay, reordering, and failure.

The Web Prolog `receive` operator, while blocking the process at the point of a `receive/1-2` call, allows for the implementation of very flexible communication protocols through mechanisms like selective pattern matching and timeouts. The selective pattern matching allows the process to wait for a particular message but can also defer out-of-order messages for later processing. By specifying a timeout to avoid the `receive` call being blocked indefinitely, alternative actions can be taken if an expected message does not arrive in time. This helps reduce the chance of breakdowns and allows for more adaptive communication.

Whenever the actor runs a `receive/1-2` call, a message is selected from the mailbox and processed sequentially. The result often depends on the state of the process, and as part of processing, a new state for the actor may be computed. This will be the current state for processing the next message selected.

Some actors, such as the `echo` actor, are stateless. Stateful actors represent their state and perform state transitions in a variety of ways: as values threaded through

the arguments of a receive loop (for example an integer updated by calls to `is/2` in the count actor, or a list of atoms in the fridge simulation), as the explicit states and transitions of a state machine, or implicitly through the choice points created by a failure-driven loop. These techniques all encode state in the control flow or data flow of the program itself. In addition to these, Web Prolog offers a more direct – though non-logical – mechanism for maintaining actor-local state: a private dynamic Prolog database.

3.2.3 The dynamic (and still private) Prolog database

Each actor owns a private Prolog database that exists for the lifetime of the actor. While the actor remains alive, it may update this database using predicates such as `assert/1`, `retract/1`, and `retractall/1`. In accordance with the ISO standard, predicates modified in this way must be declared using the `dynamic/1` directive.

Such updates are strictly actor-local. A clause asserted by one actor is added only to that actor's private database and is not visible to the shell, to other actors on the same node, or to actors elsewhere on the Prolog Web. Dynamically asserted clauses also share the lifetime of the actor: when the actor terminates, its private database and any changes made to it disappear.

For example, assuming that `foo/1` has been declared dynamic, the following goal is permitted:

```
?- spawn(assert(foo(a)), Pid).
Pid = 32861299.
?-
```

The spawned actor asserts the clause `foo(a)` in its own database and then terminates. The clause is never added to the database of the shell and disappears together with the actor that asserted it. Although this particular use of `spawn/2` has little practical value, it illustrates both the ownership and the lifetime of dynamically asserted clauses.

The dynamic database gives an actor a direct way to represent evolving state without threading an explicit state term through the arguments of its predicates. An actor may, for example, retract clauses describing its current state and assert clauses describing a new state before processing the next message. This can occasionally be convenient, but it should be used judiciously.

Private mutation is not shared mutation

The familiar objections to the Prolog dynamic database still apply. Database updates are not undone on backtracking, and the meaning of a dynamic predicate may therefore depend on the operational history of the actor rather than solely on the

arguments of a particular call. Dynamic predicates consequently introduce hidden mutable state and may make programs more difficult to understand, test, and debug.

Their actor-local scope nevertheless changes the concurrency picture considerably. Two actors cannot race to update the same private dynamic predicate, because each actor owns a separate database. For the same reason, `assert/1` and `retract/1` cannot be used as an implicit mechanism for inter-process communication. Actor-locality does not make database mutation logical, but it prevents private mutation from becoming uncontrolled shared-state mutation.

A useful comparison can be made with the Erlang process dictionary. Each Erlang process has a private key-value store that cannot be accessed directly by other processes. Like the dynamic database, the process dictionary is generally used sparingly because it introduces hidden state and undermines referential transparency. The Prolog dynamic database is considerably more expressive: it stores clauses rather than key-value pairs and allows their contents to be queried relationally. This makes it more useful, but also more tempting to overuse.

The restriction to actor-local databases does not rule out explicit forms of shared storage. Web Prolog could, for example, provide access to a transactional RDF store or to a Linda-style blackboard. Such facilities would, however, constitute separate abstractions with their own policies for access, lifetime, consistency, and concurrency; they would not turn the private databases of individual actors into shared state.

For the time being, Web Prolog deliberately restricts the ordinary dynamic database to the private scope of individual actors. Throughout the Prolog Trinity ecosystem, private state is treated as local and volatile, whereas shared data is governed by explicit node-level policies and exposed only under controlled conditions.

3.2.4 Private beliefs and shared knowledge – shadowing

A predicate defined in an actor's private database takes precedence over a predicate with the same name and arity in the shared database. Thus, for an unqualified call made from the actor's private database, Prolog first looks for a private definition and otherwise continues to the shared database. We say that the private predicate *shadows* the shared predicate.

Suppose the shared database contains the following price list and default pricing rule:

```
list_price(widget, 100).
list_price(gadget, 250).
list_price(gizmo, 400).

price(Item, Price) :-
    list_price(Item, Price).
```

An actor without a private definition of `price/2` uses the shared rule:

```
?- price(widget, Price).
Price = 100.
?-
```

We can give a newly spawned actor a private pricing policy by passing it with the `src_text` option. The actor uses the private `price/2`, but its call to `list_price/2` still resolves to the shared facts:

```
?- self(Self),
   spawn(
     ( price(widget, Price),
       Self ! discounted_price(Price)
     ),
     -,
     [ src_text("
        price(Item, Price) :-
          list_price(Item, ListPrice),
          Price is ListPrice * 0.9.
      ")
     ]
   ),
   receive({discounted_price(Price) -> true}).
Self = 34001234,
Price = 90.0.
?-
```

The private pricing rule belongs only to the spawned actor. It neither modifies the shared rule nor becomes part of the parent's private database. Consequently, another actor spawned by the parent without that `src_text` option uses the shared definition:

```
?- self(Self),
   spawn(
     ( price(widget, Price),
       Self ! ordinary_price(Price)
     ),
     -,
     ),
   receive({ordinary_price(Price) -> true}).
Self = 34001234,
Price = 100.
?-
```

Shadowing applies to a complete predicate, not to individual clauses. The following actor receives a private `list_price/2` containing only one fact:

```
?- self(Self),
   spawn(
```

```

    ( findall(Item-Price,
              list_price(Item, Price),
              Prices),
      Self ! private_prices(Prices)
    ),
    -,
    [src_text("list_price(widget, 80).")]
  ),
  receive({private_prices(Prices) -> true}).
Self = 34001234,
Prices = [widget-80].
?-

```

The private fact does not combine with the shared facts. Once the actor has defined its own `list_price/2`, all shared clauses of `list_price/2` are hidden from unqualified calls made from that actor's private database. In particular, the actor does not obtain prices for `gadget` or `gizmo`.

The shared database itself remains unchanged. The parent still sees the complete shared price list:

```

?- findall(Item-Price,
           list_price(Item, Price),
           Prices).
Prices = [widget-100, gadget-250, gizmo-400].
?-

```

Shadowing is therefore local to an actor: it changes how that actor resolves a predicate without modifying the node-provided knowledge seen by other actors.

3.3 Prolog shells and other toplevel actors

As we saw already in Chapter 2, calling `self/1` in a terminal binds its argument to the pid of the currently running shell process:

```

?- self(Pid).
Pid = 72097632.
?-

```

Shells handle all the things that toplevels handle, but also add a number of useful things for when we are talking to Prolog over a terminal. This includes I/O (read and write) and utilities such as `flush/0` and the dollar notation. Such features are supplied by the *node controller* rather than a toplevel alone. (We will say more about the node controller further ahead in this chapter.)

Here is how we instruct the shell to spawn a new toplevel:

```
?- toplevel_spawn(Pid).
Pid = 16226587.
?-
```

Note that 16226587 is a toplevel actor but not a shell. As with every other Prolog actor, it comes with a mailbox, a private Prolog database, the ability to send messages to other actors, as well as the ability to create other actors.

Figure 3.3 shows the overall anatomy of a toplevel actor. The protocol at its centre is the Prolog Toplevel Communication Protocol (PTCP), which is described in detail in the next section.



Fig. 3.3 A Prolog toplevel actor. Its distinguishing feature is the built-in Prolog Toplevel Communication Protocol (PTCP), shown at the centre of the figure.

3.3.1 A Prolog toplevel is an actor with a built-in protocol

When a programmer starts a traditional Prolog system, they are usually presented with an interactive query prompt. This interactive environment is commonly referred to as the *toplevel*. Throughout this book, however, we reserve the word *shell* for this user-facing interface, and use the term *toplevel* more generally to denote the underlying abstraction, regardless of whether it is connected to a terminal, another actor, or an external programming language.

In traditional Prolog, the shell is a unique system component. A programmer can submit queries to it, inspect successive solutions, and eventually terminate the interaction, but an ordinary Prolog program cannot itself create a new toplevel, submit queries to it, or control it programmatically. Web Prolog removes this restriction by making toplevels first-class actors that can be created, addressed, and communicated with like any other actor.

A Web Prolog toplevel actor encapsulates an independent Prolog session. It behaves very much like the familiar interactive shell, except that it is no longer tied to a terminal. Instead, it can be controlled from another Web Prolog actor, from JavaScript, or from any other client capable of speaking its protocol. This makes the familiar Prolog interaction model available both to human users and to programs.

Like the traditional shell, a Web Prolog toplevel computes solutions lazily. After a query has been submitted, it produces the first solution when available, but additional solutions are generated only when explicitly requested. This preserves the interaction model that Prolog programmers are accustomed to while making it available as a reusable programming abstraction.

A toplevel actor differs from ordinary actors not because it has a special implementation, but because it follows a specialised communication protocol. The protocol defines which messages a client may send, which messages the toplevel may return, and how the conversation may legally proceed. Besides submitting queries and requesting additional solutions, the protocol also allows the running program to produce output, request input, report failures, signal errors, and eventually terminate the current query. In other words, the protocol captures the complete dialogue between a client and a Prolog session.

The protocol is closely modelled on the informal conversation that every Prolog programmer already knows. A user starts a query, receives a solution, requests another solution by typing a semicolon, accepts the current solution by pressing return, or submits a completely new query after the previous one has finished. Although these interactions feel informal when performed at a terminal, they constitute a well-defined sequence of conversational moves. At every stage, only certain messages are valid, making the interaction naturally describable as a finite-state protocol. The next section formalises this protocol using a state machine.

3.3.2 The Prolog Toplevel Communication Protocol

The diagram in Figure 3.4 is a statechart specifying the Prolog Toplevel Communication Protocol (PTCP) – a protocol for the communication between a client and a toplevel actor. The client can be any process (including another actor or (say) a JavaScript process) capable of sending the messages and signals in bold to the toplevel. The toplevel actor is responsible for returning the messages with a leading / back to the client. The use of a statechart allows us to show that no matter the current state of the protocol, **abort** will always take it to the state from which a new goal can be called and **halt** will always terminate the toplevel process. At the predicate level, the **halt** move is issued through `toplevel_halt/1-2` and implemented using the general actor exit mechanism.

A process is able to *spawn* a toplevel. After having done so, the process becomes the parent of the toplevel and can start communicating with it according to the protocol. Initially, the protocol is in state **s1**, where the toplevel rests idle, waiting for a **call** message containing a goal. When such a message arrives, the protocol



Fig. 3.4 Statechart specifying the Prolog Toplevel Communication Protocol (PTCP). Bold transitions denote messages sent from the client to the toplevel, whereas transitions prefixed by '/' denote responses sent by the toplevel back to the client.

transitions to state **s2**, where the toplevel is doing actual work. The protocol will remain in state **s2** until some work is done and the toplevel sends a message indicating either **/success**, **/failure**, or **/error**. If the actor terminates while monitored, the runtime separately delivers **/down** to the client. On the event of the toplevel sending **/failure** or **/error**, the protocol will transition back to state **s1**. This will be the case also for a **/success** message that indicates no more solutions to the query can be found (marked with **false** in the chart). However, if a **/success** message indicates more solutions may exist (marked with **true** in the chart), the protocol transitions to state **s3**, where it will wait for a message **next**, **stop**, **abort** or **halt** to arrive from the client. If the message is **stop** or **abort** the protocol will transition back to state **s1**, if it is **next** it will transition to state **s2** and trigger the search for more solutions, and finally, if it is **halt**, the toplevel terminates and the runtime delivers **/down** for each installed monitor. Any other messages will be deferred to the code being executed by the toplevel.

Web Prolog comes with six built-in predicates which allow a client to spawn a toplevel actor and interact with it during its lifecycle:

<code>toplevel_spawn/1-2</code>	<code>toplevel_call/2-3</code>	<code>toplevel_next/1-2</code>
<code>toplevel_stop/1</code>	<code>toplevel_abort/1</code>	<code>toplevel_halt/1-2</code>

As witnessed by some of the examples in Chapter 2, such predicates can be defined in terms of the more primitive `spawn/1-3`, `!/2` and `receive/1-2` predicates.

The sections that follow introduce the `toplevel_*` predicates and their most important options through examples. Complete signatures, option descriptions, and default values are given in Appendix B.

Toplevel actors are stateful

A Prolog toplevel is a kind of server in the Erlang sense (see Section 2.4.1). Like any server, it maintains state across successive interactions with its clients.

Interestingly, a toplevel remains stateful even if the program it executes neither threads an explicit state through its predicates nor uses the dynamic database. Instead, the state resides in the suspended Prolog computation itself. After producing a solution, the toplevel retains the execution state – including any remaining choice points – needed to resume the search if the client later requests another solution. In other words, it remembers where the search left off. This implicit computational state is what enables the familiar lazy interaction model of Prolog, where successive solutions are generated only on demand.

State **s3** of the PTCP exists precisely because the toplevel retains this suspended computation while waiting for the client to decide whether to continue the search. If the client sends a **next** message, execution resumes where it previously left off by backtracking to the next remaining choice point. If it sends **stop**, the suspended computation and its remaining choice points are simply discarded.

3.3.3 Shell talking to a Prolog toplevel

Below, we show an example of how to create and interact with a toplevel process from a shell. We start by spawning a new toplevel, using `toplevel_spawn/2` with the `monitor` option to ask the runtime to notify us with a `down` message when the process eventually terminates. The `src_list` option is used to populate the private Prolog database belonging to the toplevel actor with two simple unit clauses `p(a)` and `p(b)`:

```
?- toplevel_spawn(Pid, [
    session(true),
    monitor(true),
    src_list([p(a),p(b)])
]).
Pid = 74122981.
?-
```

With this, the toplevel actor is initiated, and with the PTCP now in state **s1**, it is ready to accept messages and signals sent by the other `toplevel_*` predicates.

The `monitor` and `src_list` options are inherited from `spawn/1-3`. However, `toplevel_spawn/2` offers a couple of additional options that can be used to specify the behaviour of the toplevel actor at creation time. In the above example the `session` option is used to configure the toplevel to run a multi-goal session rather than exit after having run one goal to completion.

Making the toplevel call a goal

Let us see what happens if we call `toplevel_call/2` with the default values for options:

```
?- toplevel_call($Pid, p(X)).
true.
?- flush.
Shell got success(74122981,[p(a),p(b)],false)
true.
?-
```

The answer is returned to the mailbox of the calling process in the form of an *answer term* with three arguments. The functor of the term represents its *type*. In this case, it shows that the goal succeeded. (In other cases it might indicate a failure or an error.) The first argument of the success term is the pid, and the list in the second argument represents the two solutions that were computed. The value `false` in the third argument of the term indicates that no more solutions exist.

Since the `session` option was set to true, the PTCP is now, after a brief visit to state `s2` for the execution of the goal, back in state `s1` and is prepared to accept another use of `toplevel_call/2-3`.

Using the template option

Below, `toplevel_call/3` is called with the goal `p(X)` and with the `template` option set to the variable `X`:

```
?- toplevel_call($Pid, p(X), [
    template(X)
]).
true.
?- flush.
Shell got success(74122981,[a,b],false)
true.
?-
```

Note how the value of the `template` option determined the form of the list of solutions in the second argument of the answer term. The relation to the Prolog built-in standard predicate `findall/3` is evident.

Using the offset and limit options

When querying a relational database using SQL or an RDF dataset using SPARQL, the use of the `OFFSET` and `LIMIT` keywords serves to control the subset of results that are returned by a query. In Web Prolog they can be used as options for the

configuration of calls to `toplevel_call/3`. The `limit` option specifies the maximum number of solutions to return, while the `offset` option specifies the number of solutions to skip before starting to return solutions. Here is what happens if `limit` is set to 1:

```
?- toplevel_call($Pid, p(X), [
    template(X),
    limit(1)
]).
true.
?- flush.
Shell got success(74122981,[a],true)
true.
?-
```

The value `true` in the success term means that the state machine is now in state **s3**, waiting for a message **next** or **stop** to appear. Calling `toplevel_next/1` produces the next solution:

```
?- toplevel_next($Pid).
true.
?- flush.
Shell got success(74122981,[b],false)
true.
?-
```

`toplevel_call/3` supports the `offset` option. Together with the `limit` option it allows the call to pick out an arbitrary slice of solutions to a goal, even when the number of solutions is infinite:²

```
?- toplevel_call($Pid, between(1,infinite,I), [
    offset(100),
    template(I),
    limit(3)
]).
true.
?- flush.
Shell got success(74122981,[101,102,103],true)
true.
?-
```

Calling `toplevel_next/1` produces three more solutions:

```
?- toplevel_next($Pid).
true.
```

² It may seem odd and not all that useful for `toplevel_call/2-3` to support the `offset` option, but it turns out to be important when implementing the stateless HTTP protocol.

```
?- flush.
Shell got success(74122981,[104,105,106],true)
true.
?-
```

Note that the value of the `limit` option set in the call to `toplevel_call/3` was still in effect. However, `toplevel_next/2` accepts the `limit` option too, which allows us to increase or decrease the limit on the number of solutions returned:

```
?- toplevel_next($Pid, [
    limit(5)
]).
true.
?- flush.
Shell got success(74122981,[107,108,109,110,111],true)
true.
?-
```

Calling `toplevel_stop/1` will stop the execution of the goal:

```
?- toplevel_stop($Pid).
true.
?-
```

The `offset` and `limit` options are particularly useful in several scenarios: pagination of results in web applications, where it is impractical to retrieve and display all records at once; implementing infinite scrolling interfaces that load data dynamically as the user scrolls; handling goals with very many or infinitely many solutions without overwhelming the client; and reducing the number of network roundtrips on the Prolog Web by fetching solutions in batches. A special case is when `limit` is set to 1, which makes it straightforward to implement a Prolog-style REPL in JavaScript, where each press of `;` triggers a new request for the next solution.

Positional paging presupposes that the goal enumerates its solutions in a stable order across the requests that page through it. Stable shared data satisfies that assumption, but owner-side updates may disturb it; what can drift, what the logical update view preserves, and what a genuine snapshot would require are developed in Section 4.2.4.

Using the `once` option

The `once` option only makes sense in combination with the `limit` option; its default is `false`. Here it is set to `true`:

```
?- toplevel_call($Pid, between(1,infinite,I), [
    offset(100),
    template(I),
    once
```

```

        limit(3),
        once(true)
    ]).
true.
?- flush.
Shell got success(74122981,[101,102,103],false)
true.
?-

```

The `once` option requests *committed* evaluation: the node returns the first slice (of up to `limit` solutions) and then discards any remaining choice points rather than retaining them for a possible `toplevel_next`. This is reflected in the answer term, whose third argument is `false` even when further solutions could in principle have been found – the call reports completion because the node has undertaken to produce nothing more. It differs from simply declining to call `toplevel_next/1`: that would leave the session suspended in state `s3`, holding resume state until the next request or an idle timeout, whereas `once(true)` releases that state at once. The option is therefore a resource directive as much as a control one, useful when a client knows it wants at most `limit` solutions and wants the node to be free of any obligation to retain the computation.

A toplevel can do a kind of I/O

Whenever a toplevel is in state `s2` and doing some real work, it is able to send messages to its parent (or elsewhere) using the ordinary send operator. However, a better idea can often be to use `output/1`:

```

?- toplevel_call($Pid, output(hello)).
true.
?- flush.
Shell got output(74122981,hello)
Shell got success(74122981,[output(hello)],false)
true.
?-

```

Input can be collected by calling `input/2`, which sends a `prompt` message to the client, which in turn can respond by calling `respond/2`:

```

?- toplevel_call($Pid, input('Input', X)),
   receive({Answer -> true}).
Answer = prompt(74122981,'Input').
?- respond($Pid, hello),
   receive({Answer -> true}).
Answer = success(74122981,[input('Input',hello)],false).
?-

```

Aborting a nonterminating goal

Our toplevel actor is still not dead so let us see what happens when we ask the toplevel to first update its private Prolog database with a silly recursive clause for a predicate `p/0` and then call it:

```
?- toplevel_call($Pid, assert((p :- p))),
   receive({Answer -> true}).
Answer = success(74122981,[assert((p:-p))],false).
?- toplevel_call($Pid, p).
true.
?-
```

Although nothing is shown in the terminal, it is clear that the toplevel is now just wasting CPU cycles to no avail. Fortunately, a nonterminating goal can be aborted by calling `toplevel_abort/1`:

```
?- toplevel_abort($Pid).
true.
?-
```

With this, the PTCP is back in state **s1**.

Halting the toplevel

When we are done talking to the toplevel we can halt it:

```
?- toplevel_halt($Pid, Reply).
Reply = true.
?- flush.
Shell got down(74122981,74122981,true)
true.
?-
```

The acknowledgement returned by `toplevel_halt/2` and the original monitor notification are distinct: halting waits on a private monitor, while the monitor installed at spawn time still delivers its own `down/3` message.

Toplevels and the message deferring mechanism

In the following example, a toplevel is spawned, and then `toplevel_next/1` is called. The protocol is in state **s1**, which is obviously not in a state where it can react to the **next** message (see Figure 3.4). The message is therefore deferred and it is not until `toplevel_call/3` is called and the protocol changes states that it has an effect.

```

?- toplevel_spawn(Pid).
Pid = 78340943.
?- toplevel_next($Pid).
true.
?- flush.
true.
?- toplevel_call($Pid, member(X,[a,b,c]), [
    limit(1),
    template(X)
]).
true.
?- flush.
Shell got success(78340943, [a], true)
Shell got success(78340943, [b], true)
true.
?-

```

Note that messages sent to a toplevel will often be handled in the right order even if they arrive in the “wrong” order (for example, **next** before **call**). This is due to the selective receive which defers their handling until the PTCP permits it. The **abort** and **halt** operations, however, will never be deferred. This is guaranteed by the fact that **abort** and **halt** are valid transitions from any state in the protocol (see Figure 3.4). They are signals rather than mailbox messages: abort unwinds the submitted goal to the PTCP boundary, whereas halt terminates the actor. Section 4.1.7 specifies the distinction.

One way to think of this is in terms of the so called *robustness principle*: “Be conservative in what you send, be liberal in what you accept.”³ Due to the deferring behaviour a toplevel is liberal in this way, but, as implied by the principle, clients are advised not to rely on this behaviour.

Redirecting answers

The **target** option can be used to redirect the answer terms produced by the toplevel to any actor of choice. By default, it is set to the pid of the parent but allows us to instruct a toplevel actor to send answer terms to a different destination. To demonstrate how this works, we first create a simple actor that can serve as a target:

```

?- spawn(( repeat,
    receive({
        Msg ->
            format("Received ~p~n", [Msg]),
            fail
    })

```

³ https://en.wikipedia.org/wiki/Robustness_principle


```

        ), Pid0).
Pid0 = 98380209.
?-

```

The pid of that actor can now be used as the value of the target option:

```

?- toplevel_spawn(Pid, [
    target($Pid0)
]).
Pid = 97919106.
?-

```

When `toplevel_call/3` is called, the success answer term is printed by the designated target actor:

```

?- toplevel_call($Pid, between(1,1000,N), [
    template(N),
    limit(5)
]).
true.
Received success(97919106,[1,2,3,4,5],true)
?-

```

So, 98380209 rather than the shell received the message from 97919106.

Calling `flush/0` shows that the mailbox belonging to the shell is empty:

```

?- flush.
true.
?-

```

The `target` option is supported by `toplevel_spawn/2`, `toplevel_call/3` and `toplevel_next/2`. It is not supported by the other `toplevel_*` predicates as they do not produce answer terms. It can be used in calls to `output/2`. See Appendix B for more details.

3.3.4 Programming with toplevel actors

The toplevel actor is an important kind of agent in the Prolog Trinity ecosystem. However, the reasons for its importance will have to be deferred to later chapters, and this section only offers a few exercises to build familiarity with it.

A synchronous predicate API to toplevels

The communication between a client and a toplevel illustrated above is asynchronous, but if we want, it is quite easy to define a synchronous alternative using a technique we have already seen. A predicate `wait_for_answer/3` can be defined like so:

```
wait_for_answer(Pid, Answer, Options) :-
    receive({
        Answer if arg(1, Answer, Pid) ->
            true
    }, Options).
```

Then `toplevel_call_synch/3-4` can be implemented as follows:

```
toplevel_call_synch(Pid, Goal, Answer) :-
    toplevel_call_synch(Pid, Goal, Answer, []).

toplevel_call_synch(Pid, Goal, Answer, Options) :-
    toplevel_call(Pid, Goal, Options),
    wait_for_answer(Pid, Answer, Options).
```

and `toplevel_next_synch/2-3` like so:

```
toplevel_next_synch(Pid, Answer) :-
    toplevel_next_synch(Pid, Answer, []).

toplevel_next_synch(Pid, Answer, Options) :-
    toplevel_next(Pid, Options),
    wait_for_answer(Pid, Answer, Options).
```

Note that since `toplevel_stop/1` does not produce a response message, no `toplevel_stop_synch/2-3` is defined, and `toplevel_stop/1` can be used as it is.

Here is a simple test that shows how it works:

```
?- toplevel_spawn(Pid).
Pid = 67590967.
?- toplevel_call_synch($Pid, mortal(Who), Answer, [
    template(Who),
    limit(1)
]).
Answer = success(67590967, [socrates], true).
?- toplevel_next_synch($Pid, Answer).
Answer = success(67590967, [plato], true).
?- toplevel_next_synch($Pid, Answer).
Answer = success(67590967, [aristotle], false).
?-
```

Reconstructing `findall/3` and `findnsols/4`

In order to specify the semantics of a new Prolog predicate API it can be a good idea to implement built-in Prolog predicates with a well-known semantics using the

API. For example, if we did not already have a built-in predicate `findall/3`, using a toplevel actor we could have implemented it like so:

```
findall(Template, Goal, List) :-
    toplevel_spawn(Pid, [
        session(false)
    ]),
    toplevel_call(Pid, Goal, [
        template(Template)
    ]),
    receive({
        success(Pid, List, false) ->
            true ;
        failure(Pid) ->
            List = [] ;
        error(Pid, Error) ->
            throw(Error)
    }).
```

Of course, implementing each of these predicates in a separate process makes little practical sense. These deliberately contrived examples merely exercise some of the features of the `toplevel_*` API. Nevertheless, they can illuminate the relationship between `findall/3` and the `toplevel_*` predicates.⁴

We can take this one step further by also implementing `findnsols/4`, which works like `findall/3`, but generates at most *N* solutions to the specified goal.

This predicate is especially useful if the goal may have an infinite number of solutions.

```
findnsols(N, Template, Goal, List) :-
    toplevel_spawn(Pid, [
        session(false)
    ]),
    toplevel_call(Pid, Goal, [
        template(Template),
        limit(N)
    ]),
    wait_solutions(Pid, List).
```

```
wait_solutions(Pid, List) :-
    receive({
        success(Pid, List, false) ->
            true ;
        success(Pid, List0, true) ->
            ( List = List0
            ; toplevel_next(Pid),
```

⁴ Here, we were inspired by (Tarau, 2000).

```

        wait_solutions(Pid, List)
    ) ;
    failure(Pid) ->
        List = [] ;
    error(Pid, Error) ->
        throw(Error)
}).

```

Ciao Prolog provides this predicate, but only in a deterministic version. The version provided by SWI-Prolog is nondeterministic, and so is our reconstruction. We borrow an example of its use from the SWI-Prolog manual:⁵

```

?- findnsols(5, I, between(1, 12, I), L).
L = [1, 2, 3, 4, 5] ;
L = [6, 7, 8, 9, 10] ;
L = [11, 12].
?-

```

Interestingly, it turns out that the version provided by SWI-Prolog serves as an important building block in our proof-of-concept implementation of the `toplevel_*` predicates. (It was implemented in SWI-Prolog by Jan Wielemaker in order to support `library(pengines)`, the library on top of which SWISH is built.)

3.4 Prolog nodes are also agents

As shown in the agent taxonomy in Figure 3.1, we regard a Prolog node as a kind of Prolog agent. This classification rests on its stable, term-based interface. Functionally, a node may also expose some of the query–answer facilities associated with a `toplevel` actor: clients can submit goals and receive solutions lazily, either one at a time or in bounded slices, in the order in which Prolog finds them. This functional overlap does not imply that a node is implemented as, or derived from, a `toplevel` actor.

The diagram in Figure 3.5 illustrates the anatomy of an executing Prolog node, which is accessed by a user-client that knows its URI and is authorised to use its services.

As suggested by the diagram, a node is comprised of components such as a node controller, web APIs, a shared database containing predicates defined by the owner, a container serving as the “home” of actors such as echo actors, supervisors and `toplevels`, and various settings for the owner to configure the node. As we shall see, not all nodes include all these components.

⁵ https://www.swi-prolog.org/pldoc/doc_for?object=findnsols/4

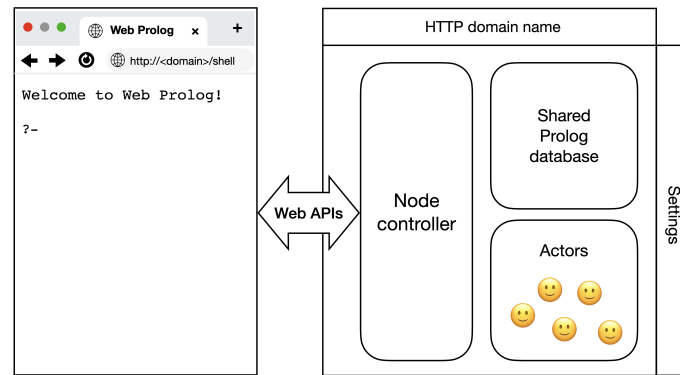


Fig. 3.5 The anatomy of a Prolog node. A terminal running in a web browser is attached to it.

3.4.1 The node controller

In the broadest profile, the node controller is entrusted with a multitude of responsibilities designed to ensure the smooth and secure operation of the node. Although not every node controller undertakes all of these tasks, the most advanced implementations are expected to manage web API connections, authenticate and authorise clients, and spawn actors upon client request. Additionally, they impose various resource restrictions to maintain optimal performance, oversee registries, monitors, and links, and guarantee that messages sent back to clients adhere to the required format. The node controller also terminates actors that have lost client connections and implements shell utilities such as the \$Var substitution mechanism and the redefinition of I/O predicates.

From an external perspective and when dealing with actors, the intricate workings of the node controller are intended to remain largely concealed, functioning much like an invisible “ether” through which messages seamlessly flow. In this manner, the node controller abstracts away the underlying technical details, allowing users to benefit from its comprehensive management facilities without the need to engage directly with its multifaceted processes.

This description does not conflict with treating the node as an agent in its own right. The two descriptions concern different faces of the same entity. At its URI-addressed boundary, where it accepts goals and returns results, the node is a correspondent; within that boundary, where its controller routes messages and governs hosted actors, it is a medium. A client meets the first face, while hosted actors experience the second.

3.4.2 The shared Prolog database

A node may host a shared Prolog database accessible over HTTP. It can be:

- *self-contained* – every predicate the exported relations need is available on any node: defined locally, a Web Prolog built-in, or a standard library predicate. Such a dataset is directly consumable by other nodes or clients.
- *dependent* – it relies on a predicate that is not guaranteed to be present wherever the database is consumed: one imported from a custom (non-standard) module, a remote predicate, or one with no provider at all. Portability is then conditional on that dependency being resolvable at the destination.

The predicates – which will be referred to as the *node-resident* predicates – are typically maintained by the owner of the node. External users are not able to make any changes to them, only the owner is allowed to do so. Any client connected with the node has access to these predicates in addition to the Web Prolog built-in predicates, and can pose queries over the web APIs formulated in terms of them. Suppose, for example, that the owner stores the following source in `mortal.pl`.

The contents of the file must be valid Web Prolog source code:

```
mortal(Who) :- human(Who).

human(socrates).
human(plato).
human(aristotle).
```

This is also what a user is presented with when steering an ordinary browser to the URI `http://n7.org`. This is an example of a shared Prolog database that is *self-contained*. In Chapter 4 we will demonstrate how this property makes it possible to achieve what we think of as *instant portability*.

However, a file might also look like this:

```
:- use_module(philosophers, [philosopher/1]).

mortal(Who) :- human(Who).

human(socrates).
human(Who) :- philosopher(Who).
```

This database is *dependent*: it draws `philosopher/1` from a separate `philosophers` module rather than defining it here. It functions at this node only if that module is present, and it is portable to another node only if `philosophers` is deployed there as well. (Had the file simply called `philosopher/1` without providing it anywhere, it would be dependent in a stronger sense – on a predicate with no provider at all.)

3.4.3 Deploying a node

To make this as concrete as possible, let us assume that the owner of a node wants to make the data in the file `mortal.pl` available to clients. The owner uses the following command to set up a Prolog node:

```
$ prolog-node --port=80 --src=mortal.pl --settings=settings.pl
```

As a result, a new node is created and the contents of the file is loaded into its shared Prolog database. The node is configured according to the settings given in the file `settings.pl`. After having performed all the usual moves necessary for the deployment of a web server, access to the node is offered through the set of API endpoints listed in Table 3.1:

API endpoint	Resource or functionality
<code>http://n7.org</code>	The shared Prolog database
<code>http://n7.org/shell</code>	A simple shell environment
<code>http://n7.org/call</code>	The stateless HTTP API
<code>http://n7.org/toplevel.spawn</code>	The semi-stateful HTTP API
<code>ws://n7.org/ws</code>	The stateful WebSocket API

Table 3.1 Proposal for how to link URIs with targets. Note that the stateful and semi-stateful APIs have additional endpoints.

By steering an ordinary browser to the URI `http://n7.org` an authorised user may want to just browse the program. Or perhaps query it by making a visit to `http://n7.org/shell`. Or perhaps the plan is to build an end-user application talking to the node over any of its web APIs.

Note also that because a Prolog node is also a web server, it may offer other HTTP endpoints as well, some of which may not be relevant to its core functionality.

Other ecosystems show that this strategy works. In the Prolog world, Jan Wielemaker’s *ClioPatria* demonstrated how a substantial Web application could be delivered as a ready-to-run system: users provided their data, made modest extensions, and obtained a full Semantic Web portal. In the Elixir world, Phoenix goes even further by generating complete, fault-tolerant applications with supervision, telemetry, routing, and deployment already in place. Developers simply add the behaviour that makes the system theirs. These examples illustrate how a language ecosystem can supply standardised, reproducible scaffolds that capture the best practices of a community.

3.4.4 Settings

As suggested by Figure 3.5, a Web Prolog node is governed by a set of *owner-controlled settings*. Only the node owner may change these settings; some of them may be *readable* by clients over HTTP, so that clients can discover the node’s advertised profile, public features, and operational limits. Collectively, the settings determine which features the node offers, who may access them, and the resource envelope within which actors and sessions are allowed to run.

It is useful to group settings into five categories:

1. **Identity and advertised profile.** The node advertises a *profile* to clients. This is primarily a discovery and compatibility mechanism: it tells clients what kind of service they can expect.
2. **Resource governance.** The node may enforce limits on injected code size, actor-local database size, cache bounds, and the maximum number of concurrent actors that client code is allowed to spawn. At the transport layer it may also enforce connection-related limits (for example, the maximum number of connections, and the maximum number of agents sharing a connection). These limits are part of the node's resource policy and are set by the node owner.
3. **Admission control.** The node may restrict which clients may connect and which goals may be executed, for instance via allow/deny lists and related authorisation rules.
4. **Session lifecycle and PTCP timeouts.** Two lifecycle limits are particularly important for PTCP sessions (cf. the PTCP state diagram in Figure 3.4). A *running time limit*, configured by the `time.limit` option, bounds each continuous period in which the session executes a goal in the working state `s2`. If the limit is exceeded, the node aborts the current goal and returns an error such as `time.limit.exceeded` to the caller. The session actor remains alive and returns to `s1`, ready to accept another call. Time spent waiting for a continuation in `s3` does not count towards the running time limit. An *idle limit*, configured by `idle.limit`, bounds how long the session may wait inactive in `s1` or `s3`. If this limit is exceeded, the node terminates the session actor normally. A client monitoring the session subsequently observes `down(Pid,Ref,true)`. These lifecycle limits are distinct from the `timeout` on a semi-stateful HTTP request: that timeout only bounds the wait for the next session event and does not abort the goal or terminate the session. A public client may request tighter lifecycle limits, but cannot weaken the ceilings set by the node owner.
5. **Bundle loading policy.** The node may require, or merely prefer, that loaded bundles are self-contained, meaning that they do not rely on unresolved external dependencies.

Note that readable over HTTP does not mean modifiable: the intention is that the node may expose a read-only subset of the settings that are relevant to interoperability and capacity planning, while keeping sensitive policy (notably admission rules) private.

Several of these settings are best understood as *policy gates* rather than mere tuning parameters. In particular, a node distinguishes between what the owner may do and what external clients may do. Clients are intended to program *session-local* concurrent computations, while the owner configures the long-lived surface of the node and installs durable services. This distinction will later be reflected in which concurrency-oriented predicates and options are admitted for client code, especially for lifetime control (for example whether a spawned actor may outlive its parent) and, in the distributed case, which combinations of remote placement and lifetime options are permitted.

3.4.5 Two transports – three web APIs

The primary responsibility of the node controller is to facilitate web API interactions. Nodes exposing the full interaction surface are designed to support two different transports – HTTP and WebSocket – and three distinct API types: a stateless HTTP API, a semi-stateful HTTP API, and a stateful WebSocket API. This architecture enhances flexibility by increasing the likelihood that at least one API will effectively serve a given end-user application.⁶

The HTTP transport

The *Hypertext Transfer Protocol* (HTTP) is a core communication protocol of the World Wide Web, enabling the exchange of resources like web pages and data between clients and servers. Operating as a stateless request–response protocol, it supports methods such as GET, used to retrieve resources, and POST, used to submit data to a server.

A client interacting with a node by making requests over an HTTP connection is illustrated in Figure 3.6.



Fig. 3.6 Interaction between a client and a node over an HTTP connection.

HTTP provides the request–response semantics independently of the representation carried in a request or response. Media types identify that representation, while HTTP content coding can compress it without changing its underlying format. HTTPS applies TLS to the HTTP connection, providing confidentiality and integrity in transit and normally authenticating the server; client authentication depends on the deployment. TLS neither chooses between JSON and Prolog text nor replaces application-level authorisation. The protocol appendix states the precise representation and transport contracts used by Web Prolog.

The WebSocket transport

In order to enable a client to control all aspects of a set of toplevels or other kinds of actors, a node that exposes actors to clients offers a WebSocket sub-protocol. Web-

⁶ Strictly speaking, HTTP and WebSockets both live in the application layer, not the transport layer. But in practice, even if TCP is the real transport protocol under the hood, it makes sense to think of them as alternative ways of transporting data between agents.

Socket is a real-time, low latency, bidirectional protocol for asynchronous communication between a client and a node. A WebSocket sub-protocol is an application-layer protocol, which operates on top of the WebSocket protocol, leveraging the transport services provided by TCP. The three phases in the lifecycle of a WebSocket connection are illustrated in Figure 3.7.

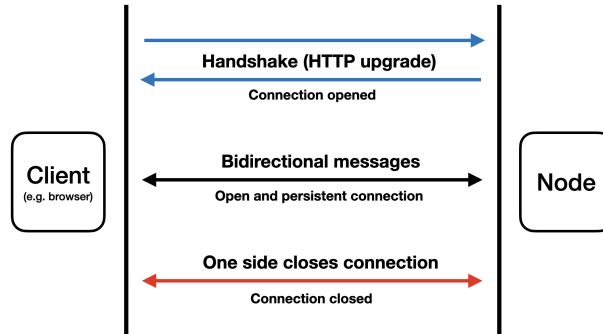


Fig. 3.7 The lifecycle of an interaction between a client and a node over a WebSocket connection.

In the first phase (in blue in the diagram) a WebSocket connection is initiated by the client via an HTTP connection which is then switched over to the WebSocket protocol. In the second phase (in black), the client and the node start talking to each other. This is where the sub-protocol comes in.

The Web Prolog WebSocket binding uses JSON objects as its protocol envelopes: client frames carry a `command` field and server frames a `type` field. Prolog goals and messages carried by fields defined as term text remain Prolog text inside those envelopes; JSON parsing alone does not turn them into Prolog terms. The HTTP bindings additionally allow a client to request either a JSON response envelope or a Prolog answer term. Compression, when enabled, belongs to the HTTP content-coding or WebSocket extension layer and does not change these representations. Exact rules and negotiation requirements are given in Appendix D.

WebSocket client libraries are available for the most common general programming languages. However, the most common use of WebSockets is in communication between a web browser and a server. In all the major web browsers, the WebSocket object provides a JavaScript API for creating and managing a WebSocket connection to a server, as well as for sending and receiving data on the connection.⁷ The role of JavaScript here is to create a new WebSocket, to define the necessary handlers for `onopen`, `onmessage`, `onerror` and `onclose` messages, and to call the methods `send` or `close` for sending messages or closing the connection.

In the next section, and with the help of a number of scenarios, the actual APIs will be demonstrated. For details, see Appendix B.

⁷ See <https://www.w3.org/TR/websockets/>

3.5 Node profiles

Prolog nodes differ in the language features and interaction surfaces they offer clients. We describe these differences using *node sub-profiles*, or simply *profiles*. A profile is a named, testable set of requirements: it states what clients may rely upon without prescribing how the node is implemented.

The four profiles form the cumulative hierarchy $\text{RELATION} \subset \text{ISOBASE} \subset \text{ISOTOPE} \subset \text{ACTOR}$, shown in Figure 3.8. Each profile includes the requirements of those inside it and adds a broader language or interaction surface.

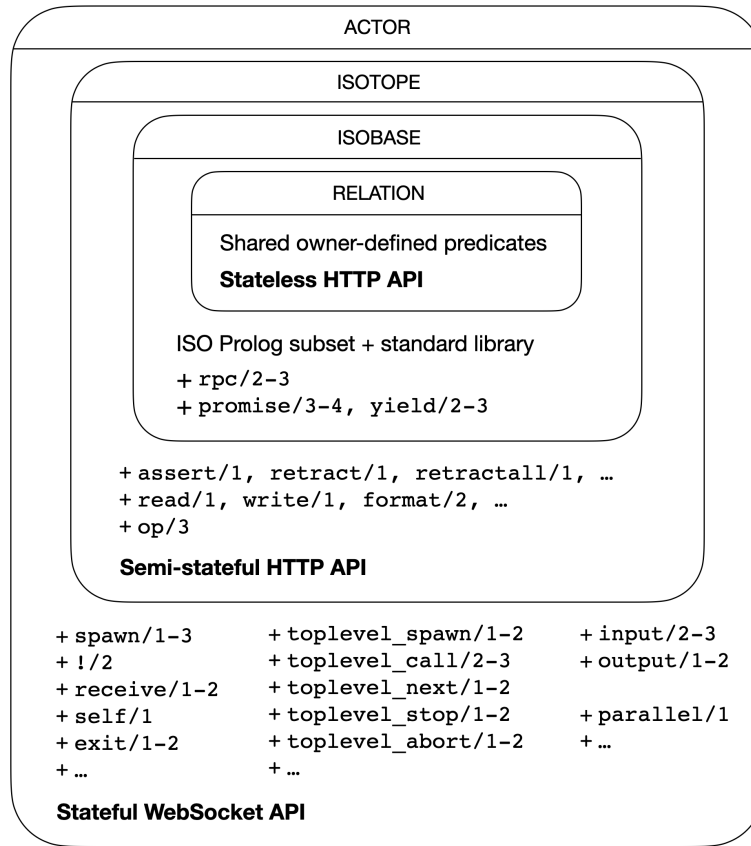


Fig. 3.8 The node-profile hierarchy. Each ring adds requirements to those inside it. On a broader-profile node, each API retains the language and interaction ceiling associated with its own ring.

RELATION requires the stateless HTTP API and a set of owner-advertised relations. A request is a query pattern over that declared schema, not a general Prolog

program. The node must return successive variable bindings, but need not contain a Prolog engine.

ISOBASE retains the stateless API and generalises its request language to goals in the ISOBASE fragment of Web Prolog. This includes owner-defined predicates, a substantial subset of ISO Prolog, standard libraries, definite clause grammars, and synchronous and split-phase RPC. Source may accompany a call, but belongs only to its temporary execution context.

ISOTOPE adds a semi-stateful HTTP API addressing a persistent toplevel actor. Definitions, dynamic database updates, operator declarations, and interactive I/O may therefore survive across requests within the session.

ACTOR adds Erlang-style concurrency and a bidirectional WebSocket API. Clients may spawn actors, exchange messages, monitor processes, and observe lifecycle events. An ACTOR implementation also supplies the owner-only service registration operations used to publish stable names.

Profiles and API ceilings

A node's highest profile is not exposed indiscriminately through every API. Each API retains its own ceiling. On an ACTOR node, `/call` operates at ISOBASE, `/toplevel_*` at ISOTOPE, and `/ws` at ACTOR. Actor operations such as `spawn/2` are consequently available through the WebSocket surface, not through the inherited HTTP APIs.

More generally, for an API offered by a node,

$$\text{effective profile} = \min(\text{node profile}, \text{API ceiling}).$$

Thus `/call` operates at RELATION on a RELATION node and at ISOBASE on every broader node. The lower surfaces remain stable as the node gains additional capabilities: a stateless client need not understand sessions or actors merely because its destination is an ACTOR node.

The hierarchy also locates the sequential and concurrent interaction partitions introduced in Chapter 1. RELATION, ISOBASE, and ISOTOPE expose sequential contracts. ACTOR additionally exposes concurrent contracts through WebSocket, while retaining the lower sequential surfaces. ISOTOPE shows why statefulness alone is not the boundary: its persistent session still presents one goal-and-answer conversation. ACTOR makes multiple processes, messages, and their relative progress observable.

RELATION is the boundary case. A client can call `wife/2` because the node advertises that relation, not because it accepts arbitrary Prolog goals. ISOBASE preserves this query behaviour while broadening the accepted language to general goals in its defined fragment. The inclusion concerns observable client behaviour, not the machinery used to produce answers.

Conformance

A conformant node must provide every API and language feature required by its advertised profile. An ACTOR node must therefore provide all three standard APIs, with each operating at its assigned ceiling; an ISOTOPE node must provide both HTTP APIs; and ISOBASE and RELATION must provide their respective stateless contracts.

A node may offer additional facilities, but clients cannot rely on them at that profile level. Conversely, omitting a mandatory facility invalidates the corresponding conformance claim. A node that provides the full actor language over WebSocket while omitting the HTTP APIs is actor-capable, but it is not ACTOR-conformant. It should advertise its actual capabilities rather than the standard profile.

Profiles are therefore portable conformance bundles, not an exhaustive taxonomy of possible nodes. A specialised or experimental node may conform to none of them. Its configured capability ceiling says what the implementation can potentially expose; an advertised profile is justified only when the complete required bundle is enabled.

Profiles give clients a stable contract and implementers a scalable commitment. Because conformance concerns observable interaction, implementations remain free to use different languages, engines, storage systems, and scheduling mechanisms. The profile also describes part of the node's security posture: RELATION exposes only declared query patterns, ISOBASE admits general goals within a controlled fragment, ISOTOPE adds persistent state and I/O, and ACTOR adds process creation, messaging, and lifecycle effects.

The following subsections illustrate RELATION and ISOBASE through the stateless HTTP API, ISOTOPE through the semi-stateful HTTP API, and ACTOR through a bidirectional WebSocket connection.

3.5.1 Browser talking to a RELATION node

As indicated by the profile hierarchy in Figure 3.8, the stateless HTTP API is the only web API that *must* be supported by a RELATION node. This does not distinguish it from nodes having a broader profile, as this is required by them too. What makes it stand out is that it is not required to offer access to any Web Prolog built-in predicates or control constructs *at all*. Even control constructs such as `,/2` and `;/2`, may not be available.

In the scenario depicted in Figure 3.9, a user equipped with an ordinary web browser is talking to the RELATION node at `http://n1.org`. The web-based terminal displays the remnants of a session during which the user executed two queries. The two question marks signify that neither the owner-defined resources nor the way answers are produced is known by the user.

The session began when the user loaded the Prolog shell at `http://n1.org/shell`. After displaying a welcome banner and a `?-` prompt, the shell used a JavaScript loop

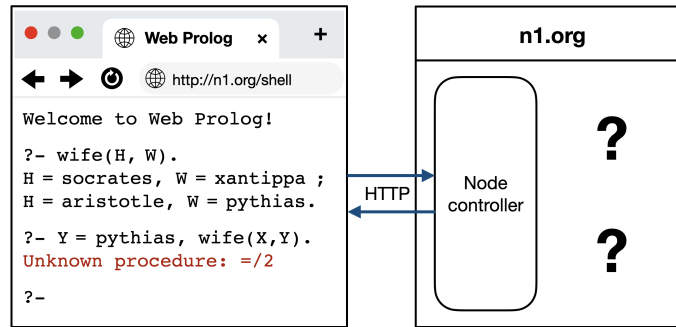


Fig. 3.9 A scenario involving a web browser in conversation over HTTP with a remote RELATION node. The two question marks signify that neither the owner-defined resources nor the way answers are produced is known by the user.

to read each query, translate it into an HTTP request, and display the resulting answer in the terminal.

Let us look at how the first solution to `?-wife(H,W)` was retrieved by using the stateless HTTP API. The URI pointing to this solution is:

```
http://n1.org/call?goal=wife(H,W)&limit=1&offset=0
```

Note the use of `limit` and `offset` parameters. Although here they are parameters rather than options, their defaults and the effect that they have on answer terms is the same as for the options accepted by `toplevel_call/3`.

URIs such as these describe a query and an answer slice, and can be bookmarked. Responses may also be cached by intermediaries when HTTP caching rules permit it. Stateless interaction makes each request self-contained; it does not by itself guarantee repeatable answers or shared-cache suitability. Section 4.2.4 separates these properties.

When the user entered the query `?-wife(H,W)`, (and assuming that a relative path was used instead of an absolute one) the following GET request was made to the node:

```
GET /call?goal=wife(H,W)&limit=1&offset=0 HTTP/1.1
Accept: application/json
```

The Accept header requested responses in JSON. (In this case the header could have been left out since JSON is the default.)⁸

The node responded with the following JSON structure, indicating that the query succeeded:

```
{ "type": "success",
  "data": [{ "H": "socrates", "W": "xantippa" }],
  "more": true }
```

⁸ A `format` parameter is also supported as this allows us to easily make a request from the address field of a web browser.

The `data` field contains the variable bindings and the value `true` of the `more` field signals that further solutions may be available. This triggered the printing of the variable bindings and served as a prompt for how to continue the exploration.

The user wanted to know if there were more solutions and typed a semicolon which led to the following request being made, looking exactly like the previous request, except that the `offset` parameter was incremented from 0 to 1:

```
GET /call?goal=wife(H,W)&limit=1&offset=1 HTTP/1.1
```

The node responded like so:

```
{ "type": "success",
  "data": [{"H":"aristotle","W":"pythias"}],
  "more": false }
```

The `more` field was now `false`, indicating that no further solutions existed, and after having printed the next set of bindings and a full stop, the shell again produced the `?-` prompt, inviting the user to enter another query.

At this stage of the interaction, it is reasonable for the user to expect that `wife/2` has the mode `wife(?, ?)` and that queries such as `?-wife(socrates, W)` or `?-wife(H, pythias)` should also succeed. A query such as `?-wife(plato, W)` should fail, however, and the response would be the following:

```
{ "type": "failure" }
```

Finally, an erroneous query, such as the attempt in the scenario to use the built-in operation `=/2`, which this `RELATION` surface does not expose, should raise an error, and show up as a response like so:

```
{ "type": "error",
  "data": "Unknown procedure: =/2" }
```

We can summarise this by saying that a `RELATION` node must offer clients a truly relational query interface over stateless HTTP. Any node failing to meet this expectation is not a conformant `RELATION` node. A `RELATION` node is required to respond to a client as if it were Prolog, thereby allowing the user to lazily back-track over possible solutions. Although a `RELATION` node may not truly perform backtracking, it must behave externally as though it does.

The lack of built-in predicates naturally makes a `RELATION` node very easy to implement. There are two quite different ways a `RELATION` node can come into existence. First, a node that already implements a richer profile can be turned into a `RELATION` node by installing a filter that inspects the goal parameter in each `/call` request and only allows it through if it matches a declared relation. Such a filter can be as simple as matching the submitted goal against a whitelist of terms – for instance, accepting only `employee(, , )` and `name(, )`, and rejecting everything else with an error. A node that is supposed to serve a particular application rather than act as a general compute server will often operate this way: it implements `ISOBASE` or more internally, but presents a `RELATION` interface to its clients.

Second, a **RELATION** node can be implemented from scratch in any programming language. (Recall that we do not require that a Prolog node is written in Prolog, only that it can talk Prolog.) At a minimum, an implementation must parse Prolog terms and produce variable bindings for queries against its data. A Python web service backed by a relational database, or a microservice wrapping a key-value store, can present itself as a conformant **RELATION** node as long as it implements the stateless HTTP API and responds with the expected JSON structure. This is the profile that makes the Prolog Web interoperable with the rest of the Web: any structured data endpoint can, in principle, participate in the ecosystem.

3.5.2 Browser talking to an ISOBASE node

For a node to adhere to the requirements of the ISOBASE profile, it *must* support the same stateless HTTP API as required by the **RELATION** profile. In our next scenario, illustrated in Figure 3.10, the user is running the same query as in the previous scenario. It means that we do not need to describe the details of requests and responses again. Instead, we focus on the language surface of the node and the contents of the shared Prolog database offered by it, and visible in the figure.

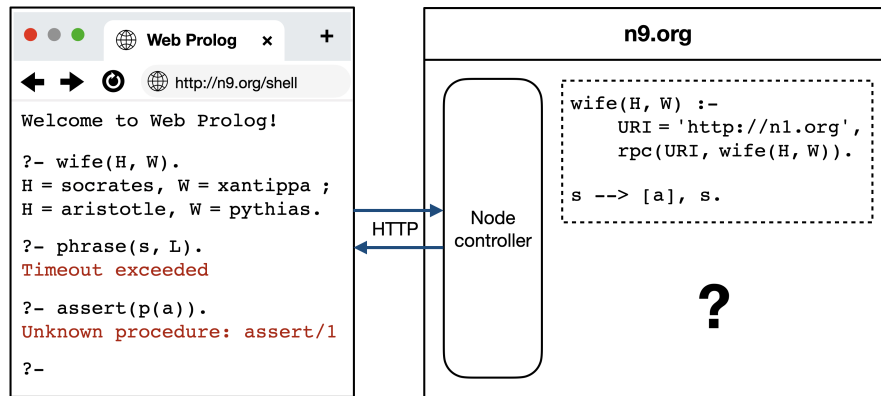


Fig. 3.10 Stateless interaction in a scenario involving a web browser and a remote node.

The main difference compared to the previous scenario is that an ISOBASE node *must* make a fairly large number of Web Prolog built-in predicates available, including a large subset of ISO Prolog predicates (recall the cloud of ISO Prolog predicates in Chapter 2), standard library predicates, definite clause grammars, as well as predicates for working with the Web.

An ISOBASE node must also support `rpc/2-3`, an easy-to-use predicate for making *nondeterministic remote procedure calls* from one node to another over

stateless HTTP, communicating in a way that is essentially *synchronous*. In the scenario in Figure 3.10, the URI `http://n1.org` in the first argument of the call to `rpc/2` shows that it accesses the relation `wife/2` made available by the node `n1.org` in the previous scenario.

In addition to the synchronous `rpc/2-3`, an ISOBASE node must also support *asynchronous* RPC using two built-in predicates: `promise/3-4` and `yield/2-3`. All three predicates can be implemented on top of the stateless HTTP API. They will be dealt with in more detail in Chapter 4.

Stateless interaction has one significant constraint: once a query has been submitted, the client has no way to abort it through the protocol. Because there is no control channel back to the running computation, ISOBASE nodes must enforce a timeout. This is what happened with the user's second query in the scenario shown in Figure 3.10: the node responded with the message `Timeout exceeded`. Unlike the previous scenario, this one allowed the user to inspect the source code defining the predicates made available by the node owner. This revealed that the node owner had apparently forgotten to add the DCG rule `s --> []`, which would have allowed the call to terminate.

Another limitation is that a truly stateless request-response protocol simply cannot support updates to the dynamic private database of an actor, nor I/O. It means that ISO predicates for modifying the dynamic database (for example, `assert/1` and `retract/1`), or I/O (for example, `read/1` and `write/1`), are *not* available. This is what happened with the user's third query, which was met with an `Unknown procedure` error message when trying to assert the clause `p(a)`. The predicate `assert/1` is simply not available to a client talking to an ISOBASE node.

3.5.3 Browser talking to an ISOTOPE node

An ISOTOPE node offers clients a shell backed by a toplevel actor. (Importantly, this does not mean that the internal `toplevel_*` predicates are directly exposed to programmers.) By using the process identifier of this actor as a session key in otherwise stateless HTTP requests, the node provides a semi-stateful API: each request is stateless in itself, but thanks to the pid they address the same long-lived toplevel actor.

The client's initial request can populate the actor's private database, which can then be updated dynamically throughout the session. Furthermore, although the client must initiate all interactions, the toplevel actor can not only respond to queries but can also prompt the client for further input or produce output at arbitrary moments. As long as the client keeps supplying the session identifier, this will work just fine.

In the scenario in Figure 3.11, the client established communication with a toplevel actor on an ISOTOPE node, which is a node that must satisfy all the requirements of ISOBASE, and then some.

The request below instructed the node controller to spawn a toplevel actor running as a session, and to populate its private database with the value of the `src_text`

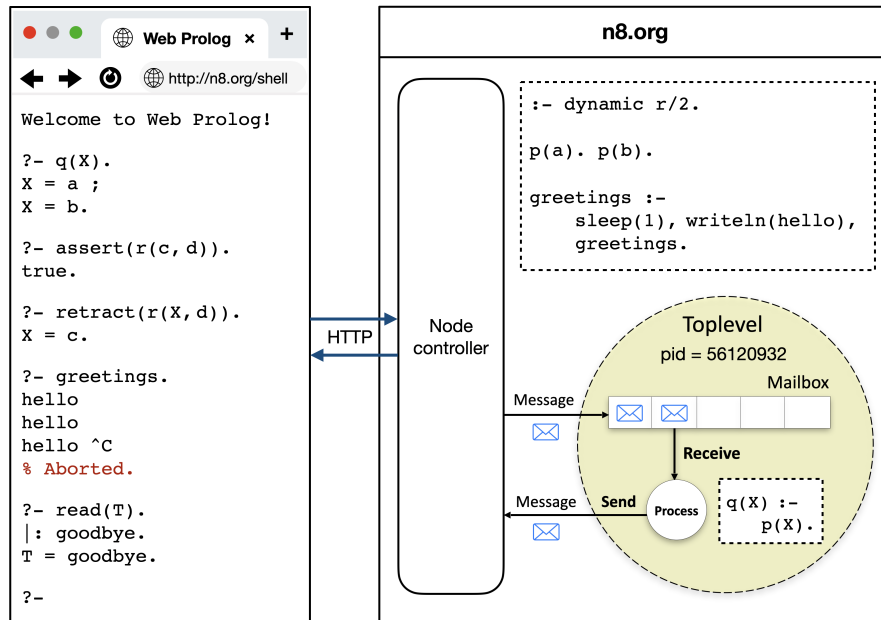


Fig. 3.11 A scenario involving a web browser in conversation over semi-stateful HTTP with a remote ISOTOPE node running a toplevel actor.

option. (Let us assume that the clause $q(X) :- p(X)$ was picked up from an editor window that is not shown in the diagram.)

```

POST /toplevel_spawn HTTP/1.1
Content-Type: application/json

```

```
{ "options": "[session(true), src_text('q(X):-p(X).')]" }
```

The node responded with a JSON structure containing the pid for the spawned toplevel actor:

```
{ "type": "spawned",
  "pid" : 56120932 }
```

This produced the prompt `?-` in the terminal, inviting the user to enter a query. The query `?-q(X)` caused the JavaScript terminal to make the following request to the node:

```

POST /toplevel_call HTTP/1.1
Content-Type: application/x-www-form-urlencoded

```

```
pid=56120932&goal=q(X)&limit=1
```

(Note that the `pid` must be sent along when querying the `toplevel`.) The request told the node controller to make a call to `toplevel_call/3` and to convert the resulting answer term into a JSON structure that encodes the first solution:

```
{ "type": "success",
  "pid" : 56120932,
  "data": [{"X":"a"}],
  "more": true }
```

Except for the `pid`, the JSON structure representing this solution is identical to the response when making a stateless request.

The value `true` of the `more` property signifies that more solutions may be available. When the user asked for another solution, the following request was made:

```
POST /toplevel_next HTTP/1.1
Content-Type: application/x-www-form-urlencoded
```

```
pid=56120932
```

This results in a similar message, except that the value of the `data` property becomes `[{"X":"b"}]` and the value of `more` becomes `false`.

Note that it took the *combination* of the clause that was loaded into the `toplevel`'s private Prolog database, and data in the shared database to answer the query `?-q(X)`.

Because the `toplevel` persists across requests, it makes sense to allow the client to update its private database using `assert/1` and `retract/1` during the session.

```
POST /toplevel_call HTTP/1.1
Content-Type: application/x-www-form-urlencoded
```

```
pid=56120932&goal=assert(r(c,d))
```

With this, the clause `r(c,d)` ended up in the private database of 56120932, but was removed again when the user decided to call `retract(r(X,d))`.

The user then called `greetings/0`, which calls `sleep(1)` and `writeln(hello)` in a tight loop. In simplified terms, the node controller redefines `writeln/1` as `writeln(Term) :- output(Term)`. This resulted in a series of output responses of the following form, returned to the client approximately once per second:

```
{ "type": "output",
  "pid" : 56120932,
  "data": "hello" }
```

For this to work over HTTP, a trick somewhat similar to so-called *long-polling* is employed: every time a JSON structure of type `output` reaches the client, it must issue a new request for a response, which can come as more output, or some other response. See Section 3.6.2 for more details.

The call to `greetings/0` will run forever unless the user does something about it. When the user hits Control-C, the nonterminating query is aborted, and the node responds with the following message:

```
{ "type": "abort",
  "pid" : 56120932 }
```

If the client does not exit the remote process, the node may terminate it after a set time with no activity. The pid will no longer be valid and any changes made to the workspace of the process will be lost.

3.5.4 Browser talking to an ACTOR node

Unlike an ISOTOPE node, an ACTOR node gives clients access to the full set of Erlang-style concurrency predicates and, in the bindings specified here, exposes them over a bidirectional WebSocket connection. This combination enables push-driven interaction, mixed-initiative communication, and concurrent actor programming – interaction patterns that the current HTTP bindings do not expose. The contract, rather than WebSocket as a transport, is what makes the surface concurrent. Figure 3.12 illustrates a scenario in which a user engages with an ACTOR node over such a connection.

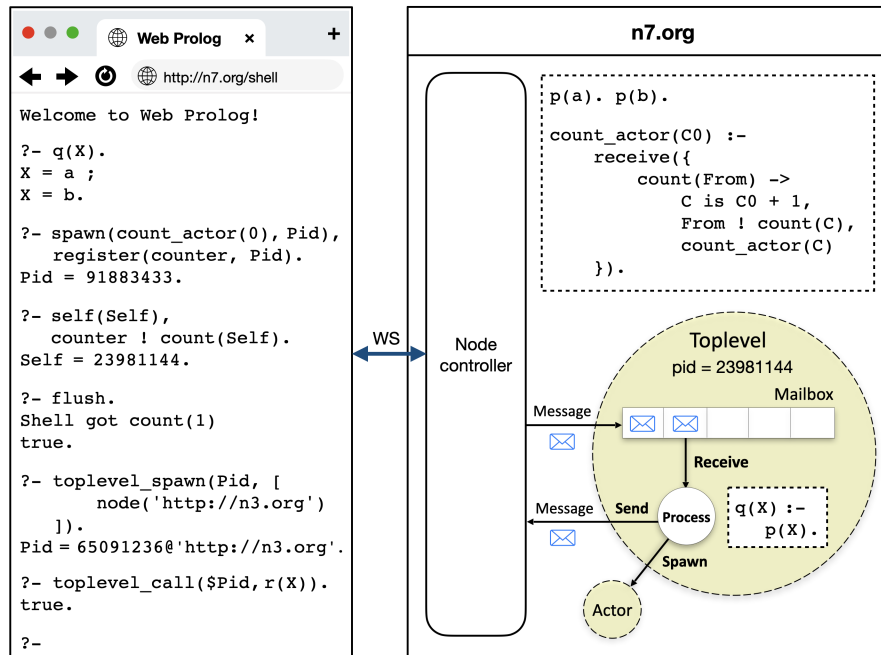


Fig. 3.12 A scenario involving a web browser in conversation with a remote node running a toplevel actor over a stateful WebSocket connection.

After having loaded the shell and established a WebSocket connection with the node, the JavaScript process sent the following command to the node's controller, instructing it to spawn a toplevel actor running a session with the clause $q(X) :- p(X)$ loaded into the actor's private database:

```
{ "command": "toplevel_spawn",
  "options": "[session(true), src_text('q(X):-p(X).')]" }
```

As requested by the client, the pid of the toplevel actor was returned to the client, not in the form of a HTTP response, but embedded in a JSON formatted message sent over the WebSocket connection:

```
{ "type": "spawned",
  "pid" : 23981144 }
```

From here on, the solving of the query $?-q(X)$ proceeds in a way very similar to how it worked in the ISOTOPE scenario, except that instead of the JavaScript shell making an HTTP GET request, it sent the following command message to the node over the WebSocket connection:

```
{ "command": "toplevel_call",
  "pid": 23981144,
  "goal": "q(X)",
  "options": "[limit(1)]" }
```

Knowing that the node's shared database offers a definition of `count_actor/1`, the user then decided to spawn a count server of the kind we saw in Chapter 2. Here is the message sent over the WebSocket connection:

```
{ "command": "toplevel_call",
  "pid": 23981144,
  "goal": "spawn(count_actor(0),Pid),register(counter,Pid)",
  "options": "[limit(1)]" }
```

This resulted in:

```
{ "type": "success",
  "pid" : 23981144,
  "data": [{"Pid":91883433}],
  "more": false }
```

While the toplevel actor continued to serve shell queries, the spawned count actor ran concurrently and received `count` messages independently of the shell interaction, illustrating the concurrent, actor based nature of the ACTOR profile. With access to the pid of the spawned count server, the user could send it a `count` message in order to try it out, and call `flush/0` to inspect the response.

Finally, the user called `toplevel_spawn/2` with the `node` option. This is yet another way to do network-transparent concurrency that will be dealt with in more detail in Chapter 4.

In this implementation, if the user leaves the terminal at `http://n7.org/shell`, closing the WebSocket causes the controller to terminate the session toplevel (via `exit/2`); before exiting, the toplevel terminates the spawned count actor. This yields a supervision-like hierarchy, discussed further in Chapter 4.

3.5.5 A terminal talking to a browser node

The scenario in the previous subsection placed the ACTOR node on a *remote* machine, reached across the network over a bidirectional WebSocket. The very same interaction can take place against a node that lives entirely within the user’s own browser tab. From the terminal nothing visible changes: the same Erlang-style concurrency predicates are typed, the same answers come back, and the same command and response messages pass to and from the toplevel actor. The session is, at the level of what the user types and reads, indistinguishable from the remote one of Figure 3.12; we therefore do not repeat the traffic here, but draw out what that indistinguishability is telling us.

A browser tab, in other words, can host a genuine ACTOR execution node – one that spawns actors, runs toplevels and serves queries with the same programming interface as a native node. It is not, however, a persistent inbound server: it shares the page’s lifetime and initiates its connections to remote nodes. Those are host-runtime properties, not differences in the actor language seen by the terminal.

The demonstrator realises this node using SWI-Prolog compiled to WebAssembly (WASM). WASM is what allows a full Prolog virtual machine to execute inside the tab, but it does not by itself provide an actor runtime. In particular, the current SWI-Prolog WASM build is single-threaded, and its engines provide cooperative interleaving rather than independently scheduled, isolated actors. The ACTOR interface therefore rests on an additional browser architecture around the WASM engine. That architecture is deliberately kept below the line of this scenario: what matters here is the node behaviour it exposes, not how the browser realises it.

The reason the browser and remote sessions can look alike is location transparency. A query is solved, an actor is spawned, a message is sent and flushed by issuing the same predicates against the same protocol; whether the node answering them sits on another machine or in the same tab changes only the *transport* those messages travel over, never their form. The protocol is the invariant; its carrier is an implementation detail.

Pids reinforce this transparency. Their canonical global form is the ordinary Prolog compound `Id@Node`; browser-local actors use the reserved node designator `localhost`. The form is part of the language contract, so programs may construct and decompose it. A shell may abbreviate a local pid when it prints it. Constructing a pid does not create an actor or establish access to one: the identifier must designate a live actor at the node, and the operation must be authorised. Given a valid pid, sending to it is the same operation regardless of where the actor came to rest.

This is the point the scenario is really drawn to make. The final entry in the terminal spawns a toplevel on a remote node with the `node` option, and the browser node honours it by opening a genuine network WebSocket – now with the browser node itself in the role of *client* to `n3.org`. A tab is therefore not merely a leaf client: it is a connection-initiating participant on the Prolog Web, reaching persistent nodes over the same WebSocket transport they use, even as it talks to its own terminal and local actors without a network at all. It does not accept inbound connections of its own. This is the location transparency the ACTOR profile was designed to provide, now seen to span the boundary between a tab and the wider network.

The one difference transparency cannot paper over is lifetime. A remote node outlives any single session – closing a connection ends a session while the node goes on serving others – which is precisely why a remote session must arrange the disciplined containment of Chapter 4, a toplevel reaping the actors it spawned before it exits. A browser node shares the fate of its page: its actors live and die with the tab, and there is no way to end the local node short of ending the page, which ends everything together. The containment a remote session must arrange has, here, nothing left to arrange.

The only fate not sealed by closing the tab is that of any *remote* actor the browser node spawned, whose survival depends on its own node’s lifecycle policy. We postpone the machinery that makes these semantics possible to Section E.4.1 in Appendix E: the limitations of the single-threaded WASM substrate, the use of a separate web worker and WASM instance for each actor, the JavaScript controller that owns routing and networking, the representation of messages across web worker and WebSocket boundaries, and the resulting isolation, resource and teardown trade-offs.

3.5.6 Social robot talking to an ACTOR node

In the previous scenarios, the shell and its toplevel actor served as the root of control: it was the first long-lived process from which other actors were spawned, monitored, and addressed. In general, however, the root of an application need not be a toplevel at all. Any actor can play that role, and a client may interact with such an actor directly, without routing interaction through a shell-style PTCP conversation.

Figure 3.13 sketches a deliberately minimal toy scenario in which a social robot interacts with a remote ACTOR node over the stateful WebSocket API.

The robot itself provides speech recognition and speech synthesis: it turns the user’s spoken input into text, sends that text to the node, receives a reply as text, and speaks it back to the user. In this toy setup the remote side does nothing clever – it simply echoes the input – so the interaction is a “parrot” loop. The purpose is not to propose a useful robot skill, but to isolate the transport-level point: unlike the shell scenarios, the client is not a terminal driving a toplevel, and yet it can still engage in persistent, bidirectional interaction with a remote node.

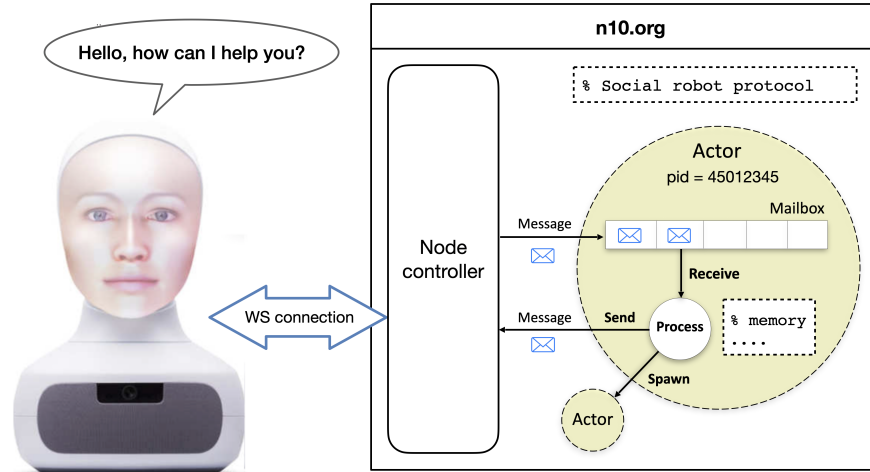


Fig. 3.13 Stateful interaction in a scenario involving a social robot and a remote ACTOR node.

At the protocol level, the scenario begins with the robot requesting that the node spawn an echo actor. The request is sent as a WebSocket message to the node controller:

```
{ "command": "spawn",
  "goal": "echo_actor",
  "options": "[monitor(true)]" }
```

The node responds by returning the pid of the spawned actor, which becomes the address the robot uses in subsequent messages. Subject to the node's policy, possession of this valid address gives the robot the capability to send to the actor.

```
{ "type": "spawned",
  "pid": 47110023 }
```

The WebSocket transport matters here because the interaction is push-driven and continuous. The robot establishes one connection and then exchanges messages in both directions as events occur: it sends new utterances as soon as they are recognised, and the node can return responses as soon as they are produced, without requiring a new HTTP request per response. This is the sense in which the interaction is stateful at the connection level: the channel remains open, latency is low, and messages can be delivered whenever either party chooses to speak.

In realistic conversational systems, of course, echoing recognised text is not enough. One typically needs additional components such as natural language understanding, a dialogue manager, grounding in a knowledge base, and response generation, as well as timing-sensitive coordination of speech, gaze, and facial expressions. We return to such agent-level concerns in Chapter 8; the present scenario

only serves to show that, with the ACTOR profile, the transport and interaction model already supports the kind of persistent, low-latency, bidirectional exchange that embodied clients require.

3.5.7 Could there be other profiles?

The hierarchy $\text{RELATION} \subset \text{ISOBASE} \subset \text{ISOTOPE} \subset \text{ACTOR}$ is useful because many applications need less than a full ACTOR node. Simpler profiles are easier to implement, govern, and deploy, while still supporting portable client behaviour.

At the same time, the hierarchy is not meant to be a closed taxonomy. The existing profiles mostly specify *what* features are exposed at the node interface. One can also imagine profiles that express *how* a node chooses to do it: restrictions, guarantees, or disciplines that cut across the profile hierarchy.

A PURE profile is one example: clients would be restricted to a declarative subset (excluding cut, non-logical arithmetic, and related extra-logical constructs), while still potentially allowing features such as source injection. Another example is a statechart discipline exposed as an optional ACTOR sub-profile for reactive conversational control. At the language level, Chapter 6 also identifies a *flat* subset of SXML (Section 6.8) for non-hierarchical machines. Unlike a node profile, this is a descriptive subset rather than a separately declared and validated offer.

3.6 Notes on node implementation

This chapter specifies node behaviour primarily in terms of externally observable contracts: profiles describe what a client may rely upon, and the web APIs describe the shapes of requests and responses. Implementation techniques are not part of the contract, and different Prolog systems – and different host platforms – will naturally realise the contracts in different ways. Nevertheless, a few recurring engineering issues are worth highlighting. They explain several design choices, clarify why multiple APIs are worth supporting, and help readers anticipate the practical costs associated with each profile.

3.6.1 Statelessness as a protocol property

The stateless HTTP API is *stateless for the client*: each request contains all information needed to interpret it, and the node is not required to remember a conversational session in order to respond. This does not prohibit the node from keeping internal caches. It merely means that caches are best-effort optimisations whose presence

must not be observable as a behavioural dependency. A node may evict cached state at any time and still remain correct and conformant.

As noted in Section 3.5.1, statelessness allows self-contained requests with meaningful URIs; safe repetition and response caching require their own conditions. A corresponding limitation is that a stateless request cannot be *externally* interrupted by the client once evaluation has begun. There is no control channel associated with the computation, and no session actor to signal. In practice, therefore, the stateless API must rely on node-enforced resource policy, most importantly running timeouts, to avoid nonterminating or overly expensive computations.

Paged solution retrieval

The `offset` and `limit` parameters make it possible to retrieve solutions in slices without prescribing how a node retains or recomputes search state. That choice is deliberately outside the node profile: clients observe answer slices, ordering, and continuation metadata, not the caching or execution strategy behind them. Chapter 4, especially Sections 4.2.2 and 4.2.4, develops restart-and-skip, save-and-resume, cursor semantics, and the consistency limits of paged retrieval in their distributed setting.

3.6.2 Semi-stateful HTTP sessions and response retrieval

The semi-stateful HTTP API differs from the stateless API in one crucial respect: each request targets a long-lived toplevel actor, identified by a `pid`, and the observable behaviour of the conversation depends on the evolving state of that actor. Nevertheless, each individual HTTP exchange is still an ordinary request–response transaction. In this sense, the protocol is *semi-stateful*: the transport remains stateless, but the addressed computation is not.

From an implementation perspective, this style aligns naturally with the architecture of SWI-Prolog’s `library(pengines)` and systems such as SWISH: the server maintains per-session engines, while the client repeatedly issues short requests that either advance the computation, or retrieve the next available response produced by that computation.

Output and prompts fit this model once they are treated as ordinary protocol events emitted by the session actor, rather than as exceptional side effects. Concretely, the toplevel actor can be understood as producing a sequence of *responses* (answers, failure, errors, output fragments, prompts, and acknowledgements) that the client retrieves over a series of short HTTP exchanges.

The statechart in Figure 3.14 extends and specialises the core PTCP in Figure 3.4 with the states and transitions needed for HTTP delivery.

The additional state `s4` shown in the figure belongs to the HTTP transport mapping rather than to PTCP itself. It records that the client has received a prompt and must

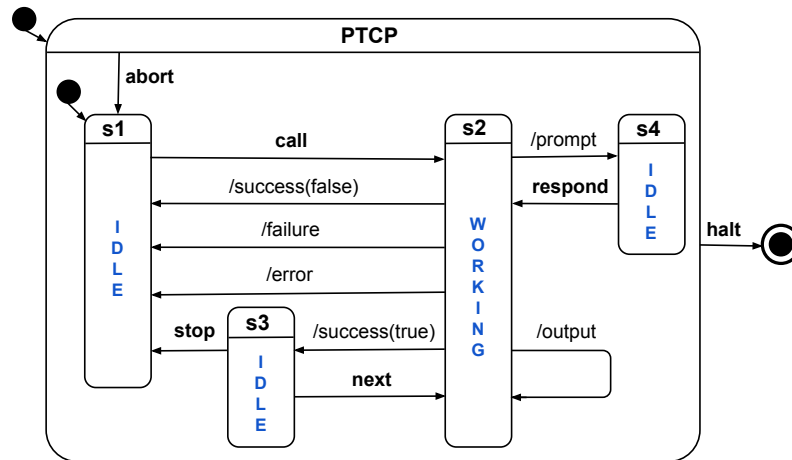


Fig. 3.14 Statechart specifying the PTCP for a successful conversation with a toplevel over HTTP.

send a **respond** request before the toplevel can resume work in **s2**. Output does not require another protocol state: an `/output` event is dequeued while the computation remains notionally in **s2**, and the client immediately calls the `/toplevel.poll` endpoint (the **poll** transition in the figure) for the next pending event. This local pull cycle repeats until the toplevel produces a terminal response (`/success`, `/failure`, `/error`).

Prompts require one additional step: when the next pending response is a prompt, the client retrieves it, presents it to the user, and sends a follow-up request that delivers the user's reply back to the session actor. From the protocol's perspective, this is still a request–response rhythm and no polling is needed here; the client is simply advancing the conversation by alternately retrieving pending events and, when prompted, supplying input. Ordering is preserved because responses are dequeued in the order they were produced. Transient transport failures may delay retrieval but do not change the logical meaning of the session.

3.6.3 Stateful WebSockets and push-driven interaction

The stateful WebSocket API exists for interactions that are naturally push-driven. Once a connection is established, the node can deliver answers, output fragments, prompts, and monitor events as soon as they are produced, without requiring a new client request for each response. This improves latency and avoids the request choreography of our long-polling trick. It also supports interleaved traffic from multiple actors over one connection, which is central to ACTOR-profile use cases.

Implementation complexity, however, shifts from request handling to connection lifecycle management. A robust node must maintain per-connection routing state (which pids belong to which client), preserve ordering within each actor stream, and enforce outbound back-pressure so that a slow client cannot cause unbounded buffering on the node. It must also define clear disconnect semantics: when a client vanishes, session actors and child actors are either terminated or detached according to policy, and monitored parties observe the corresponding `down/exit` events. In this sense, the WebSocket API is conceptually simple but operationally demanding: the protocol surface is small, while lifecycle and resource handling must be precise.

3.7 Summary

This chapter developed Prolog agents at two scales. Actor agents are independently scheduled processes with private state and message-based interfaces; node agents host programs, knowledge, and actors while presenting a stable, owner-governed boundary to clients. Toplevel actors connect the two scales through PTCP, turning the familiar query–answer conversation into an explicit protocol for incremental solutions, I/O, aborts, and termination.

The RELATION, ISOBASE, ISOTOPE, and ACTOR profiles make progressively broader language and interaction commitments observable. They let clients discover what a node promises without constraining how it is implemented. Across those profiles, the distinction between actor-private state and node-shared knowledge remains fundamental: private mutation does not become an implicit channel between processes.

The three web APIs establish how these agents can be reached through stateless requests, stateful sessions over HTTP, or bidirectional WebSocket connections. Their distributed operation, timing and failure boundaries, and security consequences belong to the next stage of the argument. Chapter 4 therefore crosses the node boundary, developing remote placement and spawning, cross-node messaging, discovery, code flow, and the open network in which node and actor agents interact.

Chapter 4

The Prolog Web

Imagine the Web wrapped in Prolog, running on top of a distributed architecture – a network of nodes exposing HTTP and WebSocket APIs, speaking web formats such as JSON. Think of it as a high-level Web, with Prolog agents capable of serving answers to queries – answers that follow from what the Web knows. Moreover, imagine it being programmable, allowing Web Prolog source code to flow in either direction: from client to node, or from node to client. This is what **the Prolog Web** is all about.

The Prolog Web – the elevator pitch

The previous chapters introduced Web Prolog as a language and Prolog agents as programmable entities. This chapter turns to the third element of the Prolog Trinity: the *Prolog Web* itself – a network of nodes hosting actors that communicate across node boundaries. In short, we extend Web Prolog into a language for distributed programming.

The model draws on Erlang’s *network-transparent concurrency*: if we know a node’s identity and we are authorised, we can spawn an actor there; if we know an actor’s pid or its registered name, we can send it a message, even across node boundaries. Unlike an Erlang cluster, however, the Prolog Web is as open as the Web itself. Here, *openness* means more than mere reachability. It means heterogeneous ownership, weak trust assumptions, communication over ordinary Web infrastructure, and an environment in which latency, disconnects, and partial failure are part of normal operation. The intended transparency is therefore selective: it unifies the programming surface for local and remote actors without pretending that their operational conditions are identical.¹ This follows the argument of Waldo et al. (1994) that distributed interaction must expose the consequences of latency, concurrency, and partial failure rather than paper them over. Openness brings risk, so the chapter pays close attention to safety: how lifetime discipline and capability separation prevent orphan processes and resource leaks.

¹ The categories “local” and “distributed” do not exhaust the possibilities. Two Prolog systems may run on the same physical computer while occupying separate processes and address spaces. Communication between them still crosses an address-space boundary, and may use the same Web Prolog protocols as communication between different computers, although it does not incur all the costs or failure modes of a physical network.

Openness also forces a basic separation of authority. On the Prolog Web there are two roles: *clients*, who temporarily borrow execution resources in order to run queries and coordinate short-lived concurrent work, and *node owners*, who are responsible for the node as an ongoing public resource. For clients, the default discipline is *containment-by-default*: spawned processes remain tied to the client session and are torn down when the session ends, preventing orphans and resource leaks. For owners, the corresponding power is *provisioning*: the ability to install node-resident predicates and start long-lived actor services meant to be shared and kept available. In practice such services are often managed under supervision, but the generic supervision patterns that support this belong to Chapter 5. This role split will recur in concrete form when we discuss remote placement and the various code-shipping mechanisms via the `src_*` predicates.

The chapter is organised in five sections, moving from the concurrent foundation through progressively higher layers of abstraction. The concurrent Prolog Web (Section 4.1) extends the actor model of Chapters 2 and 3 across node boundaries: remote spawning, cross-node messaging, orphan prevention, and the unified semantics that treats local and remote communication as instances of the same model. With the concurrent substrate in place, we can already speak concretely about node-resident services: long-lived actors started by the node owner, published under stable names, and invoked by clients over the Prolog Web. The reusable engineering patterns that structure such services – servers, supervisors, and other generic behaviours – are introduced in the next chapter and implemented in Chapter 5.

The sequential Prolog Web (Section 4.2) then introduces a synchronous, non-deterministic abstraction whose observable contract is independent of the concurrent layer. The central construct, `rpc/2-3`, can be reconstructed as a disciplined use of remote toplevel actors, which makes its incremental answer protocol easy to understand. In practice, however, the demonstrator implements it over stateless HTTP and may use cached execution contexts in save-and-resume mode. Actor machinery is one realisation of the sequential interface, not part of its contract.

The programmable Prolog Web (Section 4.3) shows how code and data flow between client and node: clients may inject task-specific predicates into their own execution contexts, while node owners install predicates and service actors that define what the node offers.

A discussion of timing and failure boundaries (Section 4.4) makes explicit where waiting becomes observable and how timeouts attach to different interaction models. Finally, a section on security (Section 4.5) asks what makes it safe to let strangers run code on a node, and answers in terms of three structural mechanisms – inexpressibility, invisibility, and containment – together with the deployment concerns that complete the picture. Chapter 9 later returns to the architectural and paradigmatic questions that these technical sections raise.

4.1 The concurrent Prolog Web

As introduced above, all the mechanisms described in the preceding chapters – spawning, sending, linking, monitoring, registering – work transparently across node boundaries. This section makes that claim concrete by introducing remote placement via the `node` option and showing how actor-style interaction spans nodes. The focus is on *where* an actor runs and how it exchanges messages across the network, but also on the safety and authority questions that network-transparent concurrency raises on an open Web. Programmability mechanisms such as `src_*` options are deliberately postponed to Section 4.3.

4.1.1 Actor talking to remote actor

For specifying the node on which we wish to create an actor process, `spawn/3` can be passed the `node` option:

```
?- spawn(echo_actor, Pid, [
      node('http://n8.org'),
      monitor(true)
    ]),
   register(echo_actor, Pid).
Pid = 99231321@'http://n8.org'.
?-
```

The call to `register/2` modifies the caller's ordinary registry. Such a binding may point to a pid on another node; it does not register or publish `echo_actor` at `n8.org`. Consequently the unqualified send below resolves the name locally and obtains the remote pid stored there.

If `n8.org` is a node conforming to the ACTOR profile, and if the node's shared database defines the predicate `echo_actor/0`, then we can have a conversation with our remote echo server like so:

```
?- self(Self),
   echo_actor ! echo(Self, hello).
Self = 71773120.
?- flush.
Shell got echo(hello)
true.
?- whereis(echo_actor, Pid),
   exit(Pid, die).
Pid = 99231321@'http://n8.org'.
?- receive({A -> true}).
A = down(99231321@'http://n8.org',99231321@'http://n8.org',die).
?-
```

4.1.2 Actors playing ping-pong while on different nodes

The actors playing ping-pong back in Chapter 2 kept their game inside one node. Passing the option `node('http://n2.org')` to one of the calls to `spawn/3` allows the game to be played between two nodes:

```
ping_pong :-
    spawn(pong, Pong_Pid, [
        node('http://n2.org')
    ]),
    spawn(ping(3, Pong_Pid)).
```

Figure 4.1 illustrates this scenario.

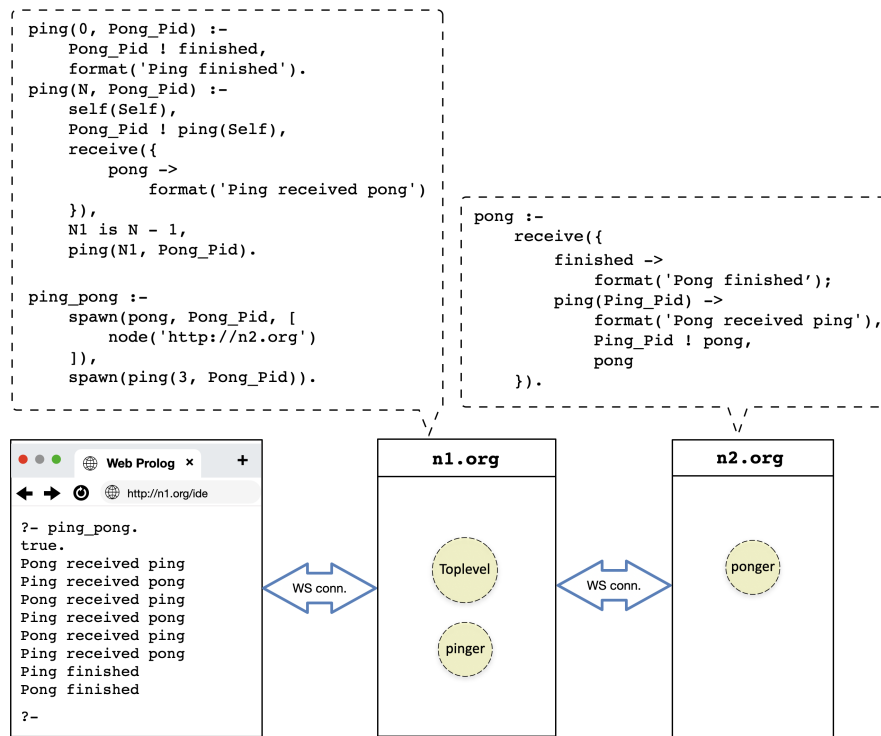


Fig. 4.1 Actors playing ping-pong while on different nodes. Output from both the local pinger and the remote ponger is returned to the terminal of the originating toplevel.

4.1.3 Terminal authority across node boundaries

Figure 4.1 illustrates a detail that is not obvious from the program text. The ponger runs on `n2.org`, yet its output appears in the browser terminal on `n1.org`, interleaved with output from the local pinger. This is not a broadcast of everything written on the remote node: the runtime routes terminal I/O back to the terminal from which the actor's process lineage originated.

The WebSocket arrows are transport channels, not terminal authority. A node-to-node connection carries events for many lineages and clients, so merely possessing it cannot confer permission to write to their terminals. The mechanism must identify both where terminal events return and which lineage is authorised to send them.

The more general rule is that terminal authority follows the process lineage, even when that lineage crosses a node boundary. A toplevel that is attached to a terminal has an I/O target. A locally spawned child inherits that target. If the child is spawned remotely, the spawning node instead supplies an opaque *terminal endpoint*, conceptually of the form

`'$io_endpoint'(Token)@HomeNode.`

The endpoint is an internal capability, not a public actor address. `HomeNode` says where terminal events must return, while the unguessable `Token` identifies the one terminal lineage to which they belong. The remote runtime installs the endpoint as the new actor's default I/O target. If that actor later spawns another actor – locally or on a third node – the same endpoint is inherited again. The rule is therefore transitive and does not depend on how many node boundaries the lineage crosses.

Box 4.1 — Erlang comparison: Group leaders and inherited I/O

Erlang provides the model for this arrangement. Every Erlang process has a *group leader*; I/O from the process group is channelled to that process, and a newly spawned process inherits the group leader of its spawner.^a This inheritance is not merely a convention of the local implementation. In Erlang's distribution protocol, a `SPAWN_REQUEST` carries both the pid of the spawning process and the pid of the group leader to assign to the new process.^b

Web Prolog borrows the semantic pattern, not Erlang's exact representation. An Erlang group leader is a pid inside a mutually trusting Erlang distribution domain. A Web Prolog terminal may instead be a browser connection on an open Web, so its inherited I/O destination is represented by a restricted capability that can be exercised only through authenticated node transport.

^a See the Erlang Run-Time System documentation for `group_leader/0`: <https://www.erlang.org/doc/apps/erts/erlang.html>.

^b See *Distribution Protocol*, under `SPAWN_REQUEST`: https://www.erlang.org/doc/apps/erts/erl_dist_protocol.html.

When the pinger calls `format/2`, its terminal event is sent to the inherited endpoint. The home node validates the endpoint and delivers the event to the browser connection that owns it. Input works in the opposite direction: a remote `input/2` sends a prompt through the same endpoint, and the home node grants that browser connection a one-use permission to return the answer to the prompting actor.

This I/O route is deliberately separate from ordinary actor communication. The remote actor's spawn reply, query answers, links, monitors, exit signals, and mailbox messages continue to use their normal targets. The terminal endpoint accepts only the small terminal protocol – output and prompts – and cannot be used as a general message-forwarding channel. Thus the mechanism does not make the display merely *look* right by copying every remote event to the browser; it gives precisely the remote descendants of a terminal lineage the authority needed to use that terminal.

The capability also has a lifecycle. A public client cannot invent or attach one, and an internal spawn that arrives without inherited terminal authority receives a non-forwarding I/O target rather than gaining authority by default. When the browser connection closes, the home node revokes its endpoints and any unanswered prompts. Consequently, the rule can be stated without mentioning locality:

An actor may perform terminal I/O when it possesses the terminal endpoint inherited from the toplevel lineage to which that terminal is attached.

The result is visible in Figure 4.1: the terminal displays output from both the pinger and the ponger. Their exact interleaving is not specified: they remain concurrent actors, and distribution adds further scheduling and network variation. What is guaranteed is not a global print order, but that each actor with the inherited capability can address the correct terminal.

4.1.4 The node option works for toplevels too!

Toplevels are actors, so it should come as no surprise that the `node` option can also be used to create a toplevel process on a remote node:

```
?- toplevel_spawn(Pid, [
    node('http://n1.org')
]).
Pid = 83110912@'http://n1.org'.
?-
```

The query `?-human(Who)` executes on the remote node `n1.org`, with answers returned to the local shell:

```
?- toplevel_call($Pid, human(Who), [
    template(Who),
    limit(1)
]).
```

```

true.
?- receive({Msg -> true}).
Msg = success(83110912@'http://n1.org',[plato],true).
?- toplevel_next($Pid).
true.
?- receive({Msg -> true}).
Msg = success(83110912@'http://n1.org',[aristotle],false).
?-

```

4.1.5 Node-resident actor processes

In addition to predicates in the shared database, the owner of a node may install *node-resident actor processes* – long-lived actors that are intended to be durable, shared, and continuously available. They are usually of a kind that we do not want unauthorised external clients to be able to create or destroy; doing this is the privilege of the node owner. This is also where supervision trees become most useful: crash recovery for node-resident services is part of the node owner’s responsibility, and Chapter 5 introduces the supervisor behaviour that makes this responsibility tractable.

A node-resident count actor

We begin with a deliberately small example. Assume that the owner of the node `n8.org` starts a count actor, detaches it from the owner’s own session, and publishes it under a stable name:

```

?- spawn(count_actor(0), Pid, [
    link(false)
]),
    register_service(counter, Pid).
Pid = 69220912.
?-

```

Two details distinguish this service from an ordinary client-created helper. The first is `link(false)`. Under the default lifetime discipline (that is, when `link` is set to `true`), the new actor would be tied to the life of the owner’s shell and torn down when that session ends – exactly the right behaviour for client-created helpers, and exactly the wrong one for a service meant to outlive any particular session. Detaching an actor from the hierarchy that created it is precisely the privilege that Section 4.5.3 reserves for node owners; here we see what the privilege is *for*.

The second detail is `register_service/2`, a publication operation distinct from ordinary `register/2`; the distinction is developed below. Publication does not itself alter an actor’s lifetime: `register_service/2` neither detaches nor supervises the

actor. Lifetime management and service publication are separate owner responsibilities, and the example performs both explicitly.

A published name can be addressed from other nodes by qualifying it with the node's URI, just as pids are: where `69220912@'http://n8.org'` is the globally qualified pid of a process at `n8.org`, the term `counter@'http://n8.org'` denotes whatever process is currently published under the name `counter` there. A client on some other node can therefore interact with the service without ever learning its pid:

```
?- self(Self).
Self = 51230945.
?- counter@'http://n8.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 1.
?- counter@'http://n8.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 3.
?-
```

The example is intentionally simple, but it already shows an important shift in perspective. This actor is not a client-local helper process created for one temporary task; it is a node-resident process with a stable public name, and it therefore behaves like a shared stateful resource. The fact that the second reply is 3 rather than 2 is not an accident of presentation, but evidence that the actor's state is shared across interactions: another client has incremented the counter in the meantime.

A node-resident publish-subscribe service

The same service perspective becomes even clearer in a publish-subscribe example. Consider the following actor:

```
pubsub_actor(Subscribers0) :-
  receive({
    publish(Msg) ->
      forall(member(Pid, Subscribers0), Pid ! msg(Msg)),
      pubsub_actor(Subscribers0);
    subscribe(Pid) ->
      pubsub_actor([Pid|Subscribers0]);
    unsubscribe(Pid) ->
      (
        select(Pid, Subscribers0, Subscribers)
      -> pubsub_actor(Subscribers)
      ;   pubsub_actor(Subscribers0)
      )
  }).
```

This actor maintains a list of subscribers. A `subscribe(Pid)` message adds a subscriber, an `unsubscribe(Pid)` message removes one, and a `publish(Msg)`

message broadcasts `Msg` to all currently subscribed processes. On its own, this is just another actor definition. It becomes a service when the owner starts it as a durable, detached, node-resident process and publishes it:

```
?- spawn(pubsub_actor([]), Pid, [
    link(false)
]),
    register_service(pubsub_service, Pid).
```

A client can then subscribe to the service and wait for messages from it:

```
?- self(Self),
    pubsub_service@'http://n8.org' ! subscribe(Self),
    repeat,
    writeln('Waiting for a message ...'),
    receive({
        msg(Message) ->
            format('Received: ~p~n', [Message]),
            fail
    }).
```

Waiting for a message ...

Received: hello

Waiting for a message ...

Note the quiet work that network transparency is doing here: the subscriber list held by the actor at `n8.org` contains *remote* pids, and `Pid ! msg(Msg)` delivers across node boundaries with the same syntax as local delivery. The message `hello` was received because some client published it to the same named actor:

```
?- pubsub_service@'http://n8.org' ! publish(hello).
true.
?-
```

This example illustrates a useful contrast with the counter. The counter service exposes shared *state*: what one client observes depends on what other clients have already done. The publish-subscribe service instead exposes shared *coordination*: clients interact indirectly through a common mediating process. In both cases the essential pattern is the same – a node-resident actor with a stable public name behaves as a service.

The implementation shown here is intentionally minimal. In practice, one would monitor subscribers so that dead ones are pruned from the list, place the service itself under supervision, possibly persist subscriptions in a more durable store, and perhaps attach access control or topic structure.

Naming, discovery, and service publication

The examples use `register_service/2` because private naming and service publication have different scopes. Clients may name their own actors with `register/2` and inspect those names with `whereis/2`. Only the node owner may add or remove names in the public service namespace, using `register_service/2` and `unregister_service/1`.

A public service is addressed as `Name@Node`. This address is resolved only through the public registry of `Node`; local names are not considered, and the service's pid is not revealed.

The same `@` syntax is used for global pids. Both `Id@Node` and `Name@Node` are ordinary terms that programs may construct and decompose. The difference lies in their resolution: `Id@Node` directly identifies a node-assigned actor, whereas `Name@Node` is looked up in the node's public registry. Constructing either address does not establish that its target is live or accessible.

Remote service resolution is deliberately non-disclosing. A send to an absent service and a send to one the sender is not authorised to use have the same observable result: the send succeeds without delivery. Applications that need confirmation must use an acknowledgement protocol.

A published name separates a stable service identifier from the actor that implements it (van Steen and Tanenbaum, 2023). The actor may be restarted or replaced without changing `Name@Node`. Because the node URI remains part of the address, moving the service elsewhere requires a directory, forwarding entry, stable front-end URI, or similar mechanism.

Publication makes a service addressable; it does not advertise it. A directory may advertise selected nodes, profiles, predicates, and services. The following example shows one such directory.

The Discovery Hub

The proof-of-concept deployment includes a Discovery Hub at <https://n0.elfenbenstornet.se/discovery-hub>, itself implemented as the Prolog node `N0`. The hub maintains a directory of the registered demonstrator nodes and periodically probes each one. It publishes the resulting information – including current reachability, advertised profile, authorisation mode, database portability, and provided predicates and services – as owner-defined relations over the stateless `/call` API. The filters in the browser interface are translated into ordinary Prolog goals and evaluated against these relations.

Node `N0` advertises the `RELATION` profile. Internally, however, it uses an actor to perform the periodic probes. That actor and the corresponding actor operations are not exposed to clients. The hub therefore provides a particularly compact illustration of an important profile distinction: a profile describes a node's observable contract, not everything used to implement it. It also demonstrates that useful ecosystem

infrastructure, including node discovery, can be constructed from ordinary relations and the stateless Web Prolog API.

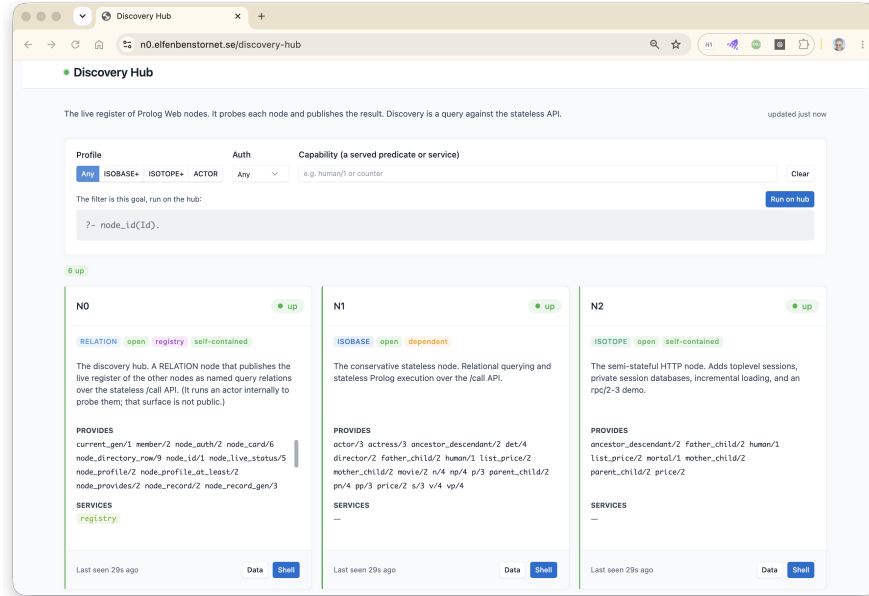


Fig. 4.2 The Discovery Hub at node N0. The browser interface queries a live directory of demonstrator nodes by profile, authorisation mode, and advertised predicate or service. Although N0 uses an actor internally to probe the registered nodes, it exposes the directory through a stateless RELATION interface.

The hub should not be understood as a mandatory central authority for the Prolog Web. It is one possible directory implementation: other hubs may coexist, participation may be voluntary, and applications may use owner-maintained peer information or other discovery mechanisms instead.

Service discovery is an owner-curated form of advertisement, distinct from publication itself. A call to `register_service/2` makes a name addressable in the public service namespace; it does not expose a client-queryable enumeration of that namespace. A node may instead advertise selected services through owner-maintained metadata, while leaving other published services unadvertised.

This separation is a design invariant. If clients could enumerate the service registry or all running processes, discovery would collapse into introspection and name discipline would be lost. Ordinary client naming operations must likewise neither expose nor remove published services. The security implications are developed in Section 4.5.2.

In the Trinity, such an advertisement can be expressed as an ordinary relation in the node's shared database. For example:

```
service(counter, meta(actor, protocol(count_v1))).
```

```
service(pubsub_service, meta(actor, protocol(pubsub_v1))).
```

The relation is maintained by the owner and may advertise all published services or only a selected subset. The owner is responsible for keeping its entries consistent with the services actually published. Richer metadata – versioning, authorisation requirements, or documentation URIs – can be represented by extending the second argument.

A client discovers advertised services with an ordinary goal-oriented query:

```
?- rpc('http://n8.org', service(Name, Meta)).
Name = counter,
Meta = meta(actor, protocol(count_v1)) ;
Name = pubsub_service,
Meta = meta(actor, protocol(pubsub_v1)).
?-
```

Discovery therefore remains declarative and compositional: clients can enumerate advertised services, filter by protocol, or combine discovery with other constraints. Because `service/2` is owner-controlled metadata rather than an enumeration of the service registry, the node reveals only what it has deliberately chosen to advertise and may apply ordinary authorisation policy to the query.

4.1.6 Uniform messaging across node boundaries

The preceding examples show that the same primitives – spawning, sending, linking, monitoring, registering – work whether the target process is local or remote. They share a programming interface and an abstract delivery model, although distribution remains visible where it affects latency, failure, authority, or ordering. This design follows the unified semantics proposed by Svensson et al. (2010): a node controller mediates side-effecting operations so that the core actor semantics do not depend on a particular transport. Chapter 9 returns to this distribution-first stance; here we define its messaging model.

Two-step messaging model

Following the *ether* abstraction of Svensson et al. (2010), messaging has two steps. A *send* creates an in-flight record in a virtual system queue; a later *delivery* appends the message to the receiver’s mailbox. The ether is a semantic device, not a required implementation structure. A local runtime may deliver immediately, while a remote runtime may carry the record over a network transport (Figure 4.3).

The in-flight step may be realised by a local queue, a direct WebSocket connection, an HTTP request, or a relay. In each case, ordering is defined over *delivery* into mailboxes. A *binding epoch* is a continuous period during which a name denotes the



Fig. 4.3 The two-step messaging model. Sending creates an in-flight record; delivery appends the message to the receiver's mailbox, where `receive/1-2` may consume it.

same actor, and a *routing epoch* is a continuous use of one ordered controller route. A local runtime remains in one routing epoch; a remote reconnect or reroute starts another. The message-ordering guarantees on which portable programs may rely are summarized below:²

Situation	Guarantee
Same sender to the same pid within one routing epoch	Send order is preserved.
Same sender through the same name within one binding and routing epoch	Send order is preserved.
One send by pid and one by name, even if both resolve to the same actor	No ordering guarantee.
Sends separated by name rebinding	No guarantee across binding epochs.
Sends separated by reconnect or rerouting	No guarantee across routing epochs.
Sends from different senders	No ordering guarantee.

The table states the minimum ordering guarantees on which portable programs may rely. An implementation may provide stronger guarantees.

4.1.7 Signals and distributed lifetime control

Signals and unwind

The two-step model applies to messages. Runtime control uses a separate *signal channel*. Signals bypass the mailbox and selective receive, so they can affect an actor while it is evaluating a goal or waiting for a particular message (Figure 4.4).

There are two kinds of signal. A *termination signal* ends the actor; it is used by `exit/2`, link propagation, PTCP `halt`, and controller policies such as resource limits or loss of an owning connection. An *interruption signal* abandons a delimited computation without ending the process. PTCP `abort` uses this second form to abandon the current goal while preserving the toplevel actor and its session.

The contract does not prescribe exact interruption points. A signal takes effect at the earliest safe point provided by the host runtime; an atomic built-in need not be interrupted part-way through. Supported cleanup handlers run as the computation unwinds.

² For messages sent with the `delay` option, send order is determined when the messages are emitted, not when the delayed sends are scheduled.

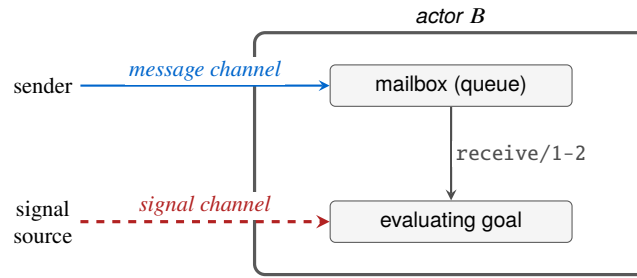


Fig. 4.4 Messages enter the mailbox and are selected by `receive/1-2`; runtime signals bypass the mailbox.

Termination unwinds the whole actor and cannot be refused by application code. Web Prolog therefore has no counterpart of Erlang's `trap_exit`: allowing a child to turn its parent's termination into an ordinary message would weaken containment. The first committed terminal outcome supplies the exit reason; later termination attempts have no further effect. Monitors report the completed lifecycle event as an ordinary `down/3` message.

PTCP abort has a narrower boundary. It passes through client-reachable exception handlers and is consumed by the privileged PTCP boundary, which resumes the persistent toplevel in state `s1`. A catch-all `catch/3` therefore cannot defeat an abort or enter its recovery branch, although cleanup still runs. Halt instead follows the termination path and destroys the actor. Figure 4.5 contrasts the two unwind extents; Section 5.3.4 gives the implementation details.

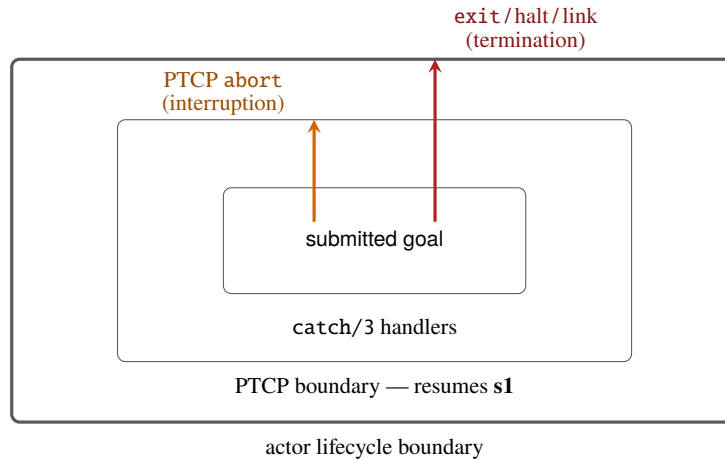


Fig. 4.5 Abort crosses client exception handlers but stops at the PTCP boundary; termination crosses the actor lifecycle boundary. Cleanup runs during either unwind.

The message and signal channels have no shared ordering guarantee. Sending a “prepare to stop” message before a termination signal therefore does not ensure that the actor processes the message first. A protocol that requires preparation must wait for an acknowledgement before terminating the actor.

Local result delivery and termination

The local cleanup algorithms in Chapter 5 require a more specific guarantee. For a worker and its caller in the same runtime, every result enqueued by an immediate send directly to the caller’s pid before the worker terminates must precede the enqueue of that worker’s termination notification to the caller. This applies both to an existing monitor and to a new monitor installed during or after termination. It is a guarantee about mailbox insertion, not about when the caller selects a message. Delayed sends, forwarded messages, and remote transports are outside its scope.

The demonstrator supplies this guarantee through synchronous local queue insertion: `thread_send_message/2` places the result in the caller’s queue before the send returns.³ If termination interrupts a send, the result is either already enqueued or will not be sent by that worker; there is no independent local delivery task that can insert it later. The worker’s exit handler delivers monitor notifications after its goal has stopped. Monitor installation, notification delivery, and removal of the pid mapping share a lifecycle mutex. A new monitor therefore either joins the notifications delivered at termination, or finds the retired pid and produces `down(Pid,Ref,no_process)`. In the latter case, any result sent by that former worker was already enqueued before retirement. Here `no_process` confirms absence; it does not recover the worker’s termination reason.

The same serialization makes local `demonitor(Ref,[flush])` sufficient to retire an old monitor: its notification is either already enqueued and flushed, or its delivery is prevented. It cannot arrive after the flush. These local guarantees do not order an incoming exit signal against an ordinary message, and do not strengthen the remote channel contract.

Remote links and monitors

For a local link, one runtime can maintain the lifetime relation directly. A remote link is maintained by the controller at the child-hosting node. When the controller learns that the parent has terminated, it propagates a reason-bearing signal. The demonstrator also scopes remotely hosted actors to the authenticated WebSocket that authorised their creation; when the connection reports close or error, the child-hosting controller terminates the connection-owned subtree.

The logical link and its transport carrier are not identical. Several links may share one connection, and one parent may die while the connection remains healthy,

³ https://www.swi-prolog.org/pldoc/doc_for?object=thread_send_message/2.

so explicit per-parent propagation is still necessary. Connection cleanup is only a backstop after transport loss has been detected. The demonstrator has no heartbeat or lease, so a silent partition need not be detected within a bounded time.

Remote monitoring observes but does not control a lifecycle. If its connection to a target closes, the runtime reports `down(Pid,Ref,connection_closed)`. This records loss of observation, not target termination; the monitor is consumed and is not restored on reconnection.

4.2 The sequential Prolog Web

The sequential and concurrent partitions classify *observable interaction contracts*. A *sequential interaction contract* exposes computation as the submission of goals followed by an ordered session of answers, committed effects, prompts, or errors. Understanding the interaction does not require the client to reason about independently addressable processes, mailbox selection, or interleavings. Ordinary single-threaded Prolog semantics, including backtracking and committed effects, supplies the governing explanation even when transport boundaries intervene. This account governs the content and solution order of completed calls; a split-phase construct may leave the wall-clock availability of a pending outcome open, including by reporting a timeout, without exposing the remote computation as an addressable process or mailbox interaction.

A *concurrent interaction contract* exposes independently progressing processes and their interactions as part of the programming model. Process identity, asynchronous delivery, mailbox selection, lifecycle events, and possible interleavings may all affect what a client or peer actor observes. Such behaviour cannot in general be reduced to one Prolog goal session.

Contracts are primary; other classifications are derivative. A construct or API belongs to the sequential partition when the interactions it exposes have the sequential form, even if actors implement it underneath. A program may compose constructs from both partitions, and a node may expose API surfaces from both. Transport, statefulness, synchrony, purity, and implementation technique are separate axes: they may correlate with the partitions in common deployments, but none defines the boundary.

One useful actor-based reconstruction of `rpc/2-3` has the following layering:

1. Actor processes and asynchronous messaging.
2. Toplevel actors, which add goal evaluation and answer-stream management to the basic actor model.
3. `rpc/2-3` over a remote toplevel – the conceptual implementation that makes the synchronous, nondeterministic interface explicit.

Each step thins the interface and hides more of the underlying machinery. The stateless HTTP implementation is an alternative realisation of the same observable `rpc/2-3` contract, not a fourth layer that every conforming node must contain. It

may use cached suspended searches, restart evaluation for each slice, or use another strategy whose internal state remains invisible to the client.

Because the concurrent layer has already been established, the actor-based reconstruction can expose how suspended search and incremental answer delivery fit together. Not every distributed program or node needs explicit concurrency, however: often a caller simply wants to ask another node to solve a goal and wait for answers. We can therefore present `rpc/2-3` and its companions `promise/3-4` and `yield/2-3` entirely through their sequential contracts. The latter pair presents a split-phase rather than a blocking interface, but the caller still observes a referenced remote-call outcome rather than an addressable remote actor or mailbox protocol. We return to alternative realisations only where they affect transport, failure, or resource ownership.

4.2.1 Nondeterministic remote procedure calls

In Web Prolog, `rpc/2-3` is a high-level meta-predicate for making *nondeterministic remote procedure calls*. It allows a process running on a node N_1 to call and attempt to solve a goal in the Prolog context of another node N_2 , taking advantage of the programs and data offered by N_2 as if they were local to N_1 . Such calls are synchronous: the caller blocks while waiting for answers. In return, `rpc/2-3` is remarkably easy to use for distributing programs across two or more nodes. It can therefore be seen as a construct for distributed programming that trades concurrency for ease of programming.

An actor may call `rpc/2-3` from a relational receive guard, thereby extending the logical environment described in Section 2.3.4 across a node boundary. Doing so also extends the guard's operational risks: because the call is synchronous, network latency delays the mailbox scan, and transport failure is subject to the guard's error-as-non-match rule unless the program translates it into an explicit logical result.

A Web Prolog client performs a remote call by providing a URI identifying the node and a goal to be solved in that node's context. The optional third argument is a list of options. Here is a simple example:

```
?- rpc('http://n1.org', human(Who)).
Who = plato ;
Who = aristotle.
?-
```

Note that solutions are returned on demand and in the familiar one-solution-at-a-time fashion: variable bindings appear as instantiations of variables occurring in the goal.

Using `rpc/3` with the `limit` option

The `rpc/3` predicate supports a `limit` option, inherited from the underlying `toplevel_call/3` predicate. Passing `limit(K)` instructs the remote node to compute at most K solutions per network roundtrip. The default is `infinite`, meaning all solutions are fetched in a single exchange. Here is what happens if we override that default with `limit(1)`:

```
?- rpc('http://n1.org', human(Who), [
    limit(1)
]).
Who = plato ;
Who = aristotle.
?-
```

From the caller's perspective, the result is identical. What changed is invisible: instead of retrieving both solutions in a single roundtrip, the call now required *two* – one per solution. In this case, the option adds cost without benefit. Later, we shall see contexts in which limiting the number of solutions per roundtrip matters in a genuinely positive way.

In addition to the `limit` option, `rpc/3` inherits the `src_*` options from `spawn/3` and the `timeout` and `once` options from the `toplevel` mechanism. Examples using `src_*` options will be given in Section 4.3.1. See Appendix B for details about the other options.

The semantics of `rpc/2-3`

For intuition it is useful to note that a call to `rpc(URI, Goal)` is semantically equivalent to the following pattern, which makes explicit that solutions arrive as instantiations of a copy that is then unified back into the original goal:

```
copy_term(Goal, Copy),
call(Copy),           % executed on node at URI
Goal = Copy.
```

The key implication is that no variables are shared across nodes: the remote execution operates on a renamed-apart copy, and only the induced substitution is returned to the caller via unification with the original goal.

This three-line characterisation is intended as a *logical* specification of `rpc/2-3`: it defines what answers the caller may observe, not how those answers are obtained. As a specification it is precise and complete for the intended domain. Web Prolog restricts remote goals to pure Herbrand terms – attributed variables are not supported, and the serialisation protocol ensures that bindings returned to the caller are always meaningful in the caller's context, ruling out node-local references such as stream handles or database pointers. What the characterisation deliberately abstracts over

is the operational machinery required to deliver those answers across a network: the creation of a remote process, the protocol that sustains nondeterminism across multiple solutions, and the handling of timeouts, exceptions, and partial failure. Chapter 5 gives two implementation-oriented accounts of that machinery.

A notable property of `rpc/2-3` is that it preserves the logical purity of the goal it calls: if the goal is pure, then the answers returned by `rpc/2-3` are exactly the logical consequences of the remote database – the network introduces no additional bindings, side effects, or observable state changes.⁴ This holds at the level of answer substitutions, assuming successful communication; in practice, timeouts, connection failures, and resource limits may prevent all answers from being delivered, but they do not corrupt the answers that are delivered. Later, we return to this observation when discussing the idea of a *pure* Prolog Web.

4.2.2 Two operational realisations

The logical specification deliberately leaves transport and process management open. One way to realise it is to spawn a toplevel actor on the target node, submit the goal using PTCP, and collect answer messages. A `success/3` message contains a slice of solutions. The caller enumerates that slice locally and requests another only when backtracking exhausts it. In this way, a sequence of deterministic protocol exchanges is presented as ordinary Prolog nondeterminism.

The demonstrator instead implements `rpc/2-3` over the stateless HTTP API. The same slice-at-a-time structure applies, but each exchange is a self-contained request to the remote node. No client-addressed actor is required, and the remote node need not expose its programmable ACTOR layer. This makes the sequential abstraction available between ISOBASE nodes and allows it to pass through ordinary HTTP infrastructure.

Statelessness at the protocol boundary does not require the implementation to discard all execution state after every response. A node may cache a suspended toplevel so that a later request can resume the search instead of repeating it. Such cached state is a disposable server-side optimisation rather than a session owned and addressed by the client. Reuse and eviction must preserve the promised observations: for pure, stably enumerable goals, recomputation can recover the same answers; mutable dependencies require the weaker paging contract or an explicitly stronger consistency mechanism discussed below.

The two realisations therefore expose the same answers while assigning transport, failure detection, and resource ownership differently. Their implementations are developed in Sections 5.4.2–5.4.3, after the toplevel behaviour on which the pedagogical actor reconstruction depends.

⁴ This is a property it shares with the `call/N` family of Prolog predicates.

4.2.3 Browser talking to a node talking to a node

In the scenario depicted in Figure 4.6, a user of a Prolog terminal running in a browser interacts with a remote node on the Prolog Web, which in turn interacts with another node.

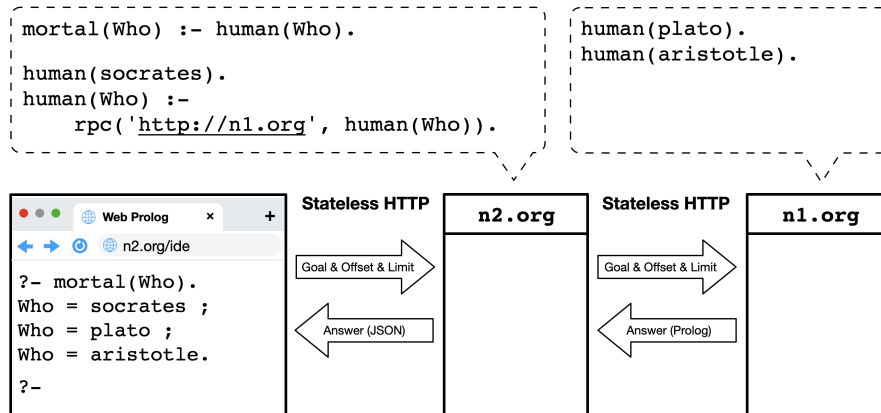


Fig. 4.6 Stateless interaction in a scenario involving a web browser and two remote nodes.

The solutions shown in the terminal follow from the *combination* of two programs: the code on the upper left in Figure 4.6 belongs to n2.org, and it contains a URI pointing to the code offered by n1.org. The browser-to-node interaction as well as the node-to-node interaction take place over HTTP and are stateless in the sense that each request can be understood and processed without relying on server-side session state.

To retrieve the first solution to `?-mortal(Who)`, the browser issues a request to n2.org asking for exactly one solution:

```
GET http://n2.org/call?goal=mortal(Who)&limit=1
```

Since the user requests solutions one-by-one, the response contains a single binding together with a flag indicating that more solutions exist:

```
{ "type": "success",
  "data": [{"Who":"socrates"}],
  "more": true }
```

When the user types a semicolon, the browser requests the next solution explicitly:

```
GET http://n2.org/call?goal=mortal(Who)&offset=1&limit=1
```


In this example, producing that next solution requires `n2.org` to consult `n1.org`. Concretely, `n2.org` evaluates `mortal/1` and, as part of doing so, triggers an `rpc/2-3` call for `human(Who)` against `n1.org`. Operationally, this involves an HTTP request to `n1.org` such as:

```
GET http://n1.org/call?goal=human(Who)&format=prolog
```

Because of the default values of the corresponding slicing parameters, the response in this case contains a complete slice with all solutions to `human(Who)` currently returned in one roundtrip. Since the URI specified `format=prolog`, this was the result:

```
success([human(plato),human(aristotle)], false)
```

At this point the implementation of `rpc/2-3` on `n2.org` must convert the returned slice into the intended nondeterministic behaviour. The usual mechanism is to keep the slice locally and enumerate it using `member/2`. This yields solutions to `human(Who)` on backtracking without further network communication, until the slice is exhausted.

Since the browser client still wants only one solution at a time, `n2.org` returns only the first derived solution, `Who=plato`, to the browser:

```
{ "type": "success",
  "data": [{"Who":"plato"}],
  "more": true }
```

When the user again types a semicolon, the browser requests another solution to `mortal(Who)`. In this particular run, no additional request to `n1.org` is needed: `n2.org` can simply backtrack within the local `member/2` enumeration of the slice already obtained from `n1.org`. The browser request therefore looks like:

```
GET http://n2.org/call?goal=mortal(Who)&offset=2&limit=1
```

and the response contains the next binding, `Who=aristotle`, now with `more:false`:

```
{ "type": "success",
  "data": [{"Who":"aristotle"}],
  "more": false }
```

This completes the description of what happens “under the hood” in the scenario of Figure 4.6: the browser drives one-solution-at-a-time enumeration through stateless HTTP requests to `n2.org`, while `n2.org` uses `rpc/2-3` to consult `n1.org` and reconstructs nondeterminism locally from slices of answers.

Large result sets and the `limit` option

If `n1.org` had millions of clauses of `human/1`, returning a huge slice in one response would be undesirable. In that case `n2.org` can pass a smaller `limit` to `rpc/2-3` to retrieve answers in manageable batches:

```
human(Who) :-
    rpc('http://n1.org', human(Who), [
        limit(1000)
    ]).
```

This avoids swamping the caller with data. A goal with very many solutions will be delivered incrementally over multiple roundtrips, but since `rpc/2-3` is non-deterministic, the call remains logically complete provided the caller keeps asking for more.

As this scenario involves only two nodes, the Prolog Web here is tiny. Later we argue that the same principles can, in principle, scale to arbitrarily many nodes. Key is the combination of statelessness (for robustness and simplicity) with an execution model that avoids shipping excessive data and avoids recomputing answers unnecessarily.

4.2.4 Backtracking as pagination

We now step back from implementation to draw out a correspondence that the rest of the book will build on.

The `offset` and `limit` parameters in the stateless HTTP API may look like a polite concession to web conventions – a way to make Prolog’s multiple solutions fit a request–response protocol. But the connection is deeper than that. Prolog backtracking and HTTP pagination instantiate the same abstract idea: incremental enumeration under external control.

A paginated endpoint does not “return the result set”; it exposes a procedure for generating results and a protocol for consuming them in slices. The server produces a page, then stops. A later request asks for the next page, which amounts to resuming enumeration at a recorded position. Prolog behaves in the same general way. A goal may have many solutions; the `toplevel` presents the next one (or the next batch) and then waits. Internally, evaluation proceeds by creating choice points, and enumeration can be resumed from a well-defined position. In an interactive `toplevel`, that position is represented by retained search state, and the user’s semicolon is simply the “resume” signal.

Offset surface, cursor semantics

Web API pagination comes in two principal forms. In *offset pagination*, the client says “skip N , return the next k ”; the server must, at least logically, regenerate the first N results and discard them. In *cursor pagination*, the server hands the client an opaque token that encodes where to resume; the client returns the token to request the next page, and the server picks up where it left off. The token is meaningless to the client – it is a handle to internal state, not a numeric position in a pre-existing result set.

Prolog backtracking is structurally cursor-based. A choice point is opaque retained state that captures exactly where to resume the search: variable bindings, unexplored branches, the position in the clause index. The semicolon hands it back and says “continue.” No re-enumeration from the top occurs; no solutions are skipped and discarded.

On the wire, however, the stateless HTTP API speaks `offset` and `limit` because that is idiomatic REST:

```
GET /call?goal=hotel(Name,paris,Rating)&offset=20&limit=10
```

This returns solutions 21–30. But the implementation need not treat the offset as an instruction to re-enumerate from the beginning and discard. A node can cache a suspended toplevel process keyed by goal and offset, so that a request for page $k+1$ resumes the process from its suspended choice point rather than replaying the search. The offset still denotes a position in the answer sequence at the public interface. Internally, the node may use that position, together with the goal and other relevant request context, to locate a retained continuation. The client acquires no guarantee that this particular process survives.

Consistency and the update view

The distinction between true offset re-execution and cached-process resumption is not merely one of cost; it has semantic consequences. True offset pagination – re-executing the goal fresh for each page – gives the client a sliding window over whatever the database contains at the moment of each request. If facts are asserted or retracted between pages, solutions may be duplicated, skipped, or reordered. Cursor-based resumption instead preserves one in-flight search: earlier branches are not replayed, and active predicate calls retain their remaining choice points.

The Prolog *logical update view*, which the ISO Prolog standard mandates, strengthens this preservation but must not be confused with a database transaction (Lindholm and O’Keefe, 1987). Each invocation of a dynamic predicate sees the clauses that existed when that invocation began, regardless of later assertions or retractions. A retained continuation therefore preserves that clause view across answer slices for predicate invocations already in progress. For a simple scan whose answers come from one such invocation, this can yield a stable sequence even while the owner updates the predicate.

It does *not*, however, freeze the actor’s entire logical environment at the start of the top-level query. An arbitrary computation may first invoke another dynamic predicate only after an answer slice has been returned; that later invocation may see updates made in the meantime. Retaining the process is therefore not, by itself, transaction-wide snapshot isolation. A node that advertises a point-in-time snapshot for general goals needs an additional mechanism, such as an immutable versioned knowledge base or a transaction whose snapshot spans the whole enumeration.

Retention introduces a separate condition. A node is free to evict cached computations – under memory pressure, after a timeout, or for any other reason (Sec-

tion 3.6.1). If a page request arrives after eviction, the node must fall back to restart-and-skip against the current database. Any search-state continuity and per-invocation logical update views belonging to the old computation are then lost. For a scan that races mutation, recomputation may consequently produce duplicates, gaps, or a stale completion flag.

The portable contract is therefore the weaker one stated in Chapter 3 (Section 3.3.3): paging by position is well defined for *stably enumerable* goals, while drift caused by owner-side mutation is possible and is not, in the base design, reported to the client. A stronger point-in-time guarantee must be advertised separately, preserve one snapshot across every slice, and either survive cache eviction or report that continuation as invalid rather than silently re-deriving it. Generation identifiers and opaque resume tokens are possible mechanisms, as discussed in Chapter 11.

The logical update view thus supplies useful per-invocation stability nearly for free. A retained process additionally avoids replaying completed search. Neither property alone is a general database snapshot; keeping these three notions separate is essential to the paging contract.

Three points on a spectrum

The relationship between Prolog’s backtracking mechanism and HTTP pagination can be understood through three retrieval strategies. The first two implement the same stateless API; the third exposes a session explicitly.

1. **Stateless offset, naïve implementation.** Each request re-executes the goal against the current database, skips N solutions, and returns the next batch. This is true offset pagination: cheap on server memory, but it repeats search and is vulnerable to inconsistency if the database changes between pages.
2. **Stateless offset, cached processes.** The wire protocol still says `offset/limit`, but the server caches a suspended toplevel process keyed by goal and offset. Offset on the surface, cursor under the hood. This preserves the suspended search and the logical update views of predicate invocations already in progress *for as long as the cached process is retained*; it is not a transaction-wide snapshot for an arbitrary goal. If the process is evicted mid-scan, a subsequent page falls back to case (1) against the current database.
3. **Semi-stateful API with process identifiers.** The server hands back an explicit process identifier, and the client uses it to request continuation. There is no offset at all. This is cursor pagination without disguise, and the closest structural match to interactive backtracking: the pid *is* the choice-point handle, and “next” *is* the semicolon.

The three styles differ in what the client’s handle refers to. In (1) and (2), the request identifies an answer slice, and the node may reconstruct the computation needed to obtain it. Only the implementation of (2) uses a cached search tree, with its choice points and bindings. In (3), the pid is an explicit handle to a full process: search tree, variable bindings, *and the process’s private database*, including any facts it has

asserted locally or code loaded via `src_*` options. Successive “next” requests do not just advance enumeration; they resume a process that may have asserted local facts, updated counters, or built up intermediate structures as part of its computation. This is closer to a database session with a temporary table than to a mere cursor.

Source loading and temporary state

The stateless API may accept `src_*` options without promising persistent client-owned state. A request such as

```
GET /call?goal=solve(X)&src_uri=http://n5.org/planner&limit=5
```

causes the node to spin up a process that loads the specified source into its private database before executing the goal. That process now carries local state – the loaded clauses – not just a position in a search tree over the shared database.

If the node caches this process for subsequent page requests, the cached handle points to a process with a populated private database: the same kind of internal execution context that a pid may refer to in the semi-stateful API. The lifetime contract differs, however: a stateless client cannot require that private context to survive between requests, while a session client explicitly addresses an existing actor.

If the node does *not* cache – naïve re-execution – then each page request reloads the code, re-executes from scratch, and skips. This is not merely inefficient for enumeration; it re-runs the entire setup for every page. And if the loaded source has changed between requests, different pages may be enumerating over different programs entirely.

Source loading therefore adds execution state without necessarily adding session state to the public protocol. Reusing the loaded context is an invisible optimisation only where reloading and recomputing preserve the promised behaviour. Stable source and dependencies make this possible for suitable goals. A client that needs a particular private database or continuation to persist should use the semi-stateful API and handle the possible expiry or termination of that session.

Caching and the Web

Three properties must be distinguished: HTTP idempotence, HTTP response caching, and continuity of a Prolog computation.

HTTP idempotence concerns the intended effect on the server of repeating a request. It does not require identical response content. A read-only GET query remains idempotent even if an owner updates the database between requests and the answers change. Conversely, merely putting an effectful goal in a GET URI does not make its execution safe to repeat; the endpoint must uphold the method’s semantics

for the computations it admits. These properties do not follow from the choice to retain or replay search.⁵

HTTP caches store response representations, not suspended Prolog engines. Storage and reuse depend on the request method, response status, cache directives, authorisation, and freshness or validation. Representation selection matters too: JSON and Prolog responses must be distinguished in the cache key, for example through `Vary: Accept` when selected by that header. A bookmarkable URI alone establishes none of these conditions.⁶ A stateless response can therefore be eligible for HTTP caching whether the node obtained it by recomputation or by resuming an internal cached process.

Execution caching has a different obligation: losing a cached process must not break the stateless contract. For pure goals over stable code and dependencies, replay and resumption can produce the same answer sequence. A request can then go to another backend that supplies the same execution context, even if that backend must recompute. The absence of session affinity does not, by itself, make different backends' code, data, or policies agree. Where dependencies change, the base paging contract admits the drift described above; retention may preserve more continuity, but clients cannot rely on that accidental benefit surviving eviction.

An explicitly addressed session makes a stronger commitment to a particular execution context. Its `next` operation advances that computation; repeating the operation can advance it again. Routing must reach the identified session, and losing it must be reported rather than silently substituting a fresh execution. That distinction concerns the operation and its lifecycle, not a general rule that every response involving a session is uncacheable.

For a small example, let a node expose this dynamic predicate in clause order:

```
:- dynamic p/1.
p(a). p(b). p(c).
```

A call of `p(X)` with `limit=1` first returns `a`. The next stateless request uses `offset=1`; a session client instead requests `next`. Table 4.1 compares evaluation at the node, assuming each request reaches it. If the owner retracts `p(a)` between pages, fresh evaluation enumerates `b, c`; offset 1 now skips `b` and returns `c`.

After the first answer	Fresh request	Cached search	Session <code>next</code>
Code and data unchanged	<code>b</code>	<code>b</code>	<code>b</code>
Owner retracts <code>p(a)</code>	<code>c</code>	<code>b</code> if retained; <code>c</code> if replayed	<code>b</code>

Table 4.1 The same query under stable data, changed data, and retained execution.

In the second row, the retained invocation of `p/1` keeps its logical update view. The read-only offset request has not become non-idempotent merely because its answer

⁵ RFC 9110, Sections 9.2.1–9.2.2, <https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2>.

⁶ RFC 9111, Sections 3–4, <https://www.rfc-editor.org/rfc/rfc9111.html#section-3>.

changed. Nor does the stable session result establish a snapshot for arbitrary multi-predicate goals: later predicate invocations may still observe newer data. An HTTP cache could separately reuse an earlier response where its caching rules allow that; such reuse neither resumes a computation nor supplies a snapshot across different page URIs.

4.2.5 Programming with `rpc/2-3`

Because `rpc/2-3` is a simple, synchronous, nondeterministic interface, it is easy to build small programming schemes on top of it. We sketch a few patterns here.

The `parallel/1` predicate

The first and simplest pattern is a meta-predicate `parallel/1` that takes a list of goals and attempts to solve them in parallel rather than one after another. We state its intended behaviour as a small set of requirements. A call to `parallel(Goals)` should

1. block until the outcome is determined, and no longer than necessary,
2. succeed, with variable bindings, if all goals succeed,
3. fail as quickly as possible if any goal fails, and
4. rethrow an error thrown by a goal when that error is the first abnormal outcome observed, also as quickly as possible.

In the normal case, running a list of goals in parallel should produce the result of running them sequentially, but potentially complete sooner. For this to work properly, the goals must be independent: they must not communicate through variables shared between list elements, global state, or other side channels. From the caller's perspective, `parallel/1` is a Prolog-style combinator whose implementation happens to orchestrate actors.

Crucially, `parallel/1` does not expose a new process or messaging protocol. Provided the listed goals are independent and deterministic, a call whose goals all succeed returns the same variable bindings as sequential conjunction; if one or more goals fail and none throws, the call fails; and if exactly one goal throws while the others succeed, that error is propagated.

This equivalence has two limits. First, `parallel/1` is semi-deterministic, committing to the first solution of each goal rather than enumerating combinations on backtracking. Second, the requirements above do not impose sequential left-to-right precedence when several workers have different abnormal outcomes. Web Prolog adopts a *first-observed* rule: the first failure or exception notification accepted for this invocation determines its abnormal outcome, and the remaining workers are cancelled. A failure makes the call fail; an exception is rethrown. In a multi-fault race, the selected outcome is therefore scheduling-dependent and need not be the one

a left-to-right conjunction would encounter first. Subject to that abnormal-outcome qualification, `parallel/1` remains a synchronous Prolog predicate surface whose concurrent worker machinery is hidden from ordinary successful use.

Making RPC calls in parallel

Two RPC calls in sequence take the running time of the first call plus the running time of the second:

```
?- rpc('http://n1.org', foo(X)),
    rpc('http://n2.org', bar(Y)).
```

If the caller node conforms to the ACTOR profile, we may combine RPC calls with `parallel/1`:

```
?- parallel([
    rpc('http://n1.org', foo(X)),
    rpc('http://n2.org', bar(Y))
]).
```

The time spent waiting is then approximately the maximum of the two response times. Note, however, that this is still a synchronous pattern: `parallel/1` blocks until both calls have returned. It therefore differs from actor-style interaction, where sends are asynchronous by default.

In its basic form, `parallel/1` commits to the first solution of each goal; it does not explore combinations by backtracking, so if `foo/1` or `bar/1` are nondeterministic, the above returns only one solution from each call. To aggregate lists of solutions, use `findall/3`:

```
?- parallel([
    rpc('http://n1.org', findall(X, foo(X), Xs)),
    rpc('http://n2.org', findall(Y, bar(Y), Ys))
]),
    append(Xs, Ys, Solutions).
```

Remote modules

Below we sketch an implementation of *remote modules*.⁷

```
:- op(700, xfx, as).
```

```
use_remote_module(URI, ImportList, Options) :-
```

⁷ What we here call *remote modules* is similar in spirit to Ciao Prolog's *active modules*. The operator `as` is local notation for this sketch, declared by `:- op(700, xfx, as).`; it is not part of Web Prolog's syntax.


```

maplist(import(URI, Options), ImportList).

import(URI, Options, Functor1/Arity as Functor2) :- !,
    functor(Head1, Functor1, Arity),
    Head1 =.. [Functor1|Args],
    Head2 =.. [Functor2|Args],
    assertz((Head2 :- rpc(URI, Head1, Options))).
import(URI, Options, Functor/Arity) :-
    functor(Head, Functor, Arity),
    assertz((Head :- rpc(URI, Head, Options))).

```

Thanks to `src_*` predicates, an import directive can define local wrappers even for predicates not directly exported by the remote node, provided they can be expressed in terms of predicates that are exported.

Global goals

Our second example takes a step towards even more network transparency and is inspired by (Loke, 2006). The idea is to support *global goals* of the form `?- *Goal`, where a local peer database maps goal patterns to peers and options:

```

peer(foo(_), 'http://n1.org', [limit(10)]).
peer(bar(_), 'http://n2.org', [timeout(1),limit(100)]).
peer(bar(_), 'http://n3.org', [timeout(3),limit(100)]).
peer(baz(_), 'http://n3.org', [timeout(1),limit(50)]).

```

Given such a database,⁸ `* /1` can be implemented like so:

```

:- op(100, fx, *).

* Goal :-
    peer(Goal, Peer, Options),
    catch(rpc(Peer, Goal, Options), time_limit_exceeded, false).

```

A registry-based variation can be obtained by retrieving `peer/3` information remotely:

```

* Goal :-
    rpc('http://registry.pl', peer(Goal, Peer, Options)),
    catch(rpc(Peer, Goal, Options), time_limit_exceeded, false).

```

⁸ We make no assumptions on how the peer database is updated.

4.2.6 Promise and yield over stateless HTTP

The `rpc/2-3` predicate is synchronous: the caller waits until the remote node returns an answer. For many applications this is fine, but sometimes a client wants to initiate a remote computation, continue with other work, and collect the result later. The `promise/3-4` and `yield/2-3` predicates support this split-phase pattern. They do not use the semi-stateful HTTP API: `promise/3-4` initiates an asynchronous call through the stateless HTTP API, while `yield/2-3` waits on a local reference for the answer to that call.

A call to `promise/3-4` sends a query to a remote node but returns immediately, handing back a reference that identifies the pending computation:

```
?- promise('http://n1.org', human(Who), Ref).
Ref = 19303011.
?-
```

The remote node begins evaluating `human(Who)`. Meanwhile, the caller is free to do other work. When it's ready for the answer, it calls `yield/2-3` with the reference:

```
?- yield($Ref, Answer).
Answer = success([human(plato), human(aristotle)], false).
?-
```

If the answer has already arrived, `yield/2-3` returns immediately. If not, it waits. When the answer arrives, the caller's runtime retains it under both the reference and the identity of the issuing process; `yield/2-3` must therefore be called by the process that issued `promise/3-4`. An implementation may use that process's mailbox, but a mailbox is not part of the sequential promise/yield contract.

The familiar options carry over: `template/1` controls which bindings are returned, while `offset/1` and `limit/1` control slicing. By default, `promise/3-4` requests all solutions; with `limit(K)`, the answer contains at most K solutions, allowing large result sets to be fetched in batches. Again, it makes sense to think of these batches as *pages* of an answer stream: `limit(K)` sets a page size, and `offset` selects which page to retrieve next.

Overlapping remote waits

The split-phase nature of `promise/3-4` and `yield/2-3` also makes it possible to overlap the waiting time for several remote computations. The important point is to issue the promises before yielding on any of them:

```
?- promise('http://n1.org', foo(X), R1),
   promise('http://n2.org', bar(Y), R2),
   yield(R1, A1),
   yield(R2, A2).
```

Since both remote computations are initiated before the caller waits for either answer, the elapsed time is approximately the maximum of the two response times, rather than their sum, assuming that the two remote nodes proceed independently. In this respect, the pattern resembles the use of `parallel/1` with synchronous RPC calls:

```
?- parallel([
    rpc('http://n1.org', foo(X)),
    rpc('http://n2.org', bar(Y))
]).
```

The two patterns are nevertheless not equivalent. A call to `parallel/1` behaves like a conjunction executed in parallel: the whole call succeeds only if all component goals succeed. It is therefore a mechanism for speeding up a conjunctive computation whose parts are expected to fit together as a single logical whole.

By contrast, `promise/3-4` and `yield/2-3` do not impose any collective success condition. Each promise denotes one pending remote computation, and each yield retrieves the answer associated with one such computation. The caller may choose to require all answers to be successful, but it may just as well accept partial success, ignore a failed computation, retry a timed-out one, or proceed as soon as one sufficiently useful answer has arrived. The pattern is therefore not a parallel conjunction, but a way of initiating several remote computations and later deciding how their outcomes should be interpreted.

This distinction also clarifies the issue of variable sharing. A parallel conjunction normally requires its component goals to be independent, in particular not to share variables whose bindings make the goals mutually dependent. The `promise/yield` pattern does not impose that restriction in the same way, because the promised computations are not parts of one live conjunction. Variables occurring in several promised goals should not be understood as shared variables spanning the remote computations. Each promise sends its own query, and each answer is later returned as data. Any consistency condition between the answers must therefore be imposed explicitly by the caller after collection.

For example, a caller that wants to combine two independently computed pages can do so locally:

```
?- promise('http://n1.org', foo(X), R1),
    promise('http://n2.org', bar(Y), R2),
    yield(R1, success(Page1, false)),
    yield(R2, success(Page2, false)),
    append(Page1, Page2, Solutions).
```

This is often the right design for web-facing computation: remote nodes compute independently, answers are returned as ordinary terms, and the caller decides how to compare, combine, or discard them.

Thus, `parallel/1` and `promise/yield` both allow waiting time to be reduced, but they express different ideas. The former is a parallelised conjunction with all-or-nothing success semantics. The latter is an asynchronous, split-phase interaction pattern with per-call answer handling.

Bounded waiting and timeouts

A client that cannot afford to block indefinitely can bound the wait performed by `yield/3`:

```
?- promise('http://n1.org', expensive_query(X), Ref),
   yield(Ref, Answer, [
       timeout(2.0),
       on_timeout(Answer = timeout)
   ]).
Answer = timeout,
Ref = 48468904.
?-
```

If no answer arrives within two seconds, the `on_timeout/1` goal is called and `Answer` is bound to `timeout`. Timing out does not cancel or consume the promise: the caller may invoke `yield/3` again with the same reference and collect the answer if it has since arrived.

A polling pattern therefore emerges naturally. The following code initiates an asynchronous call and periodically checks for its answer while doing other work:

```
?- promise('http://n1.org', (sleep(1), X=a), Ref),
   repeat,
       yield(Ref, Answer, [
           timeout(0.3),
           on_timeout(Answer = timeout)
       ]),
       (   Answer == timeout
       -> writeln('Still waiting...'),
           writeln('working...'),
           sleep(0.1),
           fail
       ;   !
       ).
Still waiting...
working...
Still waiting...
working...
Answer = success([(sleep(1),a=a)], false),
Ref = 13936890.
?-
```

Each iteration waits briefly for the answer. On timeout, the caller does some work and backtracks to try again. The binding `Answer = timeout` is undone by that backtracking, while the pending promise remains available. Once the answer arrives, the cut terminates the polling loop. The precise number of polling iterations depends on scheduling and network delay.

This is an explicitly operational pattern, but it remains sequential from the caller's perspective: at each step the caller either receives the answer or decides what to do after a bounded wait.

This illustrates a more general rule: on the Prolog Web, timing belongs in the protocol, at the points where a process would otherwise block.

4.3 The programmable Prolog Web

The Prolog Web is programmable in a stronger sense than the familiar “Web of APIs”. In its original form as a hypertext Web of documents, the Web was not itself a programmable execution substrate. This changed early with server-side scripts that generated documents dynamically and with client-side JavaScript. Later, remote procedure calling and open web APIs made it natural to treat the Web as a platform for composition, where independently developed services could be combined into “mashups”, often using a client language as glue.

Web Prolog supports this conventional style of integration, but also generalises it. A node may offer clients not merely a fixed set of endpoints, but a Web Prolog runtime environment that can be programmed directly by queries of varying complexity and, when permitted, by the injection of source code into long-lived processes on the node. In addition, a node owner may install node-resident predicates that become part of what clients can call. In this sense, the Prolog Web is programmable both from the outside, by clients issuing queries and spawning processes, and from the inside, by node owners shaping the execution environment with which clients interact.

This dual perspective aligns naturally with the actor model. An actor may begin with a private program supplied through the `src_*` options and subsequently acquire behaviour through interaction. Its placement determines the host policy, resource limits, and trust assumptions under which that behaviour executes.

Once code and data can cross node boundaries in either direction, a central systems question arises. Should a computation be moved to where the data resides, or should data, including code represented as data, be imported to where the computation runs? This question has no single correct answer. The optimal direction depends on data volume, update frequency, reuse patterns, latency requirements, and trust. The Prolog Web therefore does not enforce a fixed architecture, but instead exposes a small set of composable mechanisms that let programmers choose, per interaction, how code and data should flow.

The owner's side of this exchange was illustrated in Section 4.1.5: installing predicates in the shared database and running node-resident actors that define what clients can call. The remainder of this section focuses on the client's side: how the `src_*` options let a client determine not only *where* to run, but *what* to run.

4.3.1 Injecting code with the `src_*` options

Looking back at the ping-pong scenario in Section 4.1.2, what if the definition of `pong/0` is available at `n1.org`, but not at `n2.org`? The `src_predicates` option ensures that the appropriate source code is shipped from the caller's database and injected into the private database of the actor that will run at `n2.org`:

```
ping_pong :-
    spawn(pong, Pong_Pid, [
        node('http://n2.org'),
        src_predicates([pong/0])
    ]),
    spawn(ping(3, Pong_Pid)).
```

The `src_predicates` option names predicates that already exist in the caller's context. When the clauses themselves should be supplied literally, the `src_list` option serves that purpose. The following example uses it to populate a remote toplevel actor's private database with a clause that participates in the query:

```
?- toplevel_spawn(Pid, [
    node('http://n1.org'),
    src_list([(mortal(Who) :- human(Who))])
]).
Pid = 68427109@'http://n1.org'.
?-
```

The query `?- mortal(Who)` now executes on `n1.org`, combining the injected rule with the node's own `human/1` facts:

```
?- toplevel_call($Pid, mortal(Who), [
    template(Who)
]).
?- receive({A -> true}).
A = success(68427109@'http://n1.org', [plato, aristotle], false).
?-
```

4.3.2 Code shipping and data locality

The most direct way to bring code to the data in Web Prolog is to inject a small, task-specific predicate into a process placed near the data, execute it there, and return only the derived result. This corresponds closely to the classical systems idea of *function shipping*: rather than transferring a large dataset across the network, one ships a small computation to where the data resides and retrieves only a compact summary.

Conversely, remote facts or rules can be imported into a local process and reused there. Since the default value of the `node` option for `spawn/1-3` and `toplevel_spawn/1-2` is the special-purpose URI `localhost`, the same `src_*` mechanisms used for remote execution also support local composition. Code can therefore flow from client to node or from node to client, with applications choosing the direction at runtime according to latency, data size, or load. This reflects Prolog's natural blurring of the distinction between code and data: facts and rules can be moved, cached, and reused much like datasets.

4.3.3 A small design space

Taken together, the mechanisms described in this chapter form a small but expressive design space structured around a handful of recurring choices:

- **Interaction model:** one-shot remote procedure call, interactive `toplevel`, or actor-style orchestration (`rpc`, `toplevel_spawn`, or `spawn`)?
- **Placement:** locally or remotely (`node`)?
- **Program source:** from the caller's database, supplied literally, or already resident on the target node (`src_*`)?
- **Synchrony:** should the caller block, or initiate work and collect the result later (`promise/yield`)?
- **Lifetime:** one-shot request-response or session-oriented exchange?

Code shipping is also the point where portability stops being an academic concern and becomes operational. If nodes can execute code originating elsewhere, then even small semantic differences or library mismatches become visible immediately. For this reason, the Prolog Web treats mobile code as profile-scoped: a piece of code should state, implicitly or explicitly, which profile it assumes, so that execution is either well-defined or rejected early.

When a program is self-contained – all predicates on which it depends are either included or guaranteed by the target profile – it can be downloaded for local execution or shipped to another node without adaptation. We call this property *instant portability*: profile-scoped code, symmetric code flow, and uniform local and remote execution reduce relocation to a deployment decision rather than a porting effort.

4.4 Timing and failure boundaries on the Prolog Web

Distributed execution makes the duration and outcome of waiting less predictable. A goal that would ordinarily terminate promptly under local execution may, when it depends on a network or a remote node, fail to produce a result within an acceptable time. A timeout does not establish that the goal itself has failed; it marks the point at which waiting ends and the absence of a result becomes visible to the programmer.

In the Prolog Web architecture, two transports are used – HTTP and WebSocket – but three Web APIs are exposed: a stateless HTTP API, a semi-stateful HTTP API, and a stateful WebSocket API. Together they expose several blocking surfaces. In addition, `promise/3-4` and `yield/2-3` split submission from collection at the language level even though the underlying remote call uses the stateless HTTP API. Timeouts attach to operations at these surfaces, and their meaning and appropriate values depend on the interaction model.

4.4.1 Four blocking surfaces

Stateless HTTP API – request–response blocking. Blocking occurs during an HTTP request. A client submits a goal and waits for a response; a stateless remote procedure call may retrieve results through one or more such requests. Two bounds must be distinguished. The `timeout/1` option limits remote goal evaluation, subject to the ceiling imposed by the node owner, whereas `http_timeout/1` bounds the client’s HTTP operation. The former belongs to computation policy; the latter belongs to transport waiting. The `limit` and `offset` parameters bound the size of individual result slices and support incremental retrieval.

Promise/yield – deferred collection. A client initiates one asynchronous stateless call with `promise/3-4`, receives a local reference, and later invokes `yield/2-3`. A `timeout/1` option supplied to `yield/3` bounds only the current local wait for the result. It does not consume the promise, so a later invocation may still collect the answer. Short polling intervals, retries, and fallback actions through `on_timeout/1` therefore become explicit application logic. Promise cleanup and the execution and transport bounds applied to the underlying stateless request remain separate from the timeout on an individual yield.

Semi-stateful HTTP – session exchanges. Each request addresses the same long-lived toplevel actor by pid. Calls to `/toplevel_call`, `/toplevel_next`, and `/toplevel_poll` wait for the next relevant PTCP event within an ordinary HTTP exchange. An HTTP timeout bounds that exchange; it does not by itself erase the session actor or its evolving private state. Subject to session lifetime and node policy, a later request may continue the conversation or retrieve an event that has since become available.

Stateful WebSocket API – mailbox waiting. Blocking occurs inside `receive/2`. Its `timeout/1` option bounds a wait for a matching message, while `on_timeout/1` specifies the action to perform if the wait expires. The timeout therefore bounds participation in an interaction protocol rather than completion of a single computation. A mailbox wait may represent a reply, a coordination step, or a conversational turn. Its value is determined by protocol design, not merely by transport latency. Longer waits are natural in sustained dialogues, provided that recovery behaviour is defined.

The difference among these surfaces lies not only in what is computed, but in where waiting is exposed and how the absence of a timely result is interpreted.

4.4.2 Late replies

A timeout ends a wait; it does not necessarily cancel the underlying computation or actor. A result or message may therefore become available after the client has stopped waiting. For `promise/3-4` and `yield/2-3`, a timed-out `yield/3` leaves the local promise available for a later collection attempt. In the semi-stateful HTTP API, timing out an individual request need not terminate the session; a later request may continue the PTCP conversation or retrieve a queued event. In the stateful API, a message that arrives after a timeout may remain available to a subsequent `receive/2`. The corresponding behaviour of an ordinary stateless HTTP operation depends on whether the remote evaluation is also terminated.

What happens to a late result – whether it is retained, queued, delivered while the interaction context remains alive, or discarded – is part of the *semantics of timing*. It determines what the client can observe and what guarantees the API provides. The Prolog Web exposes these semantics at the language level through options such as `timeout/1` and `on_timeout/1`, rather than hiding them entirely inside transport configuration.

The complementary question – what happens to the *resources* that produced the late reply, and who is responsible for reclaiming them – belongs to lifetime discipline and is addressed in Section 4.5.3.

4.4.3 Design principle

A simple rule emerges:

Place a timeout at each operation that may block, and choose both its duration and its timeout action according to the interaction model – not merely according to transport defaults.

Under this discipline, timing becomes part of the architectural contract. Although the architecture uses only two transports – HTTP and WebSocket – its three Web APIs, together with the abstractions layered over them, expose distinct waiting and failure boundaries. Making those boundaries explicit is essential for predictable behaviour on an open network.

4.5 Security on the Prolog Web

A Web Prolog node may be configured to accept goals from any client that knows its URI. Openness of this kind raises an obvious question: what makes it safe to let strangers run code on your node?

The answer has two layers. Three mechanisms – *inexpressibility*, *invisibility*, and *containment* – define the structural security posture: what client code can express,

what it can reach, and how long its effects may persist. Operational security adds authentication, resource governance, transport protection, and host isolation. The structural mechanisms arise from the Web Prolog design; the operational mechanisms are realised jointly by node policy and deployment infrastructure.

The relevance of the structural mechanisms varies with the node profile. For profiles such as **RELATION** and **ISOBASE**, where interaction is stateless or request-response over HTTP, inexpressibility and the controls familiar from conventional web services are the dominant concerns. These include input validation, resource governance, authentication, and transport protection. Invisibility and containment become particularly important for profiles that support long-lived sessions and bi-directional communication, such as **ISOTOPE** and **ACTOR**, where process identity and message passing cross node boundaries. The remainder of this section focuses on this richer setting, where the interplay of all three structural mechanisms becomes visible.

The relationship between the structural and operational mechanisms is illustrated in Figure 4.7.

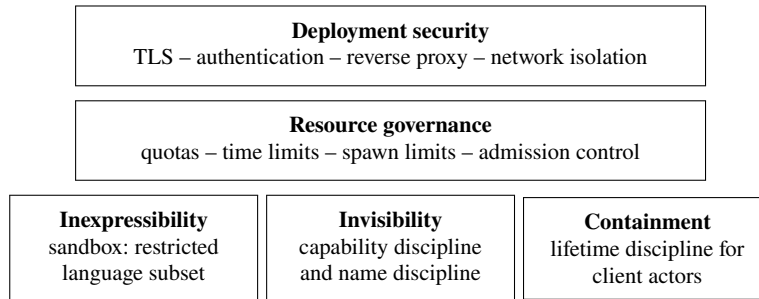


Fig. 4.7 Structural and operational security mechanisms of a Web Prolog node.

4.5.1 Inexpressibility: the sandbox

Web Prolog follows the same broad strategy as browser-based JavaScript: untrusted code executes within a restricted environment. The client language excludes file I/O, direct operating-system access, raw socket programming, and durable node storage. These facilities, if needed, are provided by the host platform and mediated through predicates and services that the node owner chooses to install – they are not directly expressible by client-injected code.

This is the price of allowing strangers to run code on your machine: certain classes of host effect must be unavailable to client code. Submitted goals and source are therefore checked against the active profile and sandbox policy before execution. Where the language permits goals to be constructed dynamically, guarded execution paths apply the policy again to the resulting goal. The sandbox substantially narrows

the client language, while host isolation provides a separate backstop should the language-level boundary fail.

Even when an operation exists in the underlying system, it is exposed to clients only through owner-controlled predicates and services. Access to those predicates and services may in turn be constrained by authentication and authorisation policy. The authorisation policy is therefore part of the node's security boundary: it defines which actions are available to which roles, and thereby what the node counts as its public surface.

4.5.2 Invisibility: name discipline

Joe Armstrong articulated a useful design principle for actor systems: if a process cannot be named, it cannot be addressed. Once process names become widely known, the system's communication surface expands. Web Prolog adopts this principle as a discipline for process identity.

A pid in Web Prolog is an inspectable and constructible process address. Its canonical global form is the ordinary Prolog compound `Id@Node`; the local presentation may be an integer. The node assigns `Id`, and the complete pair denotes the actor. Constructing the pair does not allocate an actor, establish that it is live, or bypass authorisation. A valid pid can nevertheless function as a communication capability when possession of the address, together with node policy, permits messaging. Collision resistance, non-reuse, and node authorisation support this discipline. In ordinary public client execution, `actors/1` lists actors belonging to the same client interaction. It does not reveal other clients' actors or private owner resources; node-owned runtime or administrative execution has broader visibility. Beyond their own interaction, clients normally acquire live pids through explicit disclosure. Non-disclosure therefore narrows the set of actors a client can address, but pid secrecy is not a substitute for authentication or authorisation. A pid may be necessary to address an actor without being sufficient authority to operate on it. Possession alone grants messaging authority only where node policy allows any holder of the address to send; other policies require additional credentials or permissions. This does not make a constructible pid an unforgeable reference. A restricted capability endpoint also enforces a particular scope of use: the internal terminal endpoint described earlier combines an unguessable token for one terminal lineage with authenticated node transport and terminal-specific operations.

Discoverability creates a natural tension with name discipline. If processes are protected in part because they cannot be addressed without a name, public services must still provide a controlled way for clients to learn which names are intended for use. The safe resolution is to treat discovery as an owner-curated *publication mechanism*: discovery should resolve *published* names, not reveal everything that happens to be running. One concrete realisation of this idea – a declarative service directory queryable via `rpc/2-3` – was given in Section 4.1.5.

Reply channels as capabilities

The common request–reply idiom illustrates why the distinction matters. A client often includes its own pid in a request so that the provider can reply. Where node policy permits the provider to send arbitrary messages to that address, this enables messages unrelated to the request as well as the expected reply. Disclosure alone does not override a policy that restricts such access.

A safer protocol uses a fresh, request-local reply endpoint, such as a temporary proxy actor that accepts only the expected reply and then terminates. The provider receives authority over that isolated protocol step rather than over the client actor generally. Communication authority should be as narrow and short-lived as the interaction that requires it. This discipline becomes particularly important when reply channels cross node boundaries and therefore cross trust domains.

4.5.3 Containment: lifetime discipline

On an open network, a client may disconnect mid-session or disappear without warning. If server-side actors created on that client’s behalf continue after the client has gone, they may keep consuming CPU and memory or sending messages into mailboxes that will never be read. Over time, such orphaned processes can accumulate into unbounded state and a potential security exposure.

The preventive measure is to make process lifetimes *containment-shaped*: every actor spawned by a client is lifetime-dependent on its parent. When parent termination or connection loss is detected, the child-hosting controller propagates termination through the linked subtree. For remote links it does so either from explicit knowledge of the parent’s termination or, in the current demonstrator, when the connection owning the remote subtree reports close or error, as specified in Section 4.1.7. This does not depend on a final exit packet crossing a connection already known to have failed, but neither does it promise bounded detection of a silent partition. In Web Prolog, `link(true)` is the default; creating an independent actor requires an explicit `link(false)`, and that option is reserved for node owners. A client cannot detach work from its session.

The one-way link itself is adopted from Svensson et al. (2010, Section 2.2), not introduced here. In their proposed Erlang semantics, a child links to its supervisor so that the supervisor’s termination kills the child, while a monitor carries failure information in the other direction. Web Prolog turns that asymmetric relation into an ownership discipline: it is established by spawning, made the default, and extended across node and session boundaries. The prior semantic proposal supplies the direction of dependence; the client–owner model supplies the reason for enforcing it here.

Remote placement sharpens the point. A client may place a child on a remote node via `node(URI)`, but only under lifetime dependence. Allowing a remote detached child would turn `spawn/3` into a deployment primitive and make orphan prevention

intractable under disconnects and network partitions. For that reason, client-side `remote spawn` rejects `link(false)`.

Containment addresses the resource side of the late-reply problem identified in Section 4.4. Timing semantics determine what the client observes after a timeout; lifetime discipline determines when the associated processes are reclaimed.

Should client-originated detached jobs be needed in the future, they should be introduced as an explicit abstraction – with quotas, lifetime parameters, and auditable handles – rather than as a general relaxation of the containment default.

4.5.4 Deployment concerns

The three structural mechanisms above define the security architecture of the Prolog Web, but they do not by themselves constitute a complete operational security model.

First, the node must establish *who* the caller is. Without authentication, the distinction between client and owner has no firm operational basis. Second, the node must enforce *resource governance*. A client may remain entirely within the sandbox and yet still exhaust CPU time, memory, process slots, or network bandwidth. Third, communication between client and node must be protected in transit. Without TLS or equivalent transport protection, a third party may observe or tamper with messages, undermining confidentiality and integrity.

Fourth, the runtime process itself must be constrained by the host environment. Even a correctly specified language sandbox offers limited assurance if the underlying process retains unrestricted filesystem or network access. Host isolation – through operating-system-level confinement, WebAssembly runtimes, or WASI-based embeddings – removes classes of risk that the language boundary alone cannot prevent.

These concerns extend beyond language semantics. They are realised jointly by Web Prolog runtime policy and deployment infrastructure: the node enforces identities, permissions, and resource ceilings, while the host environment and network provide transport protection and process isolation. A production node therefore requires both structural security and secure operation.

What remains after these mechanisms have done their work is the node's public surface: the predicates it exports, the services it publishes, and the policies under which they may be accessed. A well-configured node should therefore present a small, explicit, and auditable interface: everything else should be either inexpressible, invisible, or contained.

4.6 Summary

This chapter developed the Prolog Web as the third component of the Prolog Trinity, extending Web Prolog from a language for single-node programming into a language for distributed systems.

The *concurrent Prolog Web* introduced a placement-transparent actor interface: processes can be spawned on remote nodes and addressed through the same actor primitives used locally. Distribution nevertheless introduces latency, partial failure, and restrictions on the terms that can cross node boundaries. Links propagate termination through process trees, while monitors allow processes to observe the termination of others. Node-resident actors demonstrated how an owner can install long-lived, shared services, and a naming and discovery mechanism showed how such services can be advertised while preserving name discipline.

The *sequential Prolog Web* provided a disciplined request–response abstraction. Its central construct, `rpc/2–3`, realises nondeterministic remote procedure calls with familiar Prolog semantics, implemented first over a remote toplevel actor and then, more practically, over stateless HTTP. The split-phase `promise/3–4` and `yield/2–3` predicates extend this model to deferred result collection. Although the sequential layer uses the underlying process infrastructure, it presents an interface in which mailboxes and scheduling remain invisible.

The *programmable Prolog Web* showed how code and data can flow between client and node. The `src_*` options let clients supply task-specific predicates to remote execution contexts, while the same mechanisms support local composition through the `localhost` URI. Source injection is therefore a general program-composition facility rather than a remote-only feature.

The discussion of *timing and failure boundaries* identified where waiting becomes observable across the three Web APIs and the `promise/yield` abstraction over stateless HTTP: ordinary request–response blocking, deferred promise collection, semi-stateful session-event retrieval, and stateful mailbox waiting. It established that timeouts belong at blocking surfaces and should be chosen according to the interaction model rather than transport defaults. A timeout ends a wait without necessarily establishing that the goal has failed or cancelling the underlying computation.

Finally, the discussion of *security* asked what makes it safe to let strangers run code on a node. The answer combined three structural mechanisms – inexpressibility, invisibility, and containment – with the operational concerns of authentication, resource governance, transport protection, and host isolation. What these mechanisms leave exposed is the node’s *public surface*: the predicates it exports, the services it publishes, and the policies under which they may be accessed.

The chapters that follow build on this foundation. Part II develops generic behaviours – servers, supervisors, toplevels, and statecharts – that structure actor implementations. Later parts return to the architectural and paradigmatic questions raised by these technical layers.

Part II
Behaviours and other generics

Chapter 5

Implementing Web Prolog generics

Generic constructs provide reusable actor patterns that can be specialised, configured, or extended after creation, and that encapsulate common concerns such as message protocols, supervision, and state management.

It is useful to distinguish two broad categories of generics considered in Web Prolog. The first category comprises *Erlang-style behaviours*, which are message-driven components exposing a stable protocol, typically realised as a fixed vocabulary of messages and replies. They do not expose the callee’s proof search directly through caller backtracking. A client wrapper may bind an argument to an explicit reply term, but the interaction itself remains protocol-oriented. The `server_*` predicates provide a request–response service analogous to Erlang’s `gen_server`; the `supervisor_*` predicates provide a behaviour similar to an Erlang `supervisor`; and statechart actors implement a state-machine model in the spirit of `gen_statem`. The `toplevel_*` predicates also belong to this category, although a Prolog toplevel actor has no ordinary counterpart in Erlang.

The second category comprises what may be termed *Prolog-style generics*. Some are meta-predicates that orchestrate actors internally while presenting a conventional predicate-level interface. Examples include `parallel/1` and `first_solution/2-3`, which spawn worker actors, coordinate them with selective receive, and expose success, failure, exceptions, and variable bindings directly to the caller. They are semideterministic and do not enumerate further solutions on backtracking. By contrast, `rpc/2-3`, specified in Chapter 4 and implemented below, is nondeterministic: as a distributed analogue of a predicate call, it participates directly in proof search and can return multiple solutions.

This separation reflects a dual identity of the language. Web Prolog inherits explicit message passing and supervision from the actor model while retaining Prolog’s predicate interfaces and variable binding. Generics reconcile the two by factoring communication, lifecycle management, and concurrency out of application logic. Programmers can therefore work either with explicit actor protocols or with ordinary-looking predicates, according to the needs of the application.

The implementations in this chapter are implementation-oriented design sketches, not source listings of the demonstrator. Some fragments are executable in isolation;

others omit helper predicates in order to expose the protocol or programming pattern more clearly. A production implementation must also deal with such matters as module qualification, source transfer, validation, race-free cleanup, resource limits, and transport failure. Where those details do not change the point being explained, we hide them behind small helpers or omit them explicitly. Appendix B defines the normative interfaces adopted by the demonstrator. An interface introduced here but not yet present in the manual should be read as a design proposal. Readers who want to run the complete implementation, rather than copy an isolated sketch, can use the Trinity demonstrator and companion source repository described in Section 1.6.

5.1 The server behaviour

The *server behaviour* implements a classic client–server pattern in Web Prolog: a central server process maintains private state and serves an arbitrary number of clients through message passing. The behaviour is specialised by supplying an application-specific callback predicate, while the generic part provides request–response communication, state threading, monitoring, graceful shutdown, and hot code replacement.

The fragments below form one example. We first define a refrigerator callback and the reusable server loop, then add a compact client side of the behaviour. Unless a fresh server is explicitly created, the shell transcripts continue to use the same refrigerator process and therefore the same state.

To keep the example focused on the behaviour itself, we assume that callback predicates named in `server_spawn/3-4` or `server_upgrade/2` are already available to the server process. The demonstrator’s `server_upgrade/3` can instead transfer and install callback source; that fuller interface is documented in Appendix B, but its implementation is not needed to explain the server protocol.

The application-specific part is a callback describing one transition. Its four arguments are the request, the old state, the response, and the new state:

```
fridge(store(Food), Foods, ok, [Food|Foods]).
fridge(take(Food), Foods, ok(Food), Rest) :-
    select(Food, Foods, Rest), !.
fridge(take(_Food), Foods, not_found, Foods).
```

The callback contains only the application transition; process ids, correlation references, reception, and recursion belong to the generic behaviour. Its contract is semideterministic: a supported request succeeds at most once, while failure means that the request is not handled. The cut makes the second clause deterministic when `fridge/4` is called independently; the server loop also commits to the callback’s first solution.

A server is started with a callback predicate `Pred/4` and an initial state. The spawn predicates and the loop make up the server side of the behaviour:

```

server_spawn(Pred, State, Pid) :-
    server_spawn(Pred, State, Pid, []).

server_spawn(Pred, State, Pid, Options) :-
    spawn(server_loop(Pred, State), Pid, Options).

```

For an application request, the loop calls the callback as

```
call(Pred, Request, OldState, Response, NewState)
```

and passes the resulting state to its next iteration:

```

server_loop(Pred, State0) :-
    receive({
        '$call'(From, Ref, Request) ->
            (
                call(Pred, Request, State0, Response, State)
            -> From ! Ref-Response,
                server_loop(Pred, State)
            ; exit(false)
            ) ;
        '$control'(From, Ref, upgrade(Pred1)) ->
            upgrade_result(Pred1, Result),
            From ! Ref-Result,
            (
                Result == ok
            -> server_loop(Pred1, State0)
            ; server_loop(Pred, State0)
            ) ;
        '$control'(From, Ref, stop) ->
            From ! Ref-true
    }).

upgrade_result(Pred, Result) :-
    (
        current_predicate(Pred/4)
    -> Result = ok
    ; Result = error(
        error(existence_error(procedure, Pred/4), _)
    ).

```

A call message contains the client pid, a fresh correlation reference, and the application request. If the callback succeeds, the server sends the response tagged with the same reference and recurs with the new state. If no callback clause handles the request, the server exits with reason `false`. An uncaught callback exception `E` terminates it with reason `exception(E)`. After acknowledging `stop`, the loop has no recursive call and the server terminates normally, with reason `true`. An upgrade is acknowledged only after the replacement has been validated. If validation fails, the error is returned and the loop continues with its old callback and state.

5.1.1 The client interface

The client side uses the same wire protocol as the loop. The following predicates are one coherent implementation of ordinary synchronous calls, requests kept in flight, and fail-fast monitoring. We begin with the two operations from which a call is built: a *promise* sends a request, and a *yield* collects its reply.

```
server_promise(To, Request, Ref) :-
    self(Self),
    make_ref(Ref),
    To ! '$call'(Self, Ref, Request).

server_promise(To, Request, Ref, MonRef) :-
    self(Self),
    monitor(To, MonRef),
    make_ref(Ref),
    To ! '$call'(Self, Ref, Request).
```

The four-argument form installs a monitor before sending and returns both references. Monitoring before the send closes the race in which the server terminates just before the monitor is installed. The corresponding collection predicates are:

```
server_yield(Ref, Response) :-
    receive({
        Ref-Response -> true
    }).

await_reply(Ref, MonRef, Response) :-
    call_cleanup(
        receive({
            Ref-Response ->
                true ;
            down(_Pid, MonRef, Reason) ->
                throw(server_down(Reason))
        }),
        demonitor(MonRef, [flush])
    ).
```

The unmonitored yield waits only for a matching reply. `await_reply/3` additionally accepts the notification that the server has terminated. Its cleanup removes the monitor whether the receive succeeds or raises an exception. The `flush` option also removes a `down` notification that may already have been queued if the server replied and then terminated before the client cancelled its monitor. This helper is the compact sketch's counterpart of the public `server_yield/4` operation documented in Appendix B.

Figure 5.1 makes the ordering explicit. The monitor is installed before the request is sent, so either the correlated reply or a termination notification can settle the wait.

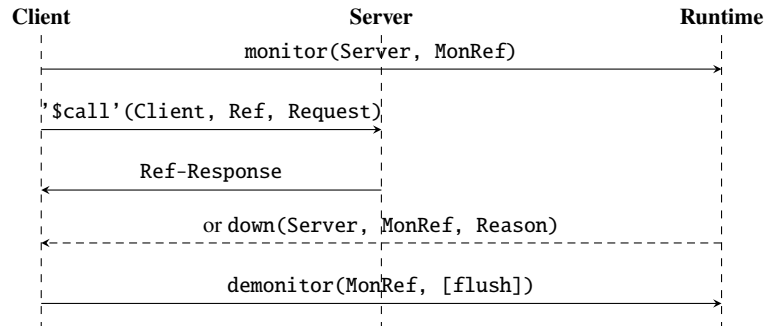


Fig. 5.1 Correlation and failure detection in a monitored server call.

An ordinary synchronous request uses the monitored forms by default:

```
server_request(To, Request, Response) :-
    server_promise(To, Request, Ref, MonRef),
    await_reply(Ref, MonRef, Response).
```

We now have a complete path through the code: `server_request/3` sends `'$call'(Self, Ref, Request)`, the server loop invokes `fridge/4`, and the client selects the resulting `Ref-Response`. For example:

```
?- server_spawn(fridge, [], Pid).
Pid = 99491660.
?- server_request($Pid, store(milk), Response).
Response = ok.
?- server_request($Pid, store(cheese), Response).
Response = ok.
?- server_request($Pid, take(milk), Response).
Response = ok(milk).
?-
```

The refrigerator's state is now `[cheese]`. To keep several requests in flight, a client instead uses the unmonitored promise and yield forms directly. Each request carries a fresh reference, so replies can be collected independently. Here the client deliberately collects the second reply first:

```
?- server_promise($Pid, store(meat), StoreRef),
    server_promise($Pid, take(meat), TakeRef).
StoreRef = 11025019,
TakeRef = 11025020.
?- server_yield($TakeRef, Taken).
Taken = ok(meat).
?- server_yield($StoreRef, Stored).
Stored = ok.
?-
```

Although the server processed the requests and sent their replies in order, selective receive found the reply tagged with `TakeRef` and left the earlier reply in the mailbox for the following yield. The state is again `[cheese]`. Code that needs both pipelining and fail-fast behaviour retains the monitor reference with the request reference and uses the same monitored collection pattern as `await_reply/3`. The fuller option-bearing interface is documented in Appendix B. Bounded waiting is omitted from this sketch because a sound timeout policy must say what happens to a reply that arrives after the wait has ended.

5.1.2 A failing callback

Monitoring an actor that has already terminated does not lose the failure notification: the caller receives a `down` message with reason `no_process`. Thus a synchronous request whose callback fails does not wait forever either. Using a fresh refrigerator so that the running example above can continue:

```
?- server_spawn(fridge, [], TypoPid),
   server_request(TypoPid, sore(milk), Response).
ERROR: Unhandled exception: server_down(false)
?-
```

The misspelling `sore(milk)` matches no clause of `fridge/4`. The callback fails, the fresh server exits with reason `false`, and `await_reply/3` reports that exit to the waiting client. The original server in `$Pid` is unaffected.

5.1.3 Hot code replacement

Upgrade and shutdown use the same monitored acknowledgement pattern as an application request. A small helper installs the monitor before sending the control message, so either an acknowledgement or the server's termination is observed:

```
server_control(To, Command, Reply) :-
    self(Self),
    monitor(To, MonRef),
    make_ref(Ref),
    catch(To ! '$control'(Self, Ref, Command), Error,
          (demonitor(MonRef, [flush]), throw(Error))),
    await_reply(Ref, MonRef, Reply).

server_upgrade(To, Pred) :-
    server_control(To, upgrade(Pred), Result),
    (   Result == ok
```

```

-> true
;   Result = error(Error), throw(Error)
).

```

The server validates the replacement before acknowledging it and recurs with the new callback while preserving `State0`; thus the client knows that the upgrade has been accepted, not merely placed in transit. A rejected upgrade raises the reported error at the client while the server continues with its previous callback and state. The demonstrator's three-argument form performs source loading before this validation and reports loading errors through the same reply.

To make the upgrade observable, the replacement callback adds an `inventory` request and delegates the existing requests to `fridge/4`:

```

fridge_with_inventory(inventory, Foods, inventory(Foods), Foods).
fridge_with_inventory(Request, Foods0, Response, Foods) :-
    fridge(Request, Foods0, Response, Foods).

```

Assuming that `fridge_with_inventory/4` is already available to the server process, the callback of our original refrigerator can now be replaced:

```

?- server_upgrade($Pid, fridge_with_inventory).
true.
?- server_request($Pid, inventory, Response).
Response = inventory([cheese]).
?-

```

The operation returns only after the server has accepted the new callback and, as the `inventory` response confirms, retains the current state. The replacement must therefore accept the existing state representation. If an upgrade changes that representation, a separate state-migration protocol is required; simple callback replacement is not enough.

5.1.4 Graceful shutdown

A server is stopped through the same acknowledged control path:

```

server_halt(To, Reply) :-
    server_control(To, stop, Reply).

```

The helper's fresh reference makes the shutdown acknowledgement subject to the same selective-receive discipline as ordinary requests and upgrades. We can now stop the refrigerator used throughout the section:

```

?- server_halt($Pid, Reply).
Reply = true.
?-

```

5.1.5 From a bespoke actor to a reusable behaviour

The refrigerator in Chapter 2 was already a stateful concurrent service, but it was implemented as a bespoke actor. The predicate `fridge/1` served simultaneously as the application logic and the actor loop: it recognised incoming messages, addressed replies to clients, carried the food list in its recursive argument, and handled termination.

The server behaviour separates these responsibilities. The `fridge/4` callback shown above describes a single application transition, mapping a request and the current state to a response and a successor state. The generic server owns the state, receives and correlates messages, invokes the callback, and continues with the resulting state. Recursion has therefore moved out of the refrigerator predicate and into the reusable behaviour. On the client side, the server interface similarly hides the construction and matching of request and reply messages performed manually in the earlier example.

The contrast is not between a relation and a concurrent service – both examples implement concurrent services – but between a purpose-built actor loop and a reusable behavioural abstraction. By factoring out the actor protocol, the server behaviour allows each application to express its state transitions as ordinary Prolog predicates without reimplementing the surrounding communication and lifecycle machinery.

5.2 The supervisor behaviour

The *supervisor behaviour* starts a set of child actors, monitors them, and applies a declared policy when one terminates. Application code describes the children; the generic supervisor owns their lifecycle. This separation is the practical form of the *let it crash* principle: a worker may terminate and discard its process state rather than attempting in-place repair (Armstrong, 2003b).

5.2.1 A restart we can observe

We continue with the refrigerator from the previous section, adding one request that deliberately terminates the server:

```
supervised_fridge(crash, _Foods, _Reply, _NewFoods) :-
    exit(simulated_failure).
supervised_fridge(Request, Foods0, Reply, Foods) :-
    fridge(Request, Foods0, Reply, Foods).
```


Assume, as before, that the callback is available to the new actor. The child specification gives the child an identifier, a start instruction, an initial state, and a restart policy. The registered name remains stable even though the child pid changes:

```
?- supervisor_spawn([
    child(fridge_service, [
        start(server(supervised_fridge,[initial_state([])])),
        restart(permanent)
    ])
], Sup, [strategy(one_for_one), intensity(3),
        period(10), monitor(true)]).
Sup = 58310427.
?- supervisor_which_children($Sup, Children).
Children = [info(fridge_service, 91467230, worker, permanent)].
?-
```

Here `worker` is the child's type. Both `server` and ordinary actor starts are workers; a nested supervisor has type `supervisor`. The compact internal specification below omits this production field since the one-for-one termination path does not inspect it.

We put some state in the child and then make it fail:

```
?- server_request(fridge_service, store(milk), Reply).
Reply = ok.
?- catch(server_request(fridge_service, crash, _), Error, true).
Error = server_down(simulated_failure).
?- supervisor_which_children($Sup, Children).
Children = [info(fridge_service, 26083951, worker, permanent)].
?-
```

The supervisor itself still has pid 58310427, but the child pid has changed from 91467230 to 26083951. The child specification was reused to start a replacement and to register it again as `fridge_service`. Process state is not recovered automatically: the replacement starts from the declared initial state.

```
?- server_request(fridge_service, take(milk), Reply).
Reply = not_found.
?-
```

The `permanent` policy requests a restart after either a normal or an abnormal termination. A `transient` child is restarted after any termination reason other than `true`, while a `temporary` child is never restarted. These are child policies; the supervisor strategy, discussed below, determines which other children are affected by a required restart.

Without the supervisor behaviour, the application would have to retain every child's start information, monitor each pid, classify its termination reason, enforce the restart budget, and register any replacement under the stable service name. That

hand-written loop would be almost identical for every application. The supervisor moves it into one reusable mechanism and leaves the application to declare only the child specification and restart policy.

5.2.2 The generic mechanism

Validation lowers the public `server(Pred, ServerOptions)` child form to a start instruction containing the extracted initial state. The loop then uses the small internal form `spec(Id, Start, Restart)`. That start instruction is either `server_start(Pred, State)` or `actor(Goal)`; both kinds of child install a monitor and a directional link:

```
start_child(Spec, running(Pid, Spec)) :-
    Spec = spec(Id, server_start(Pred, State), _),
    server_spawn(Pred, State, Pid,
        [name(Id), monitor(true), link(true)]).
start_child(Spec, running(Pid, Spec)) :-
    Spec = spec(_, actor(Goal), _),
    spawn(Goal, Pid, [monitor(true), link(true)]).

should_restart(spec(_, _, permanent), _Reason).
should_restart(spec(_, _, transient), Reason) :-
    Reason \== true.
```

There is deliberately no `should_restart/2` clause for a temporary child. The monitor reports termination; the link has the other directional purpose of ensuring that children do not survive an unexpected termination of their supervisor. A server child uses `Id` as its stable registered name. For an ordinary actor, `Id` is supervisor bookkeeping unless `Goal` registers a name itself.

The loop also needs to find a child without losing its position in start order. `take_child/5` returns the children to its left and right:

```
take_child(Pid, [running(Pid, Spec)|Right], Spec, [], Right).
take_child(Pid, [Child|Children], Spec, [Child|Left], Right) :-
    take_child(Pid, Children, Spec, Left, Right).

retire_child(spec(_, _, temporary), Left, Right, Children) :-
    append(Left, Right, Children).
retire_child(Spec, Left, Right, Children) :-
    Spec = spec(_, _, Restart),
    Restart \== temporary,
    append(Left, [running(undefined, Spec)|Right], Children).
```

A temporary child's specification is removed. A non-temporary child that is not restarted remains available for an explicit later respawn, but has no current pid.

The two `retire_child/4` clauses are exclusive, so backtracking cannot reinsert a temporary child. With these helpers the complete one-for-one termination cycle remains small:

```
supervisor_loop(Children0, Limit0) :-
    receive({
        down(Pid, _, Reason)
            if take_child(Pid, Children0, Spec, Left, Right) ->
            (
                should_restart(Spec, Reason)
            -> charge_restart(Limit0, Limit),
                start_child(Spec, Child),
                append(Left, [Child|Right], Children)
            ;
                Limit = Limit0,
                retire_child(Spec, Left, Right, Children)
            ),
            supervisor_loop(Children, Limit) ;
        down(_, _, _) ->
            supervisor_loop(Children0, Limit0)
    }).
```

The guarded clause accepts a notification for a child in the current list; a stale or unrelated `down/3` notification is consumed by the fallback clause rather than accumulating in this dedicated supervisor's mailbox. As described in Section 2.3.4, the guard may bind `Spec`, `Left`, and `Right`; `receive` commits to its first solution. Child pids are unique, so there is at most one such split. The matching specification supplies both the restart policy and the start instruction, while the two list segments preserve start order.

This listing shows only the termination path. The full loop also has branches for `supervisor_which_children/2` and other management requests, using the monitored request pattern from Section 5.1.

5.2.3 From restart to escalation

Restarting without a limit can turn one fault into an endless crash loop. The options `intensity(3)` and `period(10)` in the example establish a rolling restart budget: three restart attempts may occur within ten seconds. The next required restart in the same window makes the supervisor terminate with reason `shutdown`. Because the example also uses `monitor(true)`, the shell would then receive `down(58310427, 58310427, shutdown)`. The first two arguments coincide because `spawn-time monitor(true)` uses the monitored pid as its monitor reference. The loop charges that window as follows:

```
charge_restart(limit(Max, Period, Times0),
               limit(Max, Period, [Now|Recent])) :-
```

```

    get_time(Now),
    Cutoff is Now - Period,
    include(recent_restart(Cutoff), Times0, Recent),
    length(Recent, N),
    (    N < Max
    ->   true
    ;    exit(shutdown)
    ).

```

```

recent_restart(Cutoff, Time) :-
    Time > Cutoff.

```

The initial value for the example is `limit(3, 10, [])`. Old timestamps are discarded before the count is tested, so failures far apart do not consume a lifetime allowance. By contrast, several rapid failures are treated as one unresolved incident. The allowance is charged before `start_child/2`, so every requested replacement counts. If the supervisor gives up, its directional links terminate the remaining children, and a parent supervisor can apply a broader recovery policy.

5.2.4 Strategies, dynamic children, and trees

The loop above implements `one_for_one`: only the failed child is replaced. Under `one_for_all`, the supervisor shuts down the other children and restarts the affected group. Under `rest_for_one`, it does so only for the failed child and the children started after it. Start order is therefore part of the supervisor's state rather than an incidental property of a list. Order also matters during shutdown, when children are normally terminated in reverse start order.

Dynamic child management changes the stored set of specifications: an API may add and start a child, terminate one while retaining its specification, delete a stopped specification, or respawn it later. These operations require no new actor primitives. They use the same spawning, monitoring, linking, termination, and selective receive shown in the smaller loop; synchronous management calls additionally use the monitored request pattern developed in the preceding section.

Supervisors can themselves be children. If a supervisor exceeds its restart intensity, its parent observes the termination and applies the parent-level policy. Repeating this arrangement yields a supervision tree in which a local failure is first handled locally and escalates only when that level exhausts its restart allowance. The demonstrator implements all three strategies, dynamic child operations, timed restart intensity, and orderly shutdown. The complete public interface and child-specification format are documented in Section B.5 of Appendix B. Erlang's OTP supervision principles provide a fuller account of the model.¹

¹ https://www.erlang.org/doc/system/sup_princ.html

5.3 The toplevel behaviour

Chapter 3 showed how a client uses a toplevel actor: it submits a goal, receives a slice of solutions, and either asks for another slice or abandons the search. Here we ask a narrower implementation question: how can an actor preserve a Prolog computation between those requests and resume it later?

A resumable interaction has the following shape:

```
toplevel_call(Pid, p(X), [template(X), limit(1)])
  -> success(Pid, [a], true)
toplevel_next(Pid)
  -> success(Pid, [b], false)
```

The final argument of `success/3` says whether a suspended computation remains. The second request resumes that computation; it does not run `p(X)` again. The public operations have distinct roles:

```
toplevel_call/2-3 start a query;
toplevel_next/1-2 resume it;
toplevel_stop/1 discard it;
toplevel_abort/1 interrupt it and return to idle; and
toplevel_halt/1-2 terminate the actor.
```

Complete examples and option descriptions remain in Chapter 3 and Section B.2 of Appendix B.

These operations are easier to understand as a three-state protocol. In **s1**, the actor is idle; in **s2**, it computes a slice; and in **s3**, it retains the suspended search while waiting for the client to request another slice or stop.

The implementation developed here follows the core of the demonstrator's local toplevel closely. To keep the mechanism visible, we omit source installation, module isolation, goal validation, inference limits, `once(true)`, transport hooks, and other production machinery. None changes the three-state model.

The small wrapper below spawns an actor running that protocol:

```
toplevel_spawn(Pid) :-
    toplevel_spawn(Pid, []).

toplevel_spawn(Pid, Options) :-
    self(Self),
    option(target(Target), Options, Self),
    option(session(Session), Options, false),
    spawn(ptcp(Pid, Target, Session), Pid, Options).
```

The `target` option determines where answers are sent. The `session` option controls what happens after a query. Its default, `false`, terminates the actor once the query is exhausted or stopped; `true` returns it to idle so that it can accept another query.

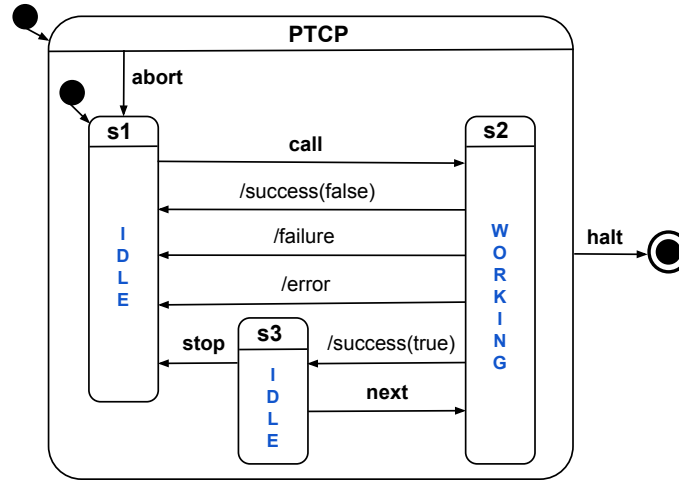


Fig. 5.2 The persistent form of the Prolog Toplevel Communication Protocol.

5.3.1 The three inner states

The implementation mirrors the three states in Figure 5.2 with the predicates `state_1/3`, `state_2/6`, and `state_3/2`.

The statechart shows the persistent, `session(true)` form of the protocol. The simplified implementation also supports a one-shot `session(false)` form, which terminates normally after its query has been exhausted or stopped; this lifecycle variation is not shown in the statechart.

The idle state `s1` is implemented by `state_1/3`:

```

state_1(Pid, Target0, Session) :-
    receive({
        '$call'(Goal, Options) ->
            option(template(Template), Options, Goal),
            option(offset(Offset), Options, 0),
            option(limit(Limit0), Options, 10000000000),
            option(target(Target1), Options, Target0),
            Limit = count(Limit0),
            Target = target(Target1),
            state_2(Goal, Template, Offset, Limit,
                    Pid, Answer),
            arg(1, Target, Out),
            Out ! Answer,
            ( arg(3, Answer, true)
            -> state_3(Limit, Target)
            ; true
    )
  
```

```

    )
  }},
  (   Session == false -> true
    ;   state_1(Pid, Target0, Session)
  ).

```

The '\$call' message supplies the goal and its options. The default limit, 100000000000, is the finite value used by the public API when no `limit/1` option is supplied; in practice it acts as an effectively unbounded first slice. It is stated explicitly in Appendix B. The limit and target are stored in mutable one-argument compounds so that a later '\$next' message can change either value. Halting is not a mailbox command and therefore requires neither a '\$halt' branch nor a mutable control term. It is implemented by the actor runtime, as described below.

The test `arg(3, Answer, true)` succeeds only for `success(Pid, Slice, true)`. It simply fails for the lower-arity `failure(Pid)` and `error(Pid, Term)` answers, so those outcomes do not enter `state_3/2`.

The working state calls the `answer/5` helper defined below and adds the toplevel actor's pid to the outward-facing answer:

```

state_2(Goal, Template, Offset, Limit, Pid, Answer) :-
    answer(Goal, Template, Offset, Limit, Answer0),
    add_pid(Answer0, Pid, Answer).

```

```

add_pid(success(Slice, More), Pid, success(Pid, Slice, More)).
add_pid(failure(Pid, failure(Pid)), Pid, failure(Pid)).
add_pid(error(Term), Pid, error(Pid, Term)).

```

When an answer reports that more solutions may remain, the actor enters `state_3/2`:

```

state_3(Limit, Target) :-
    receive({
        '$next'(Options) ->
            (   option(limit(NewLimit), Options)
              -> nb_setarg(1, Limit, NewLimit)
              ;   true
            ),
            (   option(target(NewTarget), Options)
              -> nb_setarg(1, Target, NewTarget)
              ;   true
            ),
            fail ;
        '$stop' -> true
    }).

```

After '\$next', the call to `fail/0` backtracks into `state_2/6` and resumes the suspended `findnsols/4` computation. The updated limit controls the next slice, while the updated target determines where its answer is sent.

If '\$stop' is received instead, the current search is abandoned. A persistent session returns to **s1**; a non-session toplevel then terminates.

This compact protocol assumes that '\$next' and '\$stop' belong to the currently suspended query. Because those messages carry no query reference, a stale control message can otherwise be deferred in **s1** and later affect another query. A robust multi-client interface must therefore enforce one correctly sequenced interaction per toplevel, or extend the control messages with a per-query correlation reference.

The client-side operations translate directly into protocol messages. For example:

```
toplevel_call(Pid, Goal) :-
    toplevel_call(Pid, Goal, []).

toplevel_call(Pid, Goal, Options) :-
    Pid ! '$call'(Goal, Options).
```

The predicates `toplevel_next/1-2` and `toplevel_stop/1` are defined similarly, by sending \$next and \$stop messages. These operations do not wait for an answer: replies are sent to the selected target, while selective receive in the toplevel determines when each request is admissible. Abort and halt instead use the runtime's signal path and are described with the toplevel state below.

5.3.2 Why fail/0 resumes the search

SWI-Prolog's `findnsols/4` returns at most a requested number of solutions, but is itself nondeterministic: backtracking asks it for the next slice.² Together with `offset/2`, it gives the small slicing operation used by `state_2/6`.³

```
slice(Goal, Template, Offset, Limit, Slice) :-
    findnsols(Limit, Template, offset(Offset, Goal), Slice).
```

The cleanup argument of `call_cleanup/2` runs when the call becomes deterministic. We use that fact to distinguish a final slice from one whose suspended computation may still produce another.

```
answer(Goal, Template, Offset, Limit, Answer) :-
    catch(
        call_cleanup(
            slice(Goal, Template, Offset, Limit, Slice),
            Det = true
        ),
        Error,
        true
```

² https://www.swi-prolog.org/pldoc/doc_for?object=findnsols/4

³ https://www.swi-prolog.org/pldoc/doc_for?object=offset/2


```

),
(  nonvar(Error), Error == '$abort_goal'
->  throw('$abort_goal')
;   Slice == []
->  Answer = failure
;   nonvar(Error)
->  Answer = error(Error)
;   var(Det)
->  Answer = success(Slice, true)
;   Answer = success(Slice, false)
).

```

Thus the `fail/0` in `state_3/2` backtracks through `state_2/6` into the same `findnsols/4` call. Its remaining choice points are the toplevel's implicit state. The compound count(*N*) is retained with that computation, so changing its argument with `nb.setarg/3` changes the size of the next slice without restarting the goal. The reserved abort exception is rethrown because it is a protocol control signal, not an error produced by the submitted goal.

5.3.3 Production lifecycle management

The sketch leaves resource governance to the surrounding runtime. A toplevel can otherwise remain indefinitely in `s1`, `s2`, or `s3`, and a client-side cut cannot tell it that a suspended search has been abandoned. Production systems therefore add running and idle timeouts, parent-child lifetime containment, and `once(true)` for calls that should discard their remaining choice points after the first slice. A client that abandons an ordinary resumable call should use `toplevel_stop/1`, `toplevel_abort/1`, or `toplevel_halt/1`, according to the actor's current state and the desired outcome.

5.3.4 The enclosing state: abort and halt

The three predicates above implement the inner states. Transitions attached to their enclosing state, **PTCP**, apply regardless of which inner state is active. The `abort` transition interrupts the current computation and re-enters `s1` without terminating the actor:

```

ptcp(Pid, Target, Session) :-
  catch(
    state_1(Pid, Target, Session),
    '$abort_goal',
    ptcp(Pid, Target, Session)
  )

```

).

This is why `answer/5` must rethrow `'$abort_goal'` instead of packaging it as an error answer. The `catch/3` shown here is a privileged host-level catcher inside the behaviour implementation; it is not the filtered `catch/3` exposed to submitted Web Prolog code. This PTCP boundary is the only layer permitted to consume the reserved abort control. Every client-reachable catcher propagates it, and ordinary application exceptions remain catchable in the usual way.

The local backend delivers this control through the host thread that belongs to the actor. The public predicate hides that private pid-to-thread mapping:

```
toplevel_abort(Pid) :-
  (   pid_thread(Pid, Thread)
  -> catch(
        thread_signal(Thread, throw('$abort_goal')),
        error(existence_error(_, _), _),
        true
      )
  ;   true
  ).
```

Application programs still address the actor pid, not a host thread identifier. The portable meaning of abort is the protected unwind to the PTCP boundary above; `pid_thread/2` and `thread_signal/2` belong to the local runtime rather than the actor API.

The `halt` transition, by contrast, leaves the enclosing **PTCP** state and terminates the actor. It is implemented with the general actor exit mechanism rather than a mailbox command, so it can terminate the toplevel while it is waiting in **s1** or **s3**, or while a goal is executing in **s2**. This is the operational counterpart of attaching `halt` to the enclosing state rather than to any one inner state. In particular, no separate transition from **s1** to the final state is required.

The one-argument operation requests termination immediately. The two-argument form first installs a private monitor and then waits for its matching notification:

```
toplevel_halt(Pid) :-
  exit(Pid, true).
```

```

toplevel_halt(Pid, Reply) :-
    setup_call_cleanup(
        monitor(Pid, Ref),
        (   toplevel_halt(Pid),
            receive({
                down(Pid, Ref, connection_closed) ->
                    Reply = error(connection_closed);
                down(Pid, Ref, no_process) ->
                    Reply = true;
                down(Pid, Ref, _) ->
                    Reply = true
            })
        ),
        demonitor(Ref, [flush])
    ).

```

Installing the monitor before calling `exit/2` avoids losing a fast termination. The exit signal itself is asynchronous;⁴ A matching notification with a termination reason confirms observed termination, even if the actor exited for a reason other than the requested `true`. The special reason `no_process` confirms that the target is already absent and also yields `Reply = true`; it does not establish when or why it terminated. For a remote target, `connection_closed` means that observation was lost, so the reply is `error(connection_closed)` and termination remains unconfirmed. Making `monitor/2` the setup action ensures that the private monitor is removed even if the wait is interrupted, while the `flush` option removes any late notification bearing its fresh reference and leaves notifications from other monitors untouched.

5.4 The `rpc/2-3` meta-predicate

Chapter 4 introduced `rpc/2-3` as the central construct of the sequential Prolog Web. Its interface resembles an ordinary predicate call: variables are bound on success, failure is reported as Prolog failure, exceptions are rethrown, and backtracking requests further solutions. The fact that the goal is evaluated on another node is hidden behind this familiar interface.

In the terminology introduced at the beginning of this chapter, `rpc/2-3` is a Prolog-style generic. It does not expose a message protocol to its caller. Instead, it encapsulates remote execution, answer collection, slicing, and transport behind a nondeterministic predicate interface.

There are two useful ways of implementing this abstraction. The first reconstructs it on top of a remote `toplevel` actor and makes the underlying PTCP interaction

⁴ The local implementation uses `thread_signal/2`, which queues its goal and returns immediately: https://www.swi-prolog.org/pldoc/doc_for?object=thread_signal/2.

explicit. The second uses the stateless HTTP API and is closer to the implementation provided by the demonstrator. They expose the same logical interface but make different operational and deployment trade-offs.

5.4.1 From protocol messages to Prolog nondeterminism

A toplevel actor communicates through deterministic messages. A call produces one of a small number of protocol outcomes: a slice of solutions, failure, or an error. If further solutions remain, the client may explicitly request the next slice. Nothing in this protocol is itself nondeterministic.

The task of `rpc/2-3` is to turn that deterministic exchange into ordinary Prolog control. Each received slice is enumerated locally using `member/2`. When a slice is exhausted, the choice point associated with `rpc/2-3` requests another slice. Failure of the remote goal becomes local failure, while a remote error becomes a local exception.

Two distinct forms of continuation are therefore involved. Within a slice, the continuation is an ordinary local Prolog choice point. Between slices, continuation requires another protocol exchange with the remote node. The caller does not see this distinction: pressing semicolon or backtracking into the call simply asks for the next solution.

This translation from answer messages to choice points is what makes `rpc/2-3` more than an HTTP client or a request-response wrapper. It reconstructs a remote stream of substitutions as a predicate participating directly in the caller's proof search.

5.4.2 An actor-based reconstruction

The most direct reconstruction uses the toplevel behaviour described in the preceding section. It spawns a toplevel actor on the target node, submits the goal, and then waits for PTCP answer messages. The toplevel is created with `session(false)` because it is intended to serve this one computation rather than remain available for an interactive sequence of queries.

The following code is a pedagogical reconstruction. It exposes the essential control structure but is not the implementation used by the demonstrator:

```
rpc(URI, Goal) :-
    rpc(URI, Goal, []).

rpc(URI, Goal, Options) :-
    toplevel_spawn(Pid, [
        node(URI),
        session(false),
```

```

        monitor(false)
    | Options
  ]),
  toplevel_call(Pid, Goal, Options),
  wait_answer(Pid, Goal).

```

On the ordinary protocol outcomes, the answer loop terminates on failure or error, or after the final slice has been consumed:

```

wait_answer(Pid, Goal) :-
  receive({
    success(Pid, Slice, true) ->
      (
        member(Goal, Slice)
      ;
        toplevel_next(Pid),
        wait_answer(Pid, Goal)
      ) ;
    success(Pid, Slice, false) ->
      member(Goal, Slice) ;
    failure(Pid) ->
      fail ;
    error(Pid, Error) ->
      throw(Error)
  }).

```

The two clauses handling `success/3` correspond to the two possible kinds of slice. If the final argument is `false`, the slice contains the last solutions and can simply be enumerated. If it is `true`, exhausting the local enumeration falls through to `toplevel_next/1`, after which the same answer loop is entered again. The recursion thus occurs only at slice boundaries; no network communication is needed while solutions remain in the current slice.

This is a direct use of the `receive/1-2` contract in Appendix B: selecting a message is committed, but the selected clause body may yield several solutions. Here `member/2` leaves the choice points that make `rpc/2-3` nondeterministic, and the recursive call receives another answer message only after the current slice is exhausted.

Despite being implemented with actors, the resulting abstraction is synchronous. The caller and callee proceed in request–response lockstep, and the caller performs no independent work while waiting for the next answer. The actors supply the mechanism, but the predicate presents a sequential interface.

This reconstruction also exposes lifecycle problems that the predicate interface would otherwise conceal. It does not monitor the toplevel or bound the receive operation, so a node failure or network partition may leave the caller waiting indefinitely. More subtly, the caller may abandon the computation by cutting away the `rpc/2-3` choice point. If the remote toplevel has reported that more solutions are available, it may then remain suspended waiting for a request that will never arrive.

The option `session(false)` does not by itself solve this abandonment problem. It terminates the actor when the query has been exhausted or explicitly stopped;

it cannot detect that a choice point has disappeared on another node. A production actor-based implementation would therefore need failure detection, bounded waiting, and cleanup that associates the lifetime of the remote toplevel with the lifetime of the nondeterministic call.

5.4.3 The stateless HTTP implementation

The request–response structure just described maps naturally onto HTTP. The caller submits a goal and receives a slice of solutions. It sends another request only after local backtracking has exhausted that slice. There are no unsolicited answers or server-initiated notifications, and the remote node does not need to expose its programmable actor interface.

Each exchange can therefore be represented as a self-contained request to the node's `/call` endpoint. The request carries the goal, an answer template, an offset, a limit, and the desired representation format. The response is a success, failure, or error term of the same general form used by the toplevel protocol.

The implementation below shows the central mechanism. It omits some of the validation, option processing, authentication, and error handling required by the complete demonstrator:

```
rpc(URI, Goal) :-
    rpc(URI, Goal, []).

rpc(URI, Goal, Options) :-
    parse_url(URI, Parts),
    term_variables(Goal, Vars),
    Template =.. [v|Vars],
    format(atom(GoalAtom), "~p", [Goal]),
    format(atom(TemplateAtom), "~p", [Template]),
    option(limit(Limit), Options, 100000000000),
    Request = request(
        Parts, GoalAtom, TemplateAtom, Limit, Options
    ),
    rpc_page(Template, 0, Request).

rpc_page(Template, Offset, Request) :-
    Request = request(
        Parts, GoalAtom, TemplateAtom, Limit, Options
    ),
    parse_url(URI, [
        path('/call'),
        search([
            goal=GoalAtom,
            template=TemplateAtom,
```

```

        offset=Offset,
        limit=Limit,
        format=prolog
    })
    | Parts
  ]),
  setup_call_cleanup(
    http_open(URI, Stream, Options),
    read(Stream, Answer),
    close(Stream)
  ),
  rpc_answer(Answer, Template, Offset, Request).

rpc_answer(success(Slice, true), Template, Offset, Request) :-
  !,
  (   member(Template, Slice)
  ;   Request = request(_, _, _, Limit, _),
      NextOffset is Offset + Limit,
      rpc_page(Template, NextOffset, Request)
  ).
rpc_answer(success(Slice, false), Template, _, _) :-
  member(Template, Slice).
rpc_answer(failure, _, _, _) :-
  fail.
rpc_answer(error(Error), _, _, _) :-
  throw(Error).

```

The public predicate first constructs a template containing the variables whose bindings must be returned. Goals and templates are serialised before being added to the HTTP request. The answer dispatcher then translates the response back into Prolog control: `member/2` enumerates a received slice, failure becomes failure, and an error term is thrown as an exception. When a non-final slice is exhausted, the offset is advanced and another HTTP request is made.

As in the toplevel sketch, `10000000000` is the documented default slice limit. The offset-based reconstruction may re-execute the goal to reach later slices, repeating both earlier work and any side effects performed along the way. Section E.2.2 of Appendix E explains the demonstrator's bounded continuation cache and the circumstances in which recomputation is still required.

The HTTP interface is stateless in the sense that every request contains the information needed to identify the requested portion of the answer sequence. This does not require the server to discard all execution state after each response. The demonstrator may cache a suspended toplevel and resume it when a later request asks for another slice. Such a cached computation is an internal optimisation rather than a client-addressed session: it may be reused, expired, or evicted without changing the public protocol.

This distinction is important when a caller abandons enumeration. No explicit stop request is required by the HTTP interface, because the caller does not own an addressable remote session. A cached computation may nevertheless remain unused after the final request received from that caller. The node must therefore bound its cache and terminate suspended computations on expiry, eviction, or resource pressure.

The apparent simplicity of the HTTP control structure should not obscure the transport work surrounding it. A complete implementation must serialise terms unambiguously, preserve variable relationships through templates, construct URIs correctly, validate options, enforce profiles and authorisation, and deal with TLS, redirects, authentication, timeouts, malformed responses, and connection failures.

5.4.4 Comparing the two realisations

The actor and HTTP realisations implement the same predicate-level contract. Given the same remote program and the same successful sequence of protocol exchanges, they return the same substitutions in the same order. Both enumerate answers locally within a slice and perform remote communication only when another slice is required.

Their operational boundaries are different. The actor reconstruction creates an addressable remote process and communicates with it using PTCP. This makes the relationship with the toplevel behaviour particularly clear, but it requires an ACTOR node and introduces an explicit remote lifecycle. The HTTP implementation requires only the sequential query interface. It avoids a client-addressed actor and can operate between ISOBASE nodes, but it must encode the computation in HTTP requests and rely on server-managed caching when efficient resumption is desired.

The actor version is consequently the better explanatory model: it shows how deterministic answer messages become nondeterministic Prolog execution. The HTTP version is the more practical implementation for the sequential Prolog Web: its interaction pattern fits ordinary web infrastructure and does not require callers to participate in actor lifecycle management.

Neither realisation changes the logical meaning of `rpc/2-3`. The observable abstraction remains a remote predicate call over renamed-apart terms. The differences concern transport, resource ownership, failure detection, and the management of suspended computation.

The next sections consider two further Prolog-style generics, `parallel/1` and `first_solution/2-3`. Like `rpc/2-3`, they hide actor coordination behind conventional predicate interfaces. Unlike `rpc/2-3`, however, they commit to selected solutions rather than exposing a remote computation nondeterministically on backtracking.

5.5 The parallel/1 meta-predicate

In this section we implement `parallel/1`, the meta-predicate introduced in Chapter 4 (Section 4.2.5). A call to `parallel(Goals)` should wait until its outcome is known and its workers have been cleaned up, succeed with the returned variable bindings if all goals succeed, fail promptly if any goal fails, and promptly rethrow an exception raised by any goal. These guarantees require the goals to be independent: they must not communicate through shared variables, global state, or other side channels.

The predicate is semideterministic. Each goal contributes its first success, if it has one, and `parallel([])` succeeds immediately. From the caller's perspective this is a conventional predicate-level interface; underneath, one monitored actor is used for each goal.

The listings below assume that `Goals` is a proper list and omit the public predicate's list and callability validation. The complete interface raises the errors specified in Appendix B.

5.5.1 From two goals to a list

We begin with a deliberately incomplete two-goal version:

```
parallel(Goal1, Goal2) :-
    self(Self),
    spawn((call(Goal1), Self ! Pid1-Goal1), Pid1),
    spawn((call(Goal2), Self ! Pid2-Goal2), Pid2),
    receive({Pid1-Goal1 -> true}),
    receive({Pid2-Goal2 -> true}).
```

Each worker calls its copied goal and, on success, sends its pid paired with the instantiated copy back to the parent. Selecting that message unifies the copy with the corresponding goal in the caller and thereby returns its bindings. The two replies may arrive in either order: selective receive defers the second worker's reply if it arrives before the first one being awaited.

The same pattern generalises naturally from two goals to a list:

```
parallel(Goals) :-
    maplist(par_solve, Goals, Pids),
    maplist(par_yield, Pids, Goals).

par_solve(Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Goal), Pid).

par_yield(Pid, Goal) :-
```

```
receive({Pid-Goal -> true}).
```

This version works when every goal succeeds. If a goal fails or throws an exception, however, its worker sends no result and the corresponding `receive/1` waits indefinitely. Monitors supply the missing outcome.

5.5.2 Detecting failure and errors

Each worker is now monitored, and `par_yield/3` accepts either the expected result or an abnormal termination notification from any worker belonging to this invocation:

```
parallel(Goals) :-
    maplist(par_solve, Goals, Pids),
    maplist(par_yield(Pids), Pids, Goals).

par_solve(Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Goal), Pid, [
        monitor(true)
    ]).

par_yield(Pids, Pid, Goal) :-
    receive({
        Pid-Goal ->
            receive({
                down(Pid, _, true) ->
                    true
            }) ;
        down(FailedPid, _, false)
            if memberchk(FailedPid, Pids) ->
                tidy_up_all(Pids),
                fail ;
        down(FailedPid, _, exception(E))
            if memberchk(FailedPid, Pids) ->
                tidy_up_all(Pids),
                throw(E)
    }).
```

On success, the result message is followed by a normal `down/3` notification, which consumes the worker's monitor. A failure or exception from any worker determines the outcome of the whole call, so the remaining workers are cleaned up immediately by `tidy_up_all/1`, defined in the next subsection. Matching only the current `Pid` in the abnormal clauses would unnecessarily delay detection until that particular worker was awaited.

There is a semantic cost to this promptness. If several workers fail or throw different exceptions, the first matching `down/3` notification in the mailbox determines the result. That outcome may depend on scheduling and need not be the outcome of the leftmost abnormal goal, which ordinary sequential conjunction would encounter first. The implementation is therefore equivalent to sequential conjunction only on the normal and single-abnormal cases stated in Section 4.2.5. This first-observed rule is the Web Prolog contract: it favours prompt failure and cancellation over deterministic leftmost precedence.

The guards instead admit notifications from every pid in `Pids` while leaving unrelated monitor messages in the caller's mailbox. This matters because `parallel/1` may be called by an actor that is monitoring other processes at the same time. Once a message and clause have been selected, backtracking cannot enter the receive machinery again; only choice points left by the selected body remain available.

5.5.3 Race-free cleanup

Cleanup must do more than kill the remaining workers. It must also ensure that neither result messages nor monitor notifications from this invocation are left behind in the caller's mailbox:

```
tidy_up_all(Pids) :-
    maplist(tidy_up, Pids).

tidy_up(Pid) :-
    demonitor(Pid, [flush]),
    monitor(Pid, CleanupRef),
    exit(Pid, kill),
    receive({
        down(Pid, CleanupRef, _) -> true
    }),
    drain_mailbox(Pid).

drain_mailbox(Pid) :-
    receive({
        Msg if arg(1, Msg, Pid) ->
            drain_mailbox(Pid)
    }, [
        timeout(0)
    ]).
```

The creation-time monitor is first removed and any notification it has already produced is flushed. The spawn-time monitoring contract uses the worker pid as that monitor's reference, which is why the first argument to `demonitor/2` is `Pid`. A private cleanup monitor is then installed before the exit request is sent.

These workers and their caller are local, and results are sent immediately to the caller's pid. The proof therefore uses the local enqueue and lifecycle guarantees in Section 4.1.7. Every result that survives cancellation is enqueued before the worker's termination notification is enqueued. Retiring the creation-time monitor also prevents a late notification from that old monitor. After receiving the new monitor's `down/3`, cleanup can thus drain all remaining messages from this worker with a zero-time receive: no worker result or old monitor notification can arrive afterwards.

If the worker has already exited when cleanup installs its monitor, the lifecycle mutex makes installation observe either the still-published actor or its retirement. The former case receives a termination notification; the latter receives `no_process`. In both cases, any earlier result is already in the caller's mailbox. The caller may have consumed some results already; the drain removes only those still pending. No ordering between the cleanup monitor installation and the worker's result send is needed.

This proof does not extend to remote workers or delayed result sends. In particular, `connection_closed` cannot justify a drain. A distributed version would need a delivery protocol that orders a final completion marker after all results on the same stream, with explicit handling of transport loss; merely adding a monitor would not establish that ordering.

The drain terminates because `receive/2` calls `true` by default when its zero timeout expires. Its guard deliberately applies `arg(1, Msg, Pid)` to every pending message: if `Msg` is an atom, `arg/3` raises an error, and the receive guard's error-as-non-match rule leaves that unrelated message deferred rather than aborting cleanup.

5.5.4 Behaviour and performance

We can now check success, failure, and exception propagation. The following three sleeps overlap, so the call takes approximately as long as the slowest goal rather than the sum of all three delays:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,sleep(3)),(Z=c,sleep(2))],
    time(parallel(_Goals)).
% 189 inferences, 0.000 CPU in 3.006 seconds
X = a,
Y = b,
Z = c.
?-
```

On this example the elapsed time is about three seconds, compared with about six seconds for sequential execution.

If one goal fails, the entire call fails without waiting for slower siblings:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,fail),(Z=c,sleep(2))],
    time(parallel(_Goals)).
```

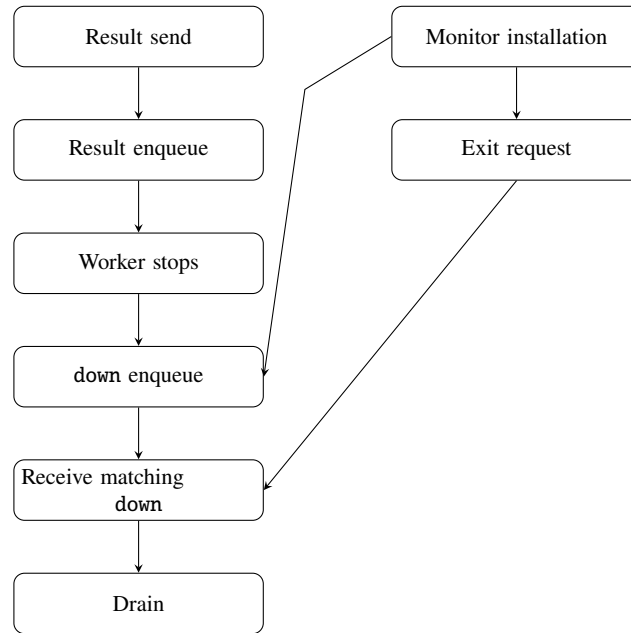


Fig. 5.3 Required local cleanup ordering when a result is sent. Arrows express required precedence; placement alone imposes none. Monitor installation and exit request are unordered with respect to result delivery and worker termination: the worker may already have exited. The path from result enqueue through worker termination to down enqueue is the local runtime guarantee, including a late monitor's no_process. If cancellation prevents the result, the first two events are absent.

```
% 105 inferences, 0.000 CPU in 0.001 seconds
false.
?-
```

An exception is likewise propagated after the other workers have been cleaned up:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,sleep(a)),(Z=c,sleep(2))],
    time(catch(parallel(_Goals),E,true)).
% 126 inferences, 0.000 CPU in 0.001 seconds
E = error(type_error(float, a), _).
?-
```

Actor creation, monitoring, message copying, and cleanup all have a cost. Goals must therefore perform enough work for their overlap to repay that cost. This implementation also creates one actor per goal; very large goal lists consequently have a correspondingly large resource footprint.

5.5.5 Relation to `concurrent/3`

SWI-Prolog's `library(thread)` provides the related predicate `concurrent/3`.⁵ Its observable success, failure, exception, and variable-binding behaviour is close to that of `parallel/1`, but the interfaces serve somewhat different purposes. The first argument of `concurrent/3` bounds the number of worker threads, and its third argument supplies thread-creation options. By contrast, `parallel/1` deliberately presents a narrower interface and creates one actor for each goal.

The compact implementation developed here should therefore not be read as a replacement for, or a simplification of, `concurrent/3`. Rather, it demonstrates how the actor primitives of Web Prolog can be composed to provide a familiar parallel-conjunction abstraction with predicate-level variable bindings. The two predicates have closely related semantics but make different choices about resource control and the abstractions exposed to the programmer.

5.6 The `first_solution/2-3` meta-predicate

SWI-Prolog's `library(thread)` also provides `first_solution/2-3`: run several alternative solvers concurrently and return the result produced by the first one to succeed.⁶ The Web Prolog version adopts this predicate-level idea but implements the race with monitored actors. Once a winner has been selected, all remaining workers are terminated and their messages are removed from the caller's mailbox.

5.6.1 Racing alternative solvers

Borrowing an example from Wielemaker, suppose that it is unclear whether a graph should be searched breadth-first or depth-first. We can start both strategies and accept whichever finds a path first:

```
search_graph(Graph, Path) :-
    first_solution(Path, [ breadth_first(Graph, Path),
                          depth_first(Graph, Path)
                        ],
    []).
```

Unlike `parallel/1`, which requires every goal to succeed, `first_solution/2-3` treats its goals as alternatives. Their copies share the same solution template, and the first result message selected by the caller binds that template.

⁵ <https://www.swi-prolog.org/pldoc/man?section=thread>

⁶ <https://www.swi-prolog.org/pldoc/man?section=thread>


```
}}).
```

Every receive head is guarded by membership in `Pids`. An unrelated pair in the caller's mailbox must not be mistaken for a winning result, and a `down/3` notification from an unrelated monitor must not affect the race. On success, `tidy_up_all/1` from the implementation of `parallel/1` supplies the same ordering barrier and race-free mailbox cleanup described in the previous section.

If no worker remains, the call fails. The predicate is therefore semideterministic: it returns the first solution selected by the mailbox but does not enumerate later solutions on backtracking.

5.6.2 Failure and error policies

The option `on_fail(continue)` removes a failed solver and keeps waiting; `on_fail(stop)` instead cleans up all workers and fails the entire call immediately. Similarly, `on_error(continue)` ignores an exception from one solver while alternatives remain, whereas `on_error(stop)` cleans up and rethrows it. The accepted values are thus `continue` and `stop`; validation and the associated domain errors are omitted from this design sketch.

The defaults make failure local to an alternative but treat an exception as significant. This is often useful when the solvers are different search strategies: one strategy may simply find no answer, while an exception may indicate a defect or resource problem that should be reported. Applications can select different policies explicitly through `first_solution/3`.

If all alternatives fail under `on_fail(continue)`, or if all remaining alternatives are discarded under `on_error(continue)`, the worker list eventually becomes empty and the whole call fails.

5.6.3 An example

Using a simple simulation of two long-running goals, here is a trial run that shows that our predicate works as intended:

```
?- time(first_solution(X, [(sleep(2),X=a),(sleep(1),X=b)])).
% 164 inferences, 0.000 CPU in 1.006 seconds
X = b.
?-
```

The second solver wins after approximately one second. Cleanup then terminates the slower worker before returning `X = b` to the caller.

5.7 Summary

The examples in this chapter illustrate two complementary ways of packaging concurrent computation in Web Prolog. Protocol-oriented generics such as the server, supervisor, and toplevel actor expose stable message protocols and hide recurring operational machinery for state management, lifecycle control, monitoring, and query execution. They do not expose their internal proof search through caller backtracking. Predicate-oriented generics such as `rpc/2-3`, `parallel/1`, and `first_solution/2-3`, by contrast, hide protocol exchanges or actor coordination behind conventional predicate interfaces.

A recurring design principle is the separation of application logic from coordination. The server callback describes how one request transforms application state; child specifications describe the topology managed by a supervisor; a toplevel actor executes submitted Prolog goals; `rpc/2-3` turns answer slices into caller-side non-determinism; and the parallel generics receive goals or alternative solvers directly as terms. The surrounding generic supplies the message protocol, process lifecycle, correlation, monitoring, transport, and cleanup appropriate to each case.

That separation does not mean that arbitrary Prolog code can be moved into a concurrent or distributed setting unchanged. Server upgrades must preserve or migrate state representations; parallel goals must satisfy their independence requirements; remotely executed RPC goals must be available and authorised on the target node; and implementations must account for resource limits, transport failure, and messages that race with process termination. The design sketches make the reusable structure visible, while the manual specifies the public contracts adopted by the demonstrator.

Several techniques recur across otherwise different abstractions. Fresh references correlate requests and replies. Selective receive lets a process react to whichever relevant outcome arrives first while deferring unrelated messages. Monitors turn silent process termination into explicit data. Acknowledgements distinguish an accepted operation from a message merely placed in transit. Cleanup monitors provide ordering barriers before mailbox draining. In the toplevel actor, controlled backtracking and mutable cells support demand-driven slices of solutions; in `rpc/2-3`, local choice points turn those slices back into a nondeterministic predicate. These examples show ordinary Prolog mechanisms and actor mechanisms reinforcing one another.

The result is not a claim that Web Prolog replaces OTP or SWI-Prolog's thread library. The server and supervisor show that established actor patterns can be expressed naturally in Web Prolog, while the toplevel, RPC, and parallel generics show how protocol exchanges and actor orchestration can be hidden behind interfaces shaped by Prolog's execution model. Together they demonstrate the central point of the chapter: combining goals, unification, and backtracking with process identity, message passing, and monitoring makes it possible to construct abstractions that remain recognisably Prolog while operating concurrently and, where required, across the Web.

Chapter 6

The statechart behaviour

The earlier chapters introduced several generic behaviours for actors in Web Prolog: the toplevel behaviour, the server behaviour, and the supervisor behaviour. Each separates recurring operational machinery from the application logic of an actor. This chapter introduces the *statechart behaviour*, in which the control logic of a single actor is represented explicitly using hierarchical states, parallel regions, and event-triggered transitions.

The behaviour is expressed in Statechart XML, or SXML, a compact XML notation whose tree structure mirrors the hierarchical structure of a statechart. An SXML document is directly executable as a Web Prolog actor: it can be launched with `statechart_spawn/2`, participate in a supervision tree, and exchange messages like any other actor. Events and data are Prolog terms, while guards and actions are ordinary Web Prolog code. The associated predicates form the `statechart` family of the ACTOR profile; a node owner may enable or disable that family as part of the node's advertised offer.

6.1 STATECHARTS

Invented by David Harel, STATECHARTS is a graphical formalism for specifying reactive systems (Harel, 1987). Building on the concept of a state machine, it adds hierarchy, parallel regions (orthogonality), and event visibility across the active states and regions. Like logic, it is a “tool for thinking” with a formal semantics (Harel and Naamad, 1996), but it is also explicitly visual – a way of “tapping the potential of high bandwidth spatial intelligence, as opposed to lexical intelligence used with textual information” (Samek, 2002).

Statecharts have been used in areas ranging from embedded control and user interfaces to conversational systems and communication protocols (Samek, 2002; Horrocks, 1999; Skantze and Moubayed, 2012; Cicirelli et al., 2009).

SXML is a lightweight textual adaptation of the statechart formalism, with Prolog as both its data model and executable content. Its surface syntax resembles SCXML, but it is intentionally not a conforming SCXML language binding. No familiarity with SCXML is assumed in this chapter. Section 6.10.5 explains the rationale for the adaptation, while Appendix C.11 records which SCXML constructs SXML retains, adapts, or omits.

6.2 Statechart actors

A *statechart actor* is a Prolog actor whose control logic is given by a statechart document. Conceptually, it behaves like an ordinary actor with a mailbox, except that incoming messages are interpreted as *events* that drive a structured state machine rather than an ad hoc `receive/1-2` loop.

Events are just messages: any incoming actor message (a Prolog term) can serve as an external event. When an event is processed, it is *visible* to all currently active parts of the machine (including parallel regions), and it may trigger one or more transitions whose actions run as Web Prolog code.

Execution follows the standard statechart *run-to-completion* intuition: processing one external event may trigger a cascade of internal work, such as taking eventless transitions and handling internally raised events, before the actor consumes the next external message. A well-formed reaction is expected to terminate; a later example shows what happens when it does not. This makes each terminating reaction behave like an atomic control step at the statechart level, even though the actor itself remains an ordinary concurrent process.

In the examples below, a statechart document can therefore be read as a compact description of (i) which states are active, (ii) which incoming events matter in those states, and (iii) what effects and state changes occur when such an event is observed. SXML takes its transition-selection and run-to-completion model from SCXML, while adapting event matching, executable content, and process invocation to Web Prolog. Section 6.7 summarises the resulting execution model; Appendix C gives the element-level contract.

6.2.1 A first statechart

In its simplest form, a statechart is just a deterministic state machine, in which state transitions are triggered by events. The simple statechart in Figure 6.1 serves as our

first example. The labelling of one of the transitions with the symbol `play` suggests that the example is drawn from the field of game development.¹

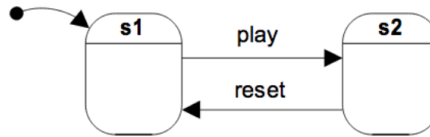


Fig. 6.1 A minimal statechart example: event-triggered transitions in a flat machine.

Here, `s1` is the initial state. If the next selected event is `play`, a transition to `s2` is taken. For any other event, the machine will remain in `s1` and the event will be discarded rather than deferred.

Statecharts within the SXML profile can be translated into documents written in an XML-based syntax. Here, a document that captures the statechart in Figure 6.1 is shown:

```

<statechart version="0.2" initial="s1">
  <state id="s1">
    <onentry>
      output('IDLE')
    </onentry>
    <go to="s2" on="play"/>
  </state>
  <state id="s2">
    <onentry>
      output('PLAYING')
    </onentry>
    <go to="s1" on="reset"/>
  </state>
</statechart>

```

The required attribute `version="0.2"` identifies the SXML notation accepted by the demonstrator. Note also that two `<onentry>` elements, the significance of which is not shown in Figure 6.1, have been added to the document. They contain *executable content* – instructions which allow the state machine to *do* things such as modifying its data model or interacting with external entities. Executable content must be specified using Web Prolog source code. In the case of the above example, the Web Prolog built-in predicate `output/1` sends `output(Pid, 'IDLE')` or `output(Pid, 'PLAYING')` to the spawning process each time `s1` or `s2` is entered, exactly as `output/1` does in other actors. Executable content may also appear in `<onexit>` and `<go>` elements.

¹ We are borrowing some of our examples from (Brusk and Lager, 2008).

6.2.2 Running a statechart actor

An SXML document is executed by an ordinary Prolog Web actor, either in a browser or on a node accessed from a browser. To create such an actor, we use `statechart_spawn/2`:

```
?- statechart_spawn(Pid, [
    src_uri('http://n6.org/game.xml'),
    monitor(true)
]).
Pid = 78230912.
?- flush.
Shell got output(78230912,'IDLE')
true.
?-
```

When `statechart_spawn/2` is called, a new actor is spawned that interprets the statechart. As specified by the `<onentry>` element in the initial state, the statechart actor immediately sends `output(78230912,'IDLE')` to its parent. Equipped with the pid, we can use `!/2` to send an event to the actor, causing a transition from `s1` to `s2`. In general, an event may be any transportable Web Prolog term; here, the relevant `on` attribute tells us that the atom `play` will do:

```
?- $Pid ! play.
true.
?- receive({output($Pid, What) -> true}).
What = 'PLAYING'.
?-
```

We may now send the event `reset` to return to `s1`, and so on. For orderly termination, a client uses `statechart_halt/2`. This lets the interpreter run the `<onexit>` content of active states before replying. The form `statechart_halt/3` bounds the wait and forcibly terminates the actor if orderly shutdown does not complete within the specified time. The lower-level `exit/2` operation remains available when immediate actor termination is required.

Since the `monitor` option was set to `true`, the runtime also notifies the parent with a `down/3` message when the actor terminates, exactly as it does for an ordinary actor.

During development, the `trace/1` option can expose event selection, transitions, configurations, and invocation. Appendix C.10 describes the emitted trace terms and their diagnostic use.

Other options to `statechart_spawn/2` work as expected. For example, we can run the statechart on a remote node by passing a `node(URI)` option, and we can provide the source directly using `src_text` rather than fetching it from a URI.

The examples in the rest of this chapter are organised around a small set of extensions that make statecharts practically useful as an actor behaviour on the

Prolog Web. Each extension adds a distinct feature – some at the state-machine level, others at the level of interaction with the surrounding actor system. Concretely, the extensions we rely on are:

- Hierarchy and history states
- Parallelism (aka orthogonality)
- Broadcast communication
- Process invocation
- Inter-process communication
- Data model and scripting language

In the sections that follow, the left column highlights statechart-level structuring mechanisms, while the right column highlights how those mechanisms connect to actors and Prolog in the Web Prolog setting.

6.3 Hierarchy and history

Statecharts may be *hierarchical*: a state may contain another statechart down to an arbitrary depth. A complex state may contain a *history state*, serving as a memory of which substate *S* the complex state was in, the last time it was left for another state. Transition to the history state implies a transition to *S*.

The statechart in Figure 6.2 exemplifies both hierarchy and the history state by enhancing our simple game with a pause-and-resume functionality.

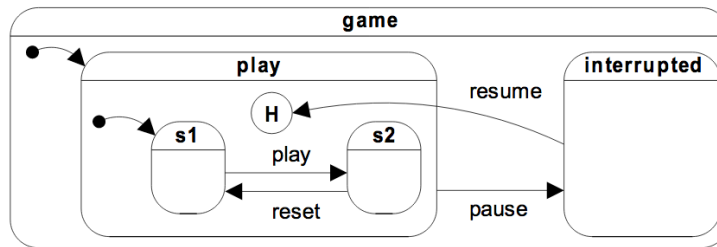


Fig. 6.2 Pause-and-resume via hierarchy and history state in the game controller.

Suppose that the current state is **s2**, and that the next selected event is **pause**. The transitions available in **s2** are considered from the inside out. The **reset** transition in **s2** does not match, but the **pause** transition inherited from the enclosing **play** state does, so a transition to **interrupted** takes place. If a **resume** event then appears, the system transitions to the history state **H**, which returns it to **s2**.

Documents are structured so that they mirror the topology of the statecharts they represent. A document that captures the statechart in Figure 6.2 is shown below:²

² A history state is shallow by default: it remembers the immediate child that was active. The optional attribute `type="deep"` instead remembers the complete active descendant configuration. The pause-and-resume example cannot distinguish the two because **s1** and **s2** are already atomic states.

```

<statechart version="0.2" initial="game">
  <state id="game" initial="play">
    <state id="play" initial="s1">
      <history id="H">
        <go to="s1" />
      </history>
      <state id="s1">
        <go to="s2" on="play" />
      </state>
      <state id="s2">
        <go to="s1" on="reset" />
      </state>
      <go to="interrupted" on="pause" />
    </state>
    <state id="interrupted">
      <go to="H" on="resume" />
    </state>
  </state>
</statechart>

```

Hierarchy supports two complementary design moves. In *refinement* (top-down), an abstract control state is elaborated into substates whose internal transitions make the behaviour explicit. In *clustering* (bottom-up), a family of closely related states is grouped under a superstate, so that shared transitions and common entry/exit behaviour can be expressed once. Both moves reduce state explosion and make large machines readable: the top level can remain a small “map” of the application, while local detail is pushed down into the regions where it belongs. Together, hierarchy and history give *interruption* and *resumption* a useful first-class representation.

6.4 Parallelism and broadcast communication

A hallmark of statecharts is *orthogonality*: a state may contain two or more regions that execute in parallel. Operationally, the machine is simultaneously in one active state in each region. The point is not CPU-level parallelism, but a way to factor one reactive controller into several facets that evolve together.

6.4.1 An NPC controller

To see why orthogonality matters, consider a toy controller for a non-player character with two independent emotion dimensions and one behaviour mode. Figure 6.3 sketches the idea: the character is simultaneously in one state in each emotion

dimension and in one behaviour state. When an event is processed, it may trigger coordinated changes across several regions.

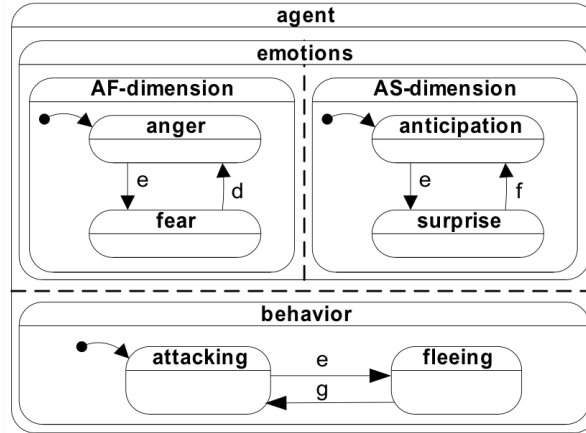


Fig. 6.3 Orthogonal regions: two emotion dimensions plus one behaviour mode in an NPC controller.

In the figure, an **e** event (think: an explosion) moves the character into **fear**, **surprise**, and **fleeing** “at once”, because all three regions observe the same event while they are active. Without parallel regions, we would be forced to encode combinations such as **anger_anticipation_attacking**, **fear_surprise_fleeing**, and so on, and the number of states grows quickly as more dimensions are added.

Here is how to render this machine in XML:

```
<statechart version="0.2" initial="agent">
  <parallel id="agent">
    <parallel id="emotions">
      <state id="AF-dimension" initial="anger">
        <state id="anger">
          <go to="fear" on="e"/>
        </state>
        <state id="fear">
          <go to="anger" on="d"/>
        </state>
      </state>
      <state id="AS-dimension" initial="anticipation">
        <state id="anticipation">
          <go to="surprise" on="e"/>
        </state>
        <state id="surprise">
          <go to="anticipation" on="f"/>
        </state>
      </state>
    </parallel>
    <state id="behavior">
      <state id="attacking">
        <go to="fleeing" on="e"/>
      </state>
      <state id="fleeing">
        <go to="attacking" on="g"/>
      </state>
    </state>
  </parallel>
</statechart>
```

```

        </state>
    </state>
</parallel>
<state id="behavior" initial="attacking">
    <state id="attacking">
        <go to="fleeing" on="e"/>
    </state>
    <state id="fleeing">
        <go to="attacking" on="g"/>
    </state>
</state>
</parallel>
</statechart>

```

6.4.2 An unbounded internal cascade

To make the broadcast mechanism completely explicit – and to expose a hazard that the mechanism makes easy to stumble into – we now switch to a deliberately minimal example: two regions that exchange events. The point is not to model two independent actors, but to show how a single statechart step can coordinate activity across regions. As we shall see, this particular machine also illustrates what goes wrong when that coordination feeds back on itself.

```

<statechart version="0.2" initial="Start">
    <parallel id="Start">
        <state id="Pinger">
            <onentry>raise(pong)</onentry>
            <go on="ping">
                writeln('Ping'), raise(pong)
            </go>
        </state>
        <state id="Ponger">
            <go on="pong">
                writeln('Pong'), raise(ping)
            </go>
        </state>
    </parallel>
</statechart>

```

Note the use of targetless `<go>` elements (elements without a `to` attribute). These transitions consume an event (and may run executable content, such as `raise/1`) without changing the active configuration, so no `<onentry>` or `<onexit>` code is triggered.

This machine does exactly what its two transitions say, and that is the problem. On entry, *Pinger* raises *pong*; the *Ponger* region consumes it and raises *ping*; the *Pinger* region consumes that and raises *pong* again; and so on without end. Because run-to-completion drains the *internal* event queue before the next *external* message is consumed (as described in the next subsection), and because each reaction replenishes that queue, the queue never empties. The actor never completes its first macrostep and never returns to consume an external event: it is livelocked. The lesson is not that broadcast or *raise/1* are unsafe, but that run-to-completion faithfully executes whatever internal cascade the document describes – including a non-terminating one. A cascade is only well-behaved if it is guaranteed to terminate. A guard that counts rounds and stops raising after a given number would fix the problem.

An orderly request made with `statechart.halt/2` cannot be processed while the actor remains trapped in such a macrostep. `statechart.halt/3` addresses this failure boundary by escalating to forced termination after its timeout.

6.4.3 What “broadcast” means here

The key semantic idea is the *configuration*: the set of active states (and regions) that the machine is currently in. Whenever the interpreter selects an event for processing, that event is matched against transitions in *all* active states in the configuration. In this sense the event is “broadcast”: it is visible everywhere in the current machine, even though only some region(s) react to it. Any internal events raised while handling the current input are processed before the next external message is consumed, so the effects within one reaction are deterministic given the current configuration, the data model, and the selected event.

6.4.4 Why not model each region as a separate actor?

A natural objection is that the Prolog Web already provides actor-level concurrency: why not represent each region as its own actor supervised by a parent, and coordinate by ordinary messaging? This is certainly possible, and it is often the right choice when true independence, failure isolation, distribution, or distinct lifetimes are required.

Orthogonality, however, is not primarily a claim about *parallel execution*; it is a way to specify a *single* reactive controller with an atomic, run-to-completion reaction. In a statechart, one selected event is visible to all active regions, transitions are chosen under a single set of priority and conflict-resolution rules, actions run, and the machine commits one coherent update of configuration and data model. An actor-only encoding can approximate the same effect, but typically only by introducing an explicit coordination protocol and a coordinator that reimplements the statechart scheduler in ad hoc code, while also making cross-region invariants harder to maintain under message interleavings.

The broader design trade-off between orthogonal regions and separate actors is taken up again in Section 6.10.2.

6.5 The Web Prolog data model

For many applications, a statechart by itself does not suffice. For example, while statecharts may be able to handle reactive aspects of AI-related applications, proactive aspects such as planning, decision making, and many other forms of problem solving and “intelligent” behaviour must also be catered for. Therefore, sophisticated conversational systems require knowledge representation and reasoning facilities as well as interaction facilities. In its current form, statecharts do not provide developers with sufficient means for knowledge representation and reasoning. To enable reactive control to be combined with logical inference, rule-based decision-making, and symbolic computation, Web Prolog can be used for data modelling and scripting.

6.5.1 Guarded transitions can derive information

SXML reuses the logical guard semantics of `receive/1-2`; it does not introduce a second kind of guard. A transition first unifies its `on` pattern with the selected event and then proves its `if` goal within the statechart actor’s logical environment. The chart’s private Prolog data model is only one component of this environment. The actor may also call predicates from the shared database of the node on which it is running. In the following example, the shared database contains the fact `online(printer2)`:

```
<statechart version="0.2" initial="ready">
  <datamodel>
    authorized(alice).
    owns(alice, printer2).
  </datamodel>
  <state id="ready">
    <go on="print(User,File)"
      if="authorized(User),
        owns(User, Printer),
        online(Printer)">
      format("Routing ~w to ~w~n", [File,Printer])
    </go>
  </state>
</statechart>
```

The guard can therefore combine private and shared knowledge while extending the bindings contributed by the event. If the event is `print(alice, 'report.pdf')`, event matching binds `User` and `File`, while the guard derives `Printer = printer2`. That binding is then in scope in the transition’s executable content. The transition

does not merely test whether printing is allowed; it derives how the event should be handled.

As with a `receive/1-2` guard, matching and guard evaluation use a tentative substitution, and the guard is evaluated for its first solution only. If the pattern or guard fails, the bindings made while trying that transition are discarded before another candidate is considered. In this limited sense, a failed candidate behaves as if it had never been tried. For portable code, however, a guard must be observationally free of side effects. The runtime is not required to enforce this restriction, and effects performed by a non-conforming guard are not undone on failure or exception. Thus the “never tried” description applies to logical bindings and transition selection, not to arbitrary effects.

6.5.2 Eventless transitions as forward chaining

So far, every transition has reacted to an event. Statecharts also support *eventless* transitions: transitions without an `on` attribute that may fire whenever their guard condition holds in the current configuration. Combined with a private and dynamic Prolog data model, this gives statecharts a form of forward-chaining inference: the machine repeatedly applies condition-action rules until no more apply, then waits for the next event. Here is a simple example, implementing Euclid’s algorithm for calculating the greatest common divisor of a set of numbers:

```
<statechart version="0.2" initial="Run">
  <datamodel>
    :- dynamic int/1.
    int(25). int(10). int(15). int(30).
  </datamodel>
  <state id="Run">
    <go if="int(X), int(Y), X > Y">
      Z is X-Y,
      retract(int(X)),
      assertz(int(Z))
    </go>
    <go to="Stop" if="int(X)">
      writeln(X)
    </go>
  </state>
  <final id="Stop"/>
</statechart>
```

The example illustrates another important piece of statechart semantics: if the children of an active state contain more than one `<go>` element, they are tried in document order. In this case, if the order between the `<go>` elements was switched, the state-

chart would just write 25 (instead of the correct 5) and then transition to the final state `Stop`, and terminate.

The entire calculation takes place within one macrostep as a cascade of eventless transitions – exactly the kind of cascade that the preceding section warned must terminate. Here termination is guaranteed because each subtraction strictly decreases the sum of the positive integers in the data model. Viewed another way, the example is a small forward-chaining production system: guarded rules repeatedly transform a Prolog knowledge base until no higher-priority rule applies.

The dynamic clauses in this example belong exclusively to the statechart actor. They are therefore not global variables or shared mutable state: no other actor can inspect or modify them, and their updates take place within the actor's sequential, run-to-completion execution. Given the same initial configuration and private database, and the same sequence of selected events, such updates introduce no additional scheduling nondeterminism. They do, however, retain the familiar non-logical properties of the Prolog dynamic database. Their effects depend on the actor's history and are not undone by backtracking or by failure later in an action. Actor-locality preserves isolation; it does not make mutation logical. See Section 3.2.3 for the more general discussion of private dynamic predicates.

6.5.3 Timed transitions

A state may also have a transition that becomes eligible after a specified delay. The `after` attribute gives the delay in seconds:

```
<state id="Color">
  <onentry>
    writeln('What color do you want?')
  </onentry>
  <go to="selectFormItem" on="says(color(Color))">
    assert(color(Color))
  </go>
  <go to="selectFormItem" after="5">
    writeln('I did not hear you.')
  </go>
</state>
```

The timer is armed when `Color` is entered. If a colour arrives first, the ordinary transition leaves the state and the timer is cancelled automatically. Otherwise, after five seconds the timed transition runs. The timeout does not interrupt executable content or a run-to-completion reaction already in progress; its firing is queued for subsequent processing. The assertion in this example has the same actor-local meaning as the updates in the preceding example: it changes only the statechart actor's private data model.

Timers are state-scoped. The precise activation, re-entry, and attribute rules are given in Appendix C.5.2.

6.5.4 Deferred events

Discarding an event that is irrelevant in the current state is usually the right default, but sometimes the event has arrived too early rather than being irrelevant. A device may receive a command while it is still initialising, for example. Such cases can be expressed with `<defer>`, a child of `<state>`:

```
<state id="initialising">
  <defer on="command(_)" />
  <go to="ready" on="initialised" />
</state>
<state id="ready">
  <go on="command(C)">
    perform(C)
  </go>
</state>
```

A `command` received in `initialising` enables no transition, but matches the `<defer>` declaration and is therefore postponed. When `initialised` changes the configuration, postponed commands are re-offered in their original order. They are processed before the actor receives a newer external message, so the reconsideration remains part of the run-to-completion machinery. A configuration change is essential: a targetless transition alone does not cause postponed events to be re-offered.

An enabled transition always takes precedence over deferral. A declaration may also have an `if` guard, with variables from its `on` pattern in scope:

```
<defer on="command(C)" if="temporarily_unavailable(C)" />
```

The postponed queue is separate from the actor mailbox and has no implicit size limit, so broad patterns in long-lived states require the same resource discipline as any other application-level buffer. Appendix C.7 states the precise search, re-offer, and discard rules.

6.6 Process invocation and communication

Statecharts become much more useful once they can spawn other processes and *communicate* with them. In the Web Prolog setting, the invoked processes are simply actors: toplevels, other statechart actors, or services running on remote nodes. For example, the statechart below spawns a remote toplevel actor, submits a query, prints each answer as it arrives, and terminates when the last answer has been received.

```

<statechart version="0.2" initial="spawn-ask-collect">
  <state id="spawn-ask-collect" initial="ask">
    <spawn type="toplevel"
      options="[node('https://n9.org'), session(false)]">
      p(a). p(b). p(c).
    </spawn>
    <state id="ask">
      <go to="collect" on="spawned(Pid)">
        toplevel_call(Pid, p(X), [
          limit(1),
          template(X)
        ])
      </go>
    </state>
    <state id="collect">
      <go on="success(Pid,Data,true)">
        writeln(Data),
        toplevel_next(Pid)
      </go>
      <go to="f" on="success(_,Data,false)">
        writeln(Data)
      </go>
    </state>
  </state>
  <final id="f"/>
</statechart>

```

How to read the example.

- Here the `<spawn>` element creates a `toplevel` actor on the node given by the option `node('https://n9.org')`. The five spawn types and their operands and option sets are specified in Appendix C.8.
- The text content of `<spawn>` is Prolog source transferred to the child as a `src_text/1` option. In this example it defines `p/1` privately in the spawned `toplevel`.
- After the child has been created, the SXML interpreter raises the internal event `spawned(Pid)`. The `pid` comes from the underlying spawn operation; the event is not a message sent by the child. The transition in state `ask` reacts by calling `toplevel_call/3`, which initiates query evaluation.
- Each answer arrives as an event of the form `success(Pid, Data, More)`. The option `template(X)` makes the successive slices `[a]`, `[b]`, and `[c]`. While `More` is `true`, the targetless transition requests the next slice with `toplevel_next/1` without exiting and re-entering `collect`. When `More` is `false`, the last answer has been delivered and the machine transitions to the final state.

Note that the intent of `<spawn>` is not merely to create a child process, but to tie its lifetime to control state: when the state that contains the invocation is exited, the invoked actor is terminated as part of leaving that state. This is the lifetime-containment discipline of Chapter 4 (Section 4.5.3) surfaced as a control construct: the invoked actor is lifetime-dependent on the invoking state in the same way that a client's spawned actors are lifetime-dependent on their session. This makes it easy to express resource ownership ("this control state owns this helper process") without relying on ad hoc cleanup code.

Executable content should not block while the statechart is stabilising. Interaction with other actors should be initiated asynchronously and completed by handling the resulting messages as events, as this example does.

The example deliberately crosses the boundary between SXML's own control events and ordinary actor traffic. The interpreter places `spawned(Pid)` on its internal event queue, whereas the PTCP `success/3` terms arrive through the actor mailbox. Once selected for processing, however, both are simply Prolog terms matched by unification. Query evaluation therefore remains asynchronous, and neither PTCP nor remote actor addressing requires a statechart-specific variant.

6.6.1 What does a confirmation confirm?

A service request illustrates why control state and application data must work together. Suppose Alice confirms a request and, while it awaits review, corrects its cost. Her earlier confirmation is still an event that occurred, but it no longer authorises the current request. The meaning of confirmation depends on the particular proposal that was presented, not merely on reaching a state called `confirmed`.

A chart can make this distinction explicit. While awaiting confirmation, it accepts an event only if the requester, revision, and exact field snapshot match the current proposal. A correction increments the revision, clears the old approval, and starts a new confirmation cycle. An old confirmation cannot advance that cycle even if it arrives while the chart is again in the same waiting state. State membership alone is therefore insufficient: the guard must also establish which request the event concerns. A trusted interface must supply the requester's identity; the event's name cannot establish it.

This is a useful division of responsibility. The statechart exposes when confirmation can have an effect; the Prolog guard establishes what is being confirmed and by whom. The complete service-request example in Section 8.4.4 applies this distinction to model-assisted proposals. A plausible proposal does not alter the meaning of approval.

6.7 The execution model in summary

The examples above convey how the statechart behaviour works in practice. Five points deserve explicit statement here; the element-level contract on which documents may rely is given in Appendix C.

1. **The execution model is based on SCXML's.** SXML follows the transition-selection and run-to-completion model stated in Section 3.13 and formalised in Appendix D of the SCXML 1.0 Recommendation (Barnett et al., 2015), instantiated with Prolog terms and goals. One selected external event may initiate a *macrostep* consisting of one or more *microsteps*. States are exited from the inside out, transition content is executed in document order, and states are entered from the outside in. The rationale for the adaptation is discussed in Section 6.10.5, and the element-by-element differences are tabulated in Appendix C.11.
2. **Events are terms and matching is unification.** At the actor level, the behaviour consumes messages in arrival order with an unconditional receive; selectivity lives in the transition rules rather than in the mailbox. A transition's *on* pattern matches an event by Prolog unification, and bindings produced by the match are in scope in the transition's *if* guard and executable content. An event that enables no transition is discarded unless it matches a `<defer>` declaration in the active configuration.
3. **Transition selection respects hierarchy and orthogonality.** For each active atomic state, candidate transitions are considered from that state outwards through its ancestors. Within one source state, document order determines which enabled transition is selected first. If two selected transitions have overlapping exit sets, the transition whose source is the descendant pre-empt the transition from its ancestor; non-conflicting transitions, such as transitions in independent parallel regions, execute together in one microstep.
4. **Internal work has a fixed priority.** After the current event has been handled, the interpreter first tests for an eventless transition. If none is enabled, it takes the next internal event, after which eventless transitions again have priority. When both sources are quiescent and the configuration has changed, postponed events are re-offered. Pending invocations are performed only after this work has quiesced; the `spawned(Pid)` events they raise enter the internal queue in turn. Only when no such work remains does the actor consume the next external message.
5. **Executable content is committed.** The Prolog content of an `<onentry>`, `<onexit>`, or `<go>` element is executed once; alternatives are not exposed through backtracking. If the goal fails, the interpreter places the internal event `error(failure(Goal))` on the queue; if it throws an exception `E`, the internal event is `error(E)`. Failure is thus an event inside the statechart rather than a control signal propagating out of it, a distinction taken up in Section 6.10.2.

The common forms of the `<go>` element, and their element-level contract, are given in Appendix C.5 (Table C.2).

6.8 A flat subset of SXML

Not every reactive controller needs Harel's structuring constructs. A protocol handler, a connection state machine, or a simple turn-taking loop is often a handful of states with no nesting and no orthogonal regions, and for such machines the full statechart vocabulary is more machinery than is needed. Rather than introduce a second behaviour for this case, we identify a flat subset of SXML: documents that omit the constructs a flat machine does not use.

We call this subset *Flat SXML*. It omits `<parallel>` (orthogonal regions), `<history>` pseudostates, and compound `<state>` elements (nested states), leaving flat states, the full range of `<go>` transition forms (Table C.2), state-scoped `<defer>` declarations, executable content, and the data model. Every document in the subset is also a valid full-SXML document; it simply uses none of the omitted constructs.

One thing the flat subset retains is the run-to-completion macrostep. RTC is not a structuring construct but part of the execution model: a flat machine still drains its internal event queue – including events raised by `raise/1` – before consuming the next external message. What the restriction removes is the *broadcast* aspect of RTC, since with no parallel regions there are no simultaneously active states for an event to be visible across. Eventless and guarded transitions, condition-action rules over the data model, internally raised events, and explicit event deferral all remain available; only hierarchy, history, and orthogonality are given up.

The benefit of identifying a subset rather than defining a separate behaviour is economy. There is one language, one interpreter, and one document syntax. Flat SXML is a descriptive name for documents written within the restricted vocabulary; the current interpreter neither requires a separate declaration nor performs a distinct Flat SXML conformance check. A machine that outgrows the subset is not rewritten in a different formalism – it simply begins using constructs the full language already provides.

6.9 From XML source to execution

The interpreter does not operate directly on the XML tree. When a statechart actor is started, its SXML source is validated as SXML 0.2 and translated into an actor-local Prolog model of the constructs described in Appendix C; embedded patterns, guards, and executable content become Prolog terms. The runtime interprets this model while maintaining the active configuration and the statechart's runtime queues, history, and timers.

The present implementation uses the same general interpreter for all SXML documents. Flat SXML nevertheless admits a simpler implementation: with only one active leaf state, an engine need not maintain hierarchical configurations, resolve conflicts between parallel transitions, or record history. A flat document could therefore be compiled into a transition table or a specialised actor loop. This is an implemen-

tation opportunity rather than a semantic distinction; the current interpreter does not perform this specialisation.

6.10 Discussion

This chapter has presented statecharts as an executable actor behaviour in Web Prolog, using an SCXML-inspired document syntax and Prolog as both data model and scripting language. The discussion section briefly situates this design relative to prior Prolog–SCXML work, clarifies what the behaviour buys compared to a hand-written `receive/1-2` loop, and then positions the result relative to Erlang/OTP’s `gen_statem` and the SCXML Recommendation.

6.10.1 Previous work

The idea of using Prolog as a data modelling and scripting language in a statechart formalism is not new. A proof-of-concept implementation of an SCXML system with Prolog as data model and scripting language was developed by a team at TU Darmstadt (Radomski et al., 2013). Their focus is multimodal conversational systems, and the paper is well worth reading, since it argues – contrary to what one might initially expect – that SCXML and Prolog form a good combination for developers of such systems. However, we take a slightly different and, we believe, simpler and more lightweight approach to integrating statechart notation with Web Prolog. Our aim is to adapt an SCXML-inspired statechart profile as a specialised Web Prolog notation, harmonised with the architecture of the Prolog Web.

6.10.2 Comparison with an ordinary message loop

At this point it is natural to ask why a dedicated statechart behaviour is introduced at all, rather than writing an ordinary message loop in Web Prolog. Small event-driven controllers are easy to code directly, but practical statecharts rely on Harel’s extensions such as hierarchy, history, and orthogonality, which buy *structure* that is difficult to recover from ad hoc `receive/1-2` code once the control problem grows. Hierarchy and history give interruption and resumption a first-class representation instead of burying it in auxiliary flags and continuations. Parallel regions support modular decomposition *within* one reactive process, and run-to-completion execution yields deterministic coordination between regions: given the same event in the same configuration, the same transitions fire in the same order. Explicit invocation ties the lifetime of spawned processes to control states, so that leaving a state can reliably terminate what it invoked. Finally, the data model makes long-lived state

a declarative Prolog object rather than an opaque tuple threaded through callback code, and the same structure that guides execution is directly visualisable, making control logic easier to review, communicate, and debug.

The statechart behaviour does not, however, depend semantically on `assert/1` and `retract/1`. Its defining machinery is the active configuration, history, internal and deferred event queues, timers, and the rules for selecting and executing transitions. Dynamic predicates are one Web Prolog representation of the optional application data that accompanies that control state. Because the database is private and reactions are sequential, deterministic database updates do not compromise actor isolation or run-to-completion determinism. This is an operational claim, not a claim of logical purity: a private dynamic database can hide part of the actor's state from the chart, and a failed action need not roll back updates already performed. The atomicity of a macrostep therefore means that another external event cannot interleave with it; it does not mean that the macrostep is a database transaction.

Beyond these structural benefits, the statechart behaviour also changes the *event-handling posture*. In a hand-written `receive/1-2` loop, selection is driven by the current mailbox contents and by pattern order, while non-matching messages typically remain in the mailbox (effectively deferred). The hand-written state machine in Chapter 2 illustrated why this must be a deliberate choice: one state deferred unknown events, while another discarded them. SXML makes the choice explicit: an unmatched selected event is discarded by default, while `<defer>` identifies the event patterns that should be reconsidered after a configuration change.

In SXML, event relevance is determined by the current configuration (the active states and regions): one selected event is matched against enabled transitions across the whole configuration, and a defer declaration in any active region may shelter it if none is enabled. Conversely, statecharts provide run-to-completion reactions: one external event may trigger a cascade of internal work (condition-only transitions, eventless transitions, internally raised events, and re-offered events) before the next external message is consumed. A reaction that never exhausts this work is livelocked and consumes no further external messages. With orthogonal regions, the selected event is visible across the entire active configuration, enabling deterministic broadcast-style coordination within one actor – something that actor-level messaging can emulate only by explicit fan-out and coordination protocols.

Three control moves are particularly awkward or easy to lose track of in an unstructured loop: eventless progress, guarded internal reactions, and targetless event handlers that consume messages without changing state.

Another operational difference concerns what failure means. `receive/1-2` commits to one selected message and one receive clause, but the fired body may backtrack through its own choice points and, if it ultimately fails, propagate that failure into the surrounding computation. Failure remains an ordinary Prolog control signal. The statechart behaviour draws the line differently: it commits to one external event and executes a deterministic run-to-completion macrostep. Within that macrostep nothing backtracks into the machine; a failing action becomes the internal event `error(failure(Goal))`. Failure-as-control versus failure-as-event is therefore an important operational boundary between the two behaviours.

6.10.3 Could PTCP itself be an SXML statechart?

The PTCP statechart of Chapter 3 invites a reflexive question: could the toplevel behaviour itself simply be supplied as an SXML document? Its outer control structure is an excellent statechart. In particular, attaching `abort` and `halt` to an enclosing state expresses exactly why those operations apply from every inner state.

The complete behaviour nevertheless requires capabilities below ordinary SXML. During state `s2`, the statechart interpreter is busy executing one run-to-completion macrostep and cannot itself receive an event that interrupts that computation; PTCP `abort` therefore requires the privileged signal mechanism of Section 4.1.7. During state `s3`, the toplevel retains a live nondeterministic Prolog computation, including its remaining choice points. Such a suspended search is not an ordinary Prolog term that can be stored in SXML's data model and later resumed by an action.

SXML can therefore describe PTCP's outer protocol, but cannot by itself supply the interruption and suspended-computation substrate that makes the protocol executable. This explains why the toplevel remains a built-in behaviour while SXML is an application-level notation. It also foreshadows Section 11.4.4: resumable Prolog computation is a distinctive runtime contract to standardise, not merely a statechart encoding technique.

6.10.4 Comparison with Erlang's `gen.statem` behaviour

Erlang's `gen.statem` behaviour offers a disciplined way to structure long-running reactive processes: the meaning of an incoming event depends on the current state, and a transition may update state data, schedule timeouts, or emit new events. In that sense, `gen.statem` and our statechart behaviour address the same engineering problem: how to express event-driven control in a maintainable way without hand-written receive loops.

The statechart behaviour differs in where it places complexity. First, it makes the control structure explicit and inspectable as a hierarchy of states rather than as callback code, which improves readability and supports direct visualisation. Second, it supports hierarchical states, history, and parallel regions as first-class control constructs, whereas `gen.statem` leaves such structure to be encoded by conventions in the callback logic. Third, the statechart run-to-completion macrostep provides deterministic intra-machine coordination (including broadcast effects across parallel regions under fixed priority rules), while Erlang's coordination across processes is intentionally mailbox-driven and concurrent. Finally, statecharts integrate process invocation as a control construct with a precise lifetime rule (invoked processes are tied to the state that invoked them), whereas `gen.statem` relies on external OTP patterns (supervision, linking, monitoring) to manage process lifetimes.

The two behaviours offer comparable state-scoped timing: `gen.statem` has state timeouts, while SXML provides `after` transitions that are armed on entry and cancelled on exit. They also share a retry-before-new discipline for premature events:

`gen_statem`'s `postpone` action defers the current event until a state change, while SXML's declarative `<defer>` associates the policy with an event pattern and control state.

The Flat SXML subset of Section 6.8 occupies roughly the same design space as `gen_statem`, while remaining directly extensible with hierarchy, history, and orthogonality.

6.10.5 Relationship to SCXML

SXML is intentionally SCXML-inspired, but it is not a full SCXML surface language. Its primary goal is to present statecharts as an executable actor behaviour in Web Prolog, with Prolog as both data model and executable content. For that reason we aim to implement the core of the SCXML 1.0 processing model, while adopting a smaller and more uniform set of constructs that harmonise with actor-style messaging and with Prolog scripting.

Several SCXML constructs are omitted because they are platform-specific, oriented toward browser integration rather than actor systems, or redundant once executable content is restricted to Web Prolog goals. SXML also renames a small number of constructs for readability in Prolog-centric examples and adds two of its own. Appendix C.11 gives the correspondence element by element, lists the SCXML constructs that SXML does not accept, and records the structural restrictions that follow from these choices.

Three further differences are semantic rather than notational. First, SCXML matches dotted event names by prefix against event descriptors, whereas SXML matches Prolog terms by unification and carries the resulting bindings into guards and actions. Second, SXML has no SCXML `_event` record: the structure and data of an event are represented directly by the term and the bindings of the `on` pattern. Third, SCXML's `<donedata>` and automatic `done.invoke` protocol are omitted. SXML uses explicit actor messages, internal `done(Id)` events, and ordinary monitoring instead. What remains central is SCXML's transition-selection and run-to-completion model, instantiated by the rules of Section 6.7.

6.10.6 Toward an actor-oriented successor to SCXML

Seen as a language for programming reactive actors, SCXML's core ideas are compelling, but the Web Prolog setting suggests a slightly different "centre of gravity": a declarative data model, a single uniform scripting language and an explicit integration with actor messaging. The statechart behaviour in this chapter can therefore be read as a concrete sketch of what an actor-oriented successor might prioritise: the standard statechart execution model, combined with Prolog-level reasoning and with explicit, programmable interaction with the surrounding actor system. What makes

the proposal more than a Prolog convenience is that the same shift applies to other host languages: the state-machine core is genuinely orthogonal to the scripting layer, and trusting that layer to carry more – communication, data modelling, scripting – is a direction a successor could take whether the host language is ECMAScript or Prolog. We develop this argument, and its implications for the SCXML lineage, in Chapter 11.

6.11 Summary

This chapter introduced the statechart behaviour: an actor behaviour in which control is expressed as a hierarchical, event-driven state machine with parallel regions, deterministic transition selection under run-to-completion execution, and executable content written in Web Prolog. Statechart actors can be spawned and supervised like ordinary actors, can communicate via messages, can use state-scoped timed transitions and event deferral, and can spawn child actors with well-defined lifetimes tied to control states. SXML retains SCXML's central execution model while replacing dotted event names with Prolog terms matched by unification and integrating action failure, final-state completion, timing, deferral, and invocation with the actor protocols of earlier chapters. The result is an SCXML-inspired but Prolog-centric profile that supports reactive control, structured coordination within one actor via configuration-wide event visibility, and integration of long-lived knowledge and decision logic through a private declarative Prolog data model.

Part III
The broader technological contexts

Chapter 7

The Prolog Web $\approx$ the Semantic Web?

Since the Prolog Web adds logic programming and automated reasoning to the conventional Web, it is natural to ask whether we are approaching something like the *Semantic Web* – the machine-readable Web vision articulated by Berners-Lee et al. (2001). The gist of their proposal is captured by the following three statements:

The Semantic Web is an extension of the current web in which information is given well-defined meaning, better enabling computers and people to work in cooperation.

For the semantic web to function, computers must have access to structured collections of information and sets of inference rules that they can use to conduct automated reasoning.

Adding logic to the Web – the means to use rules to make inferences, choose courses of action and answer questions – is the task before the Semantic Web community at the moment.

At first glance, words and phrases such as *logic*, *inference rules*, and *answer questions* could be read as an informal description of what Prolog already does. Likewise, the characterisation as an extension of the current Web fits the Prolog Web rather well. This invites a tempting but slightly misleading conclusion: perhaps the Prolog Web is simply a lightweight Semantic Web?

The resemblance is real, but it is a resemblance of ingredients rather than of engineering goals. The Semantic Web focuses on semantic interoperability by standardising shared meaning – data models, identifiers, vocabularies, and ontology semantics that many independent parties can agree on. The Prolog Web focuses on execution by standardising an executable substrate – a uniform rule language plus a node and process architecture in which predicates can be deployed as running services and agents. Both concern interoperable representations, but the Semantic Web foregrounds representation and shared meaning, whereas the Prolog Web foregrounds turning knowledge into executable behaviour.

The thesis of this chapter is therefore cooperative rather than competitive: the two are best viewed as complementary. In practice, Web Prolog can consume and produce RDF-style data, reuse established vocabularies and identifiers, and apply Prolog's execution model to “put the data to work” as queries, rules, and long-lived agent behaviour.

This comparison builds on a substantial Prolog tradition. Beginning with scalable RDF infrastructure, Jan Wielemaker and collaborators developed an integrated line of Semantic Web tools in and around SWI-Prolog: query translation and optimisation, the Triple20 ontology-manipulation tool, Web libraries, and an application architecture in which Prolog serves as the foundation for Semantic Web systems (Wielemaker et al., 2003; Wielemaker, 2005; Wielemaker et al., 2005, 2007, 2008). ClioPatria and the Semantic Web integration of SWISH carried this work into full platforms for applications and interactive development (Wielemaker et al., 2016; Beek and Wielemaker, 2016). The argument below therefore does not begin by claiming a newly discovered fit between Prolog and RDF. That fit has already been demonstrated in both infrastructure and applications. The additional question is what becomes possible when Prolog is also given a distributed node, service, and actor architecture.

The integration patterns described below are architectural patterns, not additional standard Web Prolog APIs. The Web Prolog profiles do not prescribe an RDF store, a SPARQL client, or an OWL reasoner, and the demonstrator does not provide standard predicates for them. A node owner may supply such facilities through trusted host libraries and expose them as ordinary predicates or services. Keeping this boundary explicit lets us discuss cooperation without silently enlarging the language defined in the preceding chapters.

Although the Semantic Web serves as our principal reference point, the underlying distinction between interoperable representation and executable computation applies more broadly to knowledge graphs, property graphs, and typed data APIs.

7.1 The Semantic Web tower

The Semantic Web is often presented as a layered architecture, where each layer builds on the layers below it. The lower layers are shared with the conventional Web, while the upper layers introduce machine-processable representations of data, query mechanisms, and ontology-level semantics. A commonly used illustration is the so-called *Semantic Web tower*, one variant of which is shown in Figure 7.1.

The diagram is a historical snapshot rather than a current inventory of standards. In particular, SWRL remained a 2004 W3C Member Submission,¹ whereas the RIF family later produced W3C Recommendations, including the Core, Basic Logic, and Production Rules dialects.²

The tower diagram is necessarily impressionistic, but it provides a useful scaffold for our comparison. At the bottom, URIs play the familiar role of locating resources (as URLs), but on the Semantic Web they also serve as global names for entities, relations and concepts (as IRIs). The Prolog Web uses URIs operationally to locate

¹ <https://www.w3.org/Submission/SWRL/>

² <https://www.w3.org/TR/rif-overview/>

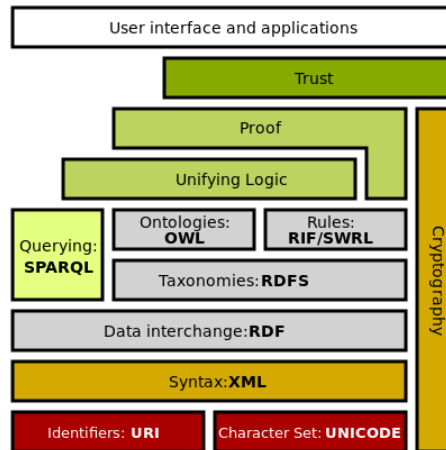


Fig. 7.1 One historical variant of the Semantic Web tower.

nodes and source; globally qualified pids then combine a process identifier with the URI of its node.

Unicode can likewise be viewed as a naming layer, ensuring that text and identifiers can be represented consistently across languages. Not all Prolog systems fully support Unicode, but a web-facing Prolog system effectively has to.

Many variants of the tower place XML at the syntactic level. In this chapter we treat XML mainly as a data interchange format among several. In the Prolog Web setting, Prolog terms provide a natural internal representation, and JSON is often the most convenient interchange format for client-facing APIs, while RDF serialisations such as RDF/XML and Turtle remain important for interoperability with existing Semantic Web tooling.

7.2 RDF as `rdf/3`

Right above the XML level of the historical Semantic Web tower we find the levels devoted to semantics proper, where RDF forms the basis. RDF represents information as subject–predicate–object triples. More complex structures are built by connecting resources and blank nodes through multiple triples. In RDF 1.1, reification has a narrower purpose: it allows an RDF graph to describe a particular triple token, but does not itself assert the triple being described.³

RDF 1.1 remains the latest W3C Recommendation. RDF 1.2 was a Candidate Recommendation Snapshot when this book was written; among other additions, it introduces triple terms and reifiers for making statements about statements, as well

³ <https://www.w3.org/TR/rdf11-mt/#reification>

as directional language-tagged strings.⁴ SPARQL 1.2 was still a Working Draft.⁵ The examples in this chapter deliberately use the stable RDF 1.1 data model and SWI-Prolog's `rdf11` library.

From a Prolog point of view, the triple pattern maps naturally to a three-argument relation. The SWI-Prolog Semantic Web libraries and *ClioPatria* therefore use `rdf/3` as their elementary query predicate (Wielemaker et al., 2016). Current SWI-Prolog applications normally use `library(semweb/rdf11)`, whose literal notation follows RDF 1.1 more closely than the older `literal(type(...))` representation. The following self-contained fragment creates a small example store:⁶

```
:- use_module(library(semweb/rdf11)).

:- rdf_register_prefix(ex,      'http://example.org/').
:- rdf_register_prefix(foaf,    'http://xmlns.com/foaf/0.1/').
:- rdf_register_prefix(schema, 'http://schema.org/').

load_example :-
    rdf_assert(ex:bob, rdf:type, foaf:'Person'),
    rdf_assert(ex:bob, foaf:knows, ex:alice),
    rdf_assert(ex:bob, foaf:knows, ex:john),
    rdf_assert(ex:alice, schema:birthDate,
               date(1990,7,4)^xsd:date),
    rdf_assert(ex:john, schema:birthDate,
               date(1992,9,6)^xsd:date).
```

Once these triples have been loaded, ordinary Prolog clauses can define a task-specific interface to the store. For example, the following relation finds the birth dates of people somebody knows:

```
known_with_birthdate(Me, Person, Date) :-
    rdf(Me, foaf:knows, Person),
    rdf(Person, schema:birthDate, Date).
```

Prefixes registered with `rdf_register_prefix/2` allow IRIs to be written in the compact form `Prefix:LocalName`; the calls to `rdf/3` still operate on the corresponding full IRIs. This reflects Berners-Lee's broader account of webizing a database, in which URIs name concepts such as tables and columns and namespace prefixes keep those names manageable. A portable namespace mechanism analogous to `rdf_register_prefix/2` would also be valuable in Web Prolog. Prefixes would provide readable notation, while the full URI would remain the public identifier used across nodes.

This would not make RDF the native data model of Web Prolog. It is important to keep the layers distinct: `rdf/3` and the prefix declarations above are supplied by the

⁴ <https://www.w3.org/TR/rdf12-concepts/>

⁵ <https://www.w3.org/TR/sparql12-query/>

⁶ The data is adapted from the RDF 1.1 Primer: <https://www.w3.org/TR/rdf11-primer/>.

SWI-Prolog host library, not by the standard Web Prolog predicate set. On a public node they would normally be installed by the owner and exposed through selected predicates rather than loaded by an untrusted client.

7.3 SPARQL-style querying in Prolog

Instead of using SPARQL, querying Semantic Web data represented as above can be done in Prolog. The close correspondence between graph patterns and Prolog goals has also supported query-language translation and optimisation (Wielemaker, 2005). The interactive session below illustrates the use of `known_with_birthdate/3` to query the triple store:

```
?- known_with_birthdate(Me, He, Date).
Me = 'http://example.org/bob',
He = 'http://example.org/alice',
Date = date(1990, 7, 4)^'^
      'http://www.w3.org/2001/XMLSchema#date' ;
Me = 'http://example.org/bob',
He = 'http://example.org/john',
Date = date(1992, 9, 6)^'^
      'http://www.w3.org/2001/XMLSchema#date'.
?-
```

The key point is modularity. Wielemaker et al. (2016) argue that the Prolog version using `known_with_birthdate/3` has several advantages over the SPARQL version. First, because the Prolog version has an explicit name, it is easy to reuse it in other queries. This allows developers to incrementally build more sophisticated functionality on top of existing, proven and tested building blocks. This can greatly simplify maintenance of complex queries.

Secondly, application logic written in Prolog is not confined to the filters and functions standardised by SPARQL. Any available predicate can serve as a guard, transformation, or derived relation at any point in the clause, and the same definition can combine RDF triples with other relations – for example, views backed by an RDBMS or calls into domain-specific code.

The query above is simple, but more complex relational queries are possible too. An ISOBASE node guarantees the usual Prolog control constructs, including conjunction, disjunction, and Negation As Failure, together with the ISO all-solutions predicates `bagof/3`, `setof/3`, and `findall/3`. SWI-based nodes may additionally expose `library(aggregate)`,⁷ which provides operators such as `count`, `sum`, `min`, and `max`. These library predicates are useful implementation facilities, but are not guaranteed by the Web Prolog ISOBASE profile.

⁷ <https://www.swi-prolog.org/pldoc/man?section=aggregate>

The same qualification applies to the SWI-Prolog library `solution_sequences`,⁸ which was developed to support direct formulations of many database-style queries and provides `distinct/1-2`, `limit/2`, `order_by/2`, and related predicates. On a node whose owner has made these two libraries available, consider the following relation `r/2`:

```
:- use_module(library(aggregate)).
:- use_module(library(solution_sequences)).

r(a, foo). r(a, bar). r(a, bar). r(c, bar). r(c, bar).
r(b, bar). r(a, foo). r(d, baz). r(a, foo). r(c, baz).
```

Here is an example of how a SPARQL-like query can be formulated using such predicates, and what the solutions would look like:

```
?- limit(10, order_by([desc(C)], aggregate(count,X,r(X,Y),C))).
C = 3, Y = bar ;
C = 2, Y = baz ;
C = 1, Y = foo.
?-
```

The call `aggregate(count, X, r(X,Y), C)` groups on the free variable `Y`. Its discriminator `X` removes duplicate `X` values within each group, so the result corresponds to `COUNT(DISTINCT X)` rather than a count of all matching clauses. The call to `order_by/2` sorts the resulting `Y-C` pairs by descending `C`, and `limit/2` truncates the sequence – conceptually similar to a SPARQL query with `GROUP BY`, `ORDER BY`, `COUNT(DISTINCT ...)`, and `LIMIT`.

The resemblance should not be mistaken for semantic identity. SPARQL has defined multiset, unbound-variable, error, and negation semantics that do not coincide in every case with Prolog’s solution sequences and Negation As Failure. The example illustrates how familiar query operations can be composed in Prolog; it is not a general semantics-preserving translation of arbitrary SPARQL.

If the owner has exposed `r/2`, `aggregate/4`, and `order_by/2` on a remote SWI-based node, the same goal can be submitted through `rpc/3`:

```
?- Goal = order_by([desc(C)], aggregate(count, X, r(X,Y), C)),
   rpc('http://n7.org', Goal, [limit(10)]).
```

The `limit(10)` option bounds the solutions returned through the RPC interface; it does not prevent `order_by/2` from collecting and sorting its complete input on the remote node. It is therefore a transport-level bound, not necessarily a computational optimisation.

An ISOBASE node is sufficient for this stateless call. ISOBASE also permits source to accompany a call through options such as `src_text/1` and `src_uri/1`. ISOTOPE becomes necessary when definitions or data must persist across a sequence

⁸ <https://www.swi-prolog.org/pldoc/man?section=solutionsequences>

of requests in a toplevel session. None of these profile guarantees includes the SWI-specific RDF or solution-sequence libraries; those remain owner-provided facilities.

The bottom line is that Prolog offers a particularly direct and idiomatic interface to RDF: triples map to relations, and rule layers can be expressed in the same language as the queries that consume them.

Wielemaker et al. (2016) summarise the position we adopt:

We do not consider SPARQL adequate for creating rich semantic web applications. SPARQL often needs additional application logic that is located near the data to provide a task-specific API that drives the user interface. Locating this logic near the data is required to avoid protocol and latency overhead. RDF-based application logic is a perfect match for Prolog and the RDF data is much easier queried through the Prolog RDF libraries than through SPARQL.

This is precisely the role the Prolog Web embraces: Prolog predicates become the task-specific API layer that lives near the data, while remaining compositional, testable, and reusable as ordinary code.

7.4 OWL as an external reasoning service

Moving up the Semantic Web tower, we reach OWL, a family of ontology languages based on description logics. OWL's purpose is not merely to provide more rules, but to support a particular kind of interoperability: shared vocabularies with explicit semantic commitments – class hierarchies, property characteristics, domain and range axioms, cardinality restrictions, and so on – that can be processed by standard OWL *reasoners*. This matters whenever independent producers and consumers of data need to agree not only on identifiers, but on the intended interpretation of those identifiers.

The Semantic Web community makes a clear distinction between knowledge representation languages (such as RDF, RDFS) and query languages (such as SPARQL). As we have seen, Prolog (including Web Prolog) can, to some extent, fill both roles – but with different semantic commitments. Web Prolog is most naturally used as an *executable rule layer*: named predicates encode task-specific queries and derived relations, composable and testable as ordinary code. OWL, by contrast, targets standardisable ontology semantics under open-world assumptions and decidable reasoning fragments, enabling interoperability through shared class and property commitments rather than shared procedural conventions. Prolog therefore excels at turning relational data into reusable executable services; OWL excels at making ontology-level meaning explicit in a form that independent tools and parties can agree on. Web Prolog is not “an OWL replacement,” and should not be presented as one. Web Prolog is grounded in Horn clause logic and, in typical use, relies on closed-world idioms including Negation As Failure. The OWL 2 DL language and OWL profiles provide standard, decidable fragments under open-world assumptions, supporting entailment and consistency reasoning that plain Horn clauses can-

not reproduce directly without changing the semantics or moving to richer logic programming extensions.

OWL also makes no unique-name assumption: two different IRIs may denote the same individual unless their distinctness is stated explicitly.⁹ This differs from ordinary Prolog term semantics, in which distinct ground terms do not unify. The contrast therefore concerns both what may be inferred from missing information and how identity is treated.

Ontology reasoning should also be distinguished from data validation. Under open-world semantics, a missing value does not normally constitute a violation. Applications that need to validate an RDF graph against structural conditions can use a shapes language such as SHACL,¹⁰ again treating the corresponding validator as a specialised external service.

This does not make the Prolog Web irrelevant to ontology-heavy applications – it merely suggests a division of labour. When OWL is the right tool for defining a shared ontology, Web Prolog can still play at least three valuable roles: *consuming* OWL-derived results (such as classifications from an external reasoner) and combining them with application-specific rules; *mediating* between heterogeneous sources by translating Semantic Web representations into executable relations exposed through the Prolog Web APIs; and providing an *agent substrate* in which long-lived actors use ontology-backed knowledge as one input among many while the agent logic itself remains straightforward Prolog.

Ontologies are thus one of the places where the Semantic Web stack contributes something genuinely distinct: not just inference, but shared semantic commitments and a mature ecosystem of reasoners and tooling. The Prolog Web complements this by making it easier to turn ontology-backed data into running services, executable behaviour, and web-native agents.

7.5 Positioning the Prolog Web alongside the Semantic Web

The preceding sections have examined individual layers of the Semantic Web tower and shown how each relates to the Prolog Web. It is now worth stepping back to compare the two ecosystems as wholes. Figure 7.2 places the Prolog Trinity alongside what, by analogy, might be called a “Semantic Trinity”: a comparable ecosystem assembled from Semantic Web technologies – RDF and SPARQL for data and querying, and a general-purpose language such as Java for application logic (and possibly OWL for ontology-level semantics, and an external reasoner for inference). The comparison is schematic. Integrated Semantic Web platforms certainly exist; the dashed arrows describe boundaries between independently standardised layers, not the quality of every implementation. Two differences nevertheless stand out. The first is integration. In the Prolog Trinity the arrows are solid, suggesting that the same

⁹ <https://www.w3.org/TR/owl2-primer/>

¹⁰ <https://www.w3.org/TR/shacl/>

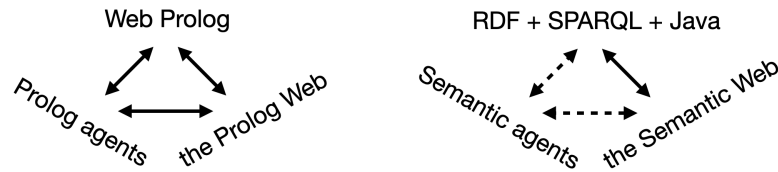


Fig. 7.2 Comparing the Prolog Trinity with a “Semantic Trinity.” Solid arrows indicate tight integration; dashed arrows indicate looser coupling.

language is used for knowledge representation, querying, rule-based computation, and agent scripting, and the execution substrate (nodes, actors, APIs) is part of the proposal rather than an afterthought. In the Semantic Trinity, the data layer (RDF), query layer (SPARQL), and application layer (Java or another host language) have distinct interfaces, and an application framework connects them.

The second difference is architectural. Semantic Web standards do not define a general process infrastructure with actor lifecycles, stateful services, and message-based coordination. These concerns are left to the chosen application framework. The Prolog Web, by contrast, includes an architecture as part of the design. Nodes serve answers over standard web protocols, calls compose local and remote resources, source can accompany calls and spawned processes, and actors coordinate over time. This is what makes the Prolog Web a *platform* for executable knowledge rather than merely a representation format.

These observations suggest a pragmatic positioning. Semantic Web standards provide interoperable representations and shared semantic commitments, but rich applications still require a host language to implement task-specific APIs, workflows, and integration with heterogeneous data sources. There is often an impedance mismatch between graph-shaped, relational data and imperative application code (Wielemaker et al., 2016). Prolog has an unusually good fit: RDF triples map directly to relations, and rule layers can be expressed in the same language as the queries that consume them. Application logic that would otherwise live as opaque client-side code or as ad hoc service glue can instead be expressed as named, reusable predicates close to the data.

From the Prolog Web perspective, then, Web Prolog can act as a programming and agent layer around Semantic Web assets: consuming RDF and SPARQL endpoints when appropriate, delegating ontology-level reasoning to OWL reasoners when needed, and exposing the resulting functionality as web-native services hosted on Prolog Web nodes. The next section makes this complementarity concrete by outlining three integration patterns that recur in practice.

7.6 Cooperation patterns

The complementary roles identified above lead to three recurring integration patterns, referred to as A, B, and C below.

7.6.1 A: Web Prolog as a SPARQL client and post-processor

In many applications the most pragmatic role for Web Prolog is to act as a consumer of Semantic Web data offered by existing endpoints. A Web Prolog program issues a SPARQL query to a remote endpoint, parses the result, and then applies ordinary Prolog predicates for filtering, aggregation, explanation, or task-specific inference. The important point is that the *logic that makes the results useful* is written as reusable predicates rather than being forced into a single monolithic SPARQL query.

Conceptually, the pattern looks like this:

1. Query a SPARQL endpoint and obtain a sequence or table of bindings.
2. Convert the bindings to a relational representation.
3. Run Prolog rules over that representation and expose the result as a Prolog Web service.

The first step is performed by an owner-installed library predicate or service that implements HTTP and one of the SPARQL result formats. The remaining steps are plain relational programming. If the application is interactive, the derived predicate can be offered through the Prolog Web APIs, allowing clients to consume its Prolog solutions incrementally. This does not imply that the underlying SPARQL endpoint is itself consumed lazily: an adapter may fetch or buffer its complete result before the Prolog relation begins producing answers.

A useful boundary case is to place a RELATION adapter directly in front of a SPARQL endpoint. The adapter advertises a fixed relation such as `rdf/3`, translates calls to that relation into SPARQL triple-pattern queries, and returns the resulting bindings through the stateless HTTP API. A client can then write:

```
?- rpc('http://sw-node.org', rdf(S, P, O)).
```

Here `rpc/2` is a client-side interface to the stateless call; it does not imply that the target accepts arbitrary ISOBASE goals. As stressed in Chapter 3, a RELATION node need not contain a Prolog engine at all. The adapter is conformant because it exposes only its advertised relation and behaves externally as a relational endpoint. In this way an existing SPARQL service can become a Prolog Web citizen without changing its storage or query engine.

7.6.2 B: RDF as `rdf/3`, Prolog rules as the API

A second pattern is to treat RDF as a storage and interchange format, but to treat Prolog as the *programming interface* to that data. In this pattern, RDF triples are ingested into a node-local indexed store queried through `rdf/3`, while the public surface consists of named Prolog predicates that encode the relevant domain.

The advantage over exposing a SPARQL endpoint is not merely syntactic. Named predicates become stable, reusable building blocks, and they can be composed with arbitrary auxiliary relations, foreign data sources, and task-specific computations. The service remains readable, testable, and evolvable: if a predicate is refactored, callers can remain unchanged as long as the predicate interface is preserved.

In a Prolog Web setting, a fixed collection of such predicates can be advertised through a `RELATION` node. An `ISOBASE` or stronger node may additionally accept general calls through `rpc/2-3`, while an `ISOTOPE` or `ACTOR` node can support longer conversations. The result is a service that is recognisably “Semantic Web” in the sense that it is grounded in RDF identifiers and vocabularies, but “Prolog Web” in the sense that its interface consists of executable relations.

A minimal bridge example

To make the idea more concrete, suppose a node maintains a local RDF store and provides the above derived predicate `known_with_birthdate/3`. A client can then query this service lazily using `rpc/3`, taking advantage of the Prolog Web’s support for bounded slices of answers:

```
?- Goal = known_with_birthdate(Me, He, Date),
   rpc('http://n7.org', Goal, [limit(10)]).
```

The point of the example is not the particular relation, but the division of labour. The use of established RDF identifiers and vocabularies makes the underlying data interoperable with Semantic Web tooling, while the exposed predicate provides an application-friendly interface that can be maintained and composed like ordinary Prolog code. If more logic is needed, it can be layered on top of `known_with_birthdate/3` rather than being duplicated across multiple SPARQL queries.

7.6.3 C: OWL reasoning as a service, Web Prolog for orchestration

A third pattern is to treat OWL reasoning as an external service rather than as something to be reimplemented inside Web Prolog. This is often the most realistic approach when an application depends on ontology-level commitments such as class subsumption and property characteristics that are supported by mature OWL reasoners.

In this pattern, OWL plays the role of *semantic backbone*, while Web Prolog plays the role of *executable glue*:

1. An OWL reasoner (local or remote) is used to compute classifications, entailments, or consistency checks.
2. The computed results are imported into Web Prolog as ordinary relations (for example, `class/2`, `subclass/2`, `entailed/1`, or additional `rdf/3` facts).
3. Prolog rules and, if needed, long-lived actors use these relations as inputs to application-specific decisions, workflows, or interactive services.

This arrangement respects the strengths of both stacks. OWL reasoners provide well-understood semantics and tool support for ontology-level inference, while Web Prolog provides a convenient substrate for integrating that inference with other sources of information, additional rule layers, network calls, and agent-style control logic. Where RDF data must also be validated, a SHACL processor can be integrated in the same way. The resulting system is typically easier to engineer than an attempt to force all application logic into SPARQL and OWL alone.

7.6.4 Summary

Pattern A consumes and post-processes existing endpoints. Pattern B places a named Prolog rule layer over an RDF store. Pattern C delegates specialised ontology reasoning or validation while Web Prolog orchestrates the application. In every case, Semantic Web assets retain their standard meanings and Web Prolog supplies the executable service and process layer.

7.7 Proof and trust

The upper layers of the Semantic Web tower are often labelled *Proof* and *Trust*. The labels point to real concerns, but the standards themselves do not prescribe a single, broadly adopted mechanism for either.

For the Prolog Web it is useful to read “trust” in a pragmatic engineering sense. A simple starting point is transparency: a node can let clients inspect the Web Prolog source code behind a published predicate or service, for example through a browsing facility, and optionally accompany it with version identifiers, digital signatures, or provenance metadata. Source visibility does not establish correctness, and a signature authenticates provenance rather than truth. These mechanisms nevertheless support auditability, reproducibility, and accountability – which is often what developers actually need.

This stance is also operationally natural in a Prolog setting. A node need not publish only answers; it can also publish the predicate definitions and provenance information required to reconstruct how those answers were obtained. Chapter 8

develops the idea further by showing how proof trees can be collected and inspected as a basis for explainable reasoning in Prolog agents.

7.8 User interfaces and applications

The cooperation patterns above are easiest to exploit in the familiar web application architecture: a JavaScript front end plus a back end that exposes task-specific predicates as services.

For example, a node can publish a UI-oriented relation over its RDF store:

```
display_card(Person, Name, Date) :-
    rdf(ex:bob, foaf:knows, Person),
    rdf(Person, foaf:name, Name),
    rdf(Person, schema:birthDate, Date).
```

A browser requests a bounded answer slice:

```
/call?goal=display_card(Person,Name,Date)&limit=10
```

The node evaluates the rule against RDF data and returns ordinary JSON binding objects; JavaScript renders each object as a card and follows the `more` flag if another slice is required. The end-to-end boundary is now clear: RDF supplies interoperable data, the named predicate supplies task-specific logic, the Prolog Web API supplies lazy delivery, and the browser supplies presentation.

The developer-facing difficulty has been formulated sharply by Verborgh and Vander Sande (2020). As Web development shifted towards front-end engineering, Semantic Web technologies remained difficult for application developers to approach, while apparently simpler stacks such as GraphQL began rediscovering problems long studied in SPARQL federation. Queries with purely local semantics cease to suffice when data comes from multiple sources. Bringing comparable expressivity within reach of front-end developers is therefore a research problem, not merely an engineering inconvenience.

The Prolog Web perspective is pragmatic: conventional Web front ends remain free to use HTML, CSS, and JavaScript, while Web Prolog provides a relational and rule-based back end over HTTP or WebSocket. A JavaScript UI can talk to services that in turn query RDF stores, SPARQL endpoints, or OWL reasoners, using Web Prolog as the application-logic layer around Semantic Web assets. ClíoPatria and SWISH have already demonstrated browser-facing Prolog environments for Semantic Web programming (Wielemaker et al., 2016; Beek and Wielemaker, 2016).

The demonstrator takes a further step by running Web Prolog in the browser through SWI-WASM. Its JavaScript bridge connects the browser-resident runtime to remote calls and actor messaging; a stable, application-facing JavaScript API over the same primitives would be a natural extension. This is a partial answer to the accessibility gap just described: front-end code can reach expressive, rule-based services through a small call-and-message interface rather than embedding

all federation and inference logic in the UI. It does not make interoperable querying easy by itself, but it lowers the entry cost at the point where much application code is written.

This is also where agents enter the picture in a concrete, engineering sense. Some agents are *UI agents*: long-lived client-side processes that maintain conversational or task state, trigger background queries, coordinate multiple services, and present results incrementally rather than as a single request–response transaction. In a conventional stack, such behaviour is implemented ad hoc in JavaScript and application glue. On the Prolog Web, the same interaction model can be carried across the client–server boundary: UI agents and back-end services can be written against the same set of message-passing and call abstractions, with the agent logic expressed as stateful actors that communicate by messages, consume RDF/OWL-backed knowledge when relevant, and expose their functionality through web-native APIs. We develop this agent perspective and the disciplined control structures needed to keep it predictable in Chapter 8.

7.9 Related work: towers of logic programming

As noted above, many variants of the Semantic Web tower have been proposed, and some align more closely with logic programming than others. The two-tower architecture of Horrocks et al. (2005), reproduced in Figure 7.3, contrasts two plausible instantiations of a layered Web stack.

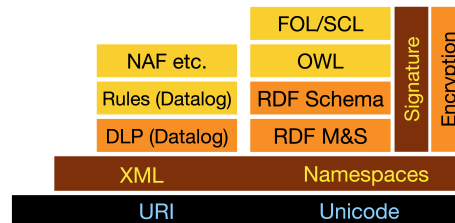


Fig. 7.3 The two-tower architecture, with the tower proposed in (Horrocks et al., 2005) to the left and a simplified version of the “original” tower to the right.

The right-hand tower captures essentially the same idea as Figure 7.1. The left-hand tower, by contrast, is more explicitly grounded in the logic programming paradigm. Here DLP denotes Description Logic Programs, a fragment designed to connect Datalog-style rule reasoning with part of the ontological expressiveness found in OWL. The later OWL 2 RL profile is especially relevant: it is a standardised, rule-oriented subset of OWL 2 designed for rule-based implementations and explicitly inspired by DLP and pD*.¹¹ Above DLP, one can formulate rules and enrich them

¹¹ <https://www.w3.org/TR/owl2-profiles/>

with features such as Negation As Failure. The “etc” in the diagram is not mere hand-waving: it points to the broad family of logic programming formalisms surveyed later in Section 10.3.1, each with its own semantic foundations and practical strengths.

The key observation is that this left-hand tower is not a single monolithic stack but a family of possible stacks, each grounded in a different fragment or extension of logic programming. The Prolog Web’s profile hierarchy (Chapter 3) provides a common interaction envelope for this diversity. A Datalog engine may expose fixed relations through the RELATION profile or a broader goal language through ISOBASE. A system using well-founded semantics or probabilistic reasoning can likewise remain addressable through standard Web Prolog protocols.

The profile name alone does not advertise the engine’s entailment regime, termination properties, treatment of negation, or probabilistic semantics. Those logical commitments require separate service metadata. The profiles instead say how a client may interact with the node – as a relation endpoint, a stateless Prolog evaluator, a persistent toplevel, or an actor system – while the protocol layer provides the common ground.

In this reading, the left-hand tower of Horrocks et al. (2005) maps directly onto the architecture described in earlier chapters. The broader vision – a Logic Programming Web rather than merely a Prolog Web – is taken up again in Chapter 10, where we consider what this architecture offers to the wider LP community.

7.10 Outlook: Prolog Trinity towers

Where does Web Prolog fit? One view places it as middleware between a conventional HTML, CSS, and JavaScript application layer and the Semantic Web stack beneath it, as suggested in Figure 7.4.

The same kind of arrangement can be built on top of the second tower, as suggested in Figure 7.5.

Or it might be regarded as a *third* tower, independent but interoperable with the other towers. Figure 7.6 suggests this option.

These pictures are complementary views rather than mutually exclusive implementation plans. Pattern B often makes Web Prolog look like middleware over an RDF store, whereas a network of independent Prolog Web nodes is better captured by the third-tower view. The latter is therefore the most faithful architectural picture of the proposal in this book: an independent but interoperable execution stack, able to sit beside Semantic Web stacks and to incorporate their assets when useful.

While the Semantic Web might be described as “layered,” the Prolog Web might be described as “integrated” – not only because the same language is used for knowledge representation and querying, but because Web Prolog is also a general-purpose programming language with a process architecture.

The case for allowing several interoperable towers was made explicitly by Kifer et al. (2005):

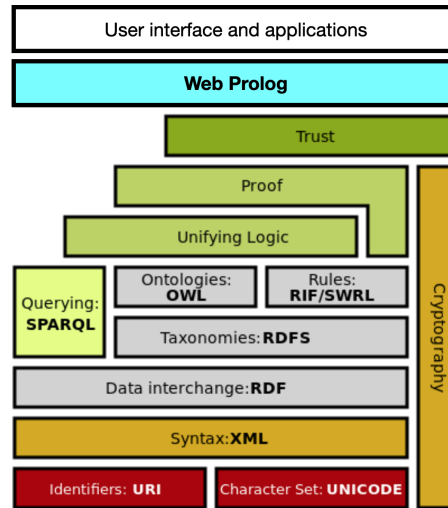


Fig. 7.4 The Semantic Web tower with an explicit Web Prolog middleware layer.

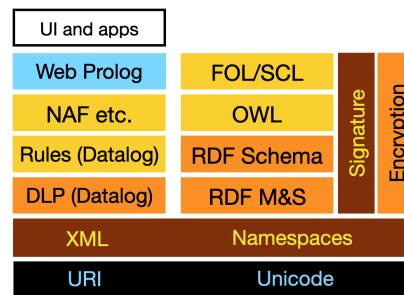


Fig. 7.5 Two-tower architecture (after (Horrocks et al., 2005)): logic-programming-oriented tower alongside the “original” tower.

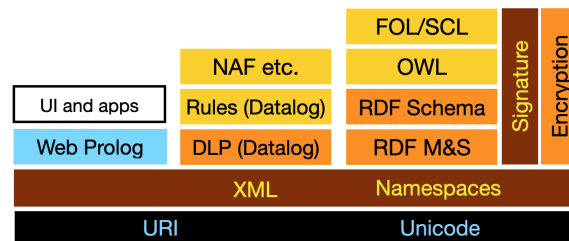


Fig. 7.6 Three-tower view: adding a Prolog Web tower alongside the two Semantic Web towers (after (Horrocks et al., 2005)).

While a single-stack architecture would hold aesthetic appeal and simplify interoperability, many workers in the field believe that such architecture is *un-realistic* and *unsustainable*. For one thing, it is presumptuous to assume that any technology will preserve its advantages forever and to require that any new development must be compatible with the old. If this were a law in the music industry (for example) then MP3 players would not be replacing compact disks, compact discs would have never displaced tapes, and we might still be using gramophones.

The authors have a point. We are tempted to think of Web Prolog programs as the vinyl records of the logic programming world, and of actors and nodes as the web-compatible gramophones on which to “play” them. Vinyl supports techniques such as *scratching* that newer media do not reproduce. Likewise, Prolog combines knowledge representation and querying with general-purpose programming, whereas the usual Semantic Web languages rely on a separate application language for that role. This integration has costs, but it is at the heart of Prolog. Without it, this would no longer be a Prolog Web.

7.11 Summary and discussion

The Prolog Web and the Semantic Web are similar in spirit: both treat the Web as a computational environment rather than merely a document store, and both rely on global naming (URIs) and on logic-based machinery to make data more useful than it would be in a purely syntactic web of documents. Both also aim at decentralisation and federation, in the sense that useful functionality should arise from composing resources offered by many independent parties rather than from a single central database.

At the same time, the two stacks emphasise different things. The Semantic Web places its centre of gravity on interoperable data models and shared semantic commitments, while the Prolog Web places its centre of gravity on executable relational services, process-level structure, and web-native agents. Table 7.1 summarises these differences.

One way to state the complementarity is that unification disciplines structure at the point of call, whereas shared vocabularies and ontologies make semantic commitments explicit. Neither substitutes for the other: structurally compatible terms can still be interpreted differently, and shared meaning does not by itself provide an execution model. Chapter 9 returns to the structural side of this distinction when treating unification as a universal data contract.

Although the discussion in this chapter uses the Semantic Web as a concrete point of comparison, the underlying issues are not specific to RDF, OWL, or W3C standards. Property graphs and large-scale knowledge graphs raise many of the same questions about graph-structured data, pattern-based querying, distributed evaluation, and the relation between shared representation and executable computation. Web Prolog can use such representations without making any one of them its native data model.

The Prolog Web	The Semantic Web
Uses the same language for querying and rule-based computation; queries can be packaged as reusable predicates	Typically separates data representation (RDF/OWL) from querying (SPARQL) and application logic
Provides a process and service architecture (nodes, APIs, long-lived actors) as part of the proposal	Provides standards for interoperable data and ontology semantics, but does not prescribe a single process architecture
Supports agent-style computation naturally through long-lived actors and message passing	Has strong support for shared ontologies and open-world reasoning through standardised languages and reasoners
Shared meaning is typically implicit in executable predicates and local conventions; shared ontologies are optional	Interoperability is centred on shared identifiers, vocabularies, and ontology axioms
Well suited as a programming and orchestration layer around heterogeneous services (including Semantic Web services)	Well suited as a lingua franca for knowledge interchange across independent parties

Table 7.1 Comparing the Prolog Web and the Semantic Web

The aim is therefore not to replace the Semantic Web, but to add the execution architecture that its representations and reasoning services do not themselves prescribe. RDF can supply interoperable data, OWL can supply shared semantic commitments, specialised reasoners can supply entailments, and Web Prolog can turn their results into named relations, distributed services, and long-lived agents. The Prolog Web is therefore best understood as an independent but interoperable execution stack: able to stand beside Semantic Web stacks, consume their representations and reasoning services, and turn those assets into executable relations, services, and agents.

Chapter 8

Prolog AI agents on the Agentic Web

Figure 8.1 illustrates a familiar way of viewing the history of artificial intelligence since the late 1950s: as a sequence of booms and busts, or “AI summers” and “AI winters.” Prolog and Lisp played prominent roles during the second boom, when expert systems were widely seen as the flagship technology of applied AI. In the current boom, by contrast, the centre of gravity has shifted towards neural methods, especially deep learning and large language models (LLMs).

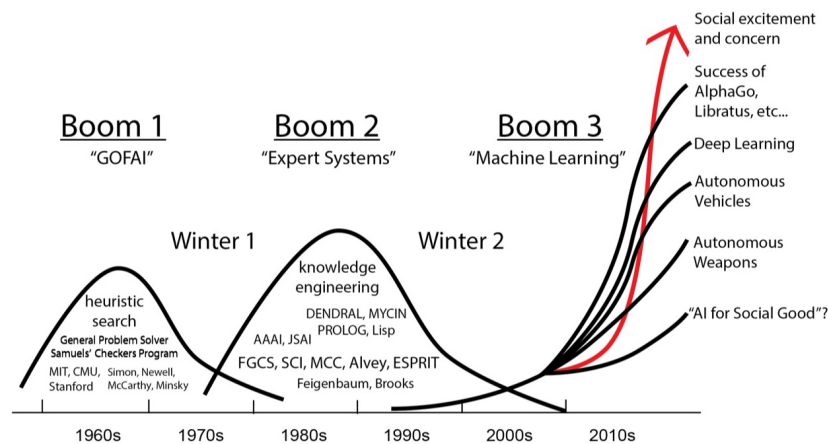


Fig. 8.1 The three booms of AI. Diagram by Colin Garvey, inspired by Yutaka Matsuo (Garvey, 2018).

This historical diagram was published in 2018 and should not be read as a chronology of the subsequent rise of generative AI. Its larger lesson remains useful: claims about AI recur in cycles, and architectural ideas can outlive the boom in which they first became prominent.

This chapter argues that the symbolic tradition remains relevant – not as a rival to contemporary AI, but as a complement to it. The reason is architectural. As the Web becomes populated by software processes that communicate, maintain state, use tools, and act over time, controllable behaviour, inspectable reasoning, and explicit constraint handling again become first-class engineering concerns. This development is often described as an *Agentic Web*. The term is still emerging rather than denoting one settled architecture; current W3C activity similarly treats agent protocols and trustworthy agent interaction as open areas of work.¹ In this chapter it means a Web in which autonomous, communicating processes operate alongside documents, services, and human users as first-class participants.

This development intersects naturally with logic programming. Earlier in the book, especially in Chapter 4, we argued that the Prolog Web already constitutes a multi-agent system in a minimal architectural sense. We now ask how Prolog can be used deliberately to structure agent behaviour, govern interaction, and combine reasoning, learning, and tool use. Because the vocabulary and products surrounding LLM-based agents change unusually quickly, specific descriptions in this chapter should be read as current in July 2026; the architectural claims are intended to outlast them.

8.1 Why Prolog agents now?

The classical Web is primarily a publishing and integration substrate: documents, hyperlinks, and request–response endpoints. The Agentic Web shifts the unit of composition towards interacting processes. By the *Prolog Web* in its agentic aspect, we mean the part of that environment in which agents are implemented as Prolog processes, exposed as web-native services, and capable of interacting through message passing.

The word *agent* itself carries three relevant senses. This book’s default, established in Chapter 3, is architectural and deliberately modest: an agent is defined by a persistent interface and behaviour, without any claim about intelligence. Classical AI uses a richer vocabulary of beliefs, desires, intentions, goals, and speech acts (Wooldridge and Jennings, 1995). Contemporary LLM engineering often uses *agent* for a model embedded in a tool-using control loop. The latter two are things that may be built on the first: a BDI agent may use an actor’s private database for beliefs and its behaviour for deliberation; an LLM agent may delegate part of that deliberation to a model. Unless qualified as *BDI agent* or *LLM agent*, the word retains the book’s architectural meaning.

Historically, the Web has standardised naming, transport, and representation far more successfully than it has standardised process structure. We have mature machinery for addressing resources and exchanging messages, but much less agreement about the lifecycle of long-lived services, the discipline of stateful interaction, the

¹ See the W3C AI Agent Protocol Community Group, <https://www.w3.org/community/agentprotocol/>.

handling of partial failure, or the way autonomous components should compose. Early discussions of web agents within the Semantic Web community made this gap explicit: representation and inference were advancing, but the process infrastructure in which agents would live and cooperate remained largely an application-level concern (Hendler, 2001).

The Prolog Web treats this gap as a design target. It preserves the Web’s strengths – global naming, standard protocols, and interoperability – while adding an explicit execution model. Nodes host long-lived actors with mailboxes and supervised life-cycles, and predicates can be exposed as message-driven services. This is why the chapter can move from “agents as an emergent property” to “agents as a deliberate design”: the substrate already has the right shape for autonomous processes, and the remaining problem becomes behavioural engineering – how to specify, constrain, explain, and govern what agents do.

That an architectural gap exists does not, by itself, explain why Web Prolog should help fill it. Two contemporary pressures make the case. The first is the need for enforceable semantics. Large language models have made fluent natural-language interaction widely available, but fluency is not behavioural reliability. Whenever an agent must satisfy non-negotiable constraints, follow explicit rules, or produce auditable justifications, part of its behaviour needs a meaning that can be inspected and tested. Web Prolog offers executable symbolic representations, explicit derivations, and programmable boundaries for failure and resource use. These properties apply to the fragment actually formalised in the program; they do not make arbitrary Prolog code terminating, consistent, or correct.

The second pressure is that the modern Web removes much of the old deployment bottleneck for symbolic AI. Symbolic systems have often remained laboratory artefacts because deployment required bespoke infrastructure, heavy integration work, and awkward maintenance. Standard protocols, cloud practices, and ubiquitous client platforms change this equation. A Prolog engine can now sit behind stable interfaces as a verifier, planner, policy engine, dialogue manager, or explanation service, and can be composed with other services without losing its semantic character.

The result is not a call for “Prolog everywhere,” but for Prolog where semantic clarity and constraint satisfaction matter most. Web Prolog is best understood as a *behavioural backplane* for agentic systems: the component in which constraints, state, control, and explanations are represented explicitly as programs, and can therefore supervise more fallible components. This positioning requires candour about Prolog’s limitations. Its learning curve is steep, its implementation landscape remains fragmented, and its mainstream ecosystem is limited. The claim is only that these costs may be justified when the alternative is to reconstruct constraint handling, control, and auditability in ad hoc form on top of tools that do not natively support them.

Having framed the architectural motivation, the remainder of the chapter develops the argument in four steps. We begin with Prolog’s symbolic competence on a single machine. We then distribute those idioms across the Prolog Web. Next, we consider neuro-symbolic architectures in which models supply linguistic or perceptual ser-

vices while Web Prolog supplies explicit reasoning and control. Finally, we turn to user-interface agents, where dialogue is treated as a problem of interaction control.

8.2 Symbolic competence in one engine

Before we distribute Prolog across the Agentic Web, it is useful to recall what a single Prolog engine already provides as an AI substrate. The point of this section is not to teach Prolog, but to fix a compact map of facilities: a handful of idioms that repeatedly recur in symbolic AI, and that will later reappear as message-driven services and interacting agents.

8.2.1 Knowledge representation and inference

A Prolog program can function simultaneously as a declarative knowledge base and as an executable inference procedure. A canonical example is a small relational theory of kinship, where recursion expresses transitive closure:

```
ancestor_descendant(X, Y) :- parent_child(X, Y).
ancestor_descendant(X, Z) :-
    parent_child(X, Y), ancestor_descendant(Y, Z).

parent_child(X, Y) :- mother_child(X, Y).
parent_child(X, Y) :- father_child(X, Y).

mother_child(trude, sally).

father_child(tom, sally).
father_child(tom, erica).
father_child(joe, tom).
```

A query such as `?-ancestor_descendant(joe, Who)` asks for logical consequences of the clauses and enumerates answers via backtracking. This combination – explicit relational structure plus a proof procedure – is the basic engine behind many symbolic systems, and in Prolog the proof procedure is built in.

8.2.2 Planning as search in a state-transition model

Many agent tasks are naturally expressed as planning: find a sequence of actions that transforms an initial state into a goal state. A small meta-program suffices when actions are represented as declarative transition rules:


```

plan(P, P, []) .
plan(P0, Pf, [A|As]) :-
    step(P0, P1, A),
    plan(P1, Pf, As).

step(start, position(monkey, bananas, chair), start).
step(position(Y,Y,Y), final, climb).
step(position(X,Y,X), position(Y,Y,Y), carry(X,Y)).
step(position(X,Y,Z), position(Z,Y,Z), walk(X,Z)).

```

Here the entire world state is a first-order term, and each `step/3` clause specifies an action's preconditions and effects. Prolog's search then discovers action sequences by exploring transition alternatives under backtracking.

For example, the query `?-plan(start, final, Plan)` produces `Plan = [start,walk(monkey,chair),carry(chair,bananas),climb]`. The example deliberately leaves search control visible: it has neither a visited set nor a depth bound, and may therefore diverge when applied to transition systems containing unproductive cycles. Practical planners add the resource and repetition controls appropriate to their state space.

8.2.3 Meta-interpreters as a control and explanation lever

A distinctive Prolog idiom is to treat programs as data and write interpreters for restricted logics, alternative control regimes, or explanation-producing variants. A minimal meta-interpreter mirrors Prolog's own proof search:

```

prove(true) :- !.
prove((A,B)) :- !, prove(A), prove(B).
prove(A) :- clause(A,B), prove(B).

```

Small changes to such an interpreter can add substantial functionality without changing the object-level knowledge base. For example, adding a second argument allows the interpreter to construct a proof object – a structured explanation that records which rules were used:

```

prove(true, true) :- !.
prove((A,B), (PA,PB)) :- !, prove(A, PA), prove(B, PB).
prove(A, A/Proof) :- clause(A,B), prove(B, Proof).

```

Proof terms of this kind are evidence-carrying data structures that can be inspected, summarised, or shown to a user. In Section 8.3 we will exploit this idiom across nodes, so that a single explanation can span distributed computation.

8.2.4 An expert system with a query-the-user facility

Expert systems were a flagship application of symbolic AI during the 1980s. They did not become a lasting mainstream paradigm, but the core pattern remains instructive, especially in Prolog: it shows how deduction can be interleaved with interaction in a disciplined way.

An expert system typically consists of a knowledge base plus an inference engine. A practical system must also be able to acquire facts that are not already stored. In Prolog, a simple way to realise this is to extend the meta-interpreter from Section 8.2.3 with a single additional clause that queries the user whenever a goal is declared askable:

```
prove(A) :- askable(A, Q), writeln(Q), read(T), T == yes.
```

The accompanying knowledge base supplies both domain rules and a set of askable/2 declarations that map predicates to human-readable prompts:

```
good_pet(X) :- bird(X), small(X).
good_pet(X) :- cuddly(X), yellow(X).

bird(X) :- has_feathers(X), tweets(X).

askable(tweets(_), 'Does it tweet?').
askable(small(_), 'Is it small?').
askable(cuddly(_), 'Is it cuddly?').
askable(has_feathers(_), 'Does it have feathers?').
askable(yellow(_), 'Is it yellow?').
```

When prove/1 encounters an askable subgoal, it suspends deduction, poses the associated question, reads the reply, and then resumes the derivation. For example:

```
?-prove(good_pet(tweety)).
Does it have feathers?
|: yes.
Does it tweet?
|: yes.
Is it small?
|: no.
Is it cuddly?
|: yes.
Is it yellow?
|: yes.
true.
?-
```

The important point is structural: the current proof attempt determines which information is requested next. The interaction is system-initiative rather than genuinely

mixed-initiative: the engine advances the derivation whenever its knowledge suffices and consults the user when its proof strategy reaches an askable subgoal. The askable declarations therefore act both as an interaction interface and as a lightweight user model: they mark which facts are expected to come from dialogue rather than from stored knowledge.

This is, of course, a toy shell. In more realistic systems one would add typed answers, richer question forms, explanation facilities, and a persistent dialogue state. For a fuller treatment of Prolog-based expert systems, see (Merritt, 1989) and (Sterling and Shapiro, 1994).

8.2.5 Parsing as deduction: DCGs

Definite Clause Grammars (DCGs) turn parsing into a form of proof search: a sentence is grammatical exactly when it can be derived from the grammar rules. DCGs are also reversible, so the same grammar can be used for generation. Operationally, naïve top-down execution makes left recursion problematic, but in tabled Prolog systems this restriction largely disappears: tabling turns parsing into dynamic programming while preserving a declarative rule notation. (Tabling is a system feature rather than part of the ISO core, so it should be read here as an optional acceleration of the same modelling stance.)

8.2.6 Summary

The examples above form a compact map of facilities. A single Prolog engine already supports (i) explicit relational knowledge representation, (ii) planning as systematic search in state-transition models, (iii) meta-level control and explanation via interpreters that construct proof objects, (iv) interactive acquisition of missing facts, and (v) grammar-based language processing as deduction.

The next section lifts these idioms onto the Agentic Web. Instead of treating predicates as internal procedures, we treat them as services: message-driven actors with interfaces, state, and failure modes. The result is an Agentic Prolog Web in which symbolic reasoning is not only executable, but deployable, composable, and inspectable across the network.

8.3 Prolog AI agents on the Agentic Prolog Web

The central claim of this section is that the idioms (i)–(v) do not need to remain local. When Prolog engines are exposed as web-native actors, the same programs can be deployed as distributed services and composed into interacting agents.

The Agentic Prolog Web is therefore not a new symbolic formalism. It is an execution and deployment stance: treat Prolog engines as networked processes with mailboxes, state, and interfaces, and let predicates become message-driven services. In this setting, what used to be a local `read/1` becomes a `prompt/2` message expecting a response; what used to be a local meta-interpreter becomes a mobile proof-producing component; and what used to be an internal predicate call may become a remote call whose provenance is part of the explanation.

The following examples cover interactive deduction, proof construction across nodes, remote parsing, the relationship to earlier agent frameworks, and the deployment properties of the resulting services.

8.3.1 An expert-system agent and a simulation

Classic expert systems faced two persistent bottlenecks: the difficulty of extracting knowledge from humans and the lack of a ubiquitous infrastructure for deployment. A web-native actor substrate does not remove the knowledge acquisition problem, but it lets elicitation be expressed as incremental interaction and lets a symbolic system be deployed as a service rather than as a local read/write loop.

To adapt the askable meta-interpreter pattern from Section 8.2.4 to an agent setting, we replace terminal I/O with an interaction predicate. Concretely, the clause that previously wrote a question and read an answer becomes:

```
prove(A) :- askable(A, Q), input(Q, T), T == yes.
```

The purpose of `input/2` is to decouple deduction from any particular user interface. In a local setting, `input/2` might still be implemented on top of `read/1`. As we saw in Chapter 4, on the Prolog Web it is implemented by sending a `prompt` to some conversational partner and waiting for a reply via `respond/2`.

We can now package the expert system as an expert-system agent by spawning a remote toplevel actor and preparing a call to `prove/1`:

```
expert_ask(Query, Pid, Options) :-
    toplevel_spawn(Pid, [
        session(false)
    | Options
    ]),
    toplevel_call(Pid, prove(Query), [
        limit(1),
        once(true)
    | Options
    ]).
```

The simulation asks an existential question: it needs one proof that the conclusion follows, not an enumeration of every proof. The `limit(1)` option restricts the returned slice to one solution, while `once(true)` commits the remote computation

after that slice and discards any remaining alternatives. The answer is therefore terminal even when another proof might exist. These options do not limit the number of prompts, which arise while that single proof is being sought.

To demonstrate the pattern, we construct a small simulation in which a second agent – in this case, the shell process – plays the role of the user. The simulation answers questions stochastically, illustrating that the expert system does not control the dialogue by a script; rather, the proof search itself determines which information must be acquired next.

```
simulation(Options) :-
    random_member(Name, [tweety,pingu]),
    format('A1: Is ~w a good pet?~n', [Name]),
    expert_ask(good_pet(Name), Pid, Options),
    interact(Pid, Name).

interact(Pid, Name) :-
    receive({
        failure(Pid) ->
            format('A2: No, ~w is not a good pet.~n', [Name]) ;
        success(Pid, _, false) ->
            format('A2: Yes, ~w is a good pet.~n', [Name]) ;
        error(Pid, Error) ->
            throw(Error);
        prompt(Pid, Question) ->
            format('A2: ~w~n', [Question]),
            random_member(Answer, [yes,no]),
            respond(Pid, Answer),
            format('A1: ~w~n', [Answer]),
            interact(Pid, Name)
    }).
```

For this run, assume that the shared database of `n9.org` contains the domain rules and declarations for the askable predicates. The inference engine is loaded into the new actor's private database, together with one task-specific fact:

```
?-simulation([
    node('http://n9.org'),
    src_predicates([prove/1]),
    src_list([yellow(tweety)])
]).
A1: Is tweety a good pet?
A2: Does it have feathers?
A1: yes
A2: Does it tweet?
A1: yes
A2: Is it small?
```

```

A1: no
A2: Is it cuddly?
A1: yes
A2: Yes, tweety is a good pet.
true.
?-

```

The architectural point is the separation between the node-resident knowledge, the privately loaded interpreter and fact, and the interaction process. The actor's questions are selected by its current proof attempt rather than by a separate dialogue script. The Web substrate supplies distribution and composability, while Prolog supplies the proof structure that drives the interaction.

The code is pedagogical. A deployed client should also monitor the actor and bound its receive operation, so that node failure or a lost connection does not leave the simulation waiting indefinitely.

8.3.2 Explainable AI across nodes: distributed proof trees

A defining advantage of logic-based, symbolic AI is the possibility of constructing explanations from explicit rules and facts. This transparency depends on the predicates involved: foreign code, side effects, cuts, and unexposed host predicates may still be opaque. On the Agentic Prolog Web, a cooperating service can preserve the explanation discipline across a distribution boundary rather than reducing a remote subgoal to a black box.

A compact meta-interpreter can achieve this by treating remote calls as proof-producing sub-derivations. When the interpreter encounters a remote goal, it ships itself to the remote node and requests a proof there, then folds the result back into the local derivation:

```

prove(true, true) :- !.
prove(rpc(URI,A), Proof) :- !,
    prove(rpc(URI,A,[]), Proof).
prove(rpc(URI,A,Options), Query@URI/Proof) :- !,
    rpc(URI, prove(A, Query/Proof), [
        src_predicates([prove/2])
    | Options
    ]).
prove((A, B), (ProofA, ProofB)) :- !,
    prove(A, ProofA),
    prove(B, ProofB).
prove(A, A/Proof) :-
    clause(A, B),
    prove(B, Proof).

```

The `src_predicates([prove/2])` option is what makes the explanation discipline recursive: the meta-interpreter is transmitted to the remote node along with the goal, so the remote node runs `prove(A, Query/Proof)` rather than the bare goal `A`. Proof construction therefore applies at every hop, not just the first.

The key idea is that a cooperating remote computation need not be treated as a black box. When the remote node accepts the transmitted interpreter and exposes the clauses it may inspect, the same explanation discipline is applied at each node, and remote contributions are marked in the resulting proof term by the `@` operator.

Suppose, as in Section 4.2.3, that the local node states that every human is mortal by means of a rule `mortal(X) :- human(X)`, and defines `human(X)` by consulting a remote node. If the fact `human(plato)` is stored on `n1.org`, we obtain a proof object that explicitly records the distribution boundary:

```
?-prove(mortal(plato), Proof).
Proof = mortal(plato)/
      (human(plato)/
       (human(plato)@'http://n1.org'/
        true)).
?-
```

The annotation `human(plato)@'http://n1.org'` documents not just what was concluded but where. The proof term is ordinary Prolog data: it can be inspected, stored, transmitted, or rendered into a human-readable explanation by a further layer of code.²

Proof-producing interpreters can of course generate very large derivations. On the Prolog Web this is best treated as a controllable engineering choice rather than an unavoidable pathology: one may bound proof depth, abstract away local details and retain only a node-level skeleton, cache recurring subproofs, or compress proof objects into summaries suitable for user-facing explanations. The benefit is that distribution does not force a loss of transparency. Explanations remain first-class data, even when computation spans multiple machines.

8.3.3 Meta-interpreters as generic behaviours

The two preceding examples share a factoring familiar from Chapter 5. The meta-interpreter defines a reasoning policy – askable deduction or proof-term construction – while the actor substrate supplies a mailbox, lifecycle, and network address. The application supplies the knowledge base. Changing the knowledge base therefore produces a different expert under the same reasoning policy; changing the interpreter applies a different policy to the same knowledge.

Packaged as an actor, such an interpreter also acquires the deployment discipline of a generic behaviour: clients see a stable process interface, while supervision and

² One practical requirement is that predicates inspected through `clause/2` must actually be visible to the interpreter. Permission to inspect compiled predicates varies between Prolog systems.

addressing remain independent of the domain clauses. The analogy should not be pressed into a proposed fixed catalogue, but it suggests useful reusable components such as bounded, tabled, interactive, or provenance-producing interpreters.

8.3.4 Parsing as remote deduction

A client may offload parsing to a node that hosts a grammar, without embedding that grammar locally:

```
?-rpc('http://n6.org', phrase(s(PT), [joe,saw,a,telescope])).
PT = s(np(pn(joe)), vp(v(saw), np(det(a), n(telescope)))).
?-
```

This vignette illustrates the broader pattern: a symbolic competence can be hosted with the resources and execution strategy it needs, then accessed through a uniform remote call. A client need not embed the grammar or reproduce its implementation assumptions. The host may run an ordinary or tabled DCG, or it may front an external grammar system such as Grammatical Framework (Ranta, 2011); the client still sends a goal and receives a structured parse.

When the grammar constructs the tree as a faithful record of the rules used, the returned PT also serves as a compact proof witness. Remote parsing is then a special case of the proof-carrying remote calls of Section 8.3.2: the desired syntactic structure is also evidence for the derivation.

8.3.5 Implementing agent frameworks in Web Prolog

Logic-based agent languages and frameworks have been implemented in Prolog for decades. Systems such as DALI (Costantini and Tocchio, 2004) and LPS (Kowalski and Sadri, 2015) show that one can retain a Horn-clause core while adding structures for events, actions, reactive rules, and persistent state. Their appeal lies in combining logic programming virtues – declarative clarity, inspectability, and verifiability – with practical mechanisms for autonomy and interaction.

This tradition is richer than an actor substrate alone. BDI architectures model deliberation in terms of beliefs, desires, and intentions, while agent communication languages attach speech-act semantics to messages. Web Prolog standardises neither mental attitudes nor an agent communication language. Its private databases, behaviours, and term messages provide places in which such notions can be represented, but their meaning belongs to the framework layered above the substrate. This distinction prevents a mailbox-bearing process from being mistaken for a complete cognitive architecture.

Web Prolog does not present itself as yet another high-level cognitive agent language with built-in planners, ontologies, or a fixed BDI-style architecture. Instead,

it provides a lower-level actor substrate in which the primary abstraction is a process with a private database and a mailbox, encapsulated behind a process identifier and supervised by reliability patterns. In this respect it is closer to an Erlang-inspired concurrent logic substrate than to a monolithic agent language.

Precisely because the substrate is low-level and concrete, many classic agent-language ideas can be reconstructed. A receive loop that pattern-matches on incoming messages, together with an explicit representation of pending goals, already suffices to encode reactive rules of the form “on event then action” or “on event and if condition, then action” as ordinary Prolog clauses over the mailbox and the private database. In statechart actors, this becomes even more structured, allowing rich reactive and mixed-initiative behaviours to be expressed as statechart specifications interpreted by Web Prolog rather than as ad hoc control code.

Seen from this perspective, Web Prolog is again best understood as an enabler rather than a competitor. It supplies the web-native execution fabric on top of which more specialised agent frameworks can be layered. LPS-style engines, DALI-like event-driven agents, and more elaborate frameworks can all be realised as conservative extensions that treat Web Prolog actors as their concrete embodiment on the open Web.

8.3.6 Scalability and interoperability on the Web

Two traits of the Web are important for symbolic agents: established deployment machinery and protocol-level interoperability.

Scalability

Web Prolog agents are processes that can be spawned, monitored, and terminated. Standard deployment practices can be used around nodes: containers can replicate stateless endpoints, and ordinary observability tools can monitor host processes. Stateful actors require more care. A deployment must preserve or discover actor location, route later messages to the correct process, and coordinate lifecycle and recovery; a conventional load balancer cannot provide these properties by itself. Inside a node, actor supervision supplies a finer-grained failure boundary. The architecture therefore supports distribution, while large-scale operation remains a deployment problem rather than an automatic consequence of using HTTP or WebSockets.

Interoperability

The Web rewards components that speak stable protocols. On the Agentic Prolog Web, terms travel as structured data, which makes it natural to expose symbolic services through documented message schemas and to interoperate with non-Prolog

components through defined encodings. Conversely, Prolog agents can call REST services, interact with browser clients, or orchestrate external systems while keeping their internal reasoning symbolic and inspectable. A single canonical Prolog–JSON mapping is not assumed; as Chapter 11 discusses, representation choices remain part of the interoperability contract.

Distribution does not make every component symbolic. Many agents incorporate neural services for language, perception, or retrieval while retaining explicit rules for the commitments that matter. The next section examines architectures in which Web Prolog supplies reasoning and control, while neural models remain replaceable components.

8.4 Neuro-symbolic AI agents

To build a robust, knowledge-driven approach to AI we must have the machinery of symbol manipulation in our toolkit. Too much useful knowledge is abstract to proceed without tools that represent and manipulate abstraction, and to date, the only known machinery that can manipulate such abstract knowledge reliably is the apparatus of symbol manipulation.

Marcus and Davis (Marcus and Davis, 2019, p. 17)

Large language models are powerful generators and interpreters of language, but sampled generation does not by itself establish that a constraint has been satisfied or that an action is permitted. Neuro-symbolic AI is the broad family of approaches that combines learned representations with explicit symbolic structure or reasoning (d’Avila Garcez and Lamb, 2023). The coupling ranges from modules that exchange structured data at run time to systems in which neural and symbolic representations participate in one learning procedure.

This section concentrates on the architectural end of that range. Models supply language, perception, retrieval, or heuristic proposals; Prolog supplies explicit rules, search, and checks; actors and statecharts control when the components may act and how their failures are contained. These roles are design choices, not properties conferred automatically by either technology.

8.4.1 Why hybridise?

The productive question is not which paradigm should displace the other, but which component should hold authority over which commitment. Models are useful at turning heterogeneous human input into candidate structures and at turning structured results back into fluent language. Prolog is useful where rules, alternatives, and constraints must be named and executed. An actor substrate adds concurrency and explicit failure boundaries.

This is deliberately more modest than presenting symbolic AI as the complete route to intelligence. The history of expert systems and the Fifth Generation project

supplies reason for caution about such totalising claims (Körner et al., 2022). Nor does the proposed division imply that every model output is wrong or every Prolog result correct. Authority should be placed in the component whose semantics match the obligation, and any guarantee extends only as far as the formalised rules, trusted code, and resource assumptions on which it depends.

Aspect	Prolog component	Model component
Representation	Explicit clauses, terms, constraints, and control.	Learned distributed representations and generated sequences.
Natural role	Checking, search, policy, planning, and structured state.	Interpretation, generation, retrieval, perception, and proposals.
Evidence	Can return derivations, failed constraints, and other inspectable artefacts.	Can explain in fluent language, but generated explanations are not proof artefacts.
Failure modes	Failure, exception, divergence, or incompleteness; bounds and monitoring must make waiting observable.	Invalid, unsupported, or unstable output; external validation is needed when correctness matters.
Change	Rules and facts can be inspected and edited directly.	Behaviour changes through context, tools, model versions, or training.

Table 8.1 Complementary engineering roles in a hybrid agent.

Table 8.1 is a division of labour, not a scorecard. A useful shorthand is *the model proposes; the logic disposes*. The model may propose an interpretation, plan, tool call, or utterance. The symbolic layer decides which commitments follow under the rules it has actually been given.

8.4.2 Web Prolog as symbolic backend and agent layer

Python dominates machine-learning practice for good reasons, and nothing here argues against it. A hybrid agent nevertheless needs more than tensor libraries. Web Prolog combines symbolic search with mailboxes, supervision, and remote communication. A model service can therefore be one actor among others, or a co-processor called by a wrapper actor, without making the model itself the owner of lifecycle and policy.

This also clarifies the role of multi-agent decomposition. Assigning subgoals to specialist actors is a concurrency and deployment pattern, not by itself a form of neuro-symbolic integration. It becomes neuro-symbolic when some of those actors exchange learned representations or model outputs with symbolic components. Actor boundaries then provide useful locations for monitoring, timeouts, provenance, and access control.

Normative reasoning as executable policy

Legal, ethical, and organisational constraints can be modelled as rule sets in Prolog, including defeasible rules, exceptions, and priority schemes. This makes it possible to treat compliance not as post-hoc documentation but as executable policy: before an action is taken, the agent can attempt to prove that it is permitted under the current facts, and if no proof exists it can either refuse, ask for missing information, or escalate to a human.

For computational-law treatments in a Prolog setting, see, for example, the discussion in *Prolog: The Next 50 Years* and related work on executable norms and obligations (Kowalski et al., 2023a; Kowalski and Dato, 2022).

Bridging, not siloing

Hybrid architectures are not an either/or choice between Prolog and Python. Janus provides fast, bidirectional interoperation adopted by more than one Prolog implementation. Prolog can retain orchestration while delegating vectorised numerics, model inference, or tensor operations to Python libraries (Andersen and Swift, 2023; Wielemaker, 2023).

8.4.3 Architectural patterns for hybrid agents

The most useful patterns can be organised by the direction and closeness of the boundary between model and program.

A model calls Prolog as a tool

A model may emit a structured request for retrieval, arithmetic, symbolic query answering, or constraint checking. The surrounding orchestrator validates the request, invokes an authorised Prolog predicate, and returns a structured result. The model proposes the call; an ordinary program holds the authority and performs the effect. Allowed predicates, resource limits, and result schemas remain part of the service boundary.

SWI-Prolog’s experimental mcp pack supplies one concrete bridge to model-oriented orchestrators.³ A declaration `mcp_tool/3` exposes a Prolog predicate as a Model Context Protocol tool with a description and JSON Schema. This complements rather than replaces the Prolog Web protocols. MCP offers a bounded, schema-described tool call; the PTCP preserves a Prolog session with incremental

³ <https://github.com/JanWielemaker/mcp>. The repository describes the package as experimental; its version and supported protocol revision may change.

nondeterministic answers, prompts, and control messages. A knowledge base may usefully offer both interfaces.

Prolog calls a model service

Conversely, Web Prolog can delegate an open-ended language task to a model and treat the call as an effectful predicate guarded by timeouts, budgets, and policy checks. The exact provider API will evolve, but the architectural point is stable: the Prolog side controls when the call is made, how its result is validated, and what happens when it fails.

The following provider-neutral sketch shows the shape of an HTTP call to a text-generation endpoint:

```
:- use_module(library(http/http_client)).
:- use_module(library(http/http_json)).
:- use_module(library(option)).

call_llm(SystemMsg, UserMsg, Options, Reply) :-
    option(endpoint(Endpoint), Options),
    option(authorization(Auth), Options),
    option(model(Model), Options),
    Payload = json([
        model=Model,
        input=[
            json([role=system, content=SystemMsg]),
            json([role=user, content=UserMsg])
        ]
    ]),
    http_post(Endpoint, json(Payload), Reply,
        [request_header('Authorization'=Auth)]).
```

This is deliberately a provider-neutral sketch, not ACTOR-profile client code. A node owner would normally place the adapter behind an authorised predicate or service and align the payload with the provider's current specification.

Containing latency and failure

The HTTP predicate blocks while the provider responds. This is acceptable in a dedicated wrapper actor whose job is to wait for that service; it is not acceptable in SXML executable content, which should initiate long-running work asynchronously and consume the result as a later event. A statechart can send a correlated request and enter a waiting state whose transitions handle reply and failure events. A state-scoped *after* transition can handle the timeout: its timer is armed on entry, cancelled

automatically when another transition leaves the state, and renewed with a fresh activation if the state is entered again. A wrapper actor may instead supply a correlated timeout event when the timeout belongs to the external operation rather than to the control state.

The wrapper is also the natural failure boundary. It may be monitored or run under the supervisor behaviour of Chapter 5; a network exception can then terminate or restart it without corrupting the governing statechart. Correlation references prevent a late reply from an earlier request from being accepted in a later waiting phase.

A statechart governs the interaction

The surrounding layer that decides when to call a model, validates its output, enforces limits, and routes control is often called an agent *harness*. In Web Prolog, this harness can be an executable statechart rather than an informal combination of prompts, conditionals, and retry logic. Its states, guards, transitions, and stopping conditions answer the question “what may happen next?” independently of the model’s disposition. The model may propose actions or emit candidate events; only transitions admitted by the chart acquire behavioural effect.

Tighter neuro-symbolic integration

Some systems integrate learning and symbolic inference more closely. DeepProbLog permits neural predicates to supply probabilistic information inside ProbLog programs and supports learning from queries (Manhaeve et al., 2021). NeurASP combines neural-network outputs with answer-set programs (Yang et al., 2020). These systems use different semantics and training mechanisms; they should not be reduced to one generic “differentiable Prolog” architecture.

From the Trinity perspective, such a component could be exposed behind a predicate boundary like any other specialised reasoner. Keeping final policy in explicit rules, so that learned scores rank permitted alternatives rather than silently expanding the permitted action set, is a possible system design constraint. It is not a guarantee supplied automatically by DeepProbLog, NeurASP, or differentiability itself.

8.4.4 A statechart-wrapped service-request agent

To make the harness pattern concrete, consider an internal service-request agent. A requester describes a service in ordinary language; a model proposes structured fields; the requester confirms a specific summary; Prolog rules select automatic fulfilment, rejection, or human review; and an asynchronous dispatch acknowledgment determines the final outcome. The example develops the distinction between

confirmation and proposal discussed in Section 6.6.1. The model helps prepare a request; it does not authenticate the requester, grant approval, or establish that dispatch succeeded.

The runnable companion is in `companions/service-request/`. It contains the complete SXML chart, Prolog helpers, an offline demonstration, and executable tests. It runs on the demonstrator's local statechart interpreter. Its model and fulfilment adapters are asynchronous test doubles: no model account or real service desk is needed.

From the companion directory, the complete successful path is run with:

```
swipl -q -s service_request.pl -g service_request:demo -t halt
```

The accompanying README gives the runtime checkout layout and the test command. The demonstration proposes a laptop costing 700, confirms the summary, dispatches it, and reports `succeeded(ticket_42)`. The tests also exercise prohibition, policy conflict, missing information, reviewer decisions, malformed fields, stale events, and timeouts.

The trusted boundary and the request revision

The chart belongs to a trusted wrapper. That wrapper authenticates the user, authorises access to the request, and derives the principal from the session; it must not accept a claimed principal from model output or a request body. The companion fixes the requester as `alice` and the reviewer as `reviewer`. Its `submit/3` is a privileged local interface that simulates this boundary, not an implementation of authentication. The wrapper keeps the chart pid and a random envelope secret private. Only trusted adapters can attach that secret to model and dispatch replies. The model receives the request text and field schema, not these capabilities.

Each request has an identifier and an integer revision. Entering `collect` starts a linked adapter actor without waiting for the model. The adapter captures a fresh correlation identifier and the current revision, then wraps its eventual reply as `model_result(Key,C,R,Result)`. The offline adapter announces a `model_request` to the demo controller and waits for `deliver(Result)`; a provider adapter would perform the external call at that boundary. An error is also a result, not an exception allowed to bypass the chart.

The initiation itself is ordinary actor code:

```
begin_model :-
    context(Key, _, Observer, Request, _),
    revision(R, Text), uuid(C),
    retractall(pending(_, _)),
    assertz(pending(model, C)), self(Self),
    spawn(service_request:model_adapter(
        Self, Key, Observer, Request, C, R, Text),
        _, [link(true)]).
```

The essential collection path is below. Module qualifications are omitted from these excerpts; the companion contains the full executable chart.

```
<state id="collect">
  <onentry>begin_model</onentry>
  <go to="confirm" on="model_result(Key,C,R,fields(Fs))"
    if="model_reply(Key,C,R), valid_fields(Fs,Canonical)">
    remember(Canonical)
  </go>
  <go to="failure" on="model_result(Key,C,R,_)"
    if="model_reply(Key,C,R)">
    finish(failed(invalid_model_reply))
  </go>
  <go to="failure" after="5">
    finish(failed(model_timeout))
  </go>
</state>
```

The full chart also distinguishes recognised model errors from malformed replies. The `model_reply/3` guard checks the envelope secret, correlation, and revision. A late reply cannot replace the current proposal. The SXML after transition supplies the timeout: the interpreter arms its timer on entry and cancels it on exit. No external component has to invent a `timeout` event. Confirmation, review, and dispatch have their own timed transitions. The companion makes the duration configurable; ignored events do not restart it.

`valid_fields/2` accepts only a ground list containing at most one `service/1` and one `cost/1`. Service comes from a fixed vocabulary; cost is a non-negative bounded integer in fixed monetary units. Unknown or duplicate keys and invalid values are rejected. A valid partial list is allowed, but cannot be dispatched. The requester identity comes from the wrapper, not a model-supplied field. The wrapper displays a canonical structured summary; this application does not display unvalidated model prose.

Confirmation, correction, and policy coverage

Confirmation binds an authenticated principal to both the revision and the exact field list shown to that principal. A term named `confirmed` would establish none of this. Here the accepting transition is:

```
<go to="assess" on="authenticated(Key,Who,confirm(R,Fs))"
  if="confirmed(Key,Who,R,Fs)">
  record_confirmation(Who,R,Fs)
</go>
```

The guard requires the wrapper's secret, the authorised requester, the current revision, and exact equality with the stored summary. The confirmation record retains who confirmed which snapshot. A correction is accepted by the enclosing pre-dispatch state, including while collecting, awaiting confirmation, or awaiting review:

```
<go to="collect" on="authenticated(Key,Who,correction(R,Text))"
  if="correct(Key,Who,R,Text)">
  apply_correction(Text)
</go>
```

This increments the revision and clears the proposal, pending correlation, confirmation, assessment, and review decision before starting a new model request. Thus even a previously authentic confirmation is stale when it names an old revision. The requester can also cancel before dispatch. Once dispatch starts, that revision is frozen: a correction cannot retract an external effect that may already be under way.

Assessment is total over validated proposals. The example uses the following explicit precedence, rather than two guards that might both fail or both succeed:

```
classify(Fs, insufficient(missing_fields)) :-
  \+ complete(Fs), !.
classify(Fs, conflicting) :- permit(Fs), forbid(Fs), !.
classify(Fs, forbidden) :- forbid(Fs), !.
classify(Fs, permitted) :- permit(Fs), !.
classify(_, insufficient(policy_gap)).
```

Its small policy permits laptop and VPN requests costing at most 1,000 and forbids restricted services and VPN requests. The VPN overlap is deliberate: a request satisfying both rules goes to review, not automatic fulfilment. A pure prohibition is rejected. If neither rule succeeds, lack of policy coverage is recorded explicitly.

Missing fields likewise produce an explicit information deficit. The chart cannot become stuck in assess:

```
<state id="assess">
  <onentry>assess</onentry>
  <go to="route" if="decision(permitted)"/>
  <go to="failure" if="decision(forbidden)">
    finish(rejected(policy))
  </go>
  <go to="review"/>
</state>
```

Human review is an implemented state. It displays the revision, confirmed fields, and assessment, then awaits an authenticated `review(R,Fs,approve)` or `review(R,Fs,reject)`. Its guards check the reviewer role, current revision, and exact snapshot just as confirmation does. The example explicitly authorises a reviewer to resolve a conflict or a policy gap for a complete request. It does not authorise overriding a pure prohibition or approving missing fields. An incomplete request must be corrected or rejected; silence ends in review timeout. A correction invalidates any outstanding review decision as well as the requester's confirmation.

Dispatch and terminal outcomes

Entering route starts a second asynchronous adapter, but only after checking a current confirmation and either permission or an authorised review approval. The adapter captures its own correlation and the frozen revision. It also receives the idempotency key `key(RequestId,Revision)` and the confirmed payload. The essential acknowledgement path is:

```
<go to="success" on="dispatch_result(Key,C,R,ok(Ticket))"
  if="dispatch_reply(Key,C,R), atom(Ticket)">
  finish(succeeded(Ticket))
</go>
<go to="failure" after="5">
  finish(failed(dispatch_timeout_unknown))
</go>
```

The full chart validates negative acknowledgements too: a known refusal or unavailability ends in `failed(dispatch(Code))`. An invalid acknowledgement has an explicit unknown-outcome result. Both final states publish the recorded outcome and terminate the chart; linked adapter workers are then stopped. Requester cancellation, reviewer rejection, invalid model output, and timeouts also have terminal results.

Dispatch timeout means that no usable acknowledgement arrived, not that the external action did not occur. The companion does not retry automatically. A production backend must durably deduplicate the key and reconcile status before an

uncertain operation is retried. Likewise, stopping a local adapter cannot undo an accepted remote operation. These are obligations of the trusted dispatch integration, not properties obtained merely by naming a state `route`.

The model provides flexibility in interpreting a request; the statechart provides the interaction sequence; and Prolog rules decide which routes are permitted under the represented policy. Replacing the model need not change the confirmation boundary, stopping conditions, or escalation paths.

None of this gives an agent “common sense” merely because its rules are written in Prolog. Commonsense reasoning is open-ended, contextual, often defeasible, and sensitive to time and incomplete knowledge. A Prolog program can implement a chosen fragment, but its conclusions inherit the coverage, negation policy, priorities, and consistency of that fragment. Learned components may broaden recall, yet their proposals still require validation when the missing knowledge concerns a safety or policy obligation.

8.4.5 Open engineering and research questions

The service-request agent illustrates a relatively clean division of labour, but assigning roles to the model, logic program, and control layer does not by itself solve the problems at their boundaries. Evidence must remain traceable as it crosses components, structured representations must retain their meaning, generated explanations must not outrun their symbolic support, and policy checks must mediate every relevant effect. The following are therefore not capabilities claimed for the present demonstrator, but engineering and research questions suggested by the architecture.

- **Tracing and audit.** Symbolic execution can produce proof objects, constraint stores, and, where explicitly captured, search traces. Model interaction produces a different kind of record: prompts, tool calls, model and version identifiers, sampling settings, and returned outputs. Correlating these records across actors and nodes into a coherent audit trail remains a practical challenge and an opportunity for web-native agent engineering.
- **Proof artefacts as user-facing explanations.** Proof trees and derivation traces are precise but not always pleasant to read. An LLM could serve as a commentary layer that renders them as structured explanations. The generated explanation should remain linked to the proof fragments that support it and must not be treated as establishing more than the underlying derivation establishes. Minimal justifications and counterfactual analyses may also require additional symbolic computation rather than language generation alone.
- **Intermediate representations.** Stable, versioned schemas at the boundary between neural and symbolic modules reduce dependence on brittle prompt conventions. JSON provides a convenient encoding, but not by itself a semantic contract; validation rules, identifiers, and provenance conventions must also be specified. JSON-LD may help where linked identities and web-visible provenance matter.

- **Safety envelopes.** When an agent must enforce policy, the authoritative checks need to remain outside the model and execute before the corresponding effects. In Web Prolog they can reside in the symbolic backplane and supervisory wrapper. The harder problem is ensuring complete mediation: indirect effects, delegated tool calls, retries, and actions performed by other agents must not bypass the same policy boundary.

8.5 User-interface agents

The Agentic Prolog Web is a substrate for multi-agent interaction in general, but an important class of agents mediates between a person and a computational environment. For these user-interface agents, the engineering problem is not only what can be inferred, but how an interaction is managed over time: when to ask, when to confirm, how to repair misunderstandings, how to handle silence and timeouts, and when to refuse or escalate.

The social-robot “parrot” of Section 3.5.6 can now be given a brain without changing its transport architecture. The robot node may continue to own speech recognition, synthesis, gaze, and embodiment, while a remote brain actor owns the dialogue governor, symbolic knowledge, and any model wrapper. Utterances, interpretations, and action requests cross the boundary as messages. This separation keeps device-specific timing and effects near the body while making the conversational policy independently inspectable, replaceable, and supervisable.

This section therefore shifts emphasis from agent-to-agent coordination to dialogue as control. Conversational behaviour should be represented as a program with states, transitions, and failure handling rather than left as an emergent property of a prompt. Web Prolog can combine dialogue-relevant facts, executable parsing, and an actor-oriented statechart controller.

8.5.1 UI agents versus MAS agents

In classic multi-agent systems, interaction is often modelled as peer-to-peer exchange: negotiation, coordination, and distributed problem solving among relatively symmetric participants. User-interface agents are asymmetric. They exist to serve a person, and they are evaluated by human-facing criteria such as clarity, trust, timing, politeness, and error recovery. The success condition is not only that the agent eventually produces a correct plan, but that it maintains an intelligible and safe interaction while doing so.

On the Web, this role frequently manifests as a conversational web agent: an agent that accepts input in natural language (text or speech) and produces responses that guide the user through a task. In such systems, the boundary between reasoning

and interaction is not clean. Questions, confirmations, clarifications, and refusals are themselves action choices that must be governed by policy.

8.5.2 Turn-taking and repair as control structure

Conversation, in the technical sense used in linguistics, is a jointly managed activity. Turn taking is locally coordinated (Sacks et al., 1974); questions and answers, offers and acceptances, and requests and responses create expectations about what may come next. When understanding is uncertain, participants use repair and grounding moves to establish enough common ground for the current purpose (Clark and Brennan, 1991).

For an engineered UI agent, the practical implication is that interaction cannot be reduced to a single request–response call. The agent needs persistent state and control over what may happen next. It must be able to suspend a task to ask a question, resume after an answer, back up after a misunderstanding, and time out or escalate when interaction fails. These are precisely the kinds of behaviours that state machines and statecharts are designed to express.

A useful mental model separates *flow* from *content*. Flow governs when the system may advance, when it must confirm, what to do on silence, and how to recover after misunderstanding. Content covers parsing, data-model updates, and reasoning. Both can be expressed as programs, but a statechart makes the permitted interaction sequence particularly visible.

8.5.3 Dialogue as proof: the missing-axioms view

A useful way to connect dialogue management to symbolic AI is the Missing Axioms Theory (Smith et al., 1995). In this view, the agent’s task can be phrased as an attempt to prove a goal. When the proof fails, the reason is often not that the rules are wrong, but that some required facts are missing. Dialogue then becomes a controlled process for acquiring the missing axioms.

This perspective is already latent in the askable expert-system pattern from Section 8.2.4 (and, in web-native form, Section 8.3.1). There, proof search determines which question must be asked next. The limitation of the toy meta-interpreter approach is not the idea, but the control envelope: real dialogue involves interruptions, repairs, topic shifts, and timeouts. To handle those robustly, we need an interaction controller with persistent state and well-defined failure handling.

8.5.4 Statecharts as dialogue managers

In Web Prolog, a statechart actor can serve as the authoritative dialogue manager, while Prolog code in its data model supplies parsing and reasoning. The following small statechart illustrates the arrangement. It collects two attributes and uses a history state to recover after a misunderstanding:

```
<statechart version="0.2" initial="Start">
  <datamodel>
    :- dynamic color/1, size/1.
    np(Attr) --> [a], adj(Attr), [apple].
    adj(color(red)) --> [red].
    adj(size(big)) --> [big].
  </datamodel>

  <state id="Start" initial="Color">
    <history id="H">
      <go to="Color"/>
    </history>
    <state id="Color">
      <onentry>
        writeln('What color do you want?')
      </onentry>
      <go to="Size"
        on="input(Input)"
        if="phrase(np(color(C)), Input)">
        assert(color(C))
      </go>
    </state>
    <state id="Size">
      <onentry>
        writeln('What size do you want?')
      </onentry>
      <go to="Final"
        on="input(Input)"
        if="phrase(np(size(S)), Input)">
        assert(size(S))
      </go>
    </state>
    <go to="H" on="input(_)">
      writeln('I did not understand.')
    </go>
  </state>
  <final id="Final">
    <onentry>
```

```

        size(S), color(C),
        format('You want a ~p ~p apple!~n', [S,C])
    </onentry>
</final>
</statechart>

```

Three elements matter here.

Repair via history

The history state supports a simple repair policy for unrecognised input. When an `input(.)` event fails to match any more specific transition, the machine emits a complaint and transitions to the history pseudostate. This re-enters the most recently active substate of the dialogue, causing its `<onentry>` prompt to run again and thereby re-asking the current question. In contrast to restarting the whole interaction, the repair stays within the current plan, resembling how human repair sequences typically preserve the larger conversational trajectory.

Parsing as a guard

Transition guards call DCG parsing directly, so the controller advances only when the incoming `input(Input)` can be parsed into the intended structured form. This makes the interface between interpretation and control explicit: the grammar determines what counts as understood, while the statechart determines when understanding is sufficient to commit to a control move.

Data-model updates as explicit effects

Executable content makes data-model updates explicit and localised. When an input is successfully parsed, the extracted fact is asserted as a named predicate in the statechart's data model. These stored facts can later be queried, audited, and constrained by ordinary Prolog rules rather than remaining implicit in conversational context.

Together, history-based repair, guard-based interpretation, and explicit data-model updates yield a compact pattern: recognise $\rightarrow$ commit $\rightarrow$ store; otherwise complain $\rightarrow$ repeat.

8.5.5 Dialogue as a governed interface

The same statechart harness that governs tool use can govern what an agent may say and do with a person. A model may be invoked for paraphrasing, interpretation, or

question generation, but its output re-enters the system through guards, schemas, and policies. The chart thereby preserves the observable interaction pattern – turn taking, repair, refusal, and escalation – even if the model is replaced. Such portability also depends on stable service contracts and shared SXML semantics; it is not guaranteed by the diagram alone.

8.6 Summary

This chapter framed Prolog’s relevance to contemporary AI as an operational question: can symbolic constraints, search, and explanations be made web-addressable and composable, so that they can participate in agentic systems? We first recalled Prolog’s symbolic competence on a single machine: knowledge as executable clauses, planning as search, proof objects, interactive acquisition of missing facts, and grammar-based language processing. We then lifted these idioms onto the Agentic Web, treating Prolog engines as networked actors and allowing proofs and explanations to span nodes.

On that foundation, we distinguished the book’s architectural agents from classical cognitive agents and contemporary LLM agents. Hybrid systems can assign linguistic and perceptual work to models while keeping represented constraints and interaction policy in explicit programs. Potentially blocking model calls belong in monitored wrapper actors; statecharts initiate the work asynchronously and react to replies, errors, and timeout events.

Finally, user-interface agents showed the same control structure from the human-facing side. Turn taking, repair, confirmation, timeouts, refusals, and escalation can be represented as executable interaction policy rather than left implicit in a prompt.

Web Prolog does not supply intelligence by itself. It supplies an executable behavioural boundary within which heterogeneous forms of intelligence can act: a distributed substrate in which state, permissible transitions, failure handling, and reasons remain explicit.

Part IV
From architecture to paradigm

Chapter 9

The Trinity as a web-native symbolic paradigm

The preceding chapters developed the three components of the Prolog Trinity: Web Prolog as a language profile, actors and nodes as computational agents, and the Prolog Web as the network over which they interact. They also introduced reusable behaviours, statechart actors, Semantic Web interoperability, and applications to AI.

Taken in isolation, none of these ideas is entirely new. Web Prolog extends established Prolog practice; its actor model draws on Erlang; remote querying has a long history; and statecharts extend conventional state machines. The interest lies in how these elements fit together.

Chapter 1 presented the two computational partitions, the affinities between Prolog and the Web, and the mutual dependence of the Trinity's components as motivations for the project. Having now developed them in detail, we can ask what programming model emerges from their integration. This chapter argues that the result is a coherent architectural stance – a candidate paradigm for executable symbolic systems on the Web.

Here, *paradigm* is used in the ordinary programming-language sense, not to claim a Kuhnian revolution. Chapter 2 used Van Roy's taxonomy (Van Roy, 2009) to locate Web Prolog among relational and logic programming, imperative search programming, and message-passing concurrency. The claim made here is that adding nodes, node-qualified process identities, profiles, and remote nondeterministic calls produces more than a collection of facilities: web distribution becomes part of the computation model, and the inherited and added concepts constrain one another strongly enough to deserve consideration as a further point in that design space.

With that framing in place, this chapter asks a disciplined question:

What conceptual pattern emerges when logic programming, actor-based concurrency, Web transport semantics, and structured behavioural abstractions are deliberately integrated into a single design?

The answer proposed here consists of six interlocking commitments: a separation between sequential and concurrent computation; portability grounded in observable interaction; agents as the primary units of structure; a logical foundation that sur-

vives distribution; distribution as part of the programming model; and interaction governance as a first-class design dimension.

These commitments are mutually constraining. The language profile determines what can be exposed safely and portably; the agent model determines which long-lived behaviours can be made robust; the network substrate determines which forms of composition are realistic at web scale. It is precisely this mutual constraint that prevents the proposal from collapsing into “Prolog over HTTP”, “actors in Prolog”, or “a distributed Prolog database”. No single strand can be redesigned in isolation without consequences for the others.

9.1 The two partitions as a foundational principle

Chapter 4 defined the sequential and concurrent partitions over observable interaction contracts. Sequential contracts present goal sessions whose answers, committed effects, prompts, and errors remain intelligible without exposing process interleavings. Concurrent contracts make independently progressing processes, asynchronous messages, mailbox selection, and lifecycle events part of what programs may observe. Constructs, programs, nodes, and profiles are classified only derivatively from the contracts they expose.

The paradigmatic point is not the definition alone but its independence from other design axes. Purity is separate: a sequential ISOTOPE session may update its private database or perform I/O, while a pure remote call can retain a logical reading. Transport and statefulness are separate: actors can implement a stateless RPC surface, and a stateful session can remain sequential. Implementation is separate for the same reason. Openness introduces yet another axis, since public execution surfaces create security and resource-governance obligations regardless of which transport carries them.

The profile hierarchy gives this independence an operational form. **RELATION**, **ISOBASE**, and **ISOTOPE** expose only sequential contracts, although **ISOBASE** adds the split-phase **promise/yield** interface and **ISOTOPE** adds session state. **ACTOR** is the first profile that also makes the concurrent partition client-expressible, while retaining the sequential surfaces inside it. The hierarchy therefore broadens the observable contract without making purity, state, or transport stand in for the partition boundary.

The boundary case **parallel/1** makes the same point from another direction. Its predicate surface is synchronous, but when several workers produce abnormal outcomes its first-observed result exposes scheduling. Thus synchrony alone does not settle the contract’s partition; the observable multi-fault rule matters too.

Real systems will commonly mix the two: pure queries for declarative reasoning and actors for coordination, state, and lifecycle management. A useful discipline is incremental relaxation – begin with the purest model that fits the problem, and introduce procedural commitments where the application requires them.

This programming-model orthogonality is compatible with the architectural mutual constraint developed in Section 9.3.2. The former separates two classes of interaction contract; the latter says that the language profile, agent model, and network substrate must be designed together. Indeed, keeping the partitions separate depends on a disciplined interface between Prolog engine state and actor process state.

9.1.1 Two levels of abstraction

The actor-based reconstruction of the sequential interface in Section 4.2 supplies the mechanism for a striking paradigm property: the Prolog Web can be read simultaneously at two levels of abstraction. On the upper level lies a *level of logic programs* – interlinked Prolog clauses distributed across nodes, where URIs serve as references between knowledge bases and `rpc/2-3` calls serve as the operational links. On the lower level lies a *level of communicating processes* – often realised as toplevel actors that are created, exchange messages, and eventually terminate.

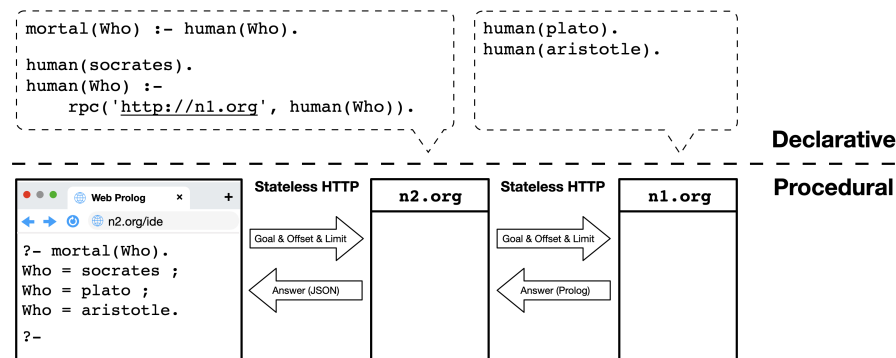


Fig. 9.1 Two levels of abstraction: interlinked logic programs above the line and communicating processes below it.

Figure 9.1 revisits the scenario of Figure 4.6, now drawing the distinction between the two levels explicitly. On the lower level, processes are created and destroyed; while alive, they send messages to each other. Nothing of this kind takes place on the level of logic programs. Entities such as processes, messages, and mailboxes do not exist there – they have been abstracted away. Drawing a line between these two levels is another way to express the distinction between declarative and procedural that so often appears in the story of Prolog. “What?” is specified at the level of logic and the corresponding “How?” is handled on the level of communicating toplevel processes.

This two-level reading is a paradigmatic property. When the sequential partition is pure, the Prolog Web admits both a *logical* reading – in which distributed computation is understood as inference over distributed knowledge bases – and a *procedural* reading – in which the same computation is understood as coordinated message exchange among network processes. Sometimes, when actors and side effects are involved in a fundamental way, only the procedural reading remains. That both readings are available at all in a distributed setting, and that the programmer can move between them, is one of the defining characteristics of the Trinity as a web programming paradigm.

9.1.2 Unification as the universal data contract

The interface between client and node needs a structural contract. In many Web APIs, request schemas, response schemas, and the code that relates them are separate artefacts.

In Prolog, unification makes the coupling direct. A goal sent to a node is simultaneously a request for information, a specification of the expected structure of answers, and the compatibility test between the two. The query

```
?- employee(Name, Dept, salary(Min, Max)).
```

both asks for employees and states the expected answer structure. If a matching fact exists, bindings flow back. If `employee/3` is undefined, the Web Prolog contract reports an existence error at the point of invocation. The query, as Chapter 1 put it, *is* the contract.

It is worth being precise about how divergence manifests, since this is where the contract earns its keep. A relation that has changed arity or disappeared should raise an error; a goal that fails to terminate may surface as a timeout; only the genuine absence of matching data should yield failure. This three-way distinction must be part of the node contract rather than left to a host Prolog’s configurable treatment of undefined predicates; Section 10.1.3 returns to this example.

Composition is native: constraints combine by conjunction, for example

```
?- employee(Name, engineering, salary(_, Max)), Max > 100000.
```

The combined query can be executed entirely on the node or split between remote and local evaluation. Because a Prolog predicate is relational, the instantiation pattern chosen by the client can provide different access modes through the same contract. Projection is expressed by choosing which variables the caller retains, rather than by defining a new response format. Evolution is also visible at the call boundary: if a relation changes name or arity, an old client receives a specific error instead of silently accepting a structurally unrelated response.

The contract is structural, not semantic. Unification can ensure that terms have compatible shapes and that shared variables satisfy the same equality constraints, but it cannot determine whether two parties attach the same meaning to a predicate

or to its arguments. That agreement must still come from documentation, a shared vocabulary, or a shared ontology. Chapter 7 develops the complementary division of labour: unification disciplines structure at the point of call, while Semantic Web vocabularies and ontologies make intended meaning explicit.

9.1.3 Backtracking as pagination

Prolog backtracking and Web pagination instantiate the same abstract idea: incremental enumeration under external control. A paginated endpoint supplies results in slices; a toplevel actor can do the same, with a retained choice point as the resume mechanism and a “next” message as the page-turn signal.

Within a slice, the caller enumerates solutions locally. Only when that slice is exhausted must it cross the network boundary to request another. The protocol can therefore expose nondeterminism incrementally without turning every individual solution into a separate remote call.

Chapter 4 developed the operational alternatives, the per-invocation stability supplied by the logical update view, and the additional mechanisms required for a point-in-time snapshot. Here the important alignment is that unification supplies the data contract, backtracking the enumeration model, and HTTP or WebSockets the transport. More generally, terms, clauses, goals, and substitutions give knowledge representation, communication, and computation a shared symbolic form.

9.2 From compatibility to interaction portability

The Prolog community has long understood portability as a matter of *compatibility*: the extent to which different systems happen to accept the same programs and produce similar results. Efforts such as the ISO Prolog standard have addressed this at the level of the core language. Yet, as surveyed in *Fifty Years of Prolog and Beyond* (Körner et al., 2022), portability issues remain widespread beyond the core – in libraries, constraint systems, concurrency models, and interfaces to the outside world.

Compatibility, even when it covers both syntax and behaviour, is an *empirical* property: it depends on the contingent alignment of implementations, and it is easily broken by innovation. It asks whether two systems happen to agree, not whether they are obliged to. The Prolog Web sharpens the problem further: in a network of nodes implemented on top of different Prolog systems, the question is no longer just whether two systems accept the same program and produce the same answers, but whether they preserve the same observable *interactions* – the same patterns of queries, answers, and message exchanges as seen by clients and peer agents.

In this work, we adopt a different perspective. Rather than treating portability as empirical compatibility between systems, we treat it as a designed property of programs, defined over their observable interactions.

Interaction portability. A program is portable across a declared class of execution environments, under stated external dependencies and observations, when relocation preserves its permitted observable interaction behaviours – the relevant queries, answers, effects, messages, and lifecycle events.

Preservation here means observational equivalence, not identical wall-clock timing or reproduction of one fixed trace. A nondeterministic program may admit several answer orders or message interleavings; relocation is portable when it preserves the permitted set rather than selecting one particular trace.

The service-request agent in Section 8.4.4 makes the question concrete. If its request actor moves to another runtime, what must remain true? A confirmation must still concern the same revision and field snapshot, an old reply must not advance a corrected request, and dispatch must remain subject to the same policy. If an acknowledgement is lost, relocation must not turn an unknown external outcome into an assertion that nothing happened. The relevant observations therefore include which effects may be authorised and what the caller can learn about them, as well as returned terms.

This does not require the same pid or elapsed time on both runtimes. It does require a declared treatment of identity, timing, and dependencies: a different policy source or trust boundary is not merely a different placement. Running the local example in two environments supplies observations under its test conditions; it does not establish preservation across all admissible interactions. The example helps specify the obligation rather than discharge it.

This shift has several important consequences. First, portability becomes relative to a well-defined class of environments: a design decision, not an accident of history. Second, interaction rather than textual acceptance becomes central: two systems may accept the same program and yet differ in observable interaction behaviour, and under this notion of portability such differences determine whether portability holds. Third, portability must be *designed* rather than inferred: it requires explicit control over the aspects of execution that affect observable interactions.¹

The Trinity provides the ingredients from which this stronger notion can be built. Web Prolog restricts the client-supplied fragment so that self-contained programs need not depend on uncontrolled host features; owner-defined predicates and optional libraries remain explicit dependencies. Agents and nodes structure execution around message passing and explicit interaction, making behaviour visible and comparable across environments. Node profiles then provide the first reference points relative to which interaction portability can be assessed.

The same principle applies to substitution as well as relocation. The statechart dialogue governors of Chapter 8, for example, constrain the observable sequence of

¹ The distinction drawn here has a parallel in the distinction between the meaning of expressions and meaning realised through their use in interaction.

turns, confirmations, repairs, and refusals while the language model behind them may be replaced or re-prompted. Node profiles support portability across execution environments; dialogue governors seek a related invariance across substitution of a fallible component. In both cases, the stable object is the declared interaction contract rather than the implementation that realises it.

9.2.1 Node profiles as interaction contracts

Returning to the primary axis – relocation across execution environments – the notion of interaction portability is meaningful only relative to a well-defined class of such environments, and those classes must therefore be made explicit. Chapter 3 introduced node profiles as conformance declarations – specifications of what language features and web APIs a node provides. A profile is not, by itself, a claim that any two nodes bearing its name have identical data, libraries, policies, or owner-defined predicates. It does, however, delimit the observable surface on which a stronger interaction contract can be stated and tested.

What varies across profiles is therefore both what a node can do and which aspects of its behaviour become relevant to portability. At each level the observable surface grows wider:

- **RELATION.** Observable interaction consists of answer substitutions delivered through the paged stateless API. Each node must provide an order stable enough for its own pages to be meaningful, as described in Section 3.3.3, but the profile specifies no canonical order shared by independent backends. Portability between two RELATION nodes therefore requires equivalent advertised relations and answers, including multiplicity and any ordering observed by the program; pages from different nodes cannot in general be spliced into one enumeration.
- **ISOBASE.** Observable interaction extends to the evaluation of goals in the ISOBASE fragment, including Prolog answer order, control constructs, and synchronous and asynchronous RPC. Client-supplied source may accompany a request, with scope limited to its temporary execution context. Incremental delivery is already present at RELATION; what ISOBASE adds is a language-level basis for asking whether answer order and continuation behaviour agree. Such agreement is required for a particular program to be interaction-portable, but it does not follow from the profile alone when owner code, libraries, or policies differ.
- **ISOTOPE.** Observable interaction extends to side effects within a persistent toplevel session: supplied definitions and dynamic database updates can survive across requests, and the session supports interactive I/O. The extension is persistent session scope; supplied source is already available at ISOBASE with temporary scope. Portability must therefore cover not merely answers and their order, but also the effects produced along the way and the state retained between requests. This is the most demanding level within the sequential partition.

- **ACTOR.** Observable interaction extends to message exchanges, mailbox semantics, and process lifecycle. Portability here depends on the delivery guarantees, the selection semantics of `receive/1-2` (Section 2.3.4), the failure-notification vocabulary, and the lifetime discipline for spawned processes. These are precisely the parts of the actor contract that must be pinned down independently of any one implementation.

The profiles thus form a hierarchy of increasingly rich observable surfaces, and interaction portability becomes more demanding as one climbs it. What may be sufficient for a **RELATION** program – agreement on answer streams and paging behaviour – is not sufficient for an **ACTOR** program, which also requires agreement on the dynamics of interaction itself. Future conformance specifications can turn selected parts of these requirements into normative profile guarantees; the present profiles should not be read as having completed that work already.

Under this view, portability is no longer a fragile byproduct of partial standardisation. It becomes a first-class design principle: programs are written against a profile rather than a specific implementation, their meaning is assessed in terms of observable interactions, and systems are free to innovate internally as long as they preserve the relevant contract. As shown in Section 4.3.3, when a program is self-contained and the target environment implements the required profile, relocation may require no adaptation. Programs that rely on owner resources, optional libraries, or particular service semantics must carry those dependencies explicitly.

9.3 Agents as the primary abstraction

At the centre of this design space stands the agent, used here in an architectural rather than a rational or intentional sense: an independently addressable participant with a local program or state, an interaction boundary, and a lifecycle. Actors and toplevels fit this definition directly; statechart actors are actors governed by an explicit model; nodes are higher-level agents that host programs and processes, expose services under a policy, and mediate between them and the network.

Actors are therefore agents. Toplevels are agents. Statechart processes are agents with explicit reactive control, and nodes are agents at a larger granularity. A node is not merely a Web server that happens to execute Prolog goals: it is a situated participant that owns resources, hosts durable knowledge and services, advertises a profile, and enforces policy. External services, browsers, and AI components can likewise be treated as agents when they participate through defined interaction boundaries.

The Trinity consequently adopts a process-and-message discipline in which identity, mailbox semantics, and lifecycle policy provide the main structure. Logical reasoning occurs *within* agents, distribution occurs *between* agents, behaviours constrain classes of agents, and statecharts govern reactive control.

The choice has far-reaching implications. It aligns naturally with distribution-first semantics: because the agent is already a process with a mailbox, the step from

local to distributed execution does not require a change of abstraction. It supports fault containment and explicit failure observation through links and monitors. It also allows sequential reasoning and reactive control to coexist inside one application without conflating their interfaces.

Once nodes host durable processes that interact over published protocols, the Prolog Web is a multi-agent system in this architectural sense: global behaviour emerges from locally scoped programs coordinating under failure and timing constraints.

We saw in Section 2.2.1 that Armstrong characterised the Erlang approach as structuring applications from large numbers of communicating parallel processes – an architectural infrastructure of well-defined, independently implementable sub-components. The same principle applies directly to the Prolog Web. But in the Trinity, the process-and-message discipline is not the whole story – it is held in place by a tighter structural property.

9.3.1 Interaction as inference

The agent-centric and logical commitments of the Trinity meet when an agent decides how to react. An incoming message contributes a partial substitution by unifying with a receive or transition pattern. A logical guard may then extend that substitution by proving a goal within the agent's *logical environment*. Selection therefore produces not only a decision, but also information that can flow into the ensuing action. In this sense, a mailbox message is a transient premise rather than the complete input to a dispatch function.

The construction also connects the two partitions of Web Prolog. Receiving a message is an effect in the concurrent partition: a message is selected and consumed under an ordering discipline. Determining whether the message enables a receive clause or statechart transition invokes the sequential partition: a Prolog goal is proved within the agent's logical environment. The outcome of that proof then determines the next concurrent action.

Logical guards are consequently a small but concrete instance of the Trinity's central claim: symbolic reasoning need not sit beside interaction as a separate service, but can participate directly in deciding what an interaction means. The operational semantics are given in Section 2.3.4; Section 6.5.1 shows their reuse in statecharts.

9.3.2 Mutual constraint

The three components of the Trinity are mutually constraining – a property more characteristic of a paradigm than a toolkit. The language profile constrains the agent model: restricting remotely supplied code narrows what agents must defend against and which operations they may expose. The agent model constrains the network substrate: identities, mailboxes, sessions, and lifecycles require naming, message

transport, and failure notification. The network substrate constrains the language profile in return: goals, answers, messages, and code fragments must have portable representations and well-defined behaviour at transport and failure boundaries.

No single component can be redesigned independently without affecting the others. This tight coupling is not a fragility; it is the source of the design’s coherence. It is also why the Trinity is not well described as “Prolog plus actors plus HTTP”. The components do not merely coexist; they shape each other.

For example, suppose a node admitted an unrestricted file-opening predicate into remotely supplied goals. The language profile would no longer provide inexpressibility as a sandbox boundary; actor containment alone could not prevent a client from reading node-owned files; and the HTTP and WebSocket bindings would need a substantially stronger authorisation and audit model. Conversely, retaining a portable profile without file I/O lets every transport and actor context rely on the same structural restriction. A change in one corner therefore changes the obligations of the other two.

9.3.3 Capability-oriented security

One consequence of this mutual shaping is the security model. Because the language profile, the agent model, and the network substrate constrain each other, part of the security posture can be structural rather than imposed only by external checks.

The structural mechanisms that make the Prolog Web safe to open – inexpressibility, invisibility, and containment – are developed in Section 4.5. What matters paradigmatically is that these mechanisms constitute a *capability-oriented* layer. Pids and narrowly scoped reply channels convey the ability to address actors. The `Id@Node` representation is public and constructible, but the node assigns identifiers and controls their use. Possession of a valid pid is necessary but not always sufficient: node policy may still require authorisation for its use. Public node URIs and published service names, by contrast, are discoverable addresses governed by ordinary access policy, not secret capabilities. Inexpressibility is enforced at the language boundary: dangerous operations cannot be named in the client fragment. Nondisclosure of a pid narrows the set of actors a client can address, but does not turn the pid into authentication. Containment is enforced by lifetime dependency: when a session ends, its actor subtree disappears. These mechanisms reduce what scattered runtime checks must accomplish, but they complement rather than replace authentication, authorisation, resource governance, auditing, and transport security.

This structural character aligns the Prolog Web with a family of contemporary execution environments – browser-based JavaScript, WebAssembly, WASI – that share the same posture: code executes inside a deliberately bounded environment and acquires authority only through explicitly provided references. Traditional Prolog systems are designed around maximal expressiveness; Web Prolog inverts the default, treating expressive power as owner-controlled authority that can be selectively granted rather than silently assumed.

9.3.4 Five complementary views

The agent-centric design refracts into several complementary perspectives, each emphasising different linked entities and activities:

The Prolog Web of source code. URIs link programs to programs. Nodes may expose source code over HTTP so that users and tools can follow those links.

The Prolog Web of query solutions. URIs link callers to answer streams. Interaction is sequential and pull-based: a client asks a node to solve a goal and enumerates its solutions incrementally.

The Prolog Web of actors. Node-qualified pids link live processes with mailboxes. Interaction is asynchronous, while links and monitors expose lifecycle dependencies and failure.

The Prolog Web of behaviours. Behaviours specify message syntax, semantics, and timing; actors are their running instances.

The Prolog Web of nodes. Nodes are the places of the system. They store and serve code, host actors, enforce policy, advertise profiles and services, and provide points of execution and accountability.

These are not competing architectures but complementary perspectives on the same system. A Prolog Web application may involve several simultaneously: source code that is browsable, goals that are queryable, actors that are addressable, behaviours that are composable, and nodes that are accountable.

9.4 The logical foundation: purity, belief, and scoped inference

The agent-centric view supplies the structural principle; the logical foundation supplies the semantic one. The Trinity is not merely a concurrent system that happens to use Prolog syntax. Logic programming plays a constitutive role, and the key to understanding that role is the notion of a *pure Prolog Web*.

9.4.1 Pure Web Prolog and the pure Prolog Web

As noted in Chapter 4, *rpc/2-3* retains the logical purity of the predicates it calls. This allows us to form the notion of *pure Web Prolog*:

$$\text{Pure Web Prolog} = \text{Pure Prolog} + \text{rpc/2-3}$$

This equation assumes that the called predicates and their logical context are pure and stable for the duration of the call. Timeouts and connection failures may truncate the answer stream, but do not change the logical status of answers already returned. The union of such fragments forms what we call the *pure Prolog Web*: an aspirational Web of Logic whose distributed answers remain consequences of pure programs.



Fig. 9.2 A pure Prolog Web spanning the globe remains aspirational, but the architecture does not preclude it.

9.4.2 Epistemic reading and scoped inference

If agents are the primary abstraction and the pure Prolog Web gives them a logical character, then a call such as `rpc(URI, Q)` admits an optional epistemic reading: it asks whether the logical environment at URI supports Q. A clause such as

```
human(X) :- rpc('http://n1.org', human(X)).
```

can, using a belief operator – equivalently, a predicate over reified formulas – be formulated as

$$\forall x[\text{believes}(n_1, \text{human}(x)) \rightarrow \text{human}(x)]$$

It makes perfect sense to say that one agent believes that `human(socrates)` is true while another does not:

```
?- rpc('http://n2.org', human(socrates)).
true.
?- rpc('http://n1.org', human(socrates)).
false.
?-
```

And the following query asks whether two agents *agree* that `human(X)`:

```
?- rpc('http://n1.org', human(X)),
   rpc('http://n2.org', human(X)).
X = plato ;
X = aristotle.
?-
```

The epistemic reading is not the only one available. More conservatively, the URI can be understood as specifying the *scope* of inference – the knowledge base with respect to which a query is to be evaluated. As argued in (Kifer et al., 2005), scoped inference is mandatory for realising Negation As Failure on the Web, because applying any form of the closed world assumption requires knowing the entire knowledge base first – something that is not possible for the Web since its boundaries cannot be clearly delineated. A third and more practical view is simply that `rpc/2-3` invokes a goal in a remote logical context whose predicates remain distinct from those of the caller. For pure relations on suitably aligned nodes, the three views – epistemic operator, scoped inference, remote context – can coincide operationally, although they remain conceptually distinct.

What about `rpc/3`? Code injection via `src_*` options does not jeopardise purity provided the final composition of the program executed is itself pure. The `limit` option is a *pragma*: it specifies the granularity of the conversation between client and node but has no effect on the meaning of the query.

9.4.3 The less pure Prolog Web

Not all of the Prolog Web is pure. ISOBASE nodes may use control constructs such as `cut`, and ISOTOPE nodes go further with side-effecting predicates, *I/O*, and dynamic database updates. The stateless HTTP API cannot preserve such updates between independent requests; stateful sessions and actors can.

More fundamentally, the concurrent Prolog Web – the world of the ACTOR profile – is inherently procedural. Actor-based programming involves process creation and destruction, message ordering, selective receive, and failure propagation – none of which have declarative readings. A realistic application will typically mix profiles freely, using pure queries for declarative reasoning and actors for coordination and state.

This spectrum from pure to impure is itself a paradigmatic feature. Rather than forcing a choice between declarative and procedural – a tension that has shadowed Prolog since its inception – the Trinity makes the boundary explicit and navigable. The programmer can reason about which parts of a system admit logical readings and which require procedural ones, and can move between the two with architectural awareness rather than accidental drift.

9.5 The distribution-first model

Within the concurrent partition, Web Prolog adopts a distribution-first view. The same actor primitives apply locally and remotely, with local placement as the simpler case. Remote placement additionally introduces serialisation, latency, authentication boundaries, disconnects, and partial failure. The sequential partition remains distinct:

it offers query evaluation rather than actor interaction, even when an actor-based protocol implements it underneath.

One useful starting point for formalising this stance is Svensson et al. (2010), who proposed a unified semantics for Erlang-like systems. Web Prolog draws two related design moves from that work. First, local and remote messages travel through a single conceptual mechanism, while a node controller handles side-effecting operations uniformly. Second, links are unidirectional rather than symmetric; their supervisor example makes the child’s lifetime depend on the supervisor’s, which is the orientation adopted by Web Prolog’s spawn hierarchy. In their abstract account, placement can be treated as a configuration detail rather than as a split between two languages, even though it remains operationally observable. The separation of node controllers from process semantics offers Web Prolog a relevant model, although a Web Prolog semantics must additionally account for Prolog engine state, backtracking, the boundary between the two partitions, and the open-Web containment policy layered on the one-way link relation.

A useful way to read the distribution-first stance is through the classic goal of *distribution transparency*: making the distribution of processes and resources invisible to applications. The Prolog Web does not aim for transparency by hiding the network behind a magic API; rather, it aims for a disciplined form in which the *same* conceptual operations – such as spawning, sending and monitoring – apply locally and remotely. Latency, serialisation, security boundaries, and failure modes change with distance, but the programming model does not bifurcate into a “local” and a “remote” language. The profile hierarchy identifies the language features and protocol surfaces available across the boundary; precise conformance rules must say which observable guarantees travel with them. Distribution-first therefore means uniform primitives with explicit operational differences, not an attempt to make the network disappear.

The combination of Prolog’s execution model with this actor semantics suggests a modular route to a formal account. Conceptually, each process has both an actor component – including its mailbox and lifecycle state – and a *Prolog engine state* consisting of a goal stack and a current substitution. Most rules respect that separation, but the components meet at the actor primitives. A send constructs a message from the current substitution; a receive removes a selected message and extends the substitution by unification; spawning, monitoring, and termination likewise connect goal execution with process lifecycle. The crucial discipline is that these crossings are committed: backtracking never unsends a message or returns a selected message to the mailbox. Backtracking therefore remains local to an actor’s engine even though the two state components interact. A full development and proof of composability lie beyond this chapter and belong to the research agenda of Chapter 11. The separation and committed interface make orthogonality a plausible design objective; they should not be mistaken for a completed formal result.

9.5.1 The browser as a Web Prolog runtime

The distribution-first model extends naturally to the client side. A browser equipped with a Web Prolog runtime hosts what Chapter 3 called a *browser node*: a node in architectural terms, but not a public endpoint. It accepts no incoming network connections and therefore participates as a client and as a host of local actors. It can nevertheless use the same interaction primitives available to server-side code. The programmer should be able to use synchronous calls when blocking is acceptable, non-blocking request-response calls when responsiveness matters, and actor-style interaction for cases that genuinely require long-lived conversational state – all without being exposed to transport-level details.

The longer-term ambition suggested by this design is Prolog on both sides of the Web. At the same time, this should not be confused with an attempt to hide the Web or to force all clients into a single language. Many developers will continue to prefer writing front-ends in JavaScript and communicating with Prolog back-ends through explicit web APIs. The Prolog Web should make such APIs visible, well-specified, and robust. A Prolog-everywhere experience may make them largely invisible in the common case, but engineering robustness requires that they remain accessible.

9.6 Control as a first-class dimension

Consider the famous equation coined by Robert Kowalski (Kowalski, 1979):

Algorithm = Logic + Control

The kind of control we have in mind here is different from search control. Conversational and reactive systems are hard not primarily because they require sophisticated algorithms, but because they require fine-grained *interaction control* – the kind that governs turn-taking, orchestration, and long-lived dialogue flow. In the Trinity, this motivates the combination of three programming models: logic programming for knowledge and inference; actor-based programming for concurrency, distribution, and interaction; and statecharts for explicit, inspectable control structure. The three are complementary: each addresses a dimension of system design that the others leave open, while – as we show below – remaining tightly coupled to them.

9.6.1 Governance of interaction

If the Trinity were only a story about distributed queries and actor concurrency, it would leave an important practical question unanswered: how does one *govern* complex interaction so that it remains inspectable, auditable, and robust? Protocol sequencing, timeouts, retries, cancellation, interruption, recovery, and exceptional

cases can all be encoded by actors, but their structure is then dispersed through the actor loop.

Statechart actors are the Trinity's response. The idea is not to replace actor programs with statecharts, but to offer a structured substrate for reactive behaviour in which states, transitions, events, timers, guards, and recovery paths are explicit and therefore reviewable. Statechart actors turn the actor model from a concurrency substrate into a *governable interaction platform*.

Control participates in the same mutual constraint as the rest of the architecture. Statechart guards, actions, and data-model code are Web Prolog goals, so there is no second scripting language or security boundary. The interpreter implements run-to-completion and the internal event queue inside an actor, while invoked-process lifetimes can be tied to the states that invoked them. The statechart layer therefore reuses actor messaging and lifecycle mechanisms while making their control structure explicit.

Chapter 6 committed to one realisation: SXML, an SCXML-inspired statechart notation with Web Prolog as its guard, action, and data-model language. The wider design space still admits alternatives, including SCXML itself with Web Prolog as a scripting layer. Either way, the relationship follows a familiar web pattern: a declarative control layer whose behaviour is scripted in a general-purpose language.

This combination has a broader consequence for the Agentic Web discussed in Chapter 8. Conversational and tool-using agents need more than components capable of producing answers: they need explicit protocol state, bounded waiting, recovery from partial failure, and coordination over long-lived interactions. Actors provide the durable interaction substrate, statecharts make its control inspectable, and logic supplies knowledge and constraints.

This also supports a pragmatic neuro-symbolic division of labour. Statistical components can interpret uncertain input or propose actions, while symbolic programs retain responsibility for protocol, policy, and lifecycle. The point is not that every agent must use all three mechanisms, but that the platform gives fallible components a governed setting in which to participate.

9.7 Executable logic on the Web

A common use of logic in Web architecture is representational: publishing structured meaning, expressing constraints, and drawing conclusions from shared data. This leaves an operational question: where does executable reasoning live, and how can computations hosted by independent sites be combined?

The Trinity treats logic on the Web as executable as well as representational. A node can publish facts, but it can also host relational services, rules, and protocols. Goals are shipped and enumerated in the sequential partition; long-lived agents exchange symbolic messages and apply constraints in the concurrent partition. Logic thereby becomes a callable, composable, and governable component of live Web sys-

tems. The sequential and concurrent partitions provide different operational forms, but both use the same terms, goals, and relations as their symbolic substrate.

This positions the Prolog Web beside, rather than against, the Semantic Web. The Semantic Web asks how independently published data can be understood and combined; the Prolog Web asks how independently hosted logic can be executed and composed. The former emphasises shared identifiers, data models, and entailment regimes; the latter adds relational services and long-lived agents. RDF or other graph data can therefore provide interoperable knowledge while the Prolog Web provides an execution stack over or alongside it. Chapter 7, especially Table 7.1, develops this relationship in detail.

9.8 Comparison to other approaches

The claim that the Trinity constitutes a candidate paradigm invites a natural sceptical response: has this not been attempted before? Four precedents are close enough to warrant explicit comparison.

9.8.1 Linda and tuple-space coordination

Linda (Gelernter, 1985) introduced coordination through a shared associative memory, with pattern matching over tuples as the retrieval mechanism. The Trinity shares the emphasis on pattern matching as coordination, but the Prolog Web is a network of active agents with explicit identities and lifecycles, not a passive shared store. Coordination is directional: a client submits a goal to a named node and receives answers through a well-defined protocol. This directedness makes reasoning about authority, lifetime, and failure tractable.

9.8.2 Concurrent Prolog and the committed-choice languages

Concurrent Prolog (Shapiro, 1987), Parlog (Clark and Gregory, 1986), and GHC (Ueda, 1986) showed that SLD-resolution and concurrency are not incompatible. Their commitment mechanisms deliberately change the usual Prolog relationship between search and backtracking, and backtracking is absent from the concurrent partition. The Trinity makes a different trade-off. The sequential partition retains ordinary nondeterministic Prolog execution; the concurrent partition adopts Erlang-style message passing, making the non-declarative character of concurrency explicit. The two are deliberately orthogonal: backtracking does not roll back mailboxes, and actor communication does not interfere with proof search.

9.8.3 Erlang/OTP and actor behaviours

Erlang is the direct precedent for the Trinity's process-and-message discipline and its treatment of failure (Armstrong, 2003b, 2007a). OTP further shows how generic behaviours and supervision can turn actor primitives into an application architecture. The Trinity adopts this lesson but gives each actor a Prolog engine and preserves ordinary nondeterministic querying in a separate sequential partition. It also makes HTTP, WebSocket, and URI addressing part of the public distribution surface rather than assuming an Erlang-only network. The specific departures collected in Table 2.1 are not miscellaneous inventions: they arise primarily from Prolog's success, failure, and relational semantics, and from the authority and lifetime pressures of an open Web.

9.8.4 Oz/Mozart and the multiparadigm kernel language

Oz (Smolka, 1995), implemented in the Mozart system (Van Roy et al., 2003), is the closest and most instructive precedent. Like the Trinity, it combines logic programming, concurrency, and distribution in a single coherent design. Distributed Oz (Van Roy et al., 1997) adds network-transparent mobility and fault tolerance, and the textbook (Van Roy and Haridi, 2004) provides an unusually clean formal account through a hierarchy of kernel languages. The comparison therefore cannot rest on missing features; it must rest on design stance.

The decisive difference is that Oz uses a fully compositional, higher-order kernel language that accommodates its paradigms uniformly, rather than preserving Prolog's characteristic combination of Horn-clause syntax, unification-driven clause selection, backtracking, and the program-as-database identity. Oz therefore belongs to a different language tradition, not to a compatibility profile of Prolog. The Trinity, by contrast, is defined as a *profile* of Prolog. It extends ISO Prolog with concurrency primitives and distribution infrastructure while remaining syntactically and semantically recognisable to anyone who already knows the language. This is a deliberately narrower ambition – but it is the right ambition for a proposal that aims to unify an existing fragmented community rather than replace it with a new one.

A second difference concerns the Web. Oz/Mozart supplies its own distribution subsystem, organised around sites and network-transparent language entities. The Trinity instead makes HTTP, WebSocket, and URI addressing part of its public interface. URIs identify nodes, while actors have pids qualified by node identity. The result is less general than Oz in the space of distribution models but more directly embedded in existing Web infrastructure.

9.9 Why the Trinity fits Prolog

The Trinity is not merely implemented in Prolog. Its components exploit the unusual degree of semantic unity already present in the language. Terms serve as data structures, messages, and query templates; goals are both executable requests and descriptions of the answers sought; clauses are program text but can also be transmitted and composed as symbolic data. Unification and backtracking then connect these forms without a separate schema language, serialisation model, or enumeration calculus.

This unity also matters inside actors. The same mechanism that selects a clause can match an incoming message, and a logical guard can extend the substitution produced by that match. Each actor is therefore not only a mailbox-driven process but also a locally scoped reasoning engine. Other languages can reproduce the Trinity's architecture, but they will typically require distinct mechanisms for data representation, message matching, rule evaluation, and remote invocation. The claim is not that the architecture is exclusive to Prolog, but that Prolog gives it an unusually compact and coherent expression.

9.10 The dual process

Chapter 1 described the project as two simultaneous movements: *webizing Prolog* and *prologizing the Web*. They acted as a forcing function. Webizing required URI-addressed nodes, open transport surfaces, and explicit security boundaries. Prologizing contributed unification, nondeterministic querying, symbolic code, actor-style concurrency, and execution in the browser. Pursued together, these requirements produced the mutual constraints described in this chapter rather than two independent collections of features.

On the webizing side, node URIs make placement and inference scope part of ordinary programs, while node-qualified pids address live actors without confusing them with public Web resources. HTTP supplies stateless and semi-stateful query surfaces; WebSocket supports longer-lived actor interaction. Profiles, explicit dependencies, and the security mechanisms of Chapter 4 turn differences between deployments into declared boundaries rather than hidden assumptions. These choices also keep the Prolog Web accessible to non-Prolog clients and aligned with the Web's culture of open interfaces.

On the prologizing side, a goal becomes a Web request whose answers are matched by unification and consumed through backtracking and paging. Symbolic programs can be shipped to an execution context rather than reduced to passive data. Erlang-style actors add persistent interaction without replacing the sequential Prolog core, and the browser runtime makes the same model available on the client side. The six commitments of this chapter are thus consequences of pursuing both directions together, not features selected independently.

The dual process leaves questions of community and execution beyond the paradigm itself. Chapter 10 considers federation as a response to fragmentation and as a way of opening the network to neighbouring communities. Chapter 11 then distinguishes the engineering paths that can follow from the demonstrated architecture.

The paradigm claim therefore does not depend on predicting an institutional destination. Its immediate test is whether the language profile, agent model, and network protocols continue to form a useful and coherent programming model in practice.

9.11 Summary

This chapter has argued for a candidate paradigm for web-native symbolic computation. Its sequential and concurrent partitions separate nondeterministic querying from actor interaction without preventing their composition. Interaction portability asks whether relocation preserves the permitted observable behaviours declared by a profile and a program's dependencies. Agents provide the main units of structure, while unification, backtracking, and scoped remote inference preserve a recognisably logical foundation.

The sequential partition can be read at two levels: as inference among interlinked logic programs and as communication among the processes that realise it. The agent abstraction spans several granularities – actors, toplevels, statechart processes, and nodes – without claiming that all are intentional agents in the AI sense. This provides the setting in which interaction can itself involve inference: messages unify with patterns, guards prove goals, and the resulting substitutions inform actions.

Within the concurrent partition, common primitives apply locally and remotely while latency, failure, and security boundaries remain visible. Statecharts add inspectable governance of long-lived interaction, and the executable-logic layer complements Semantic Web representation with hosted rules and relational services. These commitments are held together by mutual constraint: the language profile limits the execution surface, the agent model shapes the required protocols, and the Web boundary shapes portable representations and lifecycle rules.

Whether this constellation becomes a paradigm in the stronger sense – a named point in the design space, implemented independently, taught, and built upon – is for a community to determine. The immediate claim is more modest: the Trinity is not merely Prolog transported over the Web. It is an independent but interoperable execution stack, designed to run alongside existing Web and Semantic Web stacks while adding executable logic, actors, and governed interaction.

Part V

What's in it for you and your community?

Chapter 10

Communities, interoperability, and adoption

The preceding chapters developed the Prolog Trinity as a technical proposal. This chapter asks what that proposal might contribute to the communities that would have to implement, use, and sustain it. It begins with Prolog implementers, tool builders, teachers, and users, then widens the discussion to neighbouring communities in logic programming, the Semantic Web, AI, web development, and Erlang.

The Prolog community already encompasses a diverse collection of systems. Figure 10.1 offers an impressionistic reminder of that plurality. The challenge considered in this chapter is not how to remove these differences, but how to make cooperation across them possible.



Fig. 10.1 Logos of thirteen Prolog systems: B-Prolog, Ciao, ECLiPSe, GNU Prolog, JIPROLOG, Stryer Prolog, SICStus Prolog, SWI-Prolog, Tau Prolog, Trealla Prolog, tuProlog, XSB, and YAP. The collage illustrates plurality; it is not a ranking or a measure of current project activity. The logos remain those of their respective projects.

10.1 From fragmentation to federation

Most current systems implement the ISO Prolog core, but important differences remain in extensions, libraries, tooling, and choices left to the implementation. The result is not merely inconvenience when moving a program: it also complicates shared teaching material, libraries, tests, and further standardisation.

Three responses deserve comparison. Incremental proposals can enlarge the portable common ground. A unified Prolog could bring major facilities into one coherent implementation. Federation instead asks distinct systems to agree on the surfaces through which they interact at runtime. The first two seek greater internal agreement; the third seeks useful cooperation across differences that remain.

10.1.1 Incremental convergence through PIPs

One response is the *Prolog Improvements Forum*.¹ Its *Prolog Improvement Proposals* (PIPs) document language and library features supported, or intended to be supported, by more than one system. PIPs are not ISO standards, but they can record implementation experience, coordinate common APIs, and prepare material for later standardisation.

This is a realistic approach to increasing the portable part of Prolog: bottom-up, incremental, and grounded in implementation experience. Agreement on such matters as library interfaces can narrow the portability gap without requiring systems to converge wholesale, and proposals tested in several implementations can provide useful evidence for later standardisation.

Its limitation is one of scope rather than merit. Incremental convergence can reduce the cost of moving programs between systems, but it does not by itself provide a runtime through which those systems cooperate. The Trinity therefore treats PIPs and federation as complementary: one narrows avoidable dialect differences while the other specifies the boundaries across which differences may remain.

10.1.2 One Prolog to rule them all

Gupta et al. propose a more ambitious response: integrate tabling, constraints, coroutining, parallelism, and goal-directed ASP efficiently and seamlessly in one system (Gupta et al., 2023). This continues a long-standing hope that the strongest facilities associated with different Prologs might be brought into one coherent environment.

The integrative impulse is already visible in existing systems. SWI-Prolog, for example, combines broad libraries and interfaces with constraint programming,

¹ <https://prolog-lang.org/ImplementersForum/PIPs>

tabling, Just-In-Time indexing, and an implementation of s(CASP).² The unified-Prolog vision sets a stronger goal: not merely co-location of features, but principled and efficient composition among them.

That is an attractive research programme, but it answers a different question from the Trinity. A unified system assembles facilities inside one implementation; federation lets existing and future implementations cooperate while remaining distinct. Progress in either direction could benefit the other.

10.1.3 The middle way: interoperability as a federating force

Language standards usually define a common language or API so that programs can be moved or reimplemented. Federation makes a narrower agreement: participants standardise behaviour at an interaction boundary while retaining internal differences. The two approaches are compatible, but they solve different problems.

The Trinity defines such boundaries as profiles, protocols, and APIs. A node need not run a particular Prolog system; it must conform to the profile it advertises, including the predicates, transports, and behavioural contracts that profile specifies. Different nodes may realise the same surface with different optimisations, libraries, and architectures. What is shared is the *interface*, not the *engine*.

This is federation in a precise sense: independent parties agree on how to communicate without agreeing on how to think. The agreement is operational – terms have specified meanings, interactions follow stated contracts, and capabilities are advertised – while implementation choices remain local. This resembles the organisation of the Web itself: boundaries are specified closely enough to permit cooperation, while the systems behind them remain independently owned and free to evolve.

Figure 10.2 illustrates the concept in concrete terms. Rather than collapsing the distinctive strengths of SWI-Prolog, ECLiPSe, XSB, and YAP into a single uniform system, we envision an ecosystem where each system is wrapped in a node implementation that allows it to converse with others. The diversity of approaches is preserved, yet they can all contribute to a common fabric of cooperation. Participation remains voluntary; a node interface of this kind is required only by a system that chooses to join the shared Prolog Web.

The proof-of-concept implementations accompanying this book demonstrate communication among nodes owned by different parties and running in different locations. They do not yet demonstrate cross-implementation interoperability: the nodes in these experiments use the same underlying Prolog system. Heterogeneous implementations are therefore the decisive next test.

Portability and runtime interoperability should not be confused. Portability moves code between systems; interoperability lets systems cooperate during execution. They are complementary, and both require implementer cooperation, but runtime

² <https://github.com/SWI-Prolog/sCASP>

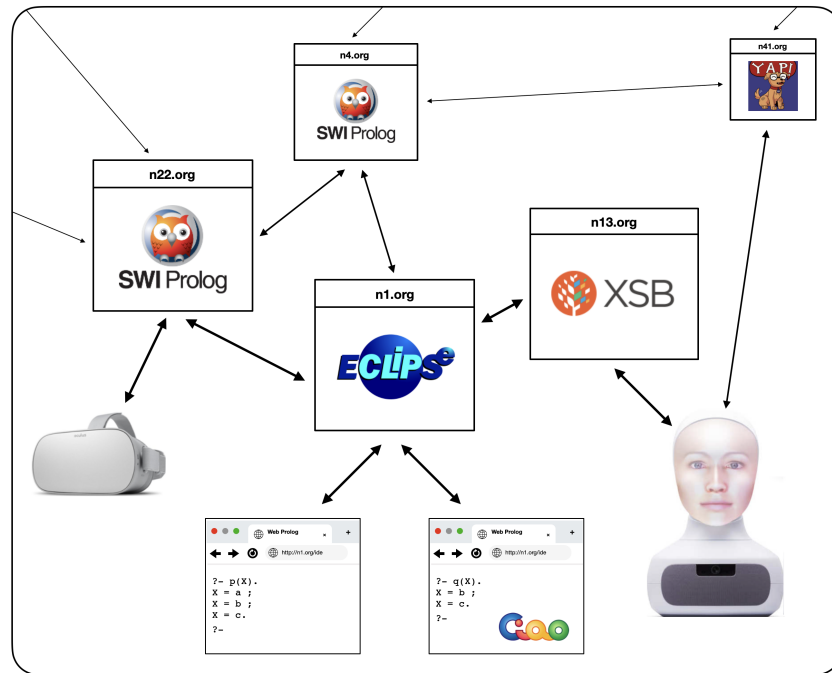


Fig. 10.2 The network introduced in Figure 1.3, now imagined with heterogeneous Prolog implementations behind its nodes. Clients and nodes share profile-defined interaction surfaces while implementations retain their own internal strengths. Later in this chapter the topology remains fixed while the picture opens to neighbouring ecosystems.

disagreements are especially costly because they become visible inside a live distributed computation. The advertised contract must therefore be precise.

Profiles must fix the latitude

Interface agreement must reach further than a common list of predicate names. ISO Prolog deliberately leaves some behaviour implementation-defined, and systems also differ in areas outside the standard. Two conforming systems may therefore disagree about matters that become observable at a network boundary: what happens when an undefined predicate is called, what a double-quoted literal denotes, which errors are raised, or how numeric limits are handled.

A Web Prolog profile must settle every such choice that affects its interaction contract. In this sense, a profile is partly a decision over the latitude left by the host language: it turns implementation-defined behaviour into profile-defined behaviour for the fragment it covers. The choice should follow from the contract, not merely from an implementation's default. For example, the ISO `unknown` flag permits

different treatments of an undefined predicate. The Trinity's distinction between an error for a missing relation, failure for the absence of a solution, and timeout for a computation that does not answer requires the profile-equivalent of `unknown = error`, rather than silent failure. Similarly, portable source loading requires the `double_quotes` setting – and therefore the meaning of double-quoted text – to be fixed rather than assumed.

The design requirement is firm even where the final setting still belongs to standardisation. The present demonstrator's profile inventory primarily specifies routes, predicates, source-loading facilities, and actor behaviour; a standards-quality profile must additionally enumerate these language-level choices and test them. Conformance is consequently not a declaration of ancestry or intent but a claim about observable behaviour, checked by a suite shared among implementations. This is how a narrow `RELATION` surface and a much richer `ACTOR` surface can both be precise without requiring the engines behind them to be alike.

Diversification through shared surfaces

Federation allows Prolog systems to retain the extensions and optimisations that distinguish them. A node may expose those facilities through owner-defined relations or services without adding them to the common profile; indeed, a specialised facility may be exactly why a client chooses that node.

Imagine a future where an application orchestrated through Web Prolog can dispatch a deductive query to a remote XSB engine, feed the results into an ECLiPSe solver, and manage the user interface via SWI-Prolog. The names are illustrative rather than commitments about a particular integration. The point is that profile conformance could allow specialised services to be composed without requiring their implementations to merge.

The profile hierarchy makes this approach inclusive. A small or single-threaded system can implement `RELATION` without taking on the actor runtime required by `ACTOR`. The work is still substantial: profile contracts, serialisation, security, implementations, and conformance tests must agree. Work in the Prolog Improvements Forum that catalogues divergences and clarifies semantics can inform those profiles; executable profiles can in turn reveal disagreements that portability work may otherwise miss.

If the Prolog Web succeeds, it will not be because every Prolog system has been replaced by a single implementation, but because many implementations have adopted compatible interaction surfaces. Success is measured by interoperation and reuse, not by replacement. The same principle extends to neighbouring communities: the Trinity does not ask the Erlang community to adopt Prolog, or the Semantic Web community to abandon RDF. It asks whether there is value in a shared execution substrate for logic-based agents on the Web.

10.2 Community infrastructure and adoption

FYPB also identifies fragmentation of the Prolog *community* and a comparatively small user base as concerns. Grouping around implementations and research traditions is natural and often productive, but it can make knowledge, tools, and teaching material harder to share. A technical architecture cannot solve that social problem. It can provide common objects around which collaboration occurs.

10.2.1 Shared objects for collaboration

The “All Things Prolog” website, sponsored by the Association for Logic Programming, already provides a valuable system-independent home for information, education, discussion, and improvement proposals.³ A Trinity portal should complement that work rather than duplicate it. Its distinctive role would be operational: discovering public nodes and services, running examples against them, displaying conformance information, and linking the resulting artefacts to documentation and discussion.

The Prolog Web also broadens what can be shared. In addition to source code, a developer could publish a bounded service or a live demonstration actor that others can use immediately. This does not replace repositories or package managers; it adds running computational resources to the things a community can exchange.

The same infrastructure can support work on the infrastructure itself. Profiles provide specifications to debate, conformance suites provide common tests, and nodes provide shared targets for clients, examples, and documentation. FYPB observes that portability problems extend beyond the language core to libraries and tools. A profile-governed node cannot make arbitrary host-system tools portable, but it can give cross-system tools a stable surface on which to operate.

10.2.2 Playgrounds and learning

The interactive and distributed character of Web Prolog suggests a useful role for browser-based playgrounds. Figure 10.3 shows three established environments: SWISH, based on SWI-Prolog; the Ciao Prolog playground; and the Tau Prolog sandbox. They let visitors program without first installing a local system and are useful for teaching, experiments, and shareable examples.

These playgrounds serve different systems and audiences, and their differences are often deliberate. They also reveal a portability boundary: a program written for one environment may depend on its dialect, libraries, or interface conventions and therefore require changes in another. This is not an argument that the existing work

³ <https://prolog-lang.org/>

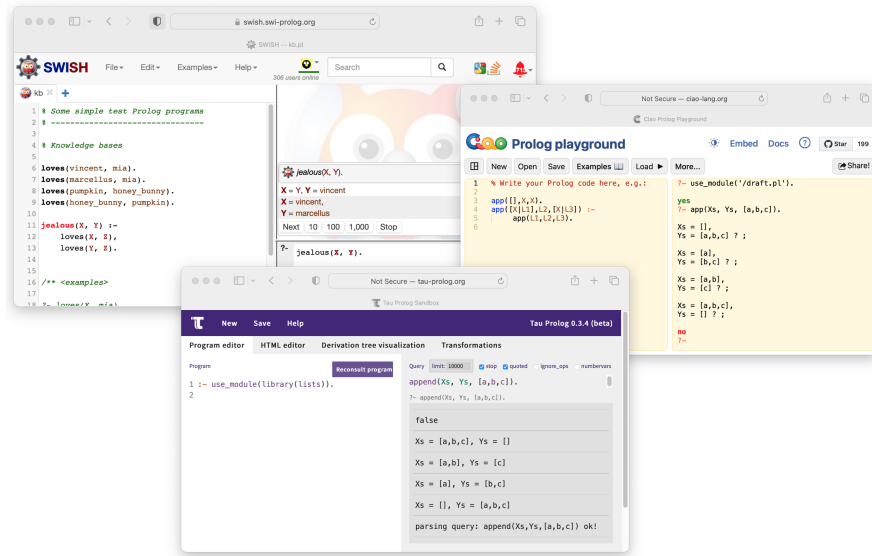


Fig. 10.3 A snapshot of three browser-based environments: SWISH for SWI-Prolog, the Ciao Prolog playground, and the Tau Prolog sandbox. Their different interfaces and dialects reflect their different purposes; a shared Web Prolog profile could add a portable teaching surface between them.

should have been centralised. It is an opportunity to add a shared layer. A common Web Prolog profile could make a useful core of exercises and queries portable while allowing each playground to retain the facilities that distinguish it.

A Web Prolog playground served by an ISOTOPE node could provide one such shared environment for general instruction. When backed by an ACTOR node, the same interface could extend from ordinary queries to actor programming and distributed interaction. It should be viewed as a complement to SWISH, the Ciao playground, and the Tau Prolog sandbox: a place to teach the common profile and its network model, not a universal replacement for system-specific environments.

Web delivery removes installation friction, but it does not remove Prolog’s conceptual learning curve. Existing Prolog programmers can begin with the sequential profiles and encounter actor concepts only when they need them. Readers familiar with Erlang will recognise much of the concurrent vocabulary, while those new to actor programming can draw on the substantial Erlang literature. The profile ladder thus provides a pedagogical sequence as well as an implementation sequence.

Interactive, browser-based examples can present Prolog as a current language for queries, services, and distributed symbolic applications. Whether this attracts new users is an empirical question, but it lowers the cost of trying the language and of sharing a runnable example.

Staying close to traditional Prolog makes it possible to reuse the many good textbooks and tutorials already available. Erlang literature can likewise explain several

principles behind Web Prolog's actor layer. Syntax, failure, logical variables, and the toplevel protocol remain specifically Prolog concerns, however, so dedicated Web Prolog tutorials are still needed. The demonstrator manual and browser tutorial are first steps rather than substitutes for such material.

10.2.3 Conditions for wider adoption

Prolog has a much smaller educational, industrial, and public presence than mainstream languages. FYPB treats this as a threat because a small user base can reinforce a small ecosystem: fewer users mean fewer libraries, contributors, examples, and deployment reports, which in turn give new users fewer reasons to begin. Technical quality alone does not break that cycle.

The Trinity can affect only some conditions for adoption. Browser access lowers the cost of trying Prolog, and runnable links lower the cost of sharing an example. A common specification and conformance tests allow clients, documentation, and services to accumulate around more than one implementation. A visible application domain can provide motivation to learn the language. None of these guarantees growth, but each removes a practical obstacle.

Usefulness must supply the pull. Developers will tolerate an unfamiliar language when it solves a problem they care about unusually well. The credible case for Web Prolog therefore lies in applications where relations, constraints, symbolic knowledge, or actor-based coordination are central, not in a general claim that every web program should be written in Prolog. Documentation, examples, responsive maintenance, and a welcoming community matter at least as much as the language design once a visitor arrives.

Familiarity remains a weakness. Prolog's syntax and execution model are foreign to many programmers trained on imperative languages. Erlang programmers will recognise much of the actor vocabulary, but unification, logical variables, failure, and backtracking still require explanation. The RELATION, ISOTOPE, and ACTOR progression can make that learning incremental: ordinary relations first, remote execution next, and explicit concurrency only when it becomes useful.

There is also a danger in gaining visibility through one domain. Prolog must not be reduced to a web language or detached from systems serving other applications. The structural answer is to define Web Prolog as a profile of general-purpose Prolog, not a replacement for it. Section 10.4 returns to that risk. The adoption claim here is deliberately modest: the Web offers a low-friction place to encounter and apply Prolog, while shared contracts let that entry point lead back to a plural implementation community.

10.3 Opening the network to neighbouring communities

The case for federation does not stop at the boundary between Prolog implementations. Rule engines, triple stores, ASP solvers, Erlang services, model-serving stacks, and web applications need not adopt Prolog or surrender their own semantics in order to participate. They may instead expose selected facilities through a specified surface, or remain behind an owner-side adapter. The adapter work and semantic mapping required at such a boundary are real engineering costs. As Section 10.1.3 emphasised, profiles fix observable contracts; they do not prescribe the internals behind them.

Figure 10.4 is the third view of the network first shown in Chapter 1 and revisited earlier in this chapter. The topology and clients have not changed. What has changed is what some nodes expose and what external components may stand behind them. At `n4.org`, the general SWI-Prolog logo has given way to *cplint*, indicating probabilistic logic programming offered through that implementation. Database cylinders suggest RDF stores behind logical interfaces, while the Python and model-provider logos suggest an owner-side adapter to a neural service.

The following examples ask what such boundaries might offer to logic programming, the Semantic Web, AI, web development, and Erlang. They do not claim that all the illustrated integrations already exist. Where the demonstrator supplies evidence, we say so; elsewhere the text describes an integration path or a research possibility.

10.3.1 For the broader logic programming community?

The logic programming landscape

Prolog is the most widely known logic programming language, but it is far from the only one. Over the past five decades, researchers have extended, restricted, or reinterpreted the Horn-clause core in many directions. The following survey is necessarily selective, but it covers formalisms and systems relevant to the possibility of a broader Logic Programming Web.

Prolog's native mode of reasoning is deduction – deriving consequences from given rules and facts – but logic programming as a field has long explored other modes of inference as well. Abductive logic programming (ALP) reverses the direction: given an observation, it searches for hypotheses that, together with background knowledge, would explain it (Kakas et al., 1992). Inductive logic programming (ILP) goes further still, learning new rules from examples and background theory (Muggleton and De Raedt, 1994; Cropper et al., 2022). Both traditions extend the reach of logic-based reasoning well beyond what a standard Prolog system provides out of the box.

Nor is deductive logic programming itself a monolith. Datalog restricts Horn clause logic to function-free programs, sacrificing Turing completeness in exchange for guaranteed termination and well-understood complexity bounds (Ceri et al., 1989;

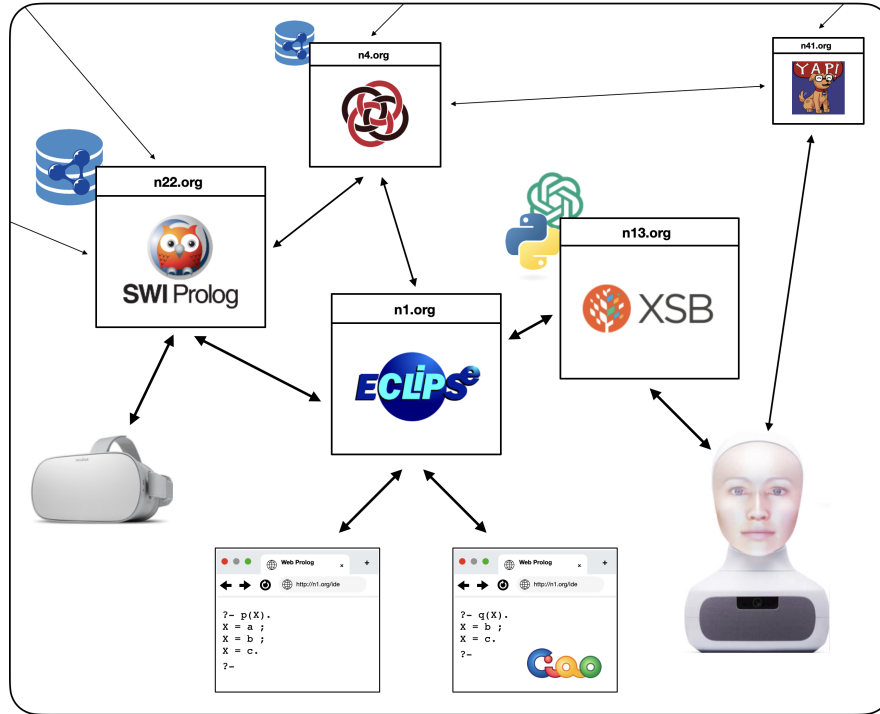


Fig. 10.4 The network of the opening chapter, now opened to neighbouring ecosystems. Node `n4.org` exposes probabilistic reasoning through *cplint*; the database, Python, and model-provider logos indicate possible backing services. The network and its clients remain unchanged.

Green et al., 2013). Well-founded semantics (WFS) addresses a different limitation, providing a three-valued account of negation in which every normal logic program has a unique minimal model assigning each ground atom the value *true*, *false*, or *undefined* (Van Gelder et al., 1991); systems such as XSB and SWI-Prolog implement WFS via tabling, giving them stronger semantic guarantees than standard Negation As Failure. Answer Set Programming (ASP) departs more radically, searching for the stable models of a logic program rather than computing answers by resolution; a recent variant, s(CASP) (Arias et al., 2018), combines ASP with constraint logic programming in a top-down, query-driven execution model with support for explanation generation. Probabilistic logic programming (PLP) equips logic programs with probabilistic reasoning: ProbLog (De Raedt et al., 2007) is a Python-based toolbox that defines distributions over possible worlds and supports inference, sampling, and parameter learning, while *cplint* (Riguzzi, 2018) is a complementary SWI-Prolog library for LPADs under the distribution semantics, accessible both locally and through “*cplint* on SWISH.” Constraint logic programming (CLP) extends Prolog’s unification with constraint solvers over domains such as finite integers, reals, and sets, and most major Prolog systems now ship with CLP libraries as standard. Finally,

Logic Production Systems (LPS) (Kowalski and Sadri, 2015; Kowalski et al., 2023b) combines logic programming with reactive, event-driven computation.

The common thread is that logic programming is not a single language but a family of formalisms. That does not make every member of the family a Prolog implementation or give it the interaction semantics required by a Web Prolog profile. It does create opportunities for narrower, explicitly mapped interfaces through which different kinds of logical service can participate.

Different semantics at a shared boundary

ASP systems, Datalog variants, constraint systems, probabilistic languages, and multi-paradigm systems are not necessarily Prolog implementations in the sense required by ISOBASE. Such a system should not advertise a profile whose language contract it does not implement. It can instead expose a deliberately small logical API through RELATION, or place a profile-conforming controller in front of its native query interface. The wrapper must translate both data and meaning: what constitutes one answer, how alternatives are ordered, how errors are represented, and which calls are authorised.

Probabilistic and specialised services

The cplint system, described above, provides a useful intermediate case. It is a SWI-Prolog library and is available through cplint on SWISH. Because SWISH builds on Pengines, the following existing Pengine interaction can call it directly:

```
?- pengine_rpc('https://cplint.ml.unife.it', prob(blue,Prob), [
    src_text("
        :- use_module(library(pita)).
        :- pita.
        :- begin_lpad.
           blue : 2/3; red : 1/3 :- urn1.
           blue : 25/55; red : 30/55 :- urn2.
           urn1 : 0.5 ; urn2 : 0.5.
        :- end_lpad.
    ")
]).
Prob = 0.5606060606060606.
?-
```

This demonstrates remote access through the protocol lineage from which Web Prolog grew; it does not by itself establish conformance to a current Web Prolog profile. A node owner could expose an equivalent, bounded relation such as `urn_probability/2` through RELATION without exposing cplint's entire programming environment. LPS, also available through SWISH, presents a similar

opportunity, though its visual results require an explicit agreement about how an execution trace is represented.

ProbLog, implemented primarily as a Python toolbox, is a different case. Its declarative character does not make it an ISOBASE implementation: ISOBASE fixes a Prolog language and toplevel contract, not merely the absence of side effects. A more honest first integration would therefore publish selected ProbLog computations as `RELATION` predicates. The same pattern applies to Picat's constraint solving, planning, and optimisation facilities. A richer profile could be justified later if multiple implementations converge on a larger common contract.

One stable model as one solution

Answer Set Programming makes the semantic mapping especially visible. For the program

```
cross :- not wait.
wait  :- not cross.
```

a straightforward adapter convention is to expose each stable model (Gelfond and Lifschitz, 1988) as one solution of an owner-defined `RELATION` predicate:

```
?- model(M).
M = [cross] ;
M = [wait].
?- 
```

The backend retains ASP semantics; the wrapper defines only how its results cross the boundary. It must choose a canonical representation and an ordering stable enough for the pagination contract. Brave and cautious consequences could be exposed as separate relations. This is an illustrative adapter convention, not a claim that ASP has been absorbed into Prolog or that `model/1` belongs to the `RELATION` profile.

The larger opportunity is federation without forced unification. A probabilistic service, an ASP solver, and a constraint solver could be composed through small declared surfaces while remaining different systems. Achieving that requires more than a thin transport shim: profiles, schemas, error vocabularies, authorisation, resource bounds, and conformance tests all matter. The Trinity offers a place to specify those agreements; adoption must show whether the cost is justified.

10.3.2 For the Semantic Web community?

Chapter 7 compared the Prolog Web with the Semantic Web in detail. The modest claim here is complementarity. RDF, OWL, and SPARQL provide standard ways to identify, describe, and query data. A Web Prolog service can sit above such resources when an application also needs rules, stateful behaviour, or coordination with other

services. It may front a SPARQL endpoint, translate selected RDF structures into terms, apply domain rules, and publish the result through a narrow logical API.

That boundary should not be mistaken for a universal representation. RDF graphs and Prolog terms have different semantics; an adapter must state how identifiers, literals, blank nodes, entailment regimes, and failures are mapped. Nor is every triple store thereby a Prolog node. The node is the component that implements the advertised Web Prolog contract; the store may be one of its private backing resources.

The practical offer is thus a programmable agent and orchestration layer that can reuse Semantic Web infrastructure without replacing it. Standardisation is discussed separately in Chapter 11; the present chapter need only establish why the communities have something concrete to discuss.

10.3.3 For the AI community?

Chapter 8 developed the case for symbolic, neural, and hybrid agents, including concrete interaction patterns between Prolog and large language models. It also described an experimental Model Context Protocol bridge. The point need not be repeated here. From the outside, the Trinity offers a control layer in which a model call is one effectful service among others: it can be authorised, timed out, checked against symbolic constraints, combined with stored knowledge, and incorporated into a longer-lived actor.

This does not turn every model endpoint into an actor. A node owner must still write an adapter, choose a stable request and result vocabulary, protect credentials, impose budgets, and decide what model output is permitted to affect. Conversely, a Prolog relation exposed to a model-oriented orchestrator may use MCP when a bounded, schema-described tool call is appropriate, while PTCP remains useful when the interaction requires a Prolog session, nondeterministic answers, or actor messaging. One knowledge base may therefore offer both interfaces without pretending that they express the same protocol.

Agents and the Web

Hendler’s early discussion of agents and the Semantic Web (Hendler, 2001), followed by his later question “Where Are All the Intelligent Agents?” (Hendler, 2007), identified an enduring problem: capable components do not by themselves supply an open architecture for long-lived, accountable interaction. Today’s model-based agents make that problem more visible, not obsolete. Some maintain state and use tools; others are short-lived wrappers around a model call. The useful contrast is therefore not “stateless neural agent” versus “stateful symbolic agent,” but whether state, policy, communication, and failure are explicit parts of the architecture.

Web Prolog actors are stationary rather than mobile. They can nevertheless reach remote knowledge by fetching a permitted representation, querying a resident service,

or spawning a child actor on the node that owns the relevant resource. Mobility could be built above that substrate, but is not required merely to delegate work.

An active W3C Community Group on Autonomous Agents on the Web studies Web-aligned, transparent, and accountable multi-agent systems.⁴ Its work is a relevant neighbour, not evidence that the Trinity already implements a Hypermedia MAS architecture. A useful research programme would compare the group's hypermedia interaction model with ACTOR messaging, identify the missing agent abstractions, and test interoperation rather than assume equivalence.

10.3.4 For web programmers?

For web programmers, the proposal is not a replacement for JavaScript or a general-purpose web stack. It is an additional tool for applications dominated by rules, constraints, symbolic data integration, or long-lived reactive behaviour. In those domains, relations can make decision logic explicit, while actors can maintain conversations and respond to events without reducing every interaction to an isolated request.

There is a learning cost. Unification, logical variables, and backtracking do not behave like assignment, mutable variables, and loops. The case for Web Prolog must therefore be demonstrated in working tools and examples rather than asserted in general terms. The browser ACTOR node described in Section 3.5.5, with implementation details in Section E.4.1, is one such piece of evidence.

Constructing user interfaces with statecharts

User interfaces must govern asynchronous events, legal operations, and transitions between modes. Statecharts make this control structure explicit. Chapter 6 combined that model with Web Prolog: guards are goals, actions are predicates, and the data model can be queried relationally. This is particularly attractive when an interface must check policies, maintain a structured domain model, or coordinate remote services. It is not a claim that every interface should be a statechart.

The relationship with JavaScript statechart tools is complementary. XState, for example, uses JavaScript functions for guards and actions; a Trinity statechart actor uses Web Prolog for executable content. Both make control state visible. The difference is the scripting and data model available inside the machine, and that difference matters most when logic and constraints are part of the application.⁵

⁴ <https://www.w3.org/groups/cg/webagents/>

⁵ <https://stately.ai/docs/xstate>

Conversational and embodied agents

The demonstrator’s portal provides a concrete starting point for a browser environment in which statechart-driven and receive-loop actors control avatars. A fuller IDE could combine SXML and Web Prolog editors with execution, logging, and state visualisation. This continues earlier work on the *Dialog Web Lab*, a browser environment for authoring and executing SCXML dialogues with a simulated embodied agent (Junger et al., 2012).

Such an IDE would let a developer inspect the state machine while the underlying artefact runs as a genuine actor on an ACTOR node. The same message-level control could also be connected, through suitable adapters, to avatars in an immersive environment or to a social robot.

Controlling a social robot.

Furhat, a tabletop social robot head developed by Furhat Robotics, provides a physical example. Its realtime API uses WebSockets and permits an external client to send commands and receive events. An owner-side adapter could translate between that JSON protocol and the terms used by a Web Prolog actor or statechart actor.⁶ The control logic for an on-screen avatar may then be reusable, but the embodiment mapping is not free: gaze, speech, timing, errors, and physical functions need an explicit adapter. That qualification identifies the precise boundary at which the reuse claim can be tested.

10.3.5 For the Erlang community?

Erlang’s influence is present throughout Web Prolog’s actor layer: isolated processes, asynchronous messages, selective receive, monitors, and generic behaviours all acknowledge that lineage (Lager, 2019). Web Prolog is not, however, Erlang with Prolog syntax. It retains logical variables, relational guards, failure, and backtracking, while changing several concurrency choices to suit an open Web in which node owners and remote clients are distinct. Its links express a default one-way parent–child lifetime dependence rather than Erlang’s bidirectional relation; a receive commits to a selected message but then executes an ordinary Prolog goal; monitoring distinguishes logical failure from successful completion and exceptions; and profiles, authorisation, sandboxing, and containment mediate an open trust boundary. These semantics were developed where the relevant operations were introduced, rather than repeated here as a comparison table.

The differences still leave room for hybrid systems. An Erlang service can call a bounded Web Prolog relation over HTTP, and a Web Prolog owner can place an

⁶ Furhat Realtime API documentation: <https://docs.furhat.io/realtime-api/intro>. The author is not affiliated with Furhat Robotics, but knows members of the team personally.

adapter in front of an Erlang service. A genuinely profile-conforming ACTOR node on the BEAM would be a larger project: it would have to implement PTCP, the term and error contracts, authorisation, and the specified Web Prolog semantics rather than merely exchange messages with an existing node.

There is also a research question here. A runtime combining BEAM-like process scheduling and fault isolation with unification, backtracking, and meta-level execution would occupy a different point in the language-design space from both Erlang and today's demonstrator.

The outward-looking examples share one principle: the offer is a boundary, not annexation. A system may expose one `RELATION` predicate, provide a richer profile-conforming controller, or remain outside the network and be called through an owner-side adapter. In every case, the important questions are observable: what is named, what one answer means, how alternatives and failures are represented, who may call the service, and which resources bound the call. Whether communities will invest in those boundaries is the adoption question to which we now turn.

10.4 Risk analysis

The proposal carries two related risks. The first is rhetorical: calling Prolog a web language may narrow the perception of a general-purpose language and repeat the damage caused when Prolog was identified too closely with earlier AI hype. Paulo Moura put the objection sharply: "The last thing we need is for Prolog to be downplayed as a Web thing."⁷ That criticism is justified. An earlier description of Web Prolog as a "special-purpose language" was too narrow.

The second risk is technical. Jan Wielemaker welcomed the possibility that Web Prolog might succeed but warned that it could become detached from Prolog systems aimed at other applications.⁸ A web-facing dialect could develop its own syntax, libraries, and assumptions until the shared name concealed two communities rather than connecting them.

The profile model is intended to answer both objections. Web Prolog is not a replacement for general-purpose Prolog; it is a deliberately bounded profile that fixes the observable choices required at an open network boundary. "Web-facing" need not mean "web-only." Implementations may expose the profile alongside their existing languages and facilities, and owner-defined services can draw on those facilities without adding them to the untrusted profile.

This answer is structural, but not automatic. Detachment would still occur if the profiles were specified without implementer participation, if teaching material presented them as a separate language, or if the web interfaces could not be maintained alongside host systems. Shared syntax where possible, explicit mappings where it

⁷ Paulo Moura, SWI-Prolog Discourse, "Encouraging industry about Prolog".

⁸ Jan Wielemaker, in the same discussion.

is not, conformance suites run by several implementations, and continued use of ordinary Prolog libraries are therefore more important than branding.

The Web remains a plausible application surface because many of its problems have a logical shape: querying and transforming structured data, evaluating policies, selecting under constraints, reasoning over graphs, and coordinating stateful services. That is a domain argument, not a silver-bullet claim. Adoption must be earned by precise interfaces and useful applications.

Figure 10.5 presents one possible long-term way to keep the web-facing and general-purpose layers connected. It extends the profile diagram from Chapter 3 with a hypothetical outer ring provisionally called *ISO Prolog 2.0*. The name is a design hypothesis in this book, not an existing standard or a claim about what a future ISO project must contain.

The outer ring restores precisely the kind of host access that Chapter 2 excludes from an open, untrusted Web Prolog execution environment: ambient file I/O and unrestricted access to sockets, streams, threads, and the host system. Those exclusions are therefore not presented as amputations from Prolog, but as stratification by trust and deployment context. A future general-purpose standard might also consider tabling, constraint logic programming, and other mature developments, but this book does not attempt to settle that inventory.

The nesting expresses the strongest version of the proposal: a future system conforming to the outer profile would also expose the Web Prolog profiles. That is a possible direction, not a present conformance rule. A separately standardised Web Prolog layer implemented by adapters is also possible. The important claim is narrower: web-facing profiles and future general-purpose Prolog work should be designed to compose rather than drift apart.

10.5 Summary

The case made in this chapter is deliberately conditional. For portability, the Trinity proposes a profile-defined fragment and conformance tests rather than the portability of arbitrary host-system programs. For systems fragmentation, it proposes runtime interoperability without asking implementations to converge internally. For tooling, it offers common targets for clients, documentation, discovery, tutorials, and playgrounds. For the post-desktop Web, it supplies browser-accessible nodes, services, and actors rather than treating the browser merely as a remote terminal for a desktop system.

Beyond Prolog, the proposal is equally limited and concrete. A probabilistic program, an ASP solver, an RDF store, a model service, an Erlang process, or a robot does not become Web Prolog merely because a node can reach it. What can be shared is a carefully described boundary: the meaning of a request and an answer, the representation of alternatives and failures, the authority to call, and the resources consumed. Some such boundaries have precursors or demonstrations; others remain research proposals.

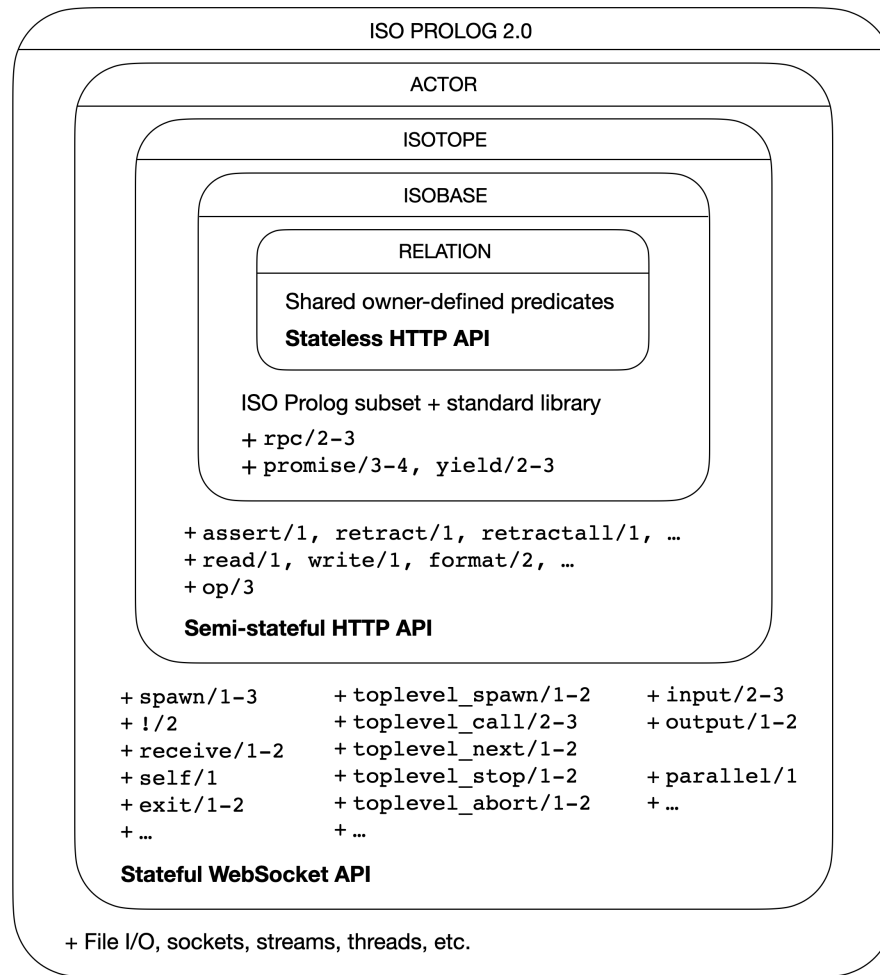


Fig. 10.5 The profile hierarchy completed by a hypothetical outer ring for ISO Prolog 2.0. Read from the inside out, the figure moves from open-web interaction surfaces towards a trusted, general-purpose environment that restores facilities such as file I/O, sockets, streams, and threads. The predicate lists are representative rather than exhaustive; the manuals define the current profile inventories.

Protocols do not manufacture communities. Multiple independent implementations, careful specifications, conformance suites, useful applications, and sustained maintenance would all be required before the benefits described here became real. The demonstrator establishes that a substantial part of the design can be built; it does not establish that other Prolog systems or neighbouring communities will adopt it, or that new users will follow.

The proposal is therefore neither that every Prolog become Web Prolog nor that the Web become Prolog's only field of application. It is that Prolog should have a well-defined way to participate in the Web while remaining recognisably Prolog, and that independently developed systems should have a practical surface on which their diversity becomes composable rather than isolating. The same surface can create points of contact with neighbouring ecosystems without absorbing them. Turning that invitation into shared infrastructure is the subject of the final chapter.

Part VI
Final words

Chapter 11

From vision to execution

Vision without execution is just hallucination.

Attributed to Thomas Alva Edison

At the beginning of this book, the Prolog Trinity ecosystem was introduced by means of a mind map. Its purpose was primarily orienting: to sketch the conceptual territory, name its principal components, and hint at the relationships between them. That diagram accompanied the reader through the early stages of the argument, serving as a visual shorthand for a still-emerging design.

In Figure 11.1 we return to the same mind map – structurally unchanged, but now adorned with three banners of the kind commonly used by the W3C.

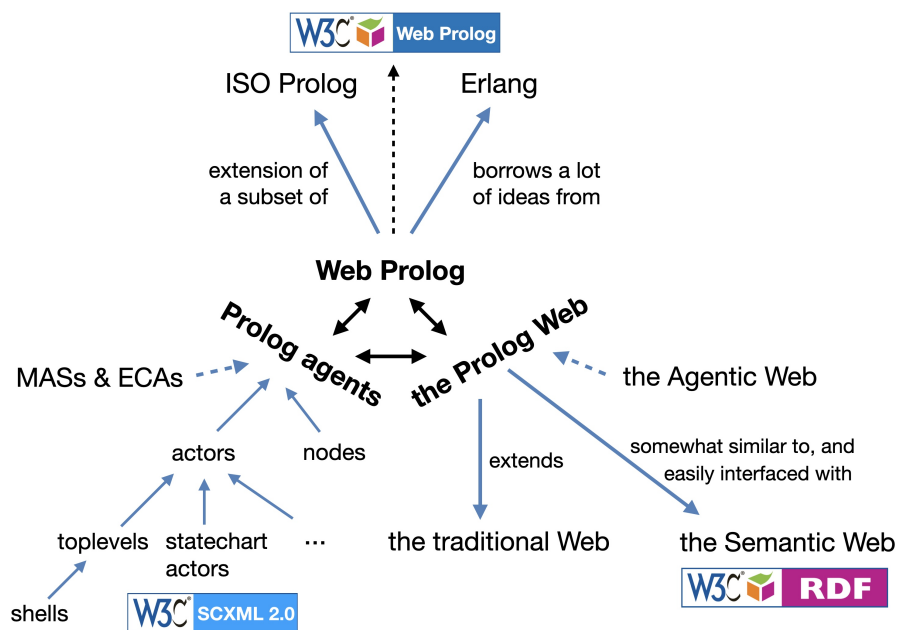


Fig. 11.1 The Prolog Trinity mind map revisited, now framed as shared infrastructure and adorned with three W3C-style technology banners. Of the three, only RDF is an actual, existing W3C standard; the other two are provisional logos of intent.

Such banners are not mere decoration. They are compact visual signs of collective agreement: indications that a technology has passed from an individual proposal to shared infrastructure, and that its definition, maintenance, and evolution are now matters of public process rather than private intent. A banner can compress years of discussion, specification work, experimentation, revision, and implementation experience into a single mark.

Of the three standards implicitly invoked by the Trinity vision, only RDF has completed that journey. The Web Prolog core and its SXML statechart notation have not. This asymmetry is neither accidental nor embarrassing. It clarifies the status of the proposal. RDF has been specified, standardised, implemented, deployed, criticised, and revised. Web Prolog and SXML remain forward-looking designs – argued for and partly implemented, but not socially ratified.

Nor is there only one possible destination. The demonstrator may develop into a serious platform centred on SWI-Prolog, with production quality valued above cross-system portability. The profile and protocol work may instead, or in parallel, become the basis of a federated programme involving several Prolog systems and perhaps standards bodies. The statechart work has yet another lineage through SCXML. These paths share much engineering, but they are not the same project and need not advance at the same pace.

Execution is therefore a set of choices rather than a predetermined roadmap. This chapter gathers the open research and engineering work, separates two plausible development routes, and identifies a layered set of specification targets if federation is pursued. It then considers the implementation, testing, and shared ownership needed to turn private design into public infrastructure. The two provisional banners in Figure 11.1 should be read in that conditional spirit: not as claims of authority, but as reminders of work that must occur outside this book.

11.1 A research and engineering agenda

The demonstrator has converted several architectural claims into running experiments, including public services, browser-hosted actors, statechart actors, and cross-node PTCP interactions. It has also made the remaining gaps more precise. The following workstreams deliberately mix product engineering, normative specification, and open research; each calls for a different kind of evidence.

Production hardening and browser operation.

The demonstrator is not yet a production platform. A serious deployment needs repeatable installation and upgrades, configuration management, persistent state and recovery policies, observability, resource accounting, overload behaviour, performance measurement, compatibility guarantees, and operational documentation. SWI-WASM additionally needs testing across browser engines and a precise account of the isolation, resource, and lifecycle guarantees offered by its worker-per-actor design. These requirements matter even if no second Prolog implementation ever joins the network.

Layered operational semantics.

Chapter 9 separated a Prolog engine's goal-and-substitution state from an actor's mailbox-and-lifecycle state and identified the crossings between them as committed effects: backtracking never unsends a message or returns a selected message to a mailbox. Extending the unified actor semantics of Svensson et al. (2010) with Prolog engine states, followed by a proof of suitable composability properties, is a research problem rather than a completed result.

Profiles and language latitude.

The profile inventories must be completed by the language-level decisions identified in Section 10.1.3: undefined-predicate behaviour, double-quoted terms, numeric limits, error terms, and every other host choice visible at the network boundary. Each decision needs a normative statement and, if interoperability is claimed, a conformance test.

Security and resource governance.

Chapter 4 established the structural principles of inexpressibility, invisibility, and containment. A deployable public-node profile still needs baseline requirements for authentication, authorisation, transport security, quotas, logging, audit, credential handling, denial-of-service resistance, and adversarial testing. Where suitable web and Internet standards already provide identity, delegated authority, discovery, secure transport, or audit, Web Prolog should bind to them rather than invent parallel mechanisms.

Terms, answers, and continuations at the web boundary.

The transformation between Prolog terms and JSON needs a stable or explicitly negotiated contract covering atoms and strings, variables, compounds, dictionaries, character and numeric rules, cyclic or otherwise unrepresentable terms, and errors. Answer representations should admit structured metadata without changing the core success, failure, and error protocol. The version-1 baseline consists of JSON envelopes and Prolog term text. A future binary term representation must be introduced through an explicitly negotiated media type or protocol capability, together with a complete mapping of Prolog values; it must not be inferred merely from a binary transport frame. Transport compression is orthogonal to that mapping. Offset/limit paging additionally needs a consistency contract: mutation may invalidate a continuation, a point-in-time snapshot may be offered, or an opaque resume token may carry the required state. Generation identifiers and resume tokens are plausible mechanisms rather than settled API. Portable dictionary and URI facilities would support this boundary but belong to the language and library track rather than being silently assumed by the protocol.

Further work on behaviours.

The server, supervisor, toplevel, and statechart examples demonstrate the value of reusable interaction abstractions, but the status of a behaviour remains underdeveloped. More work is needed on callback contracts, syntax, semantics, versioning, introspection, failure behaviour, and the relationship between a behaviour and the protocol it implements. For SXML specifically, Sections 6.7 and 6.10.5 provide a worked starting point. Timed transitions and explicit event

deferral now have implementations and stated semantics; the remaining task is to turn those and the rest of the processing model into a portable, testable contract. Independent implementations and conformance evidence.

The SWI-Prolog demonstrator is one implementation lineage. If the goal is a federated Prolog Web, at least one independently engineered implementation of each proposed interoperability layer is needed to reveal hidden assumptions. A shared test suite and implementation report should record feature-by-feature agreement rather than reduce conformance to a single badge. This item is a condition of a federation claim, not a prerequisite for improving the SWI-Prolog platform itself.

This is an agenda, not a backlog for one implementation. Its items have different owners, evidential standards, and time horizons. Treating them as a single undifferentiated project would make the whole programme harder to judge and easier to overstate.

11.2 Two routes from prototype to platform

The distinction between a production platform and a federation is important enough to make explicit. Both routes begin with the same demonstrator and can share protocols, tests, documentation, and much implementation work. They diverge in what counts as success.

11.2.1 A production-oriented SWI-Prolog platform

On the first route, the immediate goal is a dependable SWI-Prolog-specific platform. SWI-Prolog's threads, engines, module system, sandbox, HTTP stack, WebSocket support, browser port, libraries, and development tools are assets to be used fully rather than hidden merely because another Prolog system may not offer the same facilities. Success is measured by security, predictable semantics, performance, operability, useful services, and the experience of people building real applications.

This route does not make profiles or protocols irrelevant. A stable boundary between clients and nodes remains valuable even when every node uses the same engine. Profiles separate trusted from untrusted execution, make browser and server behaviour intelligible, support versioning, and give tests a public contract. What changes is the freedom to state explicitly that a facility is SWI-Prolog-specific and to optimise the platform around that fact.

A successful platform could later supply the implementation experience from which a portable core is extracted. It need not postpone useful engineering until consensus exists elsewhere.

11.2.2 A federated and standardised Prolog Web

On the second route, the defining goal is cross-system interoperability. Implementation latitude must then be constrained wherever it becomes visible at a node boundary. Profiles must be independent of SWI-Prolog defaults, term and error encodings must be portable, protocol state must be specified, and a second implementation becomes indispensable evidence.

Federation also changes the social problem. The work can no longer be owned by one implementation community alone. Other Prolog implementers must be able to alter the contract, reject assumptions, and demonstrate different internal strategies. If formal standardisation is eventually pursued, public drafts, issue tracking, conformance tests, and implementation reports become part of the engineering rather than administrative decoration.

This route is more ambitious institutionally and may be better developed in a separate specification project. It nevertheless remains technically connected to the production route: a well-run SWI-Prolog platform can be its first source of experience, while a carefully designed portable boundary can prevent that platform from becoming needlessly isolated.

11.2.3 A decision that need not be made all at once

The routes are neither mutually exclusive nor automatically compatible. A feature may begin as an SWI-Prolog extension, acquire a documented remote contract, and only later prove suitable for a shared profile. Another may rely so deeply on one runtime that standardising it would destroy its advantage. The discipline required now is therefore classification: mark what is platform-specific, what is experimentally portable, and what is claimed as a stable interoperability contract. That classification is more honest, and more useful, than pretending that the eventual institutional form has already been decided.

11.3 From incubation to standardisation

This section is conditional. It describes a plausible path *if* Web Prolog is developed as shared, cross-system infrastructure. A production SWI-Prolog platform can and should proceed without waiting for that path to be chosen.

11.3.1 What should be standardised, and when?

It is too early to standardise the whole Trinity. It is not too early to publish small observable contracts against which independent implementations can experiment: profile boundaries, term and error encodings, node discovery, request and answer protocols, actor lifecycle rules, and the tests that reveal disagreement. If these surfaces remain implicit, they may still become de facto standards, but through accidental dependence on the first implementation.

The caution is close to home. ISO/IEC 13211–2:2000 standardised Prolog modules, yet widely used systems continued to expose materially different module facilities.¹ The lesson is not that module standardisation was pointless, but that a paper contract alone does not create convergence. Proposed surfaces should be incubated beside implementations, tests, and migration experience; speculative features should not be frozen in the hope that implementations will follow.

11.3.2 Incubation and institutional division of labour

The first venue need not be a formal standards body. Public drafts, issue tracking, executable tests, and implementation reports can begin in a project open to all affected implementers. For language and library facilities shared by more than one Prolog, the Prolog Improvements Forum and its PIP process provide a natural incubation path.² A solution cursor or dictionary interface should accumulate implementation experience there before being presented as a settled language abstraction.

The existing Prolog standards belong to ISO/IEC JTC 1/SC 22, whose Prolog working group is WG 17. That work is active: ISO/IEC TS 13211–3:2025 standardises definite-clause grammar rules, and a fourth part concerning Unicode characters in quoted Prolog text is under development.³ The name *ISO Prolog 2.0* used in this book remains a design hypothesis, not the title of an approved ISO project.

W3C supplies machinery for web architecture, formats, browser-facing APIs, and Recommendation-track standards. A W3C Community Group could provide an open place to incubate specifications, tests, and use cases, but Community Group reports are not W3C standards.⁴ A later standards track would require broader support, a chartered Working Group, implementation evidence, and public review. There is presently no W3C Web Prolog project.

IETF supplies a third coordinate. Its remit includes much of the Internet substrate on which inter-agent communication depends: transport, authentication and authorisation, discovery, and security mechanisms. These determine how a service

¹ ISO/IEC 13211–2:2000, <https://www.iso.org/standard/20775.html>.

² <https://prolog-lang.org/ImplementersForum/PIPs>

³ ISO/IEC JTC 1/SC 22 work programme: <https://committee.iso.org/committee/45202/x/catalogue/p/0/u/1/w/0/d/0>.

⁴ <https://www.w3.org/community/>

is located, how delegated authority crosses organisational boundaries, and what evidence remains after a consequential action. Web Prolog should reuse suitable Internet mechanisms rather than rebuild them.

No single standards body naturally owns the whole problem. ISO has the language remit and Prolog expertise; W3C has web-application and Recommendation-track machinery; IETF has the Internet protocol and security substrate. If formal standardisation becomes appropriate, a division of labour is more credible than assigning the Trinity wholesale to one organisation.



Fig. 11.2 Turning the Trinity design into shared infrastructure may require a liaison between language and web communities. The image shows one part of a wider institutional picture and is not an indication of endorsement.

The division is conceptual before it is bureaucratic. ISO is the natural forum for questions such as a future common Prolog base, solution cursors, dictionary terms, and profile-defined language choices. W3C is a possible forum for browser APIs, web-application integration, and the statechart lineage. IETF work is relevant to transport bindings, delegated authority, discovery, and cross-domain security. Term encodings, the actor layer, and the resumable-toplevel protocol cross these boundaries and require coordination rather than a simple hand-off.

Coordination between W3C and ISO/IEC is not unprecedented. W3C's WCAG 2.2 was processed through JTC 1 as a Publicly Available Specification and published as ISO/IEC 40500:2025.⁵ This does not predict the route Web Prolog would take; it shows only that such an institutional path is exercised rather than imaginary.

In terms of banners, the possible division can be pictured as two related projects:



Fig. 11.3 Provisional banners for a future general-purpose Prolog project and a web-facing Web Prolog project. Neither denotes an existing standard or an approved standards activity.

⁵ <https://www.w3.org/press-releases/2025/wcag22-iso-pas/>

The image deliberately makes the aspiration visible. Its presence is not a prediction that either project will take this form.

11.3.3 The SXML notation and SCXML

Although SXML is treated in this book as a specialised notation within Web Prolog, its statechart semantics have a distinct standards ancestry in SCXML. It may therefore require a separate specification track without constituting a separate implementation language.

The statechart strand has a clearer ancestor than Web Prolog itself. SCXML 1.0 became a W3C Recommendation in 2015 (Barnett et al., 2015). Chapter 6 implemented its transition-selection and run-to-completion model while making the scripting language, event matching, failure semantics, and actor invocation explicit. SXML is not a conforming SCXML binding, but neither is it merely a name for an aspiration: Section 6.7 summarises the execution model, Section 6.10.5 explains the rationale for the adaptation, and Appendix C specifies the notation element by element, with its relationship to SCXML tabulated in Section C.11.

The deeper proposal is that a successor should retain a declarative, inspectable state-machine core while allowing a capable host language to carry more of the executable content and communication. Timed transitions and state-scoped event deferral are SXML additions whose value and portability would have to be assessed alongside that core. The current author served as an invited expert and editor in the W3C Working Group that produced SCXML 1.0. That experience motivates the criticism, but confers no present W3C status on SXML.

This material should not become an extended re-litigation of individual SCXML elements in the final chapter. The important research question is whether the smaller division of responsibility demonstrated by SXML remains intelligible, inspectable, and portable with more than one host language. A detailed proposal for an actor-oriented SCXML successor would be substantial enough to stand as work in its own right.

11.4 The first specification targets

If the federation route is pursued, the targets should be layered so that each stage yields an independently testable boundary:

1. Publish the profile inventories, language-choice ledger, common term and error vocabulary, extension rules for answer metadata, and conformance-test format.
2. Specify RELATION over HTTP as the smallest cross-system query surface, including paging, consistency claims, and JSON representation rules.
3. Specify the resumable toplevel and PTCP lifecycle: start, call, continue, stop, abort, success, failure, error, output, and prompt.

4. Specify the ACTOR substrate and its WebSocket binding: identity, send and selective receive, monitors, lifetime dependence, termination, local names, public service names, authorisation, and containment.
5. Incubate SXML separately against its stated SCXML relationship and a dedicated conformance suite.

This order does not imply that implementation must occur sequentially. It says only that later specifications should reuse earlier contracts rather than define competing vocabularies.

11.4.1 The actor substrate as the distinctive target

`spawn/1--3` is the point at which a Prolog goal becomes an addressable, communicating entity. The ACTOR specification must define the observable contracts of `self/1`, `!/2` and `send/2--3`, `receive/1--2`, monitoring and demonitoring, parent–child lifetime dependence, termination, local registration, and public service publication. It must preserve the deliberate differences from Erlang established in the actor chapters rather than silently inherit the semantics of one host implementation.

11.4.2 Actor identity, messaging, and lifecycle

Identity must remain globally meaningful when a pid crosses a node boundary. The constructible `Id@Node` form is part of the observable contract: construction and decomposition are supported, while creating the term alone does not create an actor, establish liveness, or confer authorisation. Message delivery must define per-sender ordering, no-process behaviour, and renamed-apart copying with aliasing preserved inside each message but never between processes or separate sends. Receive must define tentative matching, binding rollback for rejected candidates, once-only guards, the treatment of guard exceptions and effects, deferral, timeout, and committed mailbox effects. These rules require conformance tests that exercise both local and remote placement. Monitoring must standardise the `down/3` vocabulary. Local registration and the owner-controlled `register_service/2` and `unregister_service/1` namespace must remain distinct. Authorisation and lifetime containment are part of the observable Web contract, not merely deployment advice.

11.4.3 The semantics of `spawn/3`

The `spawn/3` options expose several of the design’s trust decisions. Monitoring is atomic when requested at creation. Parent–child dependence is the default, while

an unlinked remote actor requires owner permission. Source loading, module or knowledge-base selection, time limits, and sandbox policy determine the execution context. The child must execute a renamed-apart copy of the supplied goal, so its bindings cannot instantiate variables in the parent, regardless of placement. A specification should classify each option as portable, profile-specific, owner-only, or implementation-specific and say which failures occur before a pid is returned.

11.4.4 The resumable toplevel actor

The toplevel protocol is the bridge between nondeterministic Prolog execution and deterministic network messages. A call establishes one computation; success messages carry slices; `toplevel_next/1--2` resumes that same computation; stop discards the continuation; abort interrupts a running goal. Recomputation with a larger offset is an implementation strategy only when its stability and side-effect restrictions satisfy the contract. It is not the semantic definition of continuation.

This modest protocol is rich enough to support remote shells, notebooks, browser clients, RPC, teaching environments, and distributed services. It is also valuable in a single-implementation platform: resumability is an interaction property, not merely an interoperability concession.

11.4.5 Solution cursors and the ISO liaison

The protocol points toward a possible language-level abstraction: a solution cursor that creates a resumable all-solutions computation, returns the next slice, and can be closed. A PIP developed with more than one implementation community would be a natural first step. ISO WG 17 is a plausible later forum for deciding whether the abstraction belongs in a future Prolog language standard. Web-facing work would specify how a cursor-backed toplevel is represented, controlled, secured, and tested. The protocol creates a use case for the language abstraction; it does not predetermine predicate names or implementation.

11.4.6 Layering the first standards

The central rule is that shared contracts fix observable behaviour while leaving implementations free behind the boundary. Actors may be implemented using threads, lightweight processes, event loops, or browser workers. Resumable queries may use engines, continuations, virtual-machine support, or a carefully bounded recomputation strategy. Those choices become standards matters only when they change what another participant can observe.

11.5 Implementation and interoperability

We are not analyzing a world, we are building it. We are not experimental philosophers, we are philosophical engineers.

Tim Berners-Lee, 2003⁶

11.5.1 Why implementation comes early

Implementation is a design instrument, not a ceremony performed after the specification. Running systems expose ambiguous errors, lifecycle races, unstated ordering assumptions, performance cliffs, and security boundaries that prose can hide. The demonstrator has served that role throughout this book. It should not be frozen as the standard merely because it arrived first, nor dismissed as “only a prototype” when its experiments reveal real design constraints.

On the production route, the next evidence comes from sustained use: workloads, failure injection, upgrades, monitoring, and applications maintained over time. On the federation route, the decisive evidence comes from another implementation meeting the same boundary. Both kinds of evidence can overturn the design, but they answer different questions.

11.5.2 Independent implementations and test suites

For federation, the most useful next artefact is a second independent implementation of a narrow layer, preferably RELATION and its term/error encoding before the full ACTOR profile. The conformance suite should link every normative requirement to tests and publish an implementation matrix. Disagreement then becomes evidence: the prose is ambiguous, a requirement is overconstrained, or an implementation is wrong.

For the SWI-Prolog platform, redundancy means something different: separate clients, deployments, browser engines, security reviews, and applications exercise the same implementation under varied conditions. A platform can become robust without becoming portable. It should simply avoid presenting that robustness as proof of cross-system interoperability.

11.6 AI-assisted implementation and standardisation

The demonstrator and this manuscript were developed with extensive assistance from language models, including ChatGPT, Claude, and Gemini. That experience supports

⁶ <https://lists.w3.org/Archives/Public/www-tag/2003Jul/0158.html>

a bounded claim: AI assistance can reduce the cost of producing and reviewing code, tests, documentation, and alternative designs. It does not transfer responsibility for architecture, protocol semantics, security, or standards decisions. Several model-generated implementations also do not count as independent evidence when they share the same specification assumptions and human direction.

AI assistance was most effective once the intended boundary was explicit: constructing interfaces, generating routine code, proposing variants, debugging transport interactions, and keeping the demonstrator aligned with revised contracts. It was less reliable when asked to infer the contract from an incomplete implementation. This reinforces the need for small stated surfaces and executable tests.

Language models can generate protocol variations, boundary cases, and hostile inputs cheaply, but generated tests require an independently stated oracle. An AI editorial agent can also check terminology, cross-references, normative keywords, requirement identifiers, test coverage, and implementation reports, and help keep code, manuals, and manuscript examples aligned as the design changes. For sandboxing in particular, these uses supplement rather than replace threat modelling, code review, and deployment hardening. Human participants must resolve issues and own every normative decision. The attractive reflexivity of using Trinity agents to help maintain Trinity specifications should not obscure that division of responsibility.

11.7 Shared ownership

Chapter 10 considered the conditions for adoption: useful services, visible examples, documentation, and places where people can meet the technology. The narrower question here is who acquires the right to shape it. On the production route, the first constituency can be SWI-Prolog users, application builders, and operators who subject one implementation to sustained use. On the federation route, independent Prolog implementers must participate from the beginning, because interoperability cannot be established by one implementation speaking to itself.

The neighbouring communities considered in Chapter 10 are therefore partners rather than audiences. Formal-semantics researchers, Semantic Web practitioners, agent researchers, Erlang implementers, browser engineers, security reviewers, and web developers possess expertise that may change the proposal. Shared ownership means more than help with implementation: it includes the ability to challenge terminology, reject assumptions, narrow requirements, and propose different boundaries.

Protocols do not manufacture a community, but shared objects can give one something concrete to negotiate: a useful platform, a public service, a failing conformance test, a disputed requirement, or an application that could not easily have been built before. The work becomes communal when those objects can answer criticism from people who did not design them.

11.8 Closure

This chapter began by returning to the Trinity diagram and asking what its commitments demand in practice. The answer is not one predetermined roadmap. A production SWI-Prolog platform can be worthwhile even if federation never follows; a portable profile can be valuable without determining the internals of every Prolog system; and statechart work may find its most natural future alongside either route or apart from both. Keeping these paths distinct makes cooperation possible without making the largest vision a precondition for every smaller success.

Measured against the five goals stated in Section 1.4, the present result is uneven by design. The actor-oriented concurrency model has been specified and exercised in the SWI-Prolog demonstrator, but not ported independently. Local and remote querying and actor communication share a coherent interface within that implementation, but cross-system distribution remains a goal. Web Prolog defines a candidate *lingua franca*, but it cannot become one until another implementation speaks it. The examples establish a substrate on which web agents and hybrid symbolic–neural systems can be programmed, without yet constituting a complete high-level agent language. Finally, the demonstrator is an interactable node environment, not yet the secure, production-quality turn-key node envisaged in the introduction. The five goals therefore function as an evaluation frame: each separates what has been demonstrated from what still requires engineering, independent implementation, or adoption.

Execution is therefore not a final step that follows design. It is the continuing process through which designs are tested, revised, constrained, and sometimes abandoned as they encounter users, independent implementations, hostile environments, and operational limits. Some parts of the Trinity may mature quickly, others may remain speculative, and still others may prove mistaken. Shared infrastructure earns trust by making such outcomes visible.

The book's contribution is to make a design explicit, embody much of it in a running demonstrator, and identify boundaries at which others can disagree. The next step is to publish those contracts, tests, implementation reports, and unresolved questions in forms that invite independent implementation and critical review. The banners with which this chapter opened make no claim of authority; they mark possible institutional homes for work that must remain open to revision.

An ecosystem cannot simply be declared into existence. If the Trinity becomes infrastructure, it will be because a community made it answerable to reality.

Part VII

Appendices

Appendix A

Glossary

This appendix gives working definitions for the principal terms used across the book. The definitions are specific to the Prolog Trinity where that usage differs from a broader one. They describe the architecture first and the current implementation where this helps to make the concept concrete. Individual predicates, options, commands, and endpoint fields remain documented in Appendix B; they are included here only when they also name an important architectural abstraction.

Actor. An actor is an independently scheduled concurrent process with a pid, a mailbox, and an explicit message interface. It executes a goal until that goal completes or the actor is terminated. Actors may be short-lived workers or durable services. Actors such as toplevels and statechart actors are specialised Erlang-style forms.

Agent. An agent is an addressable, persistent participant in interaction, defined by its interface and behaviour rather than by its implementation language. In the book's deliberately modest architectural sense, actors (toplevels, servers, supervisors, statechart actors, ...) as well as nodes are all Prolog agents because they receive goals or messages expressed as Prolog terms and respond with answers, messages, or effects in the same term-based form. Qualified terms such as *BDI agent* and *LLM agent* retain their richer meanings from AI.

Agentic Web. The Agentic Web is the emerging conception of a Web in which autonomous, communicating processes operate alongside documents, services, and human users as first-class participants. The term does not yet denote one settled architecture or standard. The Prolog Web is proposed as one possible execution substrate within this larger design space.

Answer slice. An answer slice is a bounded list of solutions returned as one unit by a toplevel or remote call. PTCP reports whether more solutions remain. A caller enumerates the current slice locally and requests another only at a slice boundary. Slicing therefore exposes Prolog nondeterminism incrementally without requiring one network exchange per solution.

Behaviour. A behaviour is a reusable control pattern for actors. The generic part supplies such machinery as message handling, state threading, monitoring, lifecycle

control, and cleanup; an application specialises it through a callback, child specification, or statechart model. Servers, supervisors, toplevels, and statechart actors are the principal behaviours in the book. Predicate-oriented generics such as `parallel/1` use actors internally but expose an ordinary Prolog predicate interface instead of a message protocol.

Browser node. A browser node is a Web Prolog node whose execution environment is a browser. In the demonstrator's default architecture, a JavaScript node controller runs on the main thread and each local actor runs an independent SWI-Prolog WASM instance in a web worker. The design preserves the same pid and message abstractions for actors in the current tab and actors on remote nodes. A separate main-thread mode exists for self-rendering Prolog pages that require direct DOM access.

Capability. In the security sense used in this book, a capability is an unforgeable reference whose possession confers authority to perform particular operations on a resource. A pid is instead a constructible address: knowing it grants messaging authority only where node policy permits a holder to send to that actor without further authorisation. Where policy requires additional credentials or permissions, the address alone is insufficient. Neither addressability nor pid secrecy establishes unforgeability. A restricted capability endpoint combines a protected reference with enforcement of its permitted operations and scope. The internal terminal endpoint, for example, uses an unguessable token for one terminal lineage and requires authenticated node transport; it is not a general actor address. The node's authorisation layer separately assigns named policy permissions, such as `execute`, `admin`, and `internal_transport`, to a principal. Capability values and policy permissions are different mechanisms, although both delimit what a client is authorised to do.

Committed effect. A committed effect is an effect that crosses the boundary of Prolog backtracking and is not undone when a later goal fails. Sending a message, selecting and removing a message from a mailbox, spawning an actor, and performing external I/O are examples. The distinction is central to the separation between Prolog engine state and actor process state.

Concurrent, sequential, and programmable Prolog Web. These are three complementary views of the Prolog Web. The *concurrent* Prolog Web provides remote actors and message passing. The *sequential* Prolog Web hides that substrate behind nondeterministic remote calls. The *programmable* Prolog Web lets code and data accompany calls or spawned actors. They are layers of one architecture rather than separate networks.

Configuration. A statechart configuration is the set of states currently active in the machine. In a hierarchical chart it includes the active leaf states and their ancestors; parallel regions may contribute several active leaves simultaneously. Transition selection, event visibility, deferral, entry and exit, and history restoration are all defined relative to the current configuration.

Conformance. Conformance is satisfaction of the normative requirements of a declared profile or specification. A RELATION-conforming node is not deficient

because it does not implement ACTOR; it has made a narrower promise. Credible conformance requires executable tests and feature-by-feature implementation reports, especially when interoperability between independent implementations is claimed.

Containment. Containment is the structural security discipline that bounds client-created computation in lifetime and effect. By default, a spawned actor is lifetime-dependent on its parent, so a client cannot leave uncontrolled work behind after its session ends. Long-lived detached services are owner-managed exceptions rather than the default.

Distribution-first. Distribution-first is the design stance that placement, remote identity, latency, partial failure, and trust boundaries belong to the programming model from the outset. It does not mean pretending that local and remote execution have identical costs. Web Prolog keeps the same core interaction constructs across node boundaries while making the network's timing and failure boundaries observable.

Event deferral. Event deferral postpones an event that is not to be handled in the current control state. In ordinary selective receive, a non-matching mailbox message remains available to later receive operations. In SXML, an unmatched selected event is discarded by default, while a `<defer>` declaration explicitly protects matching events and offers them again after the active configuration changes.

Eventless transition. An eventless transition is considered without requiring an incoming or internally raised event. It may be unconditional or guarded by a Prolog goal and is used for automatic progress, completion-style steps, and data-driven decisions. Eventless transitions run inside the current macrostep; an unending enabled cycle creates livelock.

Federation. Federation standardises what independently developed systems share at an interaction boundary without requiring their internals to become identical. In the Trinity, profiles, protocols, representations, and observable behaviour provide the shared surface, while different Prolog systems retain their own extensions, optimisations, libraries, and runtime architectures.

Flat SXML. Flat SXML is the subset of SXML without compound states, parallel regions, or history pseudostates. It retains ordinary states, transitions, executable content, the data model, timed transitions, event deferral, and run-to-completion execution. Every Flat SXML document is also a full SXML document; the name identifies a restricted vocabulary, not a separate language or behaviour.

History state. A history state is a statechart pseudostate that records which subconfiguration of a compound state was active when that state was left. A later transition to the history state restores that configuration instead of re-entering the compound state's ordinary initial state. It is useful for interruption, resumption, and conversational repair.

Host isolation. Host isolation constrains the runtime process outside the Web Prolog language. Operating-system confinement, containers, Web workers, WebAssembly

runtimes, and WASI capability boundaries are possible mechanisms. Host isolation is a backstop to language sandboxing, not a substitute for it: one limits what the process can reach, while the other limits what submitted code can express.

Hot code replacement. Hot code replacement changes the callback used by a running server while preserving the server actor and its current state. It separates application logic from the generic protocol loop, but it is safe only when the replacement accepts the existing state representation or performs an explicit migration.

Inexpressibility. Inexpressibility is the structural security property that dangerous operations cannot be expressed in client code. The exposed language fragment excludes direct file I/O, raw operating-system or socket access, durable node storage, and other ambient-authority mechanisms. Profile checks and sandbox policy approximate this boundary in the current node.

Instant portability. Instant portability is the limiting case of portability in which relocation requires no adaptation. A program can be downloaded for local execution or shipped to another node immediately because all its dependencies are bundled with it or guaranteed by the target profile.

Interaction portability. Interaction portability is preservation of a program's permitted observable behaviours when it is relocated across a declared class of execution environments under stated dependencies. The relevant observations may include queries, answers, effects, messages, and lifecycle events. Preservation concerns the permitted set of behaviours, not identical timing or one fixed trace.

Interaction surface. An interaction surface is the externally visible contract through which systems meet: routes, APIs, message and term formats, protocol rules, errors, timing behaviour, and profile promises. The term emphasises that interoperability depends on shared boundaries rather than shared internals.

Interoperability. Interoperability is the ability of independently implemented systems to work together correctly through shared protocols, representations, and behavioural contracts. It is stronger than syntactic compatibility: a client must be able to rely on the promised observable behaviour at the advertised profile boundary.

Invisibility. Invisibility is the name discipline that limits discovery through public client interfaces. In ordinary public client execution, `actors/1` intentionally lists actors belonging to the same client interaction; it does not reveal actors belonging to other clients or private owner resources. Node-owned runtime or administrative execution has broader visibility. Node-assigned pids and owner-curated service publication support this name discipline, but non-disclosure is not an access-control guarantee. A secret pid is not, by itself, a replacement for authentication or authorisation.

Link. A link in Web Prolog is a unidirectional lifetime dependence from a child actor to its parent. With the default `link(true)`, parent termination forces the child to terminate, and dependence propagates down the actor tree. Child termination does not kill the parent. This deliberately differs from Erlang's bidirectional links; detached `link(false)` actors are normally reserved for node owners.

Logical update view. The logical update view is the Prolog database semantics under which a predicate call sees the set of clauses that existed when the call began, even if clauses are asserted or retracted while its remaining solutions are enumerated. A retained continuation can preserve this per-invocation view across answer slices. It does not by itself give an arbitrary multi-predicate goal a transaction-wide snapshot, since predicate calls first entered later may see later updates. Stateless offset-based recomputation may lose even the per-invocation continuity unless the underlying relations remain stable; a point-in-time guarantee requires an additional transaction or versioning mechanism.

Mailbox. A mailbox is an actor's queue of incoming messages. Sending is asynchronous: a message is placed in the destination mailbox and the receiving actor selects it later. Web Prolog guarantees per-sender FIFO within one stable pid or name-addressed routing epoch, but not across mixed addressing, name rebinding, reconnects, different senders, or the signal channel. Selective receive may leave older non-matching messages in the mailbox.

Monitor. A monitor is a one-way observation of another actor's lifecycle. When the observed actor terminates, each monitor produces a `down(Pid,Ref,Reason)` message, where the reference identifies the monitor instance. Monitoring reports termination but creates no lifetime dependence. Installing `monitor(true)` during spawning avoids the race between process creation and a later call to `monitor/2`. For a remote monitor, `connection_closed` reports that observation was lost, not that the target was observed to terminate; the monitor is then discharged.

Multi-agent system. A multi-agent system is a system whose behaviour arises from several interacting agents. The Prolog Web qualifies in a minimal architectural sense because nodes and actors have identities, private state, protocols, and concurrent lifecycles. This says nothing by itself about autonomy, rationality, or the BDI model traditionally associated with multi-agent systems research.

Mutual constraint. Mutual constraint is the design principle that the Trinity's language profile, agent model, and network substrate must be designed together. The language limits what can safely cross the boundary; the agent model determines the protocols and lifecycle guarantees required; and the open Web determines which representations, timing assumptions, and trust relations are realistic.

Neuro-symbolic AI. Neuro-symbolic AI combines learned neural components with explicit symbolic representation or reasoning. In the book's architectural patterns, a model may supply language or perception while Prolog supplies rules, constraints, planning, verification, explanations, or control. The division is useful only where the interface between the two components is explicit and failures remain governed.

Node. A node is a network-addressable host of execution, knowledge, and policy. It exposes one or more Web Prolog APIs, advertises a profile, enforces security and resource policy, and may host shared predicates, sessions, and node-resident actors. A node is also a Prolog agent at a larger granularity: it has a persistent identity, receives requests, returns answers or effects, and participates in the network even if it is not implemented in Prolog.

Node controller. The node controller is the component that manages the node as a collection of actors and sessions. Its responsibilities include pid allocation, actor creation and termination, registries, routing, connection lifecycle, and enforcement of node-level policy. In the browser implementation it is JavaScript on the main thread; actor logic remains in web workers.

Node-resident actor. A node-resident actor is a durable actor installed and started under the node owner's authority rather than borrowed temporarily by a client. Such actors can provide shared state, coordination, or conversational services to many clients. They normally require stable publication, supervision, recovery, and resource policies.

Open-world assumption. Under the open-world assumption, the absence of a statement does not imply that the statement is false. OWL adopts this assumption because independently published knowledge may always be incomplete. Ordinary Prolog commonly uses closed-world idioms such as Negation As Failure within a delimited knowledge base. This semantic difference prevents OWL reasoning and plain Prolog execution from being treated as interchangeable.

OWL. OWL, the Web Ontology Language, is a family of W3C languages for expressing shared ontology semantics, including class hierarchies, property characteristics, and cardinality restrictions. OWL reasoners operate under open-world semantics and, in OWL 2 DL and its profiles, within decidable fragments. The Prolog Web treats OWL reasoning as an interoperable external service rather than attempting to replace it with ordinary Horn clauses.

Owner. An owner is the authority responsible for a node's public surface and long-lived resources. The owner installs shared predicates, publishes or withdraws services, configures profiles and policy, and accepts operational responsibility for durable services, supervision, and recovery. Owner authority is a role in the security model, not merely the first client to connect.

Parallel region (orthogonality). Parallel, or orthogonal, regions are statechart regions that are active at the same time within one compound state. An event is visible across the active configuration and may enable transitions in several regions during the same macrostep. This is structured coordination within one actor, not independently scheduled actor-level parallelism.

Pid. A pid is a globally meaningful process identifier for an actor. Its canonical global form is the constructible Prolog compound `Id@Node`, where the node assigns `Id`; a shell may suppress the node component when displaying a local pid. Constructing the term neither creates an actor nor establishes liveness or authority. A valid pid is a routable handle; possession confers messaging authority only to the extent permitted by node policy (see *Capability*). It is distinct from a descriptive registered or published name.

PIP. A Prolog Improvement Proposal, or PIP, documents a language or library facility supported, or intended to be supported, by more than one Prolog system through the Prolog Improvements Forum. PIPs are not ISO standards. They provide

an incremental, implementation-led route for enlarging the portable common ground and may supply evidence for later standardisation.

Portability. Portability is the extent to which source code or an interaction pattern can move among nodes or implementations without semantic breakage or manual rewriting. The Trinity pursues portability through explicit profiles and dependencies rather than demanding complete internal uniformity. See also Interaction portability and Instant portability.

Principal. A principal is the identity on whose behalf a request is made. It is not the same thing as a user, connection, session, or pid. A principal may be an anonymous caller, a human user, a service account, or a trusted internal transport identity. Authorisation maps a principal to the operations it may perform.

Private database. A private database is the actor-local collection of clauses and facts that forms part of an actor's workspace. It can be populated at spawn time through `src.*` options and updated without modifying the node's shared database. Private definitions may shadow shared definitions, allowing one actor to specialise its logical environment.

Profile. A profile is a named, cumulative conformance contract stating which language constructs, predicate families, transports, APIs, and interaction patterns a node provides. It constrains both the node's *language surface* and its *interaction surface*. Profiles make implementation latitude explicit and allow a portable operational subset without turning Web Prolog into a separate fork of Prolog. A profile promise must include observable semantics and conformance tests, not only a feature list.

Profile hierarchy (RELATION, ISOBASE, ISOTOPE, ACTOR). The profile hierarchy is a nested sequence in which each profile extends the preceding one. RELATION exposes relational querying. ISOBASE adds a restricted ISO-style goal language and permits client-supplied source in a call's temporary execution context. ISOTOPE adds persistent toplevel sessions: supplied definitions and dynamic database updates can survive across requests within the session, which also supports interactive I/O. ACTOR adds Erlang-style concurrency, distribution, and actor behaviours. A system need only implement the narrowest profile that matches the surface it can honestly provide.

Prolog engine. A Prolog engine is the execution mechanism that evaluates a goal, maintains variable bindings and choice points, and produces successive solutions. In the Trinity architecture, engine state is distinct from actor process state: backtracking can restore substitutions and proof search, but cannot unsend messages, unspawn actors, or restore selected mailbox messages.

Prolog Trinity ecosystem. The Prolog Trinity ecosystem comprises Web Prolog as a language profile, Prolog agents as the execution entities, and the Prolog Web as the infrastructure in which those agents interact. The three components are distinct but mutually dependent: the Web is populated by agents, the agents are programmed in and communicate with Web Prolog, and the language acquires its web meaning through the nodes and protocols that host it.

Prolog Web. The Prolog Web is the network formed by network-addressable nodes that host Web Prolog knowledge, execution, actors, and services. It extends the conventional Web with executable relations, nondeterministic remote querying, long-lived communicating processes, and mobile source. It is intended as an independent but interoperable execution stack, not as a replacement for the conventional or Semantic Web.

PTCP. PTCP is the Prolog Toplevel Communication Protocol. It governs conversation with a toplevel actor: calls establish computations, success messages deliver answer slices, and *next* resumes a retained computation. *Stop* discards a continuation, *abort* interrupts a running goal and returns a persistent session to its idle state, and *halt* terminates the toplevel actor. Output, prompts, failures, errors, and normal completion are explicit protocol events.

Public surface. A node's public surface is the set of predicates, services, routes, behaviours, and policies it intentionally exposes to clients. It is smaller than the node's internal runtime and should be an explicit, auditable design choice. Everything outside it should be inexpressible, invisible, contained, or protected by deployment policy.

Pure Web Prolog and the pure Prolog Web. *Pure Web Prolog* is the pure fragment obtained when pure Prolog is extended with *rpc/2-3* calls to pure, stable remote predicates. The *pure Prolog Web* is the aspirational distributed logical environment formed by such fragments. Network failure may truncate an answer stream, but does not change the logical status of answers already derived from the stated pure context.

RDF. The Resource Description Framework is the Semantic Web data model in which information is expressed as subject–predicate–object triples. Resources and properties are identified globally by IRIs. In SWI-Prolog's Semantic Web libraries, the elementary triple pattern maps naturally to the relation *rdf/3*; those libraries are host-provided facilities rather than part of the standard Web Prolog profiles.

Registered name. A registered name is a symbolic name bound to an actor for lookup and messaging. Ordinary client-side registration is distinct from owner-controlled service publication. The former provides a convenient name within the applicable registry; the latter deliberately adds a durable service to the node's public namespace.

Reply channel. A reply channel is an endpoint created to receive one reply or a narrowly bounded class of replies. A temporary proxy actor can serve this role. Across a trust boundary, a reply channel avoids disclosing the client's general pid and grants only the authority needed for the current protocol step.

Resource governance. Resource governance controls how much work an otherwise permitted computation may consume. Time limits, memory ceilings, spawn limits, quotas, admission control, overload policy, and accounting are examples. Resource governance is distinct from sandboxing: a goal may use only allowed operations and still exhaust CPU, memory, process slots, or bandwidth.

RPC meta-predicate. *rpc/2-3* is the nondeterministic remote procedure-call interface of the sequential Prolog Web. A caller submits a goal to a node and consumes its

solutions through ordinary Prolog backtracking. The current SWI-Prolog implementation uses stateless HTTP and sliced retrieval; an actor-based reconstruction shows how the same abstraction can be expressed through a toplevel actor and PTCP.

Run-to-completion (macrostep). Run-to-completion is the statechart discipline in which one external event is processed through a finite cascade of transition microsteps, eventless transitions, and internally raised or re-offered events before the next external message is selected. The complete reaction is a *macrostep*. When it terminates, the machine has reached a stable configuration; if it does not terminate, the statechart is livelocked.

Sandboxing. Sandboxing is the language-level enforcement of which goals submitted by an untrusted client may run. Whitelists, profile checks, meta-call validation, and restrictions on dynamically constructed goals are possible mechanisms. Sandboxing answers *what may this program do?*; host isolation and resource governance answer different questions and remain necessary.

Scoped inference. Scoped inference evaluates a goal with respect to a delimited logical context, such as the knowledge base at a named node. In `rpc(URI, Goal)`, the URI identifies the context whose predicates support or fail to support the goal. Delimiting scope is particularly important for Negation As Failure, since the open Web cannot be treated as one complete closed-world database.

SCXML. SCXML is the W3C State Chart XML Recommendation, an XML-based state-machine language with a specified transition-selection and run-to-completion processing model. SXML draws on that execution model but is not a conforming SCXML language binding: it adopts Web Prolog terms, unification, actor invocation, and a smaller surface syntax.

Selective receive. Selective receive scans an actor's mailbox for the oldest message that matches one of the current receive clauses, including any guard. Non-matching messages remain available for later receive operations. Once a message is selected, its removal is committed even if the selected clause body later fails or backtracks.

Semantic Web. The Semantic Web is the W3C-associated vision and technology stack for a Web of machine-processable data with shared identifiers, vocabularies, and formally stated meaning. RDF supplies a common data model, SPARQL a query language, and OWL ontology semantics. The Prolog Web complements this representation-centred stack with executable relations, actors, and services.

Sequential and concurrent partitions. The *sequential partition* is the world of goals, relations, substitutions, backtracking, and nondeterministic answers. The *concurrent partition* is the world of actors, mailboxes, messages, and lifecycle events. Implementations may realise a sequential abstraction using actors, and actors may reason sequentially, but the programming model keeps the two partitions distinct so that proof search is not confused with interaction state.

Server. A server is a stateful actor that maintains private state and serves clients through a request–response message protocol. The generic server behaviour provides

routing, correlation, state threading, monitoring, upgrade, and shutdown machinery; an application callback defines how one request transforms the old state into a response and a new state. It is an Erlang-style protocol abstraction rather than a logical predicate interface.

Service. A service is a durable, shared facility intentionally offered by a node for repeated use by multiple clients. A service may expose a predicate or an actor protocol. An actor service differs from an arbitrary named actor because the node owner publishes it under a stable public name and accepts responsibility for its lifecycle and policy.

Service discovery. Service discovery is the process by which clients learn which published services a node offers. It resolves owner-curated public names rather than enumerating all running processes. The book's Discovery Hub illustrates a declarative directory that clients can query through `rpc/2-3`.

Service publication. Service publication is the owner-controlled act of placing a service name in a node's public service namespace. It makes the service part of the advertised public surface and is therefore distinct from ordinary client-side actor registration.

Session. A session is a bounded continuing interaction context in which state may persist across several protocol steps. A toplevel session can retain definitions, I/O state, and a current computation; an ACTOR WebSocket session can retain mailboxes and process relationships. Session lifetime, idle waiting, running computations, and transport connections have related but distinct failure boundaries.

Shadowing. Shadowing occurs when a predicate in an actor's private database takes precedence over a predicate with the same indicator in the node's shared database. It supports local specialisation without modifying shared knowledge. The precise module and lookup rules are part of the actor's logical-environment contract.

Shared database. The shared database is the node-owned collection of predicates made available as shared knowledge and skills. It is installed by the owner and may be imported into actor contexts or used by node-level query execution. Clients can combine it with private source, but cannot treat it as their own mutable workspace.

Solution cursor. A solution cursor is a proposed language-level abstraction for a resumable all-solutions computation. It would create a computation, return bounded slices on demand, and support explicit closing. PTCP supplies the motivating web-facing use case, but the book does not claim that cursor predicate names or language semantics have already been standardised.

SPARQL. SPARQL is the W3C query language for RDF graphs. It defines graph-pattern matching together with projection, grouping, ordering, aggregation, optional matches, and other operations under specified multiset and error semantics. Similar-looking Prolog queries can be concise and composable, but are not automatically semantics-preserving translations of SPARQL.

Statechart actor. A statechart actor is an actor whose control logic is given by an SXML statechart document. Incoming messages become external events that drive

an explicit hierarchy of states and transitions. The actor retains an ordinary pid and mailbox and can be monitored, supervised, and invoked like other actors.

Stateless, semi-stateful, and stateful APIs. The *stateless HTTP API* completes each request independently and retrieves solution slices by request parameters. It underlies both synchronous *rpc/2-3* and the split-phase *promise/3-4* and *yield/2-3* abstraction. The *semi-stateful HTTP API* maps PTCP onto a family of HTTP endpoints whose requests address the same persistent toplevel actor by pid. The *stateful Web-Socket API* maintains a bidirectional connection through which actors, mailboxes, and PTCP conversations persist. These APIs, and the abstractions layered over them, expose different blocking and failure boundaries.

Supervisor. A supervisor is an actor whose primary responsibility is to start, monitor, terminate, and restart child actors according to a declared policy. It embodies the “let it crash” discipline: application workers may fail while the supervisor supplies recovery structure. Supervisors are especially important for durable node-resident services.

SXML. SXML, or Statechart XML, is the book’s compact, SCXML-inspired statechart notation within Web Prolog. It provides statechart-specific control structure while using Web Prolog as its data model and executable content, Prolog terms and unification for events, explicit actor spawning for invocation, and SCXML-derived transition selection and run-to-completion semantics. Timed transitions and state-scoped event deferral are deliberate SXML additions.

Timed transition. A timed transition is an SXML transition with an *after* attribute. Its timer is armed when the source state is entered and cancelled automatically when that state is exited. The timeout therefore belongs to one activation of a control state, avoiding the manual send-and-cancel bookkeeping otherwise needed to deliver a delayed event.

Timing and failure boundary. A timing and failure boundary is an operation at which waiting becomes observable and may end through timeout, disconnect, cancellation, or remote failure. A stateless HTTP request, a *yield/2-3* wait, a semi-stateful session request, and a mailbox receive are different boundaries even when they ultimately concern the same computation. Timeout values and recovery actions therefore belong to the interaction model, not merely to transport defaults.

Toplevel actor. A toplevel actor is an actor governed by PTCP for submitting goals, delivering answer slices, handling output and prompts, and controlling the current computation. A persistent toplevel supports a sequence of queries; a one-shot toplevel terminates when its query is exhausted or stopped. Toplevel actors underpin interactive shells and provide the actor model from which remote predicate calls can be reconstructed.

Unification. Unification is the operation that makes two Prolog terms equal by finding a consistent substitution for their variables. In the Trinity it is also a structural data contract: a goal states both the request and the expected shape of its answers, while

message patterns and statechart event patterns use the same mechanism. Unification guarantees structural compatibility, not shared real-world meaning.

WASM and WASI. WebAssembly (WASM) is a portable binary execution format supported by browsers and other hosts; SWI-WASM brings SWI-Prolog into that environment. The WebAssembly System Interface (WASI) defines capability-oriented interfaces for selected host resources outside a browser. WASM is an execution substrate, not by itself a sandbox policy or a complete resource-governance mechanism.

Web Prolog. Web Prolog is the language profile of the Prolog Trinity. It starts from a selected ISO Prolog base, adds standard library and Erlang-style actor facilities, and defines profile-scoped APIs for execution over the Web. It is a proposed interoperable language surface, not a new Prolog implementation: different systems may host it while retaining different internals.

Webizing Prolog and prologizing the Web. *Webizing Prolog* means adapting Prolog to URI-addressed nodes, open protocols, browsers, security boundaries, and partial failure. *Prologizing the Web* means bringing Prolog terms, unification, non-deterministic querying, executable rules, and actors to web interaction. The Trinity arises from pursuing both directions together rather than treating either as a thin compatibility layer.

Appendix B

Manual

This manual describes the Web Prolog language and predicate APIs implemented by the current demonstrator. The node's HTTP and WebSocket interfaces are specified separately in Appendix D. Both appendices should be read as snapshots of a design in progress, not as final specifications: terminology, interfaces, and behaviour remain tentative and may change as the implementation evolves.

The classification shown beside each entry identifies the minimum public node profile in which the predicate is available, or another relevant execution context. ISOBASE provides general stateless Prolog calls, ISOTOPE adds persistent toplevel sessions, and ACTOR adds the full stateful actor-messaging model. The profiles are cumulative, so an entry marked ISOTOPE is also available in ACTOR. The classification concerns the client language, not predicates used internally to implement a profile. Entries additionally marked OWNER ONLY are reserved for code running with node-owner authority and are not available to ordinary client code.

B.1 Predicates for programming with core actors

Predicate: `self/1`

ACTOR

`self(-Pid) is det.`

Binds `Pid` to the process identifier of the calling process.

Note: A pid is an actor address whose canonical global form is `Id@Node`, where `Id` is assigned by `Node`. This ordinary Prolog compound is part of the language contract: programs may copy, store, construct, decompose, compare, and serialise it. A shell may omit `@Node` when displaying a pid on the shell's own node, but this is only a local abbreviation. Constructing a pid does not create an actor, confirm

that it is live, or grant access to it; an operation reaches an actor only when the pair designates that actor and the node authorises the operation. Arithmetic on a numeric `Id` has no defined actor-level meaning. Browser runtimes reserve `localhost` as the local node designator. Web worker actors are therefore presented as, for example, `2159438818@localhost`, and the optional main-thread SWI-WASM engine is presented as `main@localhost`.

Predicate: `spawn/1-3`

ACTOR

```
spawn(+Goal) is det.
spawn(+Goal, -Pid) is det.
spawn(+Goal, -Pid, +Options) is det.
```

Creates a new Web Prolog actor process running a copy of `Goal`. Variables occurring more than once in `Goal` retain that sharing within the copy, but no variable identity is shared between the parent and child: bindings produced by the child do not bind variables in the parent. This rule is the same for local and remote spawning. Results that must cross the process boundary have to be returned explicitly, for example in a message or through a toplevel protocol.

Valid options are:

- `node(+URI)`
Creates the process locally or on a remote compatible node. Default is `localhost`.
- `monitor(+Boolean)`
If `true`, monitoring is installed as part of process creation. Default is `false`.
- `link(+Boolean)`
If `true`, installs directional parent-to-child link cleanup. Default is `true`.
- `src.text(+AtomOrString)`
Loads the clauses specified by a Web Prolog source text into the actor's private Prolog database before calling `Goal`.
- `src.list(+ListOfClauses)`
Converts the clauses to source text and loads them into the actor's private Prolog database before calling `Goal`.
- `src.uri(+URI)`
Loads the clauses specified by a file path, file URI, or HTTP(S) URI into the actor's private Prolog database before calling `Goal`.
- `src.predicates(+ListOfPredicateIndicators)`
Loads the local predicates denoted by `ListOfPredicateIndicators` into the actor's private Prolog database before calling `Goal`.

Note: In `spawn(+Goal, -Pid, +Options)`, the option `monitor(true)` installs monitoring as part of process creation. When the child terminates, the parent receives a message `down(Pid, Ref, Reason)` where `Pid` is the terminated actor, `Reason` is its exit reason, and `Ref` identifies the monitor instance. For monitoring created via `monitor(true)`, the portable contract is `Ref = Pid`; this lets the spawning process identify and remove that monitor even though `spawn/3` returns no separate monitor

reference.

Down-reason contract: The third argument of `down(Pid, Ref, Reason)` uses the following portable vocabulary: `true` means that the actor goal succeeded; `false` means that it failed; `exception(E)` carries the public representation of the uncaught exception; an arbitrary nonvar term may be supplied explicitly through `exit/1-2`; and `no_process` means that the monitored target did not exist when the monitor was installed. The boundary reason `connection_closed` means that observation of a remote target was lost; it does not assert that the target terminated. For an uncaught SWI-Prolog `error(Formal, Context)`, the monitor notification preserves `Formal` but replaces `Context` with an unbound variable. An undefined-procedure error also omits a private generated actor-module qualification. Other exit reasons are unchanged.

Note: The semantics of `link(true)` are directional: if a parent spawns a child with linking enabled, termination of the parent causes linked children to be terminated. The link does not imply symmetric bidirectional exit propagation, and the child cannot trap the termination. For a remote child, its hosting controller enforces the relationship from explicit parent-termination information and, in the current demonstrator, from closure or failure of the authenticated connection that owns the remote subtree. No bounded silent-partition detection is claimed; see Section 4.1.7.

Note: It is possible to pass an arbitrary number of the `src_*` options to `spawn/3`, and possibly more than one instance of each variant. To ensure that clauses end up in a well-defined order, they are converted into Prolog source text before being loaded into the database. The order of clauses and directives in the resulting source text is determined by the order of the `src_*` options in the option list.

Predicate: `actors/1`

ACTOR

`actors(-Pids) is det.`

Binds `Pids` to the list of actor pids visible from the current execution context. In ordinary public client execution, this means the current actor or toplevel together with any actors that have been spawned on behalf of the same client interaction. Actors belonging to other clients are not included.

Note: Outside public client execution, for example in node-owned runtime code or administrative code, `actors/1` returns the pids of all active local actors on the node.

Predicate: `monitor/2`

ACTOR

`monitor(+PidOrName, -Ref) is det.`

Installs a monitor and returns a fresh reference in `Ref`. The first argument may be a pid or a registered local name. When the monitored process terminates, the monitoring

process receives a message of the form `down(Pid, Ref, Reason)`. If observation of a remote target is lost, the runtime instead delivers the same form with `Reason = connection_closed`. This discharges the monitor without claiming that the target terminated; reconnection does not restore it automatically.

Note: Using `monitor(true)` in `spawn/3` installs monitoring before the child can terminate, so its actual exit reason is observed. If `monitor/2` is called after the target has already terminated, the runtime still discharges the monitor with `down(Pid, Ref, no_process)`.

Predicate: `demonitor/1-2`

ACTOR

`demonitor(+Ref)` is det.

`demonitor(+Ref, +Options)` is det.

Stops monitoring identified by `Ref`. This is idempotent. There is only one valid option:

- `flush`
Non-blocking removal of all pending `down(_, Ref, _)` messages from the mailbox.

Predicate: `register/2`

ACTOR

`register(+Name, +Pid)` is det.

Registers an actor process under a name, where `Name` is an atom and `Pid` identifies the actor. The association between the name and the pid is removed when the process terminates.

Predicate: `whereis/2`

ACTOR

`whereis(+Name, -Pid)` is det.

Binds `Pid` to the actor process associated with `Name`, or to `undefined` if no such registration exists.

Predicate: `unregister/1`

ACTOR

`unregister(+Name)` is det.

Removes the association between the name and the process. The actor itself is not terminated. The operation is idempotent and succeeds without effect if `Name` is not registered.

Predicate: `register_service/2`

ACTOR, OWNER ONLY

`register_service(+Name, +Pid)` is det.

Publishes the live local actor identified by `Pid` under `Name` in the node's public service namespace. `Name` must be an atom, `Pid` must identify an actor on the publishing node, and there may be at most one published service with a given name on a node. If the name is already published, `register_service/2` raises `permission_error(register, service, Name)`, even when the existing entry denotes the same actor. Replacement is therefore explicit: the owner must first withdraw the old mapping.

The public service namespace is distinct from the ordinary registration namespace used by `register/2`; the same atom may occur in both without a collision. A qualified address `Name@Node` is resolved against the public service namespace of `Node`. `whereis/2` consults only the ordinary namespace; owner-side code may inspect the service namespace with `whereis_service/2`.

Publication neither changes the actor's links nor places it under supervision. In particular, `register_service/2` does not detach the actor from its parent. When the actor terminates, the node controller removes the service mapping atomically, so a published name never remains bound to a dead pid. A supervisor that starts a replacement actor must publish that actor again under the stable name.

Only code executing with node-owner authority may call this predicate. A call by any other principal raises `permission_error(register, service, Name)`. A non-atom `Name` raises `type_error(atom, Name)`; a `Pid` that does not identify a live actor raises `existence_error(process, Pid)`; and an attempt to publish an actor on another node raises `permission_error(register, service, Pid)`.

Predicate: `whereis_service/2`

ACTOR, OWNER ONLY

`whereis_service(+Name, -Pid)` is det.

Binds `Pid` to the live local actor published as `Name`, or to `undefined` if no such service is published. This predicate inspects only the public service namespace. Only code executing with node-owner authority may call it; other callers receive:

```
permission_error(access, actor_service_registry,
                 whereis_service(Name, Pid))
```

A non-atom `Name` raises `type_error(atom, Name)`.

Predicate: `unregister_service/1`

ACTOR, OWNER ONLY

`unregister_service(+Name)` is det.

Withdraws `Name` from the node's public service namespace. This operation removes only the name; it neither terminates nor otherwise changes the actor that the name denoted. The operation is idempotent: if no published service called `Name` exists, it succeeds without effect.

Only code executing with node-owner authority may call this predicate. A call by any other principal raises `permission_error(unregister, service, Name)`, and a non-atom `Name` raises `type_error(atom, Name)`.

Predicate: `exit/1`

ACTOR

`exit(+Reason) is det.`

Terminates the calling process with Reason.

Predicate: `exit/2`

ACTOR

`exit(+Pid, +Reason) is det.`

Sends an untrappable termination signal to the process identified by `Pid`, using Reason as its exit reason. The signal is not a mailbox message. For remote actors, the runtime routes this request through the remote node. Repeated requests after the actor's terminal outcome have no further observable effect; see Section 4.1.7.

Predicate: `!/2, send/2-3`

ACTOR

`+PidOrName ! +Message is det.`

`send(+PidOrName, +Message) is det.`

`send(+PidOrName, +Message, +Options) is det.`

Sends Message to the mailbox of the process identified as `PidOrName`. `PidOrName` may be a local pid, a local registered name, a global pid of the form `Id@Node`, or a published service address of the form `Name@Node`. A global pid may be a value returned by the runtime or a term constructed directly from a known identifier and node; the two are equivalent. Qualified service addresses are resolved only through the destination node's live service registry. If such a service is absent or the sender is not authorised to use it, the send succeeds without delivery and does not reveal which condition applied. By contrast, an unknown local registered name raises `existence_error(actor_name, Name)`; a send to an unknown pid succeeds without delivery. Valid options for `send/3` are:

- `delay(+Number)`
Delays the sending by a specified number of seconds. Default is 0.
- `id(+ID)`
ID is a user-supplied identifier that can be used by `cancel/1` to stop the sending from taking place.

Portable actor application messages are acyclic ordinary Prolog terms. Atoms, strings, numbers, variables, lists, and compound terms are portable. Each `send` constructs a fresh copy in the recipient: repeated occurrences of one variable remain shared within that copied message, but no variable identity is shared with the sender or with any separately sent message. There is consequently no distributed unification channel hidden in message passing. Attributed variables, streams, and other non-text blob values must be encoded explicitly by the application. The demonstrator validates this subset whenever serialisation is required. Native local delivery may happen to carry additional host terms, but programs using them are outside the

placement-independent contract.

Ordering: For one sender, messages sent repeatedly to the same pid are delivered in send order while one controller route remains in use. The same holds for one unchanged name binding during that routing epoch. There is no portable order across pid/name addressing, name rebinding, reconnects or rerouting, different senders, or messages and runtime signals. For delayed sends, ordering begins when the delayed operation actually emits the message. See Section 4.1.6.

Predicate: `cancel/1`

ACTOR

`cancel(+ID) is det.`

Tries to cancel all delayed messages with the specified ID that were scheduled by the calling actor. Identifiers are actor-local: another actor may use the same ID without its delayed sends being affected. Cancellation is best-effort only, since a message may already have been sent by the time the call is made.

Predicate: `output/1-2`

ACTOR

`output(+Data) is det.`
`output(+Data, +Options) is det.`

Sends a message `output(Pid, Data)` to the target process. `Pid` is the pid of the current process. Valid option:

- `target(+Pid)`
 Send the message to `Pid`. Default is the parent process.

Note that this is just a convenience predicate. Any actor may use `!/2` to send any term to any process to which it has a pid.

Predicate: `input/2-3`

ACTOR

`input(+Prompt, -Data) is det.`
`input(+Prompt, -Data, +Options) is det.`

Sends a message `prompt(Pid, Prompt)` to the target process and waits for its input. `Prompt` may be any Prolog term. `Pid` is the pid of the current process. `Data` will be bound to the term that the target process sends using `respond/2`. Valid option:

- `target(+Pid)`
 Send the `prompt` message to `Pid`. Default is the current I/O target when one has been installed, or otherwise the parent process.

Predicate: `respond/2`

ACTOR

`respond(+Pid, +Input) is det.`

Sends **Input** to the process identified by **Pid** as a response to a **prompt (Pid, Prompt)** message.

Predicate: `receive/1-2`

ACTOR

`receive(+Clauses)` is `nondet`.
`receive(+Clauses, +Options)` is `nondet`.

Waits for, selects, and consumes one message from the calling process's mailbox. A call is *single-message*, but not necessarily single-solution. During one invocation it selects and consumes at most one message. Messages are considered in arrival order, and each message is tested against the receive clauses in textual order. The first message accepted by a clause is removed from the mailbox. This selection is committed: backtracking through the invocation never selects another message or another clause.

The selected clause body is ordinary Prolog code. The call as a whole therefore has the determinism of the fired body: a body may fail, succeed once, throw an exception, or yield several solutions. Choice points left by the body remain available, but they enumerate solutions of that same body; they do not inspect the mailbox again. The selected message remains consumed if the body fails or throws, and is not restored on backtracking. Thus the `nondet` declaration concerns the possible number of logical solutions, not the number of messages consumed by one invocation. For example, after one `foo` message has arrived, `receive({foo -> member(X, [a,b])})` consumes that message once but can yield both `X = a` and `X = b`.

A cut executed in a selected clause body is scoped to that body, as if the body were invoked through `call/1`. It can prune choice points created since entry to the body, but it cannot prune choice points in the caller; nor does it alter the already committed message and clause selection.

Clauses has the following form:

```
{ Pattern1 [if Guard1] ->
    Body1 ;
    ...
    PatternN [if GuardN] ->
    BodyN
}
```

A clause accepts a message if the message unifies with its pattern and its optional guard succeeds. Candidate matching and guard evaluation use a tentative substitution. If unification or the guard fails, all bindings made while trying that candidate are discarded before the next clause or message is considered; the mailbox is not changed until a clause accepts. A guard is evaluated for its first solution only, as if by `once/1`. Bindings from the successful pattern and guard are in scope in the body.

For portable code, a guard must be observationally free of side effects; the runtime is not required to enforce this restriction, and effects performed by a non-conforming guard are not undone when that guard fails. If a guard raises an exception, the

exception and its tentative bindings are discarded and the clause does not accept the message. This error-as-non-match rule follows the selection discipline of restricted actor guards, but it also means that a guard which calls a fallible external service should translate failures into explicit results when the actor needs to distinguish them from an ordinary non-match.

Messages accepted by no clause are *deferred*: they remain in the mailbox, without any change in their contents or relative order, and are available to later receive calls. `receive({})` accepts no message and waits, subject to `Options`. Valid options for `receive/2` are:

- `timeout(+Number)`
Bounds the wait for an accepted message, in seconds. The value `0` inspects messages already in the mailbox without waiting for another. Default is no timeout.
- `on_timeout(+Goal)`
Called if the timeout expires with no message accepted. Its success, failure, exception, and any choice points determine the outcome of the receive on the timeout path. Default is `true`. This option is only meaningful when `timeout/1` is present.

Predicate: `flush/0`

ACTOR

`flush` is det.

Removes all messages currently pending in the mailbox of the calling process. Each removed message is rendered through the actor terminal-output path as a line of the form `Shell got Message`, where `Message` is the full Prolog term.

Note: `flush/0` is primarily a shell-level inspection utility. It does not wait for future messages: once the mailbox is empty at the time of the call, it succeeds immediately.

Predicate: `listing/0-1`

ISOTOPE

`listing` is det.
`listing(+What)` is det.

`listing/0` lists the content of the private database of the current actor process through the terminal-output path. Imported predicates and the injected I/O wrapper predicates are omitted.

`listing/1` lists the predicates or clauses in the current process's private database selected by `What`, following the conventional Prolog meaning of `listing/1`. `What` may be a predicate indicator (`Name/Arity` or `Name//Arity`), a callable head pattern, a clause reference, or a list of such specifications. It does not provide access to another actor's private database.

B.2 Predicates for programming with toplevel actors

Predicate: `toplevel_spawn/1-2`

ACTOR

`toplevel_spawn(-Pid)` is det.
`toplevel_spawn(-Pid, +Options)` is det.

Spawns a toplevel actor and binds `Pid` to its pid. All options accepted by `spawn/3` are also accepted by `toplevel_spawn/2`. In addition, `toplevel_spawn/2` accepts the following options:

- `session(+Boolean)`
 If set to `false`, the toplevel actor terminates after having run a goal to completion. If `true`, further interaction is expected. Default is `false` in the actor API.
- `target(+Pid)`
 Send all answer terms to `Pid`. Default is the pid of the parent.
- `name(+Name)`
 Register the toplevel actor under `Name` in the local ordinary-name namespace.
- `time_limit(+SecondsOrInfinite)`
 Bounds each continuous period spent executing a goal in PTCP state `s2`. On expiry, the target receives `error(Pid,time_limit_exceeded)`. A session toplevel remains alive and returns to `s1`, ready for another call. Time spent waiting for a continuation in `s3` does not count; the running time limit starts again when a continuation resumes. The actor API default is `infinite`; a public node may impose a finite ceiling.
- `idle_limit(+SecondsOrInfinite)`
 Bounds inactivity while a session toplevel waits for a call in PTCP state `s1` or for a continuation in `s3`. On expiry, the actor terminates normally, so a monitor observes `down(Pid,Ref,true)`. The actor API default is `infinite`; a public node may impose a finite ceiling.

Both finite lifecycle limits must be positive numbers of seconds. A public client may request tighter values, but cannot weaken the node owner's ceilings. These limits are distinct from the `timeout` on an ISOTOPE session HTTP request, which only bounds the wait for the next queued event and does not abort the goal or terminate the session.

Predicate: `toplevel_call/2-3`

ACTOR

`toplevel_call(+Pid, +Goal)` is det.
`toplevel_call(+Pid, +Goal, +Options)` is det.

Asks the toplevel `Pid` for solutions to `Goal`. Valid options are:

- `template(+Template)`
`Template` is a term sharing variables with `Goal`. By default, the template is identical to the goal.

- **offset(+Integer)**
Collect only the slice of solutions starting from **Integer**. Default is **0**.
- **limit(+Integer)**
Restrict the length of the returned list of solutions to **Integer**. Default is **10000000000**.
- **once(+Boolean)**
When **true**, commits to the first returned slice and reports **More = false**, even if **Goal** has further solutions. Default is **false**. This option is useful primarily with **limit/1**.
- **target(+Pid)**
Send the answer term to **Pid**. Default is the value of **target** when passed as an option to **toplevel_spawn/2**.

Variables in **Goal** are not bound directly in the caller. Instead, solutions and other kinds of output are returned in the form of answer messages delivered to the mailbox of the target process:

- **success(Pid, Data, More)**
Pid is the pid of the toplevel process that succeeded in finding solutions to **Goal**. **Data** is a list holding instantiations of **Template**. **More** is either **true** or **false**, indicating whether the toplevel can return more solutions if **toplevel_next/1-2** is called.
- **failure(Pid)**
Pid is the pid of the toplevel process that failed for lack of solutions.
- **error(Pid, Data)**
Pid is the pid of the toplevel throwing the error. **Data** is the error term.

Predicate: **toplevel_next/1-2**

ACTOR

toplevel_next(+Pid) is det.
toplevel_next(+Pid, +Options) is det.

Asks toplevel **Pid** for more solutions. Valid options:

- **limit(+Integer)**
If omitted, the actor API keeps using the most recent limit value from the previous toplevel call state.
- **target(+Pid)**
Send the answer term to **Pid**. Default is the value of **target** from the previous call state.

The messages delivered to the target mailbox are the same as for **toplevel_call/2-3**.

Predicate: **toplevel_stop/1**

ACTOR

toplevel_stop(+Pid) is det.

Asks toplevel Pid to stop searching for more solutions.

Predicate: `toplevel_abort/1`

ACTOR

`toplevel_abort(+Pid) is det.`

Tells the local toplevel Pid to abort the execution of the currently running goal. Abort runs supported cleanup, bypasses recovery in client `catch/3`, and returns a persistent session to PTCP state **s1**; it does not terminate the toplevel actor. An unknown or non-local pid has no effect.

Predicate: `toplevel_halt/1-2`

ACTOR

`toplevel_halt(+Pid) is det.`

`toplevel_halt(+Pid, -Reply) is det.`

Terminates toplevel actor Pid from any protocol state. The one-argument form requests termination through the actor runtime and returns without waiting for confirmation. The two-argument form installs a private monitor, requests termination, and waits for its notification. An observed termination yields `Reply = true`, regardless of its reason. The special reason `no_process` also yields `true`: the target is confirmed absent, without establishing when or why it terminated. For a remote target, `connection_closed` instead yields `Reply = error(connection_closed)`: observation was lost and termination is unconfirmed. Halt is an untrappable termination signal, not the interruption signal used by abort.

Note: The private monitor used by `toplevel_halt/2` is distinct from any existing monitor. If the toplevel was spawned with `monitor(true)`, the caller still receives its original `down(Pid, Pid, true)` notification.

B.3 Predicates for programming with server actors

Predicate: `server_spawn/3-4`

ACTOR

`server_spawn(+Pred, +State, -Pid) is det.`

`server_spawn(+Pred, +State, -Pid, +Options) is det.`

Spawns a generic server actor using callback predicate `Pred/4` and initial state `State`. The callback receives `(Request, OldState, Response, NewState)`. All options are forwarded to `spawn/3` except:

- `name(+Name)`
Register the server under `Name`.

Predicate: `server_request/3-4`

ACTOR

```
server_request(+To, +Request, -Response) is det.  
server_request(+To, +Request, -Response, +Options) is det.
```

Makes a synchronous request-response call to the server `To`, where `To` may be a pid or a registered local name. `server_request/4` accepts the options of `receive/2`, for example `timeout(+Seconds)`. Monitoring is installed automatically. If the server terminates before replying, the call promptly throws `server_down(Reason)`.

Predicate: `server_promise/3-4`

ACTOR

```
server_promise(+To, +Request, -Ref) is det.  
server_promise(+To, +Request, -Ref, -MonRef) is det.
```

Sends `Request` to the server `To` and returns a correlation reference `Ref`. The reply must later be collected with `server_yield/2-4`. The four-argument variant additionally installs a monitor and returns its reference in `MonRef`.

Predicate: `server_yield/2-4`

ACTOR

```
server_yield(+Ref, -Response) is det.  
server_yield(+Ref, -Response, +Options) is det.  
server_yield(+Ref, +MonRef, -Response, +Options) is det.
```

Waits for the response matching `Ref`. The three-argument variant accepts the options of `receive/2`. The four-argument variant uses `MonRef` to detect server termination, removes that monitor when the wait finishes normally or exceptionally, and throws `server_down(Reason)` if the server dies before replying.

Predicate: `server_upgrade/2-3`

ACTOR

```
server_upgrade(+To, +Pred) is det.  
server_upgrade(+To, +Pred, +Options) is det.
```

Replaces the callback predicate of the running server `To` without disturbing its current state. `Pred` must denote a callback predicate of arity four.

The two-argument form transfers no code and therefore requires the callback to be already available to the server process. The three-argument form accepts the following source-loading options:

- `src_text(+Text)`
Load callback source from `Text`.
- `src_list(+ListOfClauses)`
Convert the clauses to source text and load them into the server.

- `src_predicates(+ListOfPredicateIndicators)`
Copy the caller's indicated predicates into the server's private database.
- `src_uri(+URI)`
Fetch callback source from URI and load it into the server.

The operation replies only after the callback has been loaded and validated. It preserves the current state value and performs no state migration. If installation fails, `server_upgrade/2-3` rethrows the reported error and the state is unchanged. Source changes made in the server's private database before the error are not rolled back.

Predicate: `server_halt/2`

ACTOR

`server_halt(+To, -Reply)` is det.

Asks the server To to stop gracefully and binds Reply to its acknowledgement. The request carries a fresh correlation reference, so an unrelated acknowledgement in the caller's mailbox cannot satisfy the operation. A private monitor is installed before the request; if the server terminates without acknowledging it, the operation throws `server_down(Reason)`.

B.4 Predicates for programming with statechart actors

Predicate: `statechart_spawn/2`

ACTOR

`statechart_spawn(-Pid, +Options)` is det.

Spawns a statechart actor and binds Pid to its pid.

`statechart_spawn/2` requires exactly one source option:

- `src_uri(+URI)`
Load and run a statechart from a URI or file path.
- `src_text(+Text)`
Load and run a statechart from an XML text string.

In addition, `statechart_spawn/2` accepts:

- `name(+Name)`
Register the statechart actor under Name in the local ordinary-name namespace.
- `trace(+Mode)`
Set the per-actor trace mode to `true` or `false`. `true` emits trace events on the terminal-output path; `false` suppresses them. At most one `trace/1` option is accepted. Default is `true` for direct predicate calls; clients should pass the intended value explicitly.

All remaining options are passed through to `spawn/3`.

`src_list/1` and `src_predicates/1` are accepted as supplemental Prolog sources loaded into the statechart actor's private module. They do not count as the required XML source. The SXML document notation is specified in Appendix C.

Predicate: `statechart_halt/2-3`

ACTOR

`statechart_halt(+Pid, -Reply) is det.`
`statechart_halt(+Pid, -Reply, +Timeout) is det.`

Asks the statechart actor `Pid` to halt gracefully, running the exit actions of its active states, and waits for an acknowledgement in `Reply`. The two-argument form waits indefinitely.

The three-argument form waits at most `Timeout` seconds. If the actor does not reply in time, it is killed and `Reply` becomes killed.

Predicate: `raise/1`

STATECHART BUILTIN

`raise(+Event) is det.`

Enqueues `Event` on the *internal* event queue of the current statechart interpreter. It is handled after the current block of executable content and before a new external message.

Important: `raise/1` is only meaningful inside executable statechart content such as `<onentry>`, `<onexit>`, and `<go>`.

Predicate: `in/1`

STATECHART BUILTIN

`in(+Id) is semidet.`

Succeeds if the state identified by `Id` belongs to the current active configuration of the statechart interpreter.

Important: `in/1` is only meaningful inside statechart guards or executable statechart content such as `<onentry>`, `<onexit>`, and `<go>`.

B.5 Predicates for programming with supervisor actors

Predicate: `supervisor_spawn/2-3`

ACTOR

`supervisor_spawn(+ChildSpecs, -Pid) is det.`
`supervisor_spawn(+ChildSpecs, -Pid, +Options) is det.`

Spawns a supervisor actor and starts its children. Supported options are:

- **strategy(+Strategy)**
One of `one_for_one` (default), `one_for_all`, or `rest_for_one`.
- **intensity(+Integer)**
Maximum number of restarts permitted within one period. Default is 1.
- **period(+Integer)**
Restart-intensity window in seconds. Default is 5.
- **name(+Name)**
Register the supervisor under Name.

Other options are passed through to `spawn/3`.

Child specifications take the form `child(Id, ChildOptions)`. Child options are:

- **start(+Goal)**
Required. Start goal for the child. The special form
`server(Pred, ServerOptions)`
starts a generic server child. The default initial state is `[]`. It may be replaced with `initial_state(+State)`. All remaining `ServerOptions` are passed to `server_spawn/4`.
- **restart(+Policy)**
One of `permanent` (default), `transient`, or `temporary`.
- **shutdown(+Shutdown)**
One of `brutal_kill`, `infinity`, or a timeout in seconds. The default is 5 for workers and `infinity` for supervisor children.
- **type(+Type)**
`worker` (default) or `supervisor`.

Predicate: `supervisor_spawn_child/3`

ACTOR

`supervisor_spawn_child(+Pid, +ChildSpec, -Reply)` is det.

Dynamically adds and starts one child. On success, `Reply` is `ok`. If a child with the same `Id` is already present, it is `error(already_present)`. If startup fails, it is `error(start_failed)`.

Predicate: `supervisor_terminate_child/3`

ACTOR

`supervisor_terminate_child(+Pid, +Id, -Reply)` is det.

Stops child `Id` but keeps its specification. `Reply` is `ok` or `error(not_found)`.

Predicate: `supervisor_delete_child/3`

ACTOR

`supervisor_delete_child(+Pid, +Id, -Reply)` is det.

Removes a non-running child specification. Reply is `ok`, `error(running)`, or `error(not_found)`.

Predicate: `supervisor_respawn_child/3`

ACTOR

`supervisor_respawn_child(+Pid, +Id, -Reply)` is det.

Restarts a previously terminated child from its stored specification. Reply is either `ok(NewPid)`, `error(running)`, `error(not_found)`, or `error(start_failed)`.

Predicate: `supervisor_which_children/2`

ACTOR

`supervisor_which_children(+Pid, -Children)` is det.

Returns a list of:

`info(Id, Pid, Type, Restart)`

where `Pid` may be undefined for stopped children.

Predicate: `supervisor_count_children/2`

ACTOR

`supervisor_count_children(+Pid, -Counts)` is det.

Returns:

`[specs-N, active-N, supervisors-N, workers-N]`

Predicate: `supervisor_halt/1`

ACTOR

`supervisor_halt(+Pid)` is det.

Stops the supervisor and shuts down children in reverse start order.

Note on synchronous supervisor calls

The dynamic/query API above uses monitored synchronous calls and may throw:

- `supervisor_down(Reason)`
- `supervisor_call_timeout(Sup, Request)`

B.6 Prolog-style generics

Predicate: `parallel/1`

ACTOR

`parallel(+Goals) is semidet.`

Runs the proper list of independent goals `Goals` concurrently, using one monitored worker actor per goal. The call succeeds when every goal has produced its first solution and returns the resulting variable bindings. `parallel([])` succeeds immediately. Further solutions are not enumerated on backtracking.

If any goal fails, the remaining workers are terminated and the call fails. If any goal throws an exception, the remaining workers are terminated and the exception is rethrown. If several workers have abnormal outcomes, the first failure or exception notification observed by `parallel/1` determines the result; this selection may depend on scheduling and does not promise leftmost-goal precedence. Result and monitor messages belonging to the workers are removed before the predicate returns, fails, or throws.

The goals must not depend on bindings or side effects produced by sibling goals. Actor creation and message copying add overhead, and the one-worker-per-goal model gives large goal lists a correspondingly large resource footprint.

A non-list `Goals` raises `type_error(list, Goals)`. An element that is not callable raises the corresponding callable type error from its worker, which is propagated to the caller.

Predicate: `first_solution/2-3`

ACTOR

`first_solution(?Solution, +Goals) is semidet.`

`first_solution(?Solution, +Goals, +Options) is semidet.`

Runs the proper list of alternative goals `Goals` concurrently in monitored worker actors and unifies `Solution` with the result produced by the first solver to succeed. Once a winner has been selected, all remaining workers are terminated and their result and monitor messages are removed. The predicate does not enumerate later solutions on backtracking, and an empty goal list fails.

The two-argument form continues after an individual solver fails and stops on the first exception. The three-argument form accepts:

- `on_fail(stop|continue)`
`continue` removes the failed solver and keeps waiting while alternatives remain; `stop` terminates all workers and fails immediately. Default is `continue`.
- `on_error(stop|continue)`
`continue` removes the solver that raised an exception and keeps waiting while alternatives remain; `stop` terminates all workers and rethrows the exception. Default is `stop`.

If all alternatives are removed under a `continue` policy, the predicate fails. Goals should not depend on side effects produced by sibling solvers.

A non-list `Goals` or `Options` raises `type_error(list, Term)`. An action other than `stop` or `continue` raises `type_error(oneof([stop,continue]), Action)`. An unknown option raises `domain_error(first_solution_option, Option)`.

B.7 Predicates for remote Prolog-style calls

Predicate: `rpc/2-3`

ISOBASE

```
rpc(+URI, +Goal) is nondet.
rpc(+URI, +Goal, +Options) is nondet.
```

Executes a copy of `Goal` through the stateless `/call` endpoint of the node identified by `URI`, then unifies the local goal with each returned solution on backtracking. This remains a stateless call even when the destination is an `ACTOR` node: actor-only predicates such as `self/1`, `spawn/1-3`, and `receive/1-2` are not available through `rpc/2-3`.

The variable boundary across the three principal process-crossing operations can be summarised as follows:

Operation Variable semantics

<code>send</code>	A fresh message copy is delivered; sharing within the term is preserved, but no binding flows back and separate sends share no variables.
<code>spawn</code>	A renamed-apart goal runs in the child; child bindings do not bind variables in the parent.
<code>rpc</code>	A renamed-apart goal runs at the target; each returned solution is unified with the caller's original goal.

None of these operations creates a distributed shared variable. Local and remote placement obey the same portable copying rules.

The first argument alone selects the execution node. `localhost` means the node on which `rpc/2-3` is running. Source-loading options supply source for the call's temporary private database and do not change that target. For example:

```
rpc('http://n2.org', Goal, [
    src_uri('http://n3.org')
])
```

executes `Goal` on `n2.org` using source fetched from `n3.org`. All conforming native and browser runtimes accept:

- `limit(+Integer)`
Restricts the number of solutions retrieved per roundtrip. Default is `100000000000`.

- **once(+Boolean)**
When `true`, retrieves only the first slice and does not request later slices. Default is `false`.
- **timeout(+Number)**
Requests a timeout in seconds for goal execution on the destination node. The effective timeout is the minimum of this value and the node owner's configured timeout.
- **http_timeout(+Number)**
Sets the client-side HTTP transport timeout in seconds for each request. It does not change the destination node's execution timeout.
- **src_text(+AtomOrString)**
Loads source text into the temporary private database before calling `Goal`.
- **src_list(+ListOfClauses)**
Converts the clauses to source text and loads them into the temporary private database.
- **src_uri(+URI)**
Fetches source from URI and loads it into the temporary private database. This URI is a source location only.
- **src_predicates(+ListOfPredicateIndicators)**
Copies the caller's indicated predicates into the temporary private database.

Predicate: `promise/3-4`

ISOBASE

`promise(+URI, +Goal, -Ref)` is det.
`promise(+URI, +Goal, -Ref, +Options)` is det.

Starts one asynchronous stateless call to node `URI` with `Goal` and returns a local promise reference in `Ref`. As with `rpc/2-3`, the first argument selects the execution node and `localhost` denotes the current node. Source-loading options populate the promised call's temporary private database; they do not change its execution node. It accepts the same `limit/1`, `once/1`, `timeout/1`, `http_timeout/1`, and `src_*/1` options as `rpc/3`, together with:

- **template(+Template)**
`Template` is a term sharing variables with the goal. By default, the template is identical to the goal.
- **offset(+Integer)**
Selects the starting offset of the single solution slice collected by the promise. Default is `0`.

The reference returned in `Ref` can later be used by `yield/2-3` to collect the single answer term. Programs must not infer meaning from its digits or construct references themselves.

Predicate: `yield/2-3`

ISOBASE

`yield(+Ref, ?Answer)` is det.
`yield(+Ref, ?Answer, +Options)` is det.

Waits for the answer term from a previous call to `promise/3-4`. The reference is local to the runtime that created it. The answer is one of `success(Data,More)`, `failure`, or `error(Error)`, as returned by the stateless call endpoint. Valid options are:

- `timeout(+Number)`
Waits at most `Number` seconds for the promised answer. Default is no timeout.
- `on_timeout(+Goal)`
Calls `Goal` if the timeout expires before the answer is available.

A timeout does not cancel or consume the promise; a later call with the same reference may collect an answer that arrives afterward. If `on_timeout/1` is omitted, the timeout path succeeds.

Predicate: `runtime_property/1`

ISOBASE

`runtime_property(?Property) is nondet.`

Enumerates properties of the host runtime. This predicate reports host-runtime properties; it does not alter the Web Prolog profile or the semantics of the predicates above. Defined properties are:

- `implementation(swi_native|swi_wasm_worker|swi_wasm_main)`
- `persistent(+Boolean)`
- `inbound_addressable(+Boolean)`
- `dom(+Boolean)`
- `actor_isolation(native_thread|web_worker|cooperative_engine)`
- `hard_termination(+Boolean)`

Native nodes are persistent and inbound-addressable and use native threads with hard termination, but have no DOM. Web worker SWI-WASM actors have web worker isolation and hard termination but share the page's lifetime, are not inbound-addressable, and have no DOM. The optional main-thread SWI-WASM engine has DOM access but cooperative engine isolation and no hard termination; it too shares the page's lifetime and initiates rather than accepts remote connections.

B.8 Common error contract

Predicate entries above state specialised errors where they matter. Across the API, the common Prolog error vocabulary has the following roles:

Form	Meaning
<code>instantiation_error</code>	A required input was insufficiently instantiated.
<code>type_error/2</code>	The value has the wrong Prolog type; arguments identify the expected type and culprit.
<code>domain_error/2</code>	The type is acceptable but the value is outside the operation's domain.
<code>existence_error/2</code>	A required process, procedure, source, or other object does not exist.
<code>permission_error/3</code>	The operation is defined but not authorised in this execution context.
<code>resource_error/1</code>	The runtime cannot supply a required bounded resource.
<code>time_limit_exceeded</code>	A configured execution limit expired.

A direct predicate call throws the error term. PTCP reports a submitted-goal error as `error(Pid,E)`; stateless calls use `error(E)`; HTTP and WebSocket bindings place its public representation in a `type:"error"` envelope. An uncaught actor exception is observed by a monitor as `down(Pid,Ref,exception(E))`. Boundary sanitisation may remove host-private context but must preserve the portable formal error.

B.9 ISO predicates and related constructs

Web Prolog supports the following ISO predicates and related constructs from the appendix; directives and DCG rules are source-level items, and `read/1` and the write predicates are provided through the session I/O layer rather than ambient ISO stream I/O.

Control and meta-call

`!/0`, `(,)/2`, `(;)/2`, `(->)/2`, `(\+)/1`, `call/1--8`, `catch/3`, `fail/0`, `false/0`, `repeat/0`, `throw/1`, `true/0`

Directives and DCGs

`-->/2`, `discontiguous/1`, `dynamic/1`, `multifile/1`, `phrase/2--3`

Term relations, comparison, and construction

`(=)/2`, `(=..)/2`, `(==)/2`, `(@<)/2`, `(@=<)/2`, `(@>)/2`, `(@>=)/2`, `(\=)/2`, `(\==)/2`, `arg/3`, `compare/3`, `copy_term/2`, `functor/3`, `subsumes_term/2`, `term_variables/2`, `unify_with_occurs_check/2`

Arithmetic predicates

`(<)/2`, `(=:)/2`, `(=<)/2`, `(=\=)/2`, `(>)/2`, `(>=)/2`, `is/2`

Arithmetic evaluable functors

All arithmetic evaluable functors listed in the appendix are supported in arithmetic expressions.

Type testing

`acyclic_term/1`, `atom/1`, `atomic/1`, `callable/1`, `compound/1`,
`float/1`, `ground/1`, `integer/1`, `nonvar/1`, `number/1`, `var/1`

Database and all-solutions

`abolish/1`, `asserta/1`, `assertz/1`, `bagof/3`, `clause/2`, `findall/3`,
`retract/1`, `retractall/1`, `setof/3`

Atomic term processing

`atom_chars/2`, `atom_codes/2`, `atom_concat/3`, `atom_length/2`,
`char_code/2`, `number_chars/2`, `number_codes/2`, `sub_atom/5`

Session I/O

`nl/0`, `read/1`, `write/1`, `write_canonical/1`, `write_term/2`, `writeln/1`

Appendix C

SXML notation reference

This appendix is the reference manual for the XML notation accepted by the current Web Prolog statechart demonstrator. Chapter 6 introduces the ideas through examples; the present appendix instead proceeds element by element and states the operational rules on which documents may rely.

SXML is a design in progress. This appendix describes what the demonstrator implements at the time of writing.

C.1 A complete minimal document

The following document is a complete SXML statechart. It begins in `idle`, reports entry into that state, accepts a Prolog term as an event, and terminates by entering the top-level final state `finished`.

```
<statechart version="0.2" initial="idle">
  <datamodel>permitted(report).</datamodel>
  <state id="idle">
    <onentry>output(ready)</onentry>
    <go to="finished" on="start(Job)" if="permitted(Job)">
      output(starting(Job))
    </go>
  </state>
  <final id="finished"/>
</statechart>
```

The event is the Prolog term `start(Job)`. If the actor receives, for example, `start(report)`, unification binds `Job` to `report`; the same binding is visible in the guard specified by the attribute `if` and in the transition body. If `permitted(report)` succeeds, the action runs and the statechart enters `finished`. If matching or the guard fails, this transition is not enabled.

An application normally supplies the XML to `statechart_spawn/2`:

```
?- statechart_spawn(Pid, [
    src_uri('controller.xml'),
    monitor(true),
    trace(false)
]).
```

The document is supplied to `statechart_spawn/2`, whose options are specified in Appendix B.4.

C.2 Vocabulary at a glance

Element	Principal attributes	Purpose
<code><statechart></code>	version, initial, id	Document root and top-level compound state.
<code><state></code>	id, initial	Atomic or compound sequential state.
<code><parallel></code>	id	State whose child regions are simultaneously active.
<code><final></code>	id	Completion state for a compound state or for the whole statechart.
<code><history></code>	id, type	Shallow or deep remembered configuration.
<code><go></code>	to, on, if, after	Transition, optional guard, and optional Prolog action.
<code><onentry></code>	—	Prolog goal run when its parent state is entered.
<code><onexit></code>	—	Prolog goal run when its parent state is exited.
<code><datamodel></code>	—	Prolog clauses and directives loaded into the statechart's private module.
<code><defer></code>	on, if	State-scoped rule for postponing an otherwise unmatched event.
<code><spawn></code>	type, options, goal, callback, state, children	State-scoped invocation of an actor, toplevel, server, supervisor, or nested statechart.

Table C.1 SXML elements, their principal attributes, and their purposes.

Element and attribute names are case-sensitive. State identifiers are also case-sensitive: `Ready` and `ready` denote different states. Every `<state>`, `<parallel>`, `<final>`, and `<history>` element must have a document-wide unique value for the attribute `id`. Before constructing the model, the validator rejects missing or duplicate identifiers and elements placed outside the content model defined by the SXML 0.2 DTD. Section C.10 distinguishes DTD validation, cross-element semantic validation, and the further checks performed by the model parser.

C.3 XML and embedded Prolog

C.3.1 XML conventions

An SXML source is a well-formed XML document with a `<statechart>` element as its root. Ordinary XML escaping rules apply. In an attribute, a Prolog comparison such as `X < 10` must therefore be written as `X < 10`; an ampersand must be written as `&`. A large body of source that contains XML punctuation is often clearer in a CDATA section:

```
<onentry><![CDATA[
    X < 10,
    output(value(X))
]]></onentry>
```

Validation rejects unknown elements and attributes; authors and tools should use only the vocabulary in Section C.2.

C.3.2 Where Prolog occurs

SXML embeds Web Prolog in four distinct places:

- the value of the attribute `on` is a Prolog event pattern;
- the value of the attribute `if` is a Prolog goal used as a guard;
- the text in the `<onentry>`, `<onexit>`, and `<go>` elements is a Prolog goal used as executable content; and
- the text in the `<datamodel>` element is Prolog program source: clauses, directives, DCG rules, and declarations.

A block of executable content denotes one Prolog term. Several operations should be joined as a conjunction:

```
<onentry>
    retractall(count(_)),
    assertz(count(0)),
    output(ready)
</onentry>
```

The event pattern, guard, and action of one `<go>` element share variables by name. The guard is tried for its first solution. Failed matching or a failed guard leaves no logical bindings behind when the next transition candidate is tried. Guards should nevertheless be free of observable side effects: a database update, message, or output performed by a guard is not undone when that guard later fails.

Executable content is committed and is run once. Its alternatives are not exposed through backtracking. Failure raises the internal event `error(failure(Goal))`; an

exception `E` raises the internal event `error(E)`. A statechart may handle these terms with ordinary patterns in the attribute `on`. Database effects performed before failure are not rolled back.

C.4 Document root and states

C.4.1 The `<statechart>` element

```
<statechart version="0.2"
  id="machine"
  initial="start">
  ...
</statechart>
```

The `<statechart>` element is the document root. Its attributes are:

The `version` attribute (required)

The SXML notation version. The demonstrator accepts exactly `0.2`; a missing or different value is an error.

The `initial` attribute (required)

The identifier of the single top-level state entered at startup.

The `id` attribute (optional)

The identifier of the root state. If omitted, the implementation uses the internal identifier `statechart_actor`. Give the root an explicit identifier if an action or trace consumer needs to refer to it.

All embedded patterns, guards, actions, and data-model source are Web Prolog. The useful direct children are the `<datamodel>`, `<state>`, `<parallel>`, and `<final>` elements. The root participates in the state hierarchy, but normal documents do not attach transitions or invocation directly to it.

C.4.2 The `<state>` element

```
<state id="connected" initial="idle">
  <onentry>output(connected)</onentry>
  <state id="idle">...</state>
  <state id="busy">...</state>
  <go to="disconnected" on="close"/>
  <onexit>output(disconnected)</onexit>
</state>
```

A `<state>` element is *atomic* when it contains no state children and *compound* when it contains a nested control structure. When a compound state is active, one of its

sequential children is active as well. Outgoing transitions and deferral declarations on the compound state are inherited by active descendants: selection searches from the innermost active state outwards.

The attribute `id` is required. A compound state must also have the attribute `initial`, whose value identifies its default child.

Useful children are the `<onentry>`, `<onexit>`, `<go>`, `<defer>`, `<spawn>`, `<datamodel>`, `<history>`, `<state>`, `<parallel>`, and `<final>` elements. Their XML order also establishes document order for entry, exit, and transition priority.

C.4.3 The `<parallel>` element

```
<parallel id="controller">
  <state id="connection" initial="offline">...</state>
  <state id="work" initial="idle">...</state>
</parallel>
```

A `<parallel>` element makes all its child regions active. The regions do not run on separate threads. They form one configuration inside one actor, and a selected event is visible to each active region under a single run-to-completion scheduler. Compatible transitions in different regions may therefore be selected in the same microstep. Section C.5.1 specifies transition priority and conflict resolution.

The attribute `id` is required. A parallel state does not have the attribute `initial`: entry activates each child and recursively enters that child's explicit initial state where necessary. It may have `<onentry>`, `<onexit>`, `<go>`, `<defer>`, `<spawn>`, `<datamodel>`, `<history>`, `<state>`, `<parallel>`, and `<final>` child elements.

C.4.4 The `<final>` element and completion

```
<state id="processing" initial="request">
  <state id="request" initial="working">
    <state id="working">
      <go to="complete" on="reply(_)" />
    </state>
    <final id="complete" />
  </state>
  <go to="all_done" on="done(request)" />
</state>
<final id="all_done" />
```

Place this fragment inside the document root and set its `initial` attribute to `processing`. The enclosing state owns the completion transition. The target `all_done` is a top-level final state.

The attribute `id` is required. A final state may contain a `<onentry>` element. Entering a final child of a compound state with identifier `P` raises the internal Prolog event `done(P)`. When every region of a parallel state `Q` has reached a final configuration, the runtime raises `done(Q)`.

Entering a top-level final state terminates the statechart normally. Active states are exited in order, the executable content in their `<onexit>` elements runs, and invoked children owned by those states are stopped. A monitor on the statechart then receives `down(Pid,Ref,true)`. Completion data must be sent explicitly as an actor message before the statechart terminates.

C.4.5 The `<history>` element

```
<state id="work" initial="first">
  <history id="resume" type="deep">
    <go to="first"/>
  </history>
  ...
</state>
```

A history pseudostate records the active configuration below its parent when that parent is exited. A transition whose attribute `to` names the history identifier restores the recorded configuration. When the attribute `type` has the value `shallow` (the default), the statechart remembers immediate active children; with the value `deep`, it remembers the complete active atomic descendants.

The attribute `id` is required. Supply one eventless `<go>` child element as the fallback target used before any history has been recorded. The fallback is structural in the current implementation; put effects in the destination's `<onentry>` element, not in the fallback `<go>` element's body.

C.5 Transitions: the `<go>` element

```
<go to="running" on="start(Job)" if="valid(Job)">
  output(started(Job))
</go>
```

A `<go>` element may have the following attributes:

The `on` attribute

A single Prolog event pattern. It is matched by unification. Variables bound by matching are visible in the guard and action.

The `if` attribute

A Web Prolog guard. It defaults to `true`. The first solution is used. A guard without the attribute `on` makes an eventless transition.

The to attribute

One or more state identifiers separated by XML whitespace. Omit it for a targetless transition. Multiple targets are intended for a legal orthogonal-state configuration; most transitions have one target.

The after attribute

A finite, non-negative delay in seconds. Integer and fractional values are accepted. The attributes **after** and **on** are mutually exclusive. A guard supplied with the attribute **if** on a timed transition is tested when the timer fires.

At least one of the attributes **on**, **if**, **to**, or **after** should be present. The parser currently accepts an entirely empty <go> element, but it is an unconditionally enabled, eventless, targetless transition and normally creates an infinite internal cascade.

Table C.2 summarises the common forms.

Transition form	Typical use / notes
<code><go to="s" on="e">...</go></code>	Triggered state change. The ordinary event-driven case: on event pattern <i>e</i> , transition to <i>s</i> and run actions.
<code><go to="s" on="e" if="c">...</go></code>	Triggered and guarded state change. The standard ECA pattern: a matching event plus a condition enables the transition; convenient when several transitions compete on the same event.
<code><go to="s" after="n">...</go></code>	State-scoped timed transition. Arms on entry to the source state and is cancelled automatically on exit; <i>n</i> is a non-negative number of seconds.
<code><go on="e" if="c">...</go></code>	Guarded, targetless event reaction. Consumes events matching <i>e</i> , checks <i>c</i> , runs actions, and stays in the same configuration. Useful for logging, emitting output, raising internal events, or updating the data model without a state change.
<code><go to="s">...</go></code>	Eventless, unconditional state change. Useful for automatic progress (for example, completion-style transitions) or compact control-flow steps.
<code><go to="s" if="c">...</go></code>	Eventless but guarded state change. Useful for decision points driven by the data model, or for encoding automatic progress only when a condition holds.
<code><go if="c">...</go></code>	Guarded, targetless reaction (no control-state change). Useful as a condition-action rule that updates the data model, emits output, raises an internal event, logs, etc., while staying in the same configuration.

Table C.2 Common forms of the <go> element and their typical uses.

Targetless and self-targeted transitions differ. A targetless transition runs its body without exiting or entering any state. A transition whose target is its own source

performs an external self-transition: exit handlers run, timers and invoked children are cancelled, and the state is entered again.

C.5.1 Selection and execution order

For each active atomic state, candidates are considered from that state outward through its ancestors. The first matching and successful candidate in document order is selected for that region. Duplicate selections are removed. If two remaining transitions have overlapping exit sets, the transition whose source is the descendant pre-empt the transition from its ancestor. Compatible transitions selected in parallel regions may run together; document order resolves any remaining choice.

One microstep has three ordered phases:

1. exit affected active states from the inside out, running the `<onexit>` elements;
2. execute selected transition bodies in document order; and
3. enter target states from the outside in, running the `<onentry>` elements.

Eventless transitions and internal events are drained before another actor message is received. This complete reaction to one external event is a macrostep. Its atomicity means that no newer external event is selected in the middle; it does not make Prolog database effects transactional.

C.5.2 Timed transitions

A timer is armed when its source state is entered and cancelled before that state's exit actions run. Re-entry creates a fresh activation. A private activation reference prevents a late firing from an old activation from triggering the timer of a newer one. A timeout does not interrupt a Prolog action that is already running; it becomes eligible only when the current work yields to event processing.

C.6 Entry, exit, and the Prolog data model

C.6.1 The `<onentry>` and `<onexit>` elements

```
<state id="open">
  <onentry>output(opened)</onentry>
  ...
  <onexit>output(closed)</onexit>
</state>
```

The text of each element is one Web Prolog goal. There may be more than one entry or exit element; they run in document order. On entry, the state is already in the active configuration, so `in(Id)` can observe it. On exit, timers have first been invalidated, but the state remains in the active configuration while its exit action runs. Invoked children are stopped after the state's exit actions.

Actions should terminate promptly. A blocking `receive/1`, a long `sleep/1`, or a synchronous operation prevents the macrostep from stabilising and prevents the actor from selecting further external events. Use asynchronous actor interaction and handle the reply as a later event.

C.6.2 The `<datamodel>` element

```
<datamodel>
  :- dynamic count/1.
  count(0).

  permitted(Job) :- policy_allows(Job).
</datamodel>
```

The `<datamodel>` element contains ordinary Prolog source. It is loaded before the initial configuration is entered. The source may define facts, rules, DCGs, dynamic declarations, and other directives allowed by the active node policy. Code supplied through the statechart actor's `src_list/1` and `src_predicates/1` options is loaded into the same private execution module.

The statechart's own predicates shadow predicates with the same name in the node's shared database; otherwise an action may call shared predicates offered by the node. Dynamic updates belong to the statechart actor's private database and are sequentially isolated from other actors. A public node may sandbox embedded goals and reject predicates outside its advertised profile.

Although the parser accepts the `<datamodel>` element below compound states, all such source is loaded at statechart startup; it is not installed and removed on state entry and exit. Prefer one document-level element unless locality in the XML source materially improves readability.

C.6.3 Statechart built-ins

The statechart built-ins `raise/1` and `in/1` are specified in Appendix B.4. The latter is particularly useful in guards that depend on another parallel region.

Ordinary actor and I/O predicates remain the communication layer. For example, `send(Pid, Term)` or `Pid ! Term` sends an event to an actor, while `output(Term)` sends actor output along the configured I/O path. Communication and logging are thus expressed with Web Prolog predicates.

C.7 Deferred events: the `<defer>` element

```
<state id="draining">
  <defer on="job(J)" if="deferrable(J)" />
  <go to="idle" on="drained" />
</state>
```

The attribute `on` is required and contains one Prolog pattern. The optional attribute `if` contains a guard and defaults to `true`; variables from the pattern are visible in the guard.

Deferral is considered only if no transition is enabled anywhere in the active configuration. An enabled transition always wins. The runtime then searches active states from the inside out, and one matching declaration in any active region is sufficient to append the event to the postponed queue. Timed-transition implementation events are never deferrable.

After a macrostep changes the active configuration, postponed events are re-offered in original arrival order before a newer external message is selected. An event can be deferred again if the new configuration still declares it; otherwise it is handled or discarded. A configuration change is therefore essential: a targetless action alone does not cause the postponed queue to be re-offered. The queue has no implicit application-level bound, so broad patterns in a long-lived state require explicit resource discipline.

C.8 Actor invocation: the `<spawn>` element

The `<spawn>` element belongs to the state that contains it. Creation is delayed until the macrostep in which that state was entered has stabilised. A state that is entered and exited within the same macrostep never creates its child. After successful creation the interpreter raises `spawned(Pid)` on its internal event queue. This is a notification from the interpreter, not a message sent by the child.

The child remains owned by the invoking state. Exiting that state stops the child after the state's `<onexit>` content. The lifetime rule applies to all five types of the `<spawn>` element and is independent of whether `link(true)` or `monitor(true)` was also requested.

C.8.1 Common syntax and value rules

```
<spawn type="server"
  callback="counter"
  state="0"
  options="[name(counter_server), monitor(true), link(false)]">
  counter(inc, N, N1, N1) :- N1 is N + 1.
</spawn>
```

The attribute `type` is required and selects one of the five invocation contracts below. The required invocation operands are given by direct attributes such as `goal`, `callback`, `state`, and `children`. Every optional setting is a member of the Prolog list in the attribute `options`; optional settings are not accepted as direct XML attributes.

Values of required operand attributes and members of the list in the attribute `options` are parsed as Prolog terms and must be ground. The attribute `options` must itself contain a ground Prolog list. Use Prolog quoting where a value would otherwise be parsed incorrectly, for example `options="[src_uri('child.xml')]"`. Non-source option names may occur only once in the list.

Nonempty body text becomes one `src_text/1` source fragment. It is preserved as source text rather than being parsed and printed again, so clause layout and operators survive. For ordinary Prolog children it is a convenient place for callback or worker clauses. Multiple source fragments are allowed for those children and are loaded in their declared order, with the inline body first.

C.8.2 Option matrix

Type	Invocation contract
actor	Required: <code>goal(Callable)</code> via the <code>goal</code> attribute. Type-specific option: <code>target/1</code> .
toplevel	No required operand. Type-specific options: <code>session/1</code> , <code>time_limit/1</code> , <code>idle_limit/1</code> , <code>target/1</code> , and <code>name/1</code> .
server	Required: <code>callback(Callable)</code> and <code>state(Term)</code> via attributes. Type-specific option: <code>name/1</code> .
supervisor	Required: <code>children(List)</code> via the <code>children</code> attribute. Type-specific options: <code>strategy/1</code> , <code>intensity/1</code> , <code>period/1</code> , and <code>name/1</code> .
statechart	Required: exactly one XML <code>src_text/1</code> or <code>src_uri/1</code> source. Type-specific options: <code>name/1</code> and <code>trace/1</code> .

Table C.3 Invocation contracts for the five types of the <spawn> element.

All five types accept `monitor/1`, `link/1`, `node/1`, and `io_target/1`. The first two take `true` or `false`. `io_target` chooses where the child's terminal and `output/1` traffic is delivered. An actor or `toplevel` `target` chooses its protocol/result recipient; it is not a synonym for the I/O target. For a `toplevel`, an explicit `target` option replaces the containing statechart as the recipient of query results, while the containing statechart still receives its own `spawned(Pid)` event.

Actor, `toplevel`, `server`, and `supervisor` children accept `src_text/1`, `src_uri/1`, `src_list/1`, and `src_predicates/1` as Prolog sources.

For `time_limit` and `idle_limit`, a value is either `infinite` or a positive number of seconds. Supervisor `strategy`, `intensity`, and `period` have the same meanings as in the supervisor API; strategies are `one_for_one`, `one_for_all`, and

`rest_for_one`. A `target` is supported only for actor and toplevel children; the option `trace/1` is supported only for a nested statechart.

C.8.3 Actor, toplevel, server, and supervisor examples

An actor `<spawn>` element supplies its initial goal and may define that goal in the body:

```
<spawn type="actor" goal="worker" options="[monitor(true)]">
  worker :-
    receive({stop -> true;
             Job  -> perform(Job), worker}).
</spawn>
```

A toplevel body becomes its private program. Once `spawned(Pid)` is handled, an action may initiate a query with `toplevel_call/2-3`; successive `success/3` events then arrive through the actor protocol.

The server example at the start of this section uses a `<spawn>` element to create a server whose callback is `counter/4` and whose initial state is `0`. The registered name `counter_server` lets statechart actions address the child without retaining its pid. A supervisor `<spawn>` element instead places a ground list of ordinary supervisor child specifications in the attribute `children`:

```
<spawn type="supervisor"
  children="[child(worker,
                  [start(sleep(60)), restart(temporary)])]"
  options="[strategy(one_for_all), intensity(2), period(5)]"/>
```

The child specification and policy are passed to the supervisor behaviour. The statechart merely owns the supervisor actor and observes `spawned(Pid)`; supervision events and calls use the ordinary supervisor API.

C.8.4 Nested statecharts

A nested statechart needs exactly one XML source. A `src_uri` can name an external document. Inline XML must use CDATA because it is source for the child, not structural XML belonging to the parent document's `<spawn>` element:

```
<spawn type="statechart"
  options="[monitor(true),
           name(inner_chart),
           trace(false),
           src_list([child_value(ok)])]"><![CDATA[
```

```

    <statechart version="0.2" initial="waiting">
      <state id="waiting">
        <onentry>
          child_value(Value),
          output(nested(Value))
        </onentry>
        <go to="finished" on="finish"/>
      </state>
      <final id="finished"/>
    </statechart>
  ]]></spawn>

```

The CDATA body supplies the nested statechart's XML through `src_text/1`. The option `src_list/1` supplies the fact `child_value(ok)` as supplemental Prolog source for the nested statechart's private module. Selected predicates may instead be copied with `src_predicates/1`. Neither is a second XML source. Supplying zero or more than one of `src_text/1` and `src_uri/1` is an error.

Unless `io_target` says otherwise, the nested statechart's `output(nested(ok))` follows its inherited I/O path and is observable by the outer statechart as the actor event `output(ChildPid,nested(ok))`. The outer statechart must include a matching `<go>` element if it wants to act on or print that value; output is not automatically converted into a state change. The option `trace(false)` suppresses only the nested statechart's trace. It does not change the trace setting of the containing statechart.

Normal completion of a monitored nested statechart is reported by the canonical monitor event `down(Pid,Ref,true)`. The outer statechart may match `down/3`, send its own protocol messages, or use the registered child name as appropriate.

C.9 Run-to-completion and event queues

The statechart actor consumes external actor messages in arrival order. Once one message has been selected, its pattern is tested against enabled transitions across the active configuration. An unmatched event is discarded unless a `<defer>` element postpones it; unlike selective `receive/1-2`, it does not remain in the actor mailbox implicitly.

The interpreter gives work the following broad priority:

1. enabled eventless transitions;
2. events already in the internal queue, including events placed there by `raise/1`, completion and error events (`done/1` and `error/1`), `spawned/1` events, and valid timer firings;
3. postponed events re-offered after a configuration change;
4. invocation requests belonging to newly stable states; and
5. the next external actor message.

This priority makes one external reaction deterministic given the active configuration, private database, and event, but it also makes termination of internal cascades an author responsibility. An eventless rule that remains true after its action, or two `raise/1` handlers that regenerate one another, can livelock the statechart. While a macrostep is stuck, even an orderly `statechart_halt/2` request cannot be selected; use the bounded `statechart_halt/3` when forced escalation is required.

C.10 Validation, tracing, and diagnosis

Before constructing a model, the demonstrator validates every document against the versioned SXML 0.2 DTD. The implementation supplies the DTD to the XML parser, so an SXML document neither needs nor should repeat a `DOCTYPE` declaration. A running node also serves the distributable grammar at `/sxml-0.2.dtd` for editors and external validators. The DTD checks the permitted elements, their placement and textual content, and the permitted attributes. Unknown or misplaced elements and unknown attributes are errors rather than constructs that the model builder silently ignores.

DTD validation is followed by a side-effect-free semantic pass. It checks that identifiers are unique, that the attributes `initial` and `to` name existing identifiers, and that an initial state is a direct child of the element that selects it. The model parser then checks the parts of SXML that an XML grammar cannot express: Web Prolog syntax, the combination of the attributes `on` and `after`, delay values, and the type-dependent operands and options of the `<spawn>` element. Thus the DTD is the XML grammar of SXML, not the whole statechart validity contract.

Set `trace(true)` or `trace(false)` explicitly when spawning a statechart if observable tracing matters. With tracing enabled, the actor emits terms of the form `statechart_trace(TraceEvent)` on its terminal-output path. Current trace events include:

- `external_event(Event)` and `internal_event(Event)`;
- `unmatched(Event)`;
- `deferred(Event)` and `reoffered(Events)`;
- `transition(Source,Targets,ExitSet,EntrySet)`;
- `microstep(ExitSet,EntrySet)` and `configuration(States)`;
- `execution(Goal)`; and
- `invoked(Type,Pid,OwningState)`.

Trace terms expose implementation-level ordering and are chiefly diagnostic; application logic should react to its protocol events, not depend on trace delivery. For a nested `<spawn type="statechart">` element, its `trace/1` option, with value `true` or `false`, controls that statechart independently.

Spawn a statechart with `monitor(true)` while developing it. Malformed XML, missing initial states, rejected spawn options, and uncaught startup errors then appear in the monitor's `down/3` reason instead of looking like a statechart that simply failed to react. Validation detects vocabulary and cross-reference errors before the model is built; tracing the initial configuration remains useful for diagnosing the behaviour of a valid model.

C.11 Relationship to SCXML 1.0

SXML retains SCXML's central statechart ideas: atomic and compound states, parallel regions, initial and final states, shallow and deep history, configuration-wide transition selection, inside-out exit, outside-in entry, and run-to-completion processing. It then instantiates those ideas in the Web Prolog actor model. SCXML 1.0 is defined by the W3C Recommendation (Barnett et al., 2015), available at <https://www.w3.org/TR/scxml/>. SXML is not a W3C conformance profile. The following table is a translation guide, not a claim that arbitrary SCXML can be converted mechanically.

SCXML 1.0	SXML	Principal difference
<scxml>	<statechart>	No SCXML namespace or binding attributes; the SXML version is 0.2.
<transition>	<go>	Prolog-oriented spelling; no current equivalent of the attribute value <code>type="internal"</code> .
The attribute <code>event</code>	The attribute <code>on</code>	One Prolog pattern matched by unification, not a list of dotted event descriptors.
The attribute <code>cond</code>	The attribute <code>if</code>	Web Prolog goal whose first solution enables the transition.
The attribute <code>target</code>	The attribute <code>to</code>	Space-separated state identifiers; targetless semantics are retained.
The <datamodel>, <data>, and <assign> elements	The <datamodel> element containing Prolog source	Private clauses and ordinary Prolog database operations replace named data objects and XML assignment.
XML executable elements	Prolog text	Sequencing, conditionals, iteration, logging, and communication use Prolog predicates and control constructs.
The <raise> element	<code>raise(Event)</code>	Internal events are Prolog terms.
The <send> and <cancel> elements	Actor predicates	Communication uses Web Prolog pids, names, messages, delayed sends, and cancellation.
The <invoke> element	The <spawn> element	Invokes actor behaviours with state-scoped lifetime.
The delayed <send> element	The attribute <code>after</code> of the <go> element	Direct state-scoped timer with automatic cancellation on exit.
No corresponding element	The <defer> element	Explicit state-scoped postponement of unmatched Prolog event patterns.
<code>done.state.id</code>	<code>done(Id)</code>	Completion is an internal Prolog term.
<code>done.invoke.id</code> and the <donedata> element	explicit messages and <code>down/3</code>	No automatic invocation-completion data protocol.
<code>_event</code> system record	pattern bindings	Event structure is the term itself; matching exposes its components directly.

Table C.4 Correspondence between SCXML 1.0 and SXML.

The SCXML <data>, <assign>, <donedata>, <content>, <param>, <script>, <raise>, <if>, <elseif>, <else>, <foreach>, <log>, <send>, <cancel>, <invoke>, and <finalize> elements, and their associated expression attributes, are therefore not SXML elements. Their useful effects are normally expressible

in embedded Web Prolog, but their SCXML syntax and service protocols are not accepted.

There are also three current structural restrictions to remember when adapting an SCXML document:

- SXML requires explicit root and compound-state initial targets; it does not choose the first child by default.
- The SCXML `<initial>` element is not an SXML element; use the attribute `initial` on the containing state instead.
- SXML has external transition behaviour and targetless transitions, but no transition mode corresponding to the attribute value `type="internal"`.

These differences are intentional where they follow from Prolog terms and actors, and provisional where they reflect the present demonstrator. Code that must be portable to a conforming SCXML processor should be authored as SCXML, not as SXML; code that targets the Web Prolog statechart behaviour should follow this appendix rather than assuming that visually similar SCXML markup has been implemented.

C.12 Authoring checklist

Before running a new statechart, check the following:

1. The document is well-formed XML that validates as SXML 0.2, with one `<statechart>` root element whose attribute `version` is `0.2`.
2. The root element has an explicit attribute `initial`; every compound `<state>` element also has that attribute.
3. The identifiers of all `<state>`, `<parallel>`, `<final>`, and `<history>` elements are present and unique.
4. Every target named by an attribute `to` exists, and a multiple-target transition names a legal orthogonal configuration.
5. Each value of an attribute `on` or `if`, each action, and each `<spawn>` element's Prolog-valued attribute is valid Web Prolog after XML entity decoding.
6. Guards are side-effect free and all eventless/internal cascades are guaranteed to terminate.
7. A timed transition does not have both the attributes `after` and `on`.
8. Deferred-event patterns are narrow enough that their queue remains bounded by application behaviour.
9. Each `<spawn>` element has the required attribute `type`, the required operands, ground options, and no duplicate non-source option names.
10. A nested statechart has exactly one XML source and uses `src_list` or `src_predicates` only for supplemental Prolog.
11. Development runs use `monitor(true)` and an intentional trace setting so startup and selection errors are visible.

The demonstrator examples provide progressively larger executable documents, including history, parallel regions, timed and deferred transitions, the private and shared Prolog data model, and all five types of the `<spawn>` element. They are useful as runnable companions to this reference, but the contracts in this appendix take precedence over incidental details of any one example.

Appendix D

Node protocol reference

This appendix describes the public node protocols implemented by the current demonstrator. It complements the predicate reference in Appendix B: that appendix specifies the Prolog interface seen by a program, whereas this one specifies the HTTP and WebSocket interfaces seen by a client of a node.

The protocols remain a design in progress. The descriptions below distinguish the canonical interface from compatibility forms that the implementation may continue to accept. Clients should generate only the canonical forms.

D.1 Profiles and protocol surfaces

The node profiles are cumulative interaction contracts, not alternative names for transports. RELATION and ISOBASE use the same stateless endpoint but admit different classes of query. ISOTOPE adds a persistent toplevel session, and ACTOR adds general actor creation and messaging.

Profile	Interaction contract	Public surface
RELATION	Queries over advertised relations	HTTP /call
ISOBASE	Stateless Prolog execution and RPC	HTTP /call
ISOTOPE	Persistent toplevel sessions	HTTP /toplevel_*, plus lower surfaces
ACTOR	Actors, messaging, and server push	WebSocket /ws, plus lower surfaces

RELATION accepts only query patterns allowed by the relations that the node advertises. ISOBASE admits general goals and temporary source loading, subject to the node's sandbox and resource policy. Both remain stateless from the client's perspective: any dynamic update belongs to the temporary execution context and does not establish a session.

ISOTOPE associates requests with a long-lived toplevel actor. Its private database and session I/O therefore survive from one call to the next. ACTOR adds bare actors, mailboxes, monitors, and asynchronous events over a persistent full-duplex connection. Authentication and authorisation may control who may use a surface, but do not change the semantics of the profile once access has been granted.

D.2 Common conventions

D.2.1 Versioning

The current WebSocket and cross-node wire vocabulary has protocol version 1. A node reports the version, together with implementation and build information, at `/version`. An incompatible wire change requires a new version. Additive response fields may be ignored by clients that do not recognise them.

D.2.2 Terms, pids, and formats

Goals, templates, options, messages, input, monitor references, and exit reasons are Prolog terms represented as text wherever a table below says *term text*. Implementations must parse a goal together with any template that shares its variables; parsing them independently would lose variable identity.

A pid has the canonical global form `Id@Node`, where `Id` is an actor identifier assigned by `Node`. This representation is part of the protocol contract: clients may copy, construct, decompose, compare, and serialise it. In the demonstrator, a local numeric pid is represented in JSON as a JSON number, while a pid qualified with another node is represented by its `Id@Node` text. Constructing such a value does not create an actor, confirm that it is live, or grant access to it. Resolution and use remain subject to actor existence and node authorisation; arithmetic on a numeric identifier has no defined actor-level meaning.

HTTP execution endpoints accept `format=json` or `format=prolog`. JSON is the default. The Prolog form returns one answer term as plain text. In JSON, the mandatory `type` field selects the answer or event form. JSON in this specification means the data-interchange format defined by RFC 8259, whose registered media type is `application/json`.¹

A JSON response or WebSocket frame is a protocol *envelope*. Control fields such as `type`, `command`, `more`, and `pid` use the JSON forms specified in this appendix. When a field is described as *term text*, however, its JSON value is a string containing Prolog source text; parsing the surrounding JSON does not decode that string as a

¹ T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, <https://www.rfc-editor.org/rfc/rfc8259.html>.

Prolog term. Conversely, structured solution data is mapped to JSON as specified for the corresponding endpoint. The use of a JSON envelope must therefore not be read as a claim that every Prolog term has an automatic or lossless representation as a JSON value.

In this appendix, an *event* is a node-to-client protocol item selected by its *type* field. It may be carried in an HTTP response or a WebSocket server frame. Actor application payloads are called *messages*; *reply* is reserved for a correlated request-reply interaction.

D.2.3 Compression, binary formats, and transport protection

Representation, compression, and transport protection are separate layers. HTTP peers may negotiate a content coding such as *gzip* by means of *Accept-Encoding* and *Content-Encoding*; this compresses the HTTP representation without changing whether its underlying media type is JSON or Prolog text.² A WebSocket connection may likewise negotiate a compression extension during its opening handshake; the standard *permessage-deflate* extension is defined by RFC 7692.³ Neither form of compression changes the Web Prolog protocol vocabulary, and neither is required by protocol version 1 unless a deployment profile says otherwise.

Protocol version 1 defines JSON envelopes and Prolog term text; it does not define a binary term representation. Formats such as Erlang’s External Term Format or UBF are relevant precedents, but adopting either would require an explicit mapping for Prolog values and semantics rather than simply reusing its existing encoding.⁴ Any future binary representation must be identified by a negotiated media type or protocol capability and must specify terms, variables, sharing, cycles, errors, and versioning. Sending a binary WebSocket frame alone does not constitute such negotiation.

HTTPS and secure WebSocket connections use TLS to protect bytes in transit. TLS supplies confidentiality and integrity and normally authenticates the server; client authentication depends on the deployment. It does not select a term representation, compress the application data, or replace Web Prolog’s application-level authentication and authorisation rules.⁵

² See RFC 9110, Sections 8.4 and 12.5.3, <https://www.rfc-editor.org/rfc/rfc9110.html>.

³ <https://www.rfc-editor.org/rfc/rfc7692.html>.

⁴ For the External Term Format, see https://www.erlang.org/doc/apps/erts/erl_ext_dist.html; for UBF, see <https://ubf.github.io/ubf/>.

⁵ TLS 1.3 is specified by RFC 8446, <https://www.rfc-editor.org/rfc/rfc8446.html>.

D.2.4 Common answer and event forms

type	Additional fields	Meaning
success	data, more; optionally pid	A slice of solutions; more says whether another slice may exist.
failure	optionally pid	The goal has no solution, or no further solution.
error	data; optionally pid, details	Execution or protocol error.
spawned	pid	An actor or session was created.
output	pid, data	Session or actor output.
prompt	pid, data	The session is waiting for input.
timeout	pid	A blocking wait reached its timeout.
stop	pid	The current solution sequence was stopped.
abort	pid	The current computation was aborted.
responded	pid	Input or a trace setting was accepted.
halted	pid, reply	A toplevel actor was terminated.
down	pid, ref, reason	A monitored actor terminated, or remote observation was lost.
statechart_trace	pid, data	Statechart trace output.

A stateless response has no pid. Session and actor events normally do. In JSON success responses, data is a list. With the default JSON query format, each element is an object containing the named variables of one solution. With Prolog formatting, the corresponding terms are `success(Data,More)` for stateless calls and `success(Pid,Data,More)` for session calls.

Output and prompts may precede the final success, failure, or error event. A timeout at a polling or receive boundary ends that wait; it does not by itself mean that the session has been halted.

A PTCP running-limit expiry is instead an error event. Its Prolog error term is `time_limit_exceeded`; the session actor remains alive and returns to its idle state. In a JSON error event, data is the concise diagnostic intended for ordinary display. An implementation may additionally provide details, containing a display form of the Prolog exception term with implementation-supplied context elided. Both fields are presentation text, and clients must not parse them as a stable error-classification interface; in particular, details is not a claim about the exact runtime term. For a

toplevel error, Prolog-facing PTCP messages and responses using `format=prolog` retain the actual caught exception term. Monitor notifications instead use the boundary form described below. Implementations may further redact paths, goal arguments, or other sensitive information from client-facing diagnostics.

D.3 The toplevel communication protocol

PTCP is the logical protocol spoken by a toplevel actor. It is independent of whether commands arrive as actor messages, HTTP requests, or WebSocket frames. The HTTP operation `poll` is not a PTCP command: it merely retrieves an event already waiting in a transport queue.

Operation Purpose		Principal outcomes
<code>call</code>	Submit a goal and request its first solution slice	success, failure, error
<code>next</code>	Continue a suspended solution sequence	success, failure, error
<code>stop</code>	Abandon a suspended solution sequence	stop
<code>abort</code>	Interrupt the current computation and return a persistent session to its idle state	abort
<code>halt</code>	Terminate the toplevel actor from any protocol state	halted, and a down event for each monitor
<code>respond</code>	Supply input requested by a prompt	responded

The distinction between `stop`, `abort`, and `halt` is semantic. `stop` applies while the actor is waiting between solution slices. `abort` interrupts a running goal but preserves a persistent session. `halt` crosses the final lifecycle boundary and terminates the actor. Abort and halt are runtime signals, not PTCP mailbox messages. Abort bypasses application recovery while running supported cleanup and is consumed by the privileged PTCP boundary; halt is untrappable and terminates the actor. Section 4.1.7 gives the normative signal semantics.

For a remotely linked actor, the child-hosting controller maintains the lifetime relation. Known parent termination is propagated explicitly. The current demonstrator also scopes remotely hosted actors to the authenticated WebSocket that authorised their creation and tears down the connection-owned subtree when that transport reports close or error. Several logical links may share the connection, so this containment carrier does not replace explicit per-parent propagation. No bounded detection guarantee is made for a silent partition before the transport declares the connection lost.

D.4 The stateless HTTP binding

RELATION and ISOBASE expose GET /call. The only required parameter is goal.

Parameter	Meaning	Default
goal	Goal as Prolog term text	required
template	Answer template sharing variables with the goal	the goal
offset	Zero-based offset in the solution sequence	0
limit	Maximum number of solutions in the returned slice	effectively unlimited
once	Commit to the returned slice even if further solutions exist	false
src_text	Source loaded into the temporary execution context	empty
timeout	Requested execution timeout, capped by node policy	node policy
format	json or prolog	json

On a RELATION node, goal must match an advertised relation pattern and src_text is forbidden. ISOBASE permits general stateless goals and source loading, subject to policy. The client-side rpc/2-3 predicate converts its other src_* options to src_text before making this request.

The response is success, failure, or error. The server may retain an internal continuation to serve later offsets efficiently, but this cache is not part of the interaction contract: the client holds no session identity and an equivalent node may recompute the slice.

D.5 The semi-stateful HTTP binding

ISOTOPE maps PTCP onto a family of HTTP endpoints. Spawning a session through this binding returns a pid that identifies the persistent session. Every later operation is authorised against that session and requires its pid.

Endpoint	Parameters	Immediate response
/toplevel_spawn	options, statechart_trace, format	spawned or error
/toplevel_call	goal, template, offset, limit, once, src_text, timeout, format	next session event
/toplevel_next	optional limit, timeout, format	next session event
/toplevel_poll	timeout, format	next queued session event
/toplevel_respond	input, format	responded or error
/toplevel_stop	format	stop or error
/toplevel_abort	format	abort or error
/toplevel_halt	format	halted or error
/toplevel_trace	statechart_trace, format	responded or error

All ISOTOPE endpoints use POST. Parameters are normally sent as URL-encoded form data in the request body; spawn also accepts a JSON body. The `options` value of `/toplevel_spawn` is a Prolog option list, and source options belong in that list. The endpoint forces `session(true)`, irrespective of a supplied `session/1` option. The list may contain `time_limit(SecondsOrInfinite)` and `idle_limit(SecondsOrInfinite)`. The former applies only while the session executes a goal in PTCP state **s2**; the latter applies while it waits in **s1** or **s3**. Public clients may request tighter values, but cannot weaken the node owner's lifecycle ceilings. These session operations are not safe retrieval requests and their responses must not be cached: even polling dequeues an event from a persistent session.

`/toplevel_call` uses the same query parameters and defaults as `/call`, with the addition of the required session pid. If `limit` is omitted from `/toplevel_next`, the limit of the active call is inherited.

The call and next endpoints wait for one event. If output or a prompt arrives before the answer slice, the client receives that event first and uses `/toplevel_poll` to collect the following event. Polling does not advance the solution sequence. A request timeout is capped by node policy and applies at this blocking surface. It returns a non-destructive `timeout` event: it neither aborts the PTCP goal nor terminates the session, and a later poll may retrieve the result.

For a `down` event, the `reason` field preserves the contract in Appendix B: success, logical failure, uncaught exception, explicit exit term, `no_process`, or `connection_closed`. The last reports loss of a remote monitoring relationship rather than observed actor death; it discharges the monitor and is not silently reversed after reconnection. JSON encodes the corresponding public Prolog term without collapsing these cases into a single error string. When an uncaught SWI-Prolog `error(Formal, Context)` crosses the monitor boundary, `Formal` is preserved while `Context` is elided. A private generated actor-module qualification is also omitted from an undefined-procedure error. Other exit reasons are unchanged.

D.6 The stateful WebSocket binding

ACTOR exposes /ws. Each client frame is a JSON object with a mandatory `command` field; each server frame has a mandatory `type` field. Events are pushed as they occur, so no polling command is needed. Actors and sessions created through a connection are owned by it and are terminated when the connection closes.

D.6.1 Version negotiation

A client may begin with:

```
{"command": "transport_hello", "version": 1}
```

A compatible node answers with `type: "transport_welcome"`, the protocol name, and version 1. Version 1 clients may omit this exchange, but explicit negotiation is recommended for new clients.

D.6.2 Toplevel session commands

Command	Canonical fields and response
<code>toplevel_spawn</code>	<code>options</code> ; optional <code>statechart_trace</code> . Events: <code>spawned</code> , <code>error</code> .
<code>toplevel_call</code>	<code>pid</code> , <code>goal</code> , <code>options</code> . Events: <code>answer</code> , <code>output</code> , or <code>prompt</code> .
<code>toplevel_next</code>	<code>pid</code> . Events: <code>answer</code> , <code>output</code> , or <code>prompt</code> .
<code>toplevel_stop</code>	<code>pid</code> . Event: <code>stop</code> .
<code>toplevel_abort</code>	<code>pid</code> . Event: <code>abort</code> .
<code>toplevel_halt</code>	<code>pid</code> . Events: <code>down</code> , then <code>halted</code> .
<code>toplevel_respond</code>	<code>pid</code> , <code>input</code> . Event: <code>responded</code> .
<code>set_statechart_trace</code>	<code>pid</code> , <code>statechart_trace</code> . Event: <code>responded</code> .

The `options` fields are Prolog option lists represented as text. For `toplevel_spawn`, the list may contain the source options and `session/1`. Unlike the ISOTOPE spawn endpoint, WebSocket spawn does not force a persistent session; the actor predicate's default is `session(false)`.

For `toplevel_call`, the canonical options are drawn from:

```
[template(T), offset(I), limit(K), once(B), format(F)]
```

D.6.3 Bare actor commands

Command	Fields	Response or event
spawn	goal; optional options	spawned
send	pid, message	none
monitor	pid, ref	a later down event
demonitor	ref	none
exit	pid; optional reason	a later down event for each monitor

The target of `send` may be an actor owned by the connection or an owner-published service for which access is authorised. Send is fire-and-forget. A send to a missing, unpublished, or unauthorised service is accepted without delivery and does not reveal which condition applied. A binding may reject a command when no usable connection exists, but transport loss after acceptance creates no send response; delivery assurance requires an application acknowledgement. The monitor reference is supplied by the client and is returned unchanged in the corresponding down event, allowing several monitors involving the same pid to be distinguished.

D.7 Lifecycle and ownership

HTTP sessions and WebSocket-owned actors are resources scoped to their creating principal. A client cannot operate on an arbitrary pid merely by knowing its printed form. The exception is sending to an owner-published service when policy grants access; publication does not grant control over the service actor's lifecycle.

For commands originating from one connection, repeated sends to the same pid use that connection as one ordered routing epoch. Reconnection begins a new epoch and carries no portable order relative to sends accepted before the disconnect. Sends through a service name are ordered only while the name's binding is unchanged. There is no ordering guarantee between messages and runtime signals.

Closing a WebSocket terminates the actors and sessions owned by that connection. An ISOTOPE session instead survives individual HTTP requests until it is halted, expires under node policy, or the node terminates. A transport timeout, network failure, or abandoned client choice point therefore does not by itself prove that a remote computation has ended. Clients that create persistent remote work remain responsible for bounded waiting and explicit cleanup.

D.8 Starting a native node

Starting a node is an owner operation, not part of the predicate API available to client programs. From a source checkout, the normal entry point is the environment-driven launcher in `Deployment/start.node.pl`. For local development, the following

command starts an ACTOR node on port 3060 and grants the execution capability to clients connecting directly from the local machine:

```
WP_AUTH=dev WP_PORT=3060 ./Deployment/start_node.pl
```

Then open `http://localhost:3060/demonstrator`. The bare node URL, `http://localhost:3060/`, is not a landing page: by design it returns the node's shared-database source, which in this case is the default example database.

The process remains in the foreground until interrupted. It prints the effective configuration at startup and handles an interrupt or termination signal by putting the node into maintenance, allowing current work a bounded grace period, and then stopping the server.

Configuration is supplied through `WP_*` environment variables or a configuration file selected by `WP_CONFIG`. The most important variables are `WP_PORT`, `WP_PROFILE`, `WP_AUTH`, `WP_SANDBOX`, `WP_TIMEOUT`, and `WP_SHARED_DB_FILE`. A configuration can be validated without binding a port:

```
WP_CHECK=1 WP_PROFILE=actor WP_AUTH=private \
./Deployment/start_node.pl
```

The launcher defaults to private authentication. Starting a world-open node requires both `WP_AUTH=open` and the explicit acknowledgement `WP_ACK_PUBLIC=yes`, since such a node accepts code from untrusted network clients.

For interactive experimentation or for embedding a node in a larger native SWI-Prolog application, the underlying owner interface is `node/1-2`:

```
?- use_module(library(web_prolog)).
?- node(3060, [profile(actor), auth(dev)]).
```

The first form, `node(Port)`, is equivalent to `node(Port, [])`. The second accepts startup options for the node profile, authentication and sandbox policy, shared-database source, request timeout, `PTCP time_limit` and `idle_limit`, cache policy, resource limits, rate limits, and HTTP server configuration. It is an embedding interface for the node owner rather than a Web Prolog predicate offered to clients. The routes created by the server are therefore documented by interaction profile in the preceding sections instead of being repeated as a deployment-time list.

Appendix E

How to implement a Prolog node

The preceding chapters have described Web Prolog chiefly from the programmer's point of view. This appendix turns the system around and asks what a node implementer must provide. The answer depends on the node's profile, its host Prolog system, and the environment in which it is deployed.

At the time of writing, the demonstrator provides substantial implementations of all four profiles, including native and browser-resident actors, cross-node communication, source installation, statechart actors, authentication, policy enforcement, resource limits, and administrative facilities. It should nevertheless be regarded as a research implementation, not as a final interoperable standard or a production platform suitable for arbitrary deployment. In particular, there is not yet a second independent implementation against which its observable behaviour can be tested.

The distinction matters throughout this appendix. The public interfaces are specified in Appendix B; the code below is explanatory. Some fragments closely follow the demonstrator, while others deliberately omit machinery that would obscure the central idea. Whenever a fragment is pedagogical rather than complete, we say so and identify what a deployable implementation must add.

E.1 From a profile to a running node

A Prolog node is not simply a Prolog system with an HTTP server attached. Even a node implementing the narrowest public profile must translate between a network request and a Prolog computation while preserving the contracts for terms, answers, errors, limits, and profiles. Broader profiles add persistent execution contexts, actors, distribution, and lifecycle management.

The profiles therefore imply a progression of implementation responsibilities:

RELATION

Parse a call to an owner-defined relation, execute it within the advertised relation

surface, and encode its answers. Such a node need not implement ISO Prolog generally and need not itself be written in Prolog.

ISOBASE

Add the advertised ISOBASE language fragment, the stateless `/call` API, term and error conversion, and bounded nondeterministic query evaluation.

ISOTOPE

Add private execution contexts, source installation, and the semi-stateful toplevel API. The node must retain query state between requests and reclaim it when a session is stopped, halted, abandoned, or expired.

ACTOR

Add actors, mailboxes, selective receive, links, monitors, naming, remote placement, actor I/O, and the stateful WebSocket protocol. The actor profile also requires a clear account of isolation and of the lifetime relationship between a client, a connection, and the actors created through it.

These are language and protocol responsibilities. A public deployment adds another layer: authentication, authorisation, sandboxing, quotas, overload control, transport security, logging, observability, and administration. A small teaching implementation can omit that layer. An Internet-facing node cannot.

E.1.1 A useful decomposition

The native demonstrator separates the request path into several layers. The exact module boundaries are implementation choices, but the decomposition is portable:

1. A transport adapter accepts HTTP or WebSocket input and converts it into a typed internal request.
2. Authentication identifies a principal, and profile and permission checks decide whether that principal may perform the requested operation.
3. Input validation checks term size, syntax, source options, relation visibility, and the permitted language fragment.
4. An execution-context layer creates or selects the private database, actor, toplevel, or stateless query engine in which the goal will run.
5. The runtime applies time, inference, memory, actor, mailbox, and result limits as appropriate.
6. A response layer converts success, failure, errors, prompts, output, and lifecycle events into the representation required by the transport.

This order is significant. Validation must occur before untrusted source is loaded or an untrusted goal is called, and resource ownership must be established before work is started. Conversely, transport-specific JSON or WebSocket details should not leak into the actor and query semantics.

E.2 A native node on top of SWI-Prolog

The native demonstrator wraps a node around SWI-Prolog. Native threads supply the scheduling substrate for actors, temporary modules supply private clause databases, and the HTTP and WebSocket libraries supply the network transports. This is not the only possible design, but it shows how much of a node can be implemented as a library on top of a mature Prolog system.

E.2.1 The stateless HTTP endpoint

Chapter 5.3 developed answer terms and Chapter 4 showed how `rpc/2-3` reconstructs Prolog nondeterminism from stateless HTTP responses. We do not repeat those client-side mechanisms here. Instead, the following program implements a small but complete stateless endpoint. It accepts Prolog-formatted calls only; JSON, source installation, and policy enforcement are added later by broader-profile implementations.

```
:- use_module(library(http/http_server)).
:- use_module(library(http/http_parameters)).
:- use_module(library(solution_sequences)).

:- http_handler(root(call), call_handler, []).

node(Port) :-
    http_server(http_dispatch, [port(Port)]).

call_handler(Request) :-
    http_parameters(Request, [
        goal(GoalAtom, [atom]),
        template(TemplateAtom, [
            atom,
            default(GoalAtom)
        ]),
        offset(Offset, [integer, default(0)]),
        limit(Limit, [
            integer,
            default(1000000000000)
        ]),
        format(prolog, [default(prolog)])
    ]),
    parse_goal_template(
        GoalAtom, TemplateAtom, Goal, Template
    ),
```

```

compute_answer(Goal, Template, Offset, Limit, Answer),
reply_prolog(Answer).

parse_goal_template(GoalAtom, TemplateAtom, Goal, Template) :-
    atomic_list_concat([
        '(', GoalAtom, ')+( ', TemplateAtom, ' )'
    ], Text),
    read_term_from_atom(Text, Goal+Template, [
        syntax_errors(error)
    ]).

compute_answer(Goal, Template, Offset, Limit, Answer) :-
    once(answer(Goal, Template, Offset, Limit, Answer)).

answer(Goal, Template, Offset, Limit, Answer) :-
    catch(
        call_cleanup(
            slice(Goal, Template, Offset, Limit, Slice),
            Deterministic = true
        ),
        Error,
        true
    ),
    ( nonvar(Error)
    -> Answer = error(Error)
    ; Slice == []
    -> Answer = failure
    ; var(Deterministic)
    -> Answer = success(Slice, true)
    ; Answer = success(Slice, false)
    ).

slice(Goal, Template, Offset, Limit, Slice) :-
    findnsols(
        Limit,
        Template,
        offset(Offset, Goal),
        Slice
    ).

reply_prolog(Answer) :-
    format(
        'Content-type: application/x-prolog; charset=UTF-8~n~n'
    ),
    write_term(Answer, [

```



```

        quoted(true),
        numbervars(true),
        fullstop(true),
        nl(true)
    ]).

```

The goal and template are parsed together so that occurrences of the same variable name denote the same variable. A call to `once/1` makes the HTTP handler deterministic even though `answer/5` is itself nondeterministic. The value `10000000000` is the public default for `limit`; it is a finite stand-in for an effectively unbounded answer slice, as specified in Appendix B.

This program must be bound only to a trusted local interface. It executes arbitrary goals with the authority of the host Prolog process. The complete node surrounds the call to the query engine with parsing limits, authentication, profile and relation checks, sandbox validation, a private execution context, time and inference limits, maintenance and overload checks, and response logging. Those are substantial subsystems, so we describe their placement rather than disguise them as undefined predicates in an otherwise executable example.

E.2.2 Avoiding spurious recomputation

The `compute_answer/5` implementation above is correct, but successive HTTP requests may repeat work. Consider a goal whose first solution is expensive and whose second solution is cheap:

```

?- time((sleep(1), X=foo ; X=bar)).
X = foo ;
X = bar.
?-

```

If the two solutions are requested in separate slices, a purely offset-based implementation recomputes the expensive first branch when serving the second request. More generally, computing the n th slice requires recomputing and discarding the preceding $n - 1$ slices. Such recomputation also repeats any side effects performed while reaching those earlier solutions, so goals paged in this way should be pure whenever repeated effects would be observable or unsafe.

The demonstrator avoids this by retaining a bounded cache of suspended toplevel actors. After producing a non-final slice, a toplevel is waiting at the PTCP slice boundary with its remaining choice points intact. The node may associate that continuation with the next offset. A later request for the same computation can resume the actor rather than repeat the search.

This does not turn the HTTP interface into a client session. The cached actor is an internal optimisation: the client neither receives its pid nor gains exclusive ownership of it. The node may expire or evict it, in which case a later request is recomputed from the information carried by that request.

Keys, ownership, and cleanup

A cache key must contain every input that can affect the computation, including the goal, template, installed source, and relevant evaluation options. Treating a bounded `term_hash/2` result as a unique identity would be unsound unless collisions are checked. A simple pedagogical implementation can instead retain the canonical term itself:

```
goal_key(Goal-Template-Source-Once, Key) :-
    copy_term(Goal-Template-Source-Once, Key),
    numbervars(Key, 0, _).
```

A production key may use a hash for indexing, but it must still establish equality of the full canonical input. It may also have to include the node instance or execution-policy context so that continuations are not mixed across incompatible authorities.

The following alternative implementation exposes the control structure of the cache. It is complete except for the public `toplevel_*` predicates implemented in Section 5.3. A private message queue is used for the answer so that the HTTP worker does not consume an unrelated actor message:

```
:- use_module(library(aggregate)).

:- dynamic cached/3.

max_cache_size(100).

compute_answer_cached(Goal, Template, Offset, Limit,
                      Source, Answer) :-
    goal_key(Goal-Template-Source-false, Key),
    setup_call_cleanup(
        message_queue_create(Queue),
        compute_answer_cached(
            Queue, Key, Goal, Template, Offset, Limit, Source,
            Answer
        ),
        message_queue_destroy(Queue)
    ).

compute_answer_cached(Queue, Key, Goal, Template, Offset,
                      Limit, Source, Answer) :-
    ( take_cached(Key, Offset, Pid)
    ->  toplevel_next(Pid, [
        limit(Limit),
        target(Queue)
    ])
    ;   spawn_options(Source, Queue, SpawnOptions),
```

```

        toplevel_spawn(Pid, SpawnOptions),
        toplevel_call(Pid, Goal, [
            template(Template),
            offset(Offset),
            limit(Limit),
            target(Queue)
        ])
    ),
    await_slice(Queue, Key, Offset, Limit, Pid, Answer).

spawn_options('', Queue, [
    session(false),
    target(Queue)
]) :-
    !.
spawn_options(Source, Queue, [
    session(false),
    target(Queue),
    src_text(Source)
]).

await_slice(Queue, Key, Offset, Limit, Pid, Answer) :-
    ( thread_get_message(Queue, Message, [timeout(30)])
    -> slice_message(
        Message, Key, Offset, Limit, Pid, Answer
    )
    ; stop_toplevel(Pid),
      Answer = error(timeout)
    ).

slice_message(success(Pid, Slice, true),
    Key, Offset, Limit, Pid,
    success(Slice, true)) :-
    Next is Offset + Limit,
    save_cached(Key, Next, Pid).
slice_message(success(Pid, Slice, false),
    _, _, _, Pid, success(Slice, false)).
slice_message(failure(Pid),
    _, _, _, Pid, failure).
slice_message(error(Pid, Error),
    _, _, _, Pid, error(Error)).
slice_message(Message, _, _, _, Pid,
    error(unexpected_message(Message))) :-
    stop_toplevel(Pid).

```

```

take_cached(Key, Offset, Pid) :-
    with_mutex(
        continuation_cache,
        once(retract(cached(Key, Offset, Pid)))
    ).

save_cached(Key, Offset, Pid) :-
    with_mutex(
        continuation_cache,
        (
            assertz(cached(Key, Offset, Pid)),
            evict_excess(Evicted)
        )
    ),
    dispose_cached(Evicted).

evict_excess(Evicted) :-
    max_cache_size(Max),
    aggregate_all(count, cached(_, _, _), Count),
    (   Count > Max
    ->  once(retract(cached(_, _, Evicted)))
    ;   Evicted = none
    ).

dispose_cached(none) :-
    !.
dispose_cached(Pid) :-
    stop_toplevel(Pid).

stop_toplevel(Pid) :-
    catch(toplevel_halt(Pid), _, true).

```

The mutex makes acquisition and insertion race-safe. Eviction removes the oldest clause and terminates its toplevel rather than merely forgetting its pid. To use this version in the HTTP controller, replace its call to `compute.answer/5` with `compute.answer_cached/6` and pass the empty atom as the source.

The example has a size bound but no expiry clock. The demonstrator also associates an alarm with every cache entry, cancels the alarm when the entry is acquired, and terminates the toplevel when the entry expires. It further uses a node-specific cache key and preserves the timeout budget while discarding unrelated internal messages.

E.2.3 Implementing the actor primitives

The actor layer can be explained economically on a Prolog system that implements the ISO thread draft. The following host predicates provide the principal mechanisms:

Web Prolog	Host Prolog
<code>spawn/1-3</code>	<code>thread_create/3</code>
<code>self/1</code>	<code>thread_self/1</code>
<code>send/2, !/2</code>	<code>thread_send_message/2</code>
<code>receive/1-2</code>	<code>thread_get_message/3</code>
<code>exit/2</code>	<code>thread_signal/2</code>

This table is only a starting point. A thread identifier is not a Web Prolog pid, and a thread queue does not by itself implement links, monitors, names, remote addressing, private databases, or the specified selective-receive semantics. The implementation must place an actor layer between the public predicates and the host thread API.

Pids and process creation

The small implementation below separates pids from host thread identifiers. The mappings also make it possible for a distribution layer to replace the local pid generator without exposing thread identities to programs.

```
:- use_module(library(option)).

:- op(800, xfx, !).

:- meta_predicate
    spawn(0),
    spawn(0, -),
    spawn(0, -, +).

:- dynamic pid_thread/2.
:- dynamic thread_pid/2.
:- dynamic link/2.
:- dynamic monitor/3.
:- dynamic exit_reason/2.

:- thread_local parent/1.

:- initialization(init_main_actor, after_load).

init_main_actor :-
    thread_self(Thread),
```

```

(   thread_pid(Thread, _)
->  true
;   fresh_pid(Pid),
    assertz(pid_thread(Pid, Thread)),
    assertz(thread_pid(Thread, Pid))
).

self(Pid) :-
    thread_self(Thread),
    thread_pid(Thread, Pid).

fresh_pid(Pid) :-
    flag(web_prolog_pid, N, N + 1),
    Pid is 10 000 000 + N.

spawn(Goal) :-
    spawn(Goal, _).

spawn(Goal, Pid) :-
    spawn(Goal, Pid, []).

spawn(Goal, Pid, Options) :-
    self(Parent),
    fresh_pid(Pid),
    thread_create(
        actor_boot(Parent, Pid, Goal, Options),
        Thread,
        [at_exit(actor_stopped(Pid))]
    ),
    assertz(pid_thread(Pid, Thread)),
    assertz(thread_pid(Thread, Pid)),
    thread_send_message(Thread, '$start'),
    thread_get_message(initialized(Pid)).

actor_boot(Parent, Pid, Goal, Options) :-
    thread_get_message('$start'),
    assertz(parent(Parent)),
    install_lifetime_options(Parent, Pid, Options),
    Parent ! initialized(Pid),
    call(Goal).

install_lifetime_options(Parent, Pid, Options) :-
    option(link(Link), Options, true),
    (   Link == true
    ->  assertz(link(Parent, Pid))

```

```

;   true
),
option(monitor(Monitor), Options, false),
(   Monitor == true
->  assertz(monitor(Parent, Pid, Pid))
;   true
).

```

The counter in `fresh_pid/1` is sufficient for this single-runtime sketch. A node must prevent pid reuse and collision across concurrent allocation, and its distribution layer must add the node component of the canonical pid. The private `'$start'` message prevents the child from using its pid before the parent has installed the two mappings. The spawn-time monitor uses the pid itself as its reference, matching the public convention documented in Appendix B. A separately installed monitor would receive a fresh opaque reference.

When an actor terminates, the runtime applies the lifetime relations and reports the reason:

```

actor_stopped(Pid) :-
    termination_reason(Pid, Reason),
    forall(
        retract(link(Pid, Child)),
        exit(Child, kill)
    ),
    retractall(link(_, Pid)),
    forall(
        retract(monitor(Watcher, Pid, Ref)),
        Watcher ! down(Pid, Ref, Reason)
    ),
    retract(pid_thread(Pid, Thread)),
    retractall(thread_pid(Thread, Pid)).

termination_reason(Pid, Reason) :-
    retract(exit_reason(Pid, Reason)),
    !.
termination_reason(Pid, Reason) :-
    pid_thread(Pid, Thread),
    thread_property(Thread, status(Reason)).

```

This makes links directional: if the parent terminates, its linked children are terminated, whereas termination of a child does not by itself terminate the parent. Monitoring is observational and produces `down(Pid,Ref,Reason)`.

Sending and termination

Local sending resolves the pid to a thread and delegates copying and queueing to the host:

```
Pid ! Message :-
    send(Pid, Message).

send(Pid, Message) :-
    (   pid_thread(Pid, Thread)
    ->  catch(
            thread_send_message(Thread, Message),
            error(existence_error(_, _), _),
            true
        )
    ;   true
    ).
```

Sending to a pid that no longer exists is a no-op. Sending to an unknown registered name is different: the public interface reports an existence error because the name is a deliberate promise made by the node.

Self-termination records the reason before unwinding the actor. Termination of another local actor is implemented as a host-thread interrupt:

```
exit(Reason) :-
    self(Self),
    asserta(exit_reason(Self, Reason)),
    abort.

exit(Pid, Reason) :-
    (   pid_thread(Pid, Thread)
    ->  catch(
            thread_signal(Thread, exit(Reason)),
            error(existence_error(_, _), _),
            true
        )
    ;   true
    ).
```

The real implementation must serialise termination with monitor installation and removal so that a monitor is either removed or notified exactly once. It must also translate host termination statuses into the documented Web Prolog reasons.

Selective receive

Host message queues already support pattern-directed reception, but Web Prolog receive clauses also have ordering, optional guards, and committed selection. One clear implementation keeps unmatched messages in a thread-local deferred queue:

```
:- thread_local deferred/1.

receive(Clauses) :-
    receive(Clauses, []).

receive(Clauses, Options) :-
    thread_self(Mailbox),
    (   clause(deferred(Message), true, Ref),
        select_body(Clauses, Message, Body)
    ->  erase(Ref),
        call(Body)
    ;   receive_new(Mailbox, Clauses, Options)
    ).

receive_new(Mailbox, Clauses, Options) :-
    (   selectchk(timeout(T), Options, Rest), T > 0
    ->  get_time(Now), D is Now + T,
        scan(Mailbox, Clauses, [deadline(D)|Rest], Options)
    ;   scan(Mailbox, Clauses, Options, Options)
    ).

scan(Mailbox, Clauses, GetOptions, Options) :-
    (   thread_get_message(Mailbox, Message, GetOptions)
    ->  (   select_body(Clauses, Message, Body)
        ->  call(Body)
        ;   assertz(deferred(Message)),
            scan(Mailbox, Clauses, GetOptions, Options)
        )
    ;   option(on_timeout(Body), Options, true),
        call(Body)
    ).

select_body({Clauses}, Message, Body) :-
    select_clause(Clauses, Message, Body),
    !.

select_clause((Clause ; _), Message, Body) :-
    select_clause(Clause, Message, Body).
select_clause(_ ; Clauses), Message, Body) :-
    select_clause(Clauses, Message, Body).
```

```

select_clause((Head -> Body), Message, Body) :-
    (   nonvar(Head),
        Head = if(Pattern, Guard)
    -> Pattern = Message,
        catch(once(Guard), _, fail)
    ;   Head = Message
    ).

```

The cut commits to the first matching message and clause. The selected body is then an ordinary Prolog goal and may itself have several solutions; backtracking through that body never returns the consumed message to the mailbox.

Matching uses ordinary unification, not a subsumption check. A receive pattern may therefore specialise a nonground message: the pattern `offer(coffee)` matches a received `offer(X)` by binding `X` in the receiver's copy. The sender's variable remains unaffected. For ground messages the two matching policies agree. If matching or a guard fails, Prolog backtracking undoes the candidate's bindings before another clause or message is tried. The `nonvar/1` check distinguishes a guarded clause from a variable catch-all pattern without instantiating that pattern to `if/2`.

A positive timeout is converted once to an absolute deadline, which is reused while scanning new messages. Unmatched arrivals therefore do not restart the waiting period. The original options remain available for the `on_timeout` goal. A zero timeout is passed through unchanged: `thread_get_message/3` with `timeout(0)` inspects the queue without waiting, whereas an expired deadline/1 fails immediately even when messages are queued. The deadline bounds waiting; it does not interrupt a guard already being evaluated.

This fragment omits module propagation, detailed option checking, and the atomic interaction between deferred and newly arriving messages. Those are important in an implementation, but they do not alter the semantic idea: scan in arrival order, preserve messages that do not currently match, and commit when a receive clause is selected.

Why the real actor layer is larger

The preceding code is intentionally smaller than the demonstrator. The real native runtime additionally provides:

- collision-safe pids with the standard constructible `Id@Node` representation, and fresh monitor references;
- atomic monitor installation, dead-pid notification, and `demonitor/1-2` with optional flushing;
- registered actor names and node-resident service names;
- delayed sends and cancellation;
- spawn admission and per-actor stack limits;
- private execution modules and controlled source installation;
- hooks for remote spawn, send, exit, monitoring, and naming;

- Erlang's save-queue ordering, so that after a successful receive the deferred messages are rescanned from oldest before new arrivals: the thread-local store yields this ordering directly, but is checked in full on every receive rather than resumed mid-scan as on the BEAM — semantically equivalent, quadratic rather than amortised;
- checks that terms crossing a web worker or network boundary have a portable representation; and
- cleanup for failed actor initialisation, node shutdown, and abandoned sessions.

These are not embellishments. They are what turns a mapping from actors to threads into a node that preserves the same public semantics under failure, concurrency, and distribution. The pedagogical core remains useful because it shows where each of those additions attaches. In particular, the sketch implements the `spawn-time monitor(true)` option but deliberately does not present a naïve `monitor/2` implementation: correct monitoring of an independently supplied pid requires the atomic dead-pid and termination handling described above.

E.2.4 Actor I/O

Actor I/O is expressed as a message protocol rather than as access to the host process's terminal. The basic predicates can therefore be implemented on top of sending and receiving:

```
output(Term) :-
    output(Term, []).

output(Term, Options) :-
    self(Self),
    parent(Parent),
    option(target(Target), Options, Parent),
    Target ! output(Self, Term).

input(Prompt, Input) :-
    input(Prompt, Input, []).

input(Prompt, Input, Options) :-
    self(Self),
    parent(Parent),
    option(target(Target), Options, Parent),
    Target ! prompt(Self, Prompt),
    receive({
        '$input'(Target, Input) -> true
    }).
```

```
respond(Pid, Input) :-
    self(Self),
    Pid ! '$input'(Self, Input).
```

The sender identity in the private response term prevents an unrelated process from satisfying the prompt accidentally. Transport adapters convert the outward-facing `output/2` and `prompt/2` events into JSON, terminal output, or another presentation protocol as appropriate.

E.2.5 Private databases and source installation

An actor needs a private clause database, not merely a private stack and mailbox. A native implementation can create a temporary module for each actor, load the requested `src_*` material into it, and arrange for the actor goal to resolve against that module. Node-published predicates may be imported deliberately, while another client's session module must remain invisible.

This boundary is easy to weaken accidentally. If a spawned goal retains an unrestricted module reference to its creator, the child can observe later changes to the creator's dynamic database. That is sharing, not source installation. The implementation must instead copy explicitly requested predicates or source text into the child context and expose shared node state only through the node's public surface.

Browser actors obtain a stronger version of this separation as a consequence of their execution substrate: each web worker contains a distinct Prolog virtual machine. Native temporary modules must emulate the language-level part of that isolation inside one host process.

Fresh private state does not require recompiling immutable runtime support. The current native implementation compiles the standard actor-I/O predicates once and imports them into each trusted actor module. Requested `src_*` material, dynamic predicates, and all other actor-specific state remain in the fresh temporary module. Public sandboxed actors retain actor-local I/O wrappers, since the sandbox uses their locality to distinguish actor I/O from unrestricted stream I/O. The effect of this prepared runtime module is measured in Appendix F.

E.2.6 Distribution and toplevel actors

Network transparency is most easily implemented by keeping local actor operations behind a dispatch layer. A local pid is resolved to a thread; a remote pid is resolved to a connection and encoded as a protocol message. The public `spawn/3`, `send/2`, `exit/2`, and monitor predicates need not know which branch was taken.

Remote communication adds responsibilities not visible in the pedagogical actor core: connection ownership, handshake and version negotiation, reconnection policy, pending-command correlation, remote monitor delivery, term conversion, and

cleanup when a WebSocket closes. The owner of the remote node must also revalidate source, goals, and permissions rather than trust a decision made by the sending node.

Toplevel actors reuse the same substrate. Their PTCP implementation was developed in Section 5.3. In the current runtime, `toplevel.halt/1` terminates the actor through `exit/2`; `toplevel.halt/2` installs a private monitor and waits until termination has been observed. This works even when the toplevel is computing a nonterminating goal, provided that the substrate supports hard termination.

E.3 Sandboxing, isolation, and authority control

A production-quality node must enforce a boundary strong enough for running untrusted client code. Four concerns must be kept distinct:

Expressibility.

Which language constructs and built-in predicates may client code invoke?

Authority.

Which actors, services, node-owned predicates, and external resources may the computation affect?

Isolation.

What prevents the Prolog runtime or a fault in its sandbox from escaping into the host or interfering with another computation?

Resource consumption.

How much CPU time, memory, actor space, mailbox space, output, and network activity may the computation consume?

No single mechanism answers all four questions.

E.3.1 Whitelist and blacklist strategies

A language-level sandbox may be organised as a whitelist or a blacklist. Whitelisting permits only predicates explicitly declared safe. Blacklisting exposes a defined language fragment but excludes specified predicates, meta-call paths, and operation families.

For a full reflective Prolog system, whitelisting is the safer starting point. Authority may otherwise be reconstructed indirectly through meta-calling, predicate inspection, dynamic database operations, foreign predicates, parser changes, or unrestricted module qualification. A newly added host predicate is denied until it has been examined.

Blacklisting becomes more plausible when the exposed language is already narrow, the dangerous host facilities are absent or unreachable, and all meta-level paths are checked recursively. It remains one layer of a security architecture, not a sufficient

foundation for trust. The demonstrator supports both strategies because they make different trade-offs between compatibility and conservative denial.

E.3.2 Language sandboxing and host isolation

Language sandboxing decides what client code can express. Host isolation decides what the runtime process itself can do. A perfect predicate whitelist does not protect against a defect in the Prolog system or a foreign library. Conversely, a container with no useful filesystem or network authority does not prevent one client from reading another client's Prolog clauses if the language-level isolation is wrong.

A robust native deployment therefore places the Prolog process inside an operating-system, container, or virtual-machine boundary with minimal ambient authority. The Prolog sandbox, node authorisation policy, and host boundary then fail independently rather than standing or falling together.

E.3.3 WASM and WASI

WebAssembly provides another useful host boundary. In the browser, a WASM runtime inherits the browser's origin and web worker security model. In a standalone embedding, WASI-style capabilities can make filesystem handles, sockets, and host callbacks absent unless explicitly granted.

This substantially reduces ambient authority, but it does not solve Prolog-level reflection or resource exhaustion. Nor should a browser-resident demonstrator engine be confused with the server sandbox: code executed in the visitor's browser runs with the authority available to that page or web worker and is not governed by the remote node's Prolog sandbox.

E.3.4 Policy permissions inside the node

Even a host-isolated runtime needs an internal authority policy. Ordinary clients should receive session-scoped permissions: bounded calls, actor creation within their containment scope, messaging to permitted targets, and access to explicitly exported services.

Node owners may possess additional permissions for publishing services, registering public names, installing persistent code, modifying shared node state, inspecting operations, or detaching services from a client session. These operations must not become available merely because a client can name the predicate that implements them.

The implementation should therefore establish a principal at the transport boundary, carry that principal into nested execution and actor creation, and check authority at every operation that crosses from client-owned state into node-owned state.

E.3.5 Resource governance

Timeouts, actor limits, mailbox bounds, RPC limits, source-size limits, cache bounds, and rate limits answer a different question from sandboxing: *how much may this program do?* A program can remain within its language authority and still deny service by spawning actors, constructing enormous terms, producing unbounded output, or repeatedly invoking expensive public relations.

Limits must be associated with an owner or principal rather than applied only to individual calls. Otherwise a client can remain below every per-call limit while exceeding the node's capacity through multiplicity. The runtime also needs a policy for cleanup: refusing new work is insufficient if abandoned actors, sessions, cached continuations, and queues remain resident.

E.3.6 A layered deployment strategy

A realistic public implementation combines all four layers:

1. expose only the intended Web Prolog language and public relation surface;
2. associate every request and spawned computation with a principal and authorisation policy;
3. bound calls, sessions, actors, queues, source, results, and network activity;
4. deploy the runtime with minimal host authority and secure transport; and
5. make denials, saturation, lifecycle events, and policy changes observable to the node owner.

Security is therefore an architectural property of the request path, not a predicate that can be called immediately before `call(Goal)`.

E.4 A browser-resident node

Prolog can be brought into a browser by interpreting or compiling it in JavaScript, by compiling an existing system to WebAssembly, or by installing a browser extension. The demonstrator uses SWI-Prolog's WebAssembly port. Its implementation problem is not merely how to run a Prolog query, but how to reproduce the actor and node abstractions on a single-threaded browser substrate.

E.4.1 Implementing a browser node

Cooperative engines are not isolated actors

The SWI-Prolog WASM interface lets JavaScript call into a Prolog virtual machine. Open queries and engines can retain search state and yield control to JavaScript. A heartbeat based on an inference budget allows asynchronous query drivers to return periodically to the browser event loop.

This is useful cooperative execution, but it is not actor isolation. Engines inside one WASM instance share a virtual machine and global Prolog state, and only one of them executes WASM instructions at a time. A synchronous call, a long-running foreign operation, or JavaScript that does not yield can still seize that VM. Another engine in the same instance cannot forcibly terminate it.

The distinction determines the architecture. If actors shared one single-threaded instance, a non-cooperative actor could prevent every other actor in the node from receiving messages. It could also compromise the private-state model unless a second isolation mechanism were built inside the VM.

One web worker and one VM per actor

The demonstrator therefore runs each local browser actor in its own web worker, with a complete and private SWI-Prolog WASM instance. A JavaScript node controller remains on the main thread. It owns pid allocation, actor and name registries, monitors, delayed sends, web worker lifecycles, remote connections, stateless HTTP calls, the terminal, and the page.

An ordinary actor executes the goal passed to `spawn/1-3`. A toplevel actor runs the mailbox-driven PTCP loop. The shell's toplevel is also a PTCP actor in a web worker; it is special only because the controller creates it directly, attaches the terminal to it, and can replace its web worker during a hard shell reset.

Each web worker has its own linear memory, stacks, dynamic database, modules, atoms, and virtual filesystem. A non-yielding actor blocks only its web worker, and the controller can terminate that web worker. This is the browser equivalent of the containment that native actors obtain from the runtime's ability to interrupt a thread.

The cost is actor density. Every actor instantiates a VM and grows private memory, so web worker start-up and per-web-worker memory are much larger than the cost of an Erlang process or a native Prolog thread. Pools could reduce start-up latency, but multiplexing actors onto a shared web worker would give up some of the isolation that motivates the design.

Terms on the wire

Web workers cannot exchange Prolog heap objects. The sending web worker serialises a message as text, the controller routes the text, and the receiving web worker

parses it into a fresh term. Variable bindings are therefore lost exactly as they are in a remote send.

Using the same representation locally and remotely has a useful consequence: local browser communication is not a privileged semantic fast path. Cyclic terms, attributed variables, streams, and non-text blobs are rejected at the boundary, while ordinary acyclic Prolog terms are copied. An implementation using native threads may copy terms more directly, but it must preserve the same observable contract if location transparency is to hold.

The DOM stays on the main thread

A web worker has no direct access to `window` or `document`. This is not merely an inconvenience. The DOM is a stateful, location-bound, single-threaded resource, precisely the kind of resource that actor systems normally place behind a message interface.

The main-thread controller already performs a limited version of this role: it renders terminal output and delivers user input to the shell actor. A general application can use the same pattern. A web-worker-resident application actor sends presentation messages to a main-thread view component, and the view returns browser events as actor messages. The application logic then has the same shape whether the actor is in the current tab or on a remote node.

This arrangement also fits component frameworks such as React or Vue. Framework-owned DOM is updated by changing component state, not by allowing an external Prolog engine to mutate reconciled nodes. An actor subscription can feed that state in the same way as a WebSocket or another asynchronous data source.

The optional main-thread mode

There is one useful exception. A self-contained Prolog page may want direct DOM access and may be willing to run a Prolog engine on the main thread. The demonstrator supports such a mode. It can use the browser integration facilities of SWI-WASM directly, including yielding while waiting for asynchronous input.

The trade-off is local and explicit. The main-thread engine has DOM access but lacks web worker isolation and hard termination. Actors it spawns still run in web workers, and its calls and messages use the same public contracts as the default model. Portable code can inspect such genuine host differences through `runtime_property/1` rather than infer them from a pid or node URI.

Summary of the browser architecture

The browser node can therefore be summarised as follows:

A JavaScript controller on the main thread owns routing, networking, lifecycle, terminal, and page; each local actor owns an independent SWI-Prolog WASM instance in a web worker; messages cross web worker and node boundaries as portable Prolog terms; and an optional main-thread engine is available for the specialised case of a self-rendering page.

This is not necessarily the optimal browser architecture for all future Prolog systems. A threaded WASM port or a dedicated actor VM could support much lighter processes. The worker-per-actor design is instead a faithful realisation of actor isolation on the substrate available to the demonstrator. Its main achievement is that a local browser actor and a remote actor differ in placement and cost, but not in the programming model through which they are addressed.

E.5 Alternative implementation approaches

E.5.1 Other Prolog systems and host languages

A node need not be based on SWI-Prolog. Another Prolog system may offer threads, engines, modules, HTTP, or WebSocket facilities with different names and guarantees. The implementation task is to reproduce the observable Web Prolog contracts, not the internal structure of the demonstrator.

Nor must every node contain a general-purpose Prolog system. A **RELATION** node can be written in Python, Rust, Erlang, Java, or another host language if it accepts the specified calls, performs the advertised relation, and returns conforming answers. Broader profiles require increasingly complete implementations of unification, control, source installation, private databases, and actor semantics.

An Erlang-based implementation is particularly interesting. Erlang already provides lightweight isolated processes, mailboxes, distribution, monitors, and supervision, but it does not provide Prolog unification, backtracking, or the Web Prolog term and error contracts. A conforming **ACTOR** node on the BEAM would therefore be more than an adapter around an Erlang service: it would combine a Prolog execution engine with the specified Web Prolog process semantics.

E.5.2 Prospects for a dedicated Web Prolog VM

The BEAM demonstrates what can be achieved by a runtime designed around lightweight, isolated, pre-emptible processes. It is natural to ask whether a Web Prolog runtime could combine those properties with unification, logical variables, nondeterministic search, and dynamic symbolic code.

An earlier paper described such a runtime as a “holy grail” for Web Prolog (Lager, 2019). Richard O’Keefe, drawing on experience with both Prolog and Erlang, responded that the idea was feasible and pointed to the small processes of Logix and to the lineage connecting the WAM, JAM, BAM, and BEAM. The history does not

supply an implementation, but it does show that the design space is less alien than the phrase “dedicated VM” may suggest.

Such a VM would need more than fast process switching. It would have to specify how backtracking interacts with committed message effects, how heaps and code are isolated or shared, how a process is interrupted safely, how terms are copied, and how local execution preserves the same observable semantics as remote execution. It would also need the network, policy, and administrative layers described earlier in this appendix.

A mature existing Prolog system consequently retains major advantages: optimising compilation, libraries, debugging, tabling, constraints, foreign interfaces, and decades of operational experience. A dedicated runtime would be justified only if its gains in actor density, scheduling, isolation, or portability outweighed the cost of rebuilding that environment.

E.5.3 Narrower profiles require less work

The profile structure allows implementation to proceed incrementally. A RELATION node can expose a single owner-defined predicate without implementing ISO control constructs. An ISOBASE node can add bounded stateless querying without persistent sessions. An ISOTOPE node can then add source installation and toplevel continuation before any general actor distribution is attempted.

This is not merely a convenience for implementers. Independent implementations are easier to compare first at a narrow boundary. Agreement on term syntax, answer forms, errors, paging, and relation visibility provides a base on which the more demanding lifecycle and concurrency contracts can later be tested.

E.6 Summary

There is no single privileged way to implement a Prolog node. A native node may map actors onto threads and private modules; a browser node may map them onto web workers and independent WASM instances; a narrow relation node may not contain a Prolog runtime at all. What makes them nodes on the same Prolog Web is their agreement at the observable boundaries.

The implementation task is therefore layered. Query evaluation must preserve solutions, failure, errors, and paging. Actor execution must preserve mailboxes, selective receive, lifecycle relations, and location-transparent addressing. Source installation must preserve private execution contexts. Policy, sandboxing, host isolation, and resource governance must contain work performed for untrusted principals. Finally, transports must carry the same terms and events without quietly changing their semantics.

The demonstrator shows that these layers can be assembled on both native SWI-Prolog and SWI-WASM. It remains one implementation, however. The decisive next test is whether the contracts can be stated precisely enough for a different implementation, built on a different substrate, to interoperate without reproducing its internal design.

Appendix F

Benchmarking three actor substrates

Broad claims such as “Web Prolog is slower than Erlang” conceal what is actually being measured. The actor interface described in this book has three implementations of immediate interest: browser actors implemented as SWI-WASM web workers, native actors implemented as SWI-Prolog threads, and Erlang processes running on the BEAM virtual machine. They expose similar message-passing patterns, but they realise an actor at very different levels of granularity.

This appendix compares those three substrates. It is not a contest between Prolog and Erlang as languages, nor does it attempt to reduce the usefulness of a programming model to a single number. Its purpose is narrower: to measure some costs that a programmer can observe, relate them to the implementations described in Appendix E, and identify where optimisation work would have the greatest effect.

F.1 What is being compared

Table F.1 summarises the three implementations. The unit called an actor is intentionally the same at the interface and intentionally different underneath.

Table F.1 The three actor substrates.

Substrate	Actor representation	Communication path
Browser node	An independent SWI-WASM virtual machine in a web worker	Messages relayed between web workers by the browser-side actor bridge
Native node	An SWI-Prolog thread with a private actor context	In-process delivery through the native actor runtime
Erlang	A lightweight BEAM process	In-VM delivery through an Erlang process mailbox

These differences make the result partly predictable. A BEAM process is designed to be created cheaply and in great numbers. A native Web Prolog actor is heavier,

since it is built from a native Prolog thread and its execution context. A browser actor is heavier again: creating one means starting another web worker, instantiating SWI-WASM, installing its source, and connecting it to the page-level message router. The measurements tell us the size of these differences in the current implementations; they do not establish an intrinsic cost for Web Prolog.

F.2 Method

The browser measurements reported here were made on 29 July 2026 on a 2023 Apple iMac with an eight-core M3 processor and 16 GB of memory, running macOS 26.5.1. The native and Erlang measurements were repeated on the same machine on 29 August 2026. The native implementation used SWI-Prolog 10.1.3. The browser implementation used SWI-WASM 10.1.8 in the web-worker-based browser node. The Erlang implementation used Erlang/OTP 26 with ERTS 14.2.5.

The repeated native measurements were made after removing an unintended `: - debug.` directive from the node's loading path. That directive put SWI-Prolog into debug mode and disabled last-call optimisation. The results below use normal execution mode. Starting SWI-Prolog with `-O` was also tested, but changed the medians only within ordinary run-to-run variation, so the reported native series uses the normal command line without `-O`.

Three microbenchmarks were used:

Actor lifecycle.

Start one actor, send it an instruction to stop, wait for its monitor notification, and repeat. The timed interval therefore includes actor creation, initialisation, source installation where required, one message, termination, and monitor delivery. It is an end-to-end actor-lifecycle measurement, not a measurement of the underlying thread or process creation primitive alone.

Chained spawning.

Have each actor create one unlinked child and send it a message, producing a chain of actor creations. This measures the layer-0 native actor core separately from the full isolated node.

Steady-state ping-pong.

Create two actors before timing begins, warm them up with 100 round trips, and then time 1,000 round trips over the same pair. One round trip consists of a ping message and its pong reply. Actor creation and termination are outside the timed interval.

Elapsed wall time was measured in microseconds. A short warm-up preceded the recorded native and Erlang series. The tables report medians: three observations for the browser lifecycle test, seven for the native lifecycle test, and twenty-one for Erlang; the native ping-pong median is from eleven observations and the Erlang median from twenty-one. Browser ping-pong observations required the additional precaution described in Section F.4.1.

The chained-spawning medians use seven native observations at 1,000 actors, five at 10,000 and 100,000, twenty-one Erlang observations at 1,000 and 10,000, and eleven at 100,000. The final isolated-node check uses nine observations at 1,000 actors.

The Web Prolog and Erlang programs use the same lifecycle and message protocols. In particular, links are disabled in Web Prolog where Erlang's ordinary unlinked spawning is used, and both versions wait for monitor notifications before considering an actor terminated. The complete sources are also provided as `benchmarks/actor_benchmarks.pl` and `benchmarks/actor_benchmark.erl` in the demonstrator repository.

F.3 Actor lifecycle

Web Prolog actors are linked by default, whereas Erlang's `spawn_monitor/3` monitors the new process without linking it. The benchmark therefore disables Web Prolog's default link explicitly. It measures the following complete actor lifecycle:

```
benchmark_lifecycle(Actors, Microseconds) :-
    get_time(T0),
    forall(between(1, Actors, _),
           benchmark_one_lifecycle),
    get_time(T1),
    Microseconds is round((T1-T0)*1_000_000).

benchmark_one_lifecycle :-
    spawn(benchmark_idle, Pid, [
        link(false),
        monitor(true),
        src_predicates([benchmark_idle/0])
    ]),
    Pid ! stop,
    receive({down(Pid, _, true) -> true}).

benchmark_idle :-
    receive({stop -> true}).
```

Only one child is live at a time. The test consequently measures the cost of repeatedly creating and reclaiming actors, not the maximum number of simultaneously live actors. Capacity and memory consumption require a separate experiment.

The corresponding Erlang operation is:

```
one_lifecycle() ->
    {Pid, Ref} = spawn_monitor(?MODULE, idle, []),
    Pid ! stop,
```

```

    receive
        {'DOWN', Ref, process, Pid, normal} -> ok
    end.

idle() ->
    receive stop -> ok end.

```

Table F.2 gives the median elapsed time for a sequence of complete lifecycles.

Table F.2 Median time for complete actor lifecycles, in milliseconds.

Actors	Browser node	Native node	Erlang
10	1 271	3.297	0.012
30	3 971	9.363	0.037
100	13 107	31.230	0.121

The near-linear progression is more informative than the largest number alone. In this implementation, one web worker actor lifecycle costs about 130 ms, one native actor lifecycle about 0.312 ms, and one BEAM process lifecycle about 1.2 microseconds. At 100 actors, the browser node is about 420 times slower than the native node. For this complete lifecycle operation, the full native node is about 258 times slower than Erlang.

These ratios describe the chosen end-to-end operation. The browser and native measurements include preparation of private Prolog execution contexts; the Erlang module is already loaded. The table should therefore be read as the price of the actor abstractions supplied by the three systems, not as a comparison of their lowest-level schedulers.

The native implementation does not pool actors or reuse their mutable modules. Every actor receives a fresh private context, but imports one immutable implementation of the standard I/O wrappers. Public sandboxed actors retain actor-local wrappers, since those predicates form part of the sandbox's distinction between actor I/O and general stream I/O.

F.3.1 Chained spawning

The lifecycle ratio must not be read as a raw spawning ratio. The chained spawning program used here is adapted from an Erlang program by Cesarini and Thompson (Cesarini and Thompson, 2009). In it, every actor creates the next actor in a chain, isolating a finer-grained creation pattern. With links made explicitly equivalent to Erlang's ordinary `spawn`, its Web Prolog form is:

```

start(Num) :-
    self(Self),

```



```

    start_proc(Num, Self).

start_proc(0, Pid) :- !,
    Pid ! ok.
start_proc(Num, Pid) :-
    Num1 is Num-1,
    spawn(start_proc(Num1, Pid), NPid, [
        link(false)
    ]),
    NPid ! ok,
    receive({ok -> true}).

```

Each child receives its `ok` message from its parent and the last actor sends an `ok` to the original caller. Unlike the lifecycle test, the program installs no monitor and does not serialise creation with a complete monitored shutdown.

The current stand-alone actor core runs the goal in the spawning module's context, without the private-module isolation added by the full node. Table F.3 compares this layer-0 core with Erlang.

Table F.3 Median time for the unlinked actor chain, in milliseconds.

Actors	Native actor core	Erlang	Ratio
1 000	33.419	0.670	49.9
10 000	316.579	6.237	50.8
100 000	3 137.031	61.163	51.3

The layer-0 spawning gap is therefore about 51 times. Running the same chain through the full native node still prepares a new private Prolog context at every generation, but imports the prepared actor-I/O implementation rather than compiling it again. It also omits the post-load repair pass when an actor has no source to load: without source, no imported predicate or hook-installed module state can have been shadowed.

The distribution layer allocates pids from an unpredictable ten-digit space. Its allocator draws five bytes from the process CSPRNG and rejects the short tail that would introduce modulo bias. This avoids per-thread pseudo-random initialisation while retaining unguessable identifiers.

The full native node completes the 1,000-actor chain in 49.929 ms, against 0.670 ms for Erlang – a ratio of about 74.5. Temporary module creation and destruction by themselves cost about 2.4 microseconds per actor on this machine; complete source-free module preparation raises that to about 8.8 microseconds. Fresh private contexts are therefore no longer the dominant cost in this test.

F.4 Steady-state message passing

Actor startup would dominate a short browser ping-pong test. The benchmark therefore starts its two actors first and reuses them:

```
benchmark_ping_once(Ping, N, Microseconds) :-
    self(Self),
    get_time(T0),
    Ping ! run(N, Self),
    receive({ping_done -> true}),
    get_time(T1),
    Microseconds is round((T1-T0)*1_000_000).
```

```
benchmark_ping_server(Pong) :-
    receive({
        run(N, From) ->
            benchmark_ping_rounds(N, Pong),
            From ! ping_done,
            benchmark_ping_server(Pong);
        stop ->
            true
    }).
```

```
benchmark_ping_rounds(0, _) :- !.
benchmark_ping_rounds(N, Pong) :-
    self(Self),
    Pong ! ping(Self),
    receive({pong -> true}),
    N1 is N-1,
    benchmark_ping_rounds(N1, Pong).
```

```
benchmark_pong_server :-
    receive({
        ping(Ping) ->
            Ping ! pong,
            benchmark_pong_server;
        stop ->
            true
    }).
```

The omitted setup routine starts and monitors both actors, with their default links disabled. It performs the warm-up and calls `benchmark_ping_once/3` for each observation. Finally, it stops both actors and receives both monitor notifications. The Erlang program uses the same sequence.

Moving actor creation outside the timed interval narrows the gap, but does not remove it. The native node completes about 98 times as many round trips per second

Table F.4 Steady-state time and throughput for 1,000 round trips.

Substrate	Median time	Round trips/s	Relative rate
Browser node	1 110 ms	901	1.0
Native node	11.300 ms	88 496	98.2
Erlang	0.572 ms	1 748 252	1 940.3

as the browser node. Erlang in turn completes about 20 times as many as the native node. The browser path crosses two web worker boundaries and the page-level relay on every round trip; the native and BEAM paths remain inside one runtime.

F.4.1 A browser measurement caveat

Repeated browser batches in one page session did not behave as independent observations. Five consecutive batches of 100 round trips over one actor pair took 33, 49, 66, 84, and 103 ms. Recreating the pair did not remove all of the drift, whereas reloading the page did. The cause has not yet been isolated; possible locations include the web worker relay, retained mailbox or routing state, and demonstrator instrumentation.

The browser result in Table F.4 is therefore the median of three observations made in freshly loaded page sessions, followed by a fourth foreground run that reproduced the result. It should be treated as an indicative measurement of the current demonstrator, not as a stable limit of SWI-WASM or browser actors. The drift is also an actionable performance finding: before larger browser benchmarks are meaningful, the retained state responsible for it should be identified.

F.5 RPC, pagination, and query placement

RPC performance belongs to a separate transport and placement experiment. Consider, for example, a WordNet query distributed over two HTTP nodes:

```
rpc(URI1, type(S, v)),
rpc(URI2, word(S, W))
```

If `type(S, v)` produces many solutions, the second call is made once for each of them. The elapsed time then measures an $N+1$ remote-query plan: HTTP request construction, network round trips, connection reuse, and repeated remote setup. Options such as the `solution-page limit` and HTTP keep-alive policy can change the result substantially without changing the logical meaning of the query.

Moving a join to the node that holds the relevant data, batching requests, keeping connections alive, and choosing a suitable page size can therefore matter more

than the raw speed of either Prolog engine. A reproducible RPC benchmark should distinguish local execution, one warm persistent connection, one fresh connection per call, and a genuinely remote network path; record the dataset and node policies; and report both latency to the first answer and throughput for all answers.

F.6 Interpretation and limits

For these two workloads, the ordering is unambiguous: the browser node is slowest, the native node is substantially faster, and Erlang is fastest by a wide margin. The reasons are architectural. A browser actor is a strong isolation boundary containing a Prolog virtual machine. A native actor is a lighter thread and Prolog context. A BEAM process is the unit for which the Erlang virtual machine has been specialised over several decades.

That ordering should not be generalised beyond the evidence:

- The results come from one machine and one set of runtime versions.
- The browser measurements use the demonstrator in an embedded Chromium-based browser, whose exact browser-engine build was not exposed.
- The benchmark messages are small local terms. It does not measure large-term copying, cross-node communication, HTTP, WebSocket transport, or application computation.
- Lifecycle measurements include source and context preparation. A different browser architecture – for example, several actors in one web worker – would answer a different isolation and performance question.
- No claim is made here about maximum actor population or memory per actor. Those require a capacity benchmark with resource measurements and well-defined failure criteria.

The comparison nevertheless gives useful engineering direction. Browser actors should be used at a granularity that justifies a web worker and an independent Prolog VM, rather than as replacements for millions of tiny BEAM processes. Native actors can be substantially finer-grained, although their private Prolog contexts still make them much heavier than Erlang processes. The public actor semantics remain common across the substrates; what differs is the cost at which each implementation currently provides them.

References

- Andersen C, Swift T (2023) The Janus system: A bridge to new Prolog applications. In: Prolog: The Next 50 Years, Springer, Lecture Notes in Computer Science, vol 13900, pp 93–104
- Arias J, Carro M, Salazar E, Marple K, Gupta G (2018) Constraint answer set programming without grounding. *Theory and Practice of Logic Programming* 18(3–4):337–354, DOI 10.1017/S1471068418000285
- Armstrong J (2003a) Concurrency oriented programming in Erlang. Invited talk, FFG
- Armstrong J (2003b) Making reliable distributed systems in the presence of software errors. PhD thesis, Royal Institute of Technology, Stockholm
- Armstrong J (2007a) A history of Erlang. In: Proceedings of the Third ACM SIGPLAN History of Programming Languages Conference (HOPL-III), San Diego, California, USA, 9–10 June 2007, pp 1–26, DOI 10.1145/1238844.1238850, URL <https://doi.org/10.1145/1238844.1238850>
- Armstrong J (2007b) Programming Erlang: Software for a Concurrent World. Pragmatic Bookshelf
- Barnett J, Akolkar R, Auburn R, Bodell M, Burnett D, Carter J, McGlashan S, Lager T, Helbing M, Hosn R, Reifenrath K, Rosenthal N, Roxendal J (2015) State chart XML (SCXML) state machine notation for control abstraction. W3C recommendation
- Beek W, Wielemaker J (2016) SWISH: An integrated Semantic Web notebook. CEUR Workshop Proceedings 1690, URL <https://ceur-ws.org/Vol-1690/paper115.pdf>
- Berners-Lee T, Hendler J, Lassila O (2001) The Semantic Web. *Scientific American* 284(5):34–43, URL <http://www.sciam.com/article.cfm?articleID=00048144-10D2-1C70-84A9809EC588EF21>
- Brusk J, Lager T (2008) Developing natural language enabled games in SCXML. *Journal of Advanced Computational Intelligence and Intelligent Informatics (JACIII)* 12(2):156–163, DOI 10.20965/jaciii.2008.p0156, URL <https://doi.org/10.20965/jaciii.2008.p0156>

- Ceri S, Gottlob G, Tanca L (1989) What you always wanted to know about Datalog (and never dared to ask). *IEEE Transactions on Knowledge and Data Engineering* 1(1):146–166, DOI 10.1109/69.43410
- Cesarini F, Thompson S (2009) *ERLANG Programming*, 1st edn. O'Reilly Media, Inc.
- Cicirelli F, Furfaro A, Giordano A, Nigro L (2009) Statechart-based actors for modelling and distributed simulation of complex multi-agent systems. In: *European Conference on Modelling and Simulation, ECMS 2009, Madrid, Spain, June 9–12, 2009*, pp 233–239, DOI 10.7148/2009-0233-0239, URL <https://doi.org/10.7148/2009-0233-0239>
- Clark HH, Brennan SE (1991) Grounding in communication. In: Resnick LB, Levine JM, Teasley SD (eds) *Perspectives on Socially Shared Cognition*, American Psychological Association, Washington, DC, pp 127–149, DOI 10.1037/10096-006
- Clark KL (1978) Negation as failure. In: Gallaire H, Minker J (eds) *Logic and Data Bases*, Springer, pp 293–322
- Clark KL, Gregory S (1986) *PARLOG: Parallel Programming in Logic*. ACM Press
- Costantini S, Tocchio A (2004) The DALI logic programming agent-oriented language. In: Lifschitz V, Niemelä I (eds) *Logics in Artificial Intelligence, Lecture Notes in Artificial Intelligence*, vol 3229, Springer, pp 685–688, DOI 10.1007/978-3-540-30227-8_57
- Cropper A, Dumančić S, Evans R, Muggleton SH (2022) Inductive logic programming at 30. *Machine Learning* 111(1):147–172, DOI 10.1007/s10994-021-06089-1, URL <https://doi.org/10.1007/s10994-021-06089-1>
- De Raedt L, Kimmig A, Toivonen H (2007) ProbLog: A probabilistic Prolog and its application in link discovery. In: Veloso MM (ed) *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pp 2462–2467
- Frühwirth T (2009) *Constraint Handling Rules*. Cambridge University Press
- d'Avila Garcez A, Lamb LC (2023) Neurosymbolic AI: The third wave. *Artificial Intelligence Review* 56(11):12387–12406, DOI 10.1007/s10462-023-10448-w, URL <https://doi.org/10.1007/s10462-023-10448-w>
- Garvey C (2018) Broken promises and empty threats: The evolution of AI in the USA, 1956–1996. *Technology's Stories* 6(1), DOI 10.15763/jou.ts.2018.03.16.02, URL <https://www.technologystories.org/ai-evolution/>
- Gelernter D (1985) Generative communication in Linda. *ACM Transactions on Programming Languages and Systems* 7(1):80–112, DOI 10.1145/2363.2433
- Gelfond M, Lifschitz V (1988) The stable model semantics for logic programming. In: Kowalski RA, Bowen KA (eds) *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, MIT Press, pp 1070–1080
- Green TJ, Huang SS, Loo BT, Zhou W (2013) Datalog and recursive query processing. *Foundations and Trends in Databases* 5(2):105–195, DOI 10.1561/19000000017
- Gupta G, Salazar E, Shakerin F, Arias J, Varanasi SC, Basu K, Wang H, Li F, Erbatur S, Padalkar P, Rajasekharan A, Zeng Y, Carro M (2023) Prolog: Past, present, and future. In: Warren DS, Dahl V, Eiter T, Hermenegildo MV, Kowalski R, Rossi F

- (eds) Prolog: The Next 50 Years, Lecture Notes in Computer Science, vol 13900, Springer, Cham, pp 48–61, DOI 10.1007/978-3-031-35254-6_4
- Harel D (1987) Statecharts: A visual formalism for complex systems. *Sci Comput Program* 8(3):231–274, DOI 10.1016/0167-6423(87)90035-9, URL [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
- Harel D, Naamad A (1996) The statemate semantics of statecharts. *ACM Trans Softw Eng Methodol* 5:293–333, DOI 10.1145/235321.235322
- Hébert F (2013) Learn You Some Erlang for Great Good!: A Beginner's Guide. No Starch Press, San Francisco, CA, USA
- Hendler JA (2001) Agents and the Semantic Web. *IEEE Intelligent Systems* 16(2):30–37, DOI 10.1109/5254.920597
- Hendler JA (2007) Where are all the intelligent agents? *IEEE Intelligent Systems* 22(3):2–3, DOI 10.1109/MIS.2007.62
- Hewitt C, Bishop P, Steiger R (1973) A universal modular ACTOR formalism for artificial intelligence. In: *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*. Stanford, CA, USA, August 20–23, 1973, pp 235–245, URL <https://www.ijcai.org/Proceedings/73/Papers/027B.pdf>
- Horrocks I (1999) *Constructing the user interface with statecharts*. Addison-Wesley Longman Publishing Co., Inc.
- Horrocks I, Parsia B, Patel-Schneider P, Hendler J (2005) Semantic Web architecture: Stack or two towers? In: Fages F, Soliman S (eds) *Principles and Practice of Semantic Web Reasoning: Third International Workshop, PPSWR 2005, Dagstuhl Castle, Germany, September 11–16, 2005, Proceedings*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 37–41
- Jaffar J, Lassez JL (1987) Constraint logic programming. *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)* pp 111–119, DOI 10.1145/41625.41635
- Junger D, Lager T, Roxendal J (2012) SCXML for building conversational agents in the dialog web lab. In: *Proceedings of the Swedish Language Technology Conference (SLTC) 2012*
- Kakas AC, Kowalski RA, Toni F (1992) Abductive logic programming. *Journal of Logic and Computation* 2(6):719–770, DOI 10.1093/logcom/2.6.719
- Kifer M, de Bruijn J, Boley H, Fensel D (2005) A realistic architecture for the Semantic Web. In: Adi A, Stoutenburg S, Tabet S (eds) *Rules and Rule Markup Languages for the Semantic Web: First International Conference, RuleML 2005, Galway, Ireland, November 10–12, 2005, Proceedings*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 17–29
- Körner P, Leuschel M, Barbosa J, Santos Costa V, Dahl V, Hermenegildo MV, Morales JF, Wielemaker J, Diaz D, Abreu S, Ciatto G (2022) Fifty years of Prolog and beyond. *Theory and Practice of Logic Programming* 22(6):776–858, DOI 10.1017/S1471068422000102
- Kowalski R (1974) Predicate logic as programming language. *Information Processing* 74 pp 569–574
- Kowalski R, Dato A (2022) Logical english meets legal english for swaps and derivatives. *Artificial Intelligence and Law* 30(2):163–

- 197, DOI 10.1007/s10506-021-09295-3, URL <https://doi.org/10.1007/s10506-021-09295-3>
- Kowalski R, Dávila J, Sartor G, Calejo M (2023a) Logical english for law and education. In: Warren DS, Dahl V, Eiter T, Hermenegildo M, Kowalski R, Rossi F (eds) Prolog: The Next 50 Years, Lecture Notes in Computer Science, vol 13900, Springer, pp 287–299, DOI 10.1007/978-3-031-35254-6_24
- Kowalski R, Sadri F, Calejo M, Dávila J (2023b) Combining logic programming and imperative programming in lps. In: Warren DS, Dahl V, Eiter T, Hermenegildo M, Kowalski R, Rossi F (eds) Prolog: The Next 50 Years, Lecture Notes in Computer Science, vol 13900, Springer, pp 210–223, DOI 10.1007/978-3-031-35254-6_19
- Kowalski RA (1979) Algorithm = logic + control. *Commun ACM* 22(7):424–436, DOI 10.1145/359131.359136, URL <https://doi.org/10.1145/359131.359136>
- Kowalski RA, Sadri F (2015) Reactive computing as model generation. *New Generation Computing* 33(1):33–67, DOI 10.1007/s00354-015-0103-z
- Lager T (2019) Intro to Web Prolog for Erlangers. In: *Proceeding of the 18th ACM SIGPLAN International Workshop on Erlang*, ACM, pp 18–29, DOI 10.1145/3331542.3342569
- Lager T, Myrendal J (2012) Exploring the web of coined catchy phrases. In: *Proceedings of WWW2012: Web Science Track*
- Lager T, Wielemaker J (2014) Pengines: Web logic programming made easy. *Theory and Practice of Logic Programming* 14(4-5):539–552, DOI 10.1017/S1471068414000192, URL <http://dx.doi.org/10.1017/S1471068414000192>
- Lindholm TG, O’Keefe RA (1987) Efficient implementation of a defensible semantics for dynamic Prolog code. In: Lassez JL (ed) *Logic Programming: Proceedings of the Fourth International Conference*, MIT Press, pp 21–39
- Loke SW (2006) Declarative programming of integrated peer-to-peer and web based systems: the case of Prolog. *Journal of Systems and Software* 79(4):523–536, URL <http://dx.doi.org/10.1016/j.jss.2005.04.005>
- Manhaeve R, Dumančić S, Kimmig A, Demeester T, De Raedt L (2021) Neural probabilistic logic programming in DeepProbLog. *Artificial Intelligence* 298:103504, DOI 10.1016/j.artint.2021.103504, URL <https://www.sciencedirect.com/science/article/pii/S0004370221000552>
- Marcus G, Davis E (2019) *Rebooting AI: Building Artificial Intelligence We Can Trust*. Pantheon Books, New York
- Merritt D (1989) *Building Expert Systems in Prolog*. Springer-Verlag
- Muggleton S, De Raedt L (1994) Inductive logic programming: Theory and methods. *The Journal of Logic Programming* 19–20(Supplement 1):629–679, DOI 10.1016/0743-1066(94)90035-3
- O’Keefe RA (1990) *The Craft of Prolog*. The MIT Press, Cambridge, MA
- Ousterhout JK (1998) Scripting: higher level programming for the 21st century. *Computer* 31(3):23–30
- Radomski S, Schnelle-Walka D, Radeck-Arneth S (2013) A Prolog datamodel for state chart XML. In: *Proceedings of the SIGDIAL 2013 Conference*, Associa-

- tion for Computational Linguistics, pp 127–131, URL <http://aclweb.org/anthology/W13-4019>
- Ranta A (2011) *Grammatical Framework: Programming with Multilingual Grammars*. CSLI Publications, Stanford, ISBN-10: 1-57586-626-9 (Paper), 1-57586-627-7 (Cloth)
- Riaza JA (2024) Tau Prolog: A Prolog interpreter for the web. *Theory and Practice of Logic Programming* 24(1):1–21, DOI 10.1017/S1471068423000352, URL <https://doi.org/10.1017/S1471068423000352>
- Riguzzi F (2018) *Foundations of Probabilistic Logic Programming: Languages, Semantics, Inference and Learning*. River Publishers, Gistrup, Denmark, DOI 10.1201/9781003338192
- Robinson JA (1965) A machine-oriented logic based on the resolution principle. *Journal of the ACM* 12(1):23–41, DOI 10.1145/321250.321253
- Russell SJ, Norvig P (2016) *Artificial intelligence: a modern approach*. Malaysia; Pearson Education Limited,
- Sacks H, Schegloff EA, Jefferson G (1974) A simplest systematics for the organization of turn-taking for conversation. *Language* 50(4):696–735, DOI 10.2307/412243
- Samek M (2002) *Practical statecharts in C/C++: Quantum programming for embedded systems*. CRC Press
- Shapiro EY (1987) *Concurrent Prolog: Collected Papers*. MIT Press
- Skantze G, Moubayed SA (2012) Iristk: a statechart-based toolkit for multi-party face-to-face interaction. In: *International Conference on Multimodal Interaction, ICMI '12*, Santa Monica, CA, USA, October 22–26, 2012, pp 69–76, DOI 10.1145/2388676.2388698, URL <https://doi.org/10.1145/2388676.2388698>
- Smith RW, Biermann AW, Hipp DR (1995) An architecture for voice dialog systems based on Prolog-style theorem proving. *Computational Linguistics* 21(3):281–318, URL <http://www.aclweb.org/anthology/J95-3001>
- Smolka G (1995) The Oz programming model. In: van Leeuwen J (ed) *Computer Science Today: Recent Trends and Developments*, *Lecture Notes in Computer Science*, vol 1000, Springer, Berlin, pp 324–343, DOI 10.1007/BFb0015252
- van Steen M, Tanenbaum AS (2023) *Distributed Systems*, 4th edn. distributed-systems.net, free digital edition available from distributed-systems.net
- Sterling L, Shapiro EY (1994) *The Art of Prolog: Advanced Programming Techniques*, 2nd edn. The MIT Press, Cambridge, MA
- Svensson H, Fredlund LÅ, Earle CB (2010) A unified semantics for future Erlang. In: Fritchie SL, Sagonas KF (eds) *Proceedings of the 9th ACM SIGPLAN Workshop on Erlang*, Baltimore, Maryland, USA, September 30, 2010, ACM, pp 23–32, DOI 10.1145/1863509.1863514, URL <https://doi.org/10.1145/1863509.1863514>
- Tarau P (2000) Fluents: A refactoring of Prolog for uniform reflection and interoperation with external objects. In: Lloyd JW, et al. (eds) *Computational Logic - CL 2000*, First International Conference, London, UK, 24–28 July, 2000, *Proceedings*, Springer, *Lecture Notes in Computer Science*, vol 1861, pp 1225–

- 1239, DOI 10.1007/3-540-44957-4_82, URL https://doi.org/10.1007/3-540-44957-4_82
- Ueda K (1986) Guarded horn clauses. In: Proceedings of the 1986 International Conference on Logic Programming, Springer, pp 168–179
- Van Gelder A, Ross KA, Schlipf JS (1991) The well-founded semantics for general logic programs. *Journal of the ACM* 38(3):620–650, DOI 10.1145/116825.116838
- Van Roy P (2009) Programming paradigms for dummies: What every programmer should know. *New computational paradigms for computer music* 104
- Van Roy P, Haridi S (2004) Concepts, Techniques, and Models of Computer Programming. The MIT Press, Cambridge, Massachusetts
- Van Roy P, Haridi S, Brand P, Smolka G, Mehl M, Scheidhauer R (1997) Mobile objects in Distributed Oz. *ACM Transactions on Programming Languages and Systems* 19(5):804–851, DOI 10.1145/265943.265972
- Van Roy P, Brand P, Duchier D, Haridi S, Henz M, Schulte C (2003) Logic programming in the context of multiparadigm programming: The Oz experience. *Theory and Practice of Logic Programming* 3(6):715–763, DOI 10.1017/S1471068403001741
- Verborgh R, Vander Sande M (2020) The Semantic Web identity crisis: in search of the trivialities that never were. *Semantic Web Journal* 11(1):19–27, DOI 10.3233/SW-190372, URL <https://ruben.verborgh.org/articles/the-semantic-web-identity-crisis/>
- Viriding R, Wikström C, Williams M, Armstrong J (1996) Concurrent programming in ERLANG (2nd ed.). Prentice Hall International (UK) Ltd., GBR
- Waldo J, Wyant G, Wollrath A, Kendall S (1994) A note on distributed computing. Tech. Rep. SMLI TR-94-29, Sun Microsystems Laboratories, URL <https://waldo.scholars.harvard.edu/publications/note-distributed-computing>
- Wielemaker J (2005) An optimised Semantic Web query language implementation in Prolog. In: Gabbrielli M, Gupta G (eds) *Logic Programming: 21st International Conference, ICLP 2005*, Springer, Lecture Notes in Computer Science, vol 3668, pp 128–142, URL <https://www.swi-prolog.org/download/publications/ICLP05-SeRQL.pdf>
- Wielemaker J (2023) Janus: A Bi-directional Interface Between Prolog and Python. URL <https://www.swi-prolog.org/pldoc/package/janus.html>, SWI-Prolog documentation
- Wielemaker J, Schreiber G, Wielinga B (2003) Prolog-based infrastructure for RDF: Scalability and performance. In: Fensel D, Sycara K, Mylopoulos J (eds) *The Semantic Web – ISWC 2003*, Springer, Lecture Notes in Computer Science, vol 2870, pp 644–658, DOI 10.1007/978-3-540-39718-2_41, URL <https://www.swi-prolog.org/download/publications/iswc-03.pdf>
- Wielemaker J, Schreiber G, Wielinga B (2005) Using triples for implementation: The Triple20 ontology-manipulation tool. In: *The Semantic Web – ISWC 2005*, Springer, Lecture Notes in Computer Science, vol 3729, pp 773–785, DOI 10.1007/11574620_55, URL <https://www.swi-prolog.org/download/publications/ISWC05-Triple20.pdf>

- Wielemaker J, Hildebrand M, van Ossenbruggen J (2007) Using Prolog as the fundament for applications on the Semantic Web. In: Heymans S, Polleres A, Ruckhaus E, Pearce D, Gupta G (eds) Proceedings of the 2nd Workshop on Applications of Logic Programming to the Web, Semantic Web and Semantic Web Services, CEUR-WS.org, CEUR Workshop Proceedings, vol 287, pp 84–98, URL https://ceur-ws.org/Vol-287/paper_1.pdf
- Wielemaker J, Huang Z, van der Meij L (2008) SWI-Prolog and the web. *Theory and Practice of Logic Programming* 8(3):363–392, DOI 10.1017/S1471068407003237, URL <https://doi.org/10.1017/S1471068407003237>
- Wielemaker J, Schrijvers T, Triska M, Lager T (2012) SWI-Prolog. *TPLP* 12(1-2):67–96, DOI 10.1017/S1471068411000494, URL <http://dx.doi.org/10.1017/S1471068411000494>
- Wielemaker J, Beek W, Hildebrand M, van Ossenbruggen J (2016) ClioPatria: A SWI-Prolog infrastructure for the Semantic Web. *Semantic Web* 7(5):529–541, DOI 10.3233/SW-150191, URL <https://doi.org/10.3233/SW-150191>
- Wooldridge M, Jennings NR (1995) Intelligent agents: Theory and practice. *The Knowledge Engineering Review* 10(2):115–152, DOI 10.1017/S0269888900008122, URL <https://doi.org/10.1017/S0269888900008122>
- Yang Z, Ishay A, Lee J (2020) NeurASP: Embracing neural networks into answer set programming. In: Bessiere C (ed) Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20), International Joint Conferences on Artificial Intelligence Organization, pp 1755–1762, DOI 10.24963/ijcai.2020/243, URL <https://doi.org/10.24963/ijcai.2020/243>

Index

A

- Actor 5, 71, 270, 276, **329**
 - API reference **341**
 - as agent **74**
 - bespoke 72
 - comparison with statechart regions 205
 - creation 38, 399
 - creation performance 418
 - distributed ping-pong 122
 - distribution implementation 406
 - examples 57
 - hierarchy 48
 - I/O implementation 405
 - inherited terminal authority 123
 - isolation 410
 - lifecycle **75**, 321
 - lifecycle performance 417
 - message exchange 40
 - performance 415
 - ping-pong example 62
 - primitive implementation **399**
 - primitives **37**
 - private state 74
 - production implementation 404
 - registration 44
 - remote 121
 - remote messaging 121
 - server pattern 57
 - standardisation 321
 - statechart communication 209
 - supervision 172
 - termination 43, 44, 169, 402
- Actor agent *see* Actor
- Actor model 28
- Actor process *see* Actor
- ACTOR profile *see* Profile hierarchy
- actors/1 **343**
- Admission control 409
- Adoption 291
 - community infrastructure **296**
 - conditions 298
 - risks 306
- Agent 2, **71**, **329**
 - AI 239
 - AI motivation 240
 - architectural definition 71
 - beliefs 78, 280
 - complementary views 279
 - neuro-symbolic 252
 - node 94
 - overview **71**
 - primary abstraction **276**
 - service-request 256
 - Web 303
- Agent framework 250
- Agentic Web 10, **239**, 303, **329**
 - Prolog 240
 - Prolog agents **245**
- Alternative solvers 192
- Answer set programming 302
- Answer slice 12, 85, 136, **329**
 - implementation 178
 - large result sets 139
- API reference 341
- Artificial intelligence
 - community 303
 - implementation assistance 323
- Authentication 159
- Authorisation 159, 408
 - implementation 407
 - node policy **408**

B

Backtracking 23
 actor answers 65
 as pagination *see* Backtracking as pagination
 interaction with effects 32
 Backtracking as pagination 12, 85, 116, 140, 273, 421
 BEAM 412, 415
 Behaviour 163, 283, 329
 callback 170
 Erlang 286
 meta-interpreter 249
 overview 72
 predicate generics 358
 server 164
 statechart 198
 supervisor 170
 toplevel 66, 81, 175
 Benchmarking 415
 actor lifecycle 417
 actor substrates 415
 browser timing 421
 chained spawning 418
 interpretation 422
 limitations 422
 message passing 420
 method 416
 RPC 421
 Browser
 ACTOR node 110
 ISOBASE node 106
 ISOTOPE node 107
 RELATION node 103
 two-hop call 138
 Browser node 15, 36, 112, 283, 330
 actor isolation 410
 architecture 410
 architecture summary 411
 benchmarking 421
 implementation 409
 main thread 411
 main-thread mode 411
 message encoding 410
 Browser-resident node *see* Browser node
C
 Caching 143
 remote solutions 143
 cancel/1 347
 Capability 155, 330
 reply channel 158

security 278
 terminal endpoint 123
 Closed-world assumption *see* Open-world assumption
 Code shipping 151, 284
 data locality 152
 source injection 152
 source installation 406
 Collaboration 296
 Committed effect 32, 56, 265, 281, 330
 Committed-choice language 285
 Compatibility 273
 Concurrency
 message passing 62
 Concurrent logic programming 285, 299
 Concurrent Prolog Web *see* Prolog Web, concurrent
 Concurrent, sequential, and programmable Prolog Web 330
 concurrent/3 192
 Configuration 330
 Configuration (statechart) 201, 205, 212
 Conformance 103, 275, 294, 323, 330
 test suite 323
 Containment 46, 48, 75, 158, 321, 331
 Control 243
 first-class dimension 283
 interaction 283
 Conversational agent 262, 305
 Cursor *see* Solution cursor

D

Data locality 152, 421
 <datamodel> 373
 <defer> 374
 Definite clause grammar 245
 remote 250
 Delayed sending 54, 61
 demonitor/1-2 344
 Deployment
 native node 389
 Dialogue
 as proof 263
 data model 265
 governance 265
 parsing 265
 repair 263
 repair via history 265
 turn-taking 263
 Dialogue manager 264
 Discovery *see* Service discovery
 Discovery Hub 128
 Distribution-first 130, 281, 331

Documentation 298
 DOM 411
 direct access 411
 Dynamic database 77

E

Education 296
 Embodied agent 305
 Engineering agenda 314
 Erlang
 community 305
 comparison with Trinity 286
 gen_statem 216
 influence on Web Prolog 28
 links *see* Link
 message passing 37
 Error handling 41
 Event deferral 50, 89, 209, 331
 SXML 374
 Event-driven programming 63
 Eventless transition 207, 331
 Exception 41
 Executable logic 12, 284
 Executable policy 254
 Exit signal 131
 exit/1 345
 exit/2 44, 346
 Expert system 244
 agent 246
 Explainable AI 10, 248
 Explanation 243

F

Fail-fast call 168
 Failure
 logical 41
 Federation 292, 293, 317, 331
 diversity 295
 <final> 369
 findall/3 92
 findnsols/4 92
 first_solution/2-3 192
 API reference 358
 example 194
 failure and errors 194
 racing solvers 192
 Flat SXML 213, 331
 flush/0 54, 349
 Forward chaining 207
 Furhat 305

G

Generic programming 163
 Generics *see* Behaviour
 Global goal 147
 <go> 370
 Governance of interaction 265, 283

H

Harel, David 197
 <history> 370
 History state 201, 265, 331
 Host isolation 331, 407, 408
 WebAssembly 408
 Host language 412
 Hot code replacement 168, 332
 HTTP 12
 caching 143
 overlapping calls 148
 semi-stateful binding 386
 stateless binding 386
 stateless endpoint implementation 393
 stateless RPC 184
 transport 99
 HTTP API
 semi-stateful *see* Stateless, semi-stateful,
 and stateful APIs
 stateless *see* Stateless, semi-stateful, and
 stateful APIs

I

Implementation
 independent 323
 interoperability 323
 Prolog node 391
 role in standardisation 323
 in/1 355
 Inexpressibility 36, 156, 332, 408
 Inference 242
 input/2-3 347
 Instant portability 152, 332
 Interaction
 as inference 277
 Interaction portability 272, 273, 332
 Interaction surface 271, 275, 295, 318, 332,
 392
 profile ceiling 102
 Interoperability 8, 273, 291, 332
 Agentic Web 251
 agents 251
 Erlang 305
 federating force 293

- independent implementations 323, 412
- neighbouring communities 299
- testing 323
- Invisibility **157, 332**
- ISO 317
 - liaison 318
 - solution cursors 322
- ISO Prolog
 - supported predicates **362**
- ISOBASE profile *see* Profile hierarchy
- ISOTOPE profile *see* Profile hierarchy

K

- Knowledge representation 242

L

- Large language model 10
- Late reply 155
- Let it crash principle 170
- Linda 285
- Link **46, 158, 332**
 - implementation 402
- listing/0-1 **349**
- Livelock 204
- Logic programming 23
 - community 299
 - languages 299
 - layered architectures 234
 - Prolog *see* Prolog
- Logical update view 116, **141**, 322, **333**, 395

M

- Mailbox 5, 49, **333**
 - flushing 54
 - implementation 403
- Message loop 214
- Message passing 28, **37**
 - distributed 121
 - implementation 399
 - performance 420
 - semantics 321
 - send implementation 402
 - standardisation 321
 - two-step model 130
 - uniform local and remote interface 130
- Message queue *see* Mailbox
- Meta-interpreter **243**
 - as behaviour 249
- Microbenchmark 416
- Missing axiom 263

- Model service 255
- Monitor **43, 333**
 - implementation 402
 - server calls 168
- monitor/2 **343**
- Mozart 286
- Multi-agent system 71, 262, 303, **333**
- Multiparadigm programming 31, 286
- Mutual constraint **277, 333**

N

- Negation As Failure 228, 280
- Neuro-symbolic AI 10, **252, 333**
 - architectural patterns **254**
 - failure containment 255
 - integration boundary 254
 - model calls Prolog 254
 - motivation 252
 - Prolog calls model 255
 - research directions 261
 - statechart control 256
 - tight integration 256
 - Web Prolog 253
- Node 5, 276, **333**
 - alternative implementations 412
 - as agent **94**
 - controller 95
 - cross-node messaging 122
 - deployment 96
 - deployment security 409
 - distributed inference 248
 - implementation 115, **391**
 - implementation layers 392
 - implementation responsibilities 391
 - native implementation 393
 - profiles 101
 - settings 97
 - shared database 95
 - starting from the operating system **389**
 - sub-profiles 73
- Node controller **95, 334**, 404
 - browser 410
 - routing 130, 406
- Node-resident actor **125, 296, 334**
 - count server 125
 - publish-subscribe 126
- Non-player character 202
- Nondeterminism
 - network interface 273
 - protocol mapping 182
 - remote *see* RPC meta-predicate
 - remote calls 135
- Normative reasoning 254

O

Offset pagination 140
 <onentry> **372**
 <onexit> **372**
 Open-world assumption 227, **228**, **334**
 Orthogonality *see* Parallel region
 OTP 286
 output/1-2 **347**
 OWL 222, **227**, **334**
 external reasoning service 227
 reasoning service 231
 Owner 96, 125, **334**
 Ownership
 shared 324
 Oz 286

P

<parallel> **369**
 Parallel execution 145, 187
 remote calls 146
 Parallel region **202**
 comparison with actors 205
 NPC example 202
 Parallel region (orthogonality) **334**
 parallel/1 **145**
 API reference **358**
 cleanup 189
 comparison with concurrent/3 192
 failure and errors 188
 goal lists 187
 implementation **187**
 performance 190
 Parsing 245
 remote 250
 Partial failure 153, 281
 Pid 5, 12, **334**
 implementation 399
 remote 121
 standardisation 321
 wire representation 382
 Ping-pong benchmark 420
 PIP **292**, **334**
 Planning 242
 Platform 316
 Playground 15, **296**
 Portability 292, **335**
 Principal **335**, 392, 408
 Priority queue 55
 Private database 6, **74**, 78, **335**
 dynamic predicates 77
 implementation **406**
 mutation 77

Probabilistic logic programming 301
 Process identifier *see* Pid
 Profile **335**
 API ceiling 102
 as interaction contract **275**
 interoperability 294
 Profile hierarchy **101**, 322
 ACTOR 110, 113
 conformance 103
 extensions 115
 implementation **391**
 implementation cost 413
 ISOBASE 106, 413
 implementation 393
 ISOTOPE 107
 protocol surfaces **381**
 RELATION 103, 413
 Profile hierarchy (RELATION, ISOBASE, ISOTOPE, ACTOR) **335**
 Programmable Prolog Web *see* Prolog Web, programmable
 Prolog
 AI facilities 242
 browser implementations 13
 current ecosystem 8
 essence 23
 fragmentation 8, 292
 history 3
 unification of systems 292
 Web history **13**
 Prolog actor *see* Actor
 Prolog agent *see* Agent
 Prolog agents 12
 Prolog community 291, 296, 324
 Prolog engine 270, **335**
 cooperative 410
 Prolog Improvements Forum 318
 Prolog node *see* Node
 Prolog systems
 node implementation 412
 Prolog Trinity ecosystem 1, 2, **335**
 as paradigm **269**
 comparisons 285
 demonstrator 4
 fit with Prolog **287**
 five views 279
 layered architecture 235
 motivation 8
 mutual constraint 277
 Prolog Web 2, 12, **119**, **336**
 AI agents 245
 concurrent **121**
 effects 281
 programmable **151**

- design space 153
 - relationship to Semantic Web 221, **228**
 - Semantic Web cooperation 230
 - sequential **134**
 - standardisation 317
- Promise and yield **148**
 - overlapping waits 148
 - server behaviour 166
 - timeouts 150
- promise/3-4 **360**
- Proof
 - Semantic Web 232
- Proof tree
 - distributed 248
- Protocol
 - binary representation 383
 - compression 383
 - lifecycle and ownership 389
 - statelessness 115
 - versioning 382
- Protocol abstraction 60, 166
- Protocol reference 381
- Provenance 232
- PTCP **82**, 322, **336**
 - abort 89
 - abort signal 131
 - calling a goal 85
 - halt 89
 - implementation 176
 - offset and limit 85
 - once option 87
 - protocol reference **385**
 - relationship to SXML 216
 - template option 85
- Public surface **336**
- Publish-subscribe 126
- Pure Web Prolog and the pure Prolog Web **279, 336**
- Purity 279
 - distributed 279

Q

- Query-the-user 244
- Quota 409

R

- Race condition 189
- raise/1 **355**
- RDF 222, **223**, 302, **336**
 - bridge example 231
 - Prolog rule layer 231
 - querying 225

- rdf/3 223
 - rule layer 231
- receive/1-2 **49, 348**
- Recomputation 395
- register/2 **344**
- register_service/2 **344**
- Registered name **44**, 128, 157, **336**
- RELATION profile *see* Profile hierarchy
- Remote module **146**
- Remote procedure call *see* RPC
 - meta-predicate
- Reply channel 90, 155, **158, 336**
- Research agenda **314**
- Resource governance 12, 159, **336**, 392, **409**
 - cursor cleanup 396
- respond/2 **347**
- Risk analysis **306**
- RPC meta-predicate 91, 134, **135, 336**
 - actor-based implementation 137, 182
 - API reference **359**
 - global goals 147
 - HTTP implementation 137, 184
 - implementation **181**
 - implementation comparison 186
 - parallel calls 146
 - performance 421
 - programming patterns 145
 - protocol mapping 182
 - remote modules 146
 - semantics **136**
 - two-hop call 138
- rpc/2-3 **359**
- rpc/3
 - limit option 136, 139
- Run-to-completion (macrostep) **212, 337**
 - SXML reference 377
- runtime_property/1 **361**

S

- Sandboxing 12, 36, 156, **337**
 - blacklist 407
 - implementation **407**
 - language level 408
 - whitelist 407
- Scalability
 - Agentic Web 251
 - agents 251
- Scoped inference 136, 277, **279, 337**
 - epistemic reading 280
- Scripting language 34
- SCXML **217**, 320, **337**
 - actor-oriented successor 217

- differences from SXML 379
- Search 242
- Security
 - capability-oriented 278
 - deployment 159
 - layered deployment 409
 - Prolog Web 155
- Selective receive 49, 337
 - deferred messages 50
 - guards 51
 - implementation 403
 - matching 49
 - nondeterminism 56
 - priority 55
 - timeouts 54
- self/1 38, 341
- Semantic Web 221, 337
 - community 302
 - cooperation patterns 230
 - layer cake 222
 - logic programming 234
 - relationship to Prolog Web 228
 - Trinity comparison 235
 - user interfaces 233
- send operator 40
 - API reference 346
- send/2-3 61, 346
- Sequential and concurrent partitions 25, 270, 337
 - abstraction levels 271
- Sequential Prolog Web *see* Prolog Web, sequential
- Server 57, 164, 337
 - API reference 352
 - client interface 60, 166
 - hot code replacement 168
 - monitoring 168
 - reusable behaviour 170
 - shutdown 169
 - state 59
- Server push 117
- server.halt/2 354
- server.promise/3-4 353
- server.request/3-4 353
- server.spawn/3-4 352
- server.upgrade/2-3 353
- server.yield/2-4 353
- Service 338
 - specialised 301
- Service discovery 128, 338
 - Discovery Hub 128
- Service publication 128, 338
- Service registry *see* Service publication
- Session 338
 - I/O 88, 405
 - semi-stateful HTTP 116
 - stateful WebSocket 117
- Shadowing 78, 338, 406
- Shared database 6, 78, 95, 338
 - mutation 77
- Shared object 296
- Shell 80
 - actor programming tools 39
 - toplevel interaction 84
- Signal 131
- Simulation 246
- sleep/1 54
- Social robot 113, 305
- Solution cursor 116, 140, 338
 - cleanup 179
 - consistency 141
 - implementation 395
 - ownership and cleanup 396
 - standardisation 322
- SPARQL 222, 225, 302, 338
 - Web Prolog client 230
- <spawn> 374
- spawn/1-3 342
- spawn/3 38
 - implementation 399
 - node option 124
 - semantics 321
- spawn.statechart/3 200
- Specification
 - initial targets 320
- src_* options 152
- Stable model semantics 302
- Standardisation 313
 - AI assistance 323
 - incremental strategy 317
 - incubation 317
 - initial targets 320
 - layering 322
 - PIPs 292
 - scope and timing 318
- <state> 368
- State machine *see* Statechart actor
- State-transition model 242
- Statechart 197
 - broadcast communication 202
 - broadcast semantics 205
 - data model 206
 - deferred events 209
 - guards 206, 265
 - hierarchy 201
 - internal event cascade 204
 - previous work 214
 - process invocation 209

- time 208
- Statechart actor 63, **198**, 283, 304, **338**
 - API reference **354**
 - comparison 214
 - gen.statem 216
 - comparison with message loop 214
 - completion 369
 - dialogue management 264
 - example 198
 - execution 200
 - governing model interaction 256
 - service-request agent 211, 256
 - XML notation 365
- <statechart> **368**
- statechart.halt/2-3 **355**
- statechart.spawn/2 **354**
- Stateless, semi-stateful, and stateful APIs
 - 84, **99**, **339**
 - hidden state 143
 - HTTP 99
 - semi-stateful 116
 - solution retrieval 142
 - stateful 117
 - statelessness 115
 - WebSocket 99
- Supervision tree 174
- Supervisor **170**, **339**
 - API reference **355**
 - example 170
 - implementation 172
 - restart intensity 173
 - restart policy 170
 - restart strategy 174
- supervisor.count.children/2 **357**
- supervisor.delete.child/3 **356**
- supervisor.halt/1 **357**
- supervisor.respawn.child/3 **357**
- supervisor.spawn/2-3 **355**
- supervisor.spawn.child/3 **356**
- supervisor.terminate.child/3 **356**
- supervisor.which.children/2 **357**
- SWI-Prolog
 - native node implementation **393**
 - native threads 415
 - production platform 316
- SXML 197, **339**
 - actor invocation 374
 - compilation 213
 - data model 206
 - example 198
 - execution semantics **212**
 - flat subset 213
 - invoke 209
 - limits 216

- notation reference **365**
- Prolog data model 373
- relationship to SCXML 217
- standardisation 217, 320
- tracing 378
- Symbolic AI 10, **242**
- Symbolic paradigm 269

T

- Term representation 410
- Terminal
 - browser node 112
 - distributed I/O 123
- Timed transition **208**, **339**
- Timeout 54
 - remote calls 150
- Timing and failure boundary **153**, 255, 281, **339**
 - blocking surfaces 154
 - design principle 155
- TLS 383
- Tool use
 - Prolog 254
- Tooling 298
- Toplevel actor 65, **66**, **80**, **339**
 - abort 89, 179
 - answer redirection 90
 - answer terms 178
 - API reference **350**
 - communication protocol 82
 - goal calls 85
 - halt 89, 179
 - I/O 88
 - implementation **175**, 406
 - lifecycle 179
 - message deferral 89
 - programming interface 91
 - protocol 81
 - remote placement 124
 - RPC 182
 - solution collection 92
 - standardisation 322
 - state machine 176
 - statefulness 84
 - synchronous API 91
- toplevel.abort/1 **352**
- toplevel.call/2-3 **350**
- toplevel.halt/1-2 **352**
- toplevel.next/1-2 **351**
- toplevel.spawn/1-2 **350**
- toplevel.stop/1 **351**
- Transition
 - SXML 370

- Transport 99
- Trinity demonstrator 4, **15**
 - development routes 316
 - production platform 316
- Triple 223
- Trust 232
- Tuple space 285
- U**
- Unification **12, 23, 272, 339**
 - RDF querying 225
 - statechart guards 206
- unregister/1 **344**
- unregister_service/1 **345**
- URI 12
- User interface 233
 - statecharts 304
- User-interface agent **262**
 - comparison with MAS agents 262
- V**
- Virtual machine
 - Web Prolog 412
- W**
- W3C 317, 320
 - incubation 318
- WASM and WASI 36, **340, 408**, 409, 410
- Web
 - executable logic 284
- Web programming 304
- Web Prolog v, 2, 12, **23, 340**
 - agent frameworks 250
 - browser execution 36, 283
 - manual **341**
 - multiparadigm character 31
 - overview **25**
 - OWL orchestration 231
 - programs and data 6
 - protocol reference **381**
 - scripting language **33**
 - security 36
 - SPARQL client 230
 - suitability for the Web 36
 - symbolic backend 253
 - virtual machine 412
 - web programmers 304
- web worker **410**, 415
- WebAssembly *see* WASM and WASI
- Webizing Prolog and prologizing the Web **12, 340**
 - addressing 12
 - dual process 287
 - infrastructure 12
 - prologizing the Web 12
 - webizing Prolog 12
- WebSocket 12
 - stateful binding **388**
 - transport 99
- WebSocket API *see* Stateless, semi-stateful, and stateful APIs
- whereis/2 **344**
- whereis_service/2 **345**
- World Wide Web
 - history 3
- X**
- XML
 - SXML 213, 365
- Y**
- yield/2-3 **360**